目录

第 1 章 C++ 基础与面向对象 · · · · · 3
1.1 类、接口与多态 · · · · · 3
1.2 对象模型、生命周期与类型 · · · · · 8
1.3 模板与初始化 · · · · · 15
1.4 指针、对象表示与调用接口 · · · · · 17
1.5 数值表示与算术边界 · · · · · 19
第 2 章 内存与资源管理 · · · · · 20
2.1 所有权与 RAII · · · · · 20
2.2 分配策略与内存优化 · · · · · 22
2.3 原始存储、分配失败与字节操作 · · · · · 23
第 3 章 现代 C++ 与模板 · · · · · 25
3.1 语言特性与类型系统 · · · · · 25
3.2 值类别、移动与泛型 · · · · · 28
3.3 Modules、协程与编译期计算 · · · · · 31
3.4 特殊成员函数控制 · · · · · 32
3.5 C++11 语言与模板补充 · · · · · 32
第 4 章 并发与线程安全 · · · · · 33
4.1 同步原语与线程管理 · · · · · 33
4.2 并发任务组织 · · · · · 37
4.3 内存模型与无锁协作 · · · · · 38
第 5 章 性能与工程实践 · · · · · 40
5.1 性能测量与基础优化 · · · · · 40
5.2 算法、缓存与 I/O · · · · · 42
5.3 链接期优化 · · · · · 44
第 6 章 编译、链接与构建 · · · · · 44
6.1 头文件与项目组织 · · · · · 44
6.2 编译、链接与库 · · · · · 46
6.3 ODR 与 ABI · · · · · 48
6.4 头文件搜索与二进制装载 · · · · · 48
6.5 指令集、ABI 与二进制兼容 · · · · · 49
第 7 章 操作系统与 Linux · · · · · 49
7.1 文件、权限与进程 · · · · · 49
7.2 I/O 多路复用 · · · · · 51
7.3 内存、进程间通信与并发模型 · · · · · 52
7.4 I/O 完成模型 · · · · · 54
7.5 进程、执行与启动链路 · · · · · 55
7.6 内核边界、调度与同步 · · · · · 55
7.7 地址转换与内存回收 · · · · · 57
7.8 文件系统、缓存与持久化 · · · · · 58
7.9 中断与 I/O 语义 · · · · · 59
7.10 Linux 内存管理机制 · · · · · 59
7.11 文件与设备数据路径 · · · · · 61
7.12 硬件执行与内核架构 · · · · · 62
第 8 章 计算机网络 · · · · · 62
8.1 HTTP 与安全连接 · · · · · 62
8.2 TCP、UDP 与传输控制 · · · · · 64
8.3 名称解析与服务端网络 · · · · · 66
8.4 协议分层与应用交互 · · · · · 68
8.5 HTTP 缓存、会话与浏览器安全 · · · · · 70
8.6 TCP 状态与应用确认 · · · · · 72
8.7 网络故障与安全 · · · · · 73
8.8 网络层与链路层 · · · · · 74
8.9 链路转发与地址配置 · · · · · 74
8.10 Linux 网络数据路径 · · · · · 75
第 9 章 数据库与缓存 · · · · · 76
9.1 Redis 与缓存设计 · · · · · 76
9.2 MySQL 索引、事务与并发控制 · · · · · 77
9.3 热点与大键治理 · · · · · 80
9.4 Redis 与缓存设计 · · · · · 80
9.5 SQL 基础与数据建模 · · · · · 81
9.6 MySQL 引擎与存储 · · · · · 84
9.7 MySQL 索引补充 · · · · · 85
9.8 MySQL 锁与恢复 · · · · · 86
9.9 MySQL 查询与复制补充 · · · · · 87
9.10 Redis 内部结构与执行模型 · · · · · 88
9.11 Redis 事务与生命周期 · · · · · 89
9.12 Redis 复制、集群与缓存 · · · · · 89
9.13 MySQL 存储机制与运维补充 · · · · · 91
第 10 章 系统设计与分布式 · · · · · 92
10.1 分布式基础与数据结构 · · · · · 92
10.2 数据处理与在线服务设计 · · · · · 94
10.3 高并发、消息与容错 · · · · · 96
10.4 共识与消息一致性 · · · · · 99
10.5 分布式协调与服务治理 · · · · · 100
10.6 系统设计补充 · · · · · 100
10.7 消息队列模型与运行 · · · · · 101
10.8 RocketMQ、Kafka 与 RabbitMQ · · · · · 102
10.9 分布式理论与一致性 · · · · · 105
10.10 分布式组件与服务治理 · · · · · 105
10.11 分布式事务与一致性 · · · · · 106
10.12 高并发业务系统设计 · · · · · 107

第 11 章 自动驾驶与实时数据 · · · · · 107
11.1 QoS 与点云处理 · · · · · 107
11.2 共享内存与时钟同步 · · · · · 108
11.3 实时数据流水线 · · · · · 108
第 12 章 C++ 进阶与工程 · · · · · 109
12.1 内存模型与缓存一致性 · · · · · 109
12.2 模板、容器与线程安全 · · · · · 111
12.3 零拷贝、异常与并发问题 · · · · · 112
第 13 章 数据结构与算法 · · · · · 115
13.1 基础动态规划与数组问题 · · · · · 115
13.2 树与图的遍历 · · · · · 117
13.3 排序、查找与有序结构 · · · · · 122
13.4 链表、字符串与基础数据结构 · · · · · 130
13.5 常见结构与搜索算法 · · · · · 136
13.6 排序、组合搜索与贪心 · · · · · 144
13.7 路径规划与动态规划 · · · · · 150
13.8 栈、滑动窗口与数组优化 · · · · · 153
13.9 STL 容器使用与选择 · · · · · 159
13.10 标准算法与容器适配器 · · · · · 160
13.11 索引结构与算法选择 · · · · · 161
第 14 章 Qt · · · · · 163
14.1 核心机制与容器 · · · · · 163
14.2 界面与绘图 · · · · · 166
14.3 并发与网络 · · · · · 172
14.4 数据、模型与事件 · · · · · 174
14.5 高级特性与 QML · · · · · 184
14.6 工程实践 · · · · · 189
第 15 章 OpenCV 与计算机视觉 · · · · · 206
15.1 图像基础与常用处理 · · · · · 206
15.2 特征、变换与视觉算法 · · · · · 208
15.3 视频分析与相机标定 · · · · · 211
第 16 章 音视频与 FFmpeg · · · · · 213
16.1 容器、编码与流媒体协议 · · · · · 213
16.2 FFmpeg 解复用、解码与时间戳 · · · · · 219
16.3 图像、音频与视频编码原理 · · · · · 221
16.4 FFmpeg 编解码 API 流程 · · · · · 228
16.5 命令行处理与播放器功能 · · · · · 229
16.6 格式、性能与播放质量 · · · · · 236
第 17 章 Python · · · · · 240
17.1 Python 对象、类型与容器 · · · · · 240
17.2 Python 函数、迭代与对象协议 · · · · · 243
17.3 Python 异常、模块与工程接口 · · · · · 247
17.4 Python 并发与 asyncio · · · · · 248
17.5 CPython 内存与执行机制 · · · · · 251
第 18 章 工具命令与容器 · · · · · 253
18.1 Linux 命令与现场排查 · · · · · 253
18.2 Git 版本控制与协作 · · · · · 254
18.3 Docker 容器与部署工具 · · · · · 258
第 19 章 后端进阶与平台工程 · · · · · 262
19.1 存储选型与代码组织 · · · · · 262
19.2 订单、认证与接口边界 · · · · · 262
19.3 容量、隔离与运维 · · · · · 263
19.4 容器编排与平台边界 · · · · · 265
第 20 章 大模型与 AI 应用工程 · · · · · 266
20.1 Agent 架构与运行控制 · · · · · 266
20.2 RAG 检索与知识治理 · · · · · 270
20.3 LLM 工具调用与协议 · · · · · 274
20.4 大模型原理与工程 · · · · · 277
20.5 LangChain 与应用框架 · · · · · 281
20.6 AI 编程与交付验证 · · · · · 282
20.7 编程 Agent 扩展与持续运行 · · · · · 284
20.8 生成内容标记与溯源 · · · · · 285
参考文献 · · · · · 286

第 1 章 C++ 基础与面向对象

1.1 类、接口与多态

001 C++ 中如何使用友元函数（Friend Function）?

答案　友元函数是得到类特别授权的普通函数。它仍然不属于这个类，却可以读取或修改该类的私有、受保护成员。授权必须由类内部的 friend 声明给出，外部函数不能自行取得权限。它也没有成员函数的隐含 this 指针，需要通过参数指定操作哪个对象。

例如输出一个金额对象时，左侧是输出流，右侧才是金额。把输出运算符写成非成员友元，就能保持这种自然的调用顺序，同时读取内部保存的分数。友元应只服务于紧密协作的少数接口；友元关系不会自动传递、继承，也不意味着所有外部代码都能访问私有数据。

代码示例

```
#include <ostream>
class Money
{
    friend std::ostream& operator<<(std::ostream&, const Money&); // 类内明确授权
    int cents_ = 0; // 其余代码不能直接访问
};
std::ostream& operator<<(std::ostream& os, const Money& m)
{ // 是非成员函数
    return os << m.cents_; // 左侧流和右侧对象都可自然参与调用
} // 只授权这个紧密协作接口；不要为方便广泛开放 private
```

002 C++ 中如何实现接口?

答案　C++ 通常用抽象基类表示接口：接口列出外部可以调用的操作，把具体实现留给派生类。把成员函数声明为纯虚函数，也就是在声明末尾写 = 0，就不能直接创建这个接口类的对象。实现类继承接口，并用 override 标明自己在实现哪个操作，签名不匹配时编译器会报错。

例如绘图代码只接收“形状”，圆形和矩形各自计算面积，调用方无需知道具体类型。若对象可能通过基类指针删除，接口应提供公共虚析构函数，让派生部分也能正确释放。只在编译期要求某个类型具有指定操作时，也可以用模板或 C++20 concept，避免为了约束类型而强行建立继承关系。

代码示例

```
#include <memory>
struct Shape
{
    virtual ~Shape() = default; // 经接口删除派生对象时正确析构
    virtual double area() const = 0; // 接口规定行为，不能直接实例化
};
struct Circle final : Shape
{
    explicit Circle(double r)
        : r(r)
    {
    }
    double r;
    double area() const override
    {
        return 3.14 * r * r;
    }
};
std::unique_ptr<Shape> s = std::make_unique<Circle>(2.0);
auto a = s->area(); // 若无需运行时异构分派，可用模板/concept 避免虚调用
```

003 C++ 中如何实现运算符重载?

答案　运算符重载就是给已有运算符提供处理自定义类型的函数。例如为二维向量定义 operator+，表达式 a + b 就会调用这个函数。成员函数适合需要修改自身的操作；非成员函数常用于对称的二元操作，让左右两边采用更一致的转换规则。流输出也通常写成非成员函数，并返回流引用以便连续输出。

设计时应让符号保留读者熟悉的含义：向量加法产生新向量，+= 修改左边对象。重载不能创造新的符号，也不能改变优先级、结合方式或操作数数量；点号、作用域解析符和条件运算符不能重载。不要为了缩短代码而让加号执行文件删除等出人意料的动作，否则接口会比普通命名函数更难理解。

代码示例

```
struct Vec2
{
```

```
    double x, y;
    Vec2& operator+=(Vec2 b)
    {
        x += b.x;
        y += b.y;
        return *this;
    }
};
Vec2 operator+(Vec2 a, Vec2 b)
{
    return a += b;
} // 非成员使两个操作数对称
Vec2 p{1, 2}, q{3, 4};
Vec2 sum = p + q; // 保持“加法产生新值”的直觉
// 不改变运算符优先级或操作数个数；不能重载 ., :: 或 ?:
// 重载前先确认使用者会把它理解成普通向量加法
```

004 C++ 中的抽象类（Abstract Class）是什么？

答案　抽象类是仍然保留至少一个未实现纯虚操作的类，不能直接创建对象。它用于表达“所有实现都必须支持这些行为”，而不承诺自己已经提供完整功能。派生类可以实现一部分操作后继续保持抽象；只有所有继承的纯虚操作都有最终实现，派生类才可以被实例化。

例如读取接口规定 read，文件读取器和网络读取器分别提供实现，使用者通过接口引用调用即可。抽象类也能拥有数据、构造函数以及普通成员函数，并不是只能放函数声明。如果调用者通过基类指针销毁实现对象，应提供虚析构函数，使整个析构链执行；实际管理对象时通常交给智能指针，减少遗漏释放的机会。

代码示例

```
struct Reader
{
    virtual ~Reader() = default;
    virtual int read() = 0; // 未实现的纯虚函数令 Reader 抽象
};
struct FileReader : Reader
{
    int read() override
    {
        return 1;
    }
};
// Reader r; // 错误：抽象类不能直接构造
Reader* r = new FileReader;
delete r; // 虚析构保证派生析构也执行；实际代码优先用智能指针
```

005 C++ 中的模板特化（Template Specialization）是什么？

答案　模板特化是在通用模板之外，为特定参数提供专门实现。全特化把所有模板参数都固定下来，例如专门处理 int；类模板的偏特化只限定参数的一部分或某种形状，例如对所有 T* 指针采用一套规则。编译器根据实参选择匹配的版本，因此差异在编译期确定，不需要运行时判断类型。

例如一般类型按值打印，而指针类型还需要判断是否为空，这时类模板偏特化就能表达规则。函数模板不能偏特化，通常用函数重载或约束表达类似选择。特化声明必须在需要选择它的位置可见，也要检查多个候选是否歧义；如果简单重载已经足够清楚，就没有必要建立复杂的特化层次。

代码示例

```
template <class T> struct Kind
{
    static constexpr int value = 0;
};
template <class T> struct Kind<T*>
{
    static constexpr int value = 1;
}; // 类模板偏特化
static_assert(Kind<int>::value == 0 && Kind<int*>::value == 1);
int score(int x)
{
    return x;
}
```

```cpp
double score(double x)
{
    return x;
} // 函数模板“偏特化”用重载表达
// 特化改变匹配类型的行为；优先用清晰重载/concept，避免歧义与声明不可见
```

006 C++ 中的类型擦除（Type Erasure）是什么？

答案　类型擦除是在统一接口背后保存不同的具体类型，让调用方只关心它们共同支持的操作。这里“擦除”的是接口上可见的类型信息，对象实际仍有具体类型。例如 std::function<int(int)> 可以先保存加法闭包，再换成乘法闭包，调用者始终用同一种函数调用方式。

常见实现先用模板生成对应类型的包装器，再通过虚函数或函数指针表转发操作。std::any 则用于保存符合要求的不同值，读取时还要检查或指定实际类型。统一接口换来了运行时替换能力，也可能增加间接调用、对象大小或动态分配。小对象是否能直接放进包装器取决于实现，不能假设所有闭包都不分配内存。

代码示例

```cpp
#include <functional>
std::function<int(int)> operation;
operation = [](int x)
{
    return x + 1;
}; // 可替换为其他同签名闭包
operation = [](int x)
{
    return x * 2;
};
int answer = operation(3); // 统一运行时接口隐藏闭包具体类型
// 类型擦除通常带间接调用；目标过大时可能分配，SBO 由库实现决定
```

007 C++ 中的纯虚函数（Pure Virtual Function）是什么？

答案　纯虚函数是在虚函数声明后写 = 0 的成员函数，用来要求具体派生类提供实现。只要类还存在没有完成实现的纯虚操作，它就是抽象类，不能直接实例化。派生类用 override 给出正确签名的覆盖版本后，可以通过基类引用或指针调用，最终执行派生实现。

纯虚不等于绝对不能有函数体：普通纯虚函数也可以在类外提供公共实现，派生类通过限定名称调用它，但基类仍然是抽象类。纯虚析构函数尤其必须有定义，因为销毁派生对象时基类析构一定会执行。不要只写纯虚析构声明便认为“无需实现”，否则可能在链接阶段发现析构函数缺少定义。

代码示例

```cpp
struct Interface
{
    virtual ~Interface() = 0;
}; // 纯虚析构仍使类抽象
Interface::~Interface() = default; // 仍必须给析构函数提供定义
struct Impl : Interface
{ /* 可继承析构 */
};
// Interface x; // 错误：纯虚析构也算纯虚函数
Interface* p = new Impl;
delete p; // 析构链会调用 Interface::~Interface()
```

008 C++ 中的继承有几种类型？

答案　按访问控制区分，继承有 public、protected 和 private 三种。公共继承保留基类公共和受保护成员的对应级别；保护继承把它们都变成派生类的受保护成员；私有继承把它们都变成派生类的私有成员。无论选哪一种，派生类都不能因此直接读取基类自己的私有成员。

公共继承通常表示“派生对象也是一种基类对象”，可以交给接收基类的代码使用，例如文件读取器作为读取器。保护和私有继承更多限制外部转换与可见性。若只是想复用一个工具对象的功能，把它作为成员的组合方式往往更清楚，不必把实现复用误写成可以任意替代的类型关系。

代码示例

```cpp
struct Service
{
    virtual void run() = 0;
    virtual ~Service() = default;
};
struct Worker : public Service
{
```

```
    void run() override
    {
    }
}; // public 表示可替代
struct Detail
{
};
class Wrapper
{
    Detail detail_;
}; // 实现复用常可用组合，避免私有继承耦合
// protected/private 继承会改变基类 public/protected 成员可见级别
// 三种继承都不会让派生类直接访问基类 private 成员
```

009 C++ 中的虚函数是什么?

答案　虚函数让同一个基类接口在运行时表现出不同的行为。通过基类指针或引用调用虚函数时，选择依据是所指对象的实际类型，而不只是变量声明的类型。例如一个形状引用实际绑定圆形，调用面积时就执行圆形的计算逻辑。派生函数写 override 可以让编译器检查它是否真的覆盖了基类函数。

保留动态行为需要保留完整派生对象。如果把派生对象按值复制到基类变量，多出的派生部分会被切掉，这叫对象切片，此后调用只面对新的基类对象。编译器常用虚函数表完成分派，但表的具体格式与对象布局由实现和 ABI 决定，语言标准没有规定；因此不要靠猜测表位置编写可移植程序。

代码示例

```
struct Base
{
    virtual int id() const
    {
        return 0;
    }
    virtual ~Base() = default;
};
struct Child : Base
{
    int id() const override
    {
        return 1;
    }
};
Child c;
Base& ref = c;
int dynamic_result = ref.id(); // 按动态类型分派到 Child
Base sliced = c;
int sliced_result = sliced.id(); // 按值复制发生切片，结果来自 Base
// override 可检查签名；虚表是常见实现，不是标准规定的对象布局
```

010 C++ 中的虚拟继承是什么?

答案　虚继承主要解决菱形继承中“同一个祖先出现两份”的问题。例如左分支和右分支都继承根类，最终类又继承这两个分支，普通继承会让最终对象含有两个根类子对象，访问同名根成员时必须说明走哪条路径。中间两条继承都声明为 virtual 后，最终对象共享同一份虚基类子对象。

共享也带来了特殊的构造规则：虚基类由最派生类负责初始化，因此最终类创建对象时，中间分支为根类写的初始化不能代替最终类的选择。虚继承会让对象布局、地址调整和初始化关系更复杂，具体成本取决于 ABI。它和虚函数用途不同，不应仅因类里有虚函数就自动添加虚继承。

代码示例

```
struct Root
{
    explicit Root(int n)
        : n(n)
    {
    }
    int n;
};
struct Left : virtual Root
{
```

```
    Left()
        : Root(1)
    {
    }
};
struct Right : virtual Root
{
    Right()
        : Root(2)
    {
    }
};
struct Join : Left, Right
{
    Join()
        : Root(7)
        , Left()
        , Right()
    {
    }
};
Join j; // 只有一个共享 Root 子对象，值由最派生类 Join 初始化
// 普通继承会有两份 Root，访问成员可能歧义
// 虚基类布局/偏移依 ABI；虚继承也让构造顺序更复杂
```

011 C++ 中的赋值运算符重载如何实现?

答案　赋值运算符把已有对象的状态替换成另一个对象的状态，通常返回自身的引用，才能支持 a = b = c 连续赋值。对于拥有动态内存的类，关键不只是复制指针，而是明确资源是否需要独立复制，旧资源何时释放，以及自己给自己赋值时会不会先删掉仍要读取的数据。

一种可靠做法是先构造完整副本，再用不抛异常的交换操作替换自身状态；临时副本随后销毁旧资源。如果复制失败，原对象保持不变，这就是强异常保证。移动赋值则转移所有权，并让源对象仍能安全析构。只有真实实现不会抛异常时才标注 noexcept；资源已经交给容器时，优先采用编译器生成的赋值操作。

代码示例

```
#include <utility>
class Buffer
{
  public:
    Buffer& operator=(const Buffer& other)
    {
        Buffer temp(other); // 先完整构造副本；若分配失败原对象不变
        swap(*this, temp);
        return *this; // 按引用返回支持 a = b = c
    }
    friend void swap(Buffer& a, Buffer& b) noexcept
    {
        a.swap_data(b);
    }
    void swap_data(Buffer&) noexcept; // 示意：仅交换资源句柄/状态
}; // 自赋值也安全；移动赋值另需让源对象可安全析构
```

012 C++ 和 C 有什么区别?

答案　C 和 C++ 都能直接操作内存并调用系统接口，但提供的组织程序方式不同。C 主要使用函数和结构体，C++ 在此基础上增加类、继承、模板、函数重载、异常、命名空间和更多标准库设施。开发者可以按问题选择过程式、面向对象或泛型写法，不必所有代码都围绕继承组织。

C++ 并不是 C 的严格超集：部分语法、类型转换和语言规则不同，一份能通过 C 编译的源码未必能通过 C++ 编译。两者互相调用时常使用 C 风格接口，并在 C++ 一侧用 extern "C" 指定语言链接。接口还要约定资源归谁释放和错误如何返回，避免直接暴露标准库对象或让异常越过未经约定的语言边界。

代码示例

```
// C ABI 常用于让 C 与 C++ 模块共享稳定边界
extern "C" int add_numbers(int a, int b)
{
    return a + b;
```

```
}
// C++ 调用方可使用类、模板和重载；C 调用方只见 C 风格函数签名
// C++ 保留许多 C 能力，但不是严格超集，源代码规则并不完全相同
// 跨语言边界避免直接暴露 std::string、模板类型或编译器特定类布局
// 资源所有权和错误码也要约定，避免异常穿过 C ABI
```

013 C++ 多态如何分类与实现？

答案　C++ 多态可以理解为同一种调用方式适配不同类型。编译期多态通过重载、模板和约束选择实现，编译时已经知道调用目标；运行期多态通过虚函数，在基类指针或引用指向不同实际对象时选择对应实现。两者可以配合使用，而不是必须在整个工程里只选一种。

例如一个模板函数能对任何支持 area 的类型计算面积，编译器可为每种类型生成并优化代码；一组在运行时装入的形状则适合存放为基类智能指针，并通过虚接口调用。模板常便于内联，但大量实例可能增加代码体积；虚接口便于管理异构集合，却可能带来间接调用。若通过多态基类销毁对象，基类应具备合适的虚析构函数。

代码示例

```
struct Base
{
    virtual int value() const = 0;
    virtual ~Base() = default;
};
struct One : Base
{
    int value() const override
    {
        return 1;
    }
};
template <class T> int static_call(const T& x)
{
    return x.value();
} // 编译期绑定
int dynamic_call(const Base& x)
{
    return x.value();
} // 运行期按动态类型分派
One x;
int a = static_call(x), b = dynamic_call(x);
// 模板利于内联但增加实例代码；虚接口支持异构集合但有间接调用成本
```

1.2 对象模型、生命周期与类型

014 C++ 对象通常有哪些存储区？标准是否规定栈和堆？

答案　判断 C++ 对象能活多久，应先看存储期，而不是背诵固定的内存分区。自动存储期对象通常在进入作用域后创建、离开时销毁；静态存储期对象通常伴随整个程序；线程存储期对象每个线程各有一份；动态存储期对象由显式分配和释放机制管理，常交给智能指针作为所有者。

操作系统实现中确实常见代码段、全局数据区、线程栈和动态分配区，但“局部对象一定在物理栈上”“栈一定向某方向增长”都不是语言标准的保证。例如局部 unique_ptr 自身可以是自动对象，它拥有的目标却是动态对象，两个对象的存储期不同。分析悬空引用时，应追踪目标对象是否还活着，而非仅凭地址看起来属于哪一区域。

代码示例

```
#include <memory>
int global = 0; // 静态存储期：整个程序期间存在
thread_local int per_thread = 0; // 每个线程各有一份
void f()
{
    int local = 1; // 自动存储期，离开作用域销毁
    auto dynamic = std::make_unique<int>(2); // 动态对象由所有者托管
} // 判断生命周期看存储期/所有权；“栈、堆”只是常见实现布局说法
// 标准不规定对象必须落在哪个物理分区或栈的增长方向
```

015 C++ 的默认特殊成员函数有哪些？Rule of Three/Five 是什么？

答案　默认构造、析构、拷贝构造、拷贝赋值、移动构造和移动赋值，是 C++ 的特殊成员函数。编译器会依据类中成员以及程序员已经声明的函数，决定是否隐式声明它们、能否生成定义，或者将某项操作删除。因此“没有手写”不等于六个操作必然全部可用。

三法则和五法则是在提醒资源管理的一致性：一旦自己写析构释放资源，就要同时考虑复制是否产生独立资源，以及移动怎样转交所有权。仅复制拥有资源的裸指针会导致重复释放。更常用的零法则是让 string、vector、unique_ptr 等成员承担资源管理，使类无需手写特殊成员；能否复制由成员自然决定，语义更容易检查。

代码示例

```
#include <string>
#include <vector>
struct User
{
    std::string name;
    std::vector<int> scores;
}; // Rule of Zero
// vector/string 自己处理资源；编译器生成的复制/移动通常就是正确语义
struct Raw
{
    int* p;
    ~Raw()
    {
        delete p;
    }
}; // 自管裸资源需审视拷贝和移动
// Raw 若默认拷贝会双重释放；应禁拷贝并实现移动，或改用 RAII 成员
// Rule of Three/Five 是检查清单，优先通过 Rule of Zero 避免手写
```

016 const 在变量、指针、引用和成员函数中分别有什么作用?

答案　const 表达的是某条访问路径上的只读约束。const T* 不允许通过指针修改目标，但指针可以改指向；T* const 固定指针自身，却仍能修改非只读目标；const T& 则提供对象的只读别名。这些声明应分别看清“哪个东西不能改”，不要把指针只读和目标只读混为一谈。

const 成员函数不能直接修改普通成员，也不能调用要求可修改自身的普通成员函数，适合表达查询操作。mutable 成员可用于缓存或锁等内部状态，但并不自动线程安全。const 对象也可能在运行时才完成初始化；需要参与编译期求值时，应考虑 constexpr，并满足对应的常量表达式规则。

代码示例

```
int x = 1;
const int* pointee_readonly = &x; // 不能经此指针修改 x
int* const fixed_pointer = &x; // 指针不能改指向
const int& read_only_ref = x; // 只读别名
struct Cache
{
    mutable int hits = 0;
    int get() const
    {
        ++hits;
        return 1;
    }
};
// const 不等于编译期常量；constexpr 还要求表达式满足常量求值规则
```

017 explicit 关键字有何作用?

答案　explicit 用来阻止某些自动类型转换，让创建对象的意图写在调用处。例如端口类型只有一个接收整数的构造函数，如果没有 explicit，传整数时可能自动生成端口对象；加上它以后，Port{8080} 这样的直接构造仍然允许，但把普通整数隐式当成 Port 使用会被拒绝。

它主要保护接口边界，避免一个看似无关的值意外进入转换链，而不是禁止对象构造。C++11 起，用户定义的转换函数也可以声明 explicit；C++20 允许按条件决定是否显式。给已有接口补上 explicit 可能影响依赖隐式转换的旧调用，需要检查参数传递和初始化方式，并把需要的转换明确写出来。

代码示例

```
struct Port
{
    explicit Port(int n)
        : value(n)
    {
    }
    int value;
};
```

```
Port a{8080}; // 直接初始化可调用 explicit 构造
// Port b = 8080; // 错误：阻止意外的隐式转换
void connect(Port);
// connect(8080); // 错误；connect(Port{8080}) 意图清晰
// C++20 还可写 explicit(condition)；添加 explicit 时检查旧调用点
```

018 mutable 关键字是什么？

答案　mutable 允许一个非静态成员在 const 成员函数中被修改。它适用于“外部看到的值没变，但内部实现状态需要变化”的情况，例如查询时记录访问次数、延迟计算缓存，或者给保护缓存的互斥锁加锁。这样对外仍可提供只读接口，而不必把所有查询函数都声明成可修改对象。

这种权限只是绕开成员的常量限制，不会自动提供原子性或线程同步。多个线程同时更新 mutable 缓存或计数器，仍然可能产生数据竞争，需要用同一把锁或合适的原子操作保护。也不要用它修改本应保持不变的业务字段，否则 const 接口所承诺的逻辑只读就失去意义；对象本身还必须处于有效生命周期中。

代码示例

```
#include <mutex>
class Memo
{
    mutable std::mutex m_; // const 查询仍可同步缓存
    mutable bool ready_ = false;
    mutable int cached_ = 0;

  public:
    int value() const
    {
        std::lock_guard<std::mutex> lock(m_);
        if (!ready_)
        {
            cached_ = 42;
            ready_ = true;
        }
        return cached_;
    }
}; // mutable 适合逻辑上不改变结果的缓存/锁
// mutable 本身不提供同步；并发访问必须加锁或使用原子类型
```

019 sizeof 和 alignof 的作用是什么？

答案　sizeof 给出一个类型或表达式对应对象表示所占的字节数，alignof 给出类型要求的对齐。例如数组的 sizeof 包含全部元素，而指针的 sizeof 只有指针自身的大小，不能由此推断它指向多少元素。这里的字节是语言定义的存储单位，sizeof(char) 始终为一。

结构体为满足成员对齐，成员之间和末尾都可能插入空隙，所以结构体大小不一定等于各成员大小之和。调整成员顺序有时可以减少空隙，但具体结果由平台、编译器和 ABI 决定。把本机对象内存直接写入网络或文件会把填充、字节序和布局一起带出去；跨平台格式应明确各字段编码，而不是依赖一次 sizeof 的测量结果。

代码示例

```
#include <iostream>
struct Record
{
    char tag;
    double value;
};
std::cout << sizeof(Record) << ' ' << alignof(Record); // 实际数值依 ABI
std::cout << ' ' << sizeof(char); // 单个成员类型大小
// 成员间/末尾可能填充以满足对齐；数组大小含所有元素
// 不要把本机 sizeof/布局直接当网络协议或跨平台文件格式
```

020 static 在 C++ 中有哪些常见用法？

答案　static 的含义要结合声明位置。函数内的局部 static 对象只在首次执行到声明时初始化，随后多次调用共享同一个对象，存储期延续到程序结束阶段。命名空间作用域的 static 通常让名字仅在当前翻译单元可链接；类中的 static 数据成员则属于类整体，不是每个实例各有一份。

静态成员函数没有 this，因而不能直接使用某个实例的普通成员，也不能是 virtual，或带成员函数的 cv/ref 限定。非 inline 静态数据成员通常还需类外定义，C++17 可用 inline static 在类内定义。C++11 保证局部 static 的首次初始化线程安全，但初始化之后的共享读写仍须加锁或采用原子操作，不能把两件事混为一谈。

代码示例

```cpp
struct Registry
{
    static int count;
}; // 类级数据，不属于某个实例
int Registry::count = 0; // 非 inline 成员在一个源文件定义一次
int& shared_state()
{
    static int state = 0;
    return state;
}
// 首次经过时初始化由 C++11 保证线程安全
++shared_state(); // 初始化安全不代表之后的并发写安全
// 命名空间 static 通常内部链接；局部 static 生命周期贯穿程序
```

021 typedef 和 using 有何区别?

答案　typedef 和 using 都是给已有类型增加一个名字，不会创造不同的新类型。例如把 unsigned long 命名为 Id 后，Id 和原类型仍然相同，不能依靠这个别名阻止金额与编号相互传递。普通别名两种写法都能表达，现代代码中 using 的“名字等于类型”形式往往更直观。

using 还能直接定义别名模板，例如 Vec<T> 表示 vector<T>，这点 typedef 不能直接做到。面对复杂函数指针时，using 也更容易读出别名名称和完整类型。选择写法主要看可读性和工程风格；如果需要真正区分不同业务含义，并限制错误转换，应定义包装类，而不是期待类型别名带来额外的类型安全。

代码示例

```cpp
#include <vector>
typedef unsigned long LegacyId;
using Id = unsigned long; // 现代别名写法更易读
template <class T> using Vec = std::vector<T>; // 别名模板
Vec<int> values{1, 2};
// typedef 也能命名普通类型，但不能直接声明 template alias
// 两者都只是别名，不会创建新类型或阻止隐式转换
```

022 什么是 C++中的多重继承，如何避免菱形继承?

答案　多重继承允许一个类同时继承多个基类，适合一个对象实现多种独立接口。问题出现在两条继承路径又指向同一个祖先时：普通继承会保留两份祖先子对象，各自有自己的状态，最终对象直接访问祖先成员就可能歧义。这种结构被称为菱形继承，并非所有多重继承都会形成菱形。

如果业务上确实只有一份共同祖先，两条中间路径可以使用虚继承，让最终对象共享同一份虚基类。共享虚基类由最派生类负责初始化，布局和地址调整也更复杂。设计时先区分“实现多个接口”和“复用多份状态”；若只是需要工具功能，组合通常更容易维护，不必通过不断增加虚继承修补不必要的继承关系。

代码示例

```cpp
struct A
{
    int id = 0;
};
struct B : virtual A
{
};
struct C : virtual A
{
};
struct D : B, C
{
}; // 虚继承使最底层 D 共享一个 A
D d;
d.id = 1;
// 普通继承会有 B::A 和 C::A 两个子对象，d.id 有歧义
// 若只为复用实现，组合通常比多重/虚继承更易维护
```

023 什么是内联函数（inline)?

答案　inline 不是“要求编译器把函数调用展开”的命令。优化器是否把函数体放进调用位置，会综合函数大小、可见性和优化选项；没有 inline 的函数也可能被展开，有 inline 的函数也可能保留调用。它更重要的语言作用，是允许满足单一定义规则的同一函数定义出现在多个翻译单元中。

因此，头文件中需要提供的小函数常用 inline，类定义内部定义的成员函数通常也隐含这一属性。各处定义仍须满足一致性要求，不能让不同源文件看到相互冲突的实现。性能上，大量展开可能增大机器码并损害指令缓存；应通过实际基准或生成代码判断收益，同时考虑头文件中的实现给所有包含者带来的编译成本。

代码示例

```
// 适合放在头文件供多个翻译单元包含
inline int clamp_low(int value)
{
    return value < 0 ? 0 : value;
}
int result = clamp_low(-1); // inline 不保证机器码展开
// inline 的语言作用包括允许符合 ODR 的相同定义跨翻译单元出现
// 编译器仍会根据优化和调用上下文决定是否内联；过度展开增大代码
// 大函数放头文件也会扩大每个包含者的编译工作
```

024 什么是对象切片？如何避免？

答案　对象切片发生在把派生对象按基类值复制的时候。新对象只有基类部分，派生类新增的数据和对应的动态行为都没有保留下来。例如把圆形赋给一个形状变量后，得到的是独立的形状对象，后续虚调用也不会再把它当作圆形。这不是简单的“指针类型变了”，而是真的创建了另一个不完整的值。

需要多态时，接口参数应使用基类引用或指针，容器可保存基类智能指针，让完整派生对象继续存在。引用和裸指针只是借用，仍要保证目标寿命足够长；智能指针则明确管理所有权。如果业务要求复制出独立的多态对象，可设计虚拟 clone，由每个派生类复制自身，而不是依赖按基类值复制。

代码示例

```
struct Shape
{
    virtual double area() const
    {
        return 0;
    }
    virtual ~Shape() = default;
};
struct Circle : Shape
{
    double area() const override
    {
        return 12;
    }
};
Circle c;
Shape copy = c; // 按值复制切掉 Circle 部分
Shape& view = c; // 引用保留动态对象
// API 用引用/指针传多态对象；需要多态复制时定义 clone() 明确语义
// 异构容器可保存 std::unique_ptr<Shape>
```

025 什么是构造函数和析构函数的顺序？

答案　对象构造顺序由类的结构决定，不由初始化列表书写顺序决定。先构造虚基类，再按基类声明顺序构造直接基类，然后按成员在类中声明的顺序构造成员，最后执行构造函数体。因此把后声明成员写到列表前面，并不能让它先初始化，更不应让先构造的成员依赖尚未构造的成员。

正常析构按构造完成的反方向执行：先运行当前类析构函数体，再逆序销毁成员和基类。作用域内的局部对象也按相反顺序清理，动态对象则在对应删除操作中销毁。若使用基类指针删除派生对象，基类应有虚析构函数，保证从派生部分开始完整清理；否则这种普通删除场景会产生未定义行为。

代码示例

```
#include <string>
struct Part
{
    Part(const char* n)
        : name(n)
    {
        log(name);
    }
    ~Part()
    {
        log("~" + name);
```

```
    }
    std::string name;
};
struct Base
{
    Base()
    {
        log("Base");
    }
    virtual ~Base()
    {
        log("~Base");
    }
};
struct Child : Base
{
    Part first{"first"}, second{"second"};
    Child()
        : second("second")
        , first("first")
    {
    } // 实际仍按声明顺序
    ~Child()
    {
        log("~Child");
    }
}; // 构造基类→成员声明序；析构反向；虚基类先于直接基类构造
```

026 命名空间有什么作用？头文件中如何避免污染全局命名空间？

答案　命名空间为一组名字增加所属范围，解决不同库恰好使用同名类或函数的冲突。例如 graphics::Color 和 terminal::Color 可以同时存在，调用处通过限定名说明使用哪一个。名字较长时，可以在函数内用单个 using 声明引入需要的名称，把影响范围控制在局部。

公共头文件中不要随意写 using namespace，因为头文件会被文本包含，这个指令的影响也会传给所有包含者，甚至让原来正常的代码产生重载歧义。实现文件中可使用匿名命名空间，隐藏仅本文件使用的辅助函数和变量。命名空间是组织名称的工具，不是类的访问控制，不能用它代替对私有数据的封装。

代码示例

```
namespace graphics
{
struct Color
{
};
} // namespace graphics
void draw()
{
    using graphics::Color; // 仅在本函数引入这个名字
    Color c;
}
// 头文件里不要写 using namespace graphics; 会污染所有包含者
// 直接使用 graphics::Color 最明确；匿名 namespace 可隐藏 .cpp 私有名字
```

027 如何实现 C++ 中的单例模式（Singleton）？

答案　单例通过限制创建入口，让使用者访问同一个实例。常见写法把构造函数设为私有，删除复制与移动操作，并在静态访问函数中定义局部 static 对象。这样第一次调用时创建对象，后续返回同一实例的引用；C++11 起，多个线程首次同时访问时，语言保证这个初始化过程只完成一次。

但单例不等于线程安全容器：实例创建好以后，多个线程读写其中数据仍需同步。还要考虑程序退出时其他全局对象是否会继续访问已经析构的实例。单例隐藏了全局依赖，也会让测试难以替换配置或隔离状态；如果生命周期关系复杂，把接口作为参数注入通常更容易控制，不能仅为方便访问就把普通服务都做成单例。

代码示例

```
class Config
{
    Config() = default;
```

```
  public:
    Config(const Config&) = delete;
    Config& operator=(const Config&) = delete;
    static Config& instance()
    {
        static Config one;
        return one;
    }
    int read() const
    {
        return 1;
    }
};
auto value = Config::instance().read();
// 局部 static 首次初始化线程安全；后续可变状态仍需同步
// 测试和析构顺序复杂时，优先依赖注入而非全局单例
```

028 引用（Reference）与指针（Pointer）有什么区别?

答案　引用通常表示一个必须存在的对象的别名，初始化时就要绑定，之后赋值是在修改目标，而不是把引用改绑到另一个对象。指针则保存地址，可以为空，也可以重新指向其他对象，访问目标时通常使用解引用或箭头。空指针是正常状态，所谓空引用则不是合法接口应依赖的正常语义。

接口可以用引用表达必填对象，用指针表达可选对象，但这仍未说明资源归谁管理。裸引用和裸指针通常都是借用，返回它们之前要确认目标不会先销毁，否则两者都会悬空。引用在机器实现中也可能需要保存地址，占不占额外空间由具体场景决定；不能把“语法上是别名”误解成任何情况下都绝对零成本。

代码示例

```
void use_required(int& x)
{
    ++x;
} // 引用表达必须存在的对象
void use_optional(int* x)
{
    if (x)
        ++*x;
} // 指针可为空且可改指向
int value = 0;
int& alias = value;
int* ptr = &value;
int other = 1;
ptr = &other; // 可重新指向；alias = other 是赋值 value
// 引用声明时须绑定，通常不能重新绑定；两者都不自动表达所有权
// 返回借用引用/指针时，调用方必须确保目标仍存活
```

029 构造函数和析构函数分别负责什么?

答案　构造函数负责把对象建立成可正常使用的状态，例如检查参数、初始化成员、获取需要拥有的资源。这些始终应该成立的状态条件叫对象不变量。析构函数在对象寿命结束时释放它拥有的资源，例如文件句柄、锁或动态内存；成员对象随后还会自动按逆序析构，不必逐个手动调用。

把资源获取与对象生命周期绑定起来，可以让正常返回和异常退出都走同一条清理路径，这就是 RAII 的核心。析构函数只应释放自己的资源，借来的指针不能随意删除；清理过程通常也不应让异常向外传播。通过基类指针销毁派生对象时需要虚析构函数，才能完整清理派生资源，否则在通常的删除方式下属于未定义行为。

代码示例

```
#include <memory>
#include <string>
struct FileHandle
{
    explicit FileHandle(int fd)
        : fd(fd)
    {
    }
    ~FileHandle()
    {
        close_fd(fd);
    }
```

```
    int fd;
    void close_fd(int);
};
void process()
{
    auto file = std::make_unique<FileHandle>(open_file());
    read_data(file->fd); // 正常返回或异常离开时都由析构清理
}
// 构造建立对象不变量/取得资源；析构释放自己拥有的资源
// 经基类指针删除派生对象时基类析构需 virtual
```

1.3 模板与初始化

030 模板（Template）是什么?

答案　模板用参数描述一族相似的函数或类。参数可以是类型、编译期的值，也可以是另一个模板；当具体参数确定时，编译器据此实例化对应实现。例如同一个求较大值的模板，既能处理整数，也能处理支持比较的其他类型，避免为每种类型重复写相同算法。

这种选择发生在编译期，所以模板能实现静态多态，并常给优化器提供内联机会。模板也可以进行编译期计算和类型选择，但错误信息、编译时间及大量实例的代码体积都可能更复杂。使用 concept 或其他约束说明参数必须支持哪些操作，可以让接口意图更明确；“模板接受任意类型”并不表示模板体对任意类型都一定合法。

代码示例

```
template <class T> T larger(T a, T b)
{
    return a < b ? b : a;
}
int i = larger(3, 5);
double d = larger(1.0, 2.0);
// 调用时编译器实例化适合 T 的实现，形成静态多态
// 可用模板复用算法而不用运行期虚调用
// 用 concept/约束表达所需操作，减少不清楚的模板报错和过度实例化
// 同一模板参数会生成相应实例，构建体积也要留意
```

031 深拷贝与浅拷贝有什么区别?

答案　浅拷贝复制对象的成员值；如果成员是裸指针，复制的只是地址，两个对象于是指向同一块资源。一个对象修改内容会影响另一个，先销毁的一方还可能让另一方悬空，若两边都尝试释放资源则会重复释放。深拷贝会为副本另行创建资源，使两个对象拥有互不依赖的数据。

共享本身未必有错：如果业务希望多个对象共同延长资源寿命，可以使用 shared_ptr 并明确共享所有权。若资源必须独立，则要定义正确的复制和移动策略。最省心的方式通常是把数据放进 vector、string、unique_ptr 等 RAII 成员，让成员自身管理复制和释放，遵循零法则，而不是手动复制裸指针。

代码示例

```
#include <vector>
struct Shallow
{
    int* p;
}; // 默认拷贝只复制地址，两对象会别名
struct Deep
{
    std::vector<int> values;
}; // 容器复制时复制元素
int external = 1;
Shallow a{&external};
Shallow b = a; // 两者指向同一 int
*b.p = 9; // 通过 b 修改会影响 a 所指的值
// 裸指针不说明是否拥有资源；不要让两个对象都负责 delete
Deep x{{1, 2}}, y = x;
y.values[0] = 9; // x 不受影响
// 若资源应共享，明确用 std::shared_ptr；通常优先 Rule of Zero
```

032 类型转换（static_cast、dynamic_cast、const_cast、reinterpret_cast）有什么区别?

答案　四种转换应按意图选择，而不是靠强制转换消除编译错误。static_cast 表达语言已定义的转换，例如数值类型互转或安全的向上转型；向下转型只有在对象类型确实匹配时才安全。dynamic_cast 对多态类型进行运行时检查，指针转换失败得到空指针，引用转换失败抛出 bad_cast。

const_cast 只调整 const/volatile 限定，不改变对象本身是否可修改；如果原对象本来就是 const，去掉限定后再写入会导致未定义行为。reinterpret_cast 用于少数受控的低层表示操作，不会自动完成对象构造，也不保证跨平台解释一致。转换越低层越应靠近边界，并说明对象类型、对齐和生命周期等前提。

代码示例

```
#include <typeinfo>
struct Base
{
    virtual ~Base() = default;
};
struct Child : Base
{
};
Base* b = new Child;
Child* checked = dynamic_cast<Child*>(b); // 运行时检查；失败得 nullptr
int n = static_cast<int>(3.8); // 明确数值转换，可能丢信息
const Base* cb = b;
Base* writable = const_cast<Base*>(cb); // 原对象非 const，去限定后可写
// 若最初对象本来就是 const，经转换修改会是未定义行为
// reinterpret_cast 只用于受控低层表示；不保证跨平台意义
// dynamic_cast 引用下转失败会抛 std::bad_cast；用完释放 b
```

033 默认构造函数和拷贝构造函数分别是什么？

答案　默认构造函数是无需调用方提供必需参数即可创建对象的构造函数；它可以由编译器隐式声明，也可以显式写成 = default。拷贝构造函数则用一个同类型对象初始化一个新对象，常见签名是 T(const T&)。它属于初始化行为，与给已经存在的对象赋值不同，后者调用拷贝赋值运算符。

例如 Item b(a) 和 Item c = a 都是在创建新对象，调用拷贝构造；已有对象 c = b 才是赋值。编译器生成哪些特殊成员函数取决于其他构造、拷贝或移动操作的声明，以及成员能否被相应操作。自定义了带参数构造函数后，默认构造不一定还存在；若确实需要，可明确 default，并检查其成员是否也能默认构造。

代码示例

```
struct Item
{
    Item() = default;
    Item(const Item&) = default;
    int value = 0;
};
Item first;
Item second(first); // 默认构造 vs 新对象拷贝构造
Item third = first; // 这里仍是拷贝构造，不是赋值
third = second; // 这是拷贝赋值，对象已存在
struct NeedsArg
{
    explicit NeedsArg(int)
    {
    }
};
// 编译器不一定生成默认构造；用户声明构造后通常需显式 default
```

034 C++ 中列表初始化和圆括号初始化有哪些区别？

答案　花括号列表初始化会检查窄化转换，避免把小数悄悄截成整数等信息损失；圆括号直接初始化一般不做这项拒绝。花括号初始化类对象时，还会优先考虑 initializer_list 构造函数，所以同一个类型使用括号和花括号，可能选中不同重载，结果也可能完全不同。

例如 vector<int>{3, 4} 表示两个元素，而 vector<int>(3, 4) 表示三个值为 4 的元素。聚合类型也可以按成员声明顺序用花括号填入字段值。不要只凭“花括号更安全”就忽略构造函数选择：设计和调用类时都应查看 initializer_list 重载，并在容易混淆的地方使用清晰的命名工厂或直接展示预期语义。〔S2〕

代码示例

```
#include <vector>
std::vector<int> items{3, 4}; // initializer_list 构造：两个元素
std::vector<int> fill(3, 4); // 圆括号：三个值为 4 的元素
// 花括号还检查窄化，int x{3.5}; 会编译失败
struct Pair
{
    int left;
```

```
    int right;
};
Pair p{1, 2}; // 聚合按成员顺序初始化
// 类有 initializer_list 重载时，花括号会优先匹配它，可能改变重载选择
```

1.4 指针、对象表示与调用接口

035 对象指针、void* 和成员指针分别能表达什么？大小是否固定？

答案　普通对象指针指向特定类型的对象；解引用前必须保证地址非空、对齐正确，并且目标对象已经开始且尚未结束生命周期。void* 可以暂存对象地址，但不带目标对象类型，不能直接按对象解引用；应在确认实际类型后转换回相应指针。C++ 不保证任意函数指针都能安全装进 void*。

成员指针表示“某个类的哪一个成员”，单独拿到它还不能访问数据或调用成员函数，必须结合一个该类对象，通过 .* 或 ->* 使用。成员函数中的 this 指向当前对象，静态成员函数则没有 this。普通对象指针、成员数据指针和成员函数指针可能有不同表示与大小；64 位机器上普通指针常见为八字节也只是 ABI 现象，不是所有指针类型的标准保证。malloc 只给原始存储，转换结果不会自动构造任意 C++ 对象。〔N1、N2〕

代码示例

```
int value = 3;
void* storage = &value;
int* p = static_cast<int*>(storage); // 还原真实对象类型
struct Item
{
    int score;
    int read_score() const
    {
        return this->score;
    } // 非静态成员中 this 指向当前对象
};
int Item::* member = &Item::score;
int (Item::*method)() const = &Item::read_score;
Item item{7};
int score = item.*member;
int copy = (item.*method)(); // 成员函数指针须绑定对象
```

036 如何区分函数指针、返回指针的函数和函数对象？

答案　这三种写法的声明含义不同：int (*f)(int) 是指向函数的指针；int* f(int) 是接收整数并返回整数指针的函数；函数对象则是一个可以像函数一样调用的对象，通常重载 operator()。声明里括号的位置决定类型，读反了就会把“保存函数地址”和“函数返回地址”混淆。

函数对象可保存每次调用共用的状态，lambda 也是一种闭包对象；带捕获的 lambda 通常不能转换为普通函数指针。普通函数指针本身不保存独立捕获状态。模板可直接接收多种可调用对象，在编译期针对具体类型调用；若需要运行期替换不同目标，可用 std::function 统一签名，但可能带来间接调用或分配成本。返回指针的函数还必须说明所有权和有效期。〔N1、N3〕

代码示例

```
int twice(int x)
{
    return x * 2;
}
int (*function_ptr)(int) = &twice;
struct Add
{
    int bias;
    int operator()(int x) const
    {
        return x + bias;
    }
};
Add add{3};
int result = add(function_ptr(2)); // 7，函数对象还保存 bias
```

037 sizeof(1 == 1) 为何在 C 与 C++ 中可能不同？

答案　先判断相等比较表达式的结果类型，再求它的大小。在 C 语言中，内建比较表达式产生 int 类型的 0 或 1，因此 sizeof(1 == 1) 等于 sizeof(int)；在 C++ 中，比较结果类型是 bool，所以结果是 sizeof(bool)。常见平台会分别看到 int 为四字节、bool 为一字节，但语言不要求这些常见数字。

面试时的关键是区分类型规则与目标实现的大小：具体输出要查编译器和平台，不能从“64 位”直接推出 int 或 bool 的字节数。sizeof 的计量单位是字节，sizeof(char) 固定为一；一个字节包含多少位由 CHAR_BIT 说明。若结果用于二进制布局或协议，还要显式约定格式，不能假设不同实现的整数布局一致。〔N4、N5、N6〕

代码示例

```
const bool equal = (1 == 1);
const auto cpp_size = sizeof(equal); // C++ 中比较表达式的类型是 bool
// C 中比较表达式的类型是 int；具体大小由实现决定
```

038 C++ 的 struct 与 class 默认规则有何差异？和 C 的 struct 如何区别？

答案　C++ 中的 struct 和 class 都可以包含数据、成员函数、构造析构、模板成员和继承关系，主要差异是默认访问权限：struct 默认 public，class 默认 private；两者的默认继承权限也分别是 public 和 private。显式写出权限后，它们的能力基本相同，不存在“struct 只能放数据”的语言限制。

C 语言里的 struct 不具备 C++ 的成员函数、访问控制和继承机制，变量声明时通常还需写 struct 标签名，除非另用 typedef 定义别名。工程习惯上常用 struct 表达公开数据组合，用 class 表达维护不变量的抽象，但这只是可读性约定。选择 struct 不会自动让对象可安全 memcpy、跨平台序列化或变成固定布局，布局仍需按标准属性和实现条件确认。〔N7〕

代码示例

```
struct Record
{
    const char* title;
    int value;
}; // struct 默认 public
class Account
{
    int balance_ = 0;

  public:
    int balance() const
    {
        return balance_;
    }
};
Record income{"收入", 5}; // class 默认 private；成员函数可封装状态
```

039 union 如何共享存储？为何不能随意读取另一个成员？

答案　struct 的普通成员各自保存值；union 的所有成员重叠使用同一段存储，大小至少能容纳最大成员并满足对齐要求。通常任一时刻只有一个成员处于活动状态，先写整数再把同一段字节直接读成浮点数，不是通用安全的类型转换方式。违反活动成员规则可能产生未定义行为，不能把底层位模式碰巧相同当作可移植保证。

如果某种类型可能取整数或字符串等几种值，std::variant 会把当前活动类型和生命周期一同管理，访问时还能检查类型是否匹配。若因低层协议确实要用 union，就需要显式管理非平凡成员的构造和析构，并遵守语言规定的特殊情形。网络协议或文件格式应按字段逐项编码，不能直接发送本机 union 的大小、对齐和布局。〔N8〕

代码示例

```
std::variant<int, std::string> value = 12;
value = std::string("ready"); // 自动切换活动类型与生命周期
if (const auto* s = std::get_if<std::string>(&value))
{
    use(*s); // 不把同一段字节强行解释为另一种活对象
}
```

040 为什么没有虚构造函数？构造和析构期间的虚调用如何分派？

答案　构造函数建立特定类型的对象和它的基类、成员状态；初始化表达式已经指出要创建什么类型，所以 C++ 不提供 virtual 构造函数。若实际派生类型到运行时才确定，可用工厂函数依据输入决定创建哪个派生对象，并返回基类智能指针；需要多态复制时可定义虚 clone 接口。

构造或析构期间调用虚函数，分派会限制在当前正在构造或析构的类范围内，不会调用尚未开始构造或已经析构的更派生部分。因此在基类构造函数中调用一个虚操作，不应期待它执行派生实现；若该操作是纯虚，更不能这样依赖。把问题说成“此时虚表还没分配”不准确，应按语言规定的当前对象动态类型理解。〔N9、N10〕

代码示例

```
struct Reader
{
    virtual ~Reader() = default;
    virtual void start() = 0;
};
```

```
std::unique_ptr<Reader> make_reader(bool use_file)
{ // 工厂在构造完成后选择具体类型
    if (use_file)
        return std::make_unique<FileReader>();
    return std::make_unique<NetworkReader>();
}
```

041 重载、虚函数覆盖和派生类名字隐藏怎样区分?

答案　重载是多个同名函数同时参与候选，编译器根据参数类型和转换规则选择一个；只改变返回类型不能形成重载。覆盖则要求派生类提供与基类虚函数匹配的实现，基类引用或指针调用时才发生运行时分派。写 override 能明确表示覆盖意图，并让编译器检查参数、const 等签名是否真的对应。

隐藏发生在名称查找阶段：派生类声明了某个名字后，基类中同名的其他参数版本可能不再被正常找到；可以用 using Base::f 把基类重载加入派生类作用域。非虚函数同名声明只是隐藏，不能变成动态覆盖。默认参数还在编译期按静态声明类型选择，不会随着虚函数动态分派一起改变。〔N10〕

代码示例

```
struct Base
{
    virtual void f(int);
    void f(double);
};
struct Derived : Base
{
    using Base::f; // 保留基类的另一个重载
    void f(int) override; // 由编译器检查真实覆盖
};
```

042 空类、含虚函数的类和多重继承对象的大小怎样判断?

答案　空类完整对象也必须占用非零空间，这样连续数组中不同元素才有可区分的地址；常见实现会让它大小为一，但具体大小不应写死。空基类优化或 [[no_unique_address]] 可在特定子对象场景中让空对象与其他成员共享地址，不会因此允许两个独立完整对象地址相同。

类的实际尺寸还受成员、基类、对齐填充和实现的多态布局影响。常见 ABI 会为虚函数分派安排指针或表项，多重继承下还可能需要多个视图和 this 调整，但语言标准没有规定统一对象布局。不要推断“空类一定 1 字节”或“有虚函数就固定多八字节”；应在目标平台、编译器和选项下测量，并确认 ABI 工具能说明具体布局。〔N11、N6〕

代码示例

```
struct Empty
{
};
struct Poly
{
    virtual ~Poly() = default;
};
struct Value
{
    int value;
};
static_assert(sizeof(Empty) >= 1);
// sizeof(Poly)、sizeof(Value) 用目标 ABI 实测，不写跨平台固定值
```

1.5 数值表示与算术边界

043 补码为何能统一整数加减，C++ 有符号与无符号溢出的规则相同吗?

答案　补码用固定宽度的位表示有符号整数，使加减可以复用模二进制加法：减去一个数可以转成加上它的相反数。N 位补码有符号数的范围通常表达为 −2^(N−1) 到 2^(N−1)−1，负数多一个取值。注意，位的表示方式和语言规定的算术行为是两回事，不能从“硬件会回绕”推断程序可以依赖这种结果。

C++ 的无符号算术按模 2^N 回绕；有符号整数结果超出可表示范围仍是未定义行为。混合有符号和无符号数会经过整数提升与通常算术转换，负数可能转为很大的无符号值。进行边界计算时，应先验证输入范围，选择足够宽的类型，或使用有检查的运算；不要先溢出再试图通过比较结果检测。〔I47〕

代码示例

```
std::uint8_t wrapped = static_cast<std::uint8_t>(255u + 1u); // 无符号按模 256 回绕
if (value > std::numeric_limits<int>::max() - increment)
    return std::nullopt;
return value + increment; // 先检查，再执行有符号加法
```

044 浮点数怎样表示，为什么 0.1 + 0.2 可能不等于 0.3，应该怎样比较？

答案　常见二进制浮点数用符号、指数和有效数近似表示实数。像十进制 0.1 这样的数在二进制里是无限循环小数，只能存成附近可表示的值；每次运算还会舍入。因此常见 double 下 0.1 + 0.2 与 0.3 可能落到相邻的表示值，但这不意味着所有浮点运算都不精确，也不意味着应完全避开浮点数。具体格式和精度应查 std::numeric_limits 等实现信息。近似数比较要结合业务可接受误差：绝对容差适合接近零时的判断，相对容差适合不同量级的数值，再把两者按需求组合。比较前还要处理 NaN、无穷等特殊值，不能机械套一个固定 epsilon。金融金额通常应使用整数最小单位或适当的十进制类型，避免把近似浮点累计误差当成精确账目。〔I48、I49、I50〕

代码示例

```
double x = 0.1 + 0.2, y = 0.3;
double tolerance = std::max(abs_tol, rel_tol* std::max(std::abs(x), std::abs(y)));
bool close = std::abs(x - y) <= tolerance; // 容差应来自业务精度要求
```

第 2 章 内存与资源管理

2.1 所有权与 RAII

001 C++ 中如何使用智能指针？

答案　智能指针是用对象表达资源所有权的标准库类型。unique_ptr 表示独占：不能复制，但可移动给新所有者；shared_ptr 表示多个所有者共同延长对象寿命；weak_ptr 只观察 shared_ptr 管理的对象，不增加强引用。通常先选 unique_ptr，只有确实需要多个地方共同拥有时才用 shared_ptr。

用 make_unique 或 make_shared 创建对象，可以减少裸 new 造成的清理遗漏。注意 shared_ptr 解决的是共同生命周期，不自动解决对象内容的并发访问。若两个对象互相用 shared_ptr 指向对方，引用计数永远无法归零，形成泄漏环；对于不拥有对方的一侧改用 weak_ptr，并在使用时调用 lock 临时取得可用的共享所有权。

代码示例

```
#include <memory>
struct Node
{
    std::weak_ptr<Node> parent;
    std::shared_ptr<Node> child;
};
auto root = std::make_shared<Node>(); // 多方确实共享生命周期才用 shared_ptr
auto child = std::make_shared<Node>();
root->child = child;
child->parent = root; // 反向观察用 weak_ptr，避免引用环导致永不释放
auto sole = std::make_unique<Node>(); // 单一所有者优先用 unique_ptr
// weak_ptr::lock() 成功时才临时取得共享所有权
```

002 C++ 中如何管理内存？

答案　管理内存先要明确谁负责销毁对象、资源需要活多久，再选择合适的对象和所有者。局部变量通常随作用域结束析构，静态对象的存储期通常贯穿程序，线程局部对象每个线程各有一份；需要动态寿命时，可让智能指针或容器托管动态存储，避免每条控制路径手工释放。

现代 C++ 通常使用 vector、string、unique_ptr 等 RAII 类型，让构造和析构自动配对分配与释放。标准关注对象的存储期和生命周期，不规定它必须位于名为“栈”或“堆”的物理区域。例如 unique_ptr 本身可以是局部变量，它管理的对象具有动态存储期。准确表达所有权，比猜测对象在机器内存的哪块区域更有用。

代码示例

```
#include <memory>
#include <vector>
int global = 0; // 静态存储期
thread_local int scratch = 0; // 每个线程一份
void work()
{
    std::vector<int> local{1, 2}; // 自动对象，离开作用域析构
    auto heap = std::make_unique<int>(3); // 动态对象由局部所有者托管
} // 不直接依赖“栈/堆”说法；标准关心存储期与对象生命周期
// 分配细节由实现决定，确定所有权比猜物理区域更重要
```

003 C++ 中的内存对齐（Memory Alignment）是什么？

答案　内存对齐规定对象地址要满足的边界，例如某种类型可能要求地址是其对齐值的倍数。alignof 可查询类型的对齐要求，alignas 可为变量或类型请求更严格的对齐。结构体成员之间可能插入填充字节，使每个成员都位于合规地址，整个结构体大小也可能向对齐边界补齐。

调整成员声明顺序有时能减少填充，但尺寸与布局由平台、编译器和 ABI 决定。过度对齐的类型在动态分配时还要确保分配方式支持该对齐。不要把本机结构体直接写成文件或网络数据，填充、字节序和成员布局并非通用协议；应按字段定义编码格式，并在需要时显式处理对齐。

代码示例

```
#include <cstddef>
struct Plain
{
    char c;
    double value;
}; // 成员间可能出现填充
struct PackedOrder
{
    double value;
    char c;
}; // 顺序可能减少填充
struct alignas(64) Slot
{
    int value;
}; // 请求更严格对齐
static_assert(alignof(Slot) >= 64);
// 数值/布局仍依实现；过度对齐动态分配需配套正确分配接口
// 文件和网络协议逐字段编码，不直接写入结构体原始字节
```

004 C++ 中的垃圾回收与手动内存管理有什么区别?

答案　传统 C++ 通常用 RAII 管理对象和外部资源：对象在作用域退出时确定执行析构，适合及时关闭文件、释放锁、提交或回滚事务。追踪式垃圾回收则从根对象出发找仍可达的对象，之后回收不可达内存，开发者不必在每个分支手写释放动作；但回收时间由收集器策略决定，可能引入暂停。

内存回收不等于及时释放所有资源。即使 GC 管理了文件包装对象，文件句柄、锁和事务仍有各自的释放时机，不能仅靠等待不可达保证及时清理。引用计数也有循环引用的限制。选择时应考虑延迟、吞吐、资源类型和运行环境；C++ 可借助库接入 GC，但 RAII 对生命周期明确的资源仍非常有用。

代码示例

```
#include <fstream>
#include <fstream>
void process()
{
    std::ifstream file("input.txt"); // RAII 在作用域退出时关闭资源
    if (!file)
        return;
    use_file(file);
} // 即使异常栈展开，文件仍确定关闭
// 垃圾回收器管理可达内存不等于及时释放文件、锁或事务
// 追踪 GC 可能有暂停；资源释放时机应按延迟和资源类型设计
```

005 new/delete 与 malloc/free 有什么区别?

答案　new 表达式在取得存储后还会调用对象的构造函数；delete 表达式先调用析构函数，再释放存储。malloc 和 free 只申请与释放字节，不会自动构造或析构 C++ 对象。因此两组接口不能交叉配对，数组形式 new[] 也必须用 delete[]，否则对象生命周期和分配器调用都会不匹配。

业务代码通常无需直接处理原始分配：使用容器保存多项数据，用 make_unique 管理独占对象，可以让异常和提前返回时自动清理。原始存储操作主要出现在分配器、对象池等底层实现中；即便如此，也要分别处理对齐、对象构造和析构。如果必须用 malloc 得到存储，不能把它简单强转后就认为任意对象已经构造完成。

代码示例

```
#include <cstdlib>
Widget* p = new Widget(args...); // 分配存储并调用构造函数
delete p; // 调用析构后释放；必须成对使用 new/delete
void* raw = std::malloc(sizeof(Widget)); // 只取得字节，不构造 Widget
std::free(raw); // malloc/free 配对；不能 delete raw
// 数组 new[] 配 delete[]；业务代码优先容器/智能指针而非原始分配
```

006 什么是 RAII（资源获取即初始化）?

答案　RAII 是把资源的所有权绑定到对象生命周期上的设计方法：成功构造后，对象负责持有资源；对象析构时自动释放它。资源可以是动态内存，也可以是文件、互斥锁、套接字等。由于析构也会在异常栈展开和提前返回时运行，RAII 能统一各种退出路径的清理工作。

例如 lock_guard 在构造时加锁，在析构时解锁；文件流在析构时关闭。对象构造过程若失败，已经成功构造的成员会自动析构，因此要让每个资源在自身 RAII 成员中被管理，不要先把未完成的所有权交给外部。析构函数通常应避免抛异常，清理失败应使用错误状态或其他适当方式报告。

代码示例

```
#include <fstream>
#include <mutex>
void write_log(std::mutex& m)
{
    std::lock_guard<std::mutex> lock(m); // 获锁即建立资源所有权
    std::ofstream file("log.txt");
    file << "entry\n";
} // 异常或提前返回也会逆序析构：先关文件，再解锁
// 这种 RAII 清理比每条路径手写 unlock/close 更可靠
```

2.2 分配策略与内存优化

007 什么是内存池（Memory Pool）?

答案　内存池预先准备可重复使用的存储块，按固定尺寸或几个大小类别分配对象，减少频繁调用通用分配器的开销。它适合对象数量多、尺寸相近、生命周期可预测的场景。变长对象或生命周期复杂的分配模式未必适合用一个简单池解决，池本身也要决定如何增长、耗尽时如何失败以及何时归还存储。

分配存储和构造对象是两个步骤：返回一块字节区域，并不代表其中对象已构造；释放池块前也要先结束对应对象生命周期。实现还需满足类型对齐，并定义并发访问规则。标准库的 pmr 池资源可作为工具，但无同步池不能并发共享。内存池会引入保留内存和管理成本，应先用真实的分配大小与生命周期分布做基准。

代码示例

```
#include <vector>
#include <memory_resource>
std::pmr::unsynchronized_pool_resource pool; // 复用大小相近的分配块
{
    std::pmr::vector<Node> nodes{&pool}; // 元素仍需正常构造/析构
    nodes.emplace_back();
    nodes.emplace_back();
} // 容器先销毁元素，再由 pool 回收存储
// pool 耗尽、对齐与线程安全策略都要定义；并先用真实负载测速度
```

008 如何减少 C++ 程序中的内存碎片?

答案　想减少碎片，先确认问题究竟是分配器管理的外部碎片、每块内部未用空间，还是程序确实需要的峰值内存太大。可以用内存分析器、分配统计和真实负载测量对象大小、存活时间及峰值，再选择方案。尺寸稳定的对象适合大小类或对象池；生命周期相近的临时对象可用 arena 集中分配和批量回收。

池化和 arena 会让管理更简单或更快，但通常会预留内存，可能提高而非降低常驻占用；arena 中对象仍须在释放原始区块前析构。并发分配还要考虑锁竞争和线程局部池。单凭每次分配时间变快，不能证明整体更省内存；要在代表性工作负载下比较峰值、碎片、RSS 和执行时间，再决定是否值得引入。

代码示例

```
#include <vector>
#include <memory_resource>
std::pmr::monotonic_buffer_resource arena; // 一批临时对象统一分配
{
    std::pmr::vector<Node> frame{&arena};
    build_frame(frame);
    render(frame);
} // 容器先结束元素生命周期；arena 再一次性回收区块
// 同生命周期对象适合 arena；池化会多保留内存，不一定减少总占用
// 用 profiler 区分碎片、峰值占用和分配开销后再选大小类/池
```

009 如何通过 std::move 实现高效的内存管理?

答案　std::move 的作用是把一个表达式转换成可被当作右值使用的形式，本身不移动数据，也不自动调用移动构造函数。之后目标类型是否执行移动，取决于是否存在可用且匹配的移动操作；对于 unique_ptr 等资源类，移动通常把资源句柄转给新对象，避免深复制。

移动后的源对象仍须满足该类型规定的有效状态，并能够安全析构，但它具体保留什么值通常未指定。常见智能指针移动后为空，其他类型不能一概而论；需要继续使用时，应只执行类型保证安全的操作或重新赋值。不要对仍需保留值的对象随意 std::move，也不要认为给 const 对象 std::move 一定能触发移动。

代码示例

```cpp
#include <memory>
#include <utility>
auto source = std::make_unique<Buffer>(size);
auto destination = std::move(source); // 这里只转换值类别，不搬运字节
// unique_ptr 的移动构造接手句柄；source 现在为空
if (!source)
    source = std::make_unique<Buffer>(new_size); // 可重新赋值
// 普通移动后对象通常只保证有效可析构，具体值未指定
```

010 如何避免 C++ 中的内存泄漏？

答案　避免泄漏最直接的办法，是让每份资源从创建时就有明确所有者。局部变量、标准容器和 RAII 包装会在正常返回、异常和提前返回时自动清理，比在多个分支手工写 delete、close 或 unlock 可靠。unique_ptr 表达独占所有权；shared_ptr 只在确实有多个共同所有者时使用，并留意循环引用。

借用资源的一方不能负责释放所有者管理的对象；返回裸指针时也要明确它是借用还是转移所有权。AddressSanitizer、LeakSanitizer 等工具能找到已执行路径中的泄漏、越界或释放后访问，但不能证明所有路径都没有问题。测试应覆盖错误返回和异常分支，所有者边界设计清楚比事后依赖扫描工具更重要。

代码示例

```cpp
#include <memory>
std::unique_ptr<Widget> make_widget()
{
    return std::make_unique<Widget>();
}
void run()
{
    auto object = make_widget();
    if (condition())
        return; // 提前返回也自动释放
    process(*object);
} // 异常展开也清理；共享所有权仅在确需时用 shared_ptr
// weak_ptr 可打破所有权环；测试构建可启用 Address/LeakSanitizer
```

011 什么是 placement new？它和普通 new 有什么区别？

答案　placement new 在调用方提供的地址上构造对象：它执行构造并开始对象生命周期，但不负责申请底层存储。普通 new 表达式通常同时取得存储并构造对象。既然 placement new 的存储来自调用方，就不能直接对这个对象使用普通 delete 释放；应先显式结束对象生命周期，再按原本取得缓冲区的方式处理存储。〔S2〕

调用前要确认缓冲区大小足够、起始地址满足对象对齐，而且存储活得足够久。析构非平凡对象时应正确调用析构函数或合适的标准工具，再复用缓冲区。对象池和底层容器可能需要这种机制，但一般业务代码可以通过容器或智能指针管理对象，避免自己承担原始存储、构造、析构三套规则。

代码示例

```cpp
#include <cstddef>
#include <new>
#include <memory>
alignas(T) std::byte storage[sizeof(T)]; // 足够大且满足 T 对齐
T* object = ::new (static_cast<void*>(storage)) T(args...); // 只开始生命周期
use(*object);
std::destroy_at(object); // 显式析构；placement new 不负责释放存储
// 仅在原始缓冲区生命周期结束/释放策略执行后回收 storage
```

2.3 原始存储、分配失败与字节操作

012 未初始化指针和悬空指针分别怎样产生？置空能解决全部问题吗？

答案　未初始化的自动指针没有可供程序正常使用的确定值，俗称野指针；在赋值前读取、解引用或把它当作有效地址都不安全。悬空指针则曾指向活对象，但对象已经销毁、存储已经释放，或容器扩容使原地址失效。两者来源不同，都不能通过“看地址像不像有效值”来判断。

把某个变量置为 nullptr 只清除了这一个指针副本，其他指针和引用不会自动更新。应使用智能指针表达所有者，用普通指针或引用表达短期借用，并避免让借用跨越所有者销毁、容器搬迁或异步任务边界。AddressSanitizer 可帮助发现执行到的释放后使用路径，但需要结合代码中的寿命关系审查。〔N12〕

代码示例

```cpp
int* observer = nullptr;
{
    auto owner = std::make_unique<int>(42);
    observer = owner.get(); // 观察者不延长对象生命
```

```
} // owner 释放资源，observer 悬空
observer = nullptr; // 仅重置此变量；其他别名仍须单独处理
```

013 sizeof、strlen 与字符串拷贝各自需要哪些边界条件？

答案　sizeof 查询类型或对象表示的字节数：保留数组类型时得到整个数组容量，数组作为函数参数时会退化为指针，得到的则只是指针大小。strlen 从给定字符地址开始，向后扫描到第一个空字符并返回其前面的字符数，所以不包含结尾的零字节；调用者必须确保扫描期间的内存都可读。

strcpy 会复制字符串内容和结尾空字符，但没有目标缓冲区容量参数；目标空间不足或源目标区域重叠时会出错。strncpy 也不能简单视为安全替代，因为目标可能没有结尾空字符。普通文本优先用 std::string；固定缓冲区要显式验证容量并确保终止符，二进制内容应保存长度，不能用 strlen 判断。〔N6、N13、N14〕

代码示例

```
char buffer[16] = "abc";
static_assert(sizeof(buffer) == 16); // 容量，与有效文本长度不同
std::size_t length = std::strlen(buffer); // 3，不含结尾零
std::string safe_copy = buffer; // 所有权由字符串对象管理
```

014 memcpy 和 memcmp 为何不能普遍替代对象复制与相等比较？

答案　memcpy 按指定字节数复制对象表示，源和目标内存范围必须有效且不能重叠；范围重叠应改用 memmove。对满足平凡可复制条件的类型，在符合存储和生命周期规则时，字节复制可以恢复值；但含 string、虚函数或资源所有权的任意类不能靠复制字节完成正常拷贝。对象应调用它定义的拷贝或移动操作。

memcmp 比较的是每个字节，不是对象的逻辑值。结构体的填充字节可能不一致，即使成员值相同也会比较出不同；浮点正负零、指针和用户定义相等语义也不适合由原始字节推断。用成员比较、默认比较运算符或专门的业务规则表达值相等；跨平台文件和网络数据则应显式序列化字段。〔N15、N16〕

代码示例

```
struct Point
{
    int x;
    int y;
    bool operator==(const Point&) const = default;
};
Point a{1, 2}, b{1, 2};
bool equal = (a == b); // C++20，表达字段的值相等
// 不用 std::memcmp(&a, &b, sizeof(Point)) 推导一般对象相等
```

015 普通 new、nothrow new 与 malloc 分配失败时分别如何报告？

答案　普通 new 在标准分配路径无法取得存储时会抛出 std::bad_alloc，并可能按规则调用已设置的 new_handler；因此在普通 new 后检查空指针并不能替代异常处理。nothrow new 在分配失败时返回空指针，但对象构造函数自身仍可能抛异常。malloc 则失败时返回空指针，不调用构造函数，错误状态还要按平台约定处理。

分配成功得到非空地址也不代表程序未来触碰所有内存页都必然成功；操作系统可能采用过量承诺。零长度申请、数组长度溢出和极大请求也需要在接口层处理。调用方应根据业务需求选择恢复、减少工作或终止，并在处理失败时避免再进行可能需要大量内存的操作。不要把分配接口的错误方式混为一谈。〔N17、N18〕

代码示例

```
try
{
    auto buffer = std::make_unique<std::byte[]>(requested);
    use(buffer.get());
}
catch (const std::bad_alloc&)
{
    reportAllocationFailure(); // 真实业务中避免继续依赖新的大分配
}
```

016 glibc malloc 如何在空闲块、brk 与 mmap 之间分配？

答案　glibc 的 malloc 是用户态分配器接口，并不是每次申请都向内核要一块独立内存。它通常先从线程缓存、arena 或已释放的空闲块里寻找合适区域；需要扩展时，底层可以通过 brk 扩展传统堆区，也可以通过匿名 mmap 建立映射。较大请求更可能单独映射，但阈值可配置并会调整，不能用固定尺寸划出永久分界。

free 之后分配器可能保留存储供后续复用，因此 RSS 不立刻下降并不直接证明泄漏。建立映射、提交虚拟地址和操作系统实际提供物理页也不是同一时刻。分配器还为对齐与管理维护元数据，请求字节数不等于实际占用。诊断要结合 glibc 版本、线程和 arena 数量、分配模式与进程指标，避免把某次实验观察到的块头或阈值推广到所有环境。〔N18〕

代码示例

```
void* memory = std::malloc(256); // malloc 可能复用缓存块，也可能向系统申请内存
if (memory != nullptr)
```

```
{
    use(memory);
    std::free(memory);
}
// free 不保证 RSS 立即下降；分配器可能保留页面供后续复用
```

第 3 章 现代 C++ 与模板

3.1 语言特性与类型系统

001 C++ 中如何实现自定义的内存管理器？

答案　自定义内存管理器首先要把原始存储和对象本身分开：分配器取得足够大小、满足对齐的字节区域，容器再在其中构造元素，使用完还要先析构元素再归还存储。固定尺寸且频繁创建的对象可以用预分配池和空闲链表，按需复用固定大小的块。

若实现通用标准库 allocator，还要满足分配器类型的传播、相等性等语义，否则容器移动或交换时可能错误释放存储。标准 pmr 内存资源提供可复用的池等选择，但要按同步要求使用。全局重载 new/delete 会影响整个程序、第三方库和运行时，应先用性能分析证明默认分配路径是瓶颈，再选择局部 allocator 或资源。

代码示例

```
#include <vector>
#include <memory_resource>
std::pmr::unsynchronized_pool_resource pool; // 面向相近尺寸的复用池
{
    std::pmr::vector<Node> nodes{&pool}; // 容器在原始存储中构造元素
    for (int i = 0; i < 100; ++i)
        nodes.emplace_back();
} // 先析构元素，再让资源池回收存储
// 通用 allocator 还需满足传播/相等性等要求；unsynchronized_pool_resource 不可并发共享
// 自定义全局 new 影响全程序；先证明分配器是瓶颈
```

002 C++ 中的 auto 关键字是什么？

答案　auto 根据初始化表达式推导类型。普通的 auto 值推导类似模板按值推导：顶层 const 和引用通常会被去掉；auto& 会保留引用与底层 const，const auto 则让新变量成为只读值或只读引用。auto&& 在合适的模板推导上下文中可成为转发引用，需结合左值、右值规则理解。

例如从 vector 取出 string 元素时，auto 会复制一份，auto& 会绑定原元素，const auto& 可避免复制并阻止修改。decltype(auto) 使用 decltype 的规则，能保留精确引用和值类别，因此特别要检查返回值是否引用了短命对象。某些容器如 vector<bool> 返回代理对象，不一定是真正的 bool 引用；写通用遍历代码时要考虑这些特殊类型。

代码示例

```
#include <string>
#include <vector>
std::vector<std::string> names{"Ada", "Lin"};
auto copy = names[0]; // 值推导去掉引用，复制字符串
const auto& view = names[1]; // 保留只读引用，不复制元素
auto&& forwarding = names[0]; // 在适当模板/推导语境中是转发引用
// decltype(auto) 按 decltype 规则保留精确引用和值类别
// vector<bool> 的元素访问可能是代理对象，auto& 不一定适用
```

003 C++ 中的 noexcept 关键字如何使用？

答案　函数声明为 noexcept，表示不应有异常逃出函数边界；如果异常真的向外传播，程序会调用 std::terminate，而不是自动转成错误返回。noexcept(expr) 可在编译期询问表达式是否不抛，模板可以据此选择实现或设置条件异常规范。它描述的是异常承诺，不是告诉编译器“尽量优化”的装饰。

移动构造或移动赋值只有确实不会抛异常时才应标 noexcept。像 vector 扩容这样的操作可能依赖这个承诺，以决定能否安全移动元素；若移动会抛，容器可能改用复制来保留异常保证。错误地标记 noexcept 会把可恢复异常变成终止程序。实现应根据成员操作的真实异常行为声明，并用类型特征验证需要的性质。

代码示例

```
#include <type_traits>
struct MoveOnly
{
    MoveOnly(MoveOnly&&) noexcept = default;
};
static_assert(std::is_nothrow_move_constructible_v<MoveOnly>);
void never_fails() noexcept
{ /* 异常若逃逸会调用 std::terminate */
```

```
}
// vector 扩容时可能为强异常保证回退到拷贝
// 移动操作只有确实不抛才标 noexcept，不能作为优化装饰
// noexcept(expr) 还可在模板中查询表达式的异常属性
```

004 C++ 中的 typeid 是如何工作的?

答案　typeid 返回 std::type_info，可比较类型信息；它并不提供跨编译器稳定的类型名称。对普通表达式通常依据静态类型判断；若操作数是多态类的 glvalue，才会求出对象的动态类型。获取 name() 的字符串格式由实现决定，不适合作为持久化 ID 或跨进程协议字段。

使用 RTTI 的动态信息通常要求编译时启用 RTTI。对空指针解引用后再对多态对象使用 typeid，会抛出 std::bad_typeid；先检查指针更清晰。还应注意运算数求值：当动态类型检查需要运行时对象时，相关表达式可能被求值。若目的只是执行不同操作，虚函数或访问者模式通常比到处比较 typeid 更容易扩展。

代码示例

```
#include <typeinfo>
struct Base
{
    virtual ~Base() = default;
};
struct Derived : Base
{
};
Base* p = new Derived;
bool same = (typeid(*p) == typeid(Derived)); // 多态 glvalue 得到动态类型
const char* name = typeid(*p).name(); // 名称格式由实现决定
// 非多态表达式通常只报告静态类型；RTTI 需启用
// 不要解引用空多态指针再 typeid；那会抛 std::bad_typeid
delete p;
```

005 C++ 中的模板元编程（Template Metaprogramming）是什么?

答案　模板元编程利用模板实例化、类型特征和编译期常量表达式，在编译阶段计算结果或为不同类型选择实现。比如按类型是否为整数启用一个函数，可以让不满足条件的调用在编译期失败，而不是等程序运行再分支。它是一种静态计算方式，计算结果参与类型和代码生成。

复杂递归模板可能让编译变慢、错误信息难读，还可能生成过多代码。对简单数值计算，constexpr 函数常比递归模板更直观；公开接口的类型要求可用 concept 表达。模板技巧适合确有编译期选择需求时使用，不必把所有普通逻辑都移到类型系统里。还要留意编译期数值范围，避免计算本身发生溢出或递归深度过大。

代码示例

```
#include <cstddef>
template <std::size_t N> struct Factorial
{
    static constexpr std::size_t value = N * Factorial<N - 1>::value;
};
template <> struct Factorial<0>
{
    static constexpr std::size_t value = 1;
};
static_assert(Factorial<5>::value == 120); // 编译期实例化递归模板
constexpr std::size_t fact(std::size_t n)
{
    return n < 2 ? 1 : n * fact(n - 1);
}
// 常量表达式通常更清楚；深层 TMP 会增加编译时间和诊断难度
```

006 C++ 中的类型推导（Type Deduction）是什么?

答案　C++ 的类型推导没有一套脱离语境的统一规则。auto 值声明会像模板按值推导一样去掉顶层 const 与引用；auto& 保留引用；函数模板的 T、T& 和 T&& 参数也分别遵循按值、左值引用与转发引用规则。转发引用能根据实参是左值还是右值推导，并通过引用折叠保留类别。

decltype(auto) 则按 decltype 规则推导，可能得到引用类型。它常用于精确转发或保留表达式类型，但也容易意外返回局部变量引用。写返回 decltype(auto) 的函数时，要确认返回表达式对应的对象寿命长于调用方使用时间。理解声明语境和表达式值类别，比只看变量名称判断类型可靠。

代码示例

```
#include <utility>
int x = 1;
```

```
const int cx = 2;
auto a = cx; // 按值推导为 int，顶层 const 被去掉
const auto& b = cx; // 保留 const 引用
template <class T> decltype(auto) identity(T&& value)
{
    return std::forward<T>(value);
}
// 模板 T&& 对左值折叠为左值引用，对右值保持右值引用
// 返回引用时要确认对象活得足够久，避免悬空
```

007 C++11 中的 Lambda 表达式是什么?

答案　lambda 表达式会生成一个匿名闭包类型，其中有调用运算符以及捕获的状态。按值捕获会在闭包中保存副本，按引用捕获则仍依赖原变量活着；闭包若被放进任务队列或异步运行，不能引用捕获很快离开作用域的局部变量。C++14 的初始化捕获可在创建闭包时移动资源，例如把 unique_ptr 从调用方转移到任务中。闭包默认调用运算符不能修改按值捕获的副本，需要时可加 mutable。闭包的类型由编译器生成；非捕获 lambda 可转换为相容函数指针，带捕获的 lambda 则通常不能。长期保存任务时，捕获方式也决定任务是否拥有其依赖资源。

代码示例

```
#include <memory>
#include <utility>
#include <memory>
#include <utility>
auto make_task(std::unique_ptr<Job> job)
{
    return [owned = std::move(job)]
    {
        owned->run();
    }; // 初始化捕获转移所有权
}
int total = 0;
auto add = [&total](int n)
{
    total += n;
}; // 依赖 total 生命周期
add(3);
// 闭包保存捕获状态；异步保存时避免引用短生命周期局部变量
// 按值捕获可保存副本，必要时用 mutable 改闭包自己的副本
```

008 C++11 引入的 nullptr 有什么用法?

答案　nullptr 是专门表示空指针的字面量，类型为 std::nullptr_t，可转换成对象指针或函数指针。它与整数 0 不同，能在存在 int 与 char* 等重载时表达“这里要选指针语义”，避免历史上的 NULL 宏被定义为整数而选错重载。普通代码中应优先用 nullptr 表示空指针。

把 nullptr 赋给具体指针类型会得到空指针值，也可在模板或函数参数中传递；需要显式命名它的类型时包含 <cstddef>。nullptr 不表示“没有值的任意对象”，不能直接用于任意整数参数。选择正确的重载仍应让接口设计清楚，不能依赖复杂的隐式转换绕过类型含义。

代码示例

```
#include <cstddef>
void open(int);
void open(const char*);
open(nullptr); // 空指针语义，选择指针重载
open(0); // 整数 0 匹配 int，历史 NULL 宏也可能是整数
std::nullptr_t empty = nullptr;
// nullptr 可转换为对象/函数指针，不会误选整数重载
// 需要头文件 <cstddef> 才直接命名 std::nullptr_t
```

009 inline 函数与宏有什么区别?

答案　宏在预处理阶段执行文本替换，不理解 C++ 类型和作用域。宏参数如果在替换文本中出现多次，调用表达式可能被求值多次；有副作用的实参因此会造成重复操作，甚至未定义行为。inline 函数则像普通函数一样经过类型检查、只按参数规则求值，并受命名空间和访问控制约束。

inline 并不保证编译器展开机器码调用；其语言层面还允许满足 ODR 的定义出现在多个翻译单元。适合普通计算的宏通常可换成函数或模板。宏仍有条件编译、字符串化和 token 拼接等预处理用途，但应避免将其当作没有类型的函数机制。宏命名也要避免污染包含它的所有源码。

代码示例

```
inline int square(int x)
{
    return x * x;
} // 类型检查，参数只求值一次
int i = 3;
int value = square(i++); // i 只递增一次
#define SQUARE_BAD(x) ((x) * (x))
int j = 3;
// int wrong = SQUARE_BAD(j++); // 会重复递增且产生未定义行为
// inline 不保证生成机器码内联，但可满足多翻译单元 ODR 例外
// 宏仍适合条件编译；函数更安全、可调试、受作用域控制
```

3.2 值类别、移动与泛型

010 右值引用、移动语义、std::move 与 std::forward 的关系是什么？

答案　右值引用让类型能区分可复用对象与可转移资源的对象，并据此实现移动构造和移动赋值。std::move 只是把表达式转换成右值类别，之后是否移动由重载决议和目标类型决定；若目标没有合适的移动操作，仍可能复制。移动操作通常把句柄或缓冲区转给新对象，而非复制整份内容。

std::forward<T> 则只用于转发引用模板，按照推导得到的 T 保留传入实参是左值还是右值。普通函数形参即使写成 T&&，在函数体中命名后的变量也是左值表达式，因此需要 forward 转发原类别。移动后的源对象仍必须有效且可析构，但值可能未指定；只进行类型保证的操作或重新赋值。

代码示例

```
#include <string>
#include <utility>
void consume(std::string&& text)
{
    store(std::move(text));
}
template <class F, class... Args> decltype(auto) forward_call(F&& f, Args&&... args)
{
    return std::forward<F>(f)(std::forward<Args>(args)...);
}
// std::move 只转换值类别，移动由目标类型的重载实现
// std::forward<T> 仅在转发引用模板里按 T 保留实参原值类别
```

011 如何使用 C++ 实现延迟计算（Lazy Evaluation）？

答案　延迟计算是把工作先保存起来，等结果第一次被需要时再执行。例如调用方只在某个分支才读取昂贵计算结果，就可以用 lambda 或函数对象包装计算，避免用不到时提前消耗时间。若结果可能被多次访问，可用 optional 表示“尚未计算或已有结果”，计算完成后缓存起来。

多个线程同时首次请求时，要保证只进行规定次数的计算；std::call_once 可协调单次初始化，成功后后续调用不会重复执行。若初始化抛出异常，之后的调用可能再次尝试，因此还要决定重试语义。延迟任务本身也有生命周期约束：按引用捕获的外部对象必须比任务活得更久。缓存会增加状态和同步成本，仅在延迟确有收益时使用。

代码示例

```
#include <mutex>
#include <mutex>
#include <optional>
class LazyValue
{
    mutable std::optional<int> cache_;
    mutable std::once_flag once_;

  public:
    int get() const
    {
        std::call_once(once_,
                       [&]
                       {
                           cache_ = expensive_compute();
                       });
        return *cache_;
    }
};
```

```
// 闭包/函数对象可推迟计算；optional 区分未计算和结果为 0
// call_once 成功初始化后只执行一次；异常时后续调用可重试
// 引用捕获对象仍须活得比延迟任务久
```

012 如何实现 C++ 中的类型萃取（Type Traits）?

答案　类型特征是在编译期询问类型属性或变换类型的工具，例如判断是否为整数、去掉引用，或获取容器的元素类型。标准库提供的大量 trait 已经覆盖常见需求，优先复用它们可以减少模板细节和错误。类型特征只在编译期给出结果，不会在对象中存储一个运行时标记。

需要自定义时，可以用主模板提供默认结果，再用偏特化处理某种类型形状；继承 true_type 或 false_type 能得到标准布尔接口。早期代码常用 void_t 检查表达式是否合法，C++20 concept 更适合直接表达公开模板接口的要求。类型特征应封装具体检测意图，避免为了炫技构造让调用者难以理解的递归模板。

代码示例

```
#include <type_traits>
#include <utility>
#include <vector>
template <class T> struct is_pointer_like : std::false_type
{
};
template <class T> struct is_pointer_like<T*> : std::true_type
{
}; // 偏特化
static_assert(is_pointer_like<int*>::value);
using ValueType = std::remove_cvref_t<const int&>; // 先复用标准特征
static_assert(std::is_same_v<ValueType, int>); // C++20 concept 可直接写约束
```

013 C++20 Concepts 与 SFINAE 有什么区别?

答案　Concepts 用 requires 表达模板参数必须支持什么操作或满足什么性质，并参与模板候选的约束排序。调用不满足约束时，诊断通常更接近具体要求。SFINAE 的核心规则是模板替换失败时移除该候选，不把该失败当作整个程序错误；常见写法包括 enable_if 和检测惯用法。

Concepts 让代码更清楚，但不能假定它与所有 SFINAE 重载选择完全等价。约束排序有自己的规则，迁移旧模板时要检查哪些重载成为候选以及歧义是否变化。SFINAE 仍适用于旧标准兼容代码和一些细粒度重载选择；新接口通常可以优先用 concept 把要求写在定义附近。〔S1，S2〕

代码示例

```
#include <type_traits>
#include <concepts>
template <class T>
    requires std::integral<T>
T add(T a, T b)
{
    return a + b;
} // 约束直接说明所需性质
// 旧标准常用替换失败从候选集移除：
template <class T, std::enable_if_t<std::is_integral_v<T>, int> = 0> T legacy_add(T a, T b)
{
    return a + b;
}
// concepts 错误信息通常更清楚，但重载排序规则不完全等同 SFINAE
```

014 C++20 Ranges 和 View 解决了什么问题?

答案　Ranges 让算法可以接收一个范围对象，不必每次把 begin 和 end 两个迭代器分开传递。view 则允许把过滤、映射等操作串起来，并在遍历时才产生结果，因此很多处理流程不需要先创建一连串临时容器。例如可以先过滤奇数，再将每项乘二，最终只在循环取值时执行这些步骤。

惰性计算也意味着访问时可能重复执行过滤或变换函数，不能假设结果已缓存。许多 view 只保存底层数据的借用关系，并不拥有元素；如果容器销毁、扩容或迭代器失效，保存的 view 也可能不可用。返回或长期保存 view 时，必须保证底层范围寿命和迭代器有效期覆盖所有使用。〔S1，S2〕

代码示例

```
#include <ranges>
#include <vector>
std::vector<int> values{1, 2, 3, 4};
auto odds = values | std::views::filter(
                        [](int n)
                        {
```

```
                            return n % 2;
                        });
auto doubled = odds | std::views::transform(
                        [](int n)
                        {
                            return n * 2;
                        });
for (int n : doubled)
    consume(n); // 迭代时才过滤/变换，无中间容器
// view 多数不拥有元素；保存它前要保证 values 和迭代器仍有效
```

015 std::span 是什么？它和 std::vector 有何区别？

答案　std::span 是对连续元素区间的非拥有视图，通常保存起始地址与元素个数。它让函数可以统一接收数组、vector 等连续存储，而不必复制元素或只针对某个容器类型编写重载。span 只是借用数据，不负责扩容、释放，也不会把底层对象的寿命延长。

vector 与 span 的职责不同：vector 拥有存储并管理容量变化；span 只描述一段现存区间。vector 销毁或导致存储重新分配时，原来的 span 可能变成悬空视图。接受 span 的函数不应保存它超过调用者数据寿命，除非另有所有权设计。需要可调整大小并拥有数据时，继续用 vector。〔S2〕

代码示例

```
#include <span>
#include <vector>
int sum(std::span<const int> values)
{
    int total = 0;
    for (int x : values)
        total += x;
    return total;
}
std::vector<int> owned{1, 2, 3};
int result = sum(owned); // span 借用 vector 的连续存储，不复制元素
// vector 拥有并可扩容；span 只保存地址和长度，不延长生命周期
// owned 被销毁或扩容后，不得继续用原 span
```

016 std::string_view 有什么优点和生命周期陷阱？

答案　std::string_view 是带长度的只读字符视图，不拥有字符存储。把它作为解析函数参数，可以避免为了读取一段文本而复制或构造临时 string。它描述的是一段字符范围，不一定以空字符结尾；data() 只是起始地址，不能无条件传给只接受 C 字符串的接口。

视图的有效期受原始存储控制：原 string 销毁、扩容重分配、某些修改使数据位置失效，或临时字符串表达式结束后，view 都可能悬空。不要从函数返回指向局部 string 的 string_view，也不要把异步任务里的视图建立在调用者短期数据上。需要拥有内容时复制到 string，调用 C 接口前也要确保终止符和长度符合要求。〔S2〕

代码示例

```
#include <string>
#include <string_view>
#include <string_view>
int parse(std::string_view text)
{
    return parse_digits(text);
} // 无需复制字符串
std::string owner = "123";
std::string_view view = owner;
int n = parse(view); // 仅在 owner 存活且存储未失效时安全
std::string_view dangling()
{
    std::string local = "bad";
    return local;
} // 错误：悬空
// view 不保证 data() 后有 NUL；传 C API 前要复制/保证终止
```

3.3 Modules、协程与编译期计算

017 C++20 Modules 和传统头文件包含有什么区别?

答案　传统头文件是在预处理阶段把文本复制到每个翻译单元。这样宏和声明会跟着 include 进入包含者的上下文，重复解析也会增加构建工作。C++20 模块通过 export 暴露接口、通过 import 引入接口，减少文本式包含带来的宏泄漏和重复解析问题。

模块接口需要先编译为编译器可读的模块工件，常叫 BMI；构建系统必须识别模块依赖并按顺序生成。模块可以与头文件逐步共存，但编译器、标准库及构建工具支持程度会影响可用性和实际构建收益。迁移前应在项目工具链上验证依赖扫描、缓存与第三方库集成，而不是假定所有编译环境都支持同一种方式。〔S1，S2〕

代码示例

```
// math.cppm: 模块接口单元
export module math;
export int add(int a, int b)
{
    return a + b;
}
// client.cpp: import math; 然后调用 add(1, 2)
// import 提供显式接口且宏不会像文本 include 那样泄漏
// BMI 需先构建；构建系统必须安排依赖，工具链支持仍需确认
```

018 C++20 协程的基本机制是什么？协程会自动创建线程吗?

答案　协程函数包含 co_await、co_yield 或 co_return 等关键字后，编译器会按协程规则把它转换为可以暂停和恢复的状态机。promise_type 定义协程的返回对象、初始和最终挂起行为以及异常处理；awaiter 则参与决定当前操作是否挂起以及将来如何恢复。具体接口依赖协程返回类型和相关库设计。

协程只提供保存状态与挂起恢复的语言机制，不会自动创建线程，也不自动执行网络 I/O 或把任务放进调度队列。调用方或异步库必须负责何时恢复、在哪个执行器运行以及如何管理取消和资源。协程帧的分配与生命周期也要由返回对象和调用约定管理；不能因为函数表面上写成顺序逻辑就忽略异步错误处理。〔S1，S2〕

代码示例

```
#include <coroutine>
struct Task
{
    struct promise_type;
    std::coroutine_handle<promise_type> handle;
};
Task fetch()
{
    co_await wait_for_io();
    co_return;
} // 关键字让编译器生成可暂停状态机
// promise_type 定义返回对象、异常处理和初始/最终挂起行为
// awaiter 决定是否挂起及恢复方式；调度器/调用方负责何时 resume
// 协程本身不创建线程，也不自动并发或执行异步 I/O
```

019 std::expected 解决什么问题？它从哪个标准版本开始提供?

答案　std::expected<T, E> 用一个返回值表达两种结果：成功时保存 T，失败时保存 E。调用者可以检查 has_value 或直接判断对象，再读取成功值或错误原因。它比只用 optional 多携带错误信息，适合可预期且调用方有办法处理的失败；类型标明失败可能性，但不替调用者自动记录日志或恢复。

expected 从 C++23 标准库开始提供，使用时还需确认编译器和标准库实现支持。它与异常的适用边界要按项目接口约定：错误需要逐层显式处理时，expected 清楚地把错误放在返回值里；异常仍适合某些无法局部处理或跨多层传播的失败。读取 error 前要确认对象处于失败状态，调用方也不能默默丢弃错误。〔S2〕

代码示例

```
#include <expected>
#include <string>
#include <string_view>
std::expected<int, std::string> parse(std::string_view text)
{ // C++23
    if (text.empty())
        return std::unexpected(std::string("empty input"));
    return parse_integer(text);
}
auto result = parse("42");
if (result)
```

```
    use(*result);
else
    log(result.error()); // 调用方显式处理错误
// expected 携带可恢复错误；不会自动日志，也不取代所有异常
```

020 constexpr、consteval 和 constinit 有什么区别?

答案　constexpr 表示函数或变量可参与常量表达式，但满足条件时也能在运行时使用；实际能否常量求值还取决于调用参数和表达式是否符合规则。consteval 函数是立即函数，调用必须在要求的常量求值场景中完成。二者都涉及编译期求值，但强制程度不同。

constinit 是 C++20 的存储期变量声明约束，要求静态或线程存储期变量经过常量初始化，避免依赖动态初始化顺序。它不会让变量自动成为 const，程序运行后仍可修改。要选择哪一个，应先问需要“允许编译期计算”“必须立即计算”，还是“保证启动时已静态初始化”；三者不能互相替代。〔S2〕

代码示例

```
constexpr int twice(int x)
{
    return x * 2;
} // 可编译期或运行期求值
consteval int fixed(int x)
{
    return x * 2;
} // 调用必须立即常量求值
constinit int startup = twice(4); // C++20 静态初始化保证，但变量可修改
int runtime = twice(read_input()); // constexpr 函数也可运行期调用
// constinit 不能替代 const；三者约束的是不同事情
// constinit 保证静态/线程存储期变量常量初始化，但不让对象自动 const
```

3.4 特殊成员函数控制

021 显式 = default 和 = delete 怎样控制类的特殊成员函数?

答案　= default 表示请求编译器按语言规则生成特殊成员函数定义，但最终操作仍可能被删除；例如成员自身不可复制时，类默认化的拷贝操作也不能工作。= delete 则明确禁止某个函数被调用，而且该函数仍参与重载决议，适合拒绝特定类型或明确表示类不可复制。

声明一个析构、拷贝或移动成员可能改变其他特殊成员的隐式生成规则，所以要从整体上检查复制、移动和销毁语义。自管资源的类型需要考虑三法则或五法则；更好的常见选择是让 RAII 成员承担所有权，遵循零法则。default 不是“保证一定有效”，delete 也不只是把函数藏在 private 后面。〔N19、N20〕

代码示例

```
struct Handle
{
    Handle() = default;
    Handle(const Handle&) = delete;
    Handle& operator=(const Handle&) = delete;
    Handle(Handle&&) = default; // 仍须确认成员的移动语义正确
    Handle& operator=(Handle&&) = default;
};
```

3.5 C++11 语言与模板补充

022 C++11 的范围 for、enum class 和变长参数模板有何特点?

答案　范围 for 让代码直接遍历范围中的元素，少写 begin/end；它仍受迭代器规则约束，遍历中改变容器可能让当前迭代器失效。若元素较大，const auto& 可避免复制并表达只读访问。enum class 让枚举值有自己的作用域和类型，不会自动转成整数；需要底层值时要显式转换，也可以指定底层整数类型。

变长参数模板用参数包表达零个或多个类型或函数实参，实例化时可以把同一种操作展开到每个参数上。C++11 常见写法用递归重载逐项处理，C++17 才加入折叠表达式来简化一类聚合展开。写范围循环仍需考虑容器失效，枚举转换仍需考虑越界与协议编码；这几项是语言工具，不会自动提供这些额外保证。〔N21、N22、N23〕

代码示例

```
enum class State : unsigned
{
    idle,
    busy
}; // enum class 不会隐式转成 int
for (const auto& item : items)
    use(item); // 范围 for 可避免不必要的复制
```

```
template <class... Args> void print_all(const Args&... args)
{
    (print(args), ...);
} // C++17 折叠表达式
```

第 4 章 并发与线程安全

4.1 同步原语与线程管理

001 C++ 中如何使用 std::condition_variable 进行线程同步？

答案　条件变量本身不保存通知；它只是让线程在条件暂时不满足时休眠。真正的状态必须放在受互斥锁保护的谓词里，例如 ready 标志或非空任务队列。等待者先持有 unique_lock，然后调用 wait(lock, pred)：等待期间原子地释放锁，被唤醒后重新加锁并再次检查谓词。

这种循环检查可以处理虚假唤醒，也不依赖通知恰好在某个时间到达。生产者应先持锁更新状态或队列，释放锁后再通知；状态才是事实来源，通知只是提醒重新检查。若工作线程通过引用访问外部对象，线程退出前要 join，保证引用对象仍然存在。通知多次不会积累为多个待处理事件。

代码示例

```
#include <condition_variable>
#include <mutex>
#include <thread>
std::mutex m;
std::condition_variable cv;
bool ready = false;
std::thread worker(
    [&]
    {
        std::unique_lock<std::mutex> lock(m);
        cv.wait(lock,
                [&]
                {
                    return ready;
                });
        consume_data();
    }); // wait 原子解锁并重查谓词
{
    std::lock_guard<std::mutex> lock(m);
    ready = true;
} // 状态才是事实来源
cv.notify_one();
worker.join(); // 线程退出前 join，避免悬空引用
```

002 C++ 中如何实现无锁队列？

答案　无锁队列通常用原子指针或环形缓冲区索引，并以 CAS 比较并更新共享状态。CAS 失败后要重新读取状态，再判断当前操作还能否继续；不能只把算法写成“指针是 atomic”就认为队列正确。还需定义入队和出队各自何时线性化，以及采用什么内存序让节点内容对其他线程可见。

节点生命周期和 ABA 问题往往比 CAS 本身更难：一个线程读过的节点不能被另一线程提前释放，指针值也可能离开后又回到同一地址。hazard pointer 或 epoch 等机制可以延迟回收。无锁不等于每个操作都在有限步骤内完成，更不保证比互斥锁快。优先使用经过验证的实现，并进行压力与内存回收测试。

代码示例

```
// 伪代码：Michael–Scott 队列；Node 含 atomic<Node*> next 和 value，先建 dummy
std::atomic<Node*> head{dummy}, tail{dummy};
void push(Value v)
{
    Node* n = new Node(std::move(v));
    for (;;)
    {
        Node* t = tail.load(std::memory_order_acquire);
        Node* next = t->next.load(std::memory_order_acquire);
        if (t == tail.load() && !next && t->next.compare_exchange_weak(next, n))
            break;
        if (next)
            tail.compare_exchange_weak(t, next);
```

```
    } // 帮助推进尾指针
}
// 省略 dequeue/内存回收：不能在其他线程可能读取时 delete；还须处理 ABA
```

003 C++ 中如何实现条件同步机制?

答案　条件同步应围绕共享谓词设计，例如“队列非空”或“还有可用名额”。用互斥锁保护谓词及相关数据，用条件变量等待谓词成立。wait(lock, pred) 会循环检查，既处理虚假唤醒，也避免把通知本身错当成状态。若资源数量只增减，可以让计数值成为受锁保护的状态。

通知不会保存事件，也不代表计数自动加一。调用方要记住每次事件时，应在队列或计数器里记录它，再通知等待者；若是独立的计数许可，C++20 semaphore 往往更贴切。无论选哪种机制，都要定义关闭或取消时等待者如何醒来，避免线程永久睡眠。条件变量的正确性来自状态转换和锁协议，而不是通知次数。

代码示例

```
#include <condition_variable>
std::mutex m;
std::condition_variable cv;
int permits = 0;
void wait_for_work()
{
    std::unique_lock<std::mutex> lock(m);
    cv.wait(lock,
            [&]
            {
                return permits > 0;
            });
    --permits;
}
void signal_work()
{
    {
        std::lock_guard<std::mutex> lock(m);
        ++permits;
    }
    cv.notify_one();
}
// wait 用谓词循环处理虚假唤醒；通知本身不保存事件
// 若要保留每次事件，队列/计数状态须记录它；多线程计数可用 semaphore
```

004 C++ 中如何避免死锁?

答案　死锁常见原因是多个线程以相反顺序持有多把锁，彼此等待对方释放。可以规定全局锁顺序，或用 std::scoped_lock 同时获取多个互斥量，缩小持锁区间，并避免持锁时调用可能回调本代码的未知函数。try_lock 和超时锁只有在失败后有明确降级或恢复策略时才有意义，反复重试可能只是掩盖设计问题。

分析挂起时可查看线程栈及锁的持有、等待关系，找是否存在闭合等待环。避免死锁不等于保证公平，也不等于线程不会饥饿；这些是额外性质。哲学家问题可用统一顺序取筷子，或限制同时竞争人数来破坏循环等待。调查应使用受支持的调试工具，不能把某个 libc 私有锁字段当成通用接口。〔I40、I41〕

代码示例

```
#include <mutex>
std::mutex account, inventory;
void transfer()
{
    std::scoped_lock lock(account, inventory); // 一次获取多锁，避免反序死锁
    update_account();
    update_inventory(); // 只在临界区做必要工作
} // 异常或返回时 RAII 自动解锁
// 对外回调/未知代码可能反向获取锁；应避免持锁调用
// 也可规定所有路径按同一全局顺序加锁
```

005 C++ 中的 std::async 如何创建异步任务?

答案　std::async 接收可调用对象及参数，启动异步共享状态并返回 future，调用者可用 get 取得结果。默认策略允许实现选择异步运行或延迟执行：后者可能到 wait/get 才在调用线程执行。因此需要确定并发行为时，应明确指定 launch::async 或使用更可控的任务执行框架。

异步函数中的异常会存进 future，在 get 时重新抛出；get 通常只能消费结果一次。std::async 不是有界线程池，也没有通用的强制取消机制；任务数量和线程资源要由应用管理。未来对象和任务的生命周期也需要清晰：若任务捕获了引用，必须确保外部对象直到任务完成仍然存活。

代码示例

```
#include <future>
auto result = std::async(std::launch::async,
                         []
                         {
                             return compute();
                         });
// 指定 async 避免默认策略可能 deferred 到 get 时才在本线程执行
try
{
    auto value = result.get();
    use(value);
}
catch (const std::exception& e)
{
    report(e.what());
} // 异常经 future 传播
// get 只取结果一次；async 不是有界线程池，也不提供强制取消
```

006 C++ 中的 std::mutex 和 std::lock_guard 是如何工作的?

答案　std::mutex 为临界区提供互斥：一个线程成功 lock 后，其他竞争线程必须等到它 unlock。锁也建立相应的同步关系，使受保护数据能按互斥协议安全访问。互斥量应由成功获取它的线程释放，不能让另一个线程代替解锁。lock_guard 用 RAII 包住一次锁定：构造时获得锁，离开作用域时释放，因此提前返回和异常也会清理。需要提前解锁、重新加锁或采用不同锁定策略时，可使用 unique_lock；同时获取多把锁可用 scoped_lock。锁保护哪些状态应明确一致，不能只锁写操作却让其他线程无锁读取同一数据。

代码示例

```
#include <mutex>
std::mutex m;
int balance = 0;
void deposit(int amount)
{
    std::lock_guard<std::mutex> lock(m); // 构造时锁定
    balance += amount;
    if (should_throw())
        throw Error{}; // 示例：异常也会退出作用域
} // 析构自动 unlock，避免手动 unlock 漏在异常路径
// 多把锁用 std::scoped_lock；只由成功获取锁的线程释放
```

007 C++ 中的原子操作（std::atomic）如何使用?

答案　std::atomic 让对特定对象的读、写或读改写操作不可分割，并可通过 memory_order 建立线程间的可见性和顺序。原子只覆盖它自身：把账户余额和版本号分别声明为 atomic，并不能保证两者作为一个整体同时更新。多个普通字段要保持不变量时，常用互斥锁或设计更完整的原子协议。

初始实现优先使用默认顺序一致模型，先把同步关系写清楚；性能测量确有需要时，再依据 happens-before 关系放宽内存序。relaxed 适合只要求原子计数的场景，不发布其他数据。volatile 不能替代 atomic。读改写操作要选合适的函数，如 fetch_add 或 compare_exchange，并证明失败重试逻辑正确。

代码示例

```
#include <atomic>
std::atomic<int> requests{0};
requests.fetch_add(1, std::memory_order_relaxed); // 只需原子计数，不发布别的数据
std::atomic<bool> ready{false};
int payload = 0;
// 若用 ready 发布 payload，应使用 release store / acquire load
// atomic 只保护自身对象；余额与版本号等多字段不自动构成事务
// 正确性优先用默认 seq_cst，再按同步关系优化内存序
```

008 C++ 中的线程局部存储（Thread Local Storage）是什么?

答案　thread_local 声明让变量每个线程各有一份，例如每线程临时缓冲、错误码或解析上下文。不同线程读写各自副本时，不会争用同一个变量，因此可以减少为这份线程私有状态加锁的需要。线程结束时，其线程局部对象通常会析构，构造和析构时机应纳入线程生命周期设计。

线程局部存储不是跨线程通信机制，也不会保护普通全局变量；需要共享的数据仍要用锁或原子操作。大型 TLS 对象还会占用每个线程的内存。若它来自动态库，线程局部析构和库卸载还可能受平台运行时规则影响，不能在所有环境中假定相同。线程池复用线程时，线程局部状态也会跨多个任务持续存在。

代码示例

```
#include <string>
#include <string_view>
thread_local std::string scratch; // 每个线程都有独立字符串
void parse(std::string_view input)
{
    scratch.assign(input);
    process(scratch);
}
// 两线程调用时 scratch 不共享，避免为这份临时状态加锁
std::string shared; // 普通全局变量仍需同步保护
// 线程退出时会析构自己的 thread_local 对象
// TLS 不是跨线程通信方式；动态库卸载还需考虑平台运行时行为
```

009 C++ 中的线程（std::thread）如何创建和管理?

答案　构造 std::thread 后，线程函数就可能开始运行。持有 thread 的对象销毁前，必须对仍可 join 的线程调用 join 或 detach，否则析构会调用 std::terminate。join 等待工作完成，也让调用者能在后续安全清理被线程引用的数据；detach 则让线程独立继续，生命周期很难再由局部代码证明。

工作线程不能让异常逃到线程入口，否则会终止程序；应在线程函数内捕获并通过状态或 promise/future 回传错误。引用参数、lambda 引用捕获以及共享对象都要保证活得比工作线程久。优先使用 RAII 管理线程，或在支持的 C++20 环境使用会请求停止并 join 的 jthread；即便如此，停止仍需线程合作响应。

代码示例

```
#include <exception>
#include <thread>
std::thread worker(
    []
    {
        try
        {
            do_work();
        }
        catch (...)
        {
            report_failure();
        }
    });
// 线程函数不可让异常逃逸到线程入口，否则会 terminate
if (worker.joinable())
    worker.join(); // 返回前确保工作结束
// 所有引用捕获的数据都必须比 worker 活得久
// detach 会让生命周期难以证明；优先 RAII 管理或 C++20 std::jthread
```

010 C++ 中的读写锁（std::shared_mutex）是什么?

答案　std::shared_mutex 提供共享锁和独占锁：多个 reader 可同时持有共享锁读取，同一时刻 writer 需要独占锁，读写不能并发。读者通常用 shared_lock，写者用 unique_lock。锁应覆盖整个受保护数据访问过程，并由所有读写路径遵守同一协议。

它更适合读多写少、读取临界区足以抵消额外同步成本的场景。写多或临界区很短时，普通 mutex 可能更简单也更快。标准不保证具体读写公平策略，写者可能等待；不能持有共享锁后直接假定可以原子升级为独占锁，否则容易死锁或产生检查与更新间隙。需要升级时应显式设计重试协议。

代码示例

```
#include <shared_mutex>
std::shared_mutex data_mutex;
Data data;
Data snapshot()
{
    std::shared_lock<std::shared_mutex> lock(data_mutex);
    return data;
}
void update(Data next)
```

```
{
    std::unique_lock<std::shared_mutex> lock(data_mutex);
    data = std::move(next);
}
// 多个 reader 可同时读，writer 独占；锁覆盖整个数据访问
// 写多时读写锁开销/等待可能不划算；标准不保证公平策略
// 不要假设能安全地从共享锁直接升级到独占锁
```

4.2 并发任务组织

011 如何使用 C++ 实现生产者-消费者模型?

答案　生产者消费者模型通常用一个队列保存任务、用互斥量保护队列和停止标志，再用条件变量等待“队列非空”或“队列未满”。生产者在锁内检查容量并入队，解锁后通知消费者；消费者取出任务后先解锁，再执行任务，避免长时间占用队列锁。关闭时要把停止状态纳入等待谓词，并唤醒全部等待者。

有界队列还须明确满时是阻塞、拒绝还是丢弃。也可用 empty/full 信号量记录槽位和任务数量，再用互斥量保护队列；必须先获得许可，再进入队列临界区，以免持锁等待某个由其他线程释放的许可。关停时要定义剩余任务排空还是丢弃，以及等待者如何退出。通知或信号量是同步工具，任务队列和业务成功状态仍需清晰管理。〔N35〕

代码示例

```
#include <condition_variable>
#include <cstddef>
#include <mutex>
#include <queue>
#include <utility>
std::mutex m;
std::condition_variable not_empty, not_full;
std::queue<Job> q;
bool stopping = false;
constexpr std::size_t capacity = 8;
void produce(Job job)
{
    std::unique_lock<std::mutex> lk(m);
    not_full.wait(lk,
                  []
                  {
                      return q.size() < capacity || stopping;
                  });
    if (stopping)
        return;
    q.push(std::move(job));
    lk.unlock();
    not_empty.notify_one();
}
void consume()
{
    std::unique_lock<std::mutex> lk(m);
    not_empty.wait(lk,
                   []
                   {
                       return !q.empty() || stopping;
                   });
    if (q.empty())
        return;
    Job job = std::move(q.front());
    q.pop();
    lk.unlock();
    not_full.notify_one();
    run(job);
} // 锁外执行；shutdown 要唤醒等待者
```

012 如何实现线程池?

答案　线程池通常有一组工作线程、一个受互斥量保护的任务队列，以及用于等待任务或关闭状态的条件变量。工作线程醒来后取出一项任务，释放队列锁，再调用任务函数；否则一个慢任务会阻塞其他线程提交和领取任务。线程数、队列容量和队列满时的策略应根据服务负载决定。

接口还要定义任务异常如何回传、关闭时排空还是丢弃队列、是否允许继续提交，以及 shutdown 能否重复调用。停止时先发布停止状态并唤醒所有工作线程，再等待它们 join；线程仍依赖的对象必须在 join 后才销毁。线程池不会自动保证任务业务成功，任务结果与错误仍应通过 future、回调或状态返回给提交者。

代码示例

```
// 线程池伪代码：队列由 mutex 保护，cv 表示“有任务或停止”
void worker_loop()
{
    for (;;)
    {
        Task task;
        {
            std::unique_lock<std::mutex> lk(m);
            cv.wait(lk,
                    [&]
                    {
                        return stopping || !tasks.empty();
                    });
            if (stopping && tasks.empty())
                return;
            task = std::move(tasks.front());
            tasks.pop();
        }
        try
        {
            task();
        }
        catch (...)
        {
            report_task_error();
        } // 锁外执行
    }
}
void shutdown()
{
    set_stopping();
    cv.notify_all();
    join_all();
}
// API 还需规定队列满、异常传回与“排空/丢弃”策略
```

4.3 内存模型与无锁协作

013 std::memory_order 中 relaxed、acquire/release 和 seq_cst 的差异是什么？

答案　relaxed 保证对原子对象本身的操作不可分割，但不让它与其他普通内存访问建立同步关系。release 写入配合读到该值的 acquire 读取，可以建立跨线程同步，使发布之前的普通写对获取方可见。seq_cst 在这些同步规则外，还要求顺序一致操作遵守一个满足约束的单一总序，推理较直观但不一定总是必要。

选内存序应从数据依赖和 happens-before 关系出发，先写明谁发布、谁读取以及何时可见，再验证所有路径。不要把 acquire/release 当作给单个变量“加锁”；多个字段仍要有完整同步协议。只统计调用次数且不发布相关数据时 relaxed 可能足够。volatile 不能替代原子和同步。〔S2〕

代码示例

```
#include <atomic>
#include <thread>
int payload = 0;
std::atomic<bool> ready{false};
std::thread producer(
    [&]
    {
        payload = 42;
        ready.store(true, std::memory_order_release);
    });
while (!ready.load(std::memory_order_acquire))
{
```

```
} // 读到 release 后同步
int value = payload; // producer 对 payload 的写 happens-before 此读
producer.join(); // join 管理生命周期；不能用 volatile 代替 atomic
```

014 std::jthread 和 std::thread 有什么不同？stop_token 能强制终止线程吗？

答案　std::thread 若对象析构时仍处于 joinable 状态，会调用 std::terminate；C++20 的 std::jthread 析构时会请求停止，并在需要时 join，从而提供更安全的线程所有权管理。jthread 可把 stop_token 传给线程函数，线程定期查看停止请求并自行退出。

stop_token 是协作式信号，不会强行终止线程，也不会自动中断任意阻塞系统调用。线程需要主动检查、使用支持停止的等待机制，或让阻塞操作可取消。请求停止后仍须保证线程最终能返回，否则 jthread 析构等待时仍会卡住。停止逻辑还要负责唤醒条件变量上的线程，并确保退出前完成必要清理。〔S2〕

代码示例

```
#include <thread>
std::jthread worker(
    [](std::stop_token stop)
    {
        while (!stop.stop_requested())
            process_one(); // 合作式检查停止请求
    });
worker.request_stop(); // 只发信号，不会强制杀掉线程
// jthread 析构会请求停止并 join；线程体必须能最终退出
// stop_token 不会自动中断任意阻塞调用，等待应使用可停止机制
```

015 std::latch 和 std::barrier 如何选择？

答案　latch 是一次性计数门闩：参与者调用 count_down，计数归零后所有等待者通过；它不能重新计数开启下一轮。barrier 面向分阶段重复同步，每一阶段都等待设定数量的参与者到齐，再进入下一阶段，还能在阶段切换时执行 completion 回调。

如果只是让若干线程完成启动准备后一起开始一次任务，latch 结构简单合适；如果每轮算法都要等所有线程处理完各自分块再进入下一轮，例如重复图像处理阶段，就用 barrier。参与者数量和退出策略必须匹配实际运行线程，某个参与者缺席会让等待无法完成。使用前还要检查所需标准库版本与实现支持。〔S2〕

代码示例

```
#include <latch>
std::latch start_line(4); // 一次性门闩，4 个参与者汇合后开放
void worker()
{
    prepare();
    start_line.count_down();
    start_line.wait();
    run_phase();
}
// 下一轮不可重置 latch；多轮阶段同步改用 std::barrier
// barrier 每轮参与者到齐后进入下一阶段，还可执行 completion 回调
// 启动后固定参与者数量的一次汇合选 latch；重复算法阶段选 barrier
```

016 C++20 atomic::wait/notify 与忙等轮询有什么区别？

答案　C++20 atomic::wait(old) 会在原子值仍等于 old 时阻塞，而不是让线程持续循环消耗 CPU；状态改变后，其他线程可调用 notify_one 或 notify_all 唤醒等待者。notify 本身不会修改状态，也不保存待处理事件，因此必须先更新共享值，再通知。被唤醒后仍需重新读取并检查业务谓词。

相对忙等轮询，这种方式减少等待期间的 CPU 空转，但不一定适合需要极低唤醒延迟的所有场景。若状态从旧值变了又迅速回到 old，等待者可能观察不到中间变化；用单调递增序号或队列保存事件通常更可靠。atomic wait 也不是通用超时、取消或事件计数接口，应用要另外处理这些需求。〔S2〕

代码示例

```
#include <atomic>
std::atomic<int> state{0};
void waiter()
{
    while (state.load() == 0)
        state.wait(0); // 值未变时阻塞
    continue_work();
}
void signal()
{
```

```
    state.store(1);
    state.notify_one();
} // 先更新状态，再唤醒
// notify 不是事件计数；谓词/值是状态来源，醒来后仍要重查
// 相比忙等减少 CPU 空转；若值快速变回旧值，可能看不到中间状态
```

017 数据竞争（data race）和竞态条件（race condition）有什么区别?

答案　数据竞争是 C++ 内存模型中的具体错误：两个线程并发访问同一内存位置，至少一个是写，并且访问之间没有 happens-before 排序，其中又至少一个不是原子操作。发生数据竞争会导致未定义行为。用互斥锁或正确的原子同步消除它，是程序可定义运行的基本条件。

竞态条件是更广义的逻辑问题，指结果取决于线程执行时序。例如库存用 atomic 保护，两个线程各自先读“库存大于零”，再分别减一；每次访问都原子，但检查和扣减没有组成一个事务，库存仍可能变负。修复数据竞争并不能保证业务不变量，必要时把整个检查更新放在锁中，或用 CAS 循环完成条件更新。〔S2〕

代码示例

```
std::atomic<int> stock{1};
// 两次原子操作各自安全，但“检查再扣减”不是一个原子事务
if (stock.load() > 0)
    stock.fetch_sub(1); // 两线程都可能先读到 1，导致变负
// 用 CAS 循环，或把检查与更新放进同一把 mutex
// 数据竞争会导致 C++ 未定义行为；竞态条件还包括这种逻辑错误
// 修掉 data race 后仍要验证业务不变量
```

018 std::atomic_ref 有什么用途？使用时要注意什么?

答案　atomic_ref 可把现有 T 对象临时按原子方式访问，适合数据结构布局已固定、不能把成员直接改成 std::atomic<T> 的场景。它不拥有目标对象，只保存对它的引用关系；目标地址必须满足 atomic_ref<T>::required_alignment，且目标对象必须一直活到所有 atomic_ref 使用结束。

atomic_ref 存续期间，对目标的并发访问必须遵守原子访问规则。不能一个线程通过 atomic_ref 写，另一个线程同时用普通非原子读取同一对象；这仍会产生数据竞争。统一把相关并发读写都通过原子引用访问，或用锁保护普通访问。atomic_ref 是否无锁由实现和类型决定，不应把“atomic”误解为一定没有锁或固定开销。〔S2〕

代码示例

```
#include <atomic>
struct Counter
{
    alignas(std::atomic_ref<int>::required_alignment) int value = 0;
};
Counter counter;
std::atomic_ref<int> atomic_value(counter.value); // 在已有对象上原子操作
atomic_value.fetch_add(1, std::memory_order_relaxed);
// atomic_ref 存活期间，禁止对 value 并发做普通非原子读写
// Counter 和成员必须活到所有 atomic_ref 使用完；还需满足对齐要求
```

第 5 章 性能与工程实践

5.1 性能测量与基础优化

001 C++ 中如何提高程序的执行效率?

答案　提高效率的第一步不是猜哪一行慢，而是说清楚要改善什么：吞吐量、尾部延迟、CPU 时间，还是内存峰值。然后用采样 profiler、硬件计数器和代表性基准找出热点。通常先减少重复计算、改善算法复杂度和不必要分配，再看数据局部性与并行度。每次只改一个因素，才知道收益来自哪里。

近似模型下，CPU 时间与退休指令数、平均 CPI 和时钟频率相关；IPC 是每周期退休指令数。它们帮助解释执行情况，却不能单独评价不同负载，墙钟时间还包含等待 I/O 或锁的时间。可结合 cycles、instructions、branch-misses 和 cache-misses 判断瓶颈，并在相同编译选项与端到端负载上复测，避免把微基准收益直接当成用户可见提升。〔I51〕

代码示例

```
#include <chrono>
#include <chrono>
auto begin = std::chrono::steady_clock::now();
for (int i = 0; i < repetitions; ++i)
    run_representative_work();
auto elapsed = std::chrono::steady_clock::now() - begin;
report(elapsed); // 先记录正确性、吞吐/尾延迟/内存等目标指标
// 用 profiler 找热点，再一次只改一个因素并重复测量
// 固定编译选项和输入；微基准收益须用端到端负载验证
```

002 C++ 中如何避免不必要的内存拷贝？

答案　减少复制之前先区分两种接口关系：只读借用，还是转移所有权。大型只读对象可通过 const 引用接收，连续数据也可用 span；需要接管资源时可按值接收，让调用方传右值后移动进函数。这样接口能直接表达生命周期与所有权，而不是为了省一份复制返回不安全的借用引用。

从函数按值返回局部结果通常能由返回值优化消除拷贝，必要时对象也可移动；不要返回局部变量的引用。emplace 可能让元素直接在容器存储中构造，但是否省步骤取决于参数和类型，不能当成固定性能保证。避免复制的同时也要保证引用目标始终活着，先用 profiler 找到实际复制成本再调整接口。

代码示例

```
#include <vector>
void inspect(const std::vector<Record>& records)
{ // 只读借用，不复制大容器
    for (const Record& record : records)
        inspect_one(record);
}
std::vector<Record> make_records()
{
    std::vector<Record> result;
    fill(result);
    return result; // 返回局部值可做 NRVO，不要返回局部引用
}
void take_ownership(std::vector<Record> records)
{
    consume(std::move(records));
}
// 借用用引用/span；真正转移所有权时用值加移动，并检查生命周期
```

003 C++ 中的 SIMD（单指令多数据）优化如何实现？

答案　SIMD 让一条向量指令同时处理多个数据，例如并行计算一批像素的亮度。要让编译器更容易自动向量化，先使用连续数据、简单循环和互不依赖的运算，并确认输入输出没有会阻止优化的别名关系。查看编译器向量化报告或生成汇编，确认优化真的发生，再考虑手写 intrinsic 或成熟 SIMD 库。

收益受 CPU 指令集、数据对齐、循环尾部、分支和内存带宽影响；计算密集与内存密集程序的效果差别很大。手写向量代码还需处理不满一整个向量的尾部，并保证目标机器支持相关指令。应在实际部署 CPU 和代表性数据上对比吞吐，避免因一组微基准数据向量化而让通用路径变慢。

代码示例

```
#include <cstddef>
// 连续数组、彼此独立的运算利于编译器自动向量化
for (std::size_t i = 0; i < count; ++i)
    output[i] = input[i] * scale + bias;
// 先确认无重叠/别名假设，查看编译器向量化报告
// 对齐、尾部元素、分支和内存带宽都会影响收益
// 手写 intrinsic 前，在目标 CPU 和代表性数据上比较吞吐
// SIMD 只适合彼此独立且能批量执行的工作
```

004 C++ 中的内联函数（inline）如何优化性能？

答案　inline 不保证机器码一定省略函数调用。优化器会根据函数大小、优化级别、调用位置和跨模块信息决定是否展开；展开可能消除调用开销，也可能暴露常量传播与寄存器优化机会。inline 另有语言层面的 ODR 作用，允许满足规则的同一函数定义出现在多个翻译单元，这与机器码是否展开是两件事。

过多展开会增加代码体积、指令缓存压力和编译工作，反而可能拖慢程序。头文件中的小函数可能适合 inline，但性能判断要看目标构建产物、优化汇编和代表性基准。不要仅因函数短就手工内联，也不要把 inline 当作强制优化开关。

代码示例

```
// 头文件中的定义可标 inline 以符合多翻译单元 ODR
inline int normalize(int x)
{
    return x < 0 ? 0 : x;
}
int result = normalize(input); // 编译器仍可决定保留普通调用
// 内联可省调用并暴露常量传播；过度展开增大代码和指令缓存压力
// 查看优化后汇编与基准，不要把 inline 当性能保证
// ODR 允许定义跨单元出现，与优化器是否展开是两回事
```

005 C++ 中的循环展开（Loop Unrolling）是什么？

答案　循环展开把若干次迭代的工作合并到一次循环体中，可能少执行循环计数和分支，并让处理器看到更多彼此独立的指令。对非常短、计算密集的循环，这有时能减少控制开销。展开倍数不能只看循环有多短，也要考虑循环体的依赖关系和处理器资源。

过度展开会增大代码，增加寄存器压力和指令缓存未命中；尾部不足一个展开批次的数据还需额外处理。编译器通常能根据优化信息自动选择展开，手工改写后应检查汇编、正确覆盖尾部，并在代表性数据上测量。若循环受内存带宽限制，展开未必有帮助。

代码示例

```
#include <cstddef>
for (std::size_t i = 0; i + 3 < n; i += 4)
{ // 一次处理四项
    sum += values[i];
    sum += values[i + 1];
    sum += values[i + 2];
    sum += values[i + 3];
}
for (std::size_t i = n - n % 4; i < n; ++i)
    sum += values[i]; // 尾部
// 展开可能减少分支，但会增加代码体积/寄存器压力
// 现代编译器常能自动展开；先看汇编和真实基准
```

006 C++ 中的线程开销如何优化？

答案　反复为很短的任务创建、销毁线程，会增加调度、栈初始化和资源管理开销；线程池、任务批处理或合并成更粗粒度工作可以复用线程并降低每个任务的固定成本。但线程数太多会引起上下文切换、锁竞争和队列等待，最佳数量还要考虑 CPU、I/O 和任务特征。

线程上限不等于“可用内存除以固定栈大小”：还受栈与保护区、地址空间、系统和容器进程/线程限额等约束，pthread_create 可能因资源限制返回 EAGAIN。Linux 亲和性可以限制线程运行 CPU 集合，但不等于独占核心，也可能破坏负载均衡或 NUMA 放置。应同时比较吞吐和尾延迟，并评估缩小线程栈的溢出风险。〔I25、I52〕

代码示例

```
// 伪代码：启动时创建固定工作线程，后续任务复用线程
ThreadPool pool(worker_count);
for (Batch batch : split_into_coarse_batches(input))
    pool.submit(
        [batch]
        {
            process(batch);
        });
pool.wait_idle(); // 完成后继续汇总；析构前先排空并 join
// 合并过小任务可降低调度/分配开销；队列、停止和异常需定义
// 线程太多会增加切换和争用，按吞吐与尾延迟测量
```

5.2 算法、缓存与 I/O

007 如何优化 C++ 中的算法复杂度？

答案　分析算法复杂度时先说清楚输入规模以及分析口径：Big-O 常用来描述增长上界，还要区分最坏、平均、期望或均摊情形，并分别考虑辅助空间和输出空间。顺序步骤通常由增长最快的主项决定；循环应按实际迭代次数求和；递归还要计入调用深度和栈空间。

结构选择要匹配操作与数据条件：unordered_map 查找平均期望 O(1)，但最坏可能 O(n)；有序 map 的查找是 O(log n)。list 只有拿到位置迭代器时插删才便宜，按值查找仍是 O(n)。二分要求有序，滑动窗口通常要满足单调条件，动态规划适合消除重复子问题。复杂度给出增长趋势，最终还要结合输入分布和实测常数判断。

代码示例

```
#include <algorithm>
#include <list>
#include <map>
#include <unordered_map>
std::unordered_map<Key, Value> by_key; // 平均期望 O(1)，最坏 O(n)
std::map<Key, Value> ordered; // 有序查找/插入 O(log n)
std::list<Value> chain; // 已有迭代器时插删便宜，按值查找仍 O(n)
// 二分需有序；双指针/滑窗需满足单调条件，DP 消除重复子问题
// 依输入规模和操作比例选结构，再用代表性负载验证
```

008 如何优化 C++ 的循环和内存访问?

答案　循环优化常从访问模式入手：连续读取有利于硬件预取，沿指针链随机访问则容易等待内存。可根据一次处理完整对象还是批量处理单个字段，考虑 AoS 或 SoA；循环融合能减少重复读写，分块能让工作集更容易留在缓存，但两者也可能扩大工作集或降低可读性。

不要为减少一次 cache miss 就盲目重排公开结构，改动可能破坏 ABI。分支预测失败也会丢弃错误路径工作；排序数据或给 likely/unlikely 提示不是免费优化，数据重排本身有成本，编译器提示也不强制硬件预测。用 profiler、硬件计数器和代表性输入确认缓存、带宽或分支确实是瓶颈，再比较端到端收益。〔I42、I53、I54、S3〕

代码示例

```
#include <cstddef>
// 行主序矩阵按行扫描，可顺着连续地址访问
for (std::size_t r = 0; r < rows; ++r)
    for (std::size_t c = 0; c < cols; ++c)
        sum += matrix[r * cols + c];
// 分块让工作集更可能留在缓存；AoS 适合逐对象处理、SoA 适合批量单字段
// 循环融合可减少重复读写，也可能扩大工作集或降低可读性
// 用 cache-miss/带宽计数确认瓶颈，别为 ABI 贸然重排公开结构
```

009 如何优化 C++ 程序中的 I/O 性能?

答案　提升文本或文件 I/O 性能，通常先减少系统调用次数：批量读写、合适的缓冲和避免逐字符刷新都能有所帮助。iostream 在不需要与 C stdio 同步时可关闭 sync_with_stdio，并按应用需要解除 cin 与 cout 的绑定；之后不要假设 printf 和 cout 的缓冲顺序一致。交互式程序还要在需要对方响应时主动 flush。

C 流的 setvbuf 应在该流开始 I/O 前配置，它只控制 C stdio 缓冲。接口选择要与读取语义一致，不能为了吞吐破坏协议边界。std::async 只是调度一个任务，不能把阻塞文件读取自动变成内核异步 I/O；真正异步能力依平台接口与存储设备而定。改动后用真实文件大小和读写模式测量。

代码示例

```
#include <iostream>
#include <vector>
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr); // 所有 I/O 前调用
    std::vector<int> values;
    int x;
    while (std::cin >> x)
        values.push_back(x); // 批量缓冲输入
    std::cout << values.size() << '\n';
} // 关闭 sync 后勿假设与 printf 的缓冲顺序；交互场景需 flush
// 异步调度不等于内核异步 I/O；按平台选缓冲/批量接口
```

010 如何使用缓存优化提高 C++ 程序性能?

答案　缓存优化利用时间局部性和空间局部性：近期使用的数据较可能再次使用，相邻数据适合连续访问。处理器常有多级缓存，此外还有寄存器、主存和存储设备；各级容量、共享方式和包含关系依处理器而异。缓存行还涉及索引、标签和行内偏移，容量不足或映射冲突都可能造成失效。

数据布局要配合访问：逐个处理完整对象适合 AoS，一次对大量对象处理同一字段时 SoA 可避免加载无关数据。缓存行大小不是所有 CPU 都固定 64 字节，伪共享会让逻辑独立的写入争抢同一行。缓存命中也不代表数据已经持久化到存储介质。用目标 CPU 的硬件信息、工作集和计数器实际测量 miss、带宽和同步开销。〔I55〕

代码示例

```
#include <cstddef>
#include <vector>
struct AoS
{
    float x, y, z;
};
std::vector<AoS> points(count); // 整个对象处理时 AoS 连续
std::vector<float> x(count), y(count), z(count); // 批量改 x 时 SoA 少加载无关字段
for (std::size_t i = 0; i < count; ++i)
    x[i] *= scale; // 空间局部性
// 重用近期数据利用时间局部性；相邻数据利用空间局部性
// 缓存行和伪共享依硬件；用工作集与硬件计数器验证
```

5.3 链接期优化

011 LTO 和 PGO 分别做什么？可以一起使用吗？

答案　LTO 让链接阶段能够看到多个翻译单元的中间表示，进行跨文件内联和其他整体优化；PGO 则先用运行负载采集分支、调用频率等信息，再指导后续优化。二者提供的信息不同，因此可以组合使用，但会增加构建步骤、profile 管理和验证工作。

PGO 的训练输入应代表生产环境；如果热路径和分支频率明显不同，优化器可能按错误的频率布局代码。LTO/PGO 也会改变构建和调试体验，未必在所有目标上都有收益。部署前应用相同硬件和代表性工作负载比较吞吐、尾延迟、体积及启动时间，并检查 profile 是否仍适合当前版本。〔S3〕

代码示例

```
// PGO 先用代表性负载生成 profile，再用 profile 构建部署版本
// CXXFLAGS += -fprofile-generate
// run representative_workload
// CXXFLAGS += -fprofile-use -flto
// LTO 在链接时跨单元优化，PGO 按运行频率指导分支/调用优化
// 两者可组合；训练数据偏离生产会误导优化
// 比较实际延迟、吞吐和体积后再决定
```

第 6 章 编译、链接与构建

6.1 头文件与项目组织

001 #pragma once 和 #include guards 有什么区别？

答案　Include guard 是标准预处理器写法：头文件第一次展开时定义一个宏，之后再次包含时发现宏已定义，就跳过内容。#pragma once 更简洁，主流编译器普遍支持，但它不是标准 C++ 指令。两者都是防止同一翻译单元重复展开同一个头文件的常见方式。

宏 guard 名称应足够唯一，否则两个不同头文件误用相同名字，其中一个可能被静默跳过。重复包含保护也不等于消除了真正的类型依赖循环；A 需要 B 的完整定义、B 又需要 A 的完整定义时，仍需重新设计边界或采用前向声明。团队统一命名与头文件依赖规则，便于跨工具链维护。

代码示例

```
// 可移植的 include guard
#ifndef PROJECT_NET_SOCKET_HPP
#define PROJECT_NET_SOCKET_HPP
struct Socket
{
    void close();
};
#endif
// #pragma once 更短且主流编译器支持，但属于实现扩展
// 二者都防止同一翻译单元重复展开；guard 宏必须在工程内唯一
// 它们不消除真正的循环依赖
```

002 C++ 中使用 -fPIC 编译选项有什么作用？

答案　-fPIC 通常用于 Linux 等 ELF 平台，让目标文件生成位置无关代码，以便共享库可装载到不同的虚拟地址。它常见于构建动态库；静态库若未来可能链接进共享库，其中的目标文件也可能需要按 PIC 编译，否则链接器可能报告重定位问题。

PIC 的实现细节由目标架构和工具链决定，可能通过不同寻址方式访问全局数据，性能代价也不能一概而论。Windows 等平台的编译选项不同，不能把 -fPIC 当成跨平台标准参数。应依据目标系统、链接器诊断和构建规则选择选项，并测试生成库是否能按预期装载。

代码示例

```
// 共享库里的目标文件通常需位置无关代码
// g++ -fPIC -c lib.cpp -o lib.o
// g++ -shared lib.o -o libexample.so
// -fPIC 使用平台相关寻址方式，适用于多地址装载
// 静态库以后若可能被链接进共享库，也可能要编成 PIC
// 是否需要/性能影响依架构和工具链，Windows 选项不同
```

003 C++ 中动态库的符号表（Symbol Table）是什么？

答案　动态符号表保存动态链接器需要查找和绑定的导出符号及相关信息；普通符号表还可能包含调试、局部或内部符号，两者作用不相同。动态库导出哪些函数和数据，会影响其他程序能否链接，以及以后升级时必须维护的 ABI 范围。

可以把实现符号默认设为隐藏，只显式公开稳定 API，再用 readelf -Ws 或 nm -D 检查动态导出集。导出越多，后续越难重命名或改签名；修改可见性也要评估插件和用户是否依赖原符号。编译器的 visibility 属性不是跨平台标准语法，工程可定义统一宏适配各工具链。

代码示例

```
// 默认隐藏实现符号，只导出有意公开的 ABI
#if defined(__GNUC__)
__attribute__((visibility("default"))) void public_api();
#endif
// readelf -Ws libexample.so 查看 ELF 符号表
// nm -D libexample.so 只查看动态符号
// 普通符号表也可能包含调试/内部符号；导出集是 ABI 面的一部分
// visibility 属性是工具链扩展，跨平台需定义统一宏
```

004 C++ 中的 attribute((constructor)) 和 attribute((destructor)) 有何作用?

答案　__attribute__((constructor)) 和 __attribute__((destructor)) 是 GCC、Clang 等工具链提供的扩展，可让函数在常规入口前或退出阶段运行。它们不是标准 C++ 的初始化机制，执行顺序、优先级和动态库卸载时机要查对应平台文档，不能假设所有编译器行为一样。

这类隐式入口会隐藏依赖关系，多个初始化器之间的顺序也不容易维护，还可能遇到静态初始化顺序或库卸载问题。普通应用通常应使用 RAII 对象、函数内局部 static 或显式 start/stop 接口，让谁初始化、谁关闭更清楚。只有确实需要运行时约定的构造钩子时，才依赖工具链扩展并为其边界写清楚。

代码示例

```
#if defined(__GNUC__)
__attribute__((constructor)) static void before_main()
{
    init_global();
}
__attribute__((destructor)) static void after_main()
{
    shutdown_global();
}
#endif
// 这是 GCC/Clang 扩展，不是标准 C++，库卸载和顺序依平台
// 多个初始化器之间的依赖难表达，可能遇到静态初始化顺序问题
// 普通程序优先使用 RAII、函数内 static 或显式启动/停止接口
```

005 C++ 中的前向声明（Forward Declaration）是什么?

答案　前向声明只告诉编译器某个名字对应的类型或函数存在，不提供类的成员、布局和大小信息。因此把类型作为指针或引用、在函数声明中提到它，通常不需要完整定义；这有助于减少头文件包含和重编译依赖。

一旦要按值存储成员、继承该类、访问其成员或计算 sizeof，就需要完整类定义。把不必要的 include 移进源文件，可减少耦合；但函数定义若在头文件内联访问成员，调用者仍必须看到完整定义。前向声明不能替代定义，也不能解决设计上彼此需要完整布局的循环依赖。

代码示例

```
// node.hpp：若只声明指针/引用，可避免包含完整定义
class Node;
void visit(const Node& node);
class Owner
{
    Node* node_;
}; // 指针成员允许不完整类型
// node.cpp 再 #include "node_definition.hpp" 访问 Node 成员
// 按值成员、继承、sizeof 或内联访问成员需要完整类型定义
```

006 C++ 多文件项目中，头文件与源文件如何组织?

答案　头文件放调用方需要知道的声明、公共类型和必要的模板定义；普通非模板函数实现通常放进源文件，并在程序中定义一次。这样能减少重复编译，也避免多个翻译单元各自生成普通实体定义。每个头文件要自包含并有重复包含保护，不能依赖其他头文件碰巧先包含。

前向声明能减少依赖，但成员按值保存、继承和内联访问成员等场景仍需完整定义。模板通常要在实例化处看到定义，因此常放在头文件或模块接口中。拆分声明和实现也要保持 ABI/API 边界清楚，构建系统应显式列出源文件、库依赖和生成步骤。

代码示例

```
// widget.hpp：向调用方暴露声明和所需完整类型
#ifndef WIDGET_HPP
```

```cpp
#define WIDGET_HPP
class Widget
{
  public:
    void run();
};
#endif
// widget.cpp: #include "widget.hpp" 后定义 void Widget::run() { ... }
// 每个头文件应可自包含；非模板实现一般只在一个源文件定义
// 模板实现通常需在实例化点可见；前向声明可降低无关重编译
```

6.2 编译、链接与库

007 C++ 源文件如何经过编译生成可执行文件?

答案　典型 C++ 工具链会先预处理宏和头文件，再编译源代码并生成汇编，再由汇编器产生目标文件，最后由链接器解析符号、处理重定位并组合目标文件和库。日常构建命令常把这些步骤封装起来，但用预处理输出、汇编和目标文件能帮助定位问题。LTO 开启后，链接阶段还可能继续进行跨翻译单元优化。
语法和类型错误通常出现在编译阶段；undefined reference、multiple definition 等通常出现在链接阶段。阶段划分不是绝对诊断规则，但错误信息会给出线索。还要确认编译命令中的宏、包含路径和目标架构在各源文件间一致，避免产生签名或 ABI 不兼容。

代码示例

```cpp
// 逐阶段查看工具链输出（命令以 GCC/Clang 为例）
// c++ -E app.cpp -o app.ii      预处理宏与 include
// c++ -S app.ii -o app.s        编译为汇编
// c++ -c app.s -o app.o         汇编成目标文件
// c++ app.o lib.o -o app        链接符号、重定位并生成可执行文件
// 源/头错误看编译阶段；未定义/重复符号看链接阶段，LTO 会延伸优化
```

008 extern 在多文件程序中如何声明共享变量?

答案　extern 在多文件工程中常用于告诉当前翻译单元：变量由别处定义。头文件写 extern int count; 通常只是声明，不分配变量存储；恰好一个源文件写带初始化的 int count = 0; 才是定义。其他源文件包含声明后即可访问同一实体。
若在多个源文件都放普通变量定义，通常会产生重复定义链接错误。C++17 的 inline 变量允许在头文件中提供符合 ODR 的共享定义。无论用哪种方式，全局状态仍有初始化顺序和并发访问问题；需要控制生命周期或保持业务不变量时，封装为服务或对象往往比公开可变全局变量更稳妥。

代码示例

```cpp
// shared.hpp: 仅声明，不分配定义
extern int shared_count;
// state.cpp: 程序中恰好一个定义
int shared_count = 0;
// use.cpp: 包含声明后可访问 shared_count
void increment()
{
    ++shared_count;
} // 多线程时仍需同步
// C++17 可用 inline int shared_count = 0; 放头文件共享一个实体
```

009 static 变量在头文件中的作用是什么?

答案　命名空间作用域的 static 变量具有内部链接；放进头文件后，每个包含该头文件的翻译单元都会拥有自己独立的一份副本。这样看似“全局”的计数器在 a.cpp 和 b.cpp 中其实不是同一个对象，会造成状态不一致和额外存储。确实需要一个程序级共享实体时，可在头文件声明 extern，再在一个源文件定义；C++17 起也可用 inline 变量在头文件定义同一实体。还要考虑静态初始化顺序和并发读写。如果只是为了隐藏实现细节，通常把状态放在源文件私有区域或封装进类，而不是用头文件 static 意外制造多份全局状态。

代码示例

```cpp
// counter.hpp
static int counter = 0; // 每个包含该头的翻译单元各自一份
// a.cpp 和 b.cpp 各 include 后，修改的是不同对象
// 需要一个程序级共享实体时：extern 声明 + 单个源文件定义
// 或 C++17 inline int counter = 0; 允许头文件中的同一实体定义
// 全局共享仍需考虑初始化顺序和并发访问
```

010 头文件的依赖关系如何避免循环引用?

答案　头文件循环引用通常说明两个模块的接口边界相互缠绕。例如 A 头文件包含 B，B 又包含 A；include guard 只能阻止重复文本展开，不能让尚未定义完整的类型突然可用。若某个类只通过指针、引用或函数声明提及另一类，可以在头文件前向声明，把真正依赖完整定义的 include 移到源文件。

按值成员、继承、sizeof 或头文件内联代码访问成员，都需要完整类定义，因此仅前向声明不够。此时可抽出共同接口、调整所有权关系或将实现移出头文件。明确哪一侧只借用不拥有对象，有助于避免为了编译通过而放松生命周期边界。

代码示例

```
// a.hpp: 只通过引用提及 B，可前向声明
class B;
class A
{
  public:
    void use(const B&);

  private:
    B* b_;
};
// a.cpp 再 include b.hpp 并定义 A::use，因访问成员需要完整类型
// 如果 A 按值保存 B、继承 B 或内联访问 B 的字段，则前向声明不够
// include guard 防重复展开，不会消除真实依赖循环
```

011 如何定位常见的 C++ 链接错误?

答案　先把报错按类型分类，并查看链接器指出的符号名和目标文件。undefined reference 表示某个引用需要的定义没有进入链接，常见原因是实现源文件漏编译、库漏链接、签名或命名空间不一致；multiple definition 表示同一非 inline 实体重复定义。库找不到时还要核对路径、名称、平台前后缀与链接顺序。

用 nm、readelf、dumpbin 等工具检查对象文件和库里是否有对应符号，再查看构建系统打印出的完整链接命令，确认实际输入。静态库的解析顺序可能受工具链影响。若符号名存在却不匹配，还要检查编译器、运行库、宏与 ABI 配置。链接通过也不能证明 ODR 和运行时 ABI 完全正确。

代码示例

```
// undefined reference: 实现源文件/库未链接，或声明签名、命名空间不一致
// nm -C app.o libpart.a | rg 'WantedSymbol' 检查符号是否存在
// g++ app.o libpart.a -o app   # 并确认静态库位于引用者之后
// multiple definition: 查找重复定义，头文件函数考虑 inline
// 打开构建系统 verbose 命令核对实际输入文件、库路径和链接顺序
// 符号名存在但 ABI 不匹配时核对编译器/运行库/宏设置
```

012 如何调试链接错误?

答案　调试链接错误时，先判断链接器缺的是定义、重复定义，还是库文件本身。undefined reference 通常说明只有声明或调用，定义没有进入链接输入，也可能是签名、命名空间不一致；multiple definition 通常需要定位头文件中的非 inline 定义。若库找不到，则核对搜索路径、平台名称和静态库依赖顺序。

打开构建系统的 verbose 输出可看到真正执行的命令，用 nm -C、dumpbin 等核对符号名。还要检查编译器版本、运行库和宏配置，避免同一接口被不同 ABI 规则编译。源文件含头文件 guard 并不保证 ODR 一定正确，链接器发现不了所有跨翻译单元定义不一致。

代码示例

```
// header.hpp: 多翻译单元可见的相同函数定义要满足 inline 规则
inline int helper(int x)
{
    return x + 1;
}
// 若移除 inline 且 header 被 a.cpp、b.cpp 同时包含，会重复定义
// undefined reference 则相反：只有声明，定义没有进入链接输入
// 用 nm -C 查符号，构建 verbose 输出确认哪个 .o/.a 真正参与链接
// 链接成功也不能证明 ODR 一定满足
```

013 静态库和动态库的链接方式有什么区别?

答案　静态库通常是目标文件的归档，链接时把需要的目标代码纳入最终可执行文件；部署时依赖较直接，但程序可能变大，多份程序也会各自带上相关代码，更新库一般要重新链接。动态库由加载器在运行时装入，多个进程可能共享只读映射，也能单独升级，但必须能找到运行时依赖并保持 ABI 兼容。

实际行为受平台、链接选项、符号可见性和库版本策略影响。“静态”并不总等于完全没有其他动态依赖，“动态”也不保证进程能自动找到正确版本。发布时应检查依赖列表、搜索路径和兼容性。选哪种方式要综合部署便利、文件体积、更新频率、进程共享及 ABI 维护成本。

代码示例

```
// 静态库：归档目标文件，所需代码通常复制进最终程序
// ar rcs libmath.a add.o
// c++ app.o -L. -lmath -o app
// 动态库：运行时加载，可共享映射/单独升级，但要部署依赖和搜索路径
// c++ -fPIC -shared add.cpp -o libmath.so
// c++ app.o -L. -lmath -Wl,-rpath,'$ORIGIN' -o app
// 部署便利、体积、ABI 兼容性和更新方式按平台权衡
```

6.3 ODR 与 ABI

014 C++ 的 ODR（单一定义规则）是什么？违反后一定会有链接错误吗？

答案　单一定义规则（ODR）约束需要定义的类、函数、变量等实体在整个程序中的定义方式。普通非 inline 函数或变量通常只能有一个定义；类、模板及 inline 实体可以在多个翻译单元看到符合规则的定义，但内容和名字查找仍须满足一致性要求。头文件里重复出现并不意味着无条件合法。

违反 ODR 有时会由链接器报重复符号，有时无需实现诊断，程序甚至可能成功链接却产生不同翻译单元行为不一致。因此链接通过不能作为 ODR 正确的证据。C++17 inline 变量是允许多处相同定义的例外，不是允许每个源文件随意给不同初值。〔S2〕

代码示例

```
// C++17 inline 变量允许同一头文件定义出现在多个翻译单元
inline constexpr int retry_limit = 4;
// 对应的定义必须满足 ODR 要求，例如 token 序列和名字查找一致
// 普通非 inline 函数/变量若定义多次通常会链接失败
// 某些 ODR 违规无需诊断，可能链接成功却让不同单元行为不一致
// 所以“链接通过”不是 ODR 正确性的证明
```

015 C++ ABI 是什么？它和源代码兼容有什么区别？

答案　ABI 是已编译模块之间如何互通的约定，包括符号修饰、调用约定、参数和返回值传递、对象布局、虚函数分派、异常处理及运行库选择。源代码兼容关心旧源码重新编译后是否仍能工作；ABI 兼容关心旧二进制不重新编译，能否继续和新库链接及运行。

修改公共类大小、成员布局或虚函数序列，可能让旧二进制按错误偏移读写；改变调用约定、符号名或运行库也可能造成不兼容。稳定二进制边界常采用不透明句柄和 C ABI，把实现类留在库内部。即使 C ABI 也要约定字符串编码、资源释放和错误返回。版本策略应覆盖二进制依赖用户，而不只检查源码编译。〔S4〕

代码示例

```
// 稳定 C ABI 示例：C/C++ 两侧共享不透明句柄，不暴露类布局
extern "C"
{
    struct Engine;
    Engine* engine_create();
    void engine_destroy(Engine*);
    int engine_run(Engine*, const char* input);
}
// 源码兼容看重新编译后能否调用；ABI 兼容看旧二进制能否继续链接运行
// 改公开类大小、虚函数顺序、调用约定或运行库可能破坏 ABI
```

6.4 头文件搜索与二进制装载

016 头文件包含中的双引号和尖括号怎样影响搜索路径？

答案　双引号与尖括号都触发预处理器查找并展开头文件，区别主要是具体实现采用的搜索路径策略。以 GCC 为例，双引号形式先查当前包含文件所在目录，再查 -iquote 等路径和后续搜索链；尖括号通常从 -I、系统头文件目录等位置查找。命令行的 -isystem 等选项会影响具体顺序。

因此不能把双引号简单等同“项目头文件”、尖括号等同“标准库”并据此推断实际解析结果。同名文件可能来自意料之外的目录，应结合完整构建命令和预处理器搜索输出确认。采用哪种括号形式也不会自动修复循环依赖、重复定义或 ODR 问题。〔N24〕

代码示例

```
#include "project/config.hpp" // 常用于项目头文件
#include <vector> // 常用于库头文件
// GCC 可用 -E -v 查看预处理与搜索目录；实际以构建命令为准
```

017 Linux 进程启动或 dlopen 时，动态库如何映射并完成符号绑定？

答案　动态链接的 ELF 程序启动时，内核读取 ELF 信息并把控制权交给程序指定的运行时装载器。装载器根据依赖和搜索规则映射共享对象，处理必要的重定位、符号解析与初始化，然后进入程序入口。运行中 dlopen 可请求装入额外对象，dlsym 查找符号，dlclose 减少引用计数；它不保证任何情况下都立刻卸载。

共享库的只读代码页常可跨进程共享映射，但可写数据具有进程各自的语义。RUNPATH、环境、符号版本以及 RTLD_NOW/RTLD_LAZY 会影响加载与绑定时机。库文件存在不代表所有依赖都能解析，也不代表 ABI 相容；排查应结合装载器诊断工具、目标架构和符号版本信息。〔N25〕

代码示例

```
void* handle = dlopen("plugin.so", RTLD_NOW); // 运行时装入共享库
using Entry = int (*)();
auto entry = reinterpret_cast<Entry>(dlsym(handle, "plugin_start"));
if (entry)
    entry(); // 调用前应检查错误；dlclose 只减少引用计数
```

018 一次函数调用一定把所有参数压栈吗？栈帧通常保存什么？

答案　参数如何传递由目标 ABI 和调用约定决定，并非每次函数调用都把所有参数压入栈。常见 x86-64 System V ABI 会先把合适的整数和浮点参数放进指定寄存器，剩余部分才用栈空间；其他平台和类型有不同规则。返回值也可能通过寄存器或隐藏参数传递。

栈帧可能保存局部对象、溢出参数、需要由被调用方保留的寄存器、返回信息和对齐填充；优化后内联、尾调用、省略帧指针都会改变实际形态。ABI 还规定哪些寄存器由调用方或被调用方保存。调试回溯可使用展开信息，优化构建中的变量位置和源码帧不一定与未优化代码逐一对应。〔N26、N11〕

代码示例

```
long compute(long first, long second, long third)
{
    return first + second + third;
}
long result = compute(1, 2, 3); // 参数可能走寄存器，也可能走栈，取决于 ABI
// 用目标架构的反汇编和展开信息确认，不要假设所有参数都压栈
```

6.5 指令集、ABI 与二进制兼容

019 CPU、操作系统和程序的 32/64 位分别表示什么，为什么不等于指令长度？

答案　CPU 的“位数”通常描述通用寄存器、地址或基本运算能力，但指令编码长度由具体 ISA 规定；例如 RV64 处理 64 位数据时，常见基本指令编码仍为 32 位。程序的 32/64 位属性还取决于执行模式和 ABI 数据模型，不能推断所有整数类型都变成 64 位。Windows LLP64 环境中 long 仍是 32 位。

64 位系统能运行哪些 32 位程序，取决于处理器模式、内核兼容支持和对应动态库，并非自动兼容任意二进制。指针可表示的进程虚拟地址范围和机器可寻址的物理内存上限也不同：PAE 可扩展某些 32 位系统物理寻址，但不扩大普通 32 位指针范围。实际限制还受 CPU 与系统版本约束。〔I43、I44、I45、I46〕

代码示例

```
std::cout << sizeof(void*) << ' ' << sizeof(long) << '\n'; // 查看目标 ABI 的数据模型
// 指令长度、指针宽度、long 宽度和内核位数是不同概念
```

第 7 章 操作系统与 Linux

7.1 文件、权限与进程

001 Linux 文件权限 755 和 644 表示什么？

答案　755 和 644 是三组八进制权限，依次管文件所有者、所属组、其他用户。每组用读 r=4、写 w=2、执行 x=1 相加：755 就是 rwx、r-x、r-x；644 就是 rw-、r--、r--。因此普通程序常见 755，普通文本常见 644，但实际能否访问还受上级目录和其他访问控制规则影响。

目录的权限要换个角度理解：r 允许列出名字，x 允许按名字查找并进入，w 通常要配合 x 才能创建、删除目录项。删除文件主要检查所在目录，而不是文件本身是否可写。创建时请求权限还会被 umask 过滤；chmod 修改已有文件不再经过这次过滤。setuid、setgid 和 sticky 是另外的特殊位，不能混进这三组普通权限简单相加。

代码示例

```
mode_t requested = 0755; // owner/group/other：rwx、r-x、r-x
create_file(path, requested); // 创建时内核按 requested & ~process_umask 过滤
chmod(path, requested); // chmod 修改既有文件模式，不受 umask 再过滤
// 普通文件的 x 表示可执行；目录的 x 表示可搜索/穿越
// 目录中删除目录项通常需目录同时有 w+x，和目标文件权限不同
// setuid/setgid/sticky 是额外语义位；设置时仍需按安全策略检查
```

002 Linux 文件系统中的 inode 保存什么?

答案　inode 可以理解为文件的档案，记录类型、权限、所有者、大小、时间戳、链接计数及数据位置。目录项则像通讯录，保存“名字对应哪个 inode”。名字与档案分开，才能让 a、b 两个名字共同指向同一份内容；修改 a 后，从 b 也能看到修改，这就是硬链接。inode 编号只在对应文件系统内有意义，不能单凭编号跨磁盘判断是不是同一文件。符号链接另有自己的 inode，保存的是目标路径，因此可以跨文件系统，也可以指向尚不存在的目标。删除名字只是移除目录项：硬链接计数归零后，如果仍有进程打开文件，内核还要等这些引用释放才回收内容。所以删除正在写入的日志不一定立即释放空间。最终访问哪个文件还取决于路径解析、挂载点和符号链接。

代码示例

```
struct Inode
{
    Permission mode;
    Owner uid;
    Size bytes;
    BlockMap data;
    int links;
};
// 目录项保存 name -> inode；文件名不是 inode 的一部分
link("a", "b"); // 两个名字共享 inode 和内容
unlink("a"); // 链接计数减一；另一个名字仍可访问
// 最后一个目录链接删除后，若仍有打开句柄，数据也要等关闭后回收
symlink("missing-or-any-path", "shortcut"); // 符号链接有自己的 inode，可悬空
// inode 编号只在本文件系统内有意义；路径解析还受挂载点影响
```

003 Linux 进程有哪些常见状态？阻塞态何时出现?

答案　Linux 进程并不只有“运行”和“停止”。ps 的 R 表示正在运行或等待 CPU，S 表示可中断睡眠，D 表示不可中断睡眠，T 常见于作业控制暂停或被跟踪，Z 表示僵尸。阻塞是任务暂时等不到条件，例如等待数据或设备完成，调度器让它睡眠，把 CPU 交给其他任务；这通常不等于程序出了故障。

S 态常能被信号打断，D 态通常不能靠普通信号立即唤醒，持续 D 态要结合内核栈、设备和 I/O 排查。Z 态已经执行完，只留下退出状态等信息，父进程用 wait 系列回收。孤儿则是父进程先退出，存活子进程由合适的子收割者或所在 PID 命名空间的 init 接管；孤儿与僵尸描述的是两种不同关系，不能混称。〔S5〕

代码示例

```
state = inspect_with_ps_STAT(pid);
if (state == "R")
    schedule_or_running(); // 可运行/运行
if (state == "S")
    wait_interruptible_event(); // 可被普通信号打断的睡眠
if (state == "D")
    inspect_kernel_stack_and_io(); // 常见于不可中断 I/O 等待
if (state == "T")
    inspect_job_control_or_debugger();
if (state == "Z")
    parent.waitpid(pid); // 僵尸只留退出信息，等父进程回收
// 持续 D 态查设备/I/O；普通信号通常不能立即唤醒 D 态任务。
```

004 fork() 与 exec() 分别做什么?

答案　fork 用现有进程创建一个子进程，父子随后从同一处继续执行，通过返回值分辨身份：父进程拿到子 PID，子进程拿到零，失败时父进程得到负一。父子的地址空间逻辑独立，Linux 通常先借助写时复制共享物理页面，写入时才分开，所以不能把 fork 理解为立刻完整复制所有内存。

exec 则是在当前进程里换一套程序映像，成功后执行新程序，PID 保持不变，也不会返回旧代码。命令行启动外部程序通常先 fork，再由子进程 exec，父进程 wait。exec 失败必须走明确的错误路径；子进程常用 _exit 退出，避免重复执行父进程继承来的缓冲刷新等清理。父进程应处理 wait 被信号中断的情况，并最终回收退出状态。

代码示例

```
pid = fork();
if (pid == 0)
{ // 子进程：逻辑独立，物理页通常先通过写时复制共享
    execve(program, argv, envp); // 成功后替换进程映像、PID 不变且不返回
    report_exec_error(errno);
    _exit(127); // 只有失败才走到这里
}
else if (pid > 0)
{
```

```
    waitpid(pid, &status, 0); // 回收子进程并读取退出状态，避免僵尸
}
else
{
    report_fork_error(errno);
}
```

7.2 I/O 多路复用

005 select、poll、epoll 有什么区别?

答案　select、poll、epoll 都让一个线程等待多个文件描述符的就绪状态。select 每次提交描述符集合，返回后再扫描集合，而且常用 libc 的 fd_set 受 FD_SETSIZE 限制。poll 改用数组，可以表示更多描述符，但每轮仍有列表传递和扫描成本；它们的成本不是所谓“指数增长”。

epoll 把兴趣集长期保存在内核，目标就绪时组织就绪项，epoll_wait 主要返回这些有事件的项，因此大量空闲长连接时通常更合适。不过结果仍要复制到用户空间，注册变更和事件处理也有成本，不能说所有操作都是常数时间。连接少、大多活跃或业务处理很重时，应以实际负载比较。ET 模式还要使用非阻塞读写并处理到 EAGAIN。〔N49、S5〕

代码示例

```
// select：每次重建 fd_set，并扫描受 FD_SETSIZE 限制的描述符。
select_wait_and_scan(all_fds);
// poll：没有固定 fd_set 位图上限，但等待返回仍要遍历监视列表。
poll_wait_and_scan(all_fds);
// epoll：内核维护兴趣集与就绪队列，wait 主要取回有事件的 fd。
auto ready = epoll_wait(epfd);
for (auto event : ready)
    dispatch(event);
// 连接少/多数都活跃时差别可能不明显；先按实测瓶颈选择。
```

006 epoll 的 LT 与 ET 模式有什么区别?

答案　LT 是电平触发，可以理解为“只要事情没处理完，就继续提醒”。例如 socket 接收缓冲还有数据，后面的 epoll_wait 通常仍会报告可读。ET 是边沿触发，更关注就绪状态的变化；如果一次只读一小段就返回，剩余数据可能一直留在缓冲里，而程序又等不到期望的新通知。

因此 ET 必须配合非阻塞 I/O，在收到事件后循环读写，直到遇到 EAGAIN 或 EWOULDBLOCK，表示当前无法继续。读到零表示对端关闭发送方向，应按协议结束和清理；EINTR 可以重试，其他错误应明确处理。ET 有机会减少重复提醒，但对状态管理要求更高，不保证吞吐一定胜过 LT。还要防止单个连接长时间占用事件循环。

代码示例

```
// ET：注册前把 fd 设为非阻塞；就绪后循环读取直至 EAGAIN
for (;;)
{
    auto n = ::read(fd, buffer, sizeof(buffer));
    if (n > 0)
    {
        consume(buffer, n);
        continue;
    }
    if (n == 0)
    {
        close(fd);
        break;
    } // EOF 后关闭并退出，不能继续读已关闭 fd
    if (errno == EINTR)
        continue; // 信号中断但无数据，继续尝试
    if (errno == EAGAIN || errno == EWOULDBLOCK)
        break; // 当前缓冲读空
    handle_error(errno);
    break;
} // ET 漏读到 EAGAIN 后不一定再有边沿；LT 通常会继续报告可读
```

7.3 内存、进程间通信与并发模型

007 什么是缺页异常？

答案　缺页异常是 CPU 访问虚拟地址时发现当前页表不能满足映射或权限要求，于是同步转入内核处理。这里的“异常”不一定是程序错误：刚申请但尚未实际分配的匿名页、尚未调入的文件页，以及写时复制页，都可能通过缺页处理建立正确映射，然后重试原来的指令。

内核先判断地址是否属于合法区域以及访问权限是否允许，才能决定补页还是报告故障。越界、非法写只读区域等通常会导致 SIGSEGV，映射文件某些后端错误可能导致 SIGBUS。无需磁盘读取的轻缺页与需要调入数据的主缺页耗时不同，所以缺页次数不能直接当成磁盘访问次数。持续主缺页往往提示内存压力或访问局部性差，要同时看换页和工作集。

代码示例

```
address = load_or_store(virtual_address);
if (!page_table_has_mapping(address) || !permission_allows(address, operation))
{
    trap_to_kernel_page_fault(address); // CPU 同步异常，交给内核判因
    if (is_valid_demand_page(address))
    {
        map_or_load_page();
        retry_instruction();
    }
    else
        signal_process(SIGSEGV_or_SIGBUS); // 越权/无效地址不应盲目重试
}
// 匿名页分配、驻留页权限处理可无需磁盘；主缺页调入文件/交换页更慢。
// 少量缺页常见，持续主缺页要排查内存压力与访问局部性。
```

008 什么是虚拟内存？mmap 做了什么？

答案　虚拟内存让进程通过自己的虚拟地址访问数据，再由页表映射到物理页或相应后端。mmap 建立文件映射或匿名映射，成功只表示取得一段地址范围，不表示内容已进入 RAM。MAP_SHARED 的修改可以共享，MAP_PRIVATE 通常在写入时复制页面，私有修改不会按共享映射的方式写回原文件。文件映射还要处理偏移对齐、映射失败和文件被截断后的 SIGBUS。

内核用 VMA 描述属性一致的地址区间，包含权限、文件、偏移和共享方式；区间存在不代表每一页都驻留。同一进程的线程共享这套地址空间。缺页时先查 VMA 判断访问是否合法，再建立更细的页表映射。mmap、munmap、mprotect 可以拆分合并 VMA，过碎也可能碰到 max_map_count；内部数据结构会随内核变化，不能把旧版红黑树当接口保证。申请是否成功还受地址空间、RLIMIT、cgroup 和内存承诺策略影响。Linux overcommit 的模式零做启发式检查，模式一允许过量承诺，模式二按配置预算核算；CommitLimit、Committed_AS 与实际驻留量 RSS 不是一回事。即使没有 Swap，也可能回收文件页腾出内存，因此不能只拿申请量与 RAM 比较，更不能把 mmap 成功当作未来每次写入都保证成功。〔I20、N44、I16〕

代码示例

```
void* p = mmap(nullptr, length, PROT_READ | PROT_WRITE, MAP_SHARED, fd, offset);
if (p == MAP_FAILED)
    return report_mapping_error(errno); // 先检查失败
// offset 需满足页对齐等平台要求；返回映射不代表页已驻留 RAM
write_bytes(p); // MAP_SHARED 与文件映射共享修改，落盘时机另行管理
void* q = mmap(..., MAP_PRIVATE, ...); // 写入通常触发写时复制隔离
// 文件截断后访问超出新 EOF 的页可能收到 SIGBUS
munmap(p, length); // 映射长度与释放范围要匹配
```

009 如何排查进程 CPU 使用率突然升高？

答案　CPU 突然升高先要确认“谁在忙、忙在哪里”，而不是先重启。用 top、ps 或 pidstat 看目标进程和线程，结合 mpstat -P ALL 1 区分各核用户态、系统态与等待情况。用户态高可能是算法、忙循环或大量业务，系统态高可能与频繁系统调用、中断或调度有关；这些只是线索，还要拿到实际热点栈。

用 perf top、perf record 或对应运行时的采样工具定位最常出现的调用路径，再与请求量、锁竞争、错误重试和最近变更对照。JVM 可从 top -H 找十进制 TID，转成十六进制后匹配 jstack 的 nid，或用 Arthas thread 看栈。修复后在相同负载下比较吞吐和尾延迟：CPU 降低也可能只是处理请求变少，不能仅凭瞬时占用判定优化有效。

代码示例

```
top_or_ps(pid); // 先确认进程 CPU，再找高占用线程并识别用户态/系统态
for (tid : hot_threads(pid))
{ // ps -L / top -H 等工具得到线程 ID
    profile_stack(tid); // perf top/record 或采样 profiler 找热点栈
    map_tid_decimal_to_hex_for_logs(tid); // 日志/内核工具的 ID 进制可能不同
```

```
}
correlate_with_request_rate_logs_locks_syscalls_and_recent_changes();
optimize_confirmed_hot_path();
benchmark_same_load_and_compare_throughput_and_tail_latency(); // 不凭瞬时 CPU 百分比下结论
```

010 如何用 GDB 定位崩溃?

答案　用 GDB 查崩溃，首先要拿到与现场匹配的材料：core 文件、当时的可执行程序、共享库和调试符号。core 是崩溃现场快照，可能包含线程寄存器、栈和部分内存，实际保存范围受系统配置、权限和大小限制。符号文件对错比是否敲了很多调试命令更关键，版本不匹配会把调用栈和变量解释歪。

加载后先看崩溃信号和所有线程调用栈，再切到相关线程与栈帧，检查参数、对象地址和生命周期。优化编译可能让变量消失或被搬到寄存器，不能假定所有局部变量都能恢复。崩溃在释放内存或容器操作处，根因也可能是更早的越界写。可用 ASan、UBSan 或最小复现把怀疑变成证据，不能只按栈顶位置修改代码。

代码示例

```
enable_core_dump_and_reproduce();
gdb("./program", "core");
thread_apply_all("bt"); // 先看所有线程与崩溃线程栈，不把栈顶直接当根因
select_thread(crashing_thread);
frame(fault_frame);
print(arguments_and_nearby_state);
check_binary_matches_core_and_debug_symbols(); // 符号不匹配会误导栈和变量
rebuild_with_asan_ubsan_or_minimal_reproducer(); // 定位越界、空指针、释放后使用
```

011 常见的进程间通信方式有哪些?

答案　进程间通信可以按传输内容和范围选：管道适合字节流，消息队列适合离散消息，信号适合少量通知，共享内存适合同机大块数据，Unix 域 socket 适合同机双向协议，TCP 或 UDP socket 可以跨机器。信号量、进程共享锁或原子量常负责同步，而不直接承担大块数据传输；选型还要考虑容量、背压和异常退出后的恢复。

共享内存减少某些内核传递拷贝，但不会自动解决并发可见性和清理。管道有容量，所有写端引用都关闭且缓存读尽后读端才见 EOF；没有读者时写入要处理 SIGPIPE 或 EPIPE。PIPE_BUF 范围内的写入原子性只防止多个写者交错，不保证一次 read 就是一次消息。匿名管道常在 fork 后共享，FIFO 则有路径名，打开和读写还需考虑对端及非阻塞语义。〔I26、N35〕

代码示例

```
if (large_payload && same_host)
{
    shared_memory();
    add_process_shared_mutex_or_atomic();
}
else if (small_parent_child_stream)
{
    pipe();
    respect_pipe_capacity_and_backpressure();
}
else if (same_host && socket_protocol_needed)
    unix_domain_socket();
else
{
    tcp_or_udp_socket();
} // 跨主机需要考虑网络故障与协议
// 消息队列适合带排队语义的离散消息；信号只传少量通知信息。
// IPC 选型同时看数据量、主机范围、同步/容量、背压和故障恢复。
```

012 进程、线程和协程有什么区别?

答案　进程是资源与故障隔离的基本边界，通常有独立虚拟地址空间；线程共享所属进程的堆和文件描述符等资源，由内核调度。Linux NPTL 通常把每个 POSIX 线程对应为一个内核可调度任务。共享使线程传数据方便，也使数据竞争和内存损坏会影响整个进程，不能把线程理解为自带隔离的小进程。

协程则通常由运行时在用户态按挂起点切换，可以把很多等待 I/O 的任务复用到少量承载线程上；能否并行取决于运行时是否使用多个内核线程。直接调用阻塞接口可能卡住承载线程及其上的其他协程。线程触发默认致命信号通常结束整个进程，运行时捕获的语言异常却可能只结束一个任务；捕获 SIGSEGV 也不能让任意内存损坏或 C++ 未定义行为安全继续。〔N30、N34〕

代码示例

```
Process p = isolated_address_space_and_resources(); // 故障隔离较好，创建/切换成本通常较高
Thread t = shared_heap_and_file_descriptors(p); // 内核调度；共享状态需加锁/原子保护
Coroutine c = runtime_switches_at_await_points(); // 用户态轻量切换，适合大量 I/O 等待
```

```
if (coroutine_calls_blocking_io())
    block_carrier_thread(); // 可能连带卡住其他协程
if (runtime_schedules_on_multiple_kernel_threads())
    coroutine_work_can_run_in_parallel();
// 选型按隔离、共享、调度成本与阻塞行为，不按“协程一定并行”判断。
```

013 Linux 中 fork 后的 Copy-on-Write（写时复制）是怎样工作的?

答案　写时复制的关键是“先共享，只在需要独立修改时复制”。fork 后父子进程拥有逻辑独立的地址空间，但可先把可私有写入的页面映射到同一物理页，并暂时禁止直接写入。任一方写这些页面时触发缺页，内核建立该方可写的私有页面；未修改的页面继续共享，不必为另一个地址空间立刻复制整份数据。

例如父进程有一百兆数据，子进程马上 exec 新程序，大部分旧页面从未被写，就避免了无用复制。相反，如果子进程逐页修改整块数据，仍会付出页面复制、缺页处理和页表管理成本。复制粒度通常是页，不是单个变量；少量字节的分散写入也可能触碰很多页。共享映射与私有写时复制语义不同，不能把所有 fork 后映射都概括为彼此隔离。〔S5〕

代码示例

```
fork(); // 父子逻辑上各自拥有地址空间，物理只读映射可暂时共享
mark_shared_pages_read_only_and_cow();
if (parent_or_child_writes(page))
{
    page_fault(); // 写入方触发内核异常
    private_copy = allocate_and_copy(page);
    remap_writer(private_copy);
}
// 未写页面继续共享；fork 后立即 exec 常省去大部分数据复制。
// 若大量页被写，仍要复制页面并建立页表，成本按写入比例发生。
```

7.4 I/O 完成模型

014 Reactor 和 Proactor 两种 I/O 模式有什么区别?

答案　Reactor 告诉程序“这个连接可以尝试读写”，真正的非阻塞 read、write 仍由程序执行；Proactor 则先提交 I/O 操作，完成后拿到字节数或错误等结果。判断依据是收到就绪通知还是操作完成通知，而不是接口有没有回调。epoll 提供的是就绪事件，所以常用来构建 Reactor。

单个事件循环容易理解，但耗时业务会拖住其他连接。可以用有界工作池执行业务，再把结果送回拥有连接的循环；也可以由主循环接入连接，多个子循环分别管理连接状态，交接时仍需同步、唤醒和背压。线程变多不会自动改变 I/O 模型。现代 Linux 的 io_uring 能对受支持的文件和 socket 操作提供完成结果，因此不能说 Linux 只有 Reactor，也不能保证异步方案在所有负载下更快。〔S5、S6〕

代码示例

```
// Reactor：就绪通知只说明“可以尝试”，应用仍执行非阻塞读写。
for (event : epoll_wait(epfd))
{
    if (event.readable)
        read_until_eagain(event.fd);
}
// Proactor：先提交 I/O 请求，完成时通知，回调拿到结果/字节数。
submit_async_read(fd, buffer, on_completion_result);
// 判断模式看通知表示 ready 还是 operation completed，而不是看 API 有无 callback。
// Linux epoll 是就绪通知；异步库可在上层包装成完成回调式接口。
```

015 epoll 和 io_uring 的 I/O 模型有什么不同?

答案　epoll 和 io_uring 给应用的消息不同。epoll 维护描述符的兴趣集，通知“现在可以尝试读写”，应用仍要调用 read 或 write 并检查结果。io_uring 则把读写等具体操作放进提交队列，再从完成队列取回字节数或错误；在缓冲区有效期等条件上，程序需要一直负责到操作真正结束。

io_uring 的共享队列和批量提交可以减少系统调用以及请求描述信息的搬运，但普通读写不会因此自动变成零拷贝。能提交哪些操作、如何执行以及是否受安全策略限制，都与目标内核、文件类型和配置有关。部署前要核实支持情况，并用真实连接数、报文大小和存储负载比较延迟与吞吐；接口更新并不等于每种场景都更快。〔S5〕

代码示例

```
epoll_add(fd, READABLE); // 内核记录兴趣；返回就绪后仍由应用调用 read
if (epoll_wait_reports(fd))
    n = ::read(fd, buffer, size);
submit_io_uring_sqe(READ, fd, buffer, size); // 提交队列发布读写操作请求
cqe = wait_io_uring_cqe(); // 完成队列带回操作结果，而不只是“可能可读”
// io_uring 可减少逐次系统调用/复制，但受内核版本、文件类型和安全限制影响。
// 先核对目标环境支持情况，再按真实负载基准测试。
```

7.5 进程、执行与启动链路

016 一个用户态程序从 execve 到 main 经历哪些步骤？

答案　execve 成功后并不是直接跳到 main。以 Linux 的 ELF 程序为例，内核先建立新的进程映像，映射主程序的可加载段，准备包含参数、环境变量及辅助向量的初始栈。ELF 文件记录了机器入口地址，内核按它和解释器信息决定先执行哪里，原来的程序代码不会继续运行。

如果有 PT_INTERP，先进入动态装载器，由它映射依赖的共享库并完成重定位，再转交主程序入口；没有解释器则直接进入程序入口。常见工具链把这个入口安排在运行时的 _start，由运行时初始化后调用 main。静态链接、PIE、懒绑定和 ABI 会影响细节。文件具备执行权限只过了第一关，架构不符、装载器缺失或依赖不兼容仍会让启动失败。〔N27、N25、N26、N252〕

代码示例

```
char* const args[] = {const_cast<char*>("worker"), nullptr};
::execve("/usr/bin/worker", args, environ); // 成功后不返回；内核从新程序入口启动
// 动态装载器映射依赖并运行初始化，之后才进入 main
```

017 操作系统从固件到第一个用户进程如何启动？

答案　操作系统启动是一条逐层交接控制权的链。常见 Linux 机器先由固件初始化必要硬件并选择启动项，再由 bootloader 读取内核、启动参数和可能需要的 initramfs，按平台启动协议把控制权交给内核。不同机器可能采用不同固件与装载方式，不能把某一套 BIOS 步骤当成全部平台的规定。

内核随后建立页表、内存分配、调度与设备驱动等基础能力，准备或切换到真正的根文件系统，再启动第一个用户态进程 PID 1。PID 1 继续组织设备、挂载和服务依赖。排障可以沿这条链找最后成功的一步：找不到根设备与某个业务服务端口未就绪不是同层问题。看到登录界面或引导成功也不代表数据库、网络和应用依赖都准备好了。〔N28〕

代码示例

```
enum class BootStage
{
    firmware,
    bootloader,
    kernel,
    initramfs,
    rootfs,
    init
};
BootStage next(BootStage stage)
{ /* 每层完成初始化后再交接给下一层 */
    return BootStage::kernel;
}
// 真实启动由固件、引导器、内核和 PID 1 分阶段完成
```

7.6 内核边界、调度与同步

018 用户态与内核态如何切换，系统调用为什么不同于普通函数调用？

答案　用户态和内核态是 CPU 的权限边界。普通程序不能任意访问内核地址或受保护的设备，必须请求内核代办。普通函数调用通常只改变当前执行位置和调用栈，权限不变；系统调用则通过架构规定的入口保存必要现场、进入内核、校验参数及访问权限，执行服务后再回到用户态。

例如 read 的库包装最终可能进入内核读取文件，但“库函数”与“系统调用”不是同义词：一些接口可能通过 vDSO 在用户态得到结果。硬件中断和缺页等同步异常也能进入内核。还要区分权限切换与任务切换：当前线程完成一次系统调用后可以直接继续，并不必然换成另一个线程；只有阻塞、抢占等需要调度时才发生相关任务切换。〔N29〕

代码示例

```
char buffer[100];
ssize_t count = ::read(fd, buffer, sizeof(buffer)); // 系统调用跨入内核并检查 fd 与缓冲区
if (count < 0)
    handle_errno(errno); // 普通函数调用不会切换到内核权限
```

019 并发与并行怎样区分，进程和线程上下文切换主要保存什么？

答案　并发是多个任务在一段时间内都取得进展，并行则要求同一时刻真有多个任务执行。单核通过轮流运行也能并发，但不能同时做两路 CPU 计算。任务上下文切换要保存旧任务的程序计数器、栈指针、寄存器和调度状态，再恢复新任务的现场；这些状态不是把整个堆复制一遍。

如果切换到不同地址空间，还涉及页表上下文及地址转换缓存管理。同一进程的线程共享地址空间，可以省去部分工作，但仍有各自的栈和寄存器。实际成本又受 TLB 标记、缓存局部性、CPU 迁移和工作集影响，不能绝对保证某次线程切换一定比某次进程切换快。评价设计时还要统计切换频率，以及每次切换后重新暖缓存的间接成本。〔N30、N31、N32〕

代码示例

```
struct CpuContext
{
    std::uintptr_t pc, sp;
    std::array<std::uintptr_t, 8> registers;
};
CpuContext saved = save_current_context();
restore_context(other_thread); // 恢复到暂停位置；不同地址空间还要切换页表
```

020 多线程程序调用 fork 后，为什么子进程容易卡在锁上？

答案　多线程进程 fork 后，子进程只剩调用 fork 的那一个线程，但它拿到的内存快照里可能有其他线程持有的锁。假设父进程线程甲正持有分配器锁，线程乙 fork，子进程只有乙，锁却仍显示已占用。乙再调用需要这把锁的库函数就可能永远等待，因为能够解锁的甲在子进程里根本不存在。

POSIX 因而要求多线程 fork 后到 exec 前的子路径，只调用异步信号安全函数。printf、复杂内存分配和普通库调用都不能随意放进去。pthread_atfork 能协调自己掌握的锁，却很难覆盖所有第三方库状态。启动新程序时保持子路径极小，或采用满足需求的 posix_spawn；父进程稍后解锁不会改好子进程里那个独立的锁副本。〔N33〕

代码示例

```
pid_t child = ::fork();
if (child == 0)
{
    ::execve(path, argv, envp);
    ::write(STDERR_FILENO, "exec failed\n", 12);
    ::_exit(127);
}
// fork 后子进程只有调用线程；失败路径避免使用可能持锁的复杂库函数
```

021 信号、信号量和互斥量分别表达什么，为什么不能混用？

答案　信号是异步事件通知，例如退出请求或子进程状态变化；普通信号可能合并，不能当成精确计数队列。信号量保存可用资源数量，等待时扣减、发布时增加，适合管理多个空闲槽位。互斥量保护临界区和共享不变量，通常由成功加锁的线程解锁。初值为一的信号量虽然能限制同时进入人数，却不自动具备互斥量的所有者等语义。

信号的处理方式通常是进程级属性，屏蔽字又有线程维度；同步硬件故障常送给触发线程，但默认致命动作作用于整个进程。SIGKILL、SIGSTOP 不可捕获或忽略。处理函数不能随意加锁、分配或调用非异步信号安全库，通常只设置安全标志或写自管道，把复杂清理留给正常执行流。捕获未知 SIGSEGV 后返回并不等于修复了内存错误。〔N34、N35〕

代码示例

```
std::counting_semaphore<3> slots{3}; // 计数资源表示最多三个空槽
std::mutex queue_mutex;
slots.acquire();
{
    std::lock_guard lock(queue_mutex);
    queue.push(task);
} // mutex 保护临界区
slots.release(); // 信号处理函数只设置 sig_atomic_t 标志，不在其中操作队列
```

022 常见 CPU 调度策略怎样取舍，Linux 是否一直使用同一种公平调度器？

答案　调度策略是在吞吐、响应、公平与切换成本之间取舍。先来先服务简单，却可能让短任务排在长任务后；短作业优先降低平均等待，但需要估计长度；时间片轮转便于交互任务尽快得到机会；优先级和反馈队列可以照顾紧急任务，也可能使低优先任务长期饥饿。策略好坏要放到实际工作负载里看。

Linux 有普通、FIFO、RR、Deadline 等类别，公平调度的内部实现也在演进：长期使用过 CFS，较新内核逐步转向 EEVDF，不能把某版红黑树实现当永久规则。FIFO、RR 的 sched_priority 越大越优先，普通任务的 nice 越小通常权重越高，nice 并不会将它升级为实时任务。相同优先级 FIFO 不靠普通轮转时间片分配机会，RR 则增加时间片；实时预算、权限、亲和性及带宽配置也会影响结果。〔N31、N36〕

代码示例

```
enum class Policy
{
    fair,
    fifo,
    round_robin
};
// SCHED_FIFO 同优先级任务不会因普通时间片自动轮转；SCHED_RR 会轮转
sched_param param{};
param.sched_priority = priority;
pthread_setschedparam(thread.native_handle(), SCHED_RR, &param); // 需检查权限与返回码
```

023 自旋、阻塞、悲观和乐观并发控制应怎样选择？

答案　自旋与阻塞回答“等锁时做什么”：自旋持续检查，避免睡眠唤醒，但消耗 CPU；阻塞把等待者挂起，适合等待较久的场景。若持锁者被调度走，其他线程越自旋越难得到收益，因此自旋适合很短且持有者能继续运行的临界区，并不适合包住任意耗时操作。

悲观与乐观回答“何时防冲突”：悲观先取得锁再读改数据，乐观先计算，提交时用版本号或 CAS 核对是否被别人修改，冲突就重试。乐观仍需原子校验，副作用要能避免重复；冲突频繁时反而浪费工作。读写锁、递归锁也有各自语义，不能只按名字替换。Linux 内核普通自旋锁在 PREEMPT_RT 下还有实现差异，应区分用户态锁和具体内核配置。〔N37、N38〕

代码示例

```
std::atomic<int> balance{100};
int old = balance.load();
while (old >= 20 && !balance.compare_exchange_weak(old, old - 20))
{ /* 冲突后重试并重新检查 */
}
// 只有 CAS 成功才发布扣款事件；不可撤销操作不要放在重试循环里
```

024 活锁与饥饿为什么可能在没有死锁时出现？

答案　没有死锁并不代表每个任务都能完成。死锁是参与者相互等候，谁也不能前进；活锁是大家一直重试、撤退或礼让，看起来很忙却没有完成工作；饥饿则是别人持续前进，但某个任务长期拿不到 CPU 或锁。比如两个人在狭道里同步向同一侧让路，反复碰面，就是直观的活锁。

处理方法要针对原因。死锁可用统一锁序减少环路；活锁可用明确仲裁、有界重试和带随机性的退避打破同步节奏；饥饿可用公平排队、优先级老化或资源配额改善。随机退避只能降低碰撞概率，不是所有任务必然完成的数学保证。因此既要检查“有没有锁环”，也要观察任务等待时间和实际完成数量，确认系统确实有进展。〔N31、N38〕

代码示例

```
std::unique_lock<std::mutex> first(mutex_a, std::defer_lock), second(mutex_b, std::defer_lock);
if (std::try_lock(first, second) == -1)
    update_both(); // 失败就释放已取得的锁
// std::scoped_lock 可按安全顺序同时锁定；有限重试和公平队列可避免活锁/饥饿
```

025 银行家算法怎样判断资源分配安全，不安全是否等于已经死锁？

答案　银行家算法像谨慎放贷：先知道每个任务最多还要多少资源，再判断借出后能否保证大家最终做完。它维护可用量 Available、已分配量 Allocation，以及剩余需求 Need=Max−Allocation。申请不能超过自身剩余需求，也不能超过当前可用量；通过这些检查后先做一份试分配，不马上改变真实状态。

安全检查在副本上寻找一个完成顺序：只要某任务的剩余需求不大于当前工作资源，就假设它做完并归还原有分配，继续寻找下一个。所有任务都能纳入顺序才批准，否则回滚并等待。不安全表示无法保证声明的最坏需求都能满足，不等于此刻已经死锁。算法依赖可事先声明的最大需求和可归还资源，不能直接包办任意业务锁问题。〔N39〕

代码示例

```
std::array<int, 2> available{1, 0};
std::array<std::array<int, 2>, 2> need{{{1, 0}}, {{0, 1}}};
// 每轮找 need[i] <= work 的进程，假定其完成并把 allocation[i] 加回 work
bool safe = can_finish_all(available, allocation, need); // 不安全表示找不到安全序列，不等于已死锁
```

7.7 地址转换与内存回收

026 页表、多级页表、分段和 TLB 怎样参与地址转换？

答案　分页把虚拟地址拆成虚页号和页内偏移：用页号查映射与权限，再把物理页框与原偏移拼成物理地址。多级页表逐层索引，只为实际使用的地址区间建立分支，节省稀疏地址空间的表内存。TLB 缓存近期转换，未命中时需要查页表，但页面仍可能在 RAM 中，所以 TLB 未命中不等于缺页，更不等于必须读磁盘。

分段按代码、数据等逻辑区域记录基址、界限和权限，段长可变，容易有外部碎片；固定页便于分配，但页尾会有内部浪费，有些体系还把段再分页。分页不要求整个进程占连续物理内存，可高阶分配、大页和设备缓冲仍可能缺少连续块。大页扩大 TLB 覆盖、减少表项，也可能加大浪费和复制、规整成本；compound 页也不必然正使用硬件大页映射。具体页大小和级数依架构配置。〔N32、I17、I19〕

代码示例

```
constexpr std::size_t page_size = 4096;
std::size_t virtual_address = 2 * page_size + 7;
auto page_number = virtual_address / page_size, offset = virtual_address % page_size;
auto physical_address = page_table[page_number] * page_size + offset; // TLB 缓存页号到页框的映射
```

027 内存紧张时怎样回收页面，页面置换与 OOM 有什么关系？

答案　内存紧张时，内核可以丢弃可重建的干净文件缓存，对脏页安排回写，把符合条件的匿名页换出。Zone 水位等条件可唤醒 kswapd 后台回收，压力大时允许睡眠的分配者还可能直接回收，抬高请求延迟。最终可能分配失败或触发受约束的 OOM 处理，不能只根据 free 数字或认定总杀 RSS 最大进程来解释。

经典置换有 FIFO、LRU、Clock、LFU 和需要预知未来的 OPT。Clock 用访问位近似近期使用；LFU 可能留住旧热点；FIFO 甚至可能增加页框反而增加缺页，即 Belady 异常。实际内核用近似和多代管理等机制，不等于精确 LRU。swappiness 调整匿名页交换与文件页回收的相对倾向，不是到某个百分比才换出，零也非绝不使用 Swap。

若活跃工作集持续大于可用内存，页面刚换出又要换入，便会发生抖动。应结合 cgroup 限额、可回收量与工作集，减少保留数据、限制并发或扩容，而不是反复重启。分配标志还决定能否睡眠、重试和回收，不能认为所有分配失败都走同一条 OOM 路径。〔N40、I18、I19〕

代码示例

```
std::array<bool, 3> referenced{true, false, true};
std::size_t hand = 0;
while (referenced[hand])
{
    referenced[hand] = false;
    hand = (hand + 1) % referenced.size();
}
evict_page(hand); // Clock 给访问位为 1 的页一次再访问机会
```

7.8 文件系统、缓存与持久化

028 文件描述符与 inode 的关系是什么，删除文件为何可能不释放空间？

答案　文件描述符 fd 是进程内的整数索引，找到内核的打开文件描述；后者保存偏移、状态标志，并关联 inode。dup 或 fork 得到的 fd 可共享同一打开文件描述，所以也共享偏移。反复独立 open 同一文件则通常得到不同打开文件描述，虽然最终指向同一个 inode，读取位置却可以互不影响。

unlink 删除的是名字，不是强行撤销所有打开引用。没有硬链接且打开等引用都释放后，数据才有最终回收条件，因此删除日志后进程仍写它，磁盘占用可能不降。可查 /proc/PID/fd 或 lsof 的已删除文件，让程序正确关闭或轮转。VFS 用 superblock、inode、dentry、file 等对象统一不同文件系统；dentry 缓存路径查找及不存在结果，不是文件内容，也不是磁盘目录项本身。设备文件还会把操作转给驱动，并非都可像磁盘文件任意寻址。〔N41、N42、I28〕

代码示例

```
int fd = ::open("log.txt", O_RDWR);
int duplicate = ::dup(fd); // dup 与 fd 共享同一 open-file-description 和偏移
int independent = ::open("log.txt", O_RDONLY); // 另一次 open 有独立偏移
::unlink("log.txt"); // 路径删除后，已打开的描述符仍可访问文件
```

029 Page Cache 如何支持 read 与 mmap，mmap 为什么不保证一定更快？

答案　Page Cache 是内核保存文件页的缓存。普通缓冲 read 命中时从缓存复制到用户缓冲，未命中则先向存储读取。文件 mmap 让进程地址映射到相关文件页，访问时通过缺页取得映射，可能减少显式 read 和部分复制，但仍有缺页、页表、同步和存储成本；它不会把整个文件立即装入 RAM。

顺序读取能利用预读，随机访问或一次大扫描也可能读进无用页并挤出热点。posix_fadvise 等接口可提示访问方式，最终仍需看工作集和命中率，不能按文件达到多少 GB 就断定缓存无用。mmap 共享写要同步，文件截断后访问可能 SIGBUS。直接 I/O 尽量绕过文件页缓存，但有依文件系统和内核而定的对齐约束；O_DIRECT 与同步异步是不同维度，本身也不等于 fsync 式持久化。〔N43、N44、I27、N41〕

代码示例

```
char buffer[4096];
ssize_t n = ::read(fd, buffer, sizeof(buffer)); // 从 page cache 复制到用户缓冲区
void* mapped = ::mmap(nullptr, length, PROT_READ, MAP_PRIVATE, fd, 0); // 首次访问可能缺页
// mmap 少一次显式复制不代表任意负载下都更快；应测吞吐和尾延迟
```

030 write 成功与 fsync 成功的持久化保证有何差别？

答案　write 成功通常只表示相应字节已被内核接收，不表示已经进入可承受掉电的存储。调用还可能短写，后续操作也可能报告延迟错误。fflush 主要把用户态库缓冲交给内核，fsync 则请求同步文件数据及必要元信息；fdatasync 可以减少与重新取回数据无关的元信息同步，几个完成点不能混为一谈。

如果用“写临时文件，再 rename 覆盖正式文件”发布配置，仅同步文件内容还不够，目录项的变更也要按协议持久化，通常需同步包含目录。每步都要检查错误，尤其不能 fsync 失败仍宣布保存成功。最终保证受文件系统、挂载选项和设备缓存是否诚实执行刷新影响；数据库提交还取决于日志刷盘策略，网络收到字节的确认更不能替代业务或持久化确认。〔N45〕

代码示例

```
write_all(temp_fd, data, size); // 循环处理短写和 EINTR
if (::fsync(temp_fd) == 0 && ::rename(temp_path, final_path) == 0)
{
    ::fsync(directory_fd); // 同步目录项，才能确认替换操作的持久化
}
```

7.9 中断与 I/O 语义

031 硬件中断、同步异常、软中断和工作队列各适合承担什么工作？

答案　硬件中断是设备异步报告事件，例如网卡收到数据；普通 IRQ 一般可屏蔽，NMI 不受普通屏蔽控制，常用于机器检查等紧急场景，但不表示用户进程必然崩溃。CPU 进入内核后，硬中断处理应尽量短，确认来源并完成必须的紧急动作。系统调用、缺页则是由当前执行引起的入口，属于另一类同步事件。

不能睡眠的硬中断上下文可把后续工作交给软中断或工作队列。软中断有时在中断返回等路径执行，压力大时也可能交给 ksoftirqd；即使由该线程执行，也不能按普通可睡眠任务随意写。工作队列由工作线程运行，可承载可能阻塞的处理。线程化中断和 PREEMPT_RT 会改变部分上下文细节。排查软中断压力，应观察 /proc/softirqs 和 /proc/interrupts 的时间增量、CPU 分布与网卡负载，累计值本身不能表示耗时。〔N46、N47、N38、I57、I60〕

代码示例

```
enum class HandlerContext
{
    hard_irq,
    soft_irq,
    workqueue
};
void hard_irq()
{
    acknowledge_device();
    schedule_softirq();
} // 快速确认，不睡眠
void workqueue()
{
    perform_slow_io();
} // 可睡眠的慢操作放到工作队列
```

032 阻塞、非阻塞、同步与异步 I/O 怎样区分，就绪后为什么仍要检查返回值？

答案　阻塞与非阻塞描述调用遇到暂时无法完成时怎么表现：阻塞 read 可能等待，非阻塞 read 通常返回 EAGAIN。同步与异步则描述操作结果由谁取得、如何交付。经典模型中，阻塞、非阻塞、多路复用和信号驱动最终仍由应用发起 recv/read 取数据；异步 I/O 是先提交操作，完成通知带回结果。就绪提醒和完成通知不能混淆。

epoll 报可读只说明现在值得尝试，不保证数据仍在，也不保证一次读完一条完整消息：另一线程可能先取走，结果可能是 EOF，也可能是错误。事件循环通常用非阻塞 fd，根据真实返回值累积和解析字节；read 为零代表流结束，EINTR 可按条件重试，EAGAIN 表示当前读空，其他错误须处理。多路复用改善等待方式，不自动提供消息边界。〔N48、N49〕

代码示例

```
ssize_t n = ::recv(socket_fd, buffer, sizeof(buffer), 0);
if (n > 0)
    input.append(buffer, static_cast<std::size_t>(n));
else if (n == 0)
    close_peer();
else if (errno == EAGAIN || errno == EINTR)
    retry_later();
// epoll 报就绪不代表一次 recv 能读到完整业务消息
```

7.10 Linux 内存管理机制

033 NUMA 为何会出现总内存充足却访问慢或局部分配紧张？

答案　NUMA 把处理器和内存分成多个局部节点，本地访问通常比远端访问延迟更低、带宽更好。因此全机显示空闲内存充足，当前节点或 cpuset 允许使用的节点仍可能紧张。默认策略往往倾向线程所在节点；物理页常在首次触页时分配，所以由哪个线程初始化大数组、线程后来跑在哪个 CPU，都会影响局部性。

可按负载选本地优先、绑定或交错策略，并用 numastat、numa_maps 观察。cpuset 会限制允许节点，CPU 亲和性也影响本地策略的参照位置。把线程绑到某 CPU 不会自动搬迁已经存在的页面；改了策略通常只影响后续分配，页面迁移和自动 NUMA 平衡还受配置、可移动性及成本限制。因此测量要同时观察远端访问、带宽和延迟，避免把策略配置误当成已实现的数据迁移。〔I15、I18〕

代码示例

```
std::vector<int> values(count);
std::thread worker(
    [&]
    {
        for (int& value : values)
            value = initialize();
    }); // 首次触页常按本地策略分配
worker.join();
// NUMA 亲和性和内存放置需一起测量；绑核不会自动迁移已分配页面
```

034 kmalloc、vmalloc 和 alloc_pages 怎样选，GFP 标志约束什么？

答案　kmalloc、alloc_pages 和 vmalloc 面向不同分配需求。kmalloc 给较小内核对象，虚拟和物理地址都连续；alloc_pages 按 2 的幂次页阶申请连续物理页并返回页描述符。vmalloc 给连续的内核虚拟地址，但底层物理页可分散，建立映射和 TLB 管理会有成本。kvmalloc 可先尝试 kmalloc 再回退 vmalloc，释放时必须选匹配接口。

GFP 标志会约束可用 Zone、是否允许回收和睡眠。GFP_KERNEL 可能直接回收并睡眠，不能在禁止睡眠的中断上下文使用；GFP_NOWAIT 不睡眠且可能失败，GFP_ATOMIC 可用部分保留资源，但同样不能保证成功，应限制使用。设备 DMA 要走 DMA API 处理总线地址、映射和同步，普通内核指针或物理地址不能直接当设备可访问地址。分配失败必须按调用上下文设计回退。〔I17〕

代码示例

```
void* object = kmalloc(object_size, GFP_KERNEL); // 进程上下文允许睡眠，但分配仍可能失败
if (object != nullptr)
    kfree(object); // 与 kmalloc 配对释放
// 中断上下文不能睡眠；DMA 地址应通过 DMA API 获取
```

035 伙伴系统、slab 与每 CPU 页缓存怎样配合，空闲多为何高阶分配仍失败？

答案　伙伴系统管理整页空闲块，按 2 的幂次阶组织；申请小阶时可把大块拆分，释放后只有地址关系匹配且同阶空闲的伙伴才可合并。每 CPU 页缓存把常见页分配与释放暂存于本地，再批量和 Zone 空闲区交换以减少锁竞争；它缓存的是页管理状态，不是 CPU 硬件缓存。slab 类分配器再把页切成适合内核对象的槽位，和整页分阶属于不同层次。

所以空闲页总量够，不保证能拿到指定高阶连续块：空闲页可能分散在许多不能合并的位置。内存规整可以迁移可移动页面来聚合空间，但固定或不可迁移页面会阻断，过程也有成本，最终不保证成功。排查时要看目标 Zone、分配阶数、迁移条件及失败路径，不能仅依据全局空闲总量断定分配一定成功。〔I18、I19、I31、I33〕

代码示例

```
std::array<bool, 12> free_page{}; // 标记每页是否空闲
bool has_order3_block = find_aligned_run(free_page, 8); // 高阶分配要求连续且对齐
// 空闲页总数足够也可能因碎片而失败；规整会迁移可移动页面
```

036 Linux 为什么需要页面反向映射，rmap 与页表的方向有何不同？

答案　页表按虚拟地址查物理页，方向是“虚拟页到物理页”；反向映射 rmap 则让内核从一个物理页或 folio 找到哪些 VMA 和页表项映射它。共享文件页和 fork 后的写时复制页都可能映射到多个进程，因此内核回收、老化或迁移页面时，要检查并更新所有相关映射，而不能只看当前进程。

文件页可沿 address_space 的索引定位映射 VMA，匿名页则通过 anon_vma 等关系组织映射；不是为每个匿名页复制一份完整 VMA。映射计数表示映射关系，引用计数还可能包括缓存、临时持有等引用，两者都不表示访问频次，也不一定等于进程数。遍历和解除映射要按相应锁及页表同步规则执行，否则并发解除后仍使用映射可能造成错误。〔I16〕

代码示例

```
std::unordered_multimap<Page*, Mapping> reverse_map; // 物理页反查所有虚拟映射
for (const Mapping& mapping : reverse_map.equal_range(page))
    unmap(mapping);
if (page->map_count() == 0 && page->ref_count() == 0)
    reclaim(page); // 解除映射后才可回收
```

037 Linux 怎样用 Node、Zone、PFN 和 struct page 组织物理内存？

答案　Node 表示 NUMA 内存局部性，节点内部按寻址与分配约束分成 Zone；PFN 是物理页框号，struct page 等描述符记录页的管理状态，并不是页面数据本身。物理地址可能有空洞；FLATMEM、SPARSEMEM 等模型决定内核如何组织页描述信息，稀疏模型也方便描述热插拔内存。DMA、DMA32 等 Zone 的边界由架构和设备寻址限制决定，不能把 DMA 固定说成所有设备只能访问前 16 MB。

HIGHMEM 主要出现在部分 32 位配置，指没有永久内核映射的高端内存；现代 64 位机器不能机械套用。热添加的内存要上线后才能供分配，热移除通常先下线并迁移页面。MOVABLE 区提高可迁移性，固定或不可移动页面仍会阻挡下线。具体能力取决于内核配置、体系结构和设备，查看节点、Zone 与 PFN 时要把“页描述符、物理页内容、可分配状态”区分开。〔I18、I30、I32〕

代码示例

```
auto pfn = physical_address / PAGE_SIZE;
if (pfn_valid(pfn))
{
    struct page* metadata = pfn_to_page(pfn);
    inspect(metadata->zone, metadata->node);
}
// struct page 是元数据；页面内容位于对应物理页，不在描述符对象里
```

7.11 文件与设备数据路径

038 磁盘 I/O 调度怎样在寻道、吞吐和公平之间取舍？

答案　机械硬盘的寻道和旋转等待会显著影响随机 I/O。FCFS 按先来顺序服务，简单但可能来回寻道；SSTF 选当前位置最近请求，通常减少移动，却可能让远处请求挨饿。SCAN 像电梯往返，LOOK 只走到当前方向最远请求，循环变体沿一个方向服务以改善等待均匀性；它们的模型主要解释机械位置开销，不能直接作为 SSD 的性能公式。

Linux 块层还会合并相邻请求、排队并交给设备，blk-mq 提供软件及硬件多队列。none、mq-deadline、BFQ 等策略是否可用受内核、驱动和设备影响，应在目标系统检查。调度可能改善公平或延迟，也可能牺牲吞吐；SSD、虚拟化存储和控制器自身都改变排序成本。应在真实读写比例下比较吞吐、IOPS、尾延迟和不同任务的公平性，不要把 CFQ 或固定五种算法说成所有 Linux 的默认情况。〔I21〕

代码示例

```
std::vector<int> requests{10, 45, 80};
std::sort(requests.begin(),
          requests.end(),
          [&](int a, int b)
          {
              return std::abs(a - head) < std::abs(b - head);
          });
// SSTF缩短近处寻道但可能饿死远端；SSD还应实测尾延迟与公平性
```

039 文件内容怎样映射到磁盘块，连续、链式、索引和 extent 分配有何取舍？

答案　文件逻辑内容要映射到存储块，常见思路各有取舍。连续分配方便顺序和随机定位，但增长可能要搬家并产生外部碎片；链式分配易扩展，却要沿链找后续块，FAT 把链集中到表中仍有元数据成本。索引分配用直接或多级间接索引定位，支持随机访问但索引也占空间。extent 用起始块与长度概括一段连续空间，碎片少时映射紧凑，碎片多时要维护更多 extent。

文件系统还会用位图记录空闲块或 inode，并以块组等布局改善局部性。数据块耗尽和 inode 耗尽是两种独立问题：磁盘容量看起来尚有空间，也可能创建不了新文件。现代文件系统会采用 extent、树索引和延迟分配等实现，具体 inode 指针数、目录索引方式与块格式依文件系统而异，不能把一种 Unix 教材布局当成统一磁盘格式。〔I22、I36〕

代码示例

```
struct Extent
{
    std::uint64_t first_block;
    std::size_t length;
};
std::vector<Extent> extents{{100, 3}, {220, 1}}; // 连续块合并为extent
std::uint64_t physical = extents[0].first_block + 2; // 逻辑块2映射到物理块102
```

040 设备控制器、驱动与 MMIO/端口 I/O 分别承担什么职责？

答案　设备控制器是硬件暴露的命令、状态和数据接口，驱动按照设备协议操作它，并把功能接入内核框架；上层程序可能通过文件系统或设备文件使用。某些架构支持端口 I/O，用独立端口空间和 in/out 操作；MMIO 则把设备寄存器映射到 CPU 地址空间，驱动用 ioremap 建立合适映射，并通过 readl、writel 等访问器操作。

MMIO 寄存器不是普通 RAM，memcpy 或只加 volatile 都不能替代规定的访问宽度、顺序和总线规则；posted write 有时需读回确认送达。DMA 让设备搬运数据，但驱动仍须准备映射、描述符并完成缓存同步，地址要经 DMA API 转换，不能简单把 CPU 指针交给设备。设备可通过中断或轮询报告完成，DMA 也不表示 CPU 完全不参与。〔I23、I17〕

代码示例

```
void* buffer = dma_alloc_coherent(device, length, &dma_address, GFP_KERNEL);
write_descriptor(dma_address, length);
writel(START, device_register);
wait_for_completion(); // DMA映射/同步及释放必须遵循对应设备API
```

7.12 硬件执行与内核架构

041 CPU 如何从机器指令执行程序，流水线与乱序执行改变了什么？

答案　程序运行时，CPU 从程序计数器指向的位置取机器指令并译码，再读取寄存器或内存操作数、执行运算或访存，最后更新可观察的寄存器和内存状态。分支、调用、返回和异常会改变控制流；机器指令长度由指令集决定，不能从 CPU 位宽推断。C++ 一行源码可能变成多条指令，也可能被优化掉。

流水线让不同指令分别处在取指、译码、执行等阶段，超标量和乱序实现还可并行处理互不依赖的操作，但最终必须符合指令集规定的可观察结果。数据依赖、缓存未命中或分支预测错误会让流水线停顿或清空重做。因此不能把一行源码等同一条指令、一次时钟周期，也不能断言所有 CPU 都是固定四级流水线；需要用汇编、性能计数器和测量分析实际瓶颈。〔I42、I43〕

代码示例

```
int loaded = data[index];
int incremented = loaded + 1;
data[output] = incremented; // 数据依赖限制可观察结果的顺序
// CPU可乱序执行互不依赖的工作，但必须遵守ISA规定的内存语义
```

042 宏内核、微内核与混合内核怎样划分，主要取舍是什么？

答案　宏内核、微内核和混合内核的区别，重点是服务在哪个权限边界运行以及如何通信。Linux 属于模块化宏内核：调度、内存管理、文件系统和大量驱动运行在内核空间，内部调用直接，但内核模块故障可能破坏全系统。可装载驱动模块仍在内核权限内，不等于用户态隔离服务。

微内核尽量保留调度、地址空间和 IPC 等核心，把更多服务放在用户态；服务间隔离和替换更方便，但通信与系统设计成本要看实现。seL4 是一个微内核实例。Windows NT 常称混合架构，含内核、Executive、HAL 及用户态子系统，不能简化成“微内核外套一层代码”。架构标签本身不证明快慢或可靠性，要对照服务边界、故障隔离和真实工作负载。〔I58、I59〕

代码示例

```
enum class KernelLayout
{
    monolithic,
    microkernel,
    hybrid
};
KernelLayout choice = KernelLayout::microkernel; // 按性能、隔离边界与恢复成本选型
// 微内核常把驱动/服务移到进程间通信边界；具体取舍取决于实现
```

第 8 章 计算机网络

8.1 HTTP 与安全连接

001 HTTP GET 与 POST 有什么区别?

答案　GET 用来请求目标资源的表示，通常应当是安全且幂等的：安全表示调用者不要求改变资源状态，幂等表示同一请求重复执行后的预期效果相同。POST 则把内容交给目标资源按其语义处理，常用于创建或触发操作。把参数放 URL 只是常见设计，不是判断方法的标准；URL 也可能进浏览器历史、代理日志和引用来源。

缓存依据方法和响应头等规则决定，POST 并非天然不可缓存。反过来，GET 也不应被服务端偷偷设计成扣款等不可重复的写操作。HTTP 方法的安全和幂等语义与 TLS 加密无关，URL 与请求体都可能暴露于不当日志或页面脚本。GET 请求体没有通用定义的语义，库允许发送也不代表代理和服务端会按同一方式处理。〔N51〕

代码示例

```
request = HTTP_GET(resource_uri); // 获取表示；语义应安全且幂等，适合查询
request = HTTP_POST(collection_uri, payload); // 按目标资源语义处理内容，常用于创建/动作
// 不能仅凭“参数在 URL 还是 body”判定方法语义或安全性。
// GET 写操作若扣款，重试/预取会造成副作用，设计本身就不安全。
response_cache = cache_by_method_and_headers(request, response); // 按规则决定
// URL 可能进入浏览器历史和日志；敏感数据需正确认证/授权与传输保护。
```

002 HTTP/1.1 常见状态码如何分类?

答案　HTTP 状态码按首位分五类：1xx 是临时信息，2xx 表示请求已成功接收或处理，3xx 需要重定向或使用缓存，4xx 通常表示请求端问题，5xx 表示服务器或网关处理失败。常见例子是 200、201、204；301/302；400、401、403、404、429；500、502、503、504。502 是网关收到上游无效响应，504 是等待上游超时。

重定向可能影响方法：历史上不少客户端遇到 301/302 会把 POST 改为 GET；需要保留方法和请求体时应考虑 307/308。重试不能只看“不是 200”：429、503 可结合 Retry-After 和退避策略；一般 4xx 修正请求前重试无用。即使 5xx，也要考虑操作是否已经在上游完成，避免重复产生副作用。

代码示例

```
switch (response.status)
{
case 100:
case 103:
    interim_information_only();
    break; // 1xx 不是最终结果
case 200:
    success_with_representation();
    break;
case 201:
    resource_created();
    break;
case 204:
    success_without_body();
    break; // 2xx 成功类
case 301:
    follow_redirect_or_cache_by_rules();
    break; // 3xx
case 401:
    authenticate_request();
    break;
case 403:
    deny_authorized_identity();
    break;
case 404:
    resource_absent();
    break;
case 429:
    wait_retry_after_with_backoff();
    break;
case 503:
    retry_if_safe_and_retry_after_allows();
    break; // 5xx
}
// 4xx 通常先修正请求，不盲目重试；仍须考虑幂等性和截止时间
```

003 HTTP/HTTPS、TLS 和证书分别解决什么问题?

答案　HTTPS 是在 HTTP 通信外建立 TLS 安全通道。握手协商算法和会话密钥，并通过证书链把服务端公钥绑定到身份；客户端还要核验签发链、域名、有效期和用途。证书并不是“加密整段 HTTP 内容”，它支持身份认证；后续记录通常由高效的对称密钥保护机密性和完整性。TLS 不会自动保证应用授权或业务规则正确。

TLS 1.2 可用 Session ID 缓存或受保护的 Ticket 恢复会话；TLS 1.3 主要用 Ticket 得到的 PSK 恢复，但仍协商新连接参数和密钥。证书认证的临时 ECDHE/DHE 可让长期私钥泄露后仍保护过去会话；TLS 1.2 静态 RSA 密钥传输没有这种前向保密，TLS 1.3 已移除该方式。Finished 校验握手上下文；不能把 TLS 简化为“公钥加密、私钥解密”。OCSP Stapling 附带的状态响应也要校验签名与时效。

TLS 1.3 恢复时可能发送 0-RTT 早期数据，但它没有新一轮 DHE 提供的前向保密，也缺少跨连接的固有防重放保证。早期请求要能安全重放，并配合去重和服务器反重放策略；“业务幂等”单独不足以覆盖全部副作用。TLS 1.2 False Start 也只是满足条件时的提前发送优化，并非用了 ECDHE 就自动启用。〔S7、I5、I4〕

代码示例

```
client = connect_tls(server_name);
cert = peer_certificate_chain();
verify_chain_to_trusted_CA(cert);
verify_hostname(cert, server_name);
verify_expiry_and_usage(cert); // 证书提供公钥/身份链，客户端仍须检查用途和域名
keys = tls_handshake_negotiate_and_authenticate(cert); // 非对称机制用于认证/协商
secure_channel = derive_symmetric_keys(keys); // 后续记录层保护机密性与完整性
send_http_over(secure_channel); // TLS 不替代应用自身的权限检查
```

004 TCP Keep-Alive 与应用层心跳有什么区别?

答案　TCP Keep-Alive 是操作系统对空闲连接发送探测报文，尝试发现对端或路径已经失联；探测周期常由系统参数决定，默认可能很长。应用层心跳是协议明确发送的消息，可以检查会话过期、应用进程卡死或更具体的业务状态，周期可按服务目标设计。TCP 层活着不一定意味着应用能处理请求，应用有响应也要说明是哪一层在响应。

HTTP 持久连接表示多次请求复用连接，SO_KEEPALIVE 才控制 TCP 保活探测，两者不是一个开关。网线拔掉或机器断电不一定送出 FIN；空闲又无探测时，连接可能长期留存，有未确认数据则重传、用户超时会推动失败。Linux socket 通常需显式启用保活并按平台配置探测。任何探测都只是存活线索，调用本身还要有截止时间、退避和幂等保护；关闭 socket 也可能受共享引用、未读数据和 SO_LINGER 影响。〔N70、N74〕

代码示例

```
tcp_keepalive_enable(socket); // 空闲探测由 OS 参数控制，默认间隔可能很长
schedule_application_heartbeat(session, sla_period); // 检查进程卡死/会话失效
if (heartbeat_deadline_missed())
    mark_suspect_then_reconnect_with_backoff();
set_request_deadline(deadline); // 连接探测成功不代表某次业务调用会完成
retry_only_if_operation_is_idempotent_or_has_request_id();
// 代理、线程阻塞和网络分区都可能影响探测；两者只是存活线索
// 应按 SLA 配探测间隔/重试，而不是依赖操作系统默认 keepalive
```

8.2 TCP、UDP 与传输控制

005 TCP 三次握手和四次挥手如何进行？为什么需要 TIME_WAIT?

答案　TCP 建连时客户端发 SYN，服务端用 SYN+ACK 确认并给出自己的初始序号，客户端 ACK 后双方进入已建立状态。三次握手让双方确认双向可通信并同步序号，也能过滤一些旧的重复请求。关闭时两个方向可以分别结束，所以常见 FIN、ACK、FIN、ACK；报文可能因合并而少于四段，同时关闭时双方也可能进入 TIME_WAIT。

主动关闭方进入 TIME_WAIT，保留内核状态一段时间，好让最后 ACK 可以重发并隔离旧连接报文；通常应用 fd 已关闭，这不是应用线程还占着连接。优先复用长连接并查明谁在主动关闭。tcp_fin_timeout 管的是孤儿 FIN_WAIT_2，不是 TIME_WAIT。tcp_tw_reuse 的语义须按目标内核文档检查；tcp_tw_recycle 已在 Linux 4.12 删除。过早复用或强制 RST 会破坏旧报文隔离和可靠终止。〔N66、N69、N70〕

代码示例

```
client.send(SYN, seq = c);
server.reply(SYN | ACK, ack = c + 1, seq = s);
client.send(ACK, ack = s + 1); // 确认双方初始序号与双向可达后建立连接
// 关闭方向独立：FIN -> ACK -> FIN -> ACK，常见四段挥手
active_closer.send(FIN);
peer.ack_then_later_send(FIN);
active_closer.send(ACK);
active_closer.enter(TIME_WAIT); // 主动关闭方等待，必要时重传最后 ACK
wait_2MSL_then_release_tuple(); // 让旧报文消退，避免新连接误收旧段
// TIME_WAIT 占用有成本，但也保护端口复用和连接正确性
```

006 TCP 与 UDP 有什么区别?

答案　TCP 提供有序、可靠的字节流：它会重传、控制发送速度并处理拥塞，但不保留应用每次 send 的消息边界。UDP 发送独立数据报，能保留报文边界，却不承诺送达、顺序或去重，也没有内建拥塞控制。DNS 常以 UDP 查询，条件不满足可转 TCP；媒体常能容忍少量丢包，仍要依据应用需求选择。

UDP 边界也不意味着接收缓冲无限，缓冲不足时一个数据报可能被截断。QUIC 等可以在 UDP 之上实现可靠传输：它按流偏移重组数据，丢失内容用新包号重传，ACK 区间帮助发现缺口，并区分流级、连接级流控和连接拥塞控制。但裸 UDP 不会自动变可靠，DNS、媒体和 HTTP 所用传输也因协议版本及部署而异。〔I9、S7〕

代码示例

```
tcp.send(bytes); // TCP 不保留 send/recv 边界，提供有序可靠字节流
receive_with_length_prefix_delimiter_or_fixed_frame();
// TCP 用 ACK/重传、流量控制和拥塞控制；可靠性会带来重传/队头等待。
udp.send_datagram(bytes); // 保留数据报边界，但协议不承诺交付/顺序/去重
if (dns_query || loss_tolerant_realtime_media)
    choose_udp_when_app_semantics_fit();
// DNS 需要时也会转 TCP；UDP 应用需自行考虑丢包与拥塞，不能假设可靠。
```

007 TCP 如何实现可靠传输?

答案　TCP 用序列号标记字节位置、ACK 确认接收进度、校验和检测传输损坏；缺失数据会靠重传定时器或重复 ACK 等机制恢复。接收窗口限制未被接收端接受的数据量，拥塞控制根据网络反馈调节发送量。它向应用交付有序字节流，不交付“每个 send 一条消息”，所以一次 send 对应几次 recv 没有保证。

ACK 号表示下一个期望字节，累计确认只确认连续收到的前缀；后续乱序段可暂存，出现缺口不代表其他数据没到。RTO 按平滑 RTT 和变化估计并会退避，不是固定秒数。协商 SACK 后，接收方还能报告不连续已收区间，减少多个缺口下的无效重传；D-SACK 标示重复收到的区间，可帮助分析重复接收，但单凭它不能确定唯一丢包原因。连接建立时协商的 MSS、接收缓存和路径变化也会影响吞吐。〔N66、I6、I37、I38〕

代码示例

```
#include <algorithm>
segment.seq = first_byte_sequence;
segment.checksum = checksum(header, payload);
receiver.accept_in_order_bytes();
receiver.send_cumulative_ack(next_expected_seq);
if (timer_expired || duplicate_acks_indicate_gap)
    retransmit_missing_range();
limit_in_flight_by(std::min(receiver_window_rwnd, congestion_window_cwnd));
// 序列号/累计 ACK 支持有序交付；校验和发现传输损坏
// rwnd 保护接收缓存，cwnd 保护网络路径；二者较小者限制在途字节
// TCP 向应用交付字节流，无消息边界；MSS、缓存和路径也影响吞吐
```

008 TCP 流量控制与拥塞控制有什么区别?

答案　流量控制保护接收端：接收方通告 rwnd，表示接收缓冲还容得下多少未读数据。拥塞控制保护网络路径：发送方依据丢包、时延或 ECN 等反馈调整 cwnd。实际允许在途的数据通常受 rwnd 和 cwnd 较小者限制；rwnd 小可能是应用没及时读，cwnd 小可能是网络路径拥塞，诊断不能只看一个窗口。

接收窗口变成零时，发送方仍需按 TCP 规则探测窗口何时重新开放。以 Reno 模型来说，慢启动近似每 RTT 倍增 cwnd，拥塞避免近似线性增长，超时与重复 ACK 的恢复路径不同；CUBIC、BBR 等算法不都遵循相同更新规则。socket 缓冲区又是另一约束，自动调节、应用读写速度、ACK 时序和路径反馈都会影响吞吐，应结合目标主机正在使用的拥塞算法判断。〔I35、N70〕

代码示例

```
#include <algorithm>
rwnd = receiver_advertised_free_buffer(); // 接收端流控，保护消费/缓存能力
cwnd = sender_congestion_controller_estimate(); // 网络拥塞控制，保护传输路径
in_flight_limit = std::min(rwnd, cwnd);
if (rwnd == 0)
    periodically_probe_window_update(); // 对端读出数据后可重新开放
if (cwnd_is_small_or_loss_detected())
    reduce_send_rate_and_recover();
// rwnd 小先查应用读取/缓冲；cwnd 小查丢包、时延、ECN 等路径反馈
// 窗口探测不是立即发送任意数据，发送方仍须遵守协议窗口规则
```

009 TCP 滑动窗口是什么？窗口会向左移动吗?

答案　发送窗口表示可以发送但尚未确认的序列号范围；接收窗口表示接收方可容纳的数据空间。ACK 推进已确认的左边界，新的窗口通告决定右边界，所以窗口位置前移，宽度也可收缩、扩大或暂时不变。窗口缩小不是撤销已经发送的数据，理解时要把窗口范围和窗口大小分开。窗口更新也可以独立于新数据到达。

SYN 阶段可协商窗口缩放，以表示更大的接收窗口；零窗口时 persist 探测用于避免窗口更新丢失后双方永久等待，并不允许随意发送正文。带宽时延积约为带宽乘 RTT，可估计跑满路径需要多少在途字节，但不是 socket 缓冲的硬上限。Linux 可自动调节部分缓冲，手工设置可能改变调节空间。调参须同时看 rwnd、cwnd、在途量、丢包和应用处理速度，单纯调大一个 sysctl 未必改善吞吐。〔I34、N69、N70〕

代码示例

```
send_window = [oldest_unacked_seq, right_edge_allowed_by_peer_and_cwnd);
if (ack_covers_contiguous_bytes) move_left_edge_forward(ack.next_expected_seq);
if (peer_updates_receive_window) recompute_right_edge(peer.rwnd);
// ACK 可累计确认多个连续字节，窗口更新也可不伴随新数据。
if (visible_width_shrinks) do_not_retract_already_sent_bytes();
// 区分窗口位置和窗口宽度；序列号环绕/重传依 TCP 规则处理。
```

010 什么是 MTU？IP 包超过路径 MTU 会怎样?

答案　MTU 是单条链路能承载的最大网络层包尺寸；端到端路径 MTU 受路径上最小链路限制。IPv4 在允许时可由路由器分片，设了 DF 后则依赖 ICMP 错误辅助发现；IPv6 路由器不分片，源端根据 ICMPv6 Packet Too Big 等信息调整。TCP MSS 限制 TCP 负载长度，需留出 IP、TCP 及选项头部。

两端通告的 MSS 分别限制各自发送方向，路径变化或隧道封装仍可能使实际可用大小更小。IP 分片需要目的端重组，少一片就无法交付原数据报，之后 TCP 才能恢复缺失字节。若防火墙错误丢弃 ICMP，路径 MTU 发现可能形成“连接建立但大包卡住”的黑洞，可结合较小报文、抓包和分组层探测定位。不要只看接口 MTU 就断定整条路径同样大小。〔N66、I11〕

代码示例

```
path_mtu = minimum_link_mtu_on_route(); // 端到端路径上的最小链路 MTU
mss = path_mtu - ip_header_bytes - tcp_header_bytes; // TCP 负载要扣除头部
if (ipv4 && fragmentation_allowed)
    router_may_fragment_packet();
if (ipv4_df || ipv6)
```

```
    source_uses_icmp_feedback_to_adjust_packet_size();
// IPv6 路由器不分片，需由源端依据 ICMPv6 做 PMTU 发现
if (icmp_blocked_or_misconfigured())
    detect_path_mtu_black_hole_and_retransmissions();
// TCP MSS 影响分段负载；隧道封装等还会进一步减小可用 MTU
```

011 什么是 TCP 粘包/拆包？如何解决？

答案　TCP 只保证按序交付连续字节，不记住 send 调用的边界。应用连续发送两条消息，接收端可能一次 recv 读到两条；发送一条长消息，也可能分多次 recv 才读全。这些现象常叫粘包和拆包，其实不是 TCP 把消息弄坏，而是协议没有自己规定消息边界。

应用应定义帧格式：固定长度、分隔符、长度前缀或自描述编码都可以。解析端必须缓存半包、循环拆出缓冲区里的多帧，并验证长度、字符集和格式。长度前缀要设最大值，避免恶意值导致无限分配；分隔符要考虑转义和跨缓冲区查找。发送端的 send 完成也不等于对端收到完整帧。

代码示例

```
buffer.append(socket.read_available()); // TCP 是字节流，一次 send 不对应一次 recv
while (buffer.size() >= header_bytes)
{
    length = decode_big_endian_length(buffer);
    if (length > MAX_FRAME || length > SIZE_MAX - header_bytes)
        close_as_protocol_error();
    if (buffer.size() < header_bytes + length)
        break; // 半包留待后续读取
    parse_and_validate_frame(buffer.subspan(header_bytes, length));
    buffer.erase_prefix(header_bytes + length); // 同一缓冲可能含多帧
}
// 定长、分隔符或长度前缀都能定界；校验格式并限制长度防内存耗尽
// 解析循环只消费完整帧，不能把残留半帧误判为错误
```

8.3 名称解析与服务端网络

012 浏览器输入 URL 到页面显示大致经历什么？

答案　浏览器先解析 URL 和安全策略，可能由缓存或 Service Worker 直接给出结果；否则通过系统解析器找地址，并建立 TCP/TLS 或 QUIC 连接。随后发 HTTP 请求、接收响应，解析 HTML 形成 DOM、解析 CSS 形成样式信息，再布局、绘制和合成；脚本执行还可能继续改 DOM 或请求图片等子资源。因此用户看到页面之前存在多个可并行或相互阻塞的阶段。

优化前先看缓存、DNS、连接、首字节和资源加载的实际耗时。减少重定向和无用请求常有帮助，文本可评估 gzip/Brotli，已压缩媒体重复压缩收益很小。Accept-Encoding 选择内容编码，缓存应按 Vary 区分变体。合并、内联可以少一次请求，却增大失效范围或首屏负担；HTTP/2 多路复用后不能照搬旧的合并规则。Accept 中的 q 表示偏好权重，不是图片压缩质量。〔N51、N53、S7〕

代码示例

```
url = parse_and_normalize(input_url);
if (service_worker_or_cache_hit(url))
    render_cached();
ip = resolve_dns_via_browser_os_or_recursive_cache(url.host); // 未命中才向上查询
connection = connect_tcp_tls_or_quic(ip, url);
response = send_http(connection, url);
dom = parse_html(response);
styles = parse_css_and_compute_style(dom);
layout(dom, styles);
paint_and_composite();
// 脚本可能在过程中执行并请求子资源；缓存、重定向和预加载会改变顺序。
// DNS/连接/资源耗时分别影响等待，不是单一固定串行流水线。
```

013 Linux listen(backlog) 参数控制什么？SYN 队列和 accept 队列有什么区别？

答案　在 Linux 上，listen(fd, backlog) 的 backlog 表示握手已完成、等待应用 accept 的连接队列容量。尚未完成三次握手的 SYN 请求属于另一队列，受 tcp_max_syn_backlog 等机制影响。backlog 请求值还会受 somaxconn 上限约束。其他操作系统的具体语义可能不同，排查时要在目标系统核对参数和队列计数。

内核可在 accept 前完成握手并缓存连接数据，所以“应用没 accept”不等于 TCP 尚未建立。队列溢出时客户端可能重试、超时或收到 RST，取决于 tcp_abort_on_overflow 等配置和具体时序。SYN cookies 通过校验 Cookie 减少半开连接状态压力，不是另建 Cookie 队列，也解决不了 accept 处理过慢或入口带宽不足。应将队列参数、应用 accept 速率和抓包一起分析。〔S5、N67、N69〕

代码示例

```
#include <algorithm>
listen(fd, requested_backlog); // Linux 主要限制已完成握手、等待 accept 的队列
syn_queue_capacity = tcp_max_syn_backlog; // 半连接请求属于另一队列/机制
actual_accept_limit = std::min(requested_backlog, somaxconn);
while (accept_queue_growing())
    measure_application_accept_rate_and_worker_latency();
// 队列满时客户端可能重试或失败；检查半连接压力、内核参数和 accept 速度
// tcp_max_syn_backlog/somaxconn 的实际行为依 Linux 版本和配置验证
// backlog 不能代替及时 accept、连接超时和容量规划
```

014 TCP 的 Nagle 算法和 Delayed ACK 分别做什么？为什么会增加请求响应延迟?

答案　Nagle 算法减少小 TCP 段：有未确认数据时，新的小写入通常先暂存，等 ACK 到来或数据凑够后再发送。Delayed ACK 则让接收端短暂等候，试着合并 ACK 或搭配反向数据一起确认。两者时序叠加时，小请求和小响应可能额外等待一个确认周期，因此交互式协议会感到延迟。

可在确认问题后测试 TCP_NODELAY，但批量合并也可能减少报文开销，需一起测吞吐和延迟。接收端避免过小通告窗口是缓解糊涂窗口综合征的相关措施，方向与 Nagle 不同。Linux TCP_QUICKACK 只是临时提示快速确认，后续处理可能重新延迟 ACK。ACK 是否与 FIN 合并取决于时序和关闭方向，不是设置选项就固定报文数。〔S7、N70〕

代码示例

```
write_small_request(); // 有未确认数据时，Nagle 可能暂存后续小段
receiver_may_delay_ack_briefly(); // 接收端尝试合并 ACK 或搭载反向数据
// 两者叠加时，小请求/响应可能多等一个确认周期
if (interactive_latency_is_measured_bottleneck)
    setsockopt(TCP_NODELAY, true);
// 关闭 Nagle 可降低等待，但会增加小包数量和网络开销
// Delayed ACK 时机依实现/流量；测量真实 RTT 和请求大小再调整
// 批量吞吐、拥塞和服务器负载也应一起比较
```

015 HTTP/2 和 HTTP/3 在传输层和多路复用上有什么区别?

答案　HTTP/2 把多个请求流交错放进一条 TCP 连接，并用 HPACK 压缩首部；TCP 丢包后，后续字节即使属于其他流也要等缺失字节重传，形成传输层队头阻塞。HTTP/3 把 HTTP 帧放进 QUIC 流，由 QUIC 在 UDP 上实现加密、可靠传输和按流复用；单个流丢包通常不会阻止其他流的数据交付，但连接层丢包与共享拥塞控制仍会影响整条连接。HPACK 的动态表、静态表和可选 Huffman 编码用于首部，不压缩 DATA 正文。HTTP/2 还有流级和连接级流控。HTTP/3 的 QPACK 把动态表更新与字段块解耦以降低阻塞，但字段块引用未收到的表项仍可能等待，因而限制可阻塞流数。QUIC 连接 ID 有助地址改变时维持连接，迁移仍须路径验证并受端点策略限制，不是任意换地址即可绕过验证。〔S7、I9、I10〕

代码示例

```
http2.multiplex_streams_over_one_tcp_connection();
http2.compress_headers_with_HPACK();
if (tcp_segment_lost)
    later_tcp_bytes_wait_for_retransmission(); // 跨流字节队头阻塞
http3.map_http_frames_to_quic_streams_over_udp();
if (quic_stream_A_lost)
    stream_B_can_deliver_independent_bytes();
// QUIC 仍有共享拥塞控制和连接层丢包影响，不是消除所有阻塞。
// HTTP/3 通常还整合 TLS 加密；性能依网络与实现。
```

016 一次典型 DNS 解析会经过哪些角色?

答案　域名解析通常从浏览器、操作系统或本地递归缓存开始；命中时直接返回。未命中则向配置的递归解析器查询，递归器再依据 DNS 委派关系寻找根、顶级域和权威服务器，也可能从其他缓存或转发器取得答案。返回结果按 TTL 缓存，过期后再查询。实际一次查询不一定每层都访问，递归器可能已有部分记录。

DoH 和 DoT 保护客户端到递归解析器这一段传输，DoQ 则用 QUIC 传递 DNS 查询；它们不改变权威数据的层级，也不自动证明解析器给出的答案正确。排查时应说明自己问的是浏览器缓存、系统解析器还是权威 DNS。直接对权威服务器查询与普通用户使用递归服务，路径和结果可能不同。〔S8〕

代码示例

```
if (browser_cache.contains(name))
    return browser_cache[name];
if (os_or_recursive_resolver_cache.contains(name))
    return cached_answer;
answer = recursive_resolver.query_root_then_tld_then_authoritative(name);
recursive_resolver.cache(answer, answer.ttl); // 未命中才按委派关系逐层查询
```

```
// 实际递归器可能命中转发器/缓存，不一定访问每一级权威服务器。
// DoH/DoT 改变客户端到解析器的传输保护，不改变 DNS 权威层级。
```

017 SO_REUSEADDR 和 SO_REUSEPORT 有什么区别?

答案　SO_REUSEADDR 和 SO_REUSEPORT 的行为依操作系统而异。Linux 上 REUSEADDR 常用于地址重用和服务重启时绑定；它不能简单作为多个进程共享同一监听端口的开关。REUSEPORT 允许满足条件的多个 socket 在 bind 前启用选项并绑定相同地址端口，内核可以把连接分给不同监听者，但还有权限和地址匹配规则。依赖这些细节时要查目标平台文档。

通配地址和具体地址绑定可能冲突，IPv6 的 :: 是否也接收 IPv4 还取决于 IPV6_V6ONLY。客户端 connect 自动选择临时端口时，不同远端四元组可以共享同一源端口；预先显式 bind 可能改变本地二元组的占用和复用条件。所以不能用临时端口数量直接推导服务器全局连接上限。〔S5、N70〕

代码示例

```
set_option(SO_REUSEADDR); // Linux 常用于重启绑定等规则，不代表任意多进程共享监听
if (multiple_listeners && each_socket_sets_before_bind(SO_REUSEPORT))
{
    bind_same_eligible_address_and_port();
    kernel_distributes_connections();
}
// 需满足权限、地址匹配等条件；不同 Linux/BSD/Windows 语义有差异。
verify_target_os_docs_and_test(); // 不把同名选项当跨平台等价保证
```

8.4 协议分层与应用交互

018 OSI 与 TCP/IP 如何对应，socket 和常用协议处于什么位置?

答案　OSI 把通信分为物理、链路、网络、传输、会话、表示、应用七层；常用 TCP/IP 模型通常合成链路、网际、传输、应用四层。发送时，HTTP 等应用数据交给 TCP 或 UDP 加端口信息，IP 加源和目的地址，链路层再封装下一跳所用的帧。接收端逐层解封装。MAC 地址用于本地链路交付，过路由器时帧会重建，所以它不是端到端寻址。

socket 是操作系统提供的通信接口，不是某个应用协议。Nginx 的 HTTP 反向代理处理应用层语义，stream 模块也能转发传输层连接。改服务端口不会改变协议所在层或消息语义；理解抓包应判断每个头部属于哪一层，以及它对谁有效。〔N50、N51〕

代码示例

```
struct TcpSegment
{
    int source_port, destination_port;
    std::string payload;
};
struct IpPacket
{
    IpAddress source, destination;
    TcpSegment segment;
};
auto frame = ethernet_wrap(next_hop_mac, ip_wrap(server_ip, tcp_wrap(443, 53000, request)));
// socket是API抽象；路由器会重建链路帧，端到端IP通常保持
```

019 HTTP/1.1 长连接如何区分报文，并在什么情况下关闭?

答案　HTTP/1.1 长连接复用 TCP 连接，但 HTTP 报文边界由协议规则决定，不是一条 recv 对应一条消息。请求由请求行、首部、空行和可选消息体组成，响应由状态行开始。接收方按方法、状态和有效的 Transfer-Encoding、Content-Length 或连接关闭等规则判断消息体长度；例如 HEAD 响应没有消息体。

有冲突的长度字段必须严格拒绝或按规范处理，否则代理和后端解析不一致会造成请求走私风险。连接还会因 Connection: close、空闲超时、错误或进程退出而结束。HTTP 长连接解决多请求复用，TCP Keep-Alive 是空闲路径探测，两者用途不同。〔N52〕

代码示例

```
std::string buffer = receive_bytes(); // TCP只提供字节流，不保留HTTP报文边界
auto header_end = buffer.find("\r\n\r\n");
std::size_t body_size = parse_content_length(buffer); // 按协议长度读完后才解析下一条请求
```

020 HEAD、PUT、DELETE、PATCH 与 OPTIONS 分别表达什么操作?

答案　HEAD 的响应元信息语义接近 GET，但响应不带消息体，适合只检查资源状态或长度。PUT 通常把提交的表示创建或替换到指定资源，DELETE 请求移除目标当前关联；按 HTTP 语义，PUT 和 DELETE 是幂等的，但重复请求可以得到不同状态码。PATCH 只表达部分修改，是否幂等取决于补丁格式及业务定义。

OPTIONS 用来查询目标支持的通信选项，也常用于浏览器 CORS 预检。方法名不代替鉴权、输入校验或业务幂等：例如 PATCH“余额加十”重复执行会重复加值。服务端应明确哪些方法允许产生什么副作用，并为客户端超时重试定义请求 ID 或状态查询办法。〔N51〕

代码示例

```
send_request("HEAD", "/report.pdf"); // 只取状态和首部
send_request("PUT", "/config/42", complete_representation); // 创建或替换
send_request("PATCH", "/account/42", patch_document); // 幂等性取决于补丁语义
```

021 HTTP 断点续传如何使用 Range，并保证拼接的是同一份资源？

答案　断点续传不是 TCP 自动记得上次下载到哪，而是客户端记录已经取得的字节范围，再请求 Range: bytes=起点-。服务器支持时用 206 Partial Content 返回片段，并用 Content-Range 标出它在完整表示中的位置；无效范围可能返回 416。服务端也可忽略 Range 而返回完整 200，所以客户端必须检查状态、范围及实际长度再拼接。

下载期间资源若更新，按旧偏移拼新内容会损坏文件。客户端可用 If-Range 和强 ETag 验证表示未变；验证不通过就重新取完整版本。字节范围针对具体表示，内容编码变化也可能让本地偏移不再对应原始内容，因此需校验版本和哈希。〔N51〕

代码示例

```
Request request{"GET", "/large-file"};
request.headers["Range"] = "bytes=1000-";
request.headers["If-Range"] = saved_etag; // 仅拼接相同版本的资源
if (response.status == 206 && response.etag == saved_etag)
    append(temp_file, response.body);
else if (response.status == 200)
    replace(temp_file, response.body);
```

022 DNS 通常用什么端口和传输协议，何时会使用 TCP？

答案　传统 DNS 查询使用端口 53，普通查询经常以 UDP 开始，但解析器和权威服务器都必须支持 TCP。区域传送通常走 TCP；UDP 响应被截断（TC 位为 1）、太大或实现策略要求时，解析器可改用 TCP 重试。EDNS 允许协商大于早期 512 字节上限的 UDP 报文，并不意味着越大越好，IP 分片和路径 MTU 丢包都可能影响可靠性。

DoT、DoH 和 DoQ 以加密传输承载 DNS，各有自己的端口和封装，不能套用“DNS 总是 UDP 53”。如果题目说传统 DNS，答案应限定其语境；分析抓包时还要区分普通查询、区域传送和加密 DNS。〔N56〕

代码示例

```
DnsReply reply = query_dns("www.example.com", Type::A, Transport::udp);
if (reply.truncated)
    reply = query_dns("www.example.com", Type::A, Transport::tcp);
// 区域传送通常使用TCP；UDP截断时客户端可回退到TCP
```

023 已经使用 HTTP，为什么系统还会引入 RPC 框架？

答案　HTTP 规定请求响应、头部和状态码等通信语义；RPC 规定如何把远端服务表示为方法、参数和结果。二者不是互斥选项，可以组合，例如 gRPC 通常用 HTTP/2 承载远程调用。RPC 框架还可提供接口描述、跨语言代码生成、序列化、流式调用、截止时间、状态传播和拦截器，减少团队重复制定约定。

选择时看跨语言契约、浏览器兼容、调试、负载和服务治理，而不是先断言 HTTP 必然慢或 RPC 必然用某种传输。网络远程调用会超时、部分失败或重复执行，即便代码写成 user.Get() 也不具有本地函数的可靠性。需设计截止时间、错误语义和幂等策略。〔N62〕

代码示例

```
RpcReply reply = call("UserService.Get", request, deadline);
if (reply.timed_out())
    reply = query_request_status(request.id); // 超时不代表服务端没执行
if (reply.is_retryable())
    retry_with_same_id(request); // 幂等键避免重复副作用
```

024 HTTP 长连接与 WebSocket 的连接复用和双向通信有何差别？

答案　HTTP 长连接让多个 HTTP 交互复用底层连接，但它本身仍遵循 HTTP 版本定义的请求响应或流语义，不会自动变成随意双向推送的自定义通道。经典 WebSocket 先以 HTTP/1.1 请求升级，再使用独立帧协议双向传输文本或二进制消息，适合聊天和协作。不过它不自动提供业务 ACK、离线消息、重连补发或权限续期，应用要自行定义。

若只需要服务器到浏览器推送，SSE 使用 UTF-8 的 text/event-stream；原生 EventSource 发 GET，上行可另走 HTTP，也可由 fetch 读 POST 响应并解析事件格式。短轮询持续发新请求，长轮询由服务端挂起当前请求，二者都需处理重连和资源占用。WebSocket 的 101 与 Sec-WebSocket-Key/Accept 确认协议升级，不代替身份认证、Origin 校验或 wss 加密；代理超时及慢客户端背压也需考虑。〔N63、A126〕

代码示例

```
std::string event = "event: progress\ndata: 50%\n\n";
write_all(client_socket, event); // SSE适合服务端单向推送，连接保持打开
// 需要双向实时消息时再考虑WebSocket，并设计重连与背压
```

025 正向代理与反向代理分别代表谁，TLS 终止后需要注意什么？

答案　正向代理通常由客户端选择，代客户端访问外部资源，可承担出口控制、缓存或转发；反向代理位于服务入口，替后端接收请求，可路由、负载均衡和终止 TLS。两者区别是代表谁、部署在哪一侧，不取决于是否改写报文或监听哪个端口。

代理终止 TLS 后，客户端到代理与代理到源站是两条独立连接；后半段是否加密、如何验证证书要另配。代理传给后端的 X-Forwarded-For、Forwarded 等头可以被客户端伪造，代理应清理外来值并由后端只信任受控入口写入的数据。转发头不是身份凭证，也不能替代源站认证。〔N51〕

代码示例

```
TlsConnection client_to_proxy = accept_client_tls(client);
Request request = sanitize_forwarded_headers(client_to_proxy.request());
TlsConnection proxy_to_backend = connect_tls(backend, verify_certificate = true);
proxy_to_backend.send(request); // 两段TLS认证分别配置，业务授权仍由服务端检查
```

026 CDN 怎样加速内容访问，缓存键和回源有哪些边界？

答案　CDN 把请求导向较合适的边缘节点，缓存命中时就近返回，减少源站负载和跨地域传输；未命中则依规则回源并决定是否缓存。常见调度通过 DNS：权威解析把域名引到 CDN/GSLB，再结合递归解析器提供的客户端网段、运营商、URL、节点负载和健康度选择边缘。递归缓存 TTL 意味切换节点不会立刻覆盖所有用户。

缓存键和转发策略必须符合资源语义，慎重处理查询参数、Cookie、鉴权头及压缩变体，避免把用户数据共享给别人。动态请求不保证更快，源站故障时能否使用旧缓存取决于策略。更新可以使用内容哈希版本名、短 TTL 或显式刷新，但 CDN 加速不能代替源站鉴权。〔N65、N53〕

代码示例

```
using CacheKey = std::tuple<std::string, std::string, Encoding>;
CacheKey key{host, path, required_encoding};
if (cache.contains(key))
    return cache.at(key);
Response response = fetch_from_origin(key); // 私有Cookie等变化也必须纳入策略
```

8.5 HTTP 缓存、会话与浏览器安全

027 HTTP 新鲜缓存与条件验证如何配合，no-cache 和 no-store 有何差别？

答案　缓存先判断现有响应是否仍新鲜。max-age 等规则仍满足时可以直接复用；过期或必须验证时，客户端带 If-None-Match/ETag 或 If-Modified-Since/Last-Modified 请求验证。若表示没变，服务器可返回 304，客户端继续用本地实体；改变时返回新表示。缓存命中、连接复用和是否需验证是不同问题。

no-cache 允许保存响应，但复用前必须成功验证；no-store 要求这次请求或响应不被存储。还要区分 private 与共享缓存，按 Vary 区分语言或压缩等变体，并谨慎缓存鉴权内容，避免用户 A 的响应给用户 B。no-cache 并不等于“每次都返回 200”，304 本身是有效的验证结果。〔N53〕

代码示例

```
Request request = get("/page");
request.headers["If-None-Match"] = cached.etag;
Response response = send(request);
if (response.status == 304)
    refresh_expiry(cached);
else if (response.status == 200)
    cache.store(response);
// no-cache表示复用前验证；no-store表示不保存可复用副本
```

028 hosts 文件怎样影响名称解析，如何区分本地配置与 DNS 劫持？

答案　hosts 文件提供本机静态的主机名到 IP 映射；走系统解析器的程序通常按名称服务配置查询 hosts 和 DNS，具体顺序由系统配置决定。dig 等工具常直接问 DNS 服务器，可能绕过 hosts，所以 dig 显示的地址不一定是浏览器按系统解析器得到的地址。还要检查浏览器是否启用独立的安全 DNS。

定位异常时按层比较 hosts、解析器配置、本机和递归缓存、递归器回答、权威记录。DNS 劫持可能来自被改过的本地配置、恶意递归器或伪造记录。DNSSEC 可验证已签名记录的真实性与完整性；DoH/DoT 加密客户端到解析器的传输，但不会保证恶意解析器返回正确答案。〔N54、N55〕

代码示例

```
auto system_result = getaddrinfo("service.example"); // 遵循hosts与NSS顺序
DnsReply direct_result = query_dns("service.example", trusted_resolver);
if (system_result != direct_result.addresses)
```

```
    inspect_hosts_cache_and_resolver();
// 分别核对本机覆盖、递归解析与权威记录
```

029 HTTP 无状态与 Cookie、Session、Token 登录态是什么关系?

答案　HTTP 的无状态是指协议没有要求服务器记住上一次请求才能理解下一次，不是说应用不能保存会话。Cookie 是浏览器按域和路径规则保存并随请求发送的数据；Session 常指服务端保存的会话状态，客户端拿随机会话 ID 查找；Token 是身份或授权凭证的形式，可以是不透明随机串，也可以包含自描述声明。

会话 ID 可以放在 Cookie 或请求头中，Cookie 只是承载方式，不会改变 HTTP 协议无状态属性。禁用 Cookie 后可以显式携带凭证，但把它放 URL 容易落入历史记录、代理日志和引用来源。服务端仍需限制凭证权限、生命周期和撤销方式，不能把“token”这个名称当作安全保证。〔N57、N51〕

代码示例

```
Cookie cookie{}; // 使用普通赋值初始化配置，避免混用聚合与指定初始化
cookie.name = "sid";
cookie.value = random_session_id();
cookie.http_only = true;
cookie.secure = true;
cookie.same_site = SameSite::lax;
response.set_cookie(cookie);
Session session = session_store.find(request.cookie("sid")); // 服务端验证会话有效期和权限
```

030 Cookie 与 localStorage 如何选择，登录凭证需要哪些浏览器侧保护?

答案　Cookie 会按规则自动随请求发送，适合承载会话 ID；自动携带也让跨站请求伪造需要防范。localStorage 是按源隔离、由脚本读取的持久键值存储，不会自动成为 HTTP 头，常用于界面偏好。HttpOnly 限制脚本读取 Cookie，Secure 限制非安全通道发送，SameSite 控制部分跨站携带；它们解决的问题不同。

JWT 等凭证存 localStorage 时，成功执行的同源恶意脚本可以读取并带走；存 HttpOnly Cookie 可阻止直接读取，却仍要设计 CSRF 防护。HttpOnly 不能阻止 XSS 在页面里代用户发请求。登录凭证还应短期有效、限制权限并支持撤销；同时治理脚本注入，不能指望一个 Cookie 属性全面解决浏览器端风险。〔N58、N59〕

代码示例

```
local_storage.set("theme", "dark"); // 适合不敏感偏好
set_cookie("sid", session_id, HttpOnly, Secure, SameSite::Lax); // 脚本不可直接读取
// Cookie方案仍需考虑CSRF；XSS仍可能代表用户发起页面内请求
```

031 JWT 为什么便于跨实例校验，它的结构与撤销边界是什么?

答案　常见签名型 JWT 有头部、声明载荷和签名三段编码。载荷可包含签发者 iss、受众 aud、主体 sub 和有效期 exp 等声明，但编码不是加密，拿到令牌的人通常能读载荷。验证端要检查可信签名、允许的算法、签发者、受众和过期时间，不能只解码后相信字段。令牌也不应装入密码等秘密。

多实例可持有同一可信密钥或公钥离线验证，不必每次查询单一会话库；共享状态也可以扩展，所以不是唯一方案。纯离线验证的令牌泄露后难以即时撤销，可用短有效期、刷新令牌轮换、撤销列表或版本检查，但这些办法重新引入存储和运维复杂度。设计要权衡离线验证效率与撤销及时性。〔N60、N61〕

代码示例

```
Jwt jwt = parse(token);
if (!allowed_algorithm(jwt.header.alg) || !verify_signature(jwt, public_key))
    reject();
if (jwt.issuer != expected_issuer || jwt.audience != expected_audience || jwt.expired(now))
    reject();
// JWT载荷可读；需要立即撤销时还需版本号、撤销表或短有效期
```

032 跨源访问为什么受限，CORS 预检能否代替身份认证?

答案　浏览器把协议、主机和端口组成源。同源策略限制脚本读取其他源的响应；CORS 让服务器通过允许来源、方法和头部等响应字段，告诉浏览器是否放行跨源读取。部分请求会先发 OPTIONS 预检，满足简单请求条件的则可直接发送。带凭证时不能用通配来源代替明确来源，也要由服务器按规范允许凭证。

CORS 是浏览器端的读取控制，不是服务端鉴权。非浏览器客户端可以直接请求，一些跨源写请求也可能已经到达服务器，即使浏览器最后不让脚本读取响应。服务端必须独立认证、授权并防范 CSRF，不能信任“预检通过”或仅凭 Origin 放行重要操作。〔N64〕

代码示例

```
if (request.is_non_simple())
    send_preflight("OPTIONS", origin, method, headers);
Response response = send_actual_request();
if (cors_allows(response, origin))
    expose_to_browser_script(response); // CORS不是身份认证
```

033 XSS 与 CSRF 分别怎样利用浏览器身份，防护重点为何不同?

答案　XSS 是把不可信输入变成页面中的脚本执行，攻击脚本可读写脚本可见数据或代用户操作。防护重点是按输出位置编码 HTML、属性、URL 或脚本内容，尽量用安全 DOM 接口；允许富文本时做有规则的净化，再用 CSP 限制作为额外防线。不能把任意输入拼进 innerHTML 后寄希望于过滤黑名单。
CSRF 则诱使浏览器向目标站发起请求，利用浏览器自动附带的现有凭证；攻击者通常不需要读到响应或执行目标站脚本。可用不可预测的 CSRF token、合适的 SameSite Cookie、Origin 校验，并避免用 GET 做敏感写操作。HttpOnly 和 CORS 单独都不能解决全部问题；成功的 XSS 还能绕过许多 CSRF 防护。〔N75、N76〕
代码示例

```
Node* safe_comment = document.createTextNode(user_input); // 当文本插入，避免按HTML解释
comment_area.appendChild(safe_comment);
if (!valid_csrf(request) || !authorized(session, request))
    reject(); // 分别防CSRF并鉴权
```

8.6 TCP 状态与应用确认

034 如何识别一条 TCP 连接，TCP 头部字段分别解决什么问题?

答案　在同一网络命名空间和协议中，TCP 连接通常由源 IP、源端口、目的 IP、目的端口四元组识别；监听 socket 与已建立连接不是同一个实例。TCP 首部包含端口、序号、确认号、长度、控制位、窗口和校验和，选项还可协商 MSS、窗口缩放、时间戳和 SACK。最小首部 20 字节，有选项会更长。
序号以字节计并按 2^32 回绕；SYN 和 FIN 各占一个序号，纯 ACK 不占。ACK 指向连续收到前缀之后的位置，不能简单用上一包序号加负载长度推断所有状态。初始序号要兼顾旧连接隔离和不可预测性，不保证永不重复；协商时间戳后的 PAWS 可辅助拒绝过期段，但不能替代连接状态和序号验证。一个 ACK 只证明传输层进度，不证明业务操作成功。〔N66、I34〕
代码示例

```
std::uint32_t next_seq = 1001; // SYN的序号1000占用一个序号
std::uint32_t ack = next_seq + received_payload_bytes;
// TCP连接由源/目的地址与端口标识；ACK表示下一个期望字节序号
```

035 握手报文丢失或重复时 TCP 怎样恢复，accept 参与握手吗?

答案　客户端发 SYN 后等 SYN+ACK，超时可按实现重传；服务器收到 SYN 后通常记录半连接并重发 SYN+ACK，直到确认或过期。最终 ACK 丢失时，客户端可能已认为连接建立，而服务器仍重发 SYN+ACK；客户端可再次确认，后续带正确 ACK 的数据也能帮助服务端完成握手。重复 SYN 由四元组、序号和当前状态处理，不会无止境新建相同连接。
Linux 的 accept 通常从握手完成的队列取连接并返回新 fd，不是应用先 accept 才由内核发第三次握手。已建立连接遇到异常 SYN 时，可按 RFC 5961 规则发 Challenge ACK；RST 通常要求序号精确等于 RCV.NXT，窗口内但不精确匹配可挑战确认，状态不同规则也不同。无对应连接通常发 RST，但防火墙可丢包，主动打开也可能建立连接。
TCP Fast Open 在双方支持时用先前获取的 Cookie 把早期数据放进 SYN，仍保留三次握手；部署要考虑回退和重放。它需要内核、应用及路径配合。TLS ClientHello 可以与 TFO 配合，但这不专属于 TLS 1.3，也不自动等于 TLS 0-RTT 应用数据。〔N66、N67、I7、I8、N69〕
代码示例

```
send_syn(client, initial_seq);
if (received_syn_ack())
    send_ack(); // 内核完成握手后放入accept队列
int connection_fd = ::accept(listener_fd, nullptr, nullptr); // accept取连接，不发起握手
```

036 CLOSE_WAIT 和 FIN_WAIT_2 积压通常说明什么，半关闭还能做什么?

答案　CLOSE_WAIT 表示本端收到了对端 FIN 并已确认，对端不再发送，但本地应用还没有关闭自己的发送方向。大量长期 CLOSE_WAIT 常提示应用某条结束路径漏了 close，或处理任务一直卡住。FIN_WAIT_2 表示本端 FIN 已被确认，正在等对方也关闭；无应用引用的孤儿连接有内核超时策略。
半关闭允许一方用 shutdown 关闭发送方向，同时继续接收对端剩余数据。FIN 是字节流中的有序结束位置，之前的数据有缺口时，乱序 FIN 要等缺口补齐才消费；read 返回零表示有序数据读完后的 EOF，不是收到额外 EOF 字节。close 只释放当前 fd 引用，dup/fork 共享引用未关闭时未必发送 FIN；shutdown 操作的是 socket 方向。未读数据或 SO_LINGER 也可能让退出导致 RST，不能保证每次都优雅 FIN。tcp_fin_timeout 主要管无应用引用的 FIN_WAIT_2，不能控制仍被使用的半关闭连接。〔N70、N66〕
代码示例

```
::shutdown(socket_fd, SHUT_WR); // 只关闭本端发送方向
for (;;)
{
    ssize_t n = ::read(socket_fd, buffer, sizeof(buffer));
    if (n > 0)
    {
```

```
        consume(buffer, static_cast<std::size_t>(n));
        continue;
    }
    if (n == 0)
        break;
    if (errno == EINTR)
        continue;
    throw_system_error(errno);
} // EOF后关闭socket；迟迟不close会让对端停在CLOSE_WAIT
```

037 send 返回成功与 TCP ACK 分别能证明什么？

答案　send 对流式 socket 返回正数，通常只表示这些字节已被本机发送路径接收；它可能短写，也不证明远端已经收到。连接之后仍可能断开，错误也可能稍后才显现。TCP ACK 能证明对端传输层按序接收了相应字节，但应用可能尚未读取，甚至在处理前崩溃。

业务成功的确认点必须由应用协议定义，例如服务器完成持久化后返回订单 ID。客户端超时可能不知道操作究竟失败还是响应丢失，应使用请求 ID、幂等约束或状态查询收敛。不能把 send 成功或一次 ACK 当作资金、订单等业务已经提交。〔N74〕

代码示例

```
SendResult result = send_all(socket, request_with_id(idempotency_key));
if (!application_ack_before_deadline(idempotency_key))
    query_operation_status(idempotency_key);
// TCP ACK只确认传输层收到字节；应用确认才说明业务结果已知
```

8.7 网络故障与安全

038 DDoS 与 SYN Flood 会耗尽哪些资源，应怎样分层缓解？

答案　DDoS 通过多个来源耗尽链路带宽、连接状态或应用处理能力，使正常用户难以获得服务。SYN Flood 压未完成握手资源，反射放大流量压入口，TLS 握手消耗计算，DNS 或 HTTP 洪泛消耗应用容量。先监测带宽、半连接、丢包、入口错误率与请求耗时，判断瓶颈是在运营商链路、主机还是业务热点。

缓解要分层：网络侧可联合运营商清洗和边缘分发，主机侧审慎启用 SYN cookies 并调整队列，应用侧用有界并发、超时、鉴权和公平限流。Cookies 会由序号携带可校验信息，最终 ACK 到来后重算验证再建立状态；它不消除握手计算，也不解决 accept 慢、上游带宽打满或应用层洪泛。队列与重试参数需按目标内核和实际丢弃层次评估，源站绕过和误伤正常用户也要纳入演练。〔N68、N69〕

代码示例

```
if (syn_rate(source) > threshold)
    apply_edge_rate_limit(source);
if (accept_queue_full())
    inspect_accept_workers_and_syn_cookies();
// 链路饱和需上游清洗；单纯增大backlog不能恢复入口带宽
```

039 服务已经启动但请求失败或很慢，应怎样按阶段定位？

答案　服务已启动但请求失败或变慢，先记录目标地址、时间、错误和复现条件，把 DNS、连接、TLS、首字节、响应体各阶段分开计时。核对解析地址、路由、防火墙和代理，再用 ss 看监听地址及连接状态，确认没有只监听回环或错误网络命名空间。curl 的阶段计时、证书检查和经授权抓包可帮助定位；再关联服务日志、请求 ID、队列、线程池和依赖耗时。TCP 连通不证明鉴权或业务成功，超时也不必然是丢包。

抓包需对齐两端时钟与观察点，并理解 Wireshark 相对序号、窗口缩放、卸载造成的大包或校验和显示，以及分析器的推测标签。一个点没见 ACK 不证明对端没发。按时间增量查看驱动 ring、softnet、qdisc、socket 和握手队列的丢弃及重传，再联系带宽、RTT、吞吐和 PPS；小包流量可能先耗尽包处理能力。中间跳的 ICMP 被限速不等于业务丢包，驱动计数器语义也需查设备文档。〔N70、N74、I2〕

代码示例

```
auto start = Clock::now();
auto dns = resolve(host);
auto connected = connect(host);
auto tls_done = handshake(connected);
auto first_byte = request_and_wait_first_byte(connected);
log_latency(request_id, start, dns, connected, tls_done, first_byte); // 按阶段对齐两端日志
```

8.8 网络层与链路层

040 IP 与 MAC 地址如何协作，访问外网时 ARP 解析谁的地址？

答案　IP 地址用于网络层寻址和路由；MAC 地址用于本地链路上的帧交付。路由器每转发一跳，链路帧通常重新封装，源和目的 MAC 会随下一跳变化，而 IP 包继续朝最终目的地转发。主机先查路由：目的在直连网段就解析目的主机 MAC，跨网段则解析网关 MAC，不会在本地 ARP 远端服务器。IPv4 以太网用 ARP，IPv6 用邻居发现。

CIDR 前缀表示网络部分，路由通常按最长前缀匹配，默认路由只在没有更具体项时使用；不能只凭传统 A/B/C 类判断同网段。策略路由和 NAT 会改变实际路径及地址。127.0.0.1、::1 是回环本地交付，不依赖网线；localhost 是名称，可能解析成任一地址族，要核对实际解析和监听范围。MAC 也不是可验证的全球身份。〔N71、N50、S5〕

代码示例

```
IpAddress destination = parse_ip("203.0.113.20");
IpAddress next_hop = route_table.longest_prefix_match(destination).gateway_or(destination);
MacAddress mac = arp_cache.resolve(next_hop);
ethernet_send(mac, ip_packet(destination, payload));
// 外网帧目的MAC是网关；IP目的地址仍是远端服务器
```

041 ping 与 traceroute 怎样工作，ping 不通为什么不代表 HTTP 不通？

答案　ping 通常用 ICMP Echo Request/Reply 测往返时间和丢包，验证的是 ICMP 交换，不会测试 TCP 端口、TLS 或应用逻辑。traceroute 逐步提高 IP TTL/Hop Limit，让中间路由器在到期时回 ICMP Time Exceeded，据此推测路径；探测可用 UDP、ICMP 或 TCP，回程也可能不同。

防火墙或限速策略可以丢 ICMP，因此 ping 失败而 HTTP 正常；ping 成功也可能遇到端口未监听或应用异常。某跳不回应探测，不说明它没有转发业务包。UDP traceroute 到终点常见 ICMP 端口不可达，TCP 探测则可能收到 SYN+ACK 或 RST。ICMP 差错也用于路径 MTU 等诊断，但会受过滤、限速和协议差错报告规则约束；历史 Source Quench 已废弃，不能当作现代拥塞控制建议。〔N72、I39〕

代码示例

```
auto icmp_time = ping(host); // ICMP探测失败不代表服务端口不可达
auto tcp = connect_tcp(host, 443);
auto tls = start_tls(tcp);
HttpResponse response = get_health_endpoint(tls); // 直接验证应用路径
```

042 多个内网设备怎样通过 NAT/NAPT 共用一个公网地址？

答案　NAT 改写地址，常见家庭和企业网关还用 NAPT 改写端口，让多台内网设备共用一个公网地址。网关为每条流维护映射：出站源地址和端口换成公网地址及外部端口，回包再按映射恢复给内部设备。状态有超时；外部主机通常无法仅凭公网地址找到任意内网服务。

入站访问需要明确端口映射、转发或其他连通机制。NAT 影响端到端寻址，携带 IP/端口的应用协议、点对点连接和故障切换可能需要额外处理。NAT 本身不是防火墙策略，安全性仍依赖明确的访问控制和状态检查，不能把“有 NAT”当作不需鉴权。〔N73〕

代码示例

```
using NatKey = std::tuple<IpAddress, Port, IpAddress, Port>;
translations[{private_a, 51000, server, 443}] = {public_ip, 40001};
translations[{private_b, 51000, server, 443}] = {public_ip, 40002}; // 端口区分映射
```

8.9 链路转发与地址配置

043 二层交换机怎样学习 MAC 地址，未知单播和广播怎样转发？

答案　二层交换机收到帧时，先根据入站帧的源 MAC 学习“MAC 位于哪个端口”，表项通常限定在对应 VLAN 等桥接域并会老化。转发时才查询目的 MAC：已知且在别的端口就单播转发，已知在入端口则过滤；未知单播通常泛洪到该桥接域内允许的其他端口。广播也在广播域中传播，路由器通常不会把它原样跨三层边界转发。

交换机学习源地址，不是通过每个端点的目的地址来学习，也不等于配置了 IP。二层环路会让广播和未知单播持续放大，需要生成树等环路控制；VLAN、端口状态和过滤规则会改变可转发的端口集合。排障时应同时检查 MAC 表、VLAN 和端口状态。〔I1〕

代码示例

```
using MacKey = std::pair<VlanId, MacAddress>;
mac_table[{vlan10, source_mac}] = ingress_port; // 交换机从源MAC学习
if (mac_table.contains({vlan10, destination_mac}))
    forward(mac_table.at(key));
else
    flood_except(ingress_port);
```

044 IPv6 相对 IPv4 改变了哪些地址、首部和邻居发现规则？

答案　IPv6 把地址扩展到 128 位，固定基本首部为 40 字节，将一些可选功能放到扩展首部；基本首部没有 IPv4 式首部校验和。IPv6 有单播、任播和组播，没有广播。链路本地地址只用于本链路通信，不能当公网路由地址。邻居发现基于 ICMPv6，负责链路地址解析、路由器发现和邻居可达性，不再使用 IPv4 ARP。

路由器不会替 IPv6 源端分片；源端依据 Packet Too Big 等反馈调整包大小，必要时自行分片。更大的地址空间或协议支持并不自动提供认证和加密，双栈 DNS、访问控制和 MTU 仍要正确配置。迁移时需同时检查监听地址族、路由和防火墙，不能仅因接口显示 IPv6 地址就断言外部服务可达。〔I11、I12、I29〕

代码示例

```
if (!neighbor_cache.contains(next_hop))
    send_icmpv6_neighbor_solicitation(next_hop);
NeighborAdvertisement reply = wait_for_neighbor_advertisement();
neighbor_cache[next_hop] = reply.link_address; // IPv6用NDP取代ARP
```

045 DHCPv4 怎样分配与续租地址，跨子网为什么需要中继？

答案　DHCPv4 首次动态获取常见流程是 Discover、Offer、Request、ACK：客户端寻找服务器，选择租约提议并请求，服务器确认地址和参数，可能还包括掩码、网关和 DNS。客户端起初没有可用地址，常用广播。续租通常先单播向原服务器请求；进入重绑定阶段后可广播找其他服务器。租约过期仍未续得地址，就应停止使用。

普通路由器不转发客户端网段广播，跨子网时 DHCP Relay 在本地接收并转交服务器，附带链路信息让服务器选对地址池。DHCP 服务器不一定可信，拿到租约也不保证没有冲突或网络已通，应进行冲突检测并验证实际连通性。不能把整个租期的续租过程都说成四次广播。〔I13〕

代码示例

```
enum class DhcpState
{
    discover,
    offer,
    request,
    acknowledge,
    renew,
    rebind
};
send_dhcp(DhcpState::discover);
auto offer = receive_offer();
send_dhcp_request(offer.address);
auto lease = receive_ack(); // 跨子网由relay转发广播
```

046 IPv4 组播与 IGMP 分别负责什么，加入组播组是否等于建立连接？

答案　单播发给一个目的，广播发给一个广播域，组播则面向加入某组的接收者，网络只需在通往接收者的分支复制数据。主机用 IGMP 向本链路组播路由器表达成员关系，路由器通过查询和报告维护接收者；IGMPv3 可表达源过滤。跨网组播分发树还要靠组播路由协议建立，IGMP 不等于全网路由。

加入组只表示接收意愿，不建立 TCP 式连接，也不保证交付、顺序或身份认证；发送方通常无须先加入接收组。交换机可通过 IGMP Snooping 限制无关端口泛洪，但行为还取决于设备和网络配置。应用若需要可靠数据、发送者认证或重放保护，必须在更高层另行设计。〔I14、I1〕

代码示例

```
int fd = ::socket(AF_INET, SOCK_DGRAM, 0);
ip_mreq group{};
group.imr_multiaddr = multicast_address;
::setsockopt(fd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &group, sizeof(group)); // 加组不提供可靠传输
```

8.10 Linux 网络数据路径

047 Linux 收到和发送网络包时，DMA、NAPI 与 sk_buff 怎样配合？

答案　接收路径中，驱动先准备描述符和缓冲区，网卡通过 DMA 把数据写入内存；典型中断确认事件并调度 NAPI，暂时抑制重复设备中断，再按预算轮询收包。完成后可恢复中断，协议栈继续检查、路由并分发到 socket。NAPI 常在软中断处理，也支持线程化和忙轮询。

发送时数据从 socket 进入协议栈、队列和驱动，驱动提交描述符后网卡读取并发出。sk_buff 保存包元信息并引用数据区，移动首部指针或克隆时可以少复制；克隆 skb 不意味着整份 payload 都复制。发送完成只表示设备资源可回收，TCP 为未确认字节保留数据以便重传，所以绝不等同于对端 ACK 或业务成功。〔I2、I3〕

代码示例

```
enum class RxStage
{
    dma,
```

```
    interrupt,
    napi_poll,
    protocol_stack,
    socket_queue
};
void poll_rx(int budget)
{
    for (int i = 0; i < budget && packet_ready(); ++i)
        deliver_to_socket(build_skb());
}
// NAPI按预算轮询；TCP仍需保留未确认数据以便重传
```

第 9 章 数据库与缓存

9.1 Redis 与缓存设计

001 MVCC 是什么?

答案　MVCC 是多版本并发控制：更新一行时保留旧版本，让普通一致性读按事务可见性选择合适版本，读者通常不必阻塞写者。它不意味着所有读写都无锁；更新、锁定读和冲突检查仍会加锁。InnoDB 通过事务 ID、undo 版本链和 ReadView 判断版本是否可见，不可见时沿 undo 找较早版本。

在 InnoDB 中，事务自己的修改可见；ReadView 建立前已提交的版本可见，建立时仍活跃的事务版本通常不可见。RC 每次一致性读生成新视图，RR 通常在首次一致性读建立后复用，不是 BEGIN 一执行就固定。长事务会让旧版本和 undo 难以清理，增加空间及扫描成本，所以查询长时间不结束也会拖累写入。〔S9、I61、I88〕

代码示例

```
Transaction t1(db, Isolation::repeatable_read);
auto first = t1.select_balance(id); // 快照100
Transaction t2(db);
t2.update_balance(id, 80);
t2.commit();
auto second = t1.select_balance(id); // RR普通读仍看到旧版本
```

002 Redis RDB 与 AOF 持久化有什么区别?

答案　RDB 把某一时点的内存状态做成快照，文件紧凑，通常恢复较快，也适合备份；快照开始后的写入可能丢失。SAVE 在主线程做，BGSAVE 用子进程做但 fork 会带来停顿，后续写入还可能触发写时复制，增加内存和延迟。AOF 记录可重放写操作，耐久性取决于从用户缓冲写到内核、再 fsync 的策略：always 较强但成本高，everysec 后台约每秒同步，no 由系统控制。

AOF 重写扫描当前状态，省去无效历史命令；混合模式可用 RDB 数据作基础。Redis 7 起多文件 AOF 由 base、增量文件和 manifest 管理，父进程写新增量、子进程生成 base，再切换清单；base 可为 RDB 或 AOF 格式，旧重写流程不能原样套用。RDB 与 AOF 可并用，应按可接受丢失窗口和恢复时间测试。〔I77、N116〕

代码示例

```
periodically_fork_snapshot_to_RDB(); // 紧凑，恢复通常快；只含某时点，可能丢其后写入
append_write_commands_to_AOF(); // 恢复点可近，需重放且文件可能更大
configure_fsync("everysec"); // 延迟/耐久折中；always 更耐久也可能更慢
if (background_snapshot_writes)
    account_for_COW_memory_amplification();
// 监控快照年龄、AOF 重写、恢复耗时；按 RPO/RTO 实测，而非只看格式。
enable_both_if_backup_restore_speed_and_a_finer_RPO_are_both_required(); // 同时运行仍要监控资源
```

003 Redis 内存满时如何处理?

答案　Redis 达到 maxmemory 后按淘汰策略处理。noeviction 会拒绝需要增加内存的写，但读和释放内存操作仍可工作。volatile 策略只从带 TTL 的键中选，候选不足时也可能无法淘汰；allkeys 可在全部键中选择。LRU 是采样近似，LFU 用概率增长且随时间衰减的计数适应热点变化，不是保存所有键的精确访问排名；采样数和 LFU 参数会影响结果。

还要区分 maxmemory 与进程 RSS：复制、AOF 缓冲、fork 写时复制和碎片都会占额外内存。应为这些用途留余量，监控淘汰数、命令错误和尾延迟。如果业务键都没有 TTL，volatile 策略可能让写入失败；不能只按一个估算的键空间大小配置内存上限。〔I78、N116〕

代码示例

```
if (used_memory_reaches_maxmemory)
{
    if (policy == noeviction)
        return write_error_but_reads_may_work();
```

```
    candidates = policy_is_volatile ? keys_with_ttl : all_keys;
    if (candidates.empty())
        return eviction_cannot_free_key();
    evict_by_configured_policy(candidates); // LRU/LFU/TTL 等按配置近似选择
}
reserve_memory_for_replication_AOF_buffers_and_fragmentation();
monitor(evicted_keys, rejected_writes, latency);
```

004 Redis 分布式锁需要注意什么?

答案　Redis 锁常用 SET key 唯一令牌 NX PX ttl 原子尝试获取；释放和续租要用脚本原子校验令牌后删除或改期，不能 GET 再 DEL。超时代表结果未知，锁可能已由自己取得也可能没有；应有界退避、读取状态并处理通知丢失。任务若长于租约，或持有者暂停、分区后旧客户端仍继续写，就会出现迟到写入。异步复制切主也可能让两个客户端都以为锁有效。

Redlock 在互不复制的多个主节点申请同一令牌，要求多数成功，并扣除请求耗时和时钟漂移后仍有正租期；失败要尽力安全释放所有节点。它依赖时间及故障假设，多数成功不等于严格互斥。关键资源应由下游检查单调 fencing token，或用数据库条件更新拒绝旧持有者。续租和客户端取消信号本身挡不住已在途的迟到写。〔S10、N119〕

代码示例

```
std::string token = random_token();
if (redis.set(lock_key, token, nx, 30s))
    process_order();
redis.eval("if redis.call('GET',KEYS[1])==ARGV[1] then return redis.call('DEL',KEYS[1]) end",
           lock_key,
           token);
// 下游用单调fencing token拒绝过期持有者的写入
```

005 Redis 常见数据结构及适用场景是什么?

答案　Redis 的数据结构应按操作语义选：String 常用于缓存、计数；Hash 按字段更新对象；List 适合两端队列；Set 做精确去重和集合运算；ZSet 按分数排序做排行。Bitmap 用 String 位图记录签到等状态，空间受最大偏移影响，稀疏大 ID 应先映射。HyperLogLog 估算基数，标准误差约 0.81%，不能查询成员或删单个成员。GEO 基于有序集合，附近查询能力和精度有边界。

Stream 是带 ID 和消费组的追加日志，ID 在一个流内有序唯一。List 的弹出没有确认和自动重投，可先 LMOVE/BLMOVE 到处理中队列，再自行确认及回收；Pub/Sub 至多一次且不存离线消息，可靠任务可评估 Stream。还要看命令复杂度、集合规模、持久化和是否必须精确。〔I84、I85、I86、N126〕

代码示例

```
SET(key, value);
INCR(counter); // String: 缓存值/计数
HSET(user, field, value); // Hash: 字段对象; List 可做两端队列
SADD(tags, tag); // Set 去重; ZADD(rank, score, member); // ZSet 排序/Top N
XADD(stream, id, fields); // Stream 带 ID 消息流/消费组
SETBIT(flags, offset, 1);
HLL_ADD(unique_visitors, item);
GEOADD(locations, lon, lat, id);
// Bitmap / HyperLogLog / GEO 分别用于位状态、近似基数、地理查询。
// 比较命令复杂度、集合规模、精确性、持久化与恢复需求再选。
```

9.2 MySQL 索引、事务与并发控制

006 为什么 MySQL InnoDB 索引常用 B+ 树?

答案　InnoDB 常用 B+ 树，是因为内部节点保存分隔键和子页指针，扇出高、树通常较矮；叶页按键有序并链接起来，适合范围查询、排序和前缀访问。聚簇索引叶子保存整行，二级索引叶子带主键，查其他列时可能再回聚簇树。索引会占空间，也会增加写放大和页分裂。

页内记录的物理位置不要求有序，next_record 把记录按索引序串联；Page Directory 保存若干分组边界槽，二分定位后在小组内查找，常见组约 4–8 条。容量可粗估为内部扇出^(高度−1)乘叶页记录数，再考虑键宽、行宽、填充率及缓存。两千万行不是分表硬门槛；是否拆分要看工作负载、索引宽度和 I/O。哈希适合等值但不提供同样的顺序范围能力。〔S9、N94、I72、I73〕

代码示例

```
page = read_index_page(); // InnoDB 页内按序保存键和记录引用
while (!leaf_page(page))
    page = page.child_for(key); // 高扇出，降低树高/随机 I/O
record = page.find_in_leaf(key); // 叶子有序，范围查询可顺序扫描相邻叶
// 聚簇索引叶子包含整行；二级索引叶子保存主键，必要时回表。
```

```
insert_new_key_may_split_page_and_write_more_pages(); // 写入/空间开销
// 哈希查找不天然支持排序/范围；B+ 树维护成本换范围能力。
```

007 什么情况下索引可能无法有效使用?

答案　索引是否有效取决于查询条件能否形成有用的查找范围。对索引列套普通函数或算术、跳过联合索引左侧列、前导通配符，常阻碍常规定位；函数索引等情形是例外。VARCHAR 和数字比较时可能按数值语义转换列，不能笼统说所有类型转换都失效。联合索引按字典序构造区间，遇到范围后后列通常不能继续缩小定位，但仍可能用于索引条件下推或覆盖。

LIKE 'abc%' 可能定位前缀，'%abc' 一般不能找起点，却仍可能扫描覆盖索引。低选择性、返回列多、统计信息过时或 OR 分支都可能让优化器选择扫描。索引扫描不等于高效定位，应结合 EXPLAIN 的计划和实际扫描行数判断，不能仅因为查询里出现索引列就断定会走高效索引。〔I64、I65、N99〕

代码示例

```
auto plan = db.explain("SELECT user_id FROM orders WHERE status=? AND created_at>=?");
create_index("orders", {"status", "created_at", "user_id"});
// 联合索引通常先按最左前缀定位；用真实数据和执行计划验证
```

008 如何分析并优化一条慢 SQL?

答案　先用慢查询日志找频繁或耗时 SQL，再用 EXPLAIN 或 EXPLAIN ANALYZE 看访问路径、估算行数与实际行数、循环、连接顺序、回表和排序。ANALYZE 会实际执行支持的语句，要评估风险。key_len 表示所用键前缀长度，不保证所有条件都精确定位；Using filesort 说明额外排序步骤，不代表一定写磁盘。

检查过滤与联合索引顺序、返回列、排序、深分页及 N+1 往返。例如按状态过滤并按时间排序，可评估匹配的联合索引。再在真实冷热分布和写入负载下测量，检查锁等待与索引维护成本，避免只优化单次基准。估算值和计划本身都不等于真实生产成本。〔I66、N99、N106、I89〕

代码示例

```
auto plan = db.explain_analyze(query, bind_values); // 实际执行，先确认查询副作用
compare(plan.actual_rows, plan.estimated_rows);
benchmark_before_and_after(query); // 同看延迟、扫描量、排序和写入成本
```

009 如何防止 SQL 注入?

答案　防止 SQL 注入的核心是参数化或预编译：SQL 结构先确定，用户输入作为数据绑定，数据库就不会把其中的引号或关键字重新解析为语法。例如 WHERE id = ?，不要把字符串拼接进 SQL。表名、列名和排序方向通常不能作为普通值参数，因此要从固定白名单映射，而不能直接接受任意输入。

参数绑定也让不同值复用同一语句结构，有助执行计划管理。输入校验和数据库最小权限是纵深防御；手工转义容易因字符集或上下文处理错误，不能替代参数化。应用还应限制查询范围和超时，防止合法但昂贵的输入耗尽资源。

代码示例

```
auto statement = db.prepare("SELECT id, name FROM users WHERE email = ?");
statement.bind(1, user_email);
auto rows = statement.execute(); // 输入作为值绑定
// 动态列名使用白名单映射，不能把用户输入拼成SQL结构
```

010 数据库事务的 ACID 是什么?

答案　ACID 描述事务的四类保证。原子性让事务整体提交或回滚；一致性表示提交前后满足数据库约束和业务不变量；隔离性规定并发事务的可见效果；持久性表示提交后能经受约定范围内的故障恢复。数据库会结合 undo、redo、锁、MVCC 和日志同步等机制实现这些性质。

ACID 不会自动修复错误业务规则，也不能脱离配置讨论耐久性：刷盘策略、复制确认、文件系统和存储设备都会影响故障后能否保留提交。业务还需明确哪些约束由数据库强制、哪些需要应用事务保障，并通过备份恢复演练验证承诺。

代码示例

```
Transaction tx(db);
auto source = tx.select_for_update(account_a);
if (source.balance < amount)
    tx.rollback();
tx.update_balance(account_a, -amount);
tx.update_balance(account_b, amount);
tx.commit();
```

011 数据库读写分离有哪些一致性风险?

答案　主库提交后，副本可能尚未接收或应用日志，所以立刻读副本可能看不到刚写的数据。故障切换也可能丢掉未复制或未确认的提交。复制正常不代表零延迟。需要读后写一致性时，可让该会话暂时读主库，或等待副本追到指定复制位点；如果允许最终一致，应向产品明确读到旧值的可能。

还要定义副本故障、切主和写超时后的路由，以及重试是否幂等。强读后写策略也应有最长等待时间，超时后有明确回退，而不是无限等副本。不能只看复制线程“运行中”，还要监控接收和应用位置差。

代码示例

```
primary.commit(write);
replica.apply_async(write); // 复制存在延迟
read_from_replica(); // 可能读到旧值，读己之写不一定成立
if (must_read_own_write)
    read_from_primary_or_wait_until_replica_lsn >= commit_lsn;
if (failover_occurs)
    verify_acknowledged_write_was_replicated();
// 还需处理复制中断、切换和读一致性目标；异步复制可能丢最近确认写。
// 按业务在延迟、可用性、读写扩展间取舍。
```

012 数据库隔离级别与常见并发异常是什么?

答案　四种常见隔离级别从宽到严是读未提交、读已提交、可重复读、可串行化，具体实现与代价因数据库而异。脏读是看到别人的未提交值；不可重复读是同一行重读得到新值；幻读是同一范围重读时结果集合因并发插入或删除而变化。隔离级别描述并发可见性，不同数据库的规则不能混作一套。

InnoDB RR 的一致性读通常复用首次读取建立的 ReadView；锁定读和 DML 则依据当前数据与锁处理。两种读不是同一个快照。事务自己的修改始终可见，所以快照先看不到他人新增行，随后 UPDATE 命中该行，再查便能看到自己的版本。快照视图与 next-key 锁分别处理不同场景，不能声称任意语句组合都绝无结果变化；需说明隔离级别、索引和读类型。〔S9、I61、I62〕

代码示例

```
Transaction t1(db, Isolation::repeatable_read);
auto snapshot = t1.select(stock_id);
Transaction t2(db);
t2.insert(new_product);
t2.commit();
t1.update_if_exists(new_product); // 快照读与当前读/DML要分别理解
```

013 缓存穿透、击穿、雪崩分别是什么?

答案　缓存穿透是反复查询根本不存在的数据，可在入口校验、短 TTL 缓存空结果，或用 Bloom Filter 预筛。Bloom Filter 有假阳性：说“可能存在”还要查缓存或数据库。缓存击穿是单个热点键刚失效，许多请求同时回源，可用互斥重建、逻辑过期和限并发处理。缓存雪崩则是大量键同时过期或缓存集群故障，需要错开 TTL、限流、降级并保护数据库。

三种情况的触发和治理不同。热点互斥粒度要落在单键，避免一把大锁让无关请求排队；重建请求也要有超时和失败回退。逻辑过期可在后台重建期间返回旧值，但只适合业务允许短暂陈旧的数据。

代码示例

```
value = cache.get(key);
if (value == MISS)
{ // 穿透：键不存在；负缓存可短暂记住空结果
    if (bloom.definitely_absent(key))
        return NotFound; // Bloom 有假阳性，不会证明存在
    if (try_singleflight_lock(key))
        rebuild_from_database_with_ttl_jitter();
    else
        serve_stale_or_wait_with_deadline(); // 单热点失效时避免并发回源
}
if (many_keys_expire_together || cache_cluster_down)
    rate_limit_and_degrade();
// 雪崩用 TTL 抖动/限流保护 DB；重建锁粒度要小并限制等待
// 对 Bloom 命中仍要查真实存储，过滤器不能代替数据源
```

014 InnoDB 的 undo log、redo log 和 MySQL binlog 分别有什么作用?

答案　undo、redo 和 binlog 作用不同但协同工作。undo 保存旧值，用于回滚和沿版本链构造 MVCC 快照；redo 记录 InnoDB 页修改信息，按 WAL 原则支持崩溃恢复；binlog 属于 MySQL Server 层，用于复制和基于备份的时间点恢复。未提交事务的脏页也可能写盘，恢复时先按 redo 重做，再用 undo 回滚未提交修改。

普通持久表的 undo 页更新同样受 redo 保护。redo 是循环复用并受 checkpoint 限制，binlog 按保留策略清理，不是永久完整历史。ROW、STATEMENT、MIXED 分别记录行变更、语句或按规则切换。长事务会拖延 undo 清理，刷盘配置和设备会影响日志耐久性；三类日志不可互相替代。〔S9、N95、N104〕

代码示例

```
old_value = undo_log.append_for_rollback_and_snapshot_visibility();
redo_log.append_page_change_for_crash_recovery();
binlog.append_logical_server_change_for_replication_and_point_in_time_restore();
// undo 服务回滚/MVCC 版本链；redo 服务 InnoDB 崩溃恢复；binlog 属 Server 层。
// 日志格式、生命周期和持久化配置不同，不能相互替代。
```

```
if (long_transaction)
    monitor_undo_history_cleanup_lag();
```

015 MySQL 为什么要协调 InnoDB redo log 和 binlog 的两阶段提交?

答案　一次事务提交要让 InnoDB redo 状态与 Server 层 binlog 保持一致，否则崩溃恢复和复制可能看到不同的事务集合。MySQL 采用内部两阶段提交：引擎先进入 prepare，Server 写入 binlog 并按配置持久化，再通知引擎提交。崩溃恢复可依据 prepare 状态和 binlog 是否有该事务，决定完成提交或回滚。
这解决的是两套日志的逻辑协调，不能替代刷盘策略或硬件保证。若掉电前日志只在易失缓存中，是否保留仍受 innodb_flush_log_at_trx_commit、sync_binlog 和设备行为影响。应把提交顺序、复制语义和耐久窗口一起配置及测试。〔S9〕
代码示例

```
engine.prepare(txn); // 引擎先 prepare，崩溃后可被协调
server.write_binlog(txn);
flush_binlog_according_to_sync_policy();
engine.commit(txn); // Server 确认日志后通知引擎提交
// 崩溃恢复查看 prepare 状态与 binlog 记录，协调提交或回滚
// 保持 InnoDB redo 与 Server binlog 的事务集合一致，服务恢复/复制
// 刷盘频率影响掉电耐久窗口；两阶段协调不等于硬件已经持久化
// 长事务/日志积压也会影响恢复与复制延迟
```

016 InnoDB 的 gap lock 和 next-key lock 是什么?

答案　Record lock 锁索引记录，gap lock 阻止在间隙插入，next-key lock 是记录加其前方间隙。不同事务的 gap lock 可以共存，但插入意向可能被已有 gap 锁挡住。RR 下锁定读和 UPDATE/DELETE 通常按实际扫描的索引区间加锁：完整唯一键命中常退化成记录锁，未命中可锁间隙；非唯一条件还会锁匹配记录和边界间隙，二级索引操作可能连带锁聚簇记录。
没有合适索引而全表扫描时，可能锁很多记录和间隙，不等于锁升级成表锁。RC 的搜索扫描通常不用间隙锁，但外键和重复键检查有例外。IS/IX 表级意向锁用于协调行级锁与表锁，不是插入意向锁。具体区间须看版本、执行计划及 data_locks/data_lock_waits，不能仅按 SQL 文本猜锁范围。〔S9、I62〕
代码示例

```
auto rows = db.query("SELECT * FROM items WHERE id BETWEEN ? AND ? FOR UPDATE", 12, 18);
Transaction other(db);
other.execute("INSERT INTO items(id) VALUES(?)", 15); // 间隙可能使插入等待
// 用索引、执行计划与锁视图确认实际锁范围
```

9.3 热点与大键治理

017 Redis 热 Key 和大 Key 有什么区别？如何排查?

答案　热 Key 是请求集中，可能让一个分片的 CPU、网络或串行命令处理成为瓶颈；大 Key 是体积或元素数很大，读取、删除、复制和持久化可能产生长停顿和内存峰值。一个键可以既热又大。热键可用访问采样定位，评估只读场景的本地缓存或副本分流；大键可限速扫描后结合类型、元素数和 MEMORY USAGE 判断。
--bigkeys 对集合主要看元素数，不等于按字节排行。拆分、分页和 UNLINK 可降低同步操作影响，但要考虑聚合、一致性和异步回收所需的内存余量。AOF write 仍可能占主线程，no/everysec 不保证完全无阻塞；fork 写时复制复制的是被修改的内存页，不是某次改键就必然复制整个键。〔S10、N116、N125、I87〕
代码示例

```
sample_key_access_frequency_and_per_shard_cpu_network(); // 热 Key：请求集中
measure_key_bytes_element_count_and_whole_key_operation_latency(); // 大 Key：单键操作重
if (hot_and_read_only)
    add_bounded_local_cache_or_read_replicas();
if (large_collection)
    split_by_bucket_and_page_operations();
// 大 Key 的整键读/删/复制可能阻塞分片并放大内存峰值
// 热 Key 拆读流量要考虑一致性；大 Key 拆分增加聚合和更新复杂度
// 调整后继续监控分片 CPU、网络、延迟和内存峰值
```

9.4 Redis 与缓存设计

018 Redis 的进程、事件循环、持久化与集群边界是什么?

答案　Redis 通常由主执行路径串行处理键空间命令，并用事件循环和 I/O 多路复用管理很多连接，避免常见命令间的锁竞争。网络 I/O、异步释放和后台任务可由线程辅助，RDB/AOF 持久化也可能涉及子进程，所以“Redis 整个服务永远单线程”不准确。长脚本、长命令和巨型响应仍会拖住其他请求。

RDB/AOF 管重启恢复和丢失窗口，副本与 Sentinel 管可用性，Cluster 把哈希槽分到多个主节点；这些是不同边界。客户端路由到单实例、副本组或集群时语义不同。热键、异步复制、长命令和故障切换都可能影响延迟或丢失写入，内存数据库不自动意味着持久或强一致。〔N77〕

代码示例

```
while (running)
{
    auto events = poller.wait();
    for (auto& event : events)
        dispatch(event);
}
client_a.send("GET key");
client_b.send("SORT large_set"); // 主执行路径上的慢命令会拖住后续命令
```

9.5 SQL 基础与数据建模

019 关系型数据库与 NoSQL 应怎样按数据访问方式选型？

答案　关系型数据库以表、键、约束和 SQL 为核心，适合多实体关联、复杂查询以及需要事务维护不变量的业务。NoSQL 是文档、键值、列族、图等不同模型的统称，按访问路径组织数据；它不代表没有模式、没有索引或没有事务。两类系统都需要考虑数据结构和约束。

先列出主要查询、数据规模、延迟目标、跨记录原子性及一致性要求，再比较索引、分片和运维成本。订单事实可放关系库，搜索视图或缓存可以用其他模型。不要仅因预期高并发就先放弃约束，也不要假设任何 NoSQL 都能无代价水平扩展。〔N78〕

代码示例

```
Transaction tx(sql_db);
tx.insert(order);
tx.insert(order_lines);
tx.commit(); // 关系库保存事务事实
search_index.upsert(build_search_document(order)); // NoSQL/搜索视图可异步更新
```

020 第一、第二、第三范式分别在消除什么数据依赖？

答案　第一范式要求每个属性按所选数据域保存原子值，避免在一列中塞进需独立查找的重复组。第二范式在此前提下，消除非主属性只依赖复合候选键一部分的情况；例如订单号和商品号共同标识明细，但商品名称只依赖商品号。第三范式再消除非主属性通过另一非主属性传递依赖候选键的情况。

规范化能减少插入、修改和删除异常，但并不意味着表越拆越好。为了读性能，可以有意识地在读模型保存冗余字段，前提是定义更新来源、事务或事件同步机制及一致性成本。判断范式时要先明确候选键和函数依赖，而不只是看有没有重复字段。〔N79〕

代码示例

```
struct Product
{
    ProductId id;
    std::string name;
}; // 商品属性依赖商品主键
struct OrderLine
{
    OrderId order;
    ProductId product;
    int quantity;
};
// 拆分后改商品名只需更新Product；冗余读模型要明确同步与重建方式
```

021 内连接、外连接和反连接各适合什么查询？

答案　内连接只保留满足连接条件的行组合；左外连接保留左表所有行，右表无匹配时补 NULL，右外连接对称。MySQL 没有原生 FULL OUTER JOIN，可把左连接和右连接用 UNION ALL 合并，再只取右连接中左键为空的未匹配部分。查“有课程 02 但没有课程 01”时可用 EXISTS 加 NOT EXISTS，避免多对多连接放大。

外连接右表条件放在 WHERE 或 ON 会改变结果：WHERE right.col = 值会过滤掉补 NULL 的左行，ON 中条件则保留它们。连接前检查一对一或一对多基数、外键和索引；否则重复行可能来自错误连接键而不是数据重复。不要用 DISTINCT 掩盖连接条件错误，它会增加处理成本并丢失需要的行。〔N80、N81〕

代码示例

```
auto rows = db.query(R"(SELECT student.id, course.id FROM student
LEFT JOIN course ON student.id = course.student_id AND course.id = ?)",
                     course_id);
// EXISTS/NOT EXISTS表达存在性，避免一对多JOIN放大行数
```

022 MySQL 字符串列的长度、字节数和 TEXT 上限怎样区分?

答案　CHAR(M) 和 VARCHAR(M) 的 M 表示字符数，不是字节数；utf8mb4 的一个字符最多占四字节。CHAR 偏固定长度，VARCHAR 另有长度开销，比较及尾部空格行为还受类型和排序规则影响。CHAR_LENGTH 返回字符数，LENGTH 返回字节数。TEXT 的普通类型上限约 64 KiB 字节，并有更小或更大的类型。

MySQL 65535 字节行大小限制还包括长度、NULL 等开销；BLOB/TEXT 在服务器行大小计算中计有限内部开销，不表示 VARCHAR 能装 65535 个字符。InnoDB 页内记录还有限制，变长值可能移到溢出页；页内外部指针大小与 SQL 层行大小计算不是同一个概念。设计时按字符语义选长度，再看索引、字符集和行格式。〔N82、N83、I67、I63〕

代码示例

```
std::string text = "猫A";
auto characters = utf8_character_count(text);
auto bytes = text.size();
// VARCHAR长度按字符定义，索引限制还要按字符集最大字节数计算
```

023 INT 的显示宽度会改变取值范围或存储空间吗?

答案　INT(1) 和 INT(10) 中括号里的历史值是显示宽度，不限制为一位或十位，也不改变 INT 的四字节存储和有符号取值范围。它与 VARCHAR(10) 表示最多十个字符完全不同。MySQL 8.0 已弃用整数显示宽度，旧版配合 ZEROFILL 补零也只是格式效果，不是数值精度或范围控制。

需要约束业务值时选择 TINYINT、SMALLINT、INT 或 BIGINT，并说明是否允许负数，再用数据库约束和应用校验表达有效范围。若编号需要前导零，应把它当作格式化文本在输出层处理；不要依赖整数列显示属性存储编号格式，也不要把编号当可计算的数量。〔N84〕

代码示例

```
int value = 12345;
std::string display = format_with_zero_padding(value, 8); // 展示补零
// INT(M)显示宽度不改变整数范围或本身占用的存储空间
```

024 金额列为什么通常使用 DECIMAL 或整数最小货币单位?

答案　FLOAT 和 DOUBLE 用二进制近似表示，许多十进制小数无法精确表示，累加、比较和舍入可能有意外。DECIMAL(p,s) 按十进制精确保存，p 是总位数、s 是小数位；也可以用整数保存分、厘等最小单位。金额和需要十进制规则的计量通常选这两种方案，而不是浮点。

数据库类型不能替业务决定税率精度、换汇时机和舍入方向。要定义币种、最大金额、小数位与舍入规则，验证溢出边界；跨服务传输也要保持单位和精度约定，避免先转为浮点再写回。〔N85〕

代码示例

```
using Cents = std::int64_t;
Cents source_balance = 12345, transfer = 1050, target_balance = 2000;
if (source_balance >= transfer)
{
    source_balance -= transfer;
    target_balance += transfer;
}
```

025 数据库应如何统一保存 IPv4 和 IPv6 地址?

答案　用 VARCHAR 保存 IP 文本直观，但同一 IPv6 地址有多种等价写法，空间大且字符串比较不是地址数值比较。MySQL 的 INET6_ATON 可把地址转成二进制，VARBINARY(16) 紧凑保存 IPv6 并支持精确比较，再用 INET6_NTOA 转回文本。函数处理 IPv4 时返回四字节，因此要统一约定地址族编码，不能混用四字节和十六字节形式。

IPv4 可用无符号整数，但无法承接 IPv6。若要做网段匹配、范围检索或黑名单，还要在统一的网络字节序上按前缀长度计算边界，并校验 IPv4 映射 IPv6 的规则。展示文本的字典序不等于地址数值顺序；格式规范化和地址族标记应在写入时确定。〔N86〕

代码示例

```
auto a = inet_pton(AF_INET6, "2001:db8::1", &binary_a);
auto b = inet_pton(AF_INET6, "2001:0db8:0:0:0:0:0:1", &binary_b);
bool same_address = std::memcmp(&binary_a, &binary_b, sizeof(binary_a)) == 0; // 比较二进制地址
```

026 外键约束怎样维护引用完整性，有哪些使用代价?

答案　外键约束要求子表引用能匹配父表键，或按定义允许为空；父表更新、删除可以拒绝、级联或把引用置 NULL。它由数据库维护引用完整性，能防止其他程序绕过应用写入孤儿记录，但不替代事务里其他业务规则。InnoDB 对相关键要求索引，级联可能扩大锁定范围和一次写入的工作量。

跨库、跨分片或跨服务通常不能保留同样的数据库级约束，需要事件、补偿和定期对账。是否使用外键取决于数据归属、写入路径和运维方式；不能笼统说外键会拖慢所有查询，也不能在没有替代一致性机制时随意删除约束。〔N87〕

代码示例

```
Transaction tx(db);
tx.insert("customer", customer_id);
```

```
tx.insert("orders", order_id, customer_id); // 外键保证父记录存在
// RESTRICT策略下仍被引用的客户不能删除；跨服务可用事件加对账维护
```

027 IN 和 EXISTS 的语义差异是什么，怎样避免 NOT IN 的空值陷阱?

答案　IN 判断一个值是否属于候选集合；EXISTS 只判断子查询是否至少返回一行，SELECT 中选哪一列通常不影响存在性。优化器可以把 IN 或 EXISTS 改写成半连接等计划，所以不能只根据“外表大”套用固定性能口诀。先保证关联条件、索引和 NULL 语义正确，再看执行计划和实测。

NOT IN 有典型空值陷阱：候选集合只要含 NULL，比较可能得到 UNKNOWN，而 WHERE 只保留 TRUE，结果看起来像没有匹配。NOT EXISTS 通常更直接表达“没有关联行”，也可以显式排除 NULL。

例如学生选课表的课程号允许 NULL 时，用 NOT EXISTS 检查“没有课程 2”比 NOT IN 更容易审查。若能保证子查询列 NOT NULL，两种写法结果才可按该假设比较；仍要看执行计划，而不是机械改写。〔N81〕

代码示例

```
auto unsafe =
    db.query("SELECT id FROM student WHERE course_id NOT IN (SELECT course_id FROM chosen)");
auto safe = db.query("SELECT s.id FROM student s WHERE NOT EXISTS (SELECT 1 FROM chosen c WHERE "
                     "c.course_id=s.course_id)");
// NOT IN子查询含NULL时可能得到UNKNOWN；先确认列的空值语义
```

028 SQL 常见标量函数应怎样使用才不改变业务语义?

答案　SQL 标量函数可处理字符串、数值、日期和条件表达式，例如 CONCAT、SUBSTRING、ROUND、ABS、CASE、COALESCE。使用时要确认 NULL 如何传播、字符串按字符还是字节计算、舍入规则和会话时区。不能因函数名直观就假设空值和边界符合业务语义。

对索引列套普通函数可能妨碍常规索引范围查找。例如把时间列格式化后再比日期，通常改成明确的半开区间更容易利用索引；若存在函数索引或生成列，则另行评估。测试要包含时区边界、NULL 和精度样例。

COALESCE 只在值为空时取后续参数，不能用来掩盖必须拒绝的缺失数据；ROUND 的规则也应和财务约定一致。把函数用于 SELECT 展示和把它放进 WHERE 条件，对索引利用的影响不同。〔N88〕

代码示例

```
auto start = to_utc(local_day_start);
auto end = to_utc(next_local_day_start);
auto rows = db.query("SELECT * FROM orders WHERE created_at >= ? AND created_at < ?", start, end);
// 半开区间可保留时间列上的索引范围扫描
```

029 COUNT 和 NULL 的组合会怎样影响统计结果?

答案　NULL 表示未知或缺失，不等于空字符串；不能写 value = NULL，应使用 IS NULL。COUNT(*) 和 COUNT(1) 都统计结果行数，InnoDB 的处理相同；COUNT(col) 只数非 NULL，COUNT(DISTINCT col) 统计非 NULL 去重值。左连接统计子项时，应数子表非空键，否则 COUNT(*) 会把无匹配的左表行也算进去。

InnoDB 要按事务可见性计算精确行数，不能只返回一个全局计数。无 WHERE/GROUP BY 的 COUNT(*) 通常扫描最小可用二级索引，没有二级索引才扫聚簇索引，仍有成本。近似规模可看统计信息；精确计数表要与增删事务一致并避免热点。把 COUNT(*) 改为 COUNT(1) 不会自动提速。〔N89〕

代码示例

```
int rows = db.query_value("SELECT COUNT(*) FROM student LEFT JOIN course ON ...");
int matched = db.query_value("SELECT COUNT(course.id) FROM student LEFT JOIN course ON ...");
// COUNT(*)数行；COUNT(nullable_column)跳过NULL
```

030 DELETE、TRUNCATE 和 DROP 的事务及对象语义有何区别?

答案　DELETE 按条件删行并保留表结构；InnoDB 可在事务中回滚，但删除越多，日志、锁和执行时间通常越大。TRUNCATE 清空整表，是 DDL 操作，通常会隐式提交，不能当普通 DELETE 回滚，还会重置自增计数并受外键引用限制。DROP 则删除表对象和数据，后续不能把它当原表继续使用。

应先确认是否要保留结构、是否按条件清理、是否有引用及恢复方案，再选操作。大表清理可评估分批 DELETE，控制锁持有和日志增长。误执行 DDL 不能靠应用重试恢复，生产前要明确对象、备份和恢复点。〔N90〕

代码示例

```
if (keep_recent_rows)
    db.execute("DELETE FROM test WHERE created_at < ?", cutoff);
else if (clear_all_rows)
    db.execute("TRUNCATE TABLE test"); // DDL通常隐式提交
// DROP TABLE还会删除定义；破坏性操作前核对库表和恢复点
```

031 SELECT 的逻辑处理顺序与实际执行计划有什么不同?

答案　为了推理查询语义，可按 FROM/JOIN、WHERE、GROUP BY、HAVING、窗口计算、SELECT/DISTINCT、ORDER BY、LIMIT 的逻辑阶段理解。WHERE 在分组前筛行，HAVING 在分组后筛组，窗口函数在分组之后计算但保留各行。逻辑顺序也解释了为什么 SELECT 别名不能在所有子句里直接使用；MySQL 有自己的别名扩展，应看具体语法规则。

这只是语义模型，不是优化器逐阶段执行的流水线。只要结果相同，优化器可以下推条件、重排内连接或改写子查询。要知道真实访问路径和成本，需要读 EXPLAIN，并按实际数据观察。〔N91、N92〕

代码示例

```
std::erase_if(orders,
              [](const Order& x)
              {
                  return x.status != Status::paid;
              });
auto totals = group_by_user_and_sum(orders);
std::sort(totals.begin(), totals.end(), by_total_desc_then_user_id); // LIMIT最后作用于排序结果
```

032 按学生总分取第 5 到第 10 名，怎样定义并列与分页?

答案　先按学生聚合总分，再对聚合结果排名，最后取第五到第十名；若先对成绩明细 LIMIT，再汇总，得到的不是学生总分排名。ROW_NUMBER 给每行不同序号，适合必须恰好取六名且有稳定排序的场景；总分相同时加学生 ID 作为最终排序键。RANK 并列共享名次且后续跳号，DENSE_RANK 并列后不跳号，按业务含义选择。

窗口函数结果通常不能在同层 WHERE 中直接过滤，可放到 CTE 或子查询。还要明确 NULL 成绩、重复课程成绩、补考是否覆盖等计分规则，否则 SQL 写得再正确也可能产生错误名次。〔N92〕

代码示例

```
std::sort(totals.begin(),
          totals.end(),
          [](const auto& a, const auto& b)
          {
              return a.total != b.total ? a.total > b.total : a.student_id < b.student_id;
          });
auto page = std::span(totals).subspan(4, 6); // ROW_NUMBER时用稳定次序定义第5到第10名
```

9.6 MySQL 引擎与存储

033 一条查询或更新怎样经过 MySQL Server 与 InnoDB?

答案　MySQL 请求先经过连接认证，Server 层解析 SQL、检查权限并由优化器选访问路径；执行器按计划向存储引擎请求数据。InnoDB 从 Buffer Pool 取页，普通一致性读可能沿版本链找快照，更新会使用锁并产生 undo、redo，提交时再协调 binlog 与 redo。MDL、行锁和连接等待可能发生在不同阶段，MySQL 8.0 已移除旧查询缓存。

连接池能摊薄建连和认证成本，但需限制总量及借用等待。Sleep 只表示当前没有执行语句，仍可能有未提交事务或会话状态。复用前要清理事务、autocommit、隔离级别、字符集、临时表和用户变量；mysql_reset_connection 可保留网络连接但回滚事务、重置会话并释放预处理语句等资源，应遵循驱动池约定。空闲连接受 wait_timeout 影响。排障分别看池、计划、Buffer Pool、事务锁和日志刷盘，不凭总耗时归因。〔N93、N94、N95、I70、I71〕

代码示例

```
auto connection = pool.acquire();
auto plan = connection.prepare(update_sql);
plan.bind(id, status);
plan.execute();
connection.commit(); // Server解析/优化，InnoDB管理页、锁和日志
connection.reset_session();
pool.release(connection); // 归还前确认没有未提交事务
```

034 InnoDB 与 MyISAM 在事务、索引和并发上怎样取舍?

答案　InnoDB 提供事务、崩溃恢复、MVCC、行级锁和外键，适合现代 MySQL 的通用业务；聚簇索引保存整行，二级索引保存主键，因此主键越宽，各辅助索引也会变大。MyISAM 不提供同等事务和恢复语义，数据和索引分文件，写入主要依赖表级锁，只有少数明确兼容场景才考虑。

不要只因 MyISAM 的某类 COUNT 看起来快就迁移业务表。InnoDB 的“行锁”也不表示每条语句只锁一行，范围、索引和隔离级别决定锁范围。比较引擎时应一起看并发读写、故障恢复、一致性与备份方案。〔N93〕

代码示例

```
Transaction first(db), second(db);
first.update("orders", 10, Status::shipped);
second.update("orders", 11, Status::paid);
first.commit();
second.commit(); // InnoDB行级并发；引擎选择仍要看事务与恢复需求
```

035 MySQL 表结构和数据文件的布局为什么必须区分版本?

答案　数据文件布局必须带 MySQL 版本和表空间配置一起讨论。MySQL 5.7 常见 .frm 保存表定义，启用独立表空间时 InnoDB 的 .ibd 保存数据和索引；MySQL 8.0 起采用事务型数据字典并移除 .frm。InnoDB 还可用系统、通用、独立、undo 和临时表空间，因此“一张表一个 .ibd”并非任何配置下都成立。

排查前确认版本、datadir、引擎和表空间；表文件扩展名不能说明所有元数据和事务状态都在那里。备份迁移应使用受支持的一致性工具或流程，单独复制正在运行中的一个文件不能保证与字典、redo 和其他表空间匹配。〔N96、N97〕
代码示例
```
auto version = query_mysql_version();
auto datadir = query_variable("datadir");
auto tablespace = inspect_tablespace(table_name, version);
backup_consistently(datadir, tablespace);
// 先核对版本、表空间归属和备份协议，不直接复制运行中的单个.ibd
```

036 Buffer Pool 怎样减少磁盘访问，脏页又在何时落盘？

答案　可以把 Buffer Pool 想成 InnoDB 在内存里的工作桌。数据和索引按“页”放在桌上；读到不在桌上的页，才从磁盘取来。修改时先在内存里改这张页，并标记为“脏页”，意思是内存内容和磁盘版本不同。redo log 另记下这次修改，用来在崩溃后重做，因此提交通常不必立刻把整张数据页写回磁盘。后台会在刷脏、推进 checkpoint 或腾出空间时写回。
Buffer Pool 管理页时有几种不同视角：free list 是还没装内容的空位；LRU 按最近使用安排已缓存页；flush list 记录需要写回的脏页，同一脏页也同时属于 LRU。LRU 分 young/old，新页和预读页先放 old，短时间复用后才升到 young；innodb_old_blocks_time 可防一次性扫描挤掉热点。比如账户页改余额后先提交 redo，再稍后刷数据页，既能较快提交，也能在崩溃后重做。观察缓存命中、脏页和等待，并给其他进程留内存。〔N94、I68〕
代码示例
```
buffer_pool.touch(hot_page); // 热页留在young区域
for (Page page : table_scan)
    buffer_pool.add_to_old(page);
if (reused_within(page, old_blocks_time))
    buffer_pool.promote(page); // 一次性扫描不污染热区
```

9.7 MySQL 索引补充

037 InnoDB 主键为何偏好短且稳定，自增值为什么会有空洞？

答案　主键像每行的主要定位标签。在 InnoDB 中，聚簇索引叶页存整行，二级索引还要带主键才能找到行；主键越长，很多二级索引都会跟着变胖。短且稳定、写入位置较连续的主键通常省空间并减少随机页分裂。自增值像号码牌：事务申请后再回滚，号码通常不会收回来，失败插入和批量预分配也会留下空号，所以最大 ID 不是行数，空洞也不代表删过记录。
innodb_autoinc_lock_mode 决定插入语句怎样分配号码。模式 0 用传统语句级 AUTO-INC 锁；模式 1 对可预知行数的简单插入减少互斥，批量插入仍会串行分配；模式 2 允许不同语句交错取号，吞吐较好，但基于语句的复制应选 0/1 并保持主从设置一致。MySQL 8.4 默认模式 2。这个锁通常持有到当前插入语句结束，不是等整个事务提交。财务流水若要求连续编号，应另建编号协议，不要依赖主键。〔N98、N97〕
代码示例
```
Transaction tx(db);
auto id = tx.next_auto_increment();
tx.insert(id, row);
tx.rollback();
auto next_id = db.next_auto_increment(); // 回滚后可留下空洞，ID差值不等于行数
```

038 覆盖索引与索引条件下推分别减少哪一类工作？

答案　覆盖索引指查询需要的所有列都能从选中的索引中取到，不必再访问聚簇记录补列；它取决于查询投影与索引包含的列。索引条件下推则把可在索引记录上判断的条件交给存储引擎先过滤，只有通过者再回表读取整行。两种优化可以同时发生，但解决的工作不同。
复合索引后面的列即使不能进一步缩小定位范围，仍可能用于过滤或覆盖，不应叫作完全失效。EXPLAIN 的访问路径和 Extra 可以提供线索，索引过宽也会增加写入成本、缓存占用和磁盘空间。〔N99〕
代码示例
```
auto rows = db.query("SELECT last_name, age FROM users WHERE last_name=? AND age>?");
// (last_name, age)索引可覆盖此查询；请求address时需回表
```

039 字符串前缀索引怎样节省空间，有哪些功能限制？

答案　前缀索引只保存字符串开头若干字符或字节，常用于很长的文本以节省索引空间；具体计量方式受字符串类型影响。前缀越短，重复越多，查到候选后仍要比完整值。应在真实数据上测区分度，不要任意选长度。TEXT/BLOB 普通索引通常需要指定前缀，也受字符集和引擎的索引字节上限约束。
前缀索引未必能覆盖完整字符串输出或证明整列完全相等，排序、分组利用也取决于查询。唯一前缀只保证前缀唯一，会拒绝开头相同但后面不同的完整值，因此不能拿它替代完整列唯一性。〔N100〕
代码示例

```
for (int prefix : {8, 16, 32, 64})
    report(prefix, distinct_left(url_column, prefix) / total_rows);
auto matches = db.query("SELECT url FROM files WHERE url LIKE ?", prefix + "%");
// 前缀索引命中后仍需比较完整字符串，选择前先测区分度
```

9.8 MySQL 锁与恢复

040 数据库发生死锁后应用应怎样恢复，如何降低再发生概率？

答案　死锁是事务形成循环等待，单纯延长锁等待时间不会消除环。InnoDB 通常检测后回滚一个受害事务，应用应识别死锁错误，回滚并有界重试整个事务，不能只重放最后一条 SQL。两个事务即使插入不同主键，也可能因共享间隙锁或唯一键重复检查形成等待环。

用 SHOW ENGINE INNODB STATUS、data_locks 和 data_lock_waits，结合完整语句、索引和执行计划取证。统一多表或多行访问顺序、缩短事务、补合适索引可减少概率。死锁错误与锁超时的默认回滚范围可能不同，失败后要明确回滚整个业务事务再重试。关闭死锁检测会改变失败等待方式，应理解其代价。〔N101、S9、I62〕

代码示例

```
try
{
    update_in_order({account_a, account_b});
}
catch (DeadlockError&)
{
    rollback_entire_transaction();
    retry_with_backoff(idempotency_key);
}
// 重试完整业务事务；统一锁顺序可减少死锁概率
```

041 为什么 ALTER TABLE 可能让后续普通查询也排队？

答案　MDL（元数据锁）让语句使用表期间的结构保持一致。已有事务持有兼容锁时，要求更强锁的 ALTER TABLE 可能等待；一旦排他 MDL 等待队列形成，后到的普通查询也可能排在它后面，看上去整张表都堵住。根因常是一个没有结束的长事务，而不只是 DDL 执行慢。

观察 performance_schema.metadata_locks、会话和事务状态，分清真正阻塞者、DDL 等待者及被队列挡住的查询。只杀排队查询不一定解除源头。上线前控制长事务并验证具体版本的 Online DDL 能力；LOCK=NONE 不表示完全不申请任何 MDL。〔N102〕

代码示例

```
Transaction long_tx(db);
long_tx.select("orders", 1); // 保持MDL共享锁
AsyncResult ddl = db.execute_async("ALTER TABLE orders ADD COLUMN note TEXT");
// 等待中的DDL可能让后续查询排队；检查metadata_locks并结束阻塞事务
```

042 innodb_flush_log_at_trx_commit 的三个值如何影响故障丢失窗口？

答案　innodb_flush_log_at_trx_commit=1 要求每次提交相关 redo 写出并同步；2 在提交时写到文件层，通常由后台周期同步；0 提交本身不要求立即写或同步。进程崩溃、OS 崩溃和掉电影响不同；0/2 不能保证严格最多丢一秒，后台调度和设备行为会改变窗口。

binlog 还要看 sync_binlog：0 交给系统控制，1 对每个提交组同步，N>1 累积提交组后同步，并非每 N 个事务。双 1 可加强事务与复制日志的耐久，但仍依赖存储设备诚实执行刷新。组提交可摊薄 fsync 成本，不替代备份和恢复演练；也要用相同的故障类型比较配置，而不是把进程崩溃等同掉电。〔N103、N95、I69〕

代码示例

```
configure("innodb_flush_log_at_trx_commit", 1);
configure("sync_binlog", 1);
Transaction tx(db);
tx.update(account_id, -10);
tx.commit();
// 故障注入后检查已确认事务；设备写缓存也必须兑现flush语义
```

043 Redo 空间将耗尽时，为何 checkpoint 和刷脏会限制写入？

答案　Redo 是容量有限、循环复用的日志，不能覆盖崩溃恢复还需要的记录。Checkpoint 表示相关页面修改已推进到足够的落盘位置，旧 redo 才能复用。若写入产生 redo 的速度超过脏页刷盘，checkpoint 推进不足，系统会加快刷脏甚至限制前台写入，表现为延迟上升。

应观察 redo 使用量、checkpoint age、存储吞吐和脏页压力。增大日志容量可缓冲短时峰值，却不能掩盖长期写入超过设备能力。WAL 把提交关键路径从随机写多个数据页转为顺序日志写，但数据页最终仍要落盘，并没有消除存储 I/O；因此要把日志和数据盘能力一起评估。〔N104〕

代码示例

```
redo_generated_per_second = 500;
dirty_pages_flushed_per_second = 200;
if (redo_generated_per_second > dirty_pages_flushed_per_second)
    monitor_checkpoint_age();
// 扩大redo空间只能缓冲峰值，长期写入超出刷盘能力仍会限流
```

044 组提交怎样提高日志刷盘效率，代价是什么？

答案　组提交让多个并发事务共享一次日志同步，把独立 fsync 的等待和 I/O 成本分摊到一批提交上。Server 的 binlog 提交流水线按阶段聚合工作，并与 InnoDB 的提交状态协调。它提高的是日志刷盘效率，不是把多个业务事务变成一个可一起回滚的事务。

等待更多事务凑批可能提高吞吐，也可能让单个事务多等一会儿；低并发时未必受益。评估要同时看吞吐、P99 提交延迟、磁盘同步时间和耐久参数。关闭刷盘取得的高性能不能证明组提交自身的收益，应在相同耐久要求下比较。〔N95、N103〕

代码示例

```
CommitGroup group = collect_ready_commits({tx_a, tx_b, tx_c});
fsync(group.binlog);
fsync(group.redo_log);
publish_each_commit_result(group);
// 多个并发提交共享同步开销；测吞吐时也比较排队延迟
```

045 InnoDB doublewrite 为什么不能简单地由 redo 替代？

答案　数据库页通常大于底层保证的原子写入单位，崩溃时写到一半可能形成撕裂页。redo 描述对页的修改，不能保证在任意损坏的基页上安全重放。Doublewrite 先把待刷页写入受管理的双写区域并确保副本有效，再写目标文件；恢复时用完整副本修复损坏页，之后才应用 redo。

双写保护页写完整性，redo 的提交日志保护事务恢复，两者互补。MySQL 版本和存储配置会演进，设备支持也要验证；不能因 SSD 快就假设数据库页原子写并随意关闭保护。恢复演练还要确认双写区与数据文件处于预期故障域。〔N105〕

代码示例

```
write_doublewrite(page);
fsync(doublewrite_file);
write_data_page(page); // 防止崩溃造成页撕裂
if (!valid_checksum(data_page))
    restore_from_doublewrite(page);
apply_redo_records(page); // doublewrite修复页，redo重放逻辑修改
```

9.9 MySQL 查询与复制补充

046 深分页为什么常变慢，如何用游标方式继续读取？

答案　LIMIT offset,count 往往要读过并丢弃前 offset 条结果，偏移越大，扫描和回表成本越高。数据并发变化时按页号翻页还可能重复或跳行。若产品只需“下一页”，可用稳定且唯一的排序键记录上一页末尾，再按范围继续，例如创建时间加 ID 的复合游标。

若必须跳到任意页，可考虑覆盖索引先取主键再回表、限制最大深度或预先计算排序结果，但这些不能自动消除偏移扫描。游标要包含全部排序字段并处理相同值、方向和筛选条件；不能仅按 ID 排序取代业务的真实排序。〔N106、N91〕

代码示例

```
using Cursor = std::pair<TimePoint, OrderId>;
auto rows =
    db.query("SELECT * FROM orders WHERE (created_at,id) > (?,?) ORDER BY created_at,id LIMIT ?",
             cursor,
             20);
auto next_cursor = Cursor{rows.back().created_at, rows.back().id}; // 稳定翻页
```

047 MySQL 复制、半同步与 GTID 分别解决什么问题？

答案　典型 MySQL 复制由源端写 binlog，副本接收后写 relay log，再由应用线程执行；接收和应用是两个进度，所以收到日志不代表查询已可见。异步复制的提交不等待副本，半同步可等待副本确认日志接收，但通常不等于应用完成，也不自动保证无损切主或线性一致读取。

GTID 给事务标识，可用已执行集合帮助重新定位复制起点；它不是 binlog 文件偏移，也不消除复制延迟。应一并设计确认条件、超时后退化方式、切主候选和读后写路由。监控复制接收与应用差距，并通过故障切换测试实际可能丢失的提交。〔N107、N108、N109〕

代码示例

```
binlog.append(gtid, transaction);
replica.receive_to_relay_log(gtid);
if (semi_sync_acknowledged(gtid))
```

```
    reply_commit_to_client();
replica.wait_until_executed(gtid); // 收到relay log确认不等于已应用，可读主库或等待GTID
```

048 分库分表怎样选择分片键并规划扩容？

答案　分表把数据拆到不同表，分库进一步分散到不同实例，可扩展单节点容量和吞吐、缩小故障影响范围，但跨片查询、关联、全局唯一和事务会更复杂。分片键应同时满足主要查询路由、数据分布均衡和增长模式。若按买家 ID 分片，但查询只有订单主键，要么带上分片信息，要么维护路由映射或冗余键，否则只能广播。

浅页全局排序可让每片返回 offset+limit 条，再汇总排序，成本随偏移和分片数增加；连续翻页更适合稳定复合游标。任意深页可建立独立排序或搜索索引。扩容需准备迁移批次、增量追平、路由版本、校验及回滚，并提前定义跨片聚合、对账和事务边界。〔N110〕

代码示例

```
auto shard = router.for_key(order.buyer_id);
shard.insert(order);
copy_shard_range(old_shard, new_shard, range);
verify_count_and_hash(range);
router.switch_version(range, new_shard); // 双写/追增量及回滚需在切流前准备
```

9.10 Redis 内部结构与执行模型

049 Redis 的大规模有序集合为什么使用跳表配合字典？

答案　以 Redis 7.2 的典型实现为例，大型 ZSet 同时用字典按成员平均 O(1) 查分数、跳表按 score 排序，同分时按成员字节序排列。小集合可能编码为 listpack，不能把大结构的复杂度直接套给它。跳表从高层快速跳跃，再逐层下降；随机层高使查找、插入、删除期望 O(log n)，取出 m 个范围项通常 O(log n+m)，最坏仍可能线性。

每层跨度记录跨过多少底层节点，累积跨度可算名次，避免从头遍历；插入也不需整体移动数组。多层指针和跨度需要额外内存，因此它是针对内存排名与范围访问的取舍，不保证在磁盘访问或所有负载中胜过平衡树或 B+ 树。编码阈值和层高常量要按目标版本核对。〔N111、N155〕

代码示例

```
zset.add(90, "Alice");
zset.add(90, "Bob");
zset.add(70, "Carol");
auto rank = zset.rank({90, "Bob"}); // 跳表按分数和成员顺序定位，字典负责成员查找
```

050 listpack 等紧凑编码如何节省内存，为什么不适合无限增大？

答案　紧凑编码把小元素连续存放，减少指针、分配器元数据和对象头，也改善局部性；代价是查找常要扫描，插入或扩容会搬动内容。旧 ziplist 用每项记录前项长度，字段扩展可能导致后续项连锁更新；listpack 每项尾部编码自身长度，支持反向解码，避免这种级联。

实现随版本改变：Redis 6.2 的小 Hash/ZSet 常用 ziplist，7.0 改 listpack；quicklist 在 7.0 前串 ziplist，之后普通压缩节点用 listpack，大单元素可为 plain 节点；7.2 小 List 可直接 listpack，Set 也加入 listpack。小型整数 Set 可用 intset：有序数组二分查找 O(log n)，插入、删除或升级可 O(n)；升级时整体换成更宽整数编码，7.2 删除不自动降宽。超过阈值或加入非整数会按条件转 listpack/字典，这与 intset 内部位宽升级是两件事。用 OBJECT ENCODING 核对版本与配置，转换和搬移可能产生延迟。〔N112、I79、I80、I81、I82〕

代码示例

```
IntSet values{16};
values.insert(100);
values.insert(200);
values.insert(70000); // 超出16位范围时整体升级编码
// 紧凑编码省内存，但扩容/编码转换会搬移数据
```

051 Redis 字典扩容为什么要渐进迁移，迁移期间怎样读写？

答案　Redis 字典扩容时，若一次把所有桶搬到新表，可能让处理请求的线程停很久。渐进 rehash 会同时保留旧表和新表，把搬迁分散到后续操作中，直到旧表清空再结束。过渡期间查找要按规则检查两张表；新增通常进新表，更新和删除也要处理记录仍在旧表的情况。

每次迁少量桶可缩短单次停顿，但会增加暂时的内存和查找工作，碰撞严重时仍可能变慢。“渐进”不是零开销。这里说的是 Redis 实例内部字典扩容，不是把集群槽从一个节点迁到另一个节点。〔N113〕

代码示例

```
Dictionary dict;
dict.start_rehash(new_capacity);
while (dict.rehashing())
    dict.move_some_buckets(4); // 每条命令渐进搬迁
// rehash期间查找新表，也查尚未迁移的旧表桶
```

052 SDS 与 C 风格字符串相比怎样支持二进制值和高效增长？

答案　SDS 在字符串数据前保存长度和容量，长度不必像 C 字符串那样扫描到第一个零字节，所以可安全保存内嵌零字节的二进制内容。末尾仍保留一个终止符，方便部分 C 接口使用；逻辑长度始终由 SDS 元数据决定。追加前可检查空余容量并预留空间，减少反复求长和频繁重新分配。

以 Redis 7.2 的 String 对象为例，合适整数可用 int 编码，小字符串可用紧凑的 embstr，较大或修改后的对象可用 raw。这些是内部对象编码，具体阈值会随版本实现变化；应用按 Redis 命令语义操作，不应依赖内存布局。〔N114〕

代码示例

```
Sds value = make_sds({65, 0, 66}); // 长度字段允许中间包含零字节
value.append({67}); // 按length读写，不用strlen寻找第一个零
```

053 Redis 的高吞吐来自哪些因素，多线程负责什么？

答案　Redis 常见高吞吐来源包括：数据主要在内存，合适结构避免昂贵存储访问；事件循环和 I/O 多路复用可同时管理许多连接；主要命令路径串行操作键空间，少了常见锁竞争。网络往返、序列化和命令复杂度仍然有成本，长脚本或巨型返回也会拖住其他请求。

现代 Redis 会在线程中处理部分网络 I/O、异步释放和后台任务，RDB/AOF 还可能用子进程，所以“Redis 永远只有一个线程”不准确。增加 I/O 线程不会让任意慢命令并行，也不代表所有数据操作都由多个线程执行。判断性能要注明版本、命令、数据大小和持久化配置。〔N115、N116〕

代码示例

```
EventLoop loop;
loop.on_read(client, read_request);
loop.run_one_iteration();
execute_command("SORT", large_key); // 慢命令仍占用主命令执行路径并延迟其他客户端
```

9.11 Redis 事务与生命周期

054 MULTI/EXEC、WATCH、Lua 与 Pipeline 在原子性上怎样区分？

答案　MULTI 将命令排队，到 EXEC 才顺序执行命令组，期间不会插入其他客户端命令；但某条命令在执行时出错，不会自动回滚此前成功的命令。WATCH 监视键，若执行前键值变化，EXEC 会放弃，适合先读后改的乐观并发，需要重读并重试。Lua 把判断和写入放在服务端连续执行，但运行期错误同样不等于数据库式事务回滚，长脚本还会占住执行路径。

Pipeline 只是批量发送以减少网络往返，本身不保证原子性或隔离；它可与事务组合。处理库存等共享状态时先决定使用原子脚本、事务或条件更新，再明确失败、重试和幂等语义，不能依赖客户端发送顺序。〔N117、N118〕

代码示例

```
pipeline.send("GET", key);
pipeline.send("DECRBY", key, amount); // Pipeline只减少往返
if (watch(key) && exec_transaction(commands))
    return success;
// WATCH冲突后重读重试；Lua可把检查和更新放入原子脚本
```

055 键过期与内存淘汰有什么区别，更新值会保留 TTL 吗？

答案　过期是键自己的失效时间，淘汰则是在 maxmemory 压力下按策略驱逐；一个尚未到期的键也可能先被淘汰。Redis 结合访问时检查和有时间预算的主动扫描回收，逻辑失效不保证同一时刻内存立即释放。扫描轮次、样本和预算随版本、配置变化，不能把旧文章中的固定数量或比例当永久规则。

复制时由主节点过期并传播删除；副本即使暂时保存该键，普通读取也会把逻辑上已过期键视为不存在，升主后自行处理。INCR、HSET 等原地修改通常保留 TTL，普通 SET 覆盖会清掉旧 TTL；要显式带过期选项或 KEEPTTL。写入不会自动续期，滑动过期要由应用实现。〔N119、N120〕

代码示例

```
redis.set("session", value, expiry = 60s);
redis.hset("profile", "name", "Aki");
redis.set("session", new_value, keep_ttl = true); // 是否保留TTL由命令选项决定
// 过期是逻辑有效性；内存淘汰是容量策略
```

9.12 Redis 复制、集群与缓存

056 Redis 全量复制和部分重同步怎样衔接，复制能保证强一致吗？

答案　Redis 复制流用 replid 标识历史、offset 标示进度；不能用普通进程 run_id 代替。断线回来时，如果副本的历史 ID（包括可接受的旧 ID）和缺口范围仍在主节点 backlog 中，可部分重同步；否则传全量快照，再补传期间写入。全量快照可以经磁盘或无盘方式发送。

复制积压区留近期历史，副本输出缓冲保存尚待该副本消费的数据；超限可能断开连接，重连能否部分同步仍由 ID 和 offset 决定。Redis 7 的底层缓冲块可共享，不能机械认为每个副本一份独立字节副本。可按复制流速率乘可容忍断线

时间并加突发余量规划 backlog。默认异步复制切主可能丢已确认写；WAIT 和 min-replicas 可降低部分风险，仍不提供线性一致，也不能替代持久化与备份。〔N120、I83〕
代码示例
```
if (replica.offset_is_in_backlog(replid, offset))
    send_partial_sync(replica);
else
    send_full_snapshot_then_backlog(replica);
// 异步复制可能丢失未到副本的已确认写入，不提供强一致保证
```

057 Sentinel 怎样判定故障并切主，脑裂风险为何仍然存在？

答案　Sentinel 负责监控、通知和协调切主，不负责数据分片。单个 Sentinel 可将主节点标成 SDOWN；达到 quorum 个 Sentinel 同意后，才视为 ODOWN。真正执行切换还需获得 Sentinel 多数授权，所以 5 个 Sentinel、quorum=2 时，只有 2 个能通信可以判故障，却不能凑到多数授权。候选副本要满足资格，再按优先级、复制偏移及 ID 等选择。分区期间旧主可能仍为部分客户端服务，异步复制会让切主后丢写。min-replicas-to-write 和 max-lag 可以减少风险，但会牺牲可用性，不能保证零丢失。客户端仍要发现新地址、重建连接并以幂等方式重试。要区分故障判定、切换授权、候选提升和客户端更新四个阶段。〔N121、N120〕
代码示例
```
if (sentinel_votes_down >= quorum)
    mark_objectively_down(master);
if (sentinel_votes_down >= majority(sentinel_count))
    authorize_failover();
// 多数授权受网络分区影响；切换期间业务仍需幂等和防双主措施
```

058 Redis Cluster 如何按槽路由，MOVED、ASK 和 CROSSSLOT 表示什么？

答案　Redis Cluster 用 CRC16 将键映射到 16384 个槽，再把槽分配给主节点，副本用于故障转移；客户端可缓存槽到节点的路由。MOVED 表示持久归属变了，应更新路由；ASK 是迁移中的一次临时转向，客户端先发 ASKING，不能永久改映射。迁槽逐个搬 key，大键可能拖慢迁移。
多键原子命令和脚本通常要求所有键同槽，hash tag 可让键共槽，但可能制造热点。集群客户端可按节点拆 Pipeline，Pipeline 不要求所有键同槽，也不提供跨槽原子性。从副本读取要使用 READONLY 并接受复制延迟。故障副本要有资格且需持槽主节点多数授权提升；分片不消除热键、无副本时槽不可用或异步复制丢失窗口。〔N122、N118〕
代码示例
```
auto slot = crc16("order:{42}:state") % 16384;
if (reply.is_moved())
{
    slot_map.update(reply.slot, reply.node);
    retry();
}
if (reply.is_ask())
{
    send_asking(reply.node);
    retry_once();
} // ASK不更新永久槽表
```

059 本地缓存和共享 Redis 缓存怎样组合而不过度牺牲一致性？

答案　本地缓存省去网络往返，适合小而热且允许短暂陈旧的数据，但每个进程各存一份，更新传播、冷启动和内存上限都要处理。共享 Redis 让多个实例复用同一缓存，也增加网络及集群可用性依赖。二级缓存可先查本地，再查 Redis，未命中再回源；每一层都要定义 TTL、失效和降级规则。
Redis 客户端缓存跟踪可辅助通知失效，连接中断后须按协议清理或重建可信状态，不能继续相信旧的本地项。权限、余额等要求强实时判断的数据不宜只靠本地副本。提升命中率要和陈旧容忍度、数据库保护效果及内存成本一起评估。〔N123、N124〕
代码示例
```
if (local_cache.valid(key))
    return local_cache.get(key);
auto value = redis.get(key).value_or(database.load(key));
local_cache.set(key, value, short_ttl);
if (tracking_connection_lost())
    local_cache.clear(); // 余额校验仍读权威数据
```

060 为什么生产环境遍历键通常用 SCAN，游标返回意味着什么？

答案　KEYS 会一次扫描全库并返回全部匹配项，大键空间会占用命令执行线程并制造大响应。SCAN 把遍历拆成多次调用，降低单次阻塞，但不是稳定快照。完整迭代从游标 0 开始，直到再次返回 0；COUNT 只是工作量提示，某一批返回空列表不代表结束。同一键可能重复出现，处理必须幂等。

遍历过程中一直存在的键有返回保证，但新增和删除中的键不能依赖精确覆盖；Cluster 还要逐个分片扫描并考虑迁槽。扫描后删除要分批限速，避免把 KEYS 的压力转成另一种批量写压力。〔N125〕

代码示例

```
std::uint64_t cursor = 0;
do
{
    auto page = redis.scan(cursor, "cache:*", 500);
    cursor = page.next;
    process_idempotently(page.keys);
} while (cursor != 0);
// 每批可以为空；集群场景还要遍历每个节点
```

061 Cache Aside 中为什么先提交数据库再删缓存，仍有哪些竞态？

答案　Cache Aside 由应用显式读写缓存：读时先查缓存，未命中查数据库再回填；写时通常先提交数据库，再删除缓存。即便这个常见顺序也不提供强一致：旧读可能先查到旧数据库值，等写事务提交和删缓存后才回填旧值；删缓存失败也会留下旧项。Singleflight 只合并相同 key 的并发 miss，不保证跨请求一致。

可靠失效事件可作为删除信号后回查数据库最新值；若事件携带新值，要按版本原子比较，避免乱序旧事件覆盖新值。TTL、可靠重试、版本校验和按键串行重建能缩短竞态窗口，延迟双删不能作为正确性证明。〔N124〕

代码示例

```
CacheValue cached = cache.get(key);
DatabaseValue fresh = db.read(key);
cache.compare_and_set(key, cached.version, fresh); // 拒绝旧版本迟到回填覆盖新版本
// 先提交数据库再删缓存仍需处理并发回填、失效通知丢失
```

062 Redis Stream 消费者宕机后，怎样找回未确认消息？

答案　Stream ID 只在单条流内有序唯一；不同消费组各自有消费进度，同组消费者分担新消息。组读取后未确认的消息进入 PEL，业务处理成功后 XACK 移除待处理状态，但 ACK 不会删除流记录。只读新消息不会自动取回旧 PEL，因此消费者崩溃要主动检查 XPENDING。

达到空闲阈值后，可用 XCLAIM 或 XAUTOCLAIM 转交给存活消费者；重新领取不证明旧消费者已停止，所以业务要按流名和消息 ID 幂等，防止重复副作用。流裁剪、删除、重试上限、持久化和复制保留也要协调，过早删消息可能令 PEL 只剩引用却无内容。可靠投递仍需要重试和对账设计。〔N126〕

代码示例

```
auto messages = stream.read_group("orders", "worker-A", ">");
for (auto& message : messages)
{
    if (!processed(message.id))
        handle(message);
    stream.ack("orders", message.id);
}
stream.claim_idle("orders", "worker-B", 30s); // 重领后仍须按消息ID幂等
```

063 Redis 突然变慢，应怎样区分服务端、网络与客户端排队？

答案　Redis 变慢先拆分客户端连接池等待、网络往返、服务端排队和命令执行。慢日志主要统计命令执行，不包含全部网络和排队时间；结合延迟监控、INFO、CPU、网络及客户端指标定位。再检查高复杂度命令、长 Lua、大键和巨大返回值，以及 fork、写时复制、AOF 同步、碎片、换页和 CPU 竞争。

故障时不要反复跑 KEYS 或拉取全量数据扩大负载。按证据选拆键、限流、异步释放、限制批量大小或调整容量，并在相同负载下比较尾延迟和错误率。内存数据库仍可能受 CPU、网络和排队影响，不能因“数据在内存”就排除服务端瓶颈。〔N116〕

代码示例

```
auto pool_wait = pool.acquire_time();
auto network = measure_round_trip();
auto server = redis.slowlog_and_latency(command);
auto p99 = percentile(latencies, 99);
// 分开记录客户端排队、网络和服务端执行；按证据缩小瓶颈
```

9.13 MySQL 存储机制与运维补充

064 InnoDB 聚簇记录怎样保存变长列、NULL 和溢出内容？

答案　以 InnoDB COMPACT 聚簇记录为例，记录含变长字段长度、可空列 NULL 位图、5 字节记录头及字段数据。NULL 位图按位向上取整，无可空列可省略；NULL 字段不另存字段值。记录还含 6 字节事务 ID 和 7 字节回滚指针。没有显式主键时，先选合适非空唯一索引，否则生成隐藏行 ID。

变长列长度可能用一或两字节编码，不能只按字符声明长度推断。记录过大时可把部分长值放到溢出页：COMPACT 通常保留 768 字节前缀和 20 字节指针；DYNAMIC 可把长列全部外置，页内只留指针。是否外置还取决于行大小、页大小和列内容。〔I63、S9〕
代码示例
```
RecordHeader header{};
header.null_bitmap = build_null_bitmap(nullable_columns);
header.trx_id = transaction_id;
header.rollback_ptr = undo_pointer;
if (record_too_large(row))
    store_external_page(long_column); // 具体布局依行格式与版本
```

065 MySQL 一致性逻辑备份何时可用快照，何时仍需锁？

答案　逐表导出若各自看到不同时间点，会把有关联的表拼成不一致状态。FLUSH TABLES WITH READ LOCK 可建立全局读锁，限制写入及相关 DDL，但等待和持锁会影响业务，应控制窗口并确保释放。纯 InnoDB 数据可用 mysqldump --single-transaction 在 RR 一致性事务里读取，普通 DML 可继续。
MyISAM、MEMORY 不受相同 MVCC 快照保护；导出中 ALTER、DROP、RENAME、TRUNCATE 等 DDL 仍可能让结果错误或失败。要做时间点恢复，还要记录与快照匹配的 binlog 坐标并保留后续日志，相关选项可能短暂加全局读锁。长快照也会推迟 undo 清理。备份是否可用最终要通过实际恢复演练确认。〔I74、S9〕
代码示例
```
if (all_tables_are_innodb())
    mysqldump_single_transaction(snapshot, binlog_position);
else
    lock_non_transactional_tables();
restore_into_isolated_instance(backup);
verify_rows_and_foreign_keys(); // 备份必须做恢复演练
```

066 InnoDB 自适应哈希索引与 Change Buffer 分别优化什么？

答案　自适应哈希索引观察热点 B+ 树查找，在内存页上为常用键前缀建立哈希捷径，底层 B+ 树仍在；它适合部分重复等值访问，维护成本和 latch 竞争也可能抵消收益。Change Buffer 则在目标二级索引页不在 Buffer Pool 时暂存可缓冲修改，等页读入或后台时合并，减少随机读 I/O；唯一性检查通常需要读页，受益更多的是非唯一二级索引。
Change Buffer 占 Buffer Pool，合并仍会产生 I/O，也不能免除 redo 和持久化。MySQL 8.4 中 innodb_adaptive_hash_index 默认 OFF，innodb_change_buffering 默认 none，不能把旧版常见配置当当前默认。应按代表性读写负载及争用指标测量后决定是否启用。〔I75、I76、N103〕
代码示例
```
if (secondary_page_not_in_buffer_pool(index_page))
    change_buffer.merge_later(index_page, update);
if (lookup_is_repeatedly_hot(key))
    adaptive_hash_index.maybe_accelerate(key);
// 两者优化目标不同，按目标版本和读写负载评估
```

第 10 章 系统设计与分布式

10.1 分布式基础与数据结构

001 CAP 理论如何用于分布式系统选型？

答案　CAP 讨论网络分区期间，一致性与可用性的取舍。这里一致性常指线性一致读写：操作像在调用和返回间某个时刻原子生效，并遵守真实时间顺序。网络分区使两侧无法可靠协调，因此系统不能同时保证每个请求都成功返回、又保证所有副本表现为一个最新状态。
偏 CP 的系统可能让少数派拒绝写入，避免互相冲突；偏 AP 的系统可继续响应，但允许暂时不一致并在恢复后收敛。CAP 不是说正常网络下永远只能二选一，也不替业务决定最终方案。先明确分区时允许拒绝什么请求、哪些数据可以短暂陈旧，再选一致性协议和恢复方式。
代码示例
```
if (network_partition)
{
    if (require_linearizable_reads_and_writes)
        reject_or_wait_on_minority(); // CP
    else if (require_every_partition_to_respond)
        accept_writes_and_reconcile_later(); // AP
}
// CAP 约束故障分区期间的保证，不意味着正常网络下永远只能选 C 或 A。
```

```
define_consistency_semantics_availability_slo_and_partition_cases_first();
// AP 需要冲突解决/收敛；CP 多数派恢复后可继续服务。
```

002 什么是 Bloom Filter？如何选择参数？

答案　Bloom Filter 是一组位和多个哈希函数构成的快速预筛结构。插入一个元素时，把它对应的 k 个位设为 1；查询时，只要有一位是 0，就能确定该元素没插入过；如果全是 1，只能说“可能存在”，因为其他元素可能碰巧点亮这些位。标准 Bloom Filter 没有假阴性，但有假阳性，也不支持直接删除。

已知预计元素数 n 和目标误判率 p，可估算位数 m≈-n ln(p)/(ln2)^2，哈希数 k≈(m/n)ln2。实际数据超过容量，误判率会升高；要监控误判并扩容或分层，而不能把命中当作数据库事实。

代码示例

```
#include <cmath>
#include <cstddef>
#include <cstdint>
#include <vector>
// n > 0，0 < p < 1；容量超预估会提高误判率
std::size_t m = static_cast<std::size_t>(std::ceil(-n * std::log(p) / std::pow(std::log(2.0), 2)));
std::size_t k = static_cast<std::size_t>(std::round((static_cast<double>(m) / n) * std::log(2.0)));
if (k == 0)
    k = 1; // 至少使用一个哈希位置
std::vector<std::uint8_t> bits((m + 7) / 8, 0);
for (auto h : k_hashes(key))
    set_bit(bits, h % m); // 插入只置位
for (auto h : k_hashes(query))
    if (!test_bit(bits, h % m))
        return DefinitelyAbsent;
return MaybePresent; // 无假阴性、有假阳性；命中仍查真实存储
// 标准结构不能单键删除；监测误判率后扩容/分层
```

003 什么是一致性哈希？

答案　一致性哈希先把节点和键都映射到一个逻辑环；键顺时针归属遇到的第一个节点。新增或删除节点时，主要只移动它与相邻节点之间的键，避免所有数据重新分配。节点设置多个虚拟点可把一个物理节点分散到环上，减少区间大小不均；异构节点则按权重安排更多虚拟点。

这种哈希只解决映射和部分迁移，并不自动处理热点、复制或故障恢复。虚拟节点数量和权重会影响负载方差，扩缩容要渐进搬迁，限速并设置读写转发、双读校验或版本检查。否则迁移窗口可能读不到数据或覆盖新写入。

代码示例

```
for (Node node : nodes)
    for (int v = 0; v < virtual_nodes(node); ++v)
        ring[hash(node.id, v)] = node; // 虚拟节点帮助均衡区间，也可按容量加权
auto it = ring.lower_bound(hash(key));
if (it == ring.end())
    it = ring.begin();
owner = it->second; // 键映射到顺时针后继，增删节点主要移动相邻区间
// 变更节点后限速迁移，可双读/转发并校验版本后切流
// 一致性哈希不解决热点、复制和故障恢复；虚拟节点数需按负载调优
// 监控分片负载方差，不能只凭“节点均匀”假设均衡
```

004 分布式 ID 生成器要满足什么要求？

答案　分布式 ID 要先明确唯一性、吞吐和可用性，再决定是否需要大致有序、紧凑或不可预测。数据库号段可原子预留 [start,end)，进程在本地消耗，双缓冲在当前段用完前预取下一段；崩溃会留下空洞，号段耗尽或预取失败要定义等待、拒绝或降级。

Snowflake 类把时间、节点号和序列组合，需保证节点号不冲突，并处理时钟回拨和单毫秒序列耗尽。UUID 易于独立生成但较长且排序局部性有限。分片键还需匹配常见查询：若按买家分片却只凭订单 ID 查详情，要么维护路由，要么把分片信息编码进 ID，否则需要广播。严格全局单调通常要协调器，会牺牲部分可用性。

代码示例

```
id = database_allocated_range(); // 简单；需分配服务、缓存段和故障恢复
id = UUID_random(); // 易分布式生成但较长，通常不按生成时间顺序聚集
id = (timestamp_ms << (node_bits + seq_bits)) | (node_id << seq_bits) | sequence;
if (node_id_conflicts)
    reject_configuration();
if (clock_moves_backwards || sequence_exhausted)
    wait_or_reject_or_use_new_epoch();
// 明确唯一/大致有序/紧凑/不可预测等要求；严格全局单调需协调，降低可用性。
```

10.2 数据处理与在线服务设计

005 如何从超大文件中找 Top K?

答案　若数据和统计状态能放进内存，维护大小 K 的最小堆即可求 Top K：先填满，之后只有新值大于堆顶时才替换。每次调整堆需 O(log K)，总时间 O(N log K)，空间 O(K)。堆顶保存当前候选中最小的那个，因而能快速淘汰不可能进入最终前 K 的值。

文件大到无法装入内存时，可按稳定哈希把相同键分到同一桶，逐桶统计，再把每桶的前 K 个候选汇总。频率 Top K 必须先定义是否是全局聚合；稳定分桶保证同键计数不被拆开。若近似算法节省资源，要说明误差和置信范围。

代码示例

```
std::priority_queue<int, std::vector<int>, std::greater<int>> top_k; // 最小堆
for (value : stream)
{
    if (top_k.size() < K)
        top_k.push(value);
    else if (value > top_k.top())
    {
        top_k.pop();
        top_k.push(value);
    }
}
// 内存可容纳时 O(N log K)、O(K)；堆顶是候选里最小项。
// 超大文件先稳定哈希分桶相同 key，再逐桶统计/取候选并归并。
// 频率 Top K 要定义跨桶全局计数；近似算法应给误差/置信度。
```

006 如何处理分布式事务？TCC 与 Saga 有何区别?

答案　分布式事务选型要明确一致性、可用性、锁持有时间和业务可否补偿。XA/2PC 先准备再统一提交，能协调多个资源，但协调者故障和等待会影响可用性。Seata AT 通过代理数据库操作和 undo log 降低业务侵入，却受数据库、SQL 和版本支持范围限制。

TCC 显式分 Try 预留资源、Confirm 确认、Cancel 释放，控制力强，但每一步要幂等并处理空回滚、悬挂和超时。Saga 把多个本地事务串成流程，失败后执行补偿；补偿是新的业务动作，不保证精确还原，中间状态可能被看到。能用本地事务加 Outbox 时通常更简单；按隔离要求和补偿能力决定。

代码示例

```
// TCC：Try 预留；各阶段按 transaction_id 幂等，并记录事务状态
if (!inventory.try_reserve(txn_id))
    return Failed;
if (payment.try_charge(txn_id))
    inventory.confirm(txn_id);
else
{
    inventory.cancel_if_reserved(txn_id);
    payment.cancel_if_charged(txn_id);
}
// 空回滚：Try 未发生也可能收到 Cancel；悬挂：迟到 Try 不能复活已取消事务
// Saga 则串本地事务，失败执行逆序业务补偿；补偿不保证精确还原
// 能避免跨服务强事务时，用本地事务 + Outbox 降低协调复杂度
```

007 如何实现业务幂等?

答案　业务幂等是同一个逻辑请求重复到达时，最终业务结果不重复发生。为请求生成唯一业务键，在数据库唯一约束或原子条件更新下保存处理状态和结果；同键重试就返回原结果或处理中状态。先 SELECT 检查再 INSERT 有竞态，两请求可能同时通过，唯一约束才是并发防线。还要定义键作用域、保留期限及同键不同参数时如何报冲突。

Kafka 的 PID/sequence 或 Broker 去重可减少协议范围内生产重试的重复，但 ACK 丢失仍可能重投，不能替代消费者用全局业务 ID 原子去重。MySQL 可用 ON DUPLICATE KEY UPDATE；8.0.19 起插入行可用别名，VALUES(col) 从 8.0.20 起弃用。INSERT IGNORE 可能把截断等数据问题变成 warning，不是只忽略重复键；唯一索引允许多条 NULL，缺值也要冲突时应 NOT NULL。

代码示例

```
auto inserted = db.insert_if_absent("payment_request", request_id, Processing);
if (!inserted)
    return db.load_saved_result(request_id); // 唯一约束裁决并发重试
process_payment_and_store_result_in_same_transaction(request_id, amount);
```

008 如何检索 TB 级日志中的关键词?

答案　TB 级日志不应先全量扫描：按时间、服务、租户或分区缩小范围。全文检索把文本拆成词项，倒排索引通过词典定位 term，再用 postings 列表找到匹配文档，列表可含词频和位置；多关键词可合并或求交候选，再评分排序。Stored Fields 按文档 ID 取原字段，Doc Values 更适合排序和聚合。

Lucene/Elasticsearch 的 segment 常不可变，新数据形成新段，后台 merge 合并并清理已删除文档，所以索引构建和合并会消耗磁盘、CPU。压缩和列式存储减少 I/O，但仍要解压和维护索引。查询需限制时间范围、并发、返回量和敏感字段权限，避免检索本身拖垮集群。

代码示例

```
partitions = narrow_by_time_service_tenant_and_region(query);
if (frequent_full_text_search)
    candidates = inverted_index.lookup(keyword);
else
    candidates = parallel_scan_compressed_partitions(partitions, keyword);
// 列式/压缩可减少 I/O，但有解压和索引维护成本。
apply_time_range_concurrency_result_limit_and_field_acl();
// 防止无范围查询拖垮集群；对敏感字段做访问控制和审计。
```

009 如何用 Bloom Filter 判断海量键是否可能存在?

答案　海量键查重可先经过 Bloom Filter：插入时对键做多个哈希并置位，查询时有任一位为 0 就确定不存在；位全为 1 只表示可能存在，因为其他键可能置过同样的位。预期插入数量和目标误判率决定位数组大小及哈希数，超过预期容量会增加误判。

标准 Bloom Filter 不支持删除；若业务必须删除，可考虑计数型变体，但要付出更多内存。它适合在昂贵数据库查询前筛掉确定不存在的请求；正向命中仍需查真实数据，不能把 Bloom 命中当成数据存在的证明。

代码示例

```
for (key : indexed_keys)
    bloom.insert(hash_positions(key));
if (!bloom.maybe_contains(query))
    return NotFound; // 任意一位为 0 可确定不存在
record = primary_storage.lookup(query); // 位全为 1 也只是可能存在
if (measured_false_positive_rate > target)
    resize_or_layer_filter();
// 正确构造下标准 Bloom 无假阴性，但有假阳性，命中仍查真实数据
// 标准 Bloom 不支持删除；Counting Bloom 支持删除但内存更高
// n 和目标误判率 p 决定 m/k；超容量后需扩容或分层
```

010 如何设计 O(1) 的 LRU 缓存?

答案　典型 O(1) LRU 用哈希表将 key 映射到双向链表节点，链表头是最近使用，尾部是最久未用。查找平均 O(1)，命中时将节点摘下移到头部；插入新项也放头部，容量满时淘汰尾节点并删除哈希项。表与链表必须同时更新，故并发实现要保护二者一致；分片锁降低竞争，却不再是全局严格 LRU。

普通 LRU 容易被一次性扫描挤掉热点。可在准入阶段过滤、使用冷/热队列，或要求重复访问后再晋升，区分刚进入与反复命中。晋升次数和时间窗口是具体算法策略，不是所有 LRU 的定义。〔N94〕

代码示例

```
std::list<std::pair<Key, Value>> lru; // 头最近，尾最久未用
std::unordered_map<Key, decltype(lru.begin())> index;
if (index.contains(key))
{
    lru.splice(lru.begin(), lru, index[key]);
    return index[key]->second;
}
if (lru.size() == capacity)
{
    index.erase(lru.back().first);
    lru.pop_back();
}
lru.emplace_front(key, value);
index[key] = lru.begin(); // 查/移/淘汰平均 O(1)
// 并发需保护 map 与 list 原子一致；分片锁会变为局部/近似 LRU。
```

011 如何设计分布式计数器?

答案　设计计数器前先决定是否必须精确、是否允许读到延迟值，以及更新是否可能丢失或重复。高写入下可以把计数拆到多个分片分别原子累加，读时求和或后台聚合；消息驱动的计数还要用事件 ID 去重，否则重试会重复增加。

要定义热点分片、持久恢复、重置周期和跨分片快照一致性。随机后缀可把热点写分到多个桶，代价是读取时要聚合，并允许一定延迟。若业务要精确财务统计，不能用最终一致的缓存计数取代事实账本。

代码示例

```
shard = hash(request_id) % shard_count; // 分散热点写入
shards[shard].apply_once(request_id, +1); // 单分片原子去重并递增，重试不重复计数
if (read_total)
    sum = aggregate_all_shards(); // 汇总可能有短暂延迟
persist_shard_checkpoints_and_dedup_ids(); // 恢复时既不漏计也不重计
// 明确计数是否精确、读延迟与跨分片快照一致性
// 热点可拆多个随机后缀分桶；扩缩分片需迁移去重状态/计数
```

012 如何设计实时排行榜?

答案　实时排行榜需要保存成员分数和可排序索引；Redis ZSet 可用 ZADD 更新分数、ZREVRANK 查单分片名次，适合常见在线榜单。设计时先定同分排名规则、活动周期、迟到更新、持久化及读写延迟。滑动窗口榜还要及时移除过期分数或维护窗口桶。

超高写入可按用户或榜单分区，再周期合并各片候选；全局精确排名需要协调，不能只把各分片名次拼起来。榜单规模、查询 Top N 还是任意名次、更新频率都会影响数据结构和成本，应先用目标负载测量。

代码示例

```
ZADD(board, score, user_id); // 分数与排序索引一起更新
rank = ZREVRANK(board, user_id);
top = ZREVRANGE(board, 0, K - 1);
// 先定义同分排序、迟到更新、窗口过期和读延迟目标
if (single_shard_capacity_exceeded)
    write_to_partitioned_boards_then_merge_candidates();
// 分区候选汇总可扩展；全局精确 rank 仍需协调
if (sliding_window_board)
    remove_expired_scores_or_events();
// 还要考虑持久化/重启恢复、排行榜规模和查询频率
```

10.3 高并发、消息与容错

013 如何设计高并发秒杀系统?

答案　秒杀入口先承载静态商品信息，再做身份和资格校验、限流及排队；库存预热后用原子条件扣减，确保不会减到负数。成功请求可异步创建订单、交给消息队列削峰，消费者按请求 ID 幂等处理；数据库仍用库存条件更新和订单唯一约束兜底。

还要处理消息失败、超时订单释放和库存对账，并压测热库存键和队列积压。入口尽量先过滤无资格请求，减少它们争抢库存。库存预扣、订单创建和支付之间需有明确状态机，失败时按唯一订单号补偿，不能因排队接口返回“受理”就谎称下单成功。

代码示例

```
if (!authorized(user) || !rate_limiter.allow(user))
    reject();
if (!inventory.decrement_if_positive(product_id))
    return sold_out();
broker.publish({request_id, user_id, product_id}); // 消费者按request_id幂等建单
```

014 常见负载均衡算法有哪些?

答案　轮询简单，适合能力接近的无状态实例；加权轮询按容量分配；最少连接适合请求时长差异明显的负载。可是 HTTP/2 一条长连接能同时跑多个请求，连接数不等于活跃工作量；即便看活跃请求数，不同请求成本也不同。随机或延迟感知策略可缓解部分热点，一致性哈希则有助会话和缓存局部性。

负载均衡还要结合健康检查、区域成本、连接池及重试放大。健康探针应区分进程存活和业务就绪，恢复中的实例不宜立刻接满流量。发现实例故障后，安全重试需有限次数和幂等键，避免把剩余实例也压垮。

代码示例

```
if (instances_have_similar_capacity)
    choose_round_robin();
else if (capacity_weights_known)
    choose_weighted_round_robin();
else if (request_durations_vary)
    choose_least_connections();
if (session_or_cache_locality_matters)
    choose_consistent_hash();
exclude_if_unhealthy_or_not_ready(); // 活着不等于业务就绪
// 同时控制重试放大、连接池和跨区成本；策略按实际延迟/负载验证。
```

015 微服务相对单体架构有哪些权衡?

答案　单体通常部署简单，进程内调用快，事务边界直观，适合早期团队快速交付。微服务让团队按清晰领域边界独立发布和扩缩容，但每个网络调用都可能超时、重复或部分失败，还需要接口治理、分布式数据、监控和运维能力。按技术层拆成很多小服务，常会让一个业务改动跨越过多团队。

拆分前先明确数据所有权、故障边界和团队职责，并配套超时、有限重试、熔断、版本兼容及调用链观测。跨服务事务也会变成事件、补偿或一致性问题。拆分应按真实独立扩缩容、发布和隔离需求，而不是因为“微服务更先进”。

代码示例

```
if (team_is_small_and_domain_boundaries_unclear)
    start_with_modular_monolith();
// 单体进程内调用快、事务清楚、部署简单。
if (service_boundary_owned_by_team && independent_release_or_scale_needed)
    split_service();
// 微服务引入网络超时、部分失败、数据一致性、观测/发布治理成本。
add_deadlines_bounded_retries_circuit_breakers_and_version_compatibility();
// 按领域与团队所有权拆分，避免按技术层切成大量小服务。
```

016 消息队列如何尽量避免消息丢失?

答案　避免消息丢失要看生产、Broker 和消费三段。生产者等待 Broker 确认，并按需求启用持久化及副本；否则数据可能只在客户端缓冲或单个易失节点。消费者应先完成可恢复的业务处理，再提交 offset 或 ACK；若先确认再处理，崩溃就会漏业务。

至少一次投递通常会重试，因此消费者要按消息 ID 幂等，并配置退避、死信和积压监控。可在消息中维护 producer 与 partition 维度的递增序号来发现缺口，但多个生产者各有序列，不能拼成一条全局连续流。确认先后决定故障时偏向重复还是丢失，需按目标选择。

代码示例

```
producer.send(message);
await broker_durable_replica_ack(); // 持久化确认后再报成功
consumer_result = apply_business_idempotently(message.id, message.payload);
if (consumer_result.durable)
    consumer.ack_or_commit_offset(message);
// 先 ACK 再持久处理会丢业务；先处理后 ACK 崩溃会重放
retry_with_limit_and_dead_letter_after_policy();
monitor_lag_and_replica_health();
// 按消息 ID 幂等，监控复制健康/积压；Broker ACK 语义要核对配置
// 保证边界还依赖副本数、刷盘策略和故障类型
```

017 限流常用算法有哪些?

答案　固定窗口用计数器实现简单，但窗口边界可能让相邻窗口短时间内通过近两倍额度。滑动日志更平滑但要保存时间点，滑动计数用前后窗口权重近似。令牌桶按速率补令牌，可在桶容量范围内突发；漏桶以较稳定速率放行。选型要明确是限制平均速率、突发还是并发数。

限流可放网关、独立服务或进程内：集中额度容易统一但依赖网络；本地快且隔离，但多副本总额度需协调。共享桶的读改写必须原子执行，热点额度可按用户或资源分片。规则更新要带版本可靠传播；限流器故障时选择 fail-open 还是 fail-closed，关键写路径通常不能默默放行。

代码示例

```
#include <algorithm>
// 固定窗口易实现，但边界前后短时间内可接近两倍限额
if (fixed_window_count(key, now) < limit)
    allow();
// 令牌桶按 rate 补充，最多 burst_capacity，允许短突发
tokens = std::min(capacity, tokens + elapsed * rate);
if (tokens >= 1)
{
    tokens -= 1;
    allow();
}
// 滑动日志更精确但状态多；漏桶适合较匀速处理后台任务
// 分布式还要定义原子更新、公平性和状态丢失时的降级
```

018 高可用系统通常从哪些方面设计?

答案　高可用从消除单点开始：进程和机器冗余、健康检查、故障切换及可恢复备份。先定义服务 SLO 和停机预算，容灾设计才有验收标准。例如全年 99.9% 可用率约允许 8.76 小时不可用，99.99% 约 52.56 分钟。依赖调用还要设有限超时、带抖动退避的重试、熔断和可控降级。

关键依赖要明确 RTO（多久恢复）和 RPO（可接受的数据丢失范围）。备份必须定期恢复演练，监控告警、故障演练和回滚也要证明能在目标时间内起效。多副本不自动解决数据一致性或区域级故障；可用性数字要结合观察窗口和业务请求口径计算。

代码示例

```
replicate_across_processes_and_failure_domains();
health_check_readiness_and_fail_over();
call_dependency_with_deadline();
retry_with_bounded_jittered_backoff();
if (failure_rate_high)
    circuit_break_and_serve_degraded_result();
backup_data();
periodically_restore_test_and_measure_RTO_RPO();
// RTO 是恢复所需时间，RPO 是可接受的数据丢失窗口
// 监控告警、回滚和故障演练验证冗余可恢复，而非只看副本数
// 备份需定期实测恢复，才能证明目标满足
```

019 高并发 IM 或消息推送服务如何拆分?

答案　高并发 IM 可拆为连接网关、认证、用户路由和消息服务。网关维护长连接及心跳，路由定位用户所在网关，消息服务负责持久化、投递、ACK 和离线存储。按用户或会话分片、用队列吸收突发；客户端重连后按序号补拉未确认消息。多端在线要定义消息扇出、重复和设备状态过期。

直播广播若让每个实时节点处理所有房间消息，会浪费带宽和 CPU；可按 liveVideoId 分区并将房间路由到负责节点。评论发布前应做权限和内容审核，明显风险可同步拦截，边界内容可异步复核。网关故障后客户端应指数退避重连，避免所有用户同时冲击认证和路由服务。

代码示例

```
gateway.accept_long_connection();
authenticate_and_heartbeat();
route = presence_service.locate_user_gateway(user_id);
message_store.persist_with_sequence_and_ttl(message);
queue.deliver(route, message);
if (client_ack(message.id))
    mark_delivered_idempotently();
if (reconnect)
    backoff_with_jitter();
fetch_messages_after(last_acked_sequence);
// 设计多端、离线存储、重复投递、顺序和路由状态过期语义。
```

020 高并发日志系统要考虑哪些模块?

答案　高并发日志先写入异步且有界的队列，由后台线程批量格式化并写文件或远端，避免业务线程直接等待 I/O。队列有上限可控制内存；队列满时要明确定义阻塞、丢低级别日志或采样，不能无限增长。配置轮转、刷盘和进程退出时有序 flush。

日志可记录请求 ID、时间和线程信息以便排查，但要过滤密码、凭证和个人数据。监控队列深度、写入失败及丢弃数量；高峰期若选择丢日志，要保证关键审计事件另有可靠路径。异步写降低业务等待，并不代表日志已经持久化。

代码示例

```
if (pending.size() == capacity)
    apply_block_drop_low_level_or_sample_policy(record);
else
    pending.push(record); // 业务线程只排入有界异步队列
background_worker.batch_format_and_write_file_or_remote();
rotate_logs_and_flush_by_durability_policy();
on_shutdown_flush_then_join_worker();
// 记录 request ID/时间/线程但脱敏；监控队列深度、丢弃量和写入错误。
```

021 高并发网络服务器通常如何设计?

答案　常见高并发网络服务器用非阻塞 socket 和事件循环管理连接、读写及定时器；计算密集或可能阻塞的业务交给工作线程池，避免阻塞 I/O 循环。连接数、待处理任务、缓冲区大小和单请求时间都要设上限，并实现背压、超时和优雅关闭。

若事件循环里做同步磁盘 I/O 或无界计算，同一循环上的其他连接都会停等。线程数和进程模型应通过压测、profiling 后选择，不要先堆线程。接收缓冲区、业务队列和工作池要共同限流，避免把网络背压藏成内存增长。

代码示例

```
while (running)
{
    auto ready = poller.wait(deadline);
```

```
    for (event : ready)
        handle_nonblocking_socket(event); // I/O 线程保持轻量
    submit_cpu_or_blocking_work_to_bounded_pool(); // 防止磁盘/业务阻塞事件循环
    apply_backpressure_to_connection_and_task_queues();
    expire_slow_requests();
}
// 限连接、缓冲与执行时长；优雅关闭时停止接入、排空任务并回收线程。
// 压测/profiling 后再调线程数、进程模型与 CPU 亲和性。
```

10.4 共识与消息一致性

022 Raft 共识算法由哪些核心部分组成？多数派有什么作用？

答案　Raft 把复制状态机共识拆为领导者选举、日志复制和安全规则。领导者接收命令并复制日志，通常要复制到多数派后才能提交并应用。多数派相交保证：已提交的日志条目至少存在于后来多数派的交集中，符合日志新旧规则的新领导者能够继承它。

分区时失去多数派的一侧无法提交新写入，可能仍能提供受限读取；多数派条件不是说每个节点时时刻刻都有相同的最新状态。提交还取决于任期和前序日志匹配，不能只看当前副本数量就认定任意条目已安全提交。〔S11〕

代码示例

```
leader = elect_by_term_and_timeout(); // 领导者接收命令
leader.append_log(entry);
replicate_to_followers(entry);
if (replicated_to_majority && raft_term_and_prefix_rules_hold)
    commit(entry);
apply_committed_entries_to_state_machine();
// 多数派交集让新领导者能继承已提交日志；少数派不能提交新写。
// 多数不代表每个节点总有最新状态，提交还受任期/日志匹配规则约束。
```

023 Transactional Outbox 如何解决数据库更新与发消息之间的双写问题？

答案　Transactional Outbox 把业务表更新和待发送事件一起写入同一数据库事务。这样能避免业务数据提交后、程序发消息前崩溃，导致两边状态不一致。后台轮询或 CDC 再读取 Outbox 并投递到 Broker，数据库事务只保证本地两条写入一起成功或失败。

发送后、标记已发布前也可能崩溃，所以事件可能重复投递；消费者按 event ID 幂等处理。还要安排旧记录清理、发布顺序、积压告警和数据库到 Broker 的延迟。Outbox 不会自动把消费者的数据库也纳入同一个原子事务。〔S13〕

代码示例

```
Transaction tx(db);
tx.insert(order);
tx.insert(outbox_event(order.id, "OrderCreated"));
tx.commit();
for (auto event : outbox.pending())
{
    broker.publish(event);
    outbox.mark_sent(event.id);
}
// 发送后崩溃会重发，消费者按event_id幂等
```

024 消息系统里的“恰好一次”语义通常有什么边界？

答案　最多一次表示不重投，可能丢；至少一次会重试以提高送达概率，但可能重复，所以消费者要幂等。“恰好一次”必须说清范围：只保证生产者重试不重复写日志、Broker 内消费和输出原子，还是连外部数据库副作用都包括。Kafka 幂等生产者能在协议范围内抑制重试重复记录，事务可原子提交多分区输出和消费位点，read_committed 只读已提交事务。

若业务还写外部数据库，Kafka 本身不能把两边纳入同一原子操作。可用事务性 Sink、Outbox 或业务幂等键处理；还要考虑消费者提交进度与数据库提交之间的失败窗口。端到端只有在所有副作用边界都纳入设计时才能谈“恰好一次”。〔S12〕

代码示例

```
producer.enable_idempotence(); // 去除生产重试带来的同分区重复记录
begin_broker_transaction();
write_output_records();
commit_consumed_offsets_atomically();
consumer.read_committed(); // 只读已提交事务数据
// 若还写外部数据库，Kafka 事务不能把 DB 副作用纳入同一原子提交。
use_transactional_sink_or_outbox_or_idempotency_key_for_external_effect();
// 先说清“一次”覆盖生产、Broker 内消费，还是端到端业务副作用。
```

025 分布式系统中的熔断器是什么？Closed、Open 和 Half-Open 状态如何工作？

答案　熔断器根据依赖调用的失败率、超时或延迟决定是否继续调用。Closed 正常放行并统计；持续超过阈值后进入 Open，快速拒绝以给下游喘息；冷却后进入 Half-Open，只放少量探测请求，成功才逐步恢复，失败则重新打开。阈值应参考下游 SLO、错误基线和延迟分位数，避免单次尖峰造成误切。

熔断不是重试：重试会增加调用，熔断在持续故障时减少调用。降级要按业务价值定义，例如读缓存、暂停非核心功能，但不能把未完成的写伪装成功。Half-Open 探测也须限制并发，防止冷却结束后一次性涌入造成反复过载。〔S15〕

代码示例

```
switch (state)
{ // 每次请求只执行当前状态的一条路径
case Closed:
    forward_call_and_record_result();
    if (failure_rate() > threshold)
        state = Open;
    break;
case Open:
    fail_fast_or_degrade();
    if (now >= cooldown_end)
        state = HalfOpen;
    break;
case HalfOpen:
    if (allow_limited_probe())
        state = probe_succeeds() ? Closed : Open;
    break;
}
// Open 降低持续故障时的下游调用；Half-Open 限量探测恢复情况
// 熔断不是重试；失败可延长冷却，并配合超时、降级和指标告警
```

10.5 分布式协调与服务治理

026 怎样实现可重入的业务锁，并处理持有者失联？

答案　可重入锁需要保存锁名、唯一持有者和重入计数：空闲时设置 owner 并置一，同一 owner 再次获取才递增；释放时必须核对 owner，计数归零才真正释放。数据库可用固定锁记录加短事务 SELECT FOR UPDATE，Redis 可用 Hash 和 Lua 原子处理。

数据库事务锁在事务结束时释放，业务锁记录可能仍需保留，这两者不是一回事。分布式锁还要有租约、续租和恢复流程；协调服务会话过期后，旧客户端也可能恢复运行。下游应校验递增 fencing token 或版本，拒绝迟到旧持有者的写入。〔N127、N128、N129〕

代码示例

```
if (lock.owner == caller)
    ++lock.depth;
else if (lock.owner.empty())
{
    lock.owner = caller;
    lock.depth = 1;
    ++lock.fence;
}
if (request.fence < resource.last_fence)
    reject_stale_owner(); // Lua原子更新租期与持有者
```

10.6 系统设计补充

027 流量增加十倍时，怎样估算瓶颈并验证扩容方案？

答案　不能给单台 Redis 或 MySQL 一个脱离条件的固定 QPS。先明确请求分布、读写比、缓存命中率、数据大小、连接数、持久化和延迟目标，再用接近生产的数据压测，找 CPU、网络、锁、磁盘和下游最先饱和处。用吞吐和响应时间估算并发在途量，并留故障与流量突发余量。

过载时先准入、限流、降级并设有界队列，避免无界线程或重试把系统压垮。随后按瓶颈选缓存、批处理、副本、分片或异步化，逐层验证扩容后的新瓶颈和尾延迟。容量规划还要覆盖切换、积压恢复和成本，不能只看短时峰值吞吐。〔N130〕

代码示例

```
for (int qps = 1000; qps <= target; qps = static_cast<int>(qps * 1.2))
{
    auto result = load_test(qps);
    if (result.p99 > slo || result.errors > limit)
        break;
```

```
}
measure_recovery_and_backlog_drain(); // 记录拐点，再决定扩容或限流
```

028 跨服务接口超时如何沿调用链排查，而不把所有问题归因于网络？

答案　跨服务超时要按调用链分段，而不是先认定网络问题。为请求生成 trace/span ID，记录入口排队、DNS/连接、序列化、服务处理、下游和响应写回的耗时及状态；子调用 deadline 要从整体预算扣除。对照成功、超时、重试和取消请求，同时查 CPU、锁、线程池、连接池、数据库等待和消息积压。

指标看总体与尾延迟，日志看单事件细节，trace 串起因果，三者需统一采样并脱敏。客户端超时不代表服务端没执行，写请求重试可能造成重复副作用，应使用幂等键和状态查询。先限流或降级止血，再依证据调整资源、依赖或预算，不要单纯延长 timeout 掩盖问题。〔N148、N146、N130〕

代码示例

```
Deadline total = Clock::now() + 300ms;
auto reply = inventory.call(request, total - 50ms); // 给入口处理留出余量
if (reply.timed_out())
    query_status_or_retry_with_same_id(request.id);
// 超时只说明调用方停止等待，不说明服务端没执行
```

10.7 消息队列模型与运行

029 消息队列如何实现异步解耦，自研时至少需要哪些状态？

答案　消息队列让生产者把工作交给中间层，消费者按自己的速度处理，可隔离一定故障、削平突发并支持发布订阅。同一个消费组通常分工处理一份消息，不同订阅组则各自得到一份副本，二者是不同语义。队列只能暂存峰值；如果长期生产速率高于消费能力，最终仍会耗尽容量。

自研持久队列至少要定义消息 ID、主题/队列、存储恢复、确认重投、消费位点、保留清理及配额，再决定分区复制。引入 MQ 也增加端到端延迟、重复投递和运维成本，不适合所有需要立即确定结果的接口。先定义失败语义，再判断异步解耦的收益。〔N131、N132、N133〕

代码示例

```
publish(topic, event); // 主题中的同一事件可由不同消费组各自处理
group_a.dispatch_to_one_consumer(event);
group_b.dispatch_to_one_consumer(event);
if (produced_rate > consumed_rate)
    apply_backpressure_or_scale_consumers();
```

030 Kafka、RocketMQ 与 RabbitMQ 应按哪些需求比较？

答案　Kafka 擅长按分区保留和重放日志，并有流处理生态；RocketMQ 提供队列、顺序、定时和事务消息等能力；RabbitMQ 的交换机路由、确认和工作队列适合灵活任务分发。三者都能覆盖多类用法，不能简化为“某个一定更快”。对比吞吐、尾延迟、消息大小、保留周期、路由和重放、顺序范围、消费组、事务边界、运维成本与团队经验，再按目标版本压测。可靠性还取决于持久化、副本、确认和业务幂等；支持事务消息不等于能把任意外部数据库副作用原子化。〔N133、N131、N132〕

代码示例

```
Benchmark kafka = measure(kafka_config, workload);
Benchmark rabbit = measure(rabbit_config, workload);
Benchmark rocket = measure(rocket_config, workload);
choose_by(p99, recovery, ordering, replay, routing, operations_cost); // 同负载同确认级别比较
```

031 如何按订单维持消息顺序，而不把整个系统串行化？

答案　先界定要保证顺序的业务键，例如同一订单，而不是追求全局有序。把同键消息路由到同一分区、队列或受支持的消息组，生产者保持同键顺序，消费者按该范围串行处理并在成功后推进位点；不同键仍可并行。Kafka 分区和 RocketMQ FIFO 消息组都有范围与前提。

重试、切换和多生产者可能造成重复、迟到或断序。业务状态机还应校验事件版本及合法迁移。毒消息要决定阻塞后续、隔离还是补偿；跳过可能破坏因果关系，不能静默丢弃。〔N133、N134〕

代码示例

```
auto partition = hash(order.id) % partition_count;
producer.send(partition, order_created);
producer.send(partition, inventory_reserved);
// 同订单按同键进同分区，其他分区仍可并行消费
```

032 消息积压时怎样定位生产、消费或依赖瓶颈？

答案　排查积压先看条数、最老消息年龄、生产与消费速率、失败重试、消费者存活和分区分布，分辨突发峰值、单个热分区与整体能力不足。Kafka lag 应结合消费提交位点和处理耗时看，单看总长度不能判断业务已经停摆。消费者慢还可能是数据库、外部接口、锁或反复重平衡。

盲目加线程会放大下游压力，扩实例也受分区数量和顺序要求约束。短期可以限产、隔离失败消息并保护关键依赖；长期优化批处理和瓶颈资源。恢复后核对业务延迟、重复处理和保留期限，不要为了让监控归零就丢弃积压。〔N135、N136〕

代码示例

```
for (const auto& partition : topic.partitions())
    report(partition.lag(), partition.oldest_age());
if (one_partition_grows())
    inspect_hot_key_and_consumer_latency();
if (all_partitions_grow() && database_slow())
    throttle_producer();
```

033 消费失败应怎样退避重试，并让死信可以恢复处理？

答案　先区分临时故障和永久错误：网络抖动可重试，格式错误或业务拒绝通常不能原样重试。临时错误用有次数和总时长上限的指数退避，记录消息 ID、原因和尝试次数，避免立即循环重投形成热点。超过上限后转死信或隔离存储，告警并提供审计后的修复回放；消费者仍要幂等。

RabbitMQ 可用拒绝且不重入队、TTL 和队列限制等方式把消息路由到死信交换机，但死信不是自动安全永久保存的保证。TTL 配合死信可构建延迟处理，顺序、到期精度和重发布安全须核对队列类型及版本。最终还要通过业务状态对账确认没有遗失。〔N136、N137〕

代码示例

```
if (temporary(error) && message.attempts < 5)
    retry_after(backoff(message.attempts) + jitter());
else
    dead_letter(message, error.reason());
// 死信要告警并支持按原消息ID修复后重放
```

034 延迟消息关闭未支付订单时，怎样避免误关已付款订单？

答案　延迟关单可由数据库定时扫描实现，简单且易恢复但有轮询开销；进程内 DelayQueue 或时间轮成本低，却需在崩溃后重建任务。Redis ZSet 可按到期时间领取，但要做持久化、租约和补扫；过期通知不可靠，不能独自承担触发。MQ 延迟消息易扩展，TTL 加死信可补偿，但同队列不同 TTL 可能队头阻塞。

无论用哪种，定时消息只表示“该检查了”，消费者仍要验证状态与截止时间，并用条件更新原子迁移；支付回调和关单需共享状态机，库存释放按订单键幂等。保留数据库扫描对账。RocketMQ 定时精度和范围随大版本而异，不是硬实时定时器。〔N138、N127〕

代码示例

```
auto changed = db.execute(
    "UPDATE orders SET status='closed' WHERE id=? AND status='unpaid' AND deadline<=NOW()",
    order_id);
if (changed == 1)
    release_inventory_idempotently(order_id);
// 延迟消息到达时仍要以数据库状态为准
```

10.8 RocketMQ、Kafka 与 RabbitMQ

035 RocketMQ 的半消息与事务回查如何连接本地事务和消息可见性？

答案　RocketMQ 事务消息先发暂不可被消费者取走的半消息。Broker 保存成功后，生产者执行本地事务，再通知提交或回滚；提交才让消息进入正常投递。若通知丢失或状态未知，Broker 可回查生产者，生产者要按可靠保存的业务事务状态回答，不能凭易失内存变量猜测。

这一机制帮助本地事务结果与消息可见性最终一致，并未把消费者数据库纳入同一个原子事务。必须设计事务状态记录、回查服务可用性、重试和超时；消费者仍需幂等与失败恢复，不能把半消息说成任意多服务强隔离事务。〔N139〕

代码示例

```
broker.send_half_message(event);
if (database.commit_order_and_tx_state(order, "committed"))
    broker.commit_half_message(event.id);
else
    broker.rollback_half_message(event.id);
// 回查读取持久化事务状态；消费者仍需按事件ID幂等
```

036 RocketMQ 的 CommitLog、ConsumeQueue 和索引各承担什么角色？

答案　经典 RocketMQ 存储把完整消息顺序写入 CommitLog，多主题和队列可共享这份日志。ConsumeQueue 为逻辑队列建立紧凑条目，记录指向 CommitLog 的位置、长度及过滤所需信息，消费者据位点快速找到消息；IndexFile 则辅助按消息键等条件查找，不是普通顺序消费的唯一入口。

Kafka 则围绕分区日志段及其索引组织数据，不能因为都有 Broker 就假设文件布局相同。索引重建、文件清理和复制模式影响恢复及可用性，要按部署版本检查。RocketMQ 5.x 的代理和 Controller 也不能完全套用旧版主从示意图。〔N140、N131、N133〕

代码示例

```
auto entry = consume_queue.read(topic, queue, offset);
auto message = commit_log.read(entry.file_offset, entry.length);
if (lookup_by_key)
    index_file.find_commit_log_position(key);
```

037 RocketMQ NameServer 故障后，现有连接与新路由分别会怎样？

答案　NameServer 提供 Broker 注册和主题路由发现，客户端据此定位队列与 Broker，具体消息收发通常不经过 NameServer。经典部署可有多个彼此独立的 NameServer，Broker 向它们注册，客户端配置多个地址并缓存路由。短时全挂时，已有缓存且 Broker 未变化的客户端可能继续收发；新客户端、路由刷新和 Broker 变化后的访问会受影响。因此“完全没事”和“消息立刻全部中断”都不准确。应分别监控注册、路由发现、真实生产消费，设计客户端重试和 Broker 故障策略。按目标版本区分 NameServer、Controller 与 Proxy 的角色。〔N140、N131〕

代码示例

```
if (client.has_cached_route(topic))
    client.send_using_cache(topic, message);
else
    client.query_name_server(topic); // 发现面故障与Broker消息面故障要分开监控
```

038 消费组如何决定消息分工，增加消费者何时不再提高吞吐？

答案　在经典 Kafka 消费组里，一个分区同一时刻只分配给组内一个消费者；不同组各自维护消费进度。主题有十个分区，组内实例超过十个时，多出的实例可能空闲。一个消费者可以处理多个分区，但若把同分区记录随意并行，就要额外维护完成顺序和位点安全。

RocketMQ 集群消费也做组内分工；经典客户端广播消费则让每个实例各自收取副本，支持范围须查客户端版本。增加消费者前要看分区数、热键和下游容量。消费线程数、进程实例数和独立订阅组数是不同概念，不能简单用数量乘出吞吐。〔N133、N131〕

代码示例

```
std::vector<Consumer> consumers = assign_partitions(partitions, group_members);
// 消费者实例数超过分区数后不会增加分区级并行度
rebalance_on_membership_change();
monitor_downstream_capacity();
```

039 Kafka 怎样通过分区日志、批处理和拉取消费提高吞吐？

答案　Kafka 主题由多个分区组成，分区内按 offset 追加日志；消费者主动拉取并维护位置。已消费数据按保留策略继续存在，可让其他组读取或重放，不会因一次 ACK 立刻全局删除。顺序追加、批量请求、压缩和页缓存可减少寻道与往返；合适路径下零拷贝减少用户态复制。

Broker、控制器负责存储协调；Connect 接外部数据源，Streams 处理流与 KStream/KTable 状态。控制面由 ZooKeeper 向 KRaft 迁移，能用哪种模式要按部署版本确认。更多分区增加并行度，也增加文件和重平衡成本。批量会引入等待，所以高吞吐不等于每条消息都低延迟；大消息、历史随机读和副本同步都会改变表现。〔N133〕

代码示例

```
producer.append_batch(partition, {message_a, message_b, message_c});
auto records = consumer.fetch(partition, next_offset);
if (process(records))
    consumer.commit(next_offset + records.size()); // 不同组可独立重放
```

040 Kafka 位点为什么应在业务成功后推进，异步处理怎样避免跳过消息？

答案　Kafka 位点一般表示下一条要读取的位置。若先提交位点再做业务，进程崩溃后未完成消息会被跳过；业务成功后再提交则可能重投，所以需要幂等。自动提交依据拉取和周期策略，不知道异步业务任务是否完成。

同分区并行处理时要维护“连续完成水位”：后面的记录已完成，也不能越过前面未完成项提交，否则重启会漏前项。限制在途量、必要时暂停分区，并处理重平衡交接。Kafka 事务可原子提交 Kafka 输入位点和输出；外部数据库副作用仍需单独协调。〔N141、N133〕

代码示例

```
completed.insert(11);
completed.insert(12); // 10尚未完成，不能提交到13
while (completed.contains(next_offset))
    ++next_offset;
consumer.commit(next_offset); // 只推进连续完成水位，崩溃后可安全恢复
```

041 acks=all 和 ISR 怎样影响 Kafka 写入安全与可用性?

答案　ISR 是当前满足复制条件的副本集合。acks=all 要求 Leader 等待当前 ISR 的确认，但 ISR 可能缩小，所以单看 all 不能推断至少有多个安全副本；还需设置合适副本因子、min.insync.replicas、Leader 选举和持久化策略。ISR 不足时拒绝写会牺牲可用性，以守住副本要求。

生产者超时不证明 Broker 没写入：Broker 可能已接收，响应却丢失。启用幂等生产者并让业务消息携带去重键，以安全处理重试。副本选举行为随版本发展，安全结论应依据实际版本、配置和故障模型，而不能只凭“acks=all”口号。〔N133〕

代码示例

```
producer.set_acks("all");
broker.set_min_insync_replicas(2);
if (isr.size() < 2)
    reject_write();
else
    append_and_wait_for_isr(record);
// ISR少于阈值会牺牲写可用性；结合重试和幂等配置验证
```

042 Kafka 重平衡为什么会引起延迟和重复，怎样减少无效重分配?

答案　消费组成员加入、离开或订阅变化时，要重新安排分区所有权。不同协议和分配策略决定停顿范围，不能把所有 Kafka 版本都说成全组长时间停止。交接时消费者要处理在途任务和已完成位点；过早提交会漏，没提交可能重复。长时间不调用 poll、频繁重启和不稳定网络会反复触发重平衡。

限制每批和在途任务，正确设置存活、轮询和会话参数；目标版本支持时可用稳定成员身份和更平滑协议。优化目标是交接有界、恢复正确，不是把超时设得极大来隐藏失联。〔N133、N141〕

代码示例

```
void on_partition_revoked(Partition p)
{
    stop_fetching(p);
    wait_for_inflight_until(deadline);
    commit_contiguous_offset(p);
} // 晚提交可能重放；提前提交可能漏消息，消费仍应幂等
```

043 RabbitMQ 如何通过交换机、绑定和队列决定消息去向?

答案　AMQP 是协议家族，RabbitMQ 实现相关协议及扩展。连接可以开多个 channel，vhost 提供逻辑命名和权限边界。生产者通常把消息发给 exchange；交换机根据绑定和 routing key 路由到队列，消费者再从队列取消息。direct 精确匹配键，topic 按点分层模式匹配，fanout 发往所有绑定队列，headers 按头部条件匹配。

同一队列上的消费者通常竞争分工；若多个订阅者都要收到消息，应绑定到各自队列。路由成功只表示找到队列，不代表业务完成。未匹配的消息还要明确是否丢弃、退回或备份；持久化、副本、确认和权限都要分别配置。〔N132〕

代码示例

```
exchange.bind("orders", "order.*");
exchange.bind("payments", "payment.#");
exchange.publish("order.created", message); // 只路由至匹配绑定的队列
exchange.publish("payment.success", message); // 每个独立订阅者需自己的队列
```

044 RabbitMQ 的 Confirm、Return 和消费者 ACK 分别确认哪一步?

答案　Publisher Confirm 回答 Broker 是否接受发布动作，保障范围受队列类型、持久化和复制配置影响；它不表示消费者已处理。mandatory 发布没有路由到任何队列时可触发 basic.return；消息仍可能随后得到 confirm，因此生产者要分别处理“路由失败”和“Broker 接受”。消费者 ACK 则表示应用已完成一次投递，Broker 可清除未确认状态。业务提交前 ACK，崩溃会漏处理；业务提交后 ACK 丢失会重投，所以消费者应幂等。Confirm 与 delivery tag 属于不同方向和 channel，不能混用。生产者要跟踪待确认消息并安全重试，TCP 写成功不等于 Broker 落地。〔N142〕

代码示例

```
publisher.publish(exchange, routing_key, message, mandatory = true);
if (basic_return_received())
    handle_unroutable(message);
if (publisher_confirmed())
    record_broker_acceptance(message.id);
```

045 RabbitMQ prefetch 怎样限制在途消息，为什么太大会恶化公平性?

答案　手动确认时，prefetch 限制消费者或 channel 可同时持有的未确认投递数，作用范围要结合 basic.qos 和 RabbitMQ 的实现理解。它像消费端的在途窗口：太小会让工作线程等下一批消息，太大则可能把许多消息都压给慢消费者，占内存并延长故障后的重投时间，让其他消费者没工作。零通常表示不设上限，不是停止投递。

按并发处理数、消息大小、单条耗时和下游容量逐步调整，并观察吞吐与尾延迟。ACK 必须在业务完成后发送；提前 ACK 只是把 Broker 积压转移到应用内存，崩溃时更难恢复。〔N143、N142〕

代码示例
```
channel.basic_qos(4); // 每个消费者最多四条未确认消息
for (auto delivery : channel.consume(queue))
{
    process(delivery);
    channel.ack(delivery.tag);
}
// 预取过大可让慢消费者长时间占有未确认消息
```

046 RabbitMQ 加入集群为何不等于每个队列都具备多副本?

答案　RabbitMQ 集群让节点协作并共享部分元数据，但消息是否多副本取决于队列类型。仅仅加入集群不代表任意节点损坏后所有队列消息都还在。Quorum Queue 用 Raft 为一个队列建立复制组，多数派管理日志和领导者，适合需要明确复制耐久性的任务队列。

失去多数派时可用性会受限；队列数、副本数和跨节点网络都会增加成本。应按当前支持的队列类型规划副本、确认与恢复演练，不能把旧版经典镜像队列当作新版本默认实践。多副本保护 Broker 故障环节，不替代业务 ACK、幂等和备份。〔N144〕

代码示例
```
quorum_queue.append(message);
quorum_queue.wait_for_majority();
if (!has_majority())
    return unavailable(); // 少数派停止写以保护一致性
// 多节点集群不代表每个队列都有多副本，需显式配置队列类型
```

10.9 分布式理论与一致性

047 强一致、最终一致和读写一致性分别承诺什么?

答案　“一致性”要说明观察者和操作含义。线性一致性要求每个操作看起来在调用到返回之间生效，并尊重真实时间顺序；顺序一致性只要求存在符合各线程程序顺序的全局次序。最终一致允许短暂旧值，但停止更新且网络恢复后副本应收敛。BASE 概括基本可用、软状态和最终一致，表达一种分布式取舍。

软状态意味着复制期间可继续变化，不能因此允许业务随意出错；中间态要可观测，失败要有重试、冲突处理或补偿。读己之写、单调读、会话一致性是更弱但实用的保证。接口契约需写明陈旧时长、冲突合并和故障时可用性。数据库 ACID 的 C 是约束保持，不可直接等同分布式线性一致。〔N145〕

代码示例
```
if (consistency == Consistency::linearizable)
    wait_until_replica_applied(version);
else if (consistency == Consistency::eventual)
    return possibly_stale_value();
if (client_needs_read_your_writes)
    route_to_replica_at_least(version);
```

10.10 分布式组件与服务治理

048 RPC 调用链至少应设计哪些协议和失败语义?

答案　RPC 把远程请求包装成接口调用，但网络会带来超时、重复、断连、版本不兼容和部分成功。帧至少要有协议版本、类型、请求 ID 和长度，并限制最大帧长。I/O 线程负责收发和解帧，业务任务进入有界线程池；队列满要背压或拒绝，不能无限堆积。响应带请求 ID，客户端据此找到等待中的 Future。

还要定义序列化兼容、认证、deadline、取消、错误码、重试资格和幂等键。gRPC deadline 可向下游传播，重试只针对明确瞬时错误并受预算和退避约束。客户端超时无法断言服务端没执行，写操作需靠状态查询或幂等键收敛。SPI 可扩展序列化或负载策略，但扩展不能改变核心失败语义。〔N62、N146〕

代码示例
```
Frame request{.version = 2, .type = Call, .length = payload.size(), .id = 87};
if (work_queue.full())
    return overload_response(request.id);
worker.execute(request);
pending_futures.complete(request.id, make_response(request.id));
// 有界帧长、截止时间和幂等键属于协议失败语义
```

049 ZooKeeper 的 ZAB、临时节点和 Watch 怎样支持协调?

答案　ZooKeeper 用领导者和多数派协议给写请求排序并复制日志；ZAB 的目标是让已提交更新以一致顺序传播。写入要满足多数派条件，失去多数派时系统应停止无法安全完成的写，而不是伪造进度。临时节点绑定会话，客户端会话超时后节点消失，常用于注册或锁。

Watch 是一次性变更通知，收到后应重新读取状态并重新注册；它不是可永久消费的消息队列。用来选主、服务发现或锁时，要处理会话抖动、重复回调和羊群效应。旧客户端恢复后可能仍以为自己持锁，资源端须用 fencing token 或版本校验，不能只信本地状态。〔N147、N129〕

代码示例

```
auto candidate = zk.create_ephemeral_sequential("/candidate-");
watch(predecessor(candidate),
      [&]
      {
          reread_leader_and_register_watch();
      });
if (session.expired())
    discard_local_leadership(); // Watch通知后重新读取，旧主还要检查fencing token
```

050 服务注册发现怎样面对注册中心不可用和过期实例？

答案　服务实例启动后注册地址、版本、能力和健康租约；客户端订阅或轮询获取实例列表，心跳和租约过期可清理失联实例。注册中心短时不可用时，客户端可用带时间戳的缓存路由继续调用，但必须限制最大陈旧时间并定义降级，不能无限信任已下线地址。

发布期间需支持新旧版本共存和回滚。健康探针只证明探测时刻可达，不保证业务依赖正常；调用端仍须超时和隔离错误。使用 ZooKeeper、DNS、控制面或服务网格时，分别检查一致性、传播延迟、权限、证书和数据面断联行为。〔N147、N148〕

代码示例

```
if (registry.unavailable() && cache.age() < max_staleness)
    call(healthy_cached_instance());
else if (cache.age() >= max_staleness)
    return service_discovery_unavailable();
// 缓存路由设置陈旧上限；健康检查应覆盖业务就绪状态
```

10.11 分布式事务与一致性

051 Seata AT 和 TCC、Saga 的补偿边界如何比较？

答案　Seata AT 通过代理本地事务、记录回滚日志与全局锁来协调常见关系数据库操作；回滚时按镜像恢复数据，但要求参与者、SQL 和数据库版本满足支持条件。DDL、长事务和数据库外部副作用不会自动变成可回滚操作，锁冲突也会影响吞吐。

TCC 由业务显式实现预留、确认、取消，需处理幂等、空回滚、悬挂和超时；Saga 用补偿动作串起长流程，补偿可能被其他人观察且不一定精确逆转。框架不替代本地约束、Outbox、幂等和对账。按所需隔离、补偿能力、延迟及参与者类型选择。〔N149〕

代码示例

```
Transaction tx(db);
tx.decrement_inventory(sku);
tx.insert_undo_log();
tx.commit();
if (global_rollback)
    apply_undo_log();
// 外部邮件不能靠数据库undo撤回；通过Outbox或TCC协调外部副作用
```

052 Snowflake 类 ID 的位段设计怎样处理时钟回拨与序列耗尽？

答案　Snowflake 类 ID 通常把时间戳、节点标识和时间片内序列拼成整数，获得近似按生成时间排序的唯一值。节点号要通过可靠分配防止冲突；单毫秒序列耗尽时可等下一毫秒或限流拒绝。时钟回拨时不能盲目继续发号，否则可能重复；可等待追平、切逻辑纪元或暂时拒绝，策略要明确可用性取舍。

节点重启、数据中心漂移、时钟变化及位宽升级都要测试。多节点 ID 不保证严格全局顺序，也会暴露大致业务规模；若向外公开需考虑可枚举性。〔N98〕

代码示例

```
if (now_ms < last_ms)
    wait_or_fail_on_clock_rollback();
sequence = (now_ms == last_ms) ? sequence + 1 : 0;
if (sequence > sequence_mask)
    wait_for_next_millisecond();
auto id = ((now_ms - epoch) << (node_bits + sequence_bits)) | (node_id << sequence_bits) | sequence;
```

10.12 高并发业务系统设计

053 短链服务怎样生成唯一编码并支撑高频跳转？

答案　短链写入前先验证目标 URL、权限、过期规则和自定义别名；相同长链是否去重要按租户、活动和有效期定义。可用数据库序列或分布式 ID 编码为 Base62；随机码要有唯一约束并有限重试。映射保存于可靠存储并按短码索引，读时先查缓存再回源，缓存 TTL 不能超过业务过期时间。

过期短码通常不回收：旧链接、浏览器缓存的 301 或他人保存的内容可能还指向旧目标，复用会让老链接跳到新地址。点击统计要过滤爬虫和短信预览；短码还应防枚举，只允许 HTTP(S) 等受控协议，并支持恶意链接下架和监控。规模增长后可以按短码路由分片。〔N110、N98、N124〕

代码示例

```
auto code = base62(next_distributed_id());
db.insert(code, validate_http_url(target), expires_at);
if (cache.contains(code) && cache[code].expires_at > now())
    redirect(cache[code].target);
// 永不复用旧码，防止历史收藏链接指向新目标
```

054 点赞系统怎样保证用户状态幂等并承受热点写入？

答案　把点赞建模为“设置为已点赞”或“取消点赞”状态，而不是 toggle；这样重试不会把结果反转。用用户 ID 与内容 ID 的唯一约束保证一对关系至多一条，重复请求返回同一结果。关系表是可恢复的事实数据，点赞总数是派生计数。

Redis 改状态与发 MQ 之间可能崩溃，不能简单先写 Redis 再发送。可在一个 Lua 脚本中原子更新状态并写 Stream，或用 MySQL 事务 Outbox 发可重放事件。每个状态带递增版本，乱序事件只接受更高版本；消费端仅在状态真实变化时增减计数。热点计数可拆桶后汇总，并定期对账修复；本人状态和总数分别定义陈旧时间。〔N110、N124、N133〕

代码示例

```
if (likes.insert_if_absent(user_id, post_id))
    outbox.append({event_id, user_id, post_id, version});
if (consumer.mark_once(event_id))
    counters.increment(post_id); // 重复事件不重复计数
// 周期性用关系表重算派生计数，修复漏发或重放
```

第 11 章 自动驾驶与实时数据

11.1 QoS 与点云处理

001 ROS 2 DDS 的 QoS 如何按数据类型选择？

答案　ROS 2 的 QoS 是一组通信策略，包括保留历史样本的方式与深度、可靠性、持久性和 deadline 等。发布者与订阅者的 QoS 必须满足兼容关系，否则订阅端可能收不到样本。不同数据类型应按新鲜度、丢失代价和时限选择，不能给所有 topic 套同一配置。

高频相机或点云常更关心最新一帧，可用 BestEffort 和较浅队列，避免网络波动时处理很久以前的数据；关键控制指令可能需要 Reliable，但重试会占用时间和资源，必须检查是否仍满足截止时间。最终要在目标 DDS 实现、网络和负载下测量延迟、丢帧及队列积压。

代码示例

```
camera_qos = {BestEffort, KeepLast, depth = 1, deadline = frame_period}; // 保新帧
control_qos = {Reliable, KeepLast, bounded_depth}; // 关键指令要可靠但受时限约束
if (!publisher_qos_compatible_with(subscriber_qos))
    no_samples_are_delivered();
// History/Depth、Reliability、Durability、Deadline 策略须兼容
// 队列太深会积压旧图像；Reliable 重试也会增加时延和资源占用
// 在目标 DDS、网络和负载下测延迟/丢帧，再按数据重要性定配置
```

002 体素网格降采样如何实现？

答案　体素网格把三维空间切成固定边长的小立方体。对每个点，将坐标除以边长并向下取整，得到整数格子索引；把索引作为哈希键累计点坐标和数量，最后用每格坐标平均值作为代表点。对 n 个输入点，哈希处理平均为 O(n)，内存随非空体素数量增长。

体素越大，输出点越少，但细小结构、障碍边界和表面精度损失也越明显。平均值生成的质心不一定是原始点；若后续算法依赖真实边缘，可保留距离格中心最近的原始点或根据局部细节自适应格长。处理负坐标时使用 floor 一类向下取整规则，避免简单截断把负坐标分到错误体素。

代码示例

```
#include <cmath>
#include <unordered_map>
std::unordered_map<Cell, SumAndCount, CellHash> voxels;
```

```
for (const Point& p : cloud)
{
    Cell c{std::floor(p.x / size), std::floor(p.y / size), std::floor(p.z / size)};
    voxels[c].add(p); // 相同体素累计坐标和计数，floor 也正确处理负坐标
}
for (const auto& [cell, aggregate] : voxels)
    output.push_back(aggregate.centroid());
// 平均 O(n)，内存随非空体素数增长；体素越大细节/边缘精度损失越明显
// 质心不是原始点；依赖表面边缘时可保留代表点或自适应叶尺寸
```

11.2 共享内存与时钟同步

003 共享内存传递大帧如何管理所有权？

答案　大帧放进共享内存池后，生产者可只传槽位索引、长度、代际号、时间戳等轻量句柄，避免在进程间反复复制图像。句柄描述的是位置和元数据，不代表消费者有权无限期读取；应通过所有权转移、引用计数或租约状态，确保读者完成前生产者不会复用该槽。

共享槽读写状态要有跨进程同步协议，并考虑进程崩溃后的安全回收。仅因租约超时就覆盖内存可能撞上仍在访问的进程。不同进程的映射地址不保证相同，因此句柄应使用相对偏移或索引，而不是裸虚拟地址。还需验证步长、格式、对齐、NUMA 放置和后续是否发生隐式复制。

代码示例

```
slot = shared_pool.acquire();
producer.write_frame(slot); // 跨进程只传句柄而非复制大帧
publish({slot.index, slot.generation, length, timestamp, stride});
consumer.map_own_address_and_validate_layout(handle);
consumer.retain(slot);
if (consumer_done)
    release_reference(slot); // 引用归零/所有权归还前不能复用
if (owner_crashed_or_lease_expired)
    reclaim_after_safe_recovery();
// 共享槽位状态需原子同步；检查步长、缓存对齐、NUMA 和隐式复制。
```

004 多传感器如何做时间同步？

答案　多传感器的最佳同步通常来自硬件：共用触发信号或 PPS 可以让采集时间落在同一基准上；分布式设备可用 PTP/gPTP 校准时钟。采集端优先保存硬件时间戳，因为应用收到回调的时刻还叠加了驱动、队列和调度延迟。所有数据进入融合前要统一时钟域。

除了固定偏移，还要估计随时间积累的时钟漂移；启动时对齐一次不保证运行一段时间后仍同步。传感器的时间定义也要一致，例如图像曝光中点还是帧开始、雷达整圈扫描还是单点采样。匹配时按时间戳做插值或最近邻，并设置最大允许时间差，超出阈值时丢弃或标记数据质量。

代码示例

```
capture_hardware_timestamp(sensor); // 比应用收到回调时间更接近实际采集
synchronize_clocks_with_hardware_trigger_or_PPS_or_PTP();
estimate_clock_offset_and_drift_over_time();
normalize_to_common_clock_domain(); // 明确曝光中点/扫描周期等时间定义
frame = nearest_or_interpolate(sensor_a, sensor_b, max_allowed_skew);
// 只启动时校一次不能消除长期漂移；融合需设最大时间差。
```

11.3 实时数据流水线

005 实时传感器数据流水线如何避免阻塞和积压？

答案　实时传感器流水线可拆为采集、校时、预处理、融合或推理、输出等阶段，每段通过有界队列隔离短时速度波动。队列设上限是关键：无限积压会让系统持续处理已经过时的数据。队列满时要按数据语义决定丢最旧帧、丢最新帧或施加背压；实时感知常优先保留最新样本。

每帧应携带时间戳和截止时间，超时后不再把过期结果继续推给下游。监控各阶段耗时、队列深度、端到端 P95/P99 延迟和丢帧，才能看出瓶颈位于采集还是计算。控制链路还要依据安全要求处理丢弃和失联，不能把“更低延迟”当作唯一目标。

代码示例

```
capture->bounded_queue->timestamp_and_preprocess->fusion_or_inference->output;
if (queue.full())
    drop_oldest_or_newest_or_apply_backpressure_by_data_type();
if (now - frame.timestamp > deadline)
    discard_expired_frame();
record_each_stage_timestamp_and_queue_depth();
```

```
alert_on_p95_p99_latency_drops_and_backlog();
// 实时感知常优先新鲜度，不能无限排旧帧；策略按链路安全要求设定。
```

006 工业相机图像采集与处理系统如何设计？

答案　工业相机系统应让采集线程专注从 SDK 获取缓冲区并记录硬件时间戳，处理线程再通过有界队列完成去马赛克、校正或检测。这样耗时算法不会直接阻塞取帧。大图像可在共享池中传递句柄减少复制，但必须保证缓冲区在处理结束前不会被相机 SDK 重用。

每帧要验证像素格式、步长、分辨率和缓冲生命周期。队列满或处理超时后按策略丢弃过期帧，并分别记录相机设备端丢帧计数与处理队列丢帧，才能区分采集故障和算法跟不上。运行期间监控 CPU/GPU、内存、队列深度、丢帧率及 P99 延迟；过大的无界缓冲只会增加延迟。

代码示例

```
frame = camera_sdk.acquire_buffer();
timestamp = frame.hardware_timestamp();
if (!bounded_queue.try_push({frame_handle, timestamp, format, stride, dimensions}))
    count_drop();
worker_demosaic_rectify_and_detect(frame_handle); // 句柄复用可减少大帧复制
validate_pixel_format_stride_dimensions_and_buffer_lifetime();
if (processing_is_late)
    drop_expired_frame_and_record_stage_latency();
// 分开采集/处理线程，避免耗时算法阻塞 SDK 取帧
// 记录设备端丢帧计数，区分采集丢失与处理积压
// 监控 CPU/GPU、内存、丢帧和 P99；队列有界避免无界缓存
```

007 生产者永不阻塞的帧环形队列如何设计？

答案　生产者永不阻塞的帧队列通常用固定帧池和槽位序号，采集线程只写空闲槽并发布小型句柄，不在关键采集路径等待消费者。单消费者可用序号观察新帧和覆盖；若采用丢旧帧保持新鲜度，消费者应识别序号跳跃并统计丢失。槽位只可在确定读者不再访问后复用。

多消费者要分别记录读位置或引用计数，不能只靠一个“已读”标志。槽内容必须先写完，再以合适的 release 状态发布，消费者用 acquire 读取状态后才访问内容。固定槽避免大帧复制，但无锁实现仍需处理序号回绕、ABA、进程崩溃及映射生命周期；超时回收前要确认进程不再访问。

代码示例

```
slot = fixed_pool.acquire_without_wait();
if (!slot)
    return DropAndCount;
write_frame(slot.buffer);
slot.sequence = next_sequence.fetch_add(1);
slot.state.store(Ready, std::memory_order_release); // 内容写完再发布 ready
consumer.load_state(std::memory_order_acquire);
verify_generation_and_sequence();
if (reader_finished)
    release_slot(); // 槽位回收前确认无消费者仍持有
// 固定槽避免大帧复制，生产者不等待；消费者靠序号发现覆盖/丢帧
// 多消费者要追踪每个读位置或引用数；序号回绕需防 ABA
// 超时回收需确认进程已不再访问映射，不能仅因租约过期就覆盖
```

第 12 章 C++ 进阶与工程

12.1 内存模型与缓存一致性

001 C++ 中 CPU 缓存一致性和 false sharing 是什么？

答案　CPU 缓存一致性协议协调多个核心中同一缓存行副本的读写权限。例如 MESI 家族用 M、E、S、I 描述行状态；一个核心要写共享行时，通常需要取得写权限并让其他副本失效。写直达与写回描述写入何时传向下级存储，写分配描述写未命中时是否先把行载入缓存，它们是不同策略。

一致性解决的是同一位置的副本如何协调，不保证多个位置访问的整体顺序，也不能替代 C++ 原子与锁；非原子并发冲突仍是数据竞争。false sharing 是不同线程修改不同变量，但变量恰好落在同一缓存行，导致行的写权限来回转移。可对热点变量分离或按目标硬件对齐，不过会增加空间，且缓存行大小依平台；先用 profiler 验证。〔I55、I56、S2〕

代码示例

```
#include <atomic>
#include <cstdint>
#include <thread>
struct alignas(64) PaddedCounter
{
```

```
    std::atomic<std::uint64_t> value{0};
};
PaddedCounter counters[2];
std::jthread a(
    [&]
    {
        ++counters[0].value;
    });
std::jthread b(
    [&]
    {
        ++counters[1].value;
    });
// 逻辑独立但同缓存行会反复转移，形成 false sharing 性能损失
// 64 是平台假设；对齐会增加内存，应先用 profiler/硬件常量验证
```

002 C++ 中 volatile 与 std::atomic 有什么区别?

答案　volatile 主要限制编译器对该 volatile 对象访问的某些优化，常用于平台规定的内存映射设备寄存器。它不提供原子性、线程间可见性或 happens-before 关系，不能用来修复普通共享变量的数据竞争。设备访问语义仍要遵循操作系统和硬件平台要求。

std::atomic 为特定对象提供原子操作和内存序，适合线程间共享状态；release/acquire 等可发布普通数据。但一个 atomic 只保护自身对象，两个字段共同满足的业务不变量仍要用锁或经过证明的协议。要在正确性需求上区分硬件 I/O 访问和线程同步，不要把 volatile 与 atomic 当成不同“速度档”的同一工具。

代码示例

```
#include <atomic>
#include <cstdint>
volatile std::uint32_t* device_register = platform_mapped_register();
*device_register = command; // volatile 用于平台定义的设备访问语义
std::atomic<bool> ready{false}; // 线程共享状态需要原子同步
ready.store(true, std::memory_order_release);
if (ready.load(std::memory_order_acquire))
    consume_published_data();
// volatile 不保证原子性、线程可见性或 happens-before；atomic 也不保护多字段
```

003 C++ 内存模型中的 happens-before 是什么?

答案　happens-before 是 C++ 内存模型描述先后和可见性的关系。同一线程内先完成的求值通常对后续求值形成 sequenced-before；跨线程则可由互斥锁解锁/加锁或 release/acquire 等同步操作建立关系。若发布线程先写普通数据，再用 release 存入原子标志，读取到该发布值的 acquire 线程就能看到此前写入。

没有 happens-before 关系的冲突非原子读写会构成数据竞争并导致未定义行为。只看线程启动顺序或“写线程先执行了”不足以证明其他线程能看到更新。设计时要指出具体哪个同步操作把读写连接起来，并确保生命周期也正确；join 可以同步线程完成，但不替代其他需要并发通信的同步协议。

代码示例

```
#include <atomic>
#include <thread>
int payload = 0;
std::atomic<bool> published{false};
std::thread producer(
    [&]
    {
        payload = 42;
        published.store(true, std::memory_order_release);
    });
while (!published.load(std::memory_order_acquire))
{
} // 观察到发布值
int value = payload; // 写入 happens-before 此读取
producer.join(); // acquire/release 同步普通数据访问；join 回收线程
```

12.2 模板、容器与线程安全

004 SFINAE 是什么?

答案　SFINAE 是“替换失败不是错误”：模板参数替换到某个候选声明时，如果候选所需的类型或表达式不成立，编译器可以把该候选从重载集中移除，再尝试其他候选。它常被用来检测一个类型是否有成员，或只让满足条件的模板参与重载。

替换失败规则适用于特定的模板替换阶段，不是函数体里任意编译错误都能被吞掉；若模板函数体实例化后访问不存在的成员，通常仍会报错。传统 SFINAE 错误信息可能难懂，C++20 concepts 可用 requires 更直接描述约束。维护旧代码时仍需理解其候选选择和重载行为。

代码示例

```
#include <type_traits>
#include <utility>
#include <vector>
template <class T, class = void> struct has_size : std::false_type
{
};
template <class T>
struct has_size<T, std::void_t<decltype(std::declval<T>().size())>> : std::true_type
{
};
static_assert(has_size<std::vector<int>>::value); // 无 size() 的类型选主模板
// 替换失败才会从候选集中移除；函数体实例化失败不自动算 SFINAE
// C++20 requires/concepts 更直观表达约束，但旧代码仍会遇到 SFINAE
```

005 STL 容器的线程安全边界是什么?

答案　标准容器的并发安全有边界。符合标准规则的并发 const 操作通常可以进行；除 vector<bool> 这类位打包特例外，对不同元素的某些并发修改也不会彼此形成数据竞争。多个线程写同一个元素仍要同步。读写同一元素也不能因为“容器本身是线程安全的”就任意并发。

会改变容器结构的操作，如 vector 的 push_back、resize、erase 或 unordered 容器 rehash，可能使迭代器和引用失效；这些操作与遍历或访问通常需要外部同步。不同元素写入的允许范围也不代表容器可一边结构修改一边读取。要共享容器，应锁住完整结构操作，并将锁覆盖迭代器使用期。

代码示例

```
#include <thread>
#include <vector>
std::vector<int> values{1, 2, 3, 4};
std::jthread a(
    [&]
    {
        values[0] = 10;
    });
std::jthread b(
    [&]
    {
        values[1] = 20;
    }); // 修改不同普通元素
// 两个线程不应同时 push_back/resize 与遍历：结构和迭代器会改变
// 写同一元素需同步；vector<bool> 位打包有特殊风险
// 并发 const 访问也不能与未同步结构修改同时进行
```

006 shared_ptr 是线程安全的吗?

答案　不同 shared_ptr 实例若共同管理同一对象，它们共享一个控制块；引用计数的增减可按标准支持的并发方式安全进行。例如把 shared_ptr 的副本分别交给两个线程使用，不会因两边各自释放而提前销毁对象。这里说的是控制块计数，不等于任意 shared_ptr 变量都可以无同步并发读写。

若多个线程会同时对同一个 shared_ptr 变量赋值或重置，仍要用锁保护该变量，或在支持的标准库接口中使用 atomic<shared_ptr>。所指对象的成员状态也需要自己的同步设计：shared_ptr 只管理生命周期，不自动保护 Widget 内容。若所有权关系形成环，计数同样无法归零，需考虑 weak_ptr。

代码示例

```
#include <memory>
#include <thread>
std::shared_ptr<Widget> shared = std::make_shared<Widget>();
std::shared_ptr<Widget> left = shared, right = shared;
std::jthread a(
```

```
    [copy = left]
    {
        use(copy);
    }); // 独立 shared_ptr 实例
std::jthread b(
    [copy = right]
    {
        use(copy);
    }); // 共享控制块计数可并发更新
// 同一个 shared_ptr 变量若同时读写仍需同步；Widget 内部也要自行保护
```

007 std::function 与模板回调有什么取舍?

答案　std::function 为固定调用签名提供类型擦除，可以在一个字段、容器或注册表中保存不同具体类型的回调，适合运行时才决定回调对象的接口。它会隐藏目标类型并经统一接口转发，可能增加间接调用；目标太大时也可能动态分配，但小对象优化大小和行为由实现决定。

模板回调保留传入对象的具体类型，编译器容易内联并在编译期检查操作，但每种类型可能生成独立实例，接口实现通常必须对调用方可见。局部算法或一次性调用常用模板；需要异构的长期存储时 std::function 更方便。选择时考虑复制能力、生命周期、运行时替换和分配成本，而不是把其中一种当作总是更快。

代码示例

```
#include <functional>
#include <utility>
std::function<void(Event)> callback = [state](Event e)
{
    state.handle(e);
};
registry.add(callback); // 固定签名，可运行时存不同闭包类型
// 类型擦除带间接调用；目标可能分配，SBO 依具体实现
auto templated_call = [](auto&& f, auto&& value)
{
    std::forward<decltype(f)>(f)(std::forward<decltype(value)>(value));
};
templated_call(
    [](Event e)
    {
        handle(e);
    },
    event); // 具体类型利于内联
// 异构长期存储用 function；局部泛型算法通常用模板
```

008 std::vector 扩容和迭代器失效有什么影响?

答案　vector 容量不足时通常申请一块新存储，把现有元素迁移过去再释放旧存储，因此扩容会使旧元素的指针、引用和迭代器全部失效。尾部插入在多次操作中平均摊还 O(1)，但单次扩容仍可能是 O(n)。已知元素规模时 reserve 可降低重新分配次数。

reserve 本身若需要扩大容量，也会使原句柄失效；若容量足够，后续 push_back 通常至少会让旧 end 迭代器失效。erase 会把后续元素向前移动，因此被删位置及其后的迭代器和引用失效。删除循环应使用 erase 返回的新迭代器，不要继续递增已失效的旧值。

代码示例

```
#include <vector>
std::vector<Item> items;
items.reserve(100); // 已知规模时预留，降低扩容次数
items.push_back(first);
auto it = items.begin();
items.push_back(next); // 若容量不足扩容，全部元素迭代器/指针/引用失效
// reserve 只降低失效概率；erase 也会使删除点及之后句柄失效
```

12.3 零拷贝、异常与并发问题

009 如何实现零拷贝？零拷贝是否意味着完全不复制?

答案　“零拷贝”通常指让数据传输路径减少某些显式 memcpy，例如用 mmap、sendfile、共享内存或设备 DMA。它不表示数据完全没有移动：系统可能仍要更新页表或引用计数、转换格式，设备也可能实际搬运数据。要根据具体接口和平台追踪数据从源头到目标经过哪些组件。

例如 read 加 write 通常把数据先放用户缓冲区；mmap 可省去读入用户缓冲这一步，但普通 write 仍可能复制到 socket 路径，mmap 也不保证立即从磁盘读入。sendfile 可让内核连接文件与 socket 数据路径，降低 CPU 搬运，但要处理短传输、EAGAIN 和源缓冲寿命。系统调用切换到内核态不必然引发线程调度切换，不能凭固定“复制次数”推断性能翻倍。〔I24、N44〕

代码示例

```
#include <cstddef>
#include <span>
#include <vector>
void checksum(std::span<const std::byte> bytes)
{
    consume(bytes);
}
std::vector<std::byte> buffer = read_once();
checksum(buffer); // 借用缓冲，不复制字节
// mmap/sendfile/共享内存可减少特定路径 memcpy，但页表/DMA 仍有成本
// 仍须保证 buffer 生命周期、所有权移交、并发访问和背压
```

010 异常安全的三个常见保证是什么?

答案　异常安全常见分三档。不抛出保证表示异常不会逃出操作；强保证表示失败时程序状态保持原样，像事务回滚；基本保证表示对象仍然有效且资源不泄漏，但部分状态可能已经改变。具体操作是否达到哪档，应按异常发生点检查资源和对象不变量。

RAII 管理资源可以避免异常路径泄漏。先构造一个临时完整结果，再通过不抛异常的交换提交新状态，常能提供强保证，例如 copy-and-swap。noexcept 是调用者可依赖的承诺，若异常逃出，运行时会调用 terminate；只有实现真实不会抛时才能标记，不能只为让容器看起来更快。

代码示例

```
class Value
{
  public:
    Value& operator=(const Value& other)
    {
        Value temp(other); // 强保证：复制失败时当前状态不变
        swap(*this, temp);
        return *this; // ADL 选本类的非抛出 friend swap
    }
    friend void swap(Value&, Value&) noexcept;
};
// 基本保证：对象有效且无泄漏；不抛出保证：异常不外传
// noexcept 只有操作真实不抛时才能承诺
```

011 无锁队列中的 ABA 问题是什么?

答案　ABA 指 CAS 线程读到状态 A 后，另一线程把状态改为 B，又改回 A；原线程只比较当前值，发现仍等于 A，就误以为期间什么都没发生。无锁栈里节点地址释放后被重新分配到相同地址，是常见来源。算法因此可能使用已经失效的关联状态。

把版本号和指针一起比较可检测一部分 ABA；版本回绕仍需评估。hazard pointer 或 epoch 可以避免其他线程还在访问时释放节点，但这些回收方案需要完整的并发设计。解决 ABA 不能只给指针加计数，还要证明节点内容发布、访问和回收的内存序与寿命都正确。

代码示例

```
#include <atomic>
#include <cstdint>
struct TaggedHead
{
    Node* ptr;
    std::uint64_t version;
};
std::atomic<TaggedHead> head; // CAS 同比地址和版本，部分检测 ABA
bool replace(TaggedHead old, Node* next)
{
    TaggedHead desired{next, old.version + 1};
    return head.compare_exchange_strong(old, desired);
}
// 仍需 hazard pointer/epoch 确保 Node 未提前释放；版本会回绕且未必 lock-free
```

012 移动构造为什么常标记 noexcept?

答案　vector 扩容时要迁移旧元素。若移动构造承诺不抛异常，容器可以放心使用移动；若移动可能抛而复制可用，实现可能改用复制来维持强异常保证。因此对资源类型，一个正确且真实的 noexcept 移动构造可以避免不必要的深拷贝。

这不是给移动构造加速的装饰。若函数标 noexcept 却让异常逸出，会调用 terminate。标记前要检查成员的移动操作是否也不抛，并确认源对象转移后仍可安全析构。若资源移动失败语义复杂，保留可抛移动并让容器复制回退，可能比做出错误承诺更安全。

代码示例

```
#include <utility>
#include <vector>
struct Item
{
    std::vector<int> data;
    Item(Item&& other) noexcept
        : data(std::move(other.data))
    {
    } // 移动确实不抛
};
std::vector<Item> items;
items.reserve(100); // 降低扩容搬移次数
// move 可能抛而 copy 可用时，vector 为强保证可能选择 copy
// noexcept 错误标注会让逃逸异常调用 terminate
```

013 结构体成员重排为什么可能节省内存?

答案　结构体成员要满足各自的对齐要求，编译器可能在成员之间或结构体尾部插入填充。例如把一个高对齐的 double 放在 char 前面，有时能减少中间空洞；但大小和对齐取决于目标 ABI，改完必须实测。并非所有成员顺序都能通过重排降低总大小。

成员顺序也是对象布局的一部分。重排会改变成员偏移、sizeof、二进制兼容性，可能破坏与外部模块或文件的约定。若结构是公开 ABI 类型，先评估兼容影响；网络和持久化格式则明确字段编码和版本，不要直接把本机结构体原始字节写出去。优化布局还要平衡访问方式和可读性。

代码示例

```
#include <cstddef>
struct Before
{
    char tag;
    double value;
    int count;
};
struct After
{
    double value;
    int count;
    char tag;
};
std::size_t a = sizeof(Before), b = sizeof(After); // 只比较当前 ABI
// 成员对齐会插入填充；将高对齐成员前置有时能减少空隙
// 重排会改变成员偏移/sizeof 并破坏 ABI
// 网络或持久化数据应显式序列化，不能依赖本机内存布局
```

014 迭代器失效通常如何处理?

答案　迭代器失效规则由容器和操作决定。vector 扩容会使所有指向元素的迭代器、指针和引用失效；未扩容时尾部插入通常至少使旧 end 失效。list 插入一般不影响其他节点迭代器，删除只使被删节点的迭代器失效；unordered 容器 rehash 会使迭代器失效。

不能只记一个“插入是否失效”的通用结论，应查具体容器和操作。erase 常返回下一个有效迭代器，遍历删除时用返回值继续；若调用会让存储重分配的操作，需重新获取迭代器。对保存很久的引用和指针也要检查同样的生命周期，reserve 只能改变某些 vector 操作使其失效的概率。

代码示例

```
#include <vector>
for (auto it = values.begin(); it != values.end();)
{
    if (should_remove(*it))
```

```
        it = values.erase(it);
    else
        ++it;
}
// erase 返回下一个有效迭代器，删除后不要再用旧 it
// vector 扩容使所有迭代器失效；list 插入通常不影响其他节点迭代器
// unordered_map rehash 会使迭代器失效；具体规则按容器和操作查文档
```

第 13 章 数据结构与算法

13.1 基础动态规划与数组问题

001 0/1 背包问题（0/1 Knapsack）

答案　0/1 背包是在容量有限的背包里挑物品，让总价值尽量大。每件物品只有一件，因此只能选一次或不选，这就是“0/1”的含义。可以把问题拆成：只考虑前几件物品、容量为某个数时，最好能装出多少价值。用 dp[c] 记录这个最好结果，不要求把背包装满时，初始值可以全部设为零。

处理一件重量为 w、价值为 v 的物品时，有“不选它”和“选它”两种方案。前者保持 dp[c]，后者是在容量 c-w 的旧方案上加 v，取较大者。容量必须从大到小更新，因为选当前物品时要读取尚未包含它的旧状态；正序会重复使用本件物品。例如只有一件重二、价值三的物品，容量四也只能得到三。

共有 n 件物品、容量上限为 W 时，时间为 O(nW)，额外空间为 O(W)。这是依赖容量数值的算法，容量极大时并不便宜。若题目要求“恰好装满”，不能把所有状态初始化为零，应把不可达状态标记出来。零重量物品在本题也只能计入一次。

代码示例

```
// dp[c] 只由已经处理的物品决定，容量倒序是 0/1 约束的关键。
// dp[c]：已处理物品在容量不超过 c 时的最大价值；允许不选物品
std::vector<long long> dp(W + 1, 0);
for (const auto& item : items)
{
    for (int c = W; c >= item.weight; --c) // 倒序保证本轮不会重复用同件物品
        dp[c] = std::max(dp[c], dp[c - item.weight] + item.value);
}
return dp[W]; // O(nW) 时间、O(W) 空间；重量为 0 时需按约束处理
```

002 K 个一组翻转链表的思路是什么？

答案　K 个一组翻转是把链表切成连续的小组，只调换每组内部节点的连接方向。例如 K 为二时，1→2→3→4→5 变成 2→1→4→3→5；最后不足两个节点的部分保持原样。通常约定 K 为正数，K 为一时直接返回。这里移动的是节点之间的指针，不是交换节点内的数值。

每次先向后探测，确认剩余节点够一整组，再开始修改。需要保存组前节点、原组首和组后节点：组内反转后，原组尾成为新组首，原组首成为新组尾。把前一段接到新组首，再从新组尾继续处理下一组。设置一个指向原头节点的虚拟节点，就能让第一组和后续组使用相同接法。

最容易出错的是反转时丢掉下一组的入口。因此组后节点必须提前单独保存，不能边修改组尾的 next 边把它当循环终点。每个节点只经过有限次探测和修改，总时间 O(n)，只用几个指针，额外空间 O(1)。原链表会被改变，需要保留输入时应先复制。

代码示例

```
// prev/cur/next 维护组内反转；组外连接点必须先保存。
// dummy 统一处理首组；不足 k 个的尾段不反转
Node dummy{0, head};
Node* before = &dummy;
while (true)
{
    Node* end = before;
    for (int i = 0; i < k && end; ++i)
        end = end->next;
    if (!end)
        break;
    Node* oldFirst = before->next;
    Node* prev = end->next;
    Node* cur = oldFirst;
    while (cur != end->next)
    {
        Node* next = cur->next;
```

```
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    before->next = end;
    before = oldFirst;
}
return dummy.next; // 每个节点只改指针，O(n) 时间、O(1) 额外空间
```

003 三数之和如何去重并求解？

答案　三数之和要找的是三个不同位置的元素，使它们之和等于目标，常见题目把目标设为零。同样的三个数只算一组，即使数组里有很多重复值，也不能重复输出。先排序的好处是：可以根据和的大小决定下一步往哪边移动，不必枚举所有三个下标。

先固定第一个数，再在它右边放两个指针。三数和偏小，就把左指针向右移，尝试更大的数；和偏大，就把右指针向左移。找到答案后，跳过左右两侧相同的数；外层固定的数如果与上一轮相同，也直接跳过。例如 [-1,-1,0,1,2] 中，[-1,0,1] 和 [-1,-1,2] 都只保留一次。

排序之后，每次固定一个数，左右指针总共最多走过数组一次，所以主体时间为 $O(n^2)$，保存结果的空间另算。求和时要先提升到足够宽的整数类型，避免三个合法整数相加却溢出。一般目标值下不能随意套用“当前数大于零就结束”的剪枝，它只适用于零目标等满足条件的场景。

代码示例

```
// 固定首数后双指针单调逼近目标，排序后同值结果要去重。
std::sort(a.begin(), a.end());
for (int i = 0; i + 2 < a.size(); ++i)
{
    if (i && a[i] == a[i - 1])
        continue;
    int l = i + 1, r = a.size() - 1;
    while (l < r)
    {
        long long sum = (long long)a[i] + a[l] + a[r];
        if (sum < target)
            ++l;
        else if (sum > target)
            --r;
        else
        {
            out.emplace_back(a[i], a[l], a[r]);
            int x = a[l], y = a[r];
            while (l < r && a[l] == x)
                ++l;
            while (l < r && a[r] == y)
                --r;
        }
    }
} // O(n^2)；求和先扩为 64 位，避免整数溢出
```

004 不同路径数（Unique Paths）

答案　这题通常规定在矩形网格中只能向右或向下走，问从左上角到右下角有多少种走法。关键是看“最后一步”：到达一个普通格子，只可能来自它上面或左边，因此到这里的路径数等于这两处路径数相加。不同方向限制下结论会变，不能把允许回头的情况直接套进来。

用一维数组保存当前进度即可。从左到右处理一行时，尚未更新的 dp[c] 表示上方格子的路径数，已经更新的 dp[c-1] 表示左方格子的路径数，两者相加就是新状态。起点赋为一，表示有一种“从这里出发”的方式。若题目带障碍，障碍格直接设为零，表示所有经过这里的路径都被切断。

例如没有障碍的二行三列网格共有三种走法；起点或终点被堵则为零。空网格要先单独处理，路径数量较大时应使用宽整数，必要时用大整数或按题目要求取模。总时间为行数乘列数，即 O(mn)，额外空间为 O(n)，其中 n 是列数。

代码示例

```
// 一维 dp[c] 在本行代表左侧，尚未覆盖时代表上方。
// dp[c]：当前行到列 c 的路径数；障碍格状态为 0
std::vector<long long> dp(n, 0);
for (int r = 0; r < m; ++r)
    for (int c = 0; c < n; ++c)
    {
```

```
        if (grid[r][c] == '#')
            dp[c] = 0;
        else if (r == 0 && c == 0)
            dp[c] = 1;
        else
            dp[c] += c ? dp[c - 1] : 0; // 旧 dp[c] 是上方，已更新 dp[c-1] 是左方
    }
return dp[n - 1]; // O(mn) 时间、O(n) 空间；路径数大时选宽整数
```

005 买卖股票的最佳时机（Best Time to Buy and Sell Stock）

答案　这里的限制是最多买一次、卖一次，而且必须先买后卖。思路是把每一天都当作卖出日，只问“如果今天卖，之前哪一天买最便宜”。遍历过程中记住此前最低价，就不必对每个卖出日重新往前找，也不会错误地拿未来的低价充当买入价。

例如价格为 [7,1,5,3,6,4]，走到价格六时，之前最低价是一，所以能赚五。维护两个量即可：此前最低价，以及已经找到的最大利润。先用今天的价格计算卖出利润，再把今天的价格纳入最低价，供未来日期使用。这样循环中“此前”这个含义一直成立。

不交易也是合法选择，因此最大利润从零开始；如果价格一直下降，结果就是零，而不是一个负数。空数组或只有一天时也无法完成交易。整个算法只扫描一次，时间 O(n)，额外空间 O(1)。若允许多次买卖、收手续费或卖出后冷冻，状态和规则都要重新设计，不能继续只用这两个变量。

代码示例

```
// lowest 始终是当前卖出日之前的最低价格。
if (prices.empty())
    return 0;
int lowest = prices[0], best = 0;
for (int p : prices)
{
    best = std::max(best, p - lowest); // 先卖出计算，再把当天纳入未来买入价
    lowest = std::min(lowest, p);
}
return best; // 一次遍历、O(1) 空间；下跌行情返回 0
```

13.2 树与图的遍历

006 二分图检测（Bipartite Graph Detection）

答案　二分图是可以把所有顶点分成两组，而且每条边都只连接不同组的图。把两组想成红色和蓝色，就变成：能否给所有顶点染色，使相邻点颜色不同。只要任意一条边两端必须同色，就说明这样的划分不存在；不要求两组人数一样多。

从一个未染色的顶点开始，先染红色，再把它的邻居染蓝色，邻居的邻居染红色。用队列逐层扩展即可。遇到已经染色的邻居时，要检查颜色是否冲突。三角形最后会出现一条边连接两个同色点，因此不是二分图；四边形则可以交替染色。

图可能由互不连通的几块组成，所以只从第一个顶点搜索不够，必须对每个尚未染色的顶点重新启动。孤立点不会造成冲突，自环则必然冲突。邻接表表示下，每个点和每条边只处理有限次，时间 O(V+E)，颜色表与队列需要 O(V) 空间；邻接矩阵的扫描成本会更大。

代码示例

```
// 相邻点必须异色；每个连通分量都要独立启动染色。
std::vector<int> color(n, -1);
std::queue<int> q;
for (int start = 0; start < n; ++start)
    if (color[start] < 0)
    {
        color[start] = 0;
        q.push(start);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int v : graph[u])
            {
                if (color[v] < 0)
                {
                    color[v] = color[u] ^ 1;
                    q.push(v);
```

```
                }
                else if (color[v] == color[u])
                    return false; // 自环也会冲突
            }
        }
    }
return true; // 每个分量染色通过即为二分图，O(V+E) 时间、O(V) 空间
```

007 二叉搜索树的第 k 大元素

答案　二叉搜索树的基本约定是左子树的值较小、右子树的值较大。因此按“左子树、根、右子树”访问会得到升序；反过来按“右子树、根、左子树”访问，就得到降序。要找第 k 大元素，只需要在降序访问到第 k 个节点时停下，不必把所有值排序一遍。

例如一棵树的升序结果是 [2,4,6,8]，反向中序就依次看到八、六、四、二，第二个是六。递归时先进入右子树，回来后处理当前节点并减少剩余计数，计数变零就保存答案，再停止所有后续搜索。函数要区分“尚未找到”和合法的零值答案，不能把零当作未找到标志。

通常 k 从一开始，必须检查非正数和超过节点总数的情况。重复值按节点计数还是按不同数值计数，应先与题意一致。能提前停止时，时间通常为 O(h+k)，h 为树高；退化树最坏仍为 O(n)。递归额外空间 O(h)，树很深时可用显式栈替代。

代码示例

```
// 右根左顺序形成降序访问，计数到 k 后可提前终止。
// 反向中序为降序；假设 k 从 1 开始且 1 <= k <= 节点数
void visit(Node* p)
{
    if (!p || found)
        return;
    visit(p->right);
    if (!found && --k == 0)
    {
        answer = p->value;
        found = true;
    }
    visit(p->left);
}
visit(root); // O(h+k) 量级的提前停止遍历，递归栈 O(h)
// 若 k 越界由 found=false 表示未找到；极斜树可改显式栈
```

008 二叉树的前序、中序、后序遍历如何实现？

答案　前序、中序和后序的区别，是在什么时候处理当前节点。前序是“根、左、右”，先看到父节点再看孩子；中序是“左、根、右”，对二叉搜索树能得到升序；后序是“左、右、根”，先处理孩子再处理父节点，适合统计子树高度或释放树。这里的“序”指访问顺序，不是树本身变化。

递归代码的骨架完全一样：空节点直接返回，非空节点继续处理左右子树。把“记录当前值”的操作放在两次递归之前、之间或之后，就分别得到三种遍历。例如根为二、左孩子为一、右孩子为三时，前序是 [2,1,3]，中序是 [1,2,3]，后序是 [1,3,2]。

每种遍历都会访问每个节点一次，时间 O(n)。不计算输出数组时，递归调用占 O(h) 空间，h 是树高。若树退化成长链，递归太深可能耗尽调用栈，可改用自己维护的栈。迭代前序中要先压右孩子再压左孩子，因为栈是后进先出；层序遍历则应使用队列。

代码示例

```
// 递归位置决定根左右三种访问次序，空节点是递归边界。
void preorder(Node* p)
{
    if (!p)
        return;
    out.push_back(p->value);
    preorder(p->left);
    preorder(p->right);
}
void inorder(Node* p)
{
    if (!p)
        return;
    inorder(p->left);
    out.push_back(p->value);
    inorder(p->right);
```

```
}
void postorder(Node* p)
{
    if (!p)
        return;
    postorder(p->left);
    postorder(p->right);
    out.push_back(p->value);
}
// 分别按根左右、左根右、左右根访问；每次遍历 O(n)，递归栈 O(h)
// 极深退化树可用显式栈避免递归深度过大
```

009 二叉树的层次遍历

答案　层次遍历就是先访问根，再访问根的所有孩子，然后访问下一层，像按楼层逐层点名。适合使用先进先出的队列：较早加入的浅层节点先被取出，它们的孩子排到后面，因此自然保证从上到下访问。同一层通常按从左到右的顺序输出。

如果需要把每一层单独放进一个数组，就要在开始这一层时记住队列长度，只取出这么多个节点。过程中加入的孩子属于下一层，不能在本轮继续取完，否则所有层会混在一起。例如根为一，孩子为二和三，二还有孩子四，最终分组应是 [[1],[2,3],[4]]。

空树直接返回空列表。普通树没有环，每个节点只入队一次，时间 O(n)，队列所需空间与最大层宽 w 同阶，为 O(w)，最坏可以达到 O(n)。如果把相同方法用于一般图，必须额外维护已访问集合，并在入队时标记，避免环或多条路径造成重复入队。

代码示例

```
// 每轮固定队列长度，保证本层节点与新入队孩子不混淆。
if (!root)
    return levels; // 空树返回空列表
std::queue<Node*> q;
q.push(root);
while (!q.empty())
{
    int width = q.size();
    std::vector<int> level;
    while (width-- > 0)
    {
        Node* p = q.front();
        q.pop();
        level.push_back(p->value);
        if (p->left)
            q.push(p->left);
        if (p->right)
            q.push(p->right);
    }
    levels.push_back(std::move(level)); // 本轮固定 width，孩子留到下一层
}
// 每节点入队一次：O(n) 时间，队列最多 O(w) 个节点
```

010 二叉树的最大深度

答案　二叉树的最大深度，通常指从根走到最远叶子时经过的节点数。空树深度为零，只有根的树深度为一。看任何一个节点时，它下面最深的路线一定在左子树或右子树中，因此“当前树深度”就是“左右子树深度的较大值，再加当前节点这一层”。

可以让递归函数直接返回这棵子树的高度：遇到空指针返回零，先问左孩子和右孩子各有多高，再把较大值加一返回。这样最底层先算好，上层逐步汇总，根节点拿到的就是整棵树的答案。例如根下面有一条连续三节点的路径，没有更长路径时，最大深度就是三。

每个节点只计算一次，所以时间 O(n)。递归调用最多同时保留一条根到叶子的路径，空间为 O(h)，其中 h 是树高。长链状的输入可能让递归调用过深，应改为显式栈或按层遍历。还要确认题目按节点数还是边数定义深度：非空树按边计时比按节点计少一，空树则需单独约定。

代码示例

```
// 深度按节点数定义，空子树返回 0。
int depth(Node* p)
{
    if (!p)
        return 0; // 按节点数计深度，空树为 0
```

```
    int left = depth(p->left), right = depth(p->right);
    return 1 + std::max(left, right);
}
// 每节点访问一次，O(n) 时间、O(h) 递归空间；按边计时定义会不同
```

011 二叉树的最大路径和

答案　树中的路径是一条不重复经过节点的连续路线，可以从任意节点开始，在任意节点结束，不一定经过根。最大路径和就是这条路线上的值之和最大。难点在于，一条完整路径可以在某节点从左侧转向右侧，但传给父节点的路径只能从当前节点向一侧延伸，否则会出现分叉，不再是一条路径。

因此递归要同时处理两个不同的量。返回给父节点的是“从我开始，往下一条分支能取得多少收益”；更新全局答案时，则尝试“左侧收益＋当前值＋右侧收益”。某侧收益为负就不选它，相当于这一侧贡献零。例如根值一、左右孩子为二和三，完整路径二→一→三的和为六，但向父节点只能贡献一加三。

全局最大值必须从负无穷开始，不能从零开始，因为题目通常要求路径至少包含一个节点。全负树应返回最大的那个负数，而不是不存在的空路径。空树的结果要由接口另行约定。后序遍历每节点一次，时间 O(n)，额外栈空间 O(h)；数值之和可能溢出时应使用宽整数。

代码示例

```
// best 是路径可在当前节点拐弯的答案，返回值只能向父节点单边延伸。
long long gain(Node* p)
{
    if (!p)
        return 0;
    long long left = std::max(0LL, gain(p->left));
    long long right = std::max(0LL, gain(p->right));
    best = std::max(best, p->value + left + right); // 以 p 为拐点的完整路径
    return p->value + std::max(left, right); // 向父节点只能延伸一侧
}
long long best = std::numeric_limits<long long>::lowest();
gain(root);
// 全负树不能把 best 初始化成 0；时间 O(n)、栈 O(h)
```

012 二叉树的最小深度

答案　最小深度是从根到最近叶子的节点数。叶子必须左右孩子都不存在，只有一侧为空的节点还不是叶子。这个定义很重要：如果根只有一个左孩子，不能因为右侧为空就说深度是一，必须沿着左侧继续走到真正的叶子。空树的最小深度通常约定为零。

按层搜索最容易理解。根带着深度一入队，每次取一个节点，如果它是叶子就立刻返回深度；否则把存在的孩子连同“深度加一”入队。队列保证先处理浅层再处理深层，所以第一次取出的叶子一定最近，不需要继续检查所有更深节点。也可以用递归，但一侧为空时只能选择非空子树；只有两侧都存在时，才取左右深度的较小值再加一。最坏仍要遍历所有节点，时间 O(n)。队列占用与树宽有关，最坏 O(n)；递归实现占 O(h) 栈空间。不要把最大深度公式里的 max 直接机械地改成 min。

代码示例

```
// 只有左右孩子均空才是叶子；BFS 按深度递增弹出。
if (!root)
    return 0;
std::queue<std::pair<Node*, int>> q;
q.push({root, 1});
while (!q.empty())
{
    auto [p, d] = q.front();
    q.pop();
    if (!p->left && !p->right)
        return d; // BFS 首个叶子就是最近叶子
    if (p->left)
        q.push({p->left, d + 1});
    if (p->right)
        q.push({p->right, d + 1});
}
// 只有一个孩子时必须沿非空孩子走，不能把空子树当叶子；O(n) 时间
```

013 二叉树的镜像

答案　二叉树镜像就是把整棵树左右对调，好像把树放到镜子前。操作不仅要交换根的两个孩子，还要交换每个子节点的左右孩子。例如根为一、左孩子二、右孩子三，镜像后根仍为一，左变成三、右变成二；两棵子树内部也必须同样翻转。

递归做法很直接：空树不用处理，非空节点先交换左右指针，再对交换后的两棵子树继续镜像。每次只负责当前节点，重复同样的规则就覆盖了所有节点。交换后先处理哪一侧都可以，但要保证两侧各处理一次，不要错误地反复处理同一棵子树。
原地交换只改变链接，不新建节点，时间 O(n)，递归栈空间 O(h)。如果其他对象也引用这棵树，它们看到的结构也会变化；需要同时保留原树和镜像树时，应新建节点或先深拷贝，空间会增加到 O(n)。树非常深时可以使用显式栈遍历，避免调用栈过深。
代码示例
```
// 交换左右孩子后递归，原地镜像会修改输入树。
Node* invert(Node* p)
{
    if (!p)
        return nullptr;
    std::swap(p->left, p->right); // 先交换，再递归处理两侧
    invert(p->left);
    invert(p->right);
    return p;
}
// 每个节点访问一次，O(n) 时间、O(h) 递归栈；原地修改，需保留原树时先深拷贝
// 极深树可换显式栈，避免递归栈深度受限
```
014 从前序和中序遍历构造二叉树
答案　前序遍历先访问根，所以当前前序区间的第一个值就是根。中序遍历按“左、根、右”，找到根在中序区间的位置，就知道哪些值属于左子树、哪些属于右子树，还能算出左子树有多少节点。这个节点数量又能把剩余前序区间准确切成左右两段。
例如前序 [3,9,20,15,7]、中序 [9,3,15,20,7]，根是三，中序中三的左边只有九，所以前序里紧随根的一个元素属于左子树，后面的二十、十五、七属于右子树。对子区间重复同样的方法，遇到空区间就结束。用下标表示区间，避免递归时不停复制数组。
通常要求节点值互不重复，否则同一个根值可能对应多个分割位置，原树未必能唯一恢复。先建立“值到中序下标”的哈希表，平均总时间 O(n)，额外空间 O(n)，其中还包括递归栈。输入需校验长度、元素及递归区间一致性；仅集合相同也不能保证任意两种顺序都合法。
代码示例
```
// 前序当前首项给根，中序根位置给左子树长度。
// pos[value] 为中序下标；唯一值，区间均为左闭右开
Node* build(int pl, int il, int ir)
{
    if (il == ir)
        return nullptr;
    int root = pre[pl], k = pos[root], leftSize = k - il;
    Node* left = build(pl + 1, il, k);
    Node* right = build(pl + 1 + leftSize, k + 1, ir);
    return new Node(root, left, right);
}
// 预建哈希表后 O(n) 时间、O(n) 空间；先校验两遍历元素一致
```
015 从后序和中序遍历构造二叉树
答案　后序遍历的顺序是“左、右、根”，所以当前后序区间的最后一个值就是根。中序遍历把左子树、根、右子树排在一起，找到这个根的位置，就能确定左右子树的范围和节点数量。再按数量分割后序区间，递归处理即可，思路与前序加中序建树相同，只是根的位置变了。
例如中序 [1,2,3]、后序 [1,3,2]，后序最后的二是根，中序中它两边分别是一和三，因此可以直接得到左右孩子。如果采用区间参数，左右子树可以按任意顺序构造；如果改成用一个指针从后序末尾依次取值，就必须先建右子树，再建左子树，因为倒着看的顺序是“根、右、左”。
用哈希表定位中序下标，平均时间 O(n)，额外空间 O(n)。用下标范围传参能省去重复复制子数组的开销。该方法通常要求节点值唯一，而且两份序列要彼此一致；查到的根如果不在当前中序区间，说明输入非法，不能继续构造或越界访问。
代码示例
```
// 后序当前末项给根，中序根位置分割左右子树。
// pos[value] 为中序下标；后序末项为根，节点值唯一
Node* build(int pl, int pr, int il, int ir)
{
    if (il == ir)
        return nullptr;
```

```
    int root = post[pr - 1], k = pos[root], leftSize = k - il;
    Node* left = build(pl, pl + leftSize, il, k);
    Node* right = build(pl + leftSize, pr - 1, k + 1, ir);
    return new Node(root, left, right);
}
// 用区间避免复制子数组；哈希定位后 O(n) 时间、O(n) 空间
```

13.3 排序、查找与有序结构

016 冒泡排序（Bubble Sort）

答案　冒泡排序每次比较相邻两个数，如果左边更大就交换，让较大的数不断向右走。一轮结束，未排序部分的最大值就到达最右端，之后可以把这一格排除。名字里的“冒泡”只是描述大元素逐步移动的过程，不需要另外建立什么数据结构。

例如 [3,1,2]，先比较三和一，交换后得到 [1,3,2]，再比较三和二，得到 [1,2,3]。这一轮已经把最大值三送到末端。下一轮只检查前两个数。如果整轮没有发生交换，说明相邻元素都已经有序，整个数组也就有序，可以提前结束。

只在严格大于时交换，值相同的元素不会互相越过，因此冒泡排序是稳定排序；原地交换只需 O(1) 额外空间。带提前结束的版本在本来有序时只扫一轮，最好时间 O(n)，平均和最坏 O(n^2)。它适合理解排序或处理很小的数据，规模较大时交换次数太多，应选择更高效的方法。

代码示例

```
// 每轮把最大值沉到未排序区末端，无交换表示已有序。
for (int end = n - 1; end > 0; --end)
{
    bool changed = false;
    for (int i = 0; i < end; ++i)
        if (a[i] > a[i + 1])
        {
            std::swap(a[i], a[i + 1]);
            changed = true;
        }
    if (!changed)
        break; // 已有序时提前结束
}
// 相等元素不交换所以稳定；原地，最好 O(n)，平均/最坏 O(n^2)
```

017 判断一个二叉树是否为二叉搜索树（BST）

答案　判断二叉搜索树，不能只看每个节点是否“左孩子小、右孩子大”。要求是整棵左子树的所有值都小于当前值，整棵右子树的所有值都大于当前值。例如根为十，左孩子为五，五的右孩子为十二，虽然十二大于它的父节点五，却落在十的左子树里，因此仍然不合法。

可以给每个递归节点带一个允许范围。根最初没有上下界；去左子树时，把上界收紧为当前值；去右子树时，把下界收紧为当前值。这样祖先的限制会一直传下去，十二到达时仍然受到“小于十”的约束。用“边界不存在”的标记比拿类型最大最小值当哨兵更稳妥，避免误拒绝合法极值。

默认采用严格大小关系，也就是不允许重复值。若题目允许相同键放在某一侧，必须统一调整对应边界的开闭，不能两边都随意放。每节点只检查一次，时间 O(n)，递归空间 O(h)。另一种做法是检查中序序列是否严格递增，但也要处理初值及重复值的约定。

代码示例

```
// 祖先上下界同时约束节点，检查远端子树是否越界。
bool valid(Node* p, long long low, long long high)
{
    if (!p)
        return true;
    if (p->value <= low || p->value >= high)
        return false; // 严格 BST 边界
    return valid(p->left, low, p->value) && valid(p->right, p->value, high);
}
bool isBst =
    valid(root, std::numeric_limits<long long>::lowest(), std::numeric_limits<long long>::max());
// 若允许重复键，需按约定改开闭边界；O(n) 时间、O(h) 栈
```

018 判断一个二叉树是否是平衡二叉树

答案　高度平衡二叉树要求：每个节点的左右子树高度之差都不超过一，而且左右子树自己也必须平衡。只比较根的左右高度不够，因为某个深处的子树可能已经歪成一条长链。空树通常算平衡树，单节点树也满足条件。

用后序遍历可以一遍完成判断。先求左右子树高度，再检查高度差；如果子树已经不平衡，立即把失败结果传给父节点。可以约定返回负一表示不平衡，正常高度则是零或正数。父节点收到负一后就无需再把它当正常高度参与计算。这样一个返回值同时表达“是否平衡”和“高度是多少”。

如果对每个节点都重新递归计算左右高度，相同子树会被反复扫描，长链输入可能需要 $O(n^2)$ 时间。合并计算后，每节点最多处理一次，时间 O(n)，递归空间 O(h)。高度差阈值是这个定义的约定；二叉搜索树与平衡树是不同性质，判断平衡本身不要求节点值有序。

代码示例

```
// 后序返回高度，-1 作为失衡哨兵向上短路。
int heightOrBad(Node* p)
{
    if (!p)
        return 0;
    int left = heightOrBad(p->left);
    if (left < 0)
        return -1;
    int right = heightOrBad(p->right);
    if (right < 0)
        return -1;
    if (std::abs(left - right) > 1)
        return -1; // -1 表示子树已失衡
    return 1 + std::max(left, right);
}
bool balanced = heightOrBad(root) >= 0;
// 后序一次算高度，O(n) 时间、O(h) 栈；空树平衡
```

019 反转单链表的核心指针操作是什么？

答案　单链表只有指向后继的链接。反转链表就是让每个节点改为指向原来的前一个节点，使原尾变成新头。例如 1→2→3→空，处理完成后应是 3→2→1→空。通常要求原地完成，因此不需要新建节点，也不需要把节点值拷贝到数组再倒序。

维护“已经反转好的前半段”和“还未处理的后半段”。用前驱指针指向前半段的头，用当前指针指向后半段的头。每轮先保存当前节点的原后继，再把当前的 next 指向前驱，最后让前驱和当前各前进一步。先保存后继是关键，因为改完 next 后，沿原链接再也找不到剩下的节点。

当前指针变空时，所有节点都已处理，前驱指针就是新头。空链表会直接返回空，单节点链表也能自然处理。每节点只改一次链接，时间 O(n)，额外空间 O(1)。原链表的方向会被永久改变；如果调用方还需保留旧结构，应复制节点，空间随之增加到 O(n)。

代码示例

```
// 反转前保存 next，避免改写链接后丢掉未处理后缀。
Node* previous = nullptr;
Node* current = head;
while (current)
{
    Node* next = current->next; // 改链前先保存后继
    current->next = previous;
    previous = current;
    current = next;
}
head = previous; // 原尾成为新头
// 原地 O(n) 时间、O(1) 额外空间；空链表也返回 nullptr
```

020 和为 K 的子数组个数如何计算？

答案　子数组必须是原数组中连续的一段。这题可以用前缀和解决：前缀和就是“从数组开头到某位置的累计和”。如果当前累计和为 p，那么以前出现过的每一个累计和 p-K，都能与当前位置组成一段和为 K 的子数组，因为两段累计和相减，前面相同的部分会抵消。

用哈希表记每个旧前缀和出现了几次。扫描到一个元素时，先更新累计和，再把表中 p-K 的次数加入答案，最后才登记当前 p。顺序不能反，否则 K 为零时可能把长度为零的区间也算进去。初始登记“累计和零出现一次”，代表数组开始之前的位置，从第一个元素开始的合法区间才能被统计。

例如 [1,1,1]、K 为二，共有两段 [1,1]。方法适用于包含负数或零的数组，不能随意换成只适合某些非负输入的滑动窗口。哈希表操作平均为常数时，总时间 O(n)，额外空间 O(n)。累计和与区间个数都可能远大于单个元素，应使用足够宽的整数。

代码示例

```
// prefix-k 的旧频次就是以当前位置结尾的合法子数组数。
std::unordered_map<long long, long long> seen{{0, 1}};
long long prefix = 0, count = 0;
for (int x : a)
{
    prefix += x;
    count += seen[prefix - k]; // 先查旧前缀，涵盖负数与从下标 0 开始的段
    ++seen[prefix];
}
return count; // 平均 O(n) 时间、O(n) 空间；和与答案使用 64 位
```

021 图中的环检测（Cycle Detection）

答案　环是指沿着图中的边走，最后又回到已经经过的顶点。因为图可能有向，也可能无向，检测规则不能完全混用。无向图沿一条边走到孩子，再沿同一条边回父节点，并不算环；有向图则要区分“正在当前递归路径上”和“以前已经完成”的点。

无向图深度优先搜索时，保存每个节点的父节点。如果遇到已访问邻居，而那条边又不是刚才走来的父边，就发现了环。若图允许两点间有多条不同边，必须用边编号区分父边，否则平行边会被误当成同一条边。自环连接节点自身，也是一条环。

有向图可用三种状态：未访问、正在搜索、已经完成。沿边遇到“正在搜索”的点，说明存在回到当前路径的边，因此构成有向环；遇到已完成节点则不一定有环。图不连通时，必须从每个未访问节点启动搜索。用邻接表时总时间 O(V+E)，状态和递归栈需 O(V) 空间。

代码示例

```
// 有向图灰色回边与无向图非父返边的判环条件不同。
// 有向图看灰色回边；无向图忽略父边，平行边用 edgeId 区分。
bool directedCycle(int u)
{
    state[u] = 1;
    for (int v : graph[u])
        if (state[v] == 1 || (state[v] == 0 && directedCycle(v)))
            return true;
    state[u] = 2;
    return false;
}
bool undirectedCycle(int u, int parentEdge)
{
    seen[u] = true;
    for (auto [v, edgeId] : graph[u])
        if (edgeId != parentEdge && (seen[v] || undirectedCycle(v, edgeId)))
            return true;
    return false;
}
bool hasCycle(bool directed)
{
    if (directed)
    {
        for (int u = 0; u < n; ++u)
            if (state[u] == 0 && directedCycle(u))
                return true;
        return false;
    }
    std::fill(seen.begin(), seen.end(), false);
    for (int u = 0; u < n; ++u)
        if (!seen[u] && undirectedCycle(u, -1))
            return true;
    return false;
} // 遍历全部连通分量，邻接表 O(V+E)
```

022 图的广度优先搜索（BFS）

答案　广度优先搜索从一个起点出发，先访问离起点一条边的点，再访问距离两条边的点，逐层向外扩展。用先进先出的队列实现：新发现的邻居放到队尾，最早发现的节点先处理。第一次发现某个顶点时，已经找到从起点到它的最少边数路径，前提是每条边代价相同。

访问标记要在入队时设置，而不是等出队才设置。否则两个邻居可能同时把同一个节点加入队列，造成重复工作。若要还原路径，可以在首次发现节点时记住它的前驱；从终点沿前驱反向追溯，再把顺序反过来即可。不可达顶点不会入队，保持未访问状态。
邻接表下每个顶点和边至多扫描有限次，时间 O(V+E)，队列、访问状态和前驱表需要 O(V) 空间。BFS 找到的是无权图的最短边数路径；如果每条边有不同费用，就不能因为第一次遇到节点而停止，应改用 Dijkstra 等算法。
代码示例
```
// 首次入队即固定无权最短距离，并可记录 parent 重建路径。
std::queue<int> q;
distance[start] = 0;
q.push(start);
while (!q.empty())
{
    int u = q.front();
    q.pop();
    order.push_back(u);
    for (int v : graph[u])
        if (distance[v] == -1)
        {
            distance[v] = distance[u] + 1;
            parent[v] = u;
            q.push(v); // 入队即标记
        }
}
// 无权图首次发现即为最少边数；邻接表 O(V+E)，带权图不适用
```
023 图的强连通分量（Strongly Connected Components）
答案　强连通分量是有向图中的一组顶点：组内任意两个顶点都能沿有向路径互相到达。Kosaraju 算法通过两次 DFS 找出它们。第一次记录每个顶点“从它出发能到达的区域都处理完”时的完成顺序；第二次把所有边反向，再按第一次完成顺序从后往前搜索。
第一次完成得晚的顶点，在原图的连通结构中处于适合启动搜索的位置。把边反转后，从这个顶点出发，搜索会收集到恰好一个尚未收集的强连通分量。完成一次搜索后，把该组顶点标为已处理，再继续下一个起点。反转边会改变可达方向，但不会拆开或合并“彼此都可达”的顶点组。
需要保存原图、反图、完成顺序和访问标记，因此邻接表下时间 O(V+E)，空间 O(V+E)。递归 DFS 的最大深度可能等于 V，大图中可能耗尽调用栈；可以改用显式栈，或者使用同样 O(V+E) 的 Tarjan 算法。
代码示例
```
// 第一遍完成次序决定第二遍在反图搜索的 SCC 边界。
void finishDfs(int u)
{
    seen[u] = true;
    for (int v : graph[u])
        if (!seen[v])
            finishDfs(v);
    order.push_back(u);
}
void collect(int u)
{
    seen[u] = true;
    component.push_back(u);
    for (int v : reverseGraph[u])
        if (!seen[v])
            collect(v);
}
for (int u = 0; u < n; ++u)
    if (!seen[u])
        finishDfs(u);
std::fill(seen.begin(), seen.end(), false);
for (auto it = order.rbegin(); it != order.rend(); ++it)
    if (!seen[*it])
    {
        component.clear();
        collect(*it);
        groups.push_back(component);
```

```
    }
// 反图保存边方向反转；总 O(V+E) 时间、O(V+E) 空间
```

024 图的深度优先搜索（DFS）

答案　深度优先搜索会沿一条路不断往深处走，直到没有未访问邻居，再退回上一个分叉点继续。递归调用自然保存了这些“还没处理完”的位置；用显式栈也能实现同一过程。它常用于找连通分量、判断环、拓扑排序，以及树和图上的遍历。

必须在继续递归或压入栈时就把顶点标记为已访问。图里可能有环，如果等离开顶点时才标记，搜索可能再次走回它并无限循环。若想识别有向图回边，需要区分当前搜索路径上的点和已经全部处理完的点；仅一个 visited 标志不能区分两者。

邻接表下每个顶点和边扫描一次，时间 O(V+E)，访问标记及栈需要 O(V) 空间。不连通图要通过外层循环从每个未访问顶点启动。无向图判断环时还要忽略沿原边返回父节点的情况；若只需要避免重复遍历，普通 visited 就够。

代码示例

```
// visited 必须在递归前设置，并从未访问节点覆盖所有分量。
void dfs(int u)
{
    seen[u] = true;
    order.push_back(u); // 首次发现即标记，防止环中重复递归
    for (int v : graph[u])
        if (!seen[v])
            dfs(v);
}
for (int u = 0; u < n; ++u)
    if (!seen[u])
        dfs(u); // 覆盖非连通分量
// 邻接表 O(V+E) 时间、O(V) 访问标记；递归深度最坏 O(V)
```

025 图的连通性检测

答案　无向图连通，意思是任意两个顶点之间都能沿着边互相到达。只需从任意一个顶点做一次 BFS 或 DFS，数一数能访问多少个顶点；若数量等于图里的顶点总数，图就连通。若某个点没被访问，至少存在一个与其断开的部分。例如 A-B-C 和 D-E 是两块互不连接的部分，从 A 出发只能访问 A、B、C，所以图不连通。空图没有可选起点，是否视为连通取决于题目约定；只有一个顶点的图通常是连通的。孤立点也算一个连通分量，因此不能只数边而忽略顶点。

邻接表实现时，每个顶点和边只访问有限次，时间 O(V+E)，额外空间 O(V)。对于有向图，“从一个点能到达所有点”只说明单向可达，不代表强连通；反方向也必须能到达，通常用强连通分量算法或对原图和反图分别检查。

代码示例

```
// 连通性由从起点可达的节点数与顶点总数比较。
if (n == 0)
    return true; // 空图是否连通按题意定义
std::queue<int> q;
seen[0] = true;
q.push(0);
while (!q.empty())
{
    int u = q.front();
    q.pop();
    for (int v : graph[u])
        if (!seen[v])
        {
            seen[v] = true;
            q.push(v);
        }
}
bool connected = std::all_of(seen.begin(),
                             seen.end(),
                             [](bool x)
                             {
                                 return x;
                             });
// 无向图邻接表 O(V+E)；有向图强连通不能只做一次可达遍历
```

026 在旋转排序数组中查找元素

答案　旋转排序数组原本递增，只是选一个位置把前后两段交换了，例如 [4,5,6,7,0,1,2]。每次取中点时，至少有一侧仍是完整的递增区间。先识别哪侧有序，再看目标是否落在这侧范围内；若是就保留这半边，否则舍弃它，继续搜另一边。

例如左端为四、中点为七，左半区间 [4,5,6,7] 有序。如果目标是六，就缩到左边；若目标是零，则它不可能在左边，缩到右边。维护闭区间时，确认无目标后应把左界设为 mid+1 或右界设为 mid-1，确保区间严格缩小。

无重复元素时，每轮区间减半，时间 O(log n)，只用 O(1) 额外空间。重复值会让端点、中点看起来一样，无法判断旋转点在哪边；此时可以从边界收缩一格，但最坏退化到 O(n)。空数组要直接返回未找到，并留意比较端点时不要漏掉本身就是目标的中点。

代码示例

```
// 每轮先识别一侧有序区间，只保留目标可能所在半区。
int left = 0, right = static_cast<int>(a.size()) - 1;
while (left <= right)
{
    int mid = left + (right - left) / 2;
    if (a[mid] == target)
        return mid;
    if (a[left] <= a[mid])
    {
        if (a[left] <= target && target < a[mid])
            right = mid - 1;
        else
            left = mid + 1;
    }
    else
    {
        if (a[mid] < target && target <= a[right])
            left = mid + 1;
        else
            right = mid - 1;
    }
}
return -1; // 无重复时 O(log n)；有重复且端点相等需收缩，最坏 O(n)
```

027 基数排序（Radix Sort）

答案　基数排序不是直接比较两个完整数字，而是把数字按位拆开来排。例如先按个位排，再按十位排，最后按百位排。每一轮必须使用稳定排序，也就是键相同的元素仍保留上一轮的先后次序；否则排高位时会破坏低位已经建立的顺序。

从最低位开始的 LSD 做法适合位数固定的整数。以十进制为例，逐位取余得到 0 到 9，再按桶收集。处理完某位后，桶内外的收集顺序要保持稳定。字符串也能按字符位排序，但要明确字符顺序、长度补齐和编码规则，不能默认视觉字符就是一个字节。

每位计数排序时，n 是元素数、d 是位数、k 是每位可能值的个数，时间 O(d(n+k))，额外空间 O(n+k)。当数值范围有限且位数不大时有优势；键范围很大或位数很多时未必比比较排序划算。负数需另外安排符号顺序或做有序映射，不能直接把它们当作非负数处理。

代码示例

```
// 低位到高位逐位稳定排序，稳定性传递低位顺序。
// 假设键均为非负整数；每一位必须稳定排序
std::vector<int> out(a.size());
for (long long exp = 1; maxValue / exp > 0; exp *= 10)
{
    std::array<int, 10> count{};
    for (int x : a)
        ++count[(x / exp) % 10];
    for (int d = 1; d < 10; ++d)
        count[d] += count[d - 1];
    for (auto it = a.rbegin(); it != a.rend(); ++it)
        out[--count[(*it / exp) % 10]] = *it;
    std::swap(a, out); // exp 从低位到高位，稳定性保留低位次序
}
// d 位、基数 10：O(d(n+10)) 时间、O(n+10) 空间；负数需另处理
```

028 堆排序（Heap Sort）

答案　堆排序用二叉堆安排元素。升序排序时，先建大顶堆，让堆顶始终是未排序区间中的最大值。把堆顶与区间末尾交换，最大值就落在最终位置；再缩小堆的范围并把新的根节点向下调整，重复直到所有位置排好。堆通常用数组保存，不需要真的建立树节点。

为什么从数组末尾开始向前调整父节点即可建堆？叶节点本身已经是大小为一的堆，从最后一个非叶节点向根走，调整每个节点时，它的孩子子树已经满足堆性质。这样自底向上建堆只需 O(n)，不需要对每个元素分别执行一次 O(log n) 插入。

交换根与末尾后，左右子堆仍然有效，只需下滤新根，与较大的孩子交换，直到它不再小于孩子。整体时间 O(n log n)，额外空间 O(1)，但相同键可能改变相对顺序，所以不是稳定排序。若实现索引计算，要先处理长度零或一，并避免对空区间访问堆顶。

代码示例

```
// 大顶堆根是当前最大值，交换后缩小有效堆区间。
void siftDown(std::vector<int>& a, int root, int size)
{
    while (2 * root + 1 < size)
    {
        int child = 2 * root + 1;
        if (child + 1 < size && a[child] < a[child + 1])
            ++child;
        if (a[root] >= a[child])
            break;
        std::swap(a[root], a[child]);
        root = child;
    }
}
for (int i = n / 2 - 1; i >= 0; --i)
    siftDown(a, i, n); // O(n) 建大顶堆
for (int end = n - 1; end > 0; --end)
{
    std::swap(a[0], a[end]);
    siftDown(a, 0, end);
}
// 原地升序、O(n log n)，不稳定；每轮只需修复根
```

029 如何使用 C++ 实现二分查找树？

答案　二叉搜索树把较小键放在左边、较大键放在右边，查找时每比较一次，就能决定接下来只走哪棵子树。插入沿同样路径找到空位置后建立节点。删除则要分三种情况：叶节点直接移除；只有一个孩子时让父节点接上这个孩子；有两个孩子时常用右子树中的最小节点（中序后继）替换，再删除它原来的位置。

手写实现首先要确定重复键策略，例如拒绝插入重复值，或者把重复计数存进节点。还要明确谁拥有节点内存，删除时避免泄漏、悬空指针和重复释放。裸递归实现简洁，但树若持续单向插入会退化成链表，操作时间和递归深度都由树高 h 决定，最坏 O(n)。

若只是要有序键、快速查找和范围遍历，通常直接使用标准库 std::set 或 std::map；它们维护平衡结构，常见操作有 O(log n) 的复杂度。只有题目要求特定节点结构、增强信息或掌握算法原理时，才值得从头实现。即便自写，也要考虑旋转平衡，不能把普通 BST 的平均表现当作最坏保证。

代码示例

```
// BST 每一步按键比较收缩到一条搜索路径；高度未必平衡。
Node* find(Node* p, int key)
{
    while (p && p->key != key)
        p = key < p->key ? p->left : p->right;
    return p;
}
Node* insert(Node* p, int key)
{
    if (!p)
        return new Node{key};
    if (key < p->key)
        p->left = insert(p->left, key);
    else if (key > p->key)
        p->right = insert(p->right, key);
    return p; // 重复键按策略忽略
```

```
}
Node* erase(Node* p, int key)
{
    if (!p)
        return nullptr;
    if (key < p->key)
        p->left = erase(p->left, key);
    else if (key > p->key)
        p->right = erase(p->right, key);
    else if (!p->left || !p->right)
    {
        Node* child = p->left ? p->left : p->right;
        delete p;
        return child;
    }
    else
    {
        Node* s = p->right;
        while (s->left)
            s = s->left;
        p->key = s->key;
        p->right = erase(p->right, s->key);
    }
    return p;
}
// 查找/插入/删除 O(h)，双子树删除用右子树最小后继；需明确内存释放策略
```

030 如何使用 STL 实现有序容器?

答案　有序容器会按键的比较规则保存元素，所以查找和遍历都能利用顺序。std::set 保存不重复的键，std::map 保存“键到值”的映射；若允许多个相同键，用 std::multiset 或 std::multimap。它们不是按插入顺序排列，而是按比较器定义的先后顺序排列。

关联容器通常使用平衡搜索树实现，查找、插入、删除为 O(log n)，可以按序遍历，也能用 lower_bound 找到“不小于指定键”的第一个位置，再做范围查询。自定义比较器必须满足严格弱序，直观来说它需要始终一致地判断顺序；不一致的比较规则会破坏容器算法的前提。

有序结构通常不会把所有元素连续放在一块内存中，所以遍历缓存局部性不如 vector，按下标随机访问也不是它的用途。只关心快速精确查找、且不需要排序时可考虑 unordered_set 或 unordered_map；需要插入顺序时则选专门的顺序容器。要先按查询和遍历方式选择结构。

代码示例

```
// 按比较器顺序组织键，std::set/std::map 与 multi 版本决定重复键行为。
std::set<int> ordered{4, 1, 3};
ordered.insert(2);
auto firstAtLeast = ordered.lower_bound(3); // 有序范围查询 O(log n)
std::map<std::string, int> score;
score["Ada"] = 10;
for (const auto& [name, value] : score)
    consume(name, value); // 按键排序遍历
std::multiset<int> allowDuplicates;
allowDuplicates.insert(5);
allowDuplicates.insert(5);
// std::set/std::map 键唯一，multi 版本允许重复；查找插入删除 O(log n)，非连续存储
```

031 如何使用 std::map 和 std::unordered_map?

答案　std::map 和 std::unordered_map 都保存键和值，但组织方式不同。map 按比较器排序，因此遍历有序，也方便查找一段键值范围；unordered_map 根据哈希值把键分桶，查找通常很快，但遍历顺序不固定，也不能直接高效做有序范围查询。它要求键能够比较相等并提供合适的哈希函数。

哈希表查找平均 O(1)，不是无条件常数时间。如果大量键落在同一个桶里，查找会变慢，最坏 O(n)；扩容重建桶时也会花费额外时间。对 map 来说，典型查找、插入、删除是 O(log n)，操作成本更稳定，但通常有较大的节点与指针开销。

unordered_map 扩容会使迭代器失效，因此不能长期保存一个迭代器再假设它仍可用；元素本身的引用和指针规则不同，需按具体操作及标准约定检查。若预计元素很多，可考虑预留桶数减少反复扩容。按是否需要排序、可预测复杂度、内存局部性来选择，不能只比较平均查找数字。

代码示例

```
// 是否需要范围有序遍历决定 std::map 与 std::unordered_map 的选择。
std::map<std::string, int> ordered;
ordered["key"] = 1;
if (auto it = ordered.find("key"); it != ordered.end())
    use(it->second); // 有序，O(log n)
std::unordered_map<std::string, int> hashed;
++hashed["key"];
auto found = hashed.find("missing");
if (found == hashed.end())
    handleMissing();
// 哈希表平均 O(1)、最坏 O(n)，遍历顺序不稳定；std::map 支持范围查询
// std::unordered_map 扩容会使迭代器失效，选择取决于顺序和复杂度要求
```

032 如何使用 std::vector 和 std::array?

答案　vector 和 array 都能把同类型元素连续存放在一块空间中，因此支持常数时间的下标访问，也适合传给需要连续数组的接口。区别主要在长度：vector 的长度能在运行时变化，array 的元素个数是编译期确定的。可以把 vector 理解为可增长的数组，把 array 理解为大小固定的数组容器。

vector 增加元素时，如果现有容量不够，就可能申请更大的存储、把旧元素搬过去再释放旧区间。扩容会影响原来保存的迭代器、指针和引用，所以使用前要查清失效规则；已知大致大小时可先 reserve，减少多次搬移。reserve 只预留容量，不会增加元素数量。

array 通常没有动态分配，整个容器对象本身包含固定数目的元素，适合长度固定的小数组；它也可以有零长度。两者的对象大小、存储位置和生命周期取决于容器对象如何创建及类型布局。若长度不需要变化，array 更直接；否则通常选择 vector。不要误以为固定数组一定在栈上，标准并不这么保证。

代码示例

```
// std::vector 长度可变、std::array 长度编译期固定，两者元素连续。
std::vector<int> dynamic;
dynamic.reserve(100);
dynamic.push_back(7); // 运行期可变且连续
std::array<int, 3> fixed{{1, 2, 3}}; // 长度编译期固定，也连续
int first = dynamic[0], second = fixed.at(1); // 随机访问 O(1)
// std::vector 扩容可能搬迁元素；reserve 可减少扩容次数
// std::array 不动态分配长度；二者都可提供连续元素数据
```

13.4 链表、字符串与基础数据结构

033 如何删除链表倒数第 N 个节点?

答案　删除链表倒数第 N 个节点，直观做法是先知道链表总长度，再找到要删节点的前一个节点。不过这需要先后遍历两次。快慢指针可以合并这两个过程：让快指针比慢指针提前 N 个节点，之后两者一起走。快指针到尾部时，慢指针正好停在被删节点前面。

用虚拟头节点放在原头节点之前，可以统一处理所有位置。要删除第 N 个节点时，先让快指针从虚拟节点向前走 N 步，再同步移动两个指针，直到快指针后面没有节点。此时慢指针的 next 就是目标节点。把它跳过后，返回虚拟头节点的 next；这样删原头也不需要特殊分支。

例如 1→2→3→4、N 为二，目标是三，慢指针停在二，令二.next 指向四即可。N 必须大于零且不超过链表长度，否则快指针无法合法前进，应按接口约定报错或返回失败。一次遍历时间 O(n)，只增加几个指针，空间 O(1)。

代码示例

```
// 快慢指针保持 n 个节点距离，慢指针停在待删节点前。
if (n <= 0)
    return head; // 非法 n 按接口约定处理
Node dummy{0, head};
Node* fast = &dummy;
Node* slow = &dummy;
for (int i = 0; i < n && fast; ++i)
    fast = fast->next;
if (!fast)
    return head; // n 大于链表长度时不删除
while (fast->next)
{
    fast = fast->next;
    slow = slow->next;
}
slow->next = slow->next->next;
```

```
return dummy.next;
// 单趟 O(n) 时间、O(1) 空间；dummy 统一覆盖删除头节点
```

034 如何判断一个字符串是回文?

答案　回文字符串从左往右和从右往左读起来相同，例如“level”。判断时可用两个指针，一个从开头向右、一个从末尾向左，比较它们所指的字符；只要有一对不同，就不是回文。两个指针相遇或交叉时，说明所有需要比较的字符都通过了检查。

如果题目说忽略空格、标点或大小写，比较前就要按这个规则跳过字符或统一大小写；不同题目的规则不一样，不能自行假设。例如“Never odd or even”忽略空格和大小写后是回文。按字节逐个比较虽然能处理英文 ASCII，但不一定能正确处理 UTF-8 中文或组合字符。

双指针每个字符最多检查一次，时间 O(n)，额外空间 O(1)。若需求按 Unicode 字符、字素簇或规范化后的文本比较，需要先明确何为“一个字符”，并按该规则解码与标准化；视觉上相同的文本可能编码不同。大小写折叠也可能涉及多字符映射，因此要依题目范围选择合适方法。

代码示例

```
// 左右端字符按题目规范化后逐对比较，跳过规则需一致。
bool palindrome(const std::string& s)
{
    int left = 0, right = static_cast<int>(s.size()) - 1;
    while (left < right)
    {
        while (left < right && ignored(s[left]))
            ++left; // 若题目忽略标点则跳过
        while (left < right && ignored(s[right]))
            --right;
        if (lower(s[left++]) != lower(s[right--]))
            return false;
    }
    return true;
}
// O(n) 时间、O(1) 额外空间；UTF-8 应先按 Unicode 字符解码
```

035 如何判断两个链表是否相交?

答案　两条链表相交的意思是它们在某个位置开始共享同一批节点。判断标准是节点的身份或地址相同，不是节点里的数值相同；两个节点恰好都存着五，仍可能属于完全独立的链表。找到第一个共享节点后，后面的链接相同，因此会一直共享同一个尾部。

一种不需要额外集合的办法是让指针 A 从链表 A 走，走到空后改从链表 B 的头开始；指针 B 反过来走。两人最终走过的总长度都是“链表 A 长度加链表 B 长度”，于是长度差会抵消。如果存在交点，会在那里相遇；如果不相交，则同时到达空指针。

循环条件应允许空指针比较，否则两条链表长度不同或为空时会处理错误。若两条链表尾节点都存在但地址不同，可以提前判定不相交。时间 O(m+n)，空间 O(1)。指针在链表间切换时要注意确实比较节点身份；按值比较会把内容相同但没有共享链接的节点误判。

代码示例

```
// 换到另一链表后抵消两表长度差，按节点地址判断交点。
Node* a = headA;
Node* b = headB;
while (a != b)
{
    a = a ? a->next : headB;
    b = b ? b->next : headA;
}
return a; // 同时到 nullptr 表示不相交
// 相交按节点地址相同判定，不是值相等；换头抵消长度差
// O(m+n) 时间、O(1) 空间，空链表自然返回 nullptr
```

036 如何判断二叉树是否对称?

答案　对称二叉树像沿根节点向下的一面镜子：左侧的结构和值，要与右侧镜像对应。只比较左右孩子的值不够，还要检查下一层的方向。例如根的左孩子有左孩子，而右孩子对应位置应当有右孩子，不能只因为两边数值相同就认定对称。

递归时一次比较一对镜像位置的节点。两个都空表示匹配；只有一个为空或值不同就失败。接下来比较“左节点的左孩子”和“右节点的右孩子”，再比较相反的一对。这样逐层展开，检查了结构和值两个条件。空树通常被约定为对称。

每个节点最多参与一次配对检查，时间 O(n)，递归栈 O(h)。树很深时可将节点对放进队列或栈，迭代处理可以避免递归深度受限。重复值本身没有问题，关键是镜像位置对应正确。对于一般多叉树，要另行定义孩子顺序与镜像规则，不能直接套用二叉树比较。

代码示例

```
// 每次比较一对镜像位置，结构和节点值都必须对应。
bool mirror(Node* a, Node* b)
{
    if (!a || !b)
        return a == b; // 必须同时为空
    return a->value == b->value && mirror(a->left, b->right) && mirror(a->right, b->left);
}
bool symmetric = !root || mirror(root->left, root->right);
// 同时比较值与镜像结构；每节点至多访问一次，O(n) 时间、O(h) 栈
```

037 如何判断链表是否有环并找到入环点？

答案　链表有环时，某个节点会再次沿 next 到达自己之前访问过的节点，指针永远不会变空。快慢指针可用常数空间检测：慢指针每轮走一步，快指针走两步。如果链表无环，快指针最终走到空；如果存在环，快指针进入环后会每轮追近慢指针一个节点，最后必然相遇。

首次相遇只说明有环，还不是环入口。把一个指针放回头节点，另一个留在相遇点，然后两者都每次走一步，它们会在入环节点再次相遇。这个结论依赖进入环前距离和环内距离之间的关系；如果只需要回答“有没有环”，不必做第二阶段。

检查快指针时必须先保证它及 next 存在，才能安全地向前走两步。时间 O(n)，额外空间 O(1)。如果链表极长但某种并行或跨线程修改会改变 next，算法假设的稳定结构就不成立，需要先定义同步和生命周期。普通单线程、链接稳定的面试题里，双指针无需记录已访问地址。

代码示例

```
// 首次相遇用于确认有环，第二阶段交点才是入环位置。
Node* slow = head;
Node* fast = head;
do
{
    if (!fast || !fast->next)
        return nullptr;
    slow = slow->next;
    fast = fast->next->next;
} while (slow != fast);
Node* fromHead = head;
while (fromHead != slow)
{
    fromHead = fromHead->next;
    slow = slow->next;
}
return fromHead; // 两指针再次相遇即环入口
// 有环时快慢必相遇；时间 O(n)、额外空间 O(1)
```

038 如何原地合并两个有序数组？

答案　题目给第一个有序数组预留了 m+n 个位置，前 m 个是真正的数据，后 n 个位置为空余空间；第二个数组有 n 个数据。要把两者合成一个有序数组并写回第一个数组。若从开头开始写，可能覆盖第一个数组里还没比较的元素；从末尾开始填就不会碰到它们。

维护两边尚未处理部分的最后一个元素，比较大小，把较大者写到第一个数组的最后空位，然后对应的指针和写入位置都向前移动。这样每一步确定最终数组中最大的剩余元素。若第一个数组的数据先耗尽，只需把第二个数组剩余内容复制到前面；若第二个耗尽，第一个数组剩余元素已经在正确位置。

例如 [1,3,5,空,空,空] 和 [2,4,6]，依次从后向前放六、五、四、三、二、一。时间 O(m+n)，额外空间 O(1)，前提是第一个数组的容量确实至少为 m+n，且输入区间各自已排序。要验证空数组和 m 或 n 为零的情况。

代码示例

```
// 从尾部写入避免覆盖 a 尚未处理的前缀。
int i = m - 1, j = n - 1, write = m + n - 1;
while (j >= 0)
{ // 从后向前写，不覆盖 a 中尚未读取的有效元素
    if (i >= 0 && a[i] > b[j])
        a[write--] = a[i--];
    else
```

```
        a[write--] = b[j--];
}
// b 用尽时 a 剩余前缀原位有序；O(m+n) 时间、O(1) 额外空间
```

039 如何合并两个有序链表?

答案　合并两个有序链表，就是沿两个链表从头比较，每次把当前较小的节点接到结果末尾。已取走哪个节点，就只推进哪个链表的指针。因为两边各自有序，被取节点之前的值不会比后续节点更大，因此这样构造的结果仍保持有序。虚拟头节点可以消除“结果目前还是空链表”这个特殊情况。用尾指针指向结果最后一个节点；每轮选择一边的头节点接上并推进尾指针。任何一方耗尽后，另一方剩余部分本身已经有序，可以整段直接接到结果末尾，不必继续逐个比较。

复用节点时总时间 O(m+n)，额外空间 O(1)，但会改变并重新连接输入链表；要保留原输入就得创建新节点，空间 O(m+n)。相等时固定优先接左侧，可以保持相同值的原相对顺序。要确保最后接入剩余链表后没有留下旧连接造成环或重复链接。

代码示例

```
// 每次摘取较小头节点，尾指针始终指向结果末端。
Node dummy{};
Node* tail = &dummy;
while (a && b)
{
    Node*& pick = a->value <= b->value ? a : b;
    tail->next = pick;
    pick = pick->next;
    tail = tail->next; // 复用节点，左侧优先保持稳定
}
tail->next = a ? a : b;
return dummy.next;
// 一方耗尽后直接接剩余链；O(m+n) 时间、O(1) 额外空间
```

040 如何合并重叠区间?

答案　合并重叠区间的目标，是把彼此相交的多个区间合成尽量少的较大区间。先按左端点从小到大排序，再从前往后扫描，记录一个当前正在合并的区间。若新区间与它重叠，就把右端点延伸到两者右端点的较大值；若完全分开，就输出当前区间并从新区间重新开始。

例如 [1,3]、[2,5]、[8,10] 中，前两段重叠，合成 [1,5]；第三段分开，单独保留。按左端点排序后，后续区间不会跑到当前区间左侧，因此只需比较右边界，不必回头修改已经输出的区间。空列表要返回空结果。

区间端点是闭区间还是开区间会影响相接的端点是否算重叠。例如 [1,3] 和 [3,5] 在闭区间下相交，在开区间或半开区间下规则可能不同，比较符号要按题意选择。排序占 O(n log n)，扫描 O(n)；结果数组通常需要 O(n) 空间。

代码示例

```
// 已排序区间只需比较当前新区间与结果末尾的右边界。
std::sort(intervals.begin(),
          intervals.end(),
          [](const auto& a, const auto& b)
          {
              return a.start < b.start;
          });
std::vector<Interval> merged;
for (const auto& x : intervals)
{
    if (merged.empty() || merged.back().end < x.start)
        merged.push_back(x);
    else
        merged.back().end = std::max(merged.back().end, x.end); // 合并闭区间重叠/相接
}
// 空输入返回空结果；排序 O(n log n)，扫描 O(n)
// 开区间是否合并端点相接要按题意调整比较符
```

041 如何在 C++ 中实现并查集?

答案　并查集用来维护“哪些元素目前属于同一组”。最初每个元素自己是一组。查询一个元素所属的组时，顺着父指针找到该组代表；合并两个组时，把其中一个代表连到另一个代表下面。它常用来判断无向图加边后两端是否已经连通、统计连通分量，或处理 Kruskal 最小生成树。

两项优化让树保持很矮。路径压缩会在查询时把访问路径上的节点直接挂到代表下面；按秩或集合大小合并，则总是把较小的树挂到较大的树上。两者一起使用时，多次操作的摊还复杂度接近常数，严格写法是 O(α(n))，α 是增长极慢的反阿克曼函数。

合并前先比较两个代表，若已相同就无需改动；否则合并并更新集合大小或秩。并查集适合“只合并”的关系，删除一条边后如何拆开集合它无法高效处理。并查集数组占 O(n) 空间，且接口通常要检查元素编号是否合法。

代码示例

```
// 代表元标识集合；路径压缩与按大小合并控制树高。
int find(int x)
{
    return parent[x] == x ? x : parent[x] = find(parent[x]);
} // 路径压缩
bool unite(int a, int b)
{
    a = find(a);
    b = find(b);
    if (a == b)
        return false;
    if (size[a] < size[b])
        std::swap(a, b); // 小树挂大树
    parent[b] = a;
    size[a] += size[b];
    return true;
}
// 初始化 parent[i]=i、size[i]=1；合并+压缩摊还 O(α(n))
// 适合只合并不拆分的动态连通关系
```

042 如何实现 KMP 算法?

答案　KMP 用来在一个较长文本里查找模式串，而且避免失配时从头重新比较。核心是模式串自身包含的信息：当已经匹配了一段后，计算这段末尾的前缀里，有多少字符同时也是它的后缀。失配时可以保留这段相同内容，只把模式串移动到下一个可能的位置。

前缀表记录每个位置之前的子串，最长的相等真前缀和真后缀有多长。“真”表示不能把整个子串本身算进去。搜索时若文本字符和模式字符不相同，就根据前缀表回退模式下标，而文本下标保持不动；已匹配的后缀与前缀相同，所以无需重看文本中的每个字符。

先构造前缀表需要 O(m) 时间，扫描文本需要 O(n) 时间，总计 O(n+m)，空间 O(m)。不同教材可能把表定义为“包含当前位置”还是“只看当前位置之前”，实现下标会不同，但构造和回退必须采用同一约定。还要规定空模式串返回什么；比较字符串时应按实际编码或字符语义处理。

代码示例

```
// pi 记录可复用的前缀长度，失配时文本下标不回退。
// pi[i] 是 pattern[0..i] 的最长相等真前后缀长度
if (pattern.empty())
    return {0}; // 若接口要求所有边界位置，在此改为返回 0..text.size()
std::vector<int> pi(pattern.size());
for (int i = 1, j = 0; i < pattern.size(); ++i)
{
    while (j && pattern[i] != pattern[j])
        j = pi[j - 1];
    if (pattern[i] == pattern[j])
        ++j;
    pi[i] = j;
}
for (int i = 0, j = 0; i < text.size(); ++i)
{
    while (j && text[i] != pattern[j])
        j = pi[j - 1];
    if (text[i] == pattern[j])
        ++j;
    if (j == pattern.size())
    {
        matches.push_back(i - j + 1);
        j = pi[j - 1];
    }
}
// 空模式需单独定义；预处理 O(m)、扫描 O(n)，文本指针不回退
```

043 如何实现一个链表?

答案　手写链表通常要先定义节点里存什么，以及节点之间如何连接。单链表每个节点有值和 next；双链表还保存 prev，方便从已知节点向前移动。插入和删除的核心不是搬动整段元素，而是改少数几个链接，但头、尾、空表、单节点等边界会影响需要改哪些指针。

如果已知待删除节点的前驱，单链表删除只需让前驱指向目标节点的后继，然后销毁目标；若只拿到目标节点而拿不到前驱，通常还要从头查找。链表查找第 k 个元素需要逐节点前进，时间 O(n)；已知位置的链接更新可 O(1)。节点的创建和销毁必须由一个明确的所有者负责。

在现代 C++ 中，实际业务优先用 std::forward_list 或 std::list 等标准容器，避免重复造轮子并使用成熟的生命周期管理。教学或特殊布局需要自定义时，可让每个 next 用 unique_ptr 表达单向所有权，析构时递归释放；特别长的链表可能需要迭代拆链，避免深层析构。裸指针仅做链接时要清楚区分“拥有”与“借用”。

代码示例

```
// head 独占整条单链表，移动 std::unique_ptr 转移所有权。
struct Node
{
    int value;
    std::unique_ptr<Node> next;
};
std::unique_ptr<Node> head;
void pushFront(int x)
{
    auto node = std::make_unique<Node>();
    node->value = x;
    node->next = std::move(head);
    head = std::move(node);
}
void popFront()
{
    if (head)
        head = std::move(head->next);
}
// 单链表头插/头删 O(1)，std::unique_ptr 自动释放整条所有权链
// 查找仍为 O(n)；双链表还需维护 prev 与头尾边界
```

044 如何实现动态规划?

答案　动态规划不是背一堆公式，而是把大问题拆成规模更小、会重复出现的子问题，并把每个子问题的答案保存起来。首先要说清楚状态代表什么，例如 dp[i] 表示处理完前 i 个元素时的最佳收益；这个含义决定后面转移是不是正确。只写“用 dp 数组”而不解释状态，别人很难判断数组各位置代表什么。

接着想最后一步可能怎么发生，再从更小的状态推出当前答案。例如买或不买、走上格或左格，枚举所有合法选择后取最大值、最小值或相加。边界是最小规模时的答案，遍历顺序要保证依赖的状态已经计算好。只有确认旧状态之后不再需要，才能把二维表压成一维。

写完后用很小的输入手算几步，检查状态初始化、下标含义、转移覆盖范围和空输入。背包问题正序还是倒序取决于一件物品能否重复使用，不存在通用固定方向。动态规划也不总比搜索快；状态数量太大时，应考虑记忆化搜索、剪枝、稀疏状态或其他算法。

代码示例

```
// dp[x] 代表金额 x 的最优值，状态按依赖从小到大计算。
// dp[x]: 组成金额 x 所需的最少硬币数；硬币面值必须为正
const int inf = amount + 1;
std::vector<int> dp(amount + 1, inf);
dp[0] = 0;
for (int x = 1; x <= amount; ++x)
    for (int coin : coins)
    {
        if (coin <= x && dp[x - coin] != inf)
            dp[x] = std::min(dp[x], dp[x - coin] + 1);
    }
return dp[amount] == inf ? -1 : dp[amount];
// 先定义状态/边界，再按依赖从小到大计算；O(amount * coins) 时间
```

045 如何实现哈希表?

答案　哈希表的思路是先用哈希函数把键变成一个数字，再把它映射到有限数量的桶里，从而快速定位可能的位置。不同键算出同一个桶叫碰撞，并不一定代表哈希函数出错；表必须有办法继续区分它们。常见方法是每个桶挂一串元素，或在数组中按探测规则找下一个空位。

查找、插入和删除必须遵循相同的哈希与比较规则。对于自定义键，只要两个键按相等规则相等，它们的哈希值也必须相同。负载因子是元素数除以桶数；过高会增加冲突，通常需要扩容并把旧元素重新放入新桶。开放寻址的删除不能简单清空槽位，否则后续探测链可能被截断，常要用“删除标记”。

平均性能常称 O(1)，但取决于哈希分布和负载因子；大量碰撞时可能退化到 O(n)。不可信输入还可能故意制造碰撞，影响服务性能。实际使用优先标准库容器和合适的哈希函数，手写实现要分别测试碰撞、扩容、删除后查找及容量耗尽等边界。

代码示例

```
// 桶下标由哈希函数决定，桶内相等比较解决碰撞。
// 链地址法：哈希相同的键进入同桶，再用相等比较区分
if (capacity == 0)
    capacity = 1;
std::vector<std::vector<Entry>> buckets(capacity);
auto bucket = [&](const Key& key)
{
    return std::hash<Key>{}(key) % buckets.size();
};
Entry* find(const Key& key)
{
    for (Entry& e : buckets[bucket(key)])
        if (equal(e.key, key))
            return &e;
    return nullptr;
}
void insert(Key key, Value value)
{
    auto& chain = buckets[bucket(key)];
    for (Entry& e : chain)
        if (equal(e.key, key))
        {
            e.value = value;
            return;
        }
    chain.push_back({std::move(key), std::move(value)});
    ++size;
    if (static_cast<double>(size) / buckets.size() > maxLoad)
        rehash();
}
// rehash 需重算所有桶下标；开放寻址删除要用墓碑避免探测链断开
// 平均查找 O(1)，恶意碰撞或不佳分布下可退化 O(n)
```

13.5 常见结构与搜索算法

046 如何实现图的 DFS 和 BFS?

答案　图的深度优先搜索（DFS）和广度优先搜索（BFS）是两种走访顶点的方法。DFS 先沿一条路尽量走深，再回来走其他分支，常用递归或栈；BFS 先处理当前层的所有邻居，再处理下一层，使用队列。两者都需要 visited 标记，避免环路导致重复访问。

搜索时从起点只能访问它可到达的部分，所以遍历整张图时还要遍历所有顶点，对每个未访问点启动一次搜索。邻接表保存每个顶点的邻居；每个点和每条边只扫描有限次，时间 O(V+E)，visited 与栈或队列空间 O(V)。递归 DFS 的深度最坏可达顶点数 V。

无权图中，BFS 第一次到达顶点时路径的边数最少，因此适合最少步数问题；DFS 没有这个保证。若边有非负权重，应使用 Dijkstra 等加权最短路算法。无向图检测环时要忽略返回父节点的边，有向图则要区分访问中与已完成状态。

代码示例

```
// DFS 深入回溯，BFS 按层扩展并在无权图求最短边数。
void dfs(int u)
{
    seen[u] = true;
    for (int v : graph[u])
```

```
        if (!seen[v])
            dfs(v);
}
for (int u = 0; u < n; ++u)
    if (!seen[u])
        dfs(u); // 覆盖所有分量
std::fill(seen.begin(), seen.end(), false); // 重新初始化后从 start 做 BFS
std::queue<int> q;
seen[start] = true;
distance[start] = 0;
q.push(start);
while (!q.empty())
{
    int u = q.front();
    q.pop();
    for (int v : graph[u])
        if (!seen[v])
        {
            seen[v] = true;
            distance[v] = distance[u] + 1;
            q.push(v);
        }
}
// 首次发现时标记；DFS 用于深入/回溯，BFS 求无权最少边数
// 邻接表两者 O(V+E)；带权非负图用 Dijkstra
```

047 如何实现堆排序？

答案　堆排序把数组看成完全二叉树，用堆顶快速拿到最大值。升序排列时建一个大顶堆，反复把最大的堆顶放到当前区间末尾，再调整剩下部分。数组从零开始，节点 i 的两个孩子分别在 2i+1 和 2i+2；只有下标不超过堆范围时它们才存在。

建堆时从最后一个非叶节点向根逐个下滤。越靠近叶子，最多需要调整的层数越少，所以总建堆成本是 O(n)，不是对每个节点都花 O(log n)。每轮交换堆顶与末尾后，左右子树仍是堆，只需重新调整根。

排序部分有 n 轮，每次调整 O(log n)，所以总时间 O(n log n)；建堆线性不会改变这个总量级。只使用数组和几个临时变量，额外空间 O(1)。相等元素可能改变原先顺序，因此排序不稳定。空数组和单元素数组应自然返回，不要计算无效孩子下标。

代码示例

```
void sift_down(std::vector<int>& a, std::size_t i, std::size_t n)
{
    for (; 2 * i + 1 < n;)
    {
        auto child = 2 * i + 1;
        if (child + 1 < n && a[child] < a[child + 1])
            ++child;
        if (a[i] >= a[child])
            break;
        std::swap(a[i], a[child]);
        i = child;
    }
} // 建最大堆后反复把堆顶放到末尾
```

048 如何实现快速排序和归并排序？

答案　快速排序和归并排序都通过分治把数组拆小，但取舍不同。快速排序选一个基准值，把比它小和大的元素分到两边，再分别排序；归并排序先拆成左右两半各自排序，再用两个指针合并有序结果。两种算法的“分治”步骤不一样，所以稳定性和内存需求也不同。

快速排序平均 O(n log n)，但每次基准都选到极端值时会退化到 $O(n^2)$。随机基准能降低常见输入下退化的机会，三路划分把小于、等于、大于基准的部分分开，在重复值很多时更有效。它通常原地分区，但递归栈仍要占空间，并且一般不稳定。

归并排序每一层总共合并 n 个元素，递归层数约 log n，因此时间稳定为 O(n log n)。数组实现通常需要 O(n) 临时空间；相等元素先取左边可以保持稳定。若内存有限、更看重常见速度可选快速排序；若要求最坏时间稳定或稳定排序，归并更合适。

代码示例

```cpp
void quick_sort(std::vector<int>& a, int lo, int hi)
{ // [lo, hi)区间
    if (hi - lo <= 1)
        return;
    int pivot = a[lo + (hi - lo) / 2], lt = lo, i = lo, gt = hi - 1;
    while (i <= gt)
    {
        if (a[i] < pivot)
            std::swap(a[lt++], a[i++]);
        else if (a[i] > pivot)
            std::swap(a[i], a[gt--]);
        else
            ++i;
    }
    quick_sort(a, lo, lt);
    quick_sort(a, gt + 1, hi); // 三路分区跳过等值段
}
void merge(std::vector<int>& a, std::vector<int>& tmp, int lo, int mid, int hi)
{
    int i = lo, j = mid, k = lo;
    while (i < mid && j < hi)
        tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++]; // 相等先取左侧，保持稳定
    while (i < mid)
        tmp[k++] = a[i++];
    while (j < hi)
        tmp[k++] = a[j++];
    std::copy(tmp.begin() + lo, tmp.begin() + hi, a.begin() + lo);
}
void merge_sort(std::vector<int>& a, std::vector<int>& tmp, int lo, int hi)
{
    if (hi - lo <= 1)
        return;
    int mid = lo + (hi - lo) / 2;
    merge_sort(a, tmp, lo, mid);
    merge_sort(a, tmp, mid, hi);
    merge(a, tmp, lo, mid, hi); // 分半后稳定合并，O(n log n)且需要临时空间
}
```

049 如何实现栈和队列?

答案　栈和队列都保存一组元素，但取出顺序相反。栈是后进先出，像一摞盘子，只能从最上面放取；队列是先进先出，像排队，先到的先离开。两者都可以基于 vector、deque 或固定数组实现，也可以直接使用标准库的 std::stack 和 std::queue。

日常的栈操作是 push、pop、top，队列操作是 push、pop、front。容器适配器只暴露符合栈或队列的接口，隐藏底层容器细节。队列若用固定数组，环形缓冲区可以复用数组开头空出来的位置；需要用元素计数或保留空槽规则区分“头尾相等时为空还是已满”。

动态容器端部操作通常均摊 O(1)，但 pop 不一定返回被删除的值，要先读取 top 或 front。空容器上读取或弹出属于错误用法，应在接口层规定检查或异常行为。普通标准容器不支持多个线程同时无同步读写；并发场景要加锁或选择专门的并发结构。

代码示例

```cpp
// 栈操作尾端、队列操作两端，读取或弹出前要检查空状态。
std::stack<int> lifo;
lifo.push(x);
int top = lifo.top();
lifo.pop();
std::queue<int> fifo;
fifo.push(x);
int front = fifo.front();
fifo.pop();
// 栈后进先出，队列先进先出；先检查 empty 再读 top/front 或 pop
// std::stack/std::queue 是容器适配器，端操作通常 O(1)
// 固定数组队列可用 head/tail 环绕，并用计数区分满和空
```

050 如何找到二叉树的最低公共祖先（LCA）?

答案　最低公共祖先（LCA）是同时位于两个目标节点祖先链上的最深节点；一个节点也可以是自己的祖先。对普通二叉树，不知道节点值与左右位置的额外规律，常用后序递归：找到目标就把它向上传，左右两侧都报告找到了目标时，当前节点就是两者分叉的位置。

空子树返回“没找到”，命中 p 或 q 时返回当前节点。如果左右递归各返回一个非空节点，说明两个目标分别在两侧；如果只有一侧非空，则继续把这一结果向上传。若一个目标就是另一个目标的祖先，这种规则也会返回上面的目标。假设两个目标都在树中，遍历一次时间 O(n)，递归空间 O(h)。若目标可能缺失，仅凭返回一个节点会误把只找到的目标当作答案；需要额外确认两个目标都存在。二叉搜索树可以按键值与根比较，缩小到一侧，时间通常为树高 O(h)。

代码示例

```
// 左右递归都返回目标时当前节点是分叉最低点。
Node* lca(Node* p, Node* a, Node* b)
{
    if (!p || p == a || p == b)
        return p;
    Node* left = lca(p->left, a, b);
    Node* right = lca(p->right, a, b);
    if (left && right)
        return p; // 两侧各找到一个，当前节点是最低公共祖先
    return left ? left : right;
}
// 普通二叉树 O(n) 时间、O(h) 栈；若节点可能不存在需另计数确认两者均找到
```

051 如何求二叉树右视图?

答案　右视图是站在二叉树右边时，从每一层能看到的节点值。每层其他节点如果被更靠右的节点挡住，就不会出现在答案里。层序遍历天然按层处理，因此只要记录每层最后一个出队节点即可。空树的右视图为空。

另一种做法是 DFS 时优先搜索右孩子，再搜索左孩子。第一次到达某个深度的节点就是该层最右边能看到的节点，把它记下来。访问顺序不可反过来，否则每层第一次到达的会是最左节点，得到的是左视图。

无论哪种方式，每个节点只访问一次，时间 O(n)。BFS 队列最多保存一层节点，空间 O(w)；DFS 调用栈约 O(h)，其中 w 是最大层宽、h 是树高。实现按深度记录时要保证结果按层扩展；有缺失孩子也不能简单把“最右位置为空”当作该层没有可见节点。

代码示例

```
// 队列中固定的一层宽度内，最后一个弹出的节点是右视值。
if (!root)
    return view;
std::queue<Node*> q;
q.push(root);
while (!q.empty())
{
    int width = q.size();
    for (int i = 0; i < width; ++i)
    {
        Node* p = q.front();
        q.pop();
        if (i == width - 1)
            view.push_back(p->value); // 每层最后弹出者
        if (p->left)
            q.push(p->left);
        if (p->right)
            q.push(p->right);
    }
}
// O(n) 时间、队列 O(w) 空间；DFS 也可按右、左顺序首次记层
```

052 如何生成全排列或子集?

答案　全排列要把所有元素按不同顺序排列；子集则要枚举每个元素“选”或“不选”的组合。回溯把当前做出的选择放在 path 里，向下一层尝试后续选择，完成一个答案就复制保存，再撤销刚才的选择，尝试下一个分支。撤销这一步叫回溯，是避免不同分支互相污染。

排列中的元素不能重复使用，因此每层要知道哪些位置已经用过，或通过交换来限定未选区间。子集为避免同一组以不同顺序重复生成，通常从当前位置之后继续选。输入有相同值时，先排序，并跳过同一层中重复的候选项；只跳过同层重复，不能把排列里不同位置全部忽略。

n 个不同元素有 n! 个排列、2^n 个子集，光输出这些结果就可能很大，算法时间和内存至少与输出规模相关。可以边生成边处理以节约结果存储。若要求字典序，候选项按确定顺序遍历，并确保结果复制的是当前 path，而不是之后还会变化的同一份临时容器。

代码示例

```
// path 是当前选择；回溯返回时必须撤销本层选择。
void subsets(int start)
{
    out.push_back(path); // 每个递归状态本身就是一个子集
    for (int i = start; i < a.size(); ++i)
    {
        if (i > start && a[i] == a[i - 1])
            continue; // 排序后跳过同层重复值
        path.push_back(a[i]);
        subsets(i + 1);
        path.pop_back();
    }
}
void permutations()
{
    if (path.size() == a.size())
    {
        out.push_back(path);
        return;
    }
    for (int i = 0; i < a.size(); ++i)
        if (!used[i])
        {
            if (i && a[i] == a[i - 1] && !used[i - 1])
                continue;
            used[i] = true;
            path.push_back(a[i]);
            permutations();
            path.pop_back();
            used[i] = false;
        }
}
// 输入先排序以去重；输出规模可达 2^n 或 n!，结果可能占主要内存
```

053 如何用两个栈实现队列?

答案　两个栈可以模拟一个先进先出的队列。输入栈负责接收入队元素，输出栈负责提供队首。元素压进输入栈时顺序是后来者在顶部；一次性全部倒入输出栈后，最早进入的元素会翻到顶部，因此最先弹出。

只有输出栈为空时才需要倒栈。若输出栈还有旧元素，此时倒入新元素会让它们压在新元素之下，破坏先来先服务的次序。等输出栈清空后，再把输入栈全部倒过去即可。查看队首只看输出栈顶，不需要删除；删除时先检查输出栈是否为空。

每个元素最多从输入栈转到输出栈一次，之后再弹出一次。因此一次操作最坏可能搬动 n 个元素，但连续很多次操作的平均成本（摊还复杂度）是 O(1)。空队列的 front 或 pop 应规定为报错还是返回失败。两个栈共保存 n 个元素，额外空间 O(n)。

代码示例

```
// 只有输出栈空才倒序搬运，之后其栈顶即队首。
std::stack<int> input, output;
void push(int x)
{
    input.push(x);
} // 新元素暂放输入栈
int front()
{
    if (output.empty())
        while (!input.empty())
        {
            output.push(input.top());
            input.pop();
        }
```

```
    if (output.empty())
        throw EmptyQueue{};
    return output.top();
}
void pop()
{
    (void)front();
    output.pop();
}
// 仅当输出栈空才搬运，保持 FIFO；每元素搬运一次，均摊 O(1)
```

054 如何用单调栈求柱状图最大矩形?

答案　直方图里每根柱子高度已知，要找一个矩形，宽度跨过若干根柱子，高度不能超过这些柱子中的最矮值。对每根柱子，都可以问：“它作为矩形最矮的柱子时，左右最多能延伸到哪里?”知道左右第一个比它矮的位置后，宽度和面积就能直接算出。

单调栈保存柱子下标，并维持高度递增。遇到更矮的柱子时，栈顶柱子的右边界就是当前位置左侧；将它弹出后，新栈顶给出左边第一个更矮的柱子。左右更矮柱之间的宽度为 right-left-1。末尾补一个高度零的哨兵，可以把尚未弹出的柱子全部结算。

每根柱子只入栈和出栈各一次，时间 O(n)，栈空间 O(n)。高度相同的柱子可以选择严格递增或非递减规则，但相等时是否弹出要与宽度计算配套；不能一边采用一种边界解释，一边套另一种写法。空直方图答案为零，面积乘法要考虑整数溢出。

代码示例

```
// 单调栈保存尚未找到右侧更矮柱子的下标。
for (int i = 0; i <= heights.size(); ++i)
{ // i==n 视作高度 0 的哨兵
    int current = i == heights.size() ? 0 : heights[i];
    while (!st.empty() && heights[st.back()] > current)
    {
        int h = heights[st.back()];
        st.pop_back();
        int left = st.empty() ? -1 : st.back();
        best = std::max(best, h * (i - left - 1)); // 左右第一个更矮柱间的宽度
    }
    if (i < heights.size())
        st.push_back(i);
}
// 每个下标入栈出栈一次，O(n) 时间、O(n) 空间；空输入答案 0
```

055 如何用哈希表解决数组查重和两数之和?

答案　查重只需要知道一个值以前是否出现过：扫描数组时看集合里有没有它，有就发现重复，没有就加入集合。两数之和则对当前值 x 查找 target-x 是否以前出现过；若存在，两者下标就是答案。用哈希集合或哈希表可以把整次数组扫描压到平均 O(n)，代价是 O(n) 的额外存储。

为了避免把同一个元素用两次，要先查补数，再把当前值和下标写入表。例如只有一个六、目标为十二时，不能因为当前六刚插入后查到自己而回答成功。数组里值可重复时，如果前一个相同值已在表中，后一个就可以与它配对。

哈希操作平均 O(1)，冲突严重时会退化；补数 target-x 也可能在窄整数类型里溢出，应先用宽类型计算。只找一组、要找全部不同数值对、还是要找所有下标对，答案去重规则不同。查重也要说明是返回任意重复值、全部重复值，还是只判断是否存在。

代码示例

```
// 两数之和先查补数再登记当前值，避免元素自配对。
std::unordered_set<int> seen;
for (int x : a)
    if (!seen.insert(x).second)
        duplicates.push_back(x); // 查重
std::unordered_map<int, int> index;
for (int i = 0; i < a.size(); ++i)
{
    auto it = index.find(target - a[i]);
    if (it != index.end())
        return std::pair<int, int>{it->second, i}; // 先查再存，避免自配对
    index[a[i]] = i;
}
// 平均 O(n) 时间、O(n) 空间；补数运算也要防溢出
```

056 如何用队列实现栈?

答案　用一个队列实现栈，可以在每次入栈后把旧元素重新排到新元素后面。新元素入队时先到了队尾；再把它前面的 n-1 个元素依次出队并放回队尾，它就会变成队首，于是出栈可以直接删除队首，弹出的总是最后入栈的元素。
例如队列表示为 [旧顶 A, B, C]，压入 D 后成为 [A,B,C,D]，把前三个轮转后为 [D,A,B,C]。top 只看队首，不必移动；pop 删除队首。这样 push 要轮转整个旧队列，时间 O(n)，而 top 和 pop 为 O(1)。
如果改用两个队列，可以选择让 push 简单、pop 搬移，或相反。应根据主要操作选择，不要只说“一个栈或队列可以模拟”，要说明复杂度承担在哪一类操作。空栈上的 top/pop 应进行检查。队列本身仍需 O(n) 空间，单线程实现不自动提供并发安全。

代码示例

```
// 旋转旧队列元素到尾部，让新入队值位于队首模拟栈顶。
std::queue<int> q;
void push(int x)
{
    q.push(x);
    for (int i = 1; i < q.size(); ++i)
    {
        q.push(q.front());
        q.pop();
    }
}
int top()
{
    return q.front();
}
void pop()
{
    q.pop();
}
// 每次 push 后轮转旧元素，使新元素在队首：push O(n)，top/pop O(1)
// 空栈操作前需检查；双队列可改为 push O(1)、pop O(n)
```

057 如何解决背包问题?

答案　“背包问题”是一组有相似结构的优化题，不是一种单一转移公式。每个物品能选几次？重量或体积是不是整数？要最大价值、恰好装满，还是统计方案数？这些条件决定状态的含义、初值和循环方向。先把题意写清楚，再选择 0/1、完全、多重或其他背包模型。
若每件只能选一次，容量倒序更新，避免本轮重复用当前物品；若无限次选，容量正序更新，让当前物品更新出的状态能再次参与计算。每件最多 k 次时，可以把数量拆成若干组 0/1 物品，或使用针对该模型的单调队列优化。容量 W 是数值规模，基本算法 O(nW)，W 太大时要换方法。
“不超过容量”和“恰好装满”的初始化不同：前者通常所有容量的最大价值初值为零；后者应让不可达状态保持负无穷，仅容量零为零。若要输出选了哪些物品，还要记录决策或保留足够的 DP 历史，单个滚动数组往往只剩最优值。

代码示例

```
// 0/1 倒序阻止重复取用；完全背包正序允许本轮重用。
// 0/1 背包：容量倒序，同件物品本轮不能再次使用
for (const auto& item : items)
    for (int c = W; c >= item.weight; --c)
        dp[c] = std::max(dp[c], dp[c - item.weight] + item.value);
// 完全背包：容量正序，让当前物品的本轮新状态继续参与转移
for (const auto& item : reusable)
    for (int c = item.weight; c <= W; ++c)
        dp[c] = std::max(dp[c], dp[c - item.weight] + item.value);
// 基础 O(nW) 时间、O(W) 空间；多重背包可二进制拆分数量
```

058 如何计算二叉树路径总和?

答案　“二叉树路径和”可能指不同问题，必须先说清路径从哪里到哪里。若问是否存在一条“根到叶”的路径总和等于目标，就沿树向下搜索并传递还差多少；只有真正到达叶子、剩余目标正好为零才算成功。不能在某个中间节点提前结束，也不能假设节点值都非负。
若要统计任意起点到任意终点的向下路径数，则可以用前缀和频次：当前从根累加的和为 s，祖先前缀中有多少个 s-target，就有多少条以当前节点结尾的合法路径。进入节点前记录当前和，递归子树结束后要把它移除，避免另一棵兄弟子树把这段路径错误拼接进去。

两种题型的起点、终点和返回结果都不同，不能把同一个递归答案硬套进去。根到叶搜索时间 O(n)、调用栈 O(h)；任意向下路径法维护前缀表，表中保存的主要是当前根路径，空间 O(h)。负数可使累计和反复增减，正因为如此，不能用“超过目标就剪枝”的方法。

代码示例

```
// remain 只在从根向下路径累减，且仅叶节点可结束路径。
bool hasPath(Node* p, long long remain)
{
    if (!p)
        return false;
    remain -= p->value;
    if (!p->left && !p->right)
        return remain == 0; // 只有叶子才能结束路径
    return hasPath(p->left, remain) || hasPath(p->right, remain);
}
// 根到叶路径，负数也适用；O(n) 时间、O(h) 栈
// 若统计任意向下路径，改用前缀和频次并在回溯时撤销计数
```

059 如何设计支持 O(1) push、pop、top 和 getMin 的最小栈?

答案　最小栈除了普通的压入、弹出和查看栈顶，还要能在常数时间内返回当前最小值。常用做法是维护两个栈：主栈保存全部元素，辅助栈同步保存“每个位置对应的当前最小值”。因此辅助栈顶就是主栈当前部分的最小值。

压入新值 x 时，计算 x 与之前最小值谁更小，并把新最小值压入辅助栈；弹出时两边一起弹。相同最小值也要有记录，否则栈里两个最小值相同时弹掉一个后，辅助结构可能误以为最小值消失。也可以每个主栈元素一起保存压入该位置后的最小值。

push、pop、top 和 getMin 都是 O(1)，但辅助栈让额外空间增加 O(n)。若只在出现更小值时记录，弹出时必须检查被删元素是否正好等于当前最小值，并正确处理重复最小值；这种写法更省空间但更容易错。空栈调用 top、pop、getMin 应统一规定错误处理。

代码示例

```
// 每个栈项保存截至当前位置的最小值，弹出时自动恢复旧最小值。
std::stack<std::pair<int, int>> values; // 每项保存（值，到此为止的最小值）
void push(int x)
{
    int m = values.empty() ? x : std::min(x, values.top().second);
    values.emplace(x, m);
}
void pop()
{
    if (!values.empty())
        values.pop();
}
int top() const
{
    return values.top().first;
}
int getMin() const
{
    return values.top().second;
}
// push/pop/top/getMin 均 O(1)，空间 O(n)；空栈行为需定义
```

060 将二叉树转换为链表（Flatten Binary Tree to Linked List）

答案　把二叉树展开成链表，通常要求顺序等同于先序遍历：根、左子树、右子树。展开后所有 left 指针都为空，right 指针串起节点。关键是改链接时不要丢掉原树还没处理的部分，因为节点从原结构断开后可能再也找不到。

额外空间做法可以按右、左、根的逆先序遍历，并用一个 prev 指向已经接好的后缀：把当前节点的右指针接到 prev，左指针清空，再令 prev 指向当前节点。递归栈空间是 O(h)，不能把它说成 O(1)。也可用栈保存先序节点，空间 O(h)。

若题目要求真正 O(1) 额外空间，可从根向右推进：发现左子树时，找到左子树最右节点，把原右子树接到它后面；然后把左子树整体移到右侧并清空左指针。重复操作。时间通常 O(n)，但重连细节较易错，必须先保存所需子树，并用带左、右分支的例子核对先序结果。

代码示例

```
// 把左子树移到右侧前，先把原右子树接到左子树最右端。
Node* current = root;
while (current)
{
```

```
    if (current->left)
    {
        Node* rightmost = current->left;
        while (rightmost->right)
            rightmost = rightmost->right;
        rightmost->right = current->right; // 接上原右子树
        current->right = current->left;
        current->left = nullptr;
    }
    current = current->right;
}
// 原地前序展开，额外空间 O(1)、时间最坏 O(n²) 找右边界
// 逆前序递归更简洁但需 O(h) 调用栈
```

061 岛屿数量问题（Number of Islands）

答案　岛屿数量把二维网格看成图：每个陆地格是一座顶点，只与上下左右相邻的陆地相连，一个连通块就是一座岛。扫描网格时，每遇到一块尚未处理的陆地就把岛屿数加一，然后从那里做 DFS 或 BFS，把与它相连的所有陆地一次标记完。

为什么标记完整个连通块后不会漏岛？因为从起点沿相邻关系可达的每块陆地都属于同一座岛，搜索会把它们全部访问；下一次遇到的未标记陆地一定属于另一块。可以原地把陆地改成水来省 visited 数组，但这会修改输入；不能改输入时就另设标记表。

每个格子最多处理一次，时间 O(MN)，其中 M、N 是行列数。DFS 递归栈或 BFS 队列最坏可占 O(MN) 空间。默认只按上下左右连接，对角线接触不算相连；若题目定义不同，要修改邻居方向。大网格可用显式栈或队列，避免递归深度耗尽。

代码示例

```
// 每片陆地首次发现就标记，避免相邻格重复扩展。
void flood(int r, int c)
{
    if (r < 0 || r == rows || c < 0 || c == cols || grid[r][c] != '1')
        return;
    grid[r][c] = '0'; // 入栈/递归前标记，避免重复访问
    flood(r + 1, c);
    flood(r - 1, c);
    flood(r, c + 1);
    flood(r, c - 1);
}
for (int r = 0; r < rows; ++r)
    for (int c = 0; c < cols; ++c)
        if (grid[r][c] == '1')
        {
            ++islands;
            flood(r, c);
        }
// O(rows*cols) 时间、最坏 O(rows*cols) 栈/队列；输入不可改则另设 visited
```

13.6 排序、组合搜索与贪心

062 归并排序（Merge Sort）

答案　归并排序先把数组不断分成左右两半，直到每段只有一个元素；单个元素天然有序。随后把两个有序小段合并成一个更大的有序段，逐层向上合并，最后得到完整排序结果。合并时只比较两段当前最前面的元素，不必对已排好序的小段重新搜索。

例如 [4,1,3,2] 先变成 [4,1] 和 [3,2]，各自排好得到 [1,4]、[2,3]，再依次取较小的首项，形成 [1,2,3,4]。相等时先取左段元素，可以保留相等元素的原相对顺序，因此算法稳定。每一层合并总共处理 n 个元素，共约 log n 层。

时间稳定为 O(n log n)，数组实现通常需要 O(n) 临时空间；分治递归另有 O(log n) 调用栈。链表可以通过快慢指针拆分、再合并节点，避免按下标随机访问，也不必复制元素。数据装不进内存时，可把块分别排好，再做外部多路归并。

代码示例

```
// 归并时相等元素优先取左半，保留稳定次序。
void mergeSort(int left, int right)
{
    if (right - left <= 1)
        return;
    int mid = left + (right - left) / 2;
```

```
    mergeSort(left, mid);
    mergeSort(mid, right);
    int i = left, j = mid;
    tmp.clear();
    while (i < mid || j < right)
        tmp.push_back(j == right || (i < mid && a[i] <= a[j]) ? a[i++] : a[j++]);
    std::copy(tmp.begin(), tmp.end(), a.begin() + left);
}
// 相等时先取左项保证稳定；O(n log n) 时间、数组临时空间 O(n)
```

063 快速排序（Quick Sort）

答案　快速排序先选一个基准值，再把区间划分成比基准小和大的部分，然后递归处理两边。它并不需要像归并那样另建一整份合并数组，常见分区写法可以在原数组里交换元素。平均情况下划分较均匀，时间 O(n log n)，额外空间主要来自递归栈。

如果每次都选到最小或最大值，递归一侧几乎没变，另一侧只少一个元素，比较总数就会达到 $O(n^2)$。随机选基准或三数取中能降低常见坏输入造成退化的机会，但随机化不能把最坏复杂度证明为 O(n log n)。很多重复值时，三路分区把小于、等于、大于三段分开，避免反复处理相等区域。

快速排序通常不稳定：相等元素可能在分区交换中改变相对顺序。递归栈平均 O(log n)，极端划分可到 O(n)；可总是递归较小区间来限制一部分栈深，但不改变最坏时间。若需要稳定排序或标准库的复杂度保证，优先选适合需求的库算法。

代码示例

```
// 三路分区把等于 pivot 的中间段一次排定。
void quickSort(int left, int right)
{
    if (left >= right)
        return;
    int pivot = a[left + (right - left) / 2], low = left, i = left, high = right;
    while (i <= high)
    {
        if (a[i] < pivot)
            std::swap(a[low++], a[i++]);
        else if (a[i] > pivot)
            std::swap(a[i], a[high--]);
        else
            ++i;
    }
    quickSort(left, low - 1);
    quickSort(high + 1, right); // 中间等值段已完成
}
// 平均 O(n log n)，坏划分 O(n^2)；三路划分利于大量重复值，排序不稳定
```

064 打家劫舍问题的状态转移是什么？

答案　打家劫舍的限制是不能同时选择相邻房屋，要让偷到的钱最多。处理到某间房时只有两种选择：偷它，那么前一间不能偷；不偷它，那么前面已经算出的最好结果可以保留。比较这两种收益，就能得到到当前为止的最优答案。

令 dp[i] 表示前 i 间房能取得的最大钱数，第 i 间房的下标是 i-1。状态转移是 max(dp[i-1], dp[i-2]+nums[i-1])。边界可设 dp[0]=0、dp[1]=max(0, nums[0])，这样也允许什么都不偷。因为只需要前两个结果，可以用两个变量滚动计算。线性排列时间 O(n)、空间 O(1)。如果房屋首尾相邻形成一个环，不能同时偷第一和最后一间；可分别计算“排除最后一间”和“排除第一间”的线性答案，取较大者。若有负数收入，是否允许跳过负数或必须选房，需按题意改变初始化。

代码示例

```
// prev1/prev2 分别表示前一间与前两间范围的最优收益。
long long prev2 = 0, prev1 = 0; // 前 0/1 间房的最优收益
for (int money : houses)
{
    long long now = std::max(prev1, prev2 + money);
    prev2 = prev1;
    prev1 = now; // 不抢当前 vs 抢当前
}
return prev1; // O(n) 时间、O(1) 空间；允许一间不抢故初值为 0
// 环形街区需分别计算不含首间与不含末间两段再取最大
```

065 找出一个排序数组中的重复元素

答案　排序数组里相同的值会排在一起，因此扫描相邻元素就能发现重复。比如 [1,2,2,3]，比较相邻位置时会遇到两次二。若只要判断“有没有重复”，第一次发现就可以返回；若要返回所有重复值，必须说明重复值只报一次还是每多出现一次都报告。

最简单的方法是遍历整个数组，记录上一项；当前项与上一项相同则找到一组重复。若希望每种重复值只输出一次，在发现一组后跳过这一组剩余相同元素。这个方法不依赖二分，时间 O(n)、额外空间 O(1)（不计输出）。

只有在额外条件支持时才能二分。例如已知值域、数组长度和鸽笼原理，并且能判断某个范围内有多少元素时，才可对候选值范围做计数判断；不能对任意排序数组直接二分某个重复元素。若输入未排序，为了相邻查找而排序需 O(n log n)，且会改变输入顺序。

代码示例

```
// 相同元素在已排序数组中形成连续的一组。
if (a.empty())
    return;
for (int i = 0; i < a.size();)
{
    int j = i + 1;
    while (j < a.size() && a[j] == a[i])
        ++j;
    if (j - i > 1)
        duplicates.push_back(a[i]); // 每组只报告一次
    i = j;
}
// 输入已排序，单次扫描 O(n)、O(1) 额外空间；返回一个重复值可提前停
```

066 拓扑排序（Topological Sorting）

答案　拓扑排序把有向无环图中的节点排成一个顺序，使每条依赖边的起点都出现在终点之前。例如编译任务 A 依赖 B，就必须先安排 B 再安排 A。只有有向无环图（DAG）才存在这种顺序；如果有环，每个节点都等待环上的下一项，无法全部满足依赖。

Kahn 算法先计算每个节点的入度，即有多少条边指向它。把入度为零的节点放入队列，取出并加入结果，再把它所有出边删除、邻居入度减一；新变成零的节点也入队。若处理完节点数小于 V，就说明剩下部分有环。

用邻接表时，每个节点和边处理一次，时间 O(V+E)，空间 O(V)。合法拓扑序可能不止一个；如果要求字典序最小，用保存零入度节点的最小堆代替普通队列，但复杂度会增加到 O((V+E) log V) 左右。只有在图无环时，结果才包含所有顶点。

代码示例

```
// 零入度队列逐步删除依赖；未输出顶点意味着图内有环。
std::queue<int> ready;
for (int u = 0; u < n; ++u)
    if (indegree[u] == 0)
        ready.push(u);
while (!ready.empty())
{
    int u = ready.front();
    ready.pop();
    order.push_back(u);
    for (int v : graph[u])
        if (--indegree[v] == 0)
            ready.push(v);
}
if (order.size() != n)
    return {}; // 仍有入度节点说明存在有向环
// O(V+E) 时间；有多个零入度点时拓扑序不唯一
```

067 括号生成问题如何剪枝？

答案　生成有效括号时，如果先随意生成所有左右括号、最后再检查，绝大多数字符串可能都白生成了。回溯可以边生成边保证前缀合法：左括号最多放 n 个；只有已放左括号多于右括号时，才能放右括号。这样右括号永远不会提前关闭不存在的左括号。

当左右括号都放了 n 个时，长度是 2n，得到一个完整答案。比如 n 为三，前缀“)(”会立即被规则挡住，而“(()”可以继续补括号。搜索分支在每一步最多两种，剪枝后只访问可能组成合法结果的前缀。

答案数量是 Catalan 数，增长很快，所以总时间至少与输出字符串总长度同阶，不能期望对大 n 只花多项式时间。保存所有结果会占很多内存；可以按需逐条输出。n=0 时通常有一个空字符串作为唯一合法结果，但接口也可能另有约定。

代码示例

```
// close 不得超过 open，所有递归前缀始终合法。
void generate(int open, int close)
{
    if (path.size() == 2 * n)
    {
        out.push_back(path);
        return;
    }
    if (open < n)
    {
        path.push_back('(');
        generate(open + 1, close);
        path.pop_back();
    }
    if (close < open)
    {
        path.push_back(')');
        generate(open, close + 1);
        path.pop_back();
    }
}
// close<=open 保证每个前缀合法；n=0 通常输出空串
// 时间至少与 Catalan 数量级的输出量相同
```

068 插入排序（Insertion Sort）

答案　插入排序把数组分成左侧有序区和右侧未处理区。每轮取出右侧第一个元素，在左侧从右向左找位置，把比它大的元素逐个向右挪一格，再把当前元素放进空位。这样每处理完一个元素，左侧仍然有序。

例如 [3,1,2]，先把一拿出来，将三右移，放到三前面，得到 [1,3,2]；再处理二，把三移开，插入到一和三之间，得到 [1,2,3]。实现时是“移动更大元素”而不是把相邻元素反复交换，值相等时停止移动，因此相等记录保持相对顺序。

插入排序稳定、原地，空间 O(1)。已有序时几乎不需要移动，最好时间 O(n)；平均和最坏 $O(n^2)$。适合小数组、近乎有序的数据，也常用于大型混合排序的短区间。若数组很大且无序，归并或内省排序通常更合适。

代码示例

```
// 每轮开始时左侧区间有序，将当前值插入其正确位置。
for (int i = 1; i < n; ++i)
{
    int value = a[i], j = i;
    while (j > 0 && a[j - 1] > value)
    {
        a[j] = a[j - 1];
        --j;
    } // 有序前缀右移
    a[j] = value;
}
// 相等元素不移动，故稳定；原地，近乎有序最好 O(n)，平均/最坏 O(n^2)
```

069 数组中的第 K 大元素如何求解?

答案　第 K 大元素是把元素按从大到小排后位于第 K 个的值，K 通常从一开始。若只需一个值，不必完整排序。快速选择用类似快速排序的基准分区，但分区后只继续处理包含目标排名的一侧；另一侧已经知道不可能含答案，所以可以丢掉。

从小到大排序时，第 K 大对应下标 n-K。分区后比较该位置与目标下标：相等就返回当前值，目标更小只找左边，目标更大只找右边。随机基准下平均时间 O(n)，最坏仍可能 $O(n^2)$。相同值默认按元素位置计排名；如果题目问第 K 个不同值，定义和处理都要改变。

若数据一次到来后不能全部保存，大小为 K 的最小堆适合流式维护目前最大的 K 个数，堆顶就是第 K 大，时间 O(n log K)、空间 O(K)。如果只查询一次且数组可修改，快速选择通常更省；若要多次查询或保留输入，可选择完整排序或堆，按约束权衡。

代码示例

```
// partition 将 pivot 放到最终排名，只继续目标所在一侧。
// 三路 partition 避免重复值堆在某一侧；目标下标为 n-k
if (k < 1 || k > a.size())
    throw InvalidInput{};
int target = static_cast<int>(a.size()) - k, left = 0, right = a.size() - 1;
while (left <= right)
```

```
{
    int pivot = a[left + (right - left) / 2], low = left, i = left, high = right;
    while (i <= high)
    {
        if (a[i] < pivot)
            std::swap(a[low++], a[i++]);
        else if (a[i] > pivot)
            std::swap(a[i], a[high--]);
        else
            ++i;
    }
    if (target < low)
        right = low - 1;
    else if (target > high)
        left = high + 1;
    else
        return pivot;
}
// 快速选择平均 O(n)、最坏 O(n²)；随机 pivot 可降低常见退化
// 流式数据可改用大小 K 的最小堆，时间 O(n log K)、空间 O(K)
```

070 斐波那契数列（Fibonacci Sequence）

答案　斐波那契数列从 F0=0、F1=1 开始，之后每一项等于前两项之和，例如 0、1、1、2、3、5、8。直接递归计算 F(n)=F(n-1)+F(n-2) 会把相同子问题重复算很多次，调用次数随 n 指数增长。

只求一个数时，不需要保存全部历史。保留相邻两项 a、b，每轮算 c=a+b，再把 a、b 向前滚动，直到第 n 项。这样做 n 次加法，时间 O(n)，额外空间 O(1)。当 n 为零或一时直接按定义返回对应初值。

数列增长很快，定宽整数在较大的 n 会溢出，甚至中间相加就可能超出类型范围；按题目约束选择大整数、模运算或显式报告溢出。若 n 极大且需要精确的单项值，快速倍增可以把递推步骤降到 O(log n)，但大整数运算本身仍会随位数变贵。

代码示例

```
// prev/current 始终保存相邻两项，迭代避免重复递归。
long long fib(int n)
{
    if (n < 0)
        throw InvalidInput{};
    if (n <= 1)
        return n; // F0=0、F1=1
    long long prev = 0, current = 1;
    for (int i = 2; i <= n; ++i)
    {
        long long next = prev + current;
        prev = current;
        current = next;
    }
    return current;
}
// O(n) 时间、O(1) 空间；定宽整数可能溢出，模运算按需逐步执行
```

071 旋转排序数组中的最小值

答案　旋转排序数组原本递增，只是把某个位置前后的部分调了次序，例如 [3,4,5,1,2]。无重复值时，可比较中点与右端：若中点比右端大，最小值一定在中点右侧；否则最小值在左侧并且可能就是中点。每次保留仍可能含最小值的一半。

循环维护闭区间 [left,right]。当 nums[mid] > nums[right] 时，把 left 移到 mid+1；否则把 right 移到 mid。循环到左右重合时，该位置就是最小值。若数组没有发生旋转，右半判断仍会逐步把区间缩到首元素。

有重复元素且中点与右端相等时，无法从这两个值判断最小值在哪边，可以安全地把 right 减一；但重复很多时最坏要检查 O(n) 个位置。空数组没有最小值，应返回可表达“无结果”的状态。求最小值和返回最小值下标有时不是同一个接口，需看题意。

代码示例

```
// 与右端比较后保留包含最小值的闭区间。
if (a.empty())
    throw EmptyInput{};
int left = 0, right = static_cast<int>(a.size()) - 1;
```

```
while (left < right)
{
    int mid = left + (right - left) / 2;
    if (a[mid] > a[right])
        left = mid + 1;
    else if (a[mid] < a[right])
        right = mid;
    else
        --right; // 重复值无法判定旋转点在哪边
}
return a[left]; // 无重复 O(log n)，含重复最坏 O(n)；未旋转也返回首元素
```

072 最大矩形问题（Maximal Rectangle）

答案　最大矩形题给一个只含 0 和 1 的矩阵，要找完全由 1 组成、面积最大的矩形。可以把“矩形底边在当前行”的问题转成直方图：统计每一列在当前行及其上方连续出现了多少个 1，这个连续长度就是该列高度。于是每处理一行，就能用直方图最大矩形算法求一遍。

为什么这样不会漏掉答案？任何矩形都有一个最底行；到那一行时，矩形覆盖的每列高度至少等于它的高度，因此直方图算法一定会考虑到这个底边。若当前格为 0，该列连续高度清零；为 1，则在旧高度上加一。逐行更新即可覆盖所有可能的底边。

若矩阵为 m 行 n 列，每格更新一次，每行再在线性时间扫描一次直方图，总时间 O(mn)，高度数组和单调栈用 O(n) 空间。空行或空列返回零。面积乘法可能溢出窄整数；如果想恢复具体矩形，还需在记录最大面积时同时保存其左右边界和上下边界。

代码示例

```
// height[c] 是当前行为底时该列连续 1 的高度。
std::vector<int> height(cols, 0);
for (const auto& row : matrix)
{
    for (int c = 0; c < cols; ++c)
        height[c] = row[c] == '1' ? height[c] + 1 : 0;
    best = std::max(best, largestHistogram(height)); // 当前行作矩形底边
}
return best; // 每行 O(cols)，总 O(rows*cols)，额外 O(cols)
// 空矩阵/空列返回 0；直方图子程序用单调栈求最大面积
```

073 最小生成树：Kruskal 算法

答案　最小生成树（MST）是在连通无向带权图里，选出一些边把所有顶点连接起来，并让总权重最小。它必须恰好选 V-1 条边且不能成环。Kruskal 算法从全局最便宜的边开始考虑：如果加上这条边不会把两个已连通的部分闭成环，就保留它；否则跳过。

用并查集维护目前哪些顶点已经连在一起。每条边按权重从小到大排序后，查看两端代表元：不同就选择并合并，相同表示已经有另一条路径相连，这条边会成环。负权边没有问题，因为只要按权重排序，正确性仍成立。

排序成本 O(E log E)，并查集操作近似线性。如果处理完所有边仍只选到少于 V-1 条，原图不连通，算法得到的是最小生成森林而不是一棵覆盖全图的树。多个边权相同可能存在多棵同样最优的树，Kruskal 返回其中一棵即可。

代码示例

```
// 仅合并不同并查集的边，已连通边会形成环而跳过。
std::sort(edges.begin(),
          edges.end(),
          [](const Edge& a, const Edge& b)
          {
              return a.weight < b.weight;
          });
for (const Edge& e : edges)
    if (dsu.unite(e.u, e.v))
    {
        mst.push_back(e);
        total += e.weight;
        if (mst.size() == n - 1)
            break;
    }
if (mst.size() != n - 1)
    reportForest(); // 原图不连通
// O(E log E) 排序加近线性并查集；负权无向边可正常处理
```

074 最小生成树：Prim 算法

答案　Prim 算法也求最小生成树，但思路是从一个顶点开始逐渐长大。已加入的顶点形成一块“树”；每次从所有跨越“树内到树外”的边里选最便宜的一条，把它连接的树外顶点纳入。边权可以为负，只要是无向图，选择规则仍然适用。

常见实现把候选边放进最小堆。堆里可能有已经过时的边，因为另一路径后来已先访问了它的终点；取出时若终点已经访问，直接跳过即可。无需在堆里查找和删除任意旧元素，这种懒删除写法通常更简单。

邻接表和二叉堆时，常见复杂度 O(E log V)；稠密图可用邻接矩阵、线性扫描最小候选点，以 $O(V^2)$ 完成。若图不连通，从尚未访问的每个顶点再启动 Prim，会得到最小生成森林。若题目保证连通，则从任意顶点启动一次就够。

代码示例

```
// 候选堆维护已选集合的外边界，每个顶点只纳入一次。
using Candidate = std::pair<int, int>; // (权重, 顶点)
std::priority_queue<Candidate, std::vector<Candidate>, std::greater<Candidate>> q;
for (int start = 0; start < n; ++start)
    if (!used[start])
    {
        q.push({0, start});
        while (!q.empty())
        {
            auto [w, u] = q.top();
            q.pop();
            if (used[u])
                continue;
            used[u] = true;
            total += w;
            for (auto [v, cost] : graph[u])
                if (!used[v])
                    q.push({cost, v});
        }
    }
// 外层覆盖不连通分量得最小生成森林；堆优化常见 O(E log V)
```

13.7 路径规划与动态规划

075 最小路径和（Minimum Path Sum）

答案　最小路径和通常指从左上角走到右下角，只能向右或向下，每经过一个格子就把它的数值加进总和。设 dp[i][j] 是到当前格子的最小累计代价，最后一步只能从上面或左边来，所以在两条可能路线中选代价较小的一条，再加当前格子的数值。

首行每格只能从左边到达，首列每格只能从上方到达；其他方向不存在，不能把未初始化状态误当成零。用一维数组可从上到下、每行从左向右更新：更新前 dp[j] 还是上方结果，dp[j-1] 已更新为左方结果。障碍格或不可达格要按题目规定另行表示。

矩阵 m×n 时，时间 O(mn)，滚动数组空间 O(n)。即使格子里有负数也能使用，因为只能向右和向下，不会回头绕出环，状态依赖仍是有向无环的。如果允许向左、向上等来回移动，负数可能导致无限降低路径代价，就不再适用这个递推。

代码示例

```
// dp[c] 在本行更新前是上方，更新后是左方最优代价。
if (rows == 0 || cols == 0)
    return 0;
const int inf = std::numeric_limits<int>::max() / 4;
std::vector<int> dp(cols, inf);
dp[0] = grid[0][0];
for (int r = 0; r < rows; ++r)
    for (int c = 0; c < cols; ++c)
    {
        if (r == 0 && c == 0)
            continue;
        int fromAbove = dp[c], fromLeft = c ? dp[c - 1] : inf;
        dp[c] = grid[r][c] + std::min(fromAbove, fromLeft);
    }
return dp[cols - 1]; // O(rows*cols) 时间、O(cols) 空间；避免 INF 加法溢出
```

076 最短路径问题：Bellman-Ford 算法

答案　Bellman-Ford 用来求单源最短路，可以处理负权边。开始时源点距离为零，其余点记为无穷大。每一轮把所有边 u→v 都检查一遍：如果走到 u 再经过这条边比当前 v 的距离更短，就更新 v。最多重复 V-1 轮，因为没有负环时最短简单路径最多经过 V-1 条边。

若某一整轮没有距离变化，就可以提前结束。完成 V-1 轮后再扫描所有边：若从源点可达的边仍然能继续缩短，就存在源点可达的负权环，最短距离没有有限下界。不可达顶点距离为无穷，不能拿无穷加负权来做计算，否则会制造错误的更新。

时间 O(VE)，比非负边的 Dijkstra 慢，但适合有负权边的情况，也能检测可达负环。若只关心某个目标点，需要确认是否还要报告它受可达负环影响。初始化和加法应避免“无穷哨兵”溢出，工程实现可用可选值或安全饱和加法表达不可达。

代码示例

```
// 每轮松弛允许最短路再多经过一条边，V-1 轮覆盖简单路径。
std::vector<long long> dist(n, INF);
dist[source] = 0;
for (int round = 1; round < n; ++round)
{
    bool changed = false;
    for (auto [u, v, w] : edges)
        if (dist[u] != INF && dist[u] + w < dist[v])
        {
            dist[v] = dist[u] + w;
            changed = true;
        }
    if (!changed)
        break;
}
for (auto [u, v, w] : edges)
    if (dist[u] != INF && dist[u] + w < dist[v])
        reportReachableNegativeCycle();
// O(VE) 时间，可处理负边；只从可达点松弛以免 INF 溢出
```

077 最短路径问题：Dijkstra 算法

答案　Dijkstra 求一个源点到其他点的最短距离，要求所有可达边的权重都非负。算法每次选出当前暂定距离最小的顶点，把它的距离确定下来，再用这个距离尝试改进邻居。因为后续边不会带来负的额外费用，这个最小距离不可能再被绕路变得更小。

常见实现用最小堆保存候选距离。标准库堆不方便更新旧条目时，可以发现更短距离就再压入一个新条目；弹出时若条目距离已不等于最新距离，就说明过期，跳过。这样避免实现 decrease-key，但堆里可能暂时留有重复候选。

邻接表配二叉堆通常 O((V+E) log V)，空间 O(V+E)。不可达点保持无穷。任何可达负权边都会破坏算法“最小者已确定”的理由，即使某个特定输入看起来仍给对答案也不能保证正确；这时选 Bellman-Ford 等适合算法。

代码示例

```
// 堆顶暂定距离若已过期就跳过，避免重复处理旧状态。
std::vector<long long> dist(n, INF);
dist[source] = 0;
using State = std::pair<long long, int>;
std::priority_queue<State, std::vector<State>, std::greater<State>> q;
q.push({0, source});
while (!q.empty())
{
    auto [d, u] = q.top();
    q.pop();
    if (d != dist[u])
        continue; // 跳过过期条目
    for (auto [v, w] : graph[u])
        if (d + w < dist[v])
        {
            dist[v] = d + w;
            q.push({dist[v], v});
        }
}
// 仅适用于非负边；邻接表+堆 O((V+E)log V)，不可达点保持 INF
```

078 最长公共子串（Longest Common Substring）

答案　最长公共子串是在两个字符串中找一段完全连续、字符相同的片段。它不同于“子序列”：子串不能跳过字符。可以定义 dp[i][j] 为分别以 A[i]、B[j] 结尾的最长公共子串长度。若两个字符相等，就在左上角状态上加一；若不同，连续性被打断，状态必须归零。

扫描所有字符对时，把发现的最大长度和它在某个字符串中的结束位置一起保存。比如 A 是“abcde”、B 是“xbcdy”，最长公共子串是“bcd”，长度三。若只比较全局历史最大值而不记录位置，就能得到长度但无法还原具体片段。

二维动态规划时间 O(mn)、空间 O(mn)。可以压缩成一维，但更新要从右往左，确保 dp[j-1] 仍是上一行的左上状态；否则会把同一轮更新过的值重复使用。压缩后空间 O(min(m,n))，长字符串时可明显省内存。

代码示例

```
// 子串必须连续，字符不等时以该字符结尾长度归零。
std::vector<int> dp(b.size() + 1, 0);
int best = 0;
for (char x : a)
    for (int j = static_cast<int>(b.size()); j > 0; --j)
    {
        dp[j] = x == b[j - 1] ? dp[j - 1] + 1 : 0; // 不同字符必须清零
        best = std::max(best, dp[j]);
    }
return best; // 子串连续；逆序更新保护上一行左上状态
// O(mn) 时间、O(n) 空间；可把较短串放在列维
```

079 最长公共子序列（Longest Common Subsequence, LCS）

答案　最长公共子序列（LCS）要求两个字符串按相同先后顺序取字符，但中间可以跳过；所以它不一定连续。例如“abcde”和“ace”的 LCS 是“ace”，长度三。设 dp[i][j] 表示两个字符串前 i、前 j 个字符的最长公共子序列长度。如果两个前缀的最后字符相同，就可以把它们配成一对，在左上状态基础上加一；如果不同，最优方案至少会舍弃其中一个最后字符，所以取“去掉 A 最后字符”和“去掉 B 最后字符”两种结果中较大者。空前缀与任何字符串的 LCS 都是零。

二维表时间 O(mn)。只求长度时可用滚动数组，把空间压到 O(min(m,n))；但若还要输出具体序列，需要保存表或用 Hirschberg 等方法重新构造。这里不匹配时不能像公共子串那样归零，因为之前匹配的字符仍可跨过不同字符继续组成子序列。

代码示例

```
// 子序列允许跳过字符，不匹配时取删去一侧末字符的较优状态。
std::vector<std::vector<int>> dp(a.size() + 1, std::vector<int>(b.size() + 1, 0));
for (int i = 1; i <= a.size(); ++i)
    for (int j = 1; j <= b.size(); ++j)
    {
        if (a[i - 1] == b[j - 1])
            dp[i][j] = dp[i - 1][j - 1] + 1;
        else
            dp[i][j] = std::max(dp[i - 1][j], dp[i][j - 1]);
    }
return dp.back().back(); // 子序列允许跳过字符，不要求连续
// O(mn) 时间；只求长度可滚动为 O(min(m,n)) 空间，重建需留决策
```

080 最长无重复字符子串如何求解？

答案　最长无重复字符子串要找一个连续窗口，使里面没有重复字符。用左右边界表示当前窗口，向右扫描每个字符；哈希表记住每种字符最近一次出现的位置。遇到重复字符时，把左边界移到旧位置之后，窗口就重新变成无重复。

左边界只能向右移动，不能因遇到窗口外很早以前的重复字符而倒退。因此要更新为 max(left, last[c]+1)。例如“abba”：第二个 b 把左边界移到第二个 b 后；最后一个 a 的旧位置在当前窗口之外，不能再把左边界移回字符串开头。

每个字符最多扫描一次，哈希表更新一次，平均时间 O(n)，空间取决于字符种类。返回长度时，每步更新窗口长度最大值；若要返回子串起止位置，也要在最大值更新时记录窗口边界。C++ string 的下标是字节，对 UTF-8 字符应先解码或明确题目按字节比较。

代码示例

```
// 窗口左边界只能前进，哈希表记录字符最近出现下标。
std::unordered_map<char, int> last;
int left = 0, best = 0;
for (int right = 0; right < s.size(); ++right)
{
    auto it = last.find(s[right]);
    if (it != last.end())
        left = std::max(left, it->second + 1); // 窗口左边界不后退
```

```
    last[s[right]] = right;
    best = std::max(best, right - left + 1);
}
return best; // 每字符处理一次，平均 O(n) 时间，空间与字符集有关
// UTF-8 输入若按码点判断，需先解码而不能直接以字节为字符
```

081 最长递增子序列（Longest Increasing Subsequence, LIS）

答案　最长递增子序列（LIS）找的是按原顺序取出的一串严格递增值，不要求它们挨在一起。例如 [3,1,2,5] 的答案可以是 [1,2,5]，长度三。平方时间动态规划会逐项尝试前面更小的值；还有一个更快的做法维护 tails：长度为 L 的递增序列，能够达到的最小尾值。

扫描新数 x 时，在 tails 中二分找到第一个 “不小于 x” 的位置，用 x 替换它；若没有这样的位置，就把 x 追加到末尾。这个数组的长度就是目前 LIS 长度。替换尾值表示我们找到了一个同样长但尾部更小、以后更容易继续延长的候选，并不表示 tails 本身就是原数组里真实出现的完整子序列。

二分查找使总时间 O(n log n)，tails 用 O(n) 空间。严格递增要找首个 ⩾x；非递减序列要找首个 >x，两者条件不同。若要恢复实际子序列，需保存每个元素的前驱位置和它对应的 tails 层，并回溯构造。

代码示例

```
// tails 保存各长度的最小结尾，用 std::lower_bound 替换但不直接存真实子序列。
std::vector<int> tails;
for (int x : a)
{
    auto it = std::lower_bound(tails.begin(), tails.end(), x);
    if (it == tails.end())
        tails.push_back(x);
    else
        *it = x;
}
return tails.size(); // tails[i] 是长度 i+1 的最小可行结尾
// std::lower_bound 保证严格递增；O(n log n) 时间、O(n) 空间
// 需要恢复具体序列时另存每项前驱和所在长度
```

13.8 栈、滑动窗口与数组优化

082 有效括号如何用栈判断?

答案　括号合法意味着每个右括号都能和它之前最近的、尚未匹配的左括号对应起来，且最后不能剩下未匹配的左括号。遇到左括号时压入栈；遇到右括号时先检查栈非空且栈顶类型匹配，再弹出。这样栈顶自然代表最近一个还未闭合的左括号。

例如 “([])” 合法：右方括号匹配方括号，右圆括号匹配圆括号；“([)]” 不合法，因为右圆括号不能越过尚未闭合的方括号。只检查左右括号数量相等会漏掉这种顺序错误。若字符串还包含普通字符，要按题目规定选择忽略它们还是判非法。

每个字符最多入栈、出栈各一次，时间 O(n)，栈空间最坏 O(n)。遍历结束时栈还必须为空。若只有一种括号，可用余额计数器省掉栈；混合括号必须记录类型。空字符串一般合法，但仍要符合题目对输入的定义。

代码示例

```
// 每个右括号只匹配最近未配对的左括号。
std::stack<char> st;
for (char c : s)
{
    if (isOpening(c))
        st.push(c);
    else if (isClosing(c))
    {
        if (st.empty() || !matches(st.top(), c))
            return false;
        st.pop();
    }
    else if (otherCharactersAreInvalid)
        return false;
}
return st.empty(); // 所有左括号都必须匹配
// O(n) 时间、O(n) 空间；单一种类括号可仅用计数器
```

083 查找数组中的最大值和最小值

答案　在线性时间里找数组最大值和最小值，只需维护两个候选值，并逐项更新。数组非空时，最好用第一个元素同时初始化最大和最小，再检查剩余元素。这样无需假定数值范围，因此全负数、全相同值和任意类型都能按比较规则正确处理。

例如 [-8,-3,-10]，初始化时最大和最小都是 -8；读到 -3 后更新最大，读到 -10 后更新最小。若错误地把最大初始化为零，结果会变成零，但零根本不在输入里。空数组没有最大或最小元素，应返回失败状态或由调用方先检查。

普通方法每个后续元素最多比较两次，时间 O(n)、额外空间 O(1)。若比较操作很昂贵，可两个元素先互相比，再分别与当前最小最大比较，减少最坏比较次数；但代码更复杂，且需额外处理奇数个元素。通常优先写清晰的线性扫描。

代码示例

```
// 从首元素初始化可兼容全负数组，不依赖 0 作为初值。
if (a.empty())
    return std::nullopt;
int low = a[0], high = a[0];
for (int x : a)
{
    low = std::min(low, x);
    high = std::max(high, x);
}
return std::pair<int, int>{low, high};
// 全负数也正确；O(n) 时间、O(1) 额外空间
// 需最少比较时可成对比较，再分别更新全局极值
```

084 查找数组中的重复元素

答案　查找数组中的重复值，可用哈希表记录每个值出现过没有。第一次遇到时登记；再次遇到时就是重复。如果只要返回所有不同的重复值，还要记录哪些重复值已经输出过，避免第三次、第四次出现时重复加入答案。若要求出现次数，则要保存计数而不只是布尔标志。

例如 [2,1,2,2,3,1]，不同的重复值是二和一；按额外出现次数计算则是二出现两次、一再出现一次，二者是不同结果。题目必须明确输出语义。若输入可修改且数值范围固定，可以考虑原地标记省空间，但负数、重复计数和标记恢复都会增加复杂度。

哈希表平均时间 O(n)、额外空间 O(n)，适合取值范围广且内存充足的情况。要统计频次时把映射值设为计数器；只需要是否重复时，集合足够。碰撞严重会影响哈希性能。只判断有无重复时一旦发现第二次出现即可提前结束。

代码示例

```
// 频次大于 1 的键才是重复值；若只求存在可提前返回。
std::unordered_map<int, int> frequency;
for (int x : a)
    ++frequency[x];
for (const auto& [value, count] : frequency)
    if (count > 1)
        duplicates.push_back(value);
// 计数表可同时得到重复次数；平均 O(n) 时间、O(n) 空间
// 只需判断是否有重复时，集合发现第二次即可提前返回
```

085 查找数组中缺失的最小正整数

答案　未排序数组里缺失的最小正整数，答案一定在 1 到 n+1 之间，其中 n 是数组长度。因为数组只有 n 个位置，不可能同时装下 1 到 n+1 所有不同的正整数。把每个合法数 x 放到下标 x-1 后，理想情况数组会变成 [1,2,3,...]。

原地摆放时反复检查当前位置的值。若它在 1..n 范围内且目标位置不是同一个值，就把它交换到对应下标，再继续检查换回来的值；若值越界或目标位置已有同值，就前进，避免重复值互相交换循环。摆完后扫描数组，第一个 nums[i] 不等于 i+1 的位置给出答案。

例如 [3,4,-1,1] 摆放后会把一放到下标零、三放到下标二，扫描时下标一缺少二，因此答案是二。每次有效交换都会让一个值回到目标位置，整体 O(n)，额外空间 O(1)，但会修改输入。空数组答案为一。

代码示例

```
// 把值 x 放到下标 x-1，越界和重复值停止交换。
for (int i = 0; i < n; ++i)
{
    while (a[i] >= 1 && a[i] <= n && a[a[i] - 1] != a[i])
        std::swap(a[i], a[a[i] - 1]);
}
for (int i = 0; i < n; ++i)
    if (a[i] != i + 1)
        return i + 1;
```

```
return n + 1; // 1..n 都归位后答案为 n+1
// 越界和重复值直接停止交换；O(n) 时间、O(1) 空间但会修改输入
```

086 柱状图中的最大矩形（Largest Rectangle in Histogram）

答案　直方图最大矩形的高度必须不超过所覆盖柱子的最矮高度。可以针对每根柱子，找左边和右边第一个严格更矮的柱子；该柱子作为最矮高度时，最大宽度就在两边界之间。单调栈按高度保存下标，遇到当前更矮的柱子时，就知道被弹出的柱子右边界已经确定。

弹出后新的栈顶，是左侧第一个更矮位置；若栈空，则说明左边没有更矮柱，可一直延伸到下标零。宽度为右边界减左边界再减一。最后加高度零的虚拟柱，把其余候选矩形全部结算。相同高度的处理可以用 >= 或 >，但必须与保留的下标和边界解释一致。

每个下标入栈、出栈各一次，时间 O(n)，辅助栈 O(n)。空数组直接返回零。面积是高度乘宽度，较大输入下 32 位整数可能溢出，应根据约束使用 64 位或更宽类型。零高度也可作为哨兵帮助弹出更高柱子，不必单独分支处理。

代码示例

```
std::vector<int> heights = {2, 1, 5, 6, 2, 3};
std::stack<int> increasing;
for (int right = 0; right <= static_cast<int>(heights.size()); ++right)
{
    int h = right == heights.size() ? 0 : heights[right];
    while (!increasing.empty() && heights[increasing.top()] >= h)
    {
        auto i = increasing.top();
        increasing.pop();
        update_area(heights[i], right, increasing);
    }
    if (right < heights.size())
        increasing.push(right);
} // 单调栈找到每根柱子的最大左右边界
```

087 桶排序（Bucket Sort）

答案　桶排序先按数值范围把元素分到多个桶里，在每个桶中单独排序，再按桶的顺序把结果连起来。它依赖对输入分布或值域的了解：如果数据大致均匀，元素会分散到许多小桶，桶内排序成本低；如果大量数据落在同一个桶里，就失去优势。

以已知范围 [min,max] 为例，可把区间等宽切分，再按比例计算桶编号。最大值可能恰好落在最右边界，因此索引要单独截到最后一个桶，避免越界。每个桶内可以用插入排序或比较排序，最后依次输出。浮点值需明确 NaN、无穷和边界处理。

均匀分布的理想情形下性能可接近线性，但最坏复杂度取决于桶内排序，可能退化；桶数组和临时元素需要额外空间。若没有可靠的值域或分布假设，比较排序通常更稳妥。桶的数量太少会造成拥挤，太多则增加初始化和空桶成本，应根据数据范围与内存预算选择。

代码示例

```
// 桶索引按值域线性映射，最大边界需限制在最后一桶。
// 空输入直接返回；非空时假设 high>low 且桶数 n>0
if (a.empty())
    return out;
std::vector<std::vector<double>> buckets(n);
for (double x : a)
{
    int id = static_cast<int>((x - low) / (high - low) * n);
    id = std::min(n - 1, id);
    buckets[id].push_back(x);
}
for (auto& bucket : buckets)
    std::sort(bucket.begin(), bucket.end());
for (const auto& bucket : buckets)
    out.insert(out.end(), bucket.begin(), bucket.end());
// 均匀分布时平均接近 O(n)；集中一桶时退化为桶内排序复杂度
// 额外空间 O(n)，上界映射需 clamp 防止下标等于 n
```

088 正则表达式匹配（Regular Expression Matching）

答案　这里的正则匹配题通常只支持两种特殊符号：点号“.”匹配任意一个字符，星号“*”表示它前面的字符可以出现零次或多次。它不是完整正则表达式引擎，不能默认支持括号、加号、字符类等其他语法。动态规划能明确处理每种模式符号如何消耗文本。

令 dp[i][j] 表示文本前 i 个字符与模式前 j 个字符能否匹配。普通字符或点号匹配时，问题缩小到前面各少一个字符。星号有两类选择：让前一个模式字符出现零次，则跳过“字符+星号”；若该字符能匹配当前文本字符，则可以让文本前进一格、模式仍停在星号处，继续尝试重复。
空文本也可能匹配由若干“字符*”组成的模式，因此初始化要依次检查连续配对，不能只把 dp[0][0] 设为真。表格时间和空间 O(mn)，长字符串可研究滚动空间。每一步索引都要防止星号没有前置字符；通常这种模式输入会保证格式合法。
代码示例
```
// 星号转移分为跳过前项与匹配后重复两种情况。
// dp[i][j]：文本前 i 字符与模式前 j 字符匹配；模式不以 * 开头
std::vector<std::vector<bool>> dp(s.size() + 1, std::vector<bool>(p.size() + 1));
dp[0][0] = true;
for (int j = 2; j <= p.size(); ++j)
    if (p[j - 1] == '*')
        dp[0][j] = dp[0][j - 2];
for (int i = 1; i <= s.size(); ++i)
    for (int j = 1; j <= p.size(); ++j)
    {
        if (p[j - 1] == '*')
            dp[i][j] = dp[i][j - 2] || ((p[j - 2] == '.' || p[j - 2] == s[i - 1]) && dp[i - 1][j]);
        else
            dp[i][j] = (p[j - 1] == '.' || p[j - 1] == s[i - 1]) && dp[i - 1][j - 1];
    }
// * 可取零次或延长一次；O(mn) 时间、空间，不代表完整正则语法
```
089 求解最大子数组和（Maximum Subarray Sum / Kadane's Algorithm）
答案　最大子数组和要找一段非空连续元素，使它们的和最大。Kadane 算法扫描时只维护两项：以当前位置结尾的最大子数组和，以及到目前为止看到的全局最大值。遇到当前值 x 时，要么把旧子数组延长为 cur+x，要么从 x 重新开始，取较大者。
若以当前位置结尾的旧和为负，继续带着它只会让后面的总和更小，所以重新开始不会更差。这解释了状态转移 max(x, cur+x)，不是凭经验丢掉前面的数。全局答案要从首元素初始化，而不是零，否则全负数组会错误返回零。
例如 [-2,3,-1,4]，扫描到三时重新开始，之后把负一和四接上，最大和为六。时间 O(n)、额外空间 O(1)。空数组没有非空子数组，需按接口约定返回错误或空结果。若要恢复区间起止位置，在“重新开始”时记录候选起点，在全局最大值更新时保存边界。
代码示例
```
// ending 是必须以当前位置结尾的最优和，best 是全局最优。
if (a.empty())
    return 0; // 空数组按接口约定处理
int ending = a[0], best = a[0];
for (int i = 1; i < a.size(); ++i)
{
    ending = std::max(a[i], ending + a[i]); // 从当前重开或延长前段
    best = std::max(best, ending);
}
return best; // 全负数也正确；O(n) 时间、O(1) 空间
```
090 滑动窗口最大值如何在线性时间求解？
答案　固定长度窗口的最大值，要对每个长度为 k 的连续片段输出其中最大元素。每次重新扫描窗口需要 O(k)，总成本 O(nk)。双端队列可以把候选最大值按从大到小保存，队列里放的是数组下标，而不是只有值；下标才能判断元素是否已经滑出窗口。
每处理一个新位置，先把窗口外的队首下标移除；再从队尾删掉所有值不大于新值的候选，因为新值更新、且位置更靠右，将至少同样晚才离开窗口，旧候选再也不会成为最大值。把新下标放到队尾，队首就是当前窗口最大值。
等窗口至少包含 k 个元素后才输出；此前是未填满的部分，不是完整窗口。每个下标只入队、出队一次，时间 O(n)，额外空间 O(k)。若 k<=0 或 k>n，按接口定义返回空或报错。重复最大值可保留较新的一个，因为它更晚过期。
代码示例
```
// 队列下标递增且对应值递减，过期从头、较小候选从尾删除。
if (k <= 0 || k > a.size())
    return maxima;
std::deque<int> q; // 下标对应值单调递减，队首为窗口最大值
for (int i = 0; i < a.size(); ++i)
{
    while (!q.empty() && q.front() <= i - k)
```

```
        q.pop_front();
    while (!q.empty() && a[q.back()] <= a[i])
        q.pop_back();
    q.push_back(i);
    if (i >= k - 1)
        maxima.push_back(a[q.front()]);
}
// 每下标最多进出队一次，O(n) 时间、O(k) 空间；仅完整窗口输出
```

091 爬楼梯问题（Climbing Stairs）

答案　爬楼梯假设每次能走一阶或两阶，问到第 n 阶有多少种走法。考虑最后一步：如果最后走一阶，之前在 n-1 阶；如果最后走两阶，之前在 n-2 阶。因此走到 n 的方案数是这两个较小问题的方案数之和，和斐波那契数列有相同递推结构。

先定义边界，避免递推时索引越界。若把“站在地面”当作到零阶的一种方法，则 ways[0]=1、ways[1]=1；对 n≥2 才使用前两项相加。只求最终数量时，用两个变量滚动，不必保留整张表。

时间 O(n)、空间 O(1)，但方案数按指数速度增长，定宽整数会溢出。某些题目把起点和台阶的计数方式定义不同，n=0 可能返回零或一，因此要按题意确认边界。若每次可走 1、2、3 阶，转移需把前三项相加，不能沿用原公式。

代码示例

```
// 每阶最后一步只能来自前一阶或前两阶。
if (n == 0)
    return 1; // 空台阶通常有一种空走法
long long twoBack = 1, oneBack = 1; // F(0)=F(1)=1
for (int step = 2; step <= n; ++step)
{
    long long ways = twoBack + oneBack;
    twoBack = oneBack;
    oneBack = ways;
}
return oneBack; // O(n) 时间、O(1) 空间；大 n 注意计数溢出
```

092 线性查找和二分查找有什么区别？二分查找如何实现？

答案　线性查找从头到尾逐项比较，不要求数据有序，代码简单，但最坏要看 n 个元素，时间 O(n)。二分查找利用有序性，从中点比较后一次丢掉一半不可能的范围，时间 O(log n)。但它还要求能快速访问中间位置，链表上反复走到中点并不会得到这个复杂度优势。

实现前要先选择区间约定。闭区间 [left,right] 的循环常写 left<=right，找到不匹配后移动到 mid-1 或 mid+1；半开区间 [left,right) 常写 left<right，边界更新不同。把两套写法混在一起很容易漏掉最后一个元素或陷入不退出的循环。中点用 left+(right-left)/2 防止加法溢出。

有重复元素时，“找任意一个”“第一个不小于目标的位置（下界）”和“最后一个不大于目标的位置”是不同问题。下界二分在相等时仍收缩到左半边，最终位置也是插入点。空数组返回未找到或插入位置零；测试首尾、中点、重复值和目标缺失等边界。

代码示例

```
// 线性扫描无需有序；二分半开区间返回 std::lower_bound 所在位置。
// 线性查找不要求有序，逐项比较 O(n)
for (int i = 0; i < a.size(); ++i)
    if (a[i] == target)
        return i;
// 二分要求有序且可 O(1) 访问中点；半开区间找首个 >= target
int left = 0, right = a.size();
while (left < right)
{
    int mid = left + (right - left) / 2;
    if (a[mid] < target)
        left = mid + 1;
    else
        right = mid;
}
return left; // std::lower_bound 插入位置，O(log n)；mid 公式避免加法溢出
```

093 编辑距离（Levenshtein Distance）

答案　编辑距离（Levenshtein distance）计算把一个字符串变成另一个字符串，最少需要多少次插入、删除或替换字符。设 dp[i][j] 表示把第一个字符串前 i 个字符变成第二个字符串前 j 个字符的最少操作数。处理前缀而不是下标字符本身，能让空串边界更清晰。

若最后一个字符相同，不用新增操作，答案就是两个更短前缀的距离；若不同，最后一步可以是删除、插入或替换中的一种，分别从相应更小前缀状态加一，取最小值。首行或首列表示把一个前缀全部插入或删除，所以初始化为长度。两个字符串长度为 m、n 时，时间 O(mn)，整张表空间 O(mn)。只求距离可以只保留前一行和当前行，降为 O(min(m,n)) 空间；但要恢复具体操作序列，需要保存决策或用其他重建方法。若不同操作的代价不同，分别给各转移加对应费用。

代码示例

```
// diagonal 保存上一行左上，dp[j] 与 dp[j-1] 表示删除/插入候选。
std::vector<int> dp(b.size() + 1);
std::iota(dp.begin(), dp.end(), 0); // 空前缀需插入 j 次
for (int i = 1; i <= a.size(); ++i)
{
    int diagonal = dp[0];
    dp[0] = i; // 删除 i 次
    for (int j = 1; j <= b.size(); ++j)
    {
        int old = dp[j];
        dp[j] = a[i - 1] == b[j - 1] ? diagonal : 1 + std::min({diagonal, dp[j], dp[j - 1]});
        diagonal = old; // 保存上一行左上值
    }
}
// O(mn) 时间、O(n) 空间；恢复操作序列需保存路径/决策
```

094 计数排序（Counting Sort）

答案　计数排序利用“整数值落在有限且不太大的范围”这一条件。先数每个值出现多少次，再按从小到大的顺序把这些值写回。它不比较元素，因此当值域跨度小于或接近 n 时很高效；若数值从负数到极大正数，计数数组可能大得不实际。

设最小值为 lo，最大值为 hi，值 x 的计数位置是 x-lo，计数数组长度为 hi-lo+1。统计后按位置逐个重复输出相应次数。若数组可能为空，不能直接读首项来找范围。值域跨度 k 应使用更宽整数计算，避免 hi-lo+1 本身溢出。

简单输出整数时，时间 O(n+k)、空间 O(k)。如果排序的是带附加信息的记录且要求稳定，不能只按频次重建：要计算每种值在结果中的结束位置，再从输入尾部向前放入临时数组，这会用 O(n+k) 空间。负数通过偏移自然处理。

代码示例

```
// count[x-low] 统计偏移值出现次数，按计数回填整数。
if (a.empty())
    return;
int low = *std::min_element(a.begin(), a.end()), high = *std::max_element(a.begin(), a.end());
std::vector<int> count(high - low + 1, 0);
for (int x : a)
    ++count[x - low];
int write = 0;
for (int i = 0; i < count.size(); ++i)
    while (count[i]-- > 0)
        a[write++] = i + low;
// O(n+k) 时间、O(k) 空间，k=high-low+1；值域很大/稀疏时不适合
// 稳定排序记录需改用累积位置并从输入尾部放置
```

095 选择排序（Selection Sort）

答案　选择排序每一轮从未排序部分找到最小元素，再把它放到未排序区的开头。完成一轮后，左边界之前的元素已经是整个数组里最小的若干项，且位置固定。它易于理解，也容易证明结果有序。

例如 [4,2,3]，第一轮在整个数组里找到二，与首项四交换；第二轮只在 [4,3] 找到三并放到第二位，结果为 [2,3,4]。无论数组原本是否有序，都要做约 n(n-1)/2 次比较，所以时间 $O(n^2)$。

选择排序最多交换 n-1 次，写入次数可能比插入排序少；但普通交换实现不稳定，可能让相同键的记录越过彼此。空间 O(1)。在小数据或写入特别昂贵的场景可以考虑；近乎有序数组通常更适合插入排序。若稳定性重要，可调整插入方式，但移动次数会改变。

代码示例

```
// 每轮定位未排序区最小项，只做一次最终交换。
for (int i = 0; i < n; ++i)
{
    int minIndex = i;
    for (int j = i + 1; j < n; ++j)
        if (a[j] < a[minIndex])
            minIndex = j;
    if (minIndex != i)
        std::swap(a[i], a[minIndex]); // 每轮最多一次交换
```

```
}
// 比较次数约 n(n-1)/2、空间 O(1)，交换少但时间仍 O(n²)
// 标准交换版不稳定，近乎有序也不会明显减少比较
```

096 链表表示的两个数如何相加?

答案　两个链表各自按数字位保存一个整数，通常最低位在链表头，例如 342 写成 2→4→3。可以像小学竖式加法那样同时从最低位开始相加：缺少的位当零，和的个位写入新节点，十位作为进位带到下一位。
循环结束条件不只是“两个输入都还有节点”，还必须包括进位不为零。否则 9+1 会在最后漏掉额外的 1。新建结果链表可避免修改输入，也能独立管理结果节点。每一步至少处理一个输入节点或最终进位，所以算法会终止。
输入长分别为 m、n，时间 O(max(m,n))，结果空间同阶。要验证空链表表示零还是非法输入，以及数字是否包含前导零。若最高位在头部，需先反转输入、用栈存位，或递归处理末端；不能从头直接相加。复用节点则需明确谁拥有这些节点，避免两条链表共享时破坏输入。
代码示例

```
// carry 保存十进位；循环条件必须包含尚未输出的末尾进位。
Node dummy{};
Node* tail = &dummy;
int carry = 0;
while (a || b || carry)
{
    int sum = carry + (a ? a->value : 0) + (b ? b->value : 0);
    carry = sum / 10;
    tail->next = new Node{sum % 10};
    tail = tail->next;
    if (a)
        a = a->next;
    if (b)
        b = b->next;
}
return dummy.next; // 循环包含末尾进位，低位在前
// O(max(m,n)) 时间；新建结果节点，不修改输入链表
```

097 除自身以外数组的乘积如何在 O(n) 且不除法求解?

答案　对数组每个位置 i，要求其他所有元素的乘积，不能直接用“总乘积除以当前值”，因为数组可能包含零，而且题目禁止除法。可以分两遍：第一遍写下标 i 左边元素的乘积；第二遍从右向左维护右侧乘积，把两边相乘。
例如 [1,2,3,4]，左侧乘积为 [1,1,2,6]，右侧乘积依次提供 24、12、4、1，最后结果为 [24,12,8,6]。对于零值也自然工作：只有一个零时，只有它的位置可能是非零乘积；多个零时结果会全为零，不需要单独分支。
时间 O(n)，输出数组不计入额外空间时只用一个滚动乘积变量，故额外空间 O(1)。所有中间乘积都可能溢出，应按约束选 64 位、大整数或明确溢出处理。若允许空数组，返回空结果；一项数组时该项以外没有元素，通常乘积定义为一。
代码示例

```
// 前缀和后缀乘积均跳过当前位置，不使用除法。
std::vector<long long> out(a.size(), 1);
long long prefix = 1;
for (int i = 0; i < a.size(); ++i)
{
    out[i] = prefix;
    prefix *= a[i];
}
long long suffix = 1;
for (int i = static_cast<int>(a.size()) - 1; i >= 0; --i)
{
    out[i] *= suffix;
    suffix *= a[i];
}
// 前缀与后缀都不包含当前位置；O(n) 时间、输出外 O(1) 空间
// 无除法可自然处理零值，乘积类型仍需符合数值范围
```

13.9 STL 容器使用与选择

098 std::vector 中 emplace_back 和 push_back 有什么区别?

答案　push_back 和 emplace_back 都把新元素放到 vector 尾部。push_back 接收一个已经构造好的对象，容器再把它复制或移动进去；emplace_back 接收构造该对象所需的参数，并在容器末尾直接构造。例如 Point 有两个整数参数时，可以写 emplace_back(x,y)。

“原地构造”不代表每次都快很多。如果你已经有一个 T 对象，push_back(std::move(obj)) 和 emplace_back(obj) 通常都要移动一次；若参数会触发 vector 扩容，已有元素也可能搬迁。emplace 的完美转发还可能选中不同构造函数，因此行为需按实参类型确认。
通常优先选意图最清楚的写法：已有对象用 push_back，直接用构造参数创建新对象时用 emplace_back。不要只为追求“零拷贝”把所有 push_back 机械替换成 emplace_back。扩容会影响原有迭代器、指针和引用，capacity 变化时尤其要查看失效规则。〔S2〕
代码示例

```
std::vector<Point> points;
Point p{3, 4};
points.push_back(p);
points.push_back(std::move(p));
points.emplace_back(5, 6);
// emplace直接用参数构造；已有Point时不保证比push_back少一次移动
```

099 std::vector 和 std::deque 如何选择？

答案　std::vector 和 std::deque 都支持随机访问和尾部操作，但内存布局不同。vector 把元素连续放在一段空间里，缓存局部性好，也能通过 data() 交给需要连续缓冲区的接口；代价是扩容时可能整体搬迁。deque 由多个区块组成，整体不保证连续，但在两端增加或删除元素通常更方便。
两者的下标访问都是 O(1)。vector 尾插均摊 O(1)，中间插入需要移动后续元素；deque 两端操作通常 O(1)，中间插入仍需移动元素。迭代器、引用和指针失效规则不同，不能把 vector 的规则套给 deque，也不能假设两端操作永不影响任何引用。
需要连续内存、扫描性能或和 C API 交互时选 vector；两端频繁增删且不要求整体连续时可选 deque。若大量在任意位置插入或删除，list 只有在已持有位置迭代器等场景可能合适，代价是节点开销和较差缓存局部性。先看数据访问模式，再选容器。〔S2〕
代码示例

```
std::vector<int> contiguous;
contiguous.reserve(100);
contiguous.push_back(1);
std::deque<int> both_ends;
both_ends.push_front(0);
both_ends.push_back(1);
// vector连续存储；deque适合两端插入但整体不连续
```

13.10 标准算法与容器适配器

100 std::priority_queue 怎样维护堆序？常见操作是什么复杂度？

答案　std::priority_queue 是标准库提供的优先队列容器适配器，默认让优先级最高（默认比较下是最大）的元素处于 top。它通常以 vector 为底层容器并维护堆序，但标准规定的是接口行为和复杂度，不要求必须用某一种内部实现。读取 top 通常 O(1)。push 将新项加入并调整堆，pop 移除堆顶并调整剩余元素，核心调整比较次数为 O(log n)；但底层 vector 扩容时，单次 push 还可能有搬迁成本。默认比较器 std::less 形成大顶堆，使用 std::greater 常见地形成小顶堆。
它不提供任意位置的高效删除或 decrease-key。Dijkstra 等算法可在距离变小时压入新条目，取出时跳过已过期项目；这会产生重复条目，需考虑堆的峰值空间。若必须更新指定元素，可用带索引的自定义堆。空队列不能调用 top 或 pop。〔N150〕
代码示例

```
std::priority_queue<int, std::vector<int>, std::greater<int>> pending;
pending.push(7);
pending.push(2);
int smallest = pending.top(); // 2，查看不删除
pending.pop(); // 删除堆顶，不返回原值
```

101 std::sort 的复杂度保证与常见内省排序实现是什么关系？

答案　std::sort 要求输入满足随机访问迭代器，且比较器符合严格弱序；通常重载的比较次数有 O(N log N) 复杂度保证。标准没有要求它必须使用快速排序、堆排序或某一种固定组合，所以“std::sort 一定是某某算法”不是可移植的保证。不同标准库版本也可能改变内部实现。
常见实现会用内省排序（introsort）：先用快速排序分区以获得常见情况下的效率；递归太深时切换到堆排序，防止极端输入造成平方级比较；很小的区间改用插入排序，减少常数开销。这是实现策略，帮助理解它如何兼顾性能与复杂度，不是调用方能依赖的内部细节。

std::sort 不保证相等元素原顺序，要求稳定性时用 std::stable_sort。链表迭代器不支持随机访问，应调用 list::sort。比较器若不一致，或对 NaN 等浮点值定义的比较关系不满足严格弱序，算法前提就被破坏。相同数据下观察到的顺序也不应当作稳定承诺。〔N151、N152〕

代码示例

```
std::sort(values.begin(), values.end(), strict_weak_order); // 标准保证O(N log N)比较复杂度
std::stable_sort(values.begin(), values.end(), strict_weak_order); // 需要稳定性时选择它
// 快排分区+堆排序兜底+小段插入排序是常见实现，不是标准要求
```

13.11 索引结构与算法选择

102 AVL 与红黑树如何限制高度，读写性能能否只凭树名判断?

答案　AVL 树和红黑树都是平衡二叉搜索树，靠限制高度避免普通搜索树退化成链表，所以查找、插入和删除都能保持 O(log n)。AVL 更严格：每个节点的左右子树高度差最多一。红黑树通过节点颜色和根到叶路径黑节点数等规则，间接保证高度不会太大。

严格平衡通常让 AVL 搜索路径较短，但插入和删除可能需要更多旋转或高度维护；红黑树的平衡条件更宽松，更新时维护成本常较少。这只是结构上的倾向，实际快慢还受比较函数、节点大小、内存分布、缓存和读写比例影响，不能说某一种在所有程序里更快。

如果需要按序遍历和范围查询，平衡搜索树很合适。C++ 标准保证 std::map 的语义与复杂度要求，但不强制实现必须采用红黑树；具体库可能选其一或其他符合要求的结构。面试回答应区分“数据结构理论保证”和“特定库当前实现”，并依据需求与测量选型。〔N153、N154〕

代码示例

```
if (height(left) - height(right) > 1)
    rotate_avl(node); // AVL维护更严格的高度平衡
if (red_black_rule_violated(node))
    recolor_and_rotate(node); // 红黑树更新时修复颜色约束
// 两类平衡树操作均为O(log n)，实际读写取舍应测量
```

103 跳表怎样利用随机层级加速有序访问，与平衡树有哪些取舍?

答案　跳表是一组“带快速通道的有序链表”。最底层包含所有元素，上面的层只包含部分元素，并给这些节点增加较长距离的前进指针。查找从最高层开始，尽量向右跳而不越过目标；不能再前进时下降一层继续，最终在底层找到位置。

插入时先沿各层找到新节点前面的节点，再按随机抽到的高度连接相应层；删除时要拆除这些链接。随机高度使查找和更新在常见随机化假设下期望 O(log n)，但最坏仍可能 O(n)，不同于平衡树的确定性高度约束。高层指针也需要额外空间和指针访问。

跳表容易按顺序遍历和做范围查询，结构相对直观。Redis 较大的有序集合实现可将字典与跳表结合，节点跨度还支持名次相关查询；较小集合可能使用紧凑编码，因此不能说所有 ZSet 都一定使用跳表。选型还要考虑随机数质量、内存和最坏延迟。〔N155〕

代码示例

```
Node* current = head;
for (int level = max_level; level >= 0; --level)
    while (current->next[level] && current->next[level]->key < key)
        current = current->next[level];
// 跳表随机分层，期望O(log n)；记录每层前驱后即可插入
```

104 Trie 为什么适合前缀检索，怎样区分完整单词与已有路径?

答案　Trie（字典树）把一个字符串拆成字符路径。比如“cat”和“car”共享从根开始的“ca”，只需要为共享前缀存一份节点。查找时沿字符逐个走，路径长度为 L；若子节点查找近似常数时间，查找成本与键长成正比，也容易找出所有具有同一前缀的词。

走到某个节点只代表这段前缀存在，并不表示它本身是一个完整单词。因此终点需要有 end 标记或关联值。若只插入“car”，查“ca”时路径虽存在，但没有结束标记，就应返回“不是完整词”。前缀查询可以匹配到路径后，再枚举该节点之下的词。

固定字母表用数组访问快，但每个节点可能浪费大量空槽；稀疏映射省空间但增加映射成本，压缩 Trie 还会合并单支路径。中文等文本要先确定边按 UTF-8 字节、Unicode 码点还是其他单位切分，L 的含义随之不同。输出所有匹配还要加上结果遍历和输出成本。〔N156〕

代码示例

```
struct TrieNode
{
    std::unordered_map<char, std::unique_ptr<TrieNode>> next;
    bool terminal = false;
};
void insert(TrieNode& root, std::string_view word)
```

```
{
    TrieNode* node = &root;
    for (char c : word)
    {
        auto& child = node->next[c];
        if (!child)
            child = std::make_unique<TrieNode>();
        node = child.get();
    }
    node->terminal = true; // 路径存在表示前缀，terminal才表示完整单词
}
```

105 待排序数据装不进内存时，怎样实施外部归并排序？

答案　外部归并排序适用于数据太大、不能全部放进内存的情况。第一步按内存预算读一块数据，在内存中排序，再写成一个有序临时文件段；重复到原始数据处理完。第二步同时读取多个有序段，每次选各段当前最小元素中最小的一个写入结果，从而归并成更大的有序段。

所有段一次合并可能需要过多输入缓冲区、文件描述符或 I/O 通道。若当前有 R 段、每轮最多合并 k 段，就需要多轮，轮数约为 ceil(log_k R)。最小堆或败者树可快速挑出各段首项；通常 I/O 比较昂贵，所以块大小、顺序读写和缓冲策略很关键。

临时结果可能接近原数据大小，要预留足够磁盘空间。失败时应能清理临时段并保留原输入，完整结果写好后再原子发布。若要求稳定排序，相同键还需按原始段/位置定义优先顺序。临时文件也要校验、设置合适权限，并防止被其他进程替换。〔N157〕

代码示例

```
while (input.has_more())
{
    auto chunk = input.read_bounded(memory_budget);
    std::sort(chunk.begin(), chunk.end());
    write_run(chunk);
}
while (runs.size() > 1)
    runs = merge_at_most_k_runs(runs, k, output_buffer);
// 每轮用最小堆合并多个有序段；发布前校验临时文件
```

106 邻接矩阵、邻接表与边列表各适合什么图操作？

答案　图的存储结构要按主要操作选择。邻接矩阵用 V×V 表格表示任意两个顶点间有没有边，检查一条边通常 O(1)，但空间 O(V^2)，枚举某点邻居要扫一整行。稠密图或频繁查边时很直接，稀疏图会浪费大量空格。

邻接表给每个顶点存一份邻居列表，空间 O(V+E)，DFS、BFS 扫描邻居时只看实际存在的边，适合稀疏图。若列表没有额外索引，问 u 到 v 是否有边就要在 u 的邻居中找。边列表则直接保存每条边的端点和权重，按权排序或逐边松弛很方便，但不适合快速取得某个点的全部邻居。

无向图通常在两端的邻接表都保存一次同一条边；有向图只记录出边。构图前应明确是否允许自环、重边、方向和权重，否则后续算法可能重复计数或误判。没有一种结构适合所有查询：查边、遍历、排序边和内存限制要综合考虑。〔N158〕

代码示例

```
std::vector<std::vector<int>> adjacency(vertex_count); // 稀疏图邻接表占O(V+E)
adjacency[u].push_back(v);
std::vector<Edge> edges{{u, v, weight}};
// 稠密图可用邻接矩阵O(V^2)换O(1)查边；边列表适合扫描全部边
```

107 贪心算法何时能替代动态规划，怎样证明局部选择不会破坏最优解？

答案　贪心算法每一步都直接选一个看起来最好的局部选项，做完通常不会回头修改；动态规划则保存许多子问题结果，并比较不同决策的影响。两者都可能利用“最优子结构”，也就是整体最优解能由子问题最优解组成；但这并不意味着任何局部选择规则都正确。

要使用贪心，必须证明当前局部选择可以出现在某个整体最优解里，而且选择后剩余问题仍属于同类。常见证明是交换论证：从任意最优解出发，把它的第一步逐渐换成贪心选择，证明结果不变差。活动安排按最早结束时间选可行；硬币问题不能只挑最大面额，币值 1、3、4 凑 6 时，4+1+1 比 3+3 多用硬币。

如果无法证明，不要只拿几个样例就认定贪心正确；应尝试构造反例，或用动态规划覆盖多种选择。反过来，DP 也可能因状态数太大而不可行，算法选择要结合约束和复杂度。清楚说明证明依据，比单纯写出“每次选最大的”更重要。〔N159、N160〕

代码示例

```
std::sort(events.begin(),
          events.end(),
          [](auto a, auto b)
```

```
        {
            return a.end < b.end;
        });
for (auto event : events)
    if (event.start >= last_end)
    {
        selected.push_back(event);
        last_end = event.end;
    }
// 活动选择的“最早结束”需用交换论证证明最优；不是所有问题都适合贪心
```

第 14 章 Qt

14.1 核心机制与容器

001 QByteArray 的使用场景是什么?

答案　QByteArray 可以看成有长度的字节袋：它只保存一串数值，不判断内容是文字、图片还是压缩包。因此，读取文件、收发套接字数据、拼装二进制协议时都很合适，内容中出现零字节也没有问题。它提供长度管理、查找和追加操作，比把任意数据当作以零结尾的字符串更稳妥。

如果袋子中装的是文字，还必须约定编码。例如“猫”转成 UTF-8 后占三个字节，接收端要用相同编码还原成 QString。字节数、UTF-16 代码单元数与人眼看到的字符数是不同概念；协议按字节切片可以，显示文字则应先解码。访问 constData() 得到的是借用地址，对象修改或销毁后不能继续使用。

代码示例

```
QByteArray wire = QStringLiteral("猫").toUtf8(); // 显式编码成字节
wire.append('\n'); // 协议分隔符也是字节
send(socket, wire); // 二进制数据不附带字符集信息
const QString decoded = QString::fromUtf8(wire); // 读回时按 UTF-8 解码
const int characters = decoded.size(); // 字符计数不等于 wire.size()
QByteArray compressed = qCompress(wire); // 同一类型也可装任意二进制内容
```

002 QList、QVector、QLinkedList 有什么区别?

答案　先看 Qt 版本，再比较这些容器。Qt 6 的 QList 使用连续内存，QVector 是它的兼容别名，因此不能再沿用“二者底层完全不同”的旧结论。连续存储便于按下标访问和顺序遍历，也容易配合标准算法；尾部追加通常效率很好，中间插入则可能搬动后面的元素并使已有引用失效。

QLinkedList 已从 Qt 6 移除，迁移旧项目时可以评估 std::list。链表把元素放在独立节点中，已知迭代器位置时插入、删除不必整体搬移，但寻找这个位置仍需遍历，而且节点指针、内存分配和缓存访问都有成本。普通数据列表优先选连续容器；只有确实需要链表特性并经测量受益时才改用链表。

代码示例

```
QList<int> values{4, 1, 3}; // Qt 6 中 QList 连续存储
values.append(2); // 尾部追加与顺序遍历常见
std::sort(values.begin(), values.end()); // Qt 容器可接标准算法
// Qt 6 的 QVector 是 QList 的兼容别名，QLinkedList 已移除
std::list<int> nodes{4, 1, 3}; // 旧链表代码可评估 std::list
// 已知位置插入虽方便，链表节点和缓存未命中也有成本
```

003 QMap 和 QHash 有什么区别?

答案　QMap 适合“按键排好队”，QHash 适合“凭键快速找”。QMap 保持键的比较顺序，查找、插入和删除通常是对数复杂度，既能有序遍历，也方便从某个键开始做范围查询。QHash 根据哈希值定位元素，平均查找接近常数时间，但遍历次序不构成接口保证，不能把当前打印顺序当作排序结果。

例如成绩表若要按学号输出，可直接使用 QMap；缓存只需根据用户编号命中记录，则可选择 QHash。自定义哈希键需要相等比较以及与之匹配的 qHash：相等的键必须产生相同哈希值。冲突多时性能会变差；若哈希表结果最终仍需排序，应把排序成本一并考虑。只读查询也应避免用会插入默认值的下标操作。

代码示例

```
QMap<QString, int> ordered{{QStringLiteral("b"), 2}, {QStringLiteral("a"), 1}};
for (auto it = ordered.cbegin(); it != ordered.cend(); ++it)
    consume(it.key(), it.value()); // QMap 按键排序
QHash<QString, int> fast;
fast.insert(QStringLiteral("a"), 1);
const auto found = fast.constFind(QStringLiteral("a"));
if (found != fast.cend())
```

```
    consume(found.value()); // 不依赖遍历顺序
// 自定义 QHash 键还需相等比较与 qHash 实现
```

004 QString 和 std::string 有什么区别?

答案　QString 面向 Unicode 文本，内部以 UTF-16 代码单元表示内容，并提供文本查找、大小写转换、翻译和控件接口。std::string 保存 char 序列，本身不记录字符编码；同样一串字节可能代表 UTF-8、其他编码，甚至二进制内容。因此，二者差异不只是函数名称不同，而是接口对内容含义的约定不同。

在网络或标准库边界应明确编码：发送前用 QString::toUtf8()，收到 UTF-8 字节后用 QString::fromUtf8()。构造字节串时传入长度，可以保留中间的零字节。QString 的长度也不等于所有情况下的人眼字符数，例如某些符号需要代理对表示；std::string 的字节下标更不能直接当中文字符位置。需要按用户可见字符截取时，应考虑字素边界。

代码示例

```
const QString text = QStringLiteral("雪");
const QByteArray utf8 = text.toUtf8(); // 明确写出编码
std::string bytes(utf8.constData(), static_cast<std::size_t>(utf8.size()));
const QString restored = QString::fromUtf8(bytes.data(), static_cast<qsizetype>(bytes.size()));
// std::string 只保存字节，不携带编码；不要按字节下标当 Unicode 字符
showText(restored); // Qt 控件继续使用 QString
```

005 QVariant 是什么？有什么应用场景?

答案　QVariant 是一个同时保存“值”和“值的类型”的容器。它适合接口中可能出现不同类型、但仍希望统一传递的场景，例如属性系统、数据库查询结果和视图模型的数据角色。同一个模型接口可以为显示角色返回姓名文本，为编辑角色返回年龄整数，调用者通过角色约定知道应当怎样解释结果。

取值时仍需核对类型或检查转换是否成功。例如文本“十八”转成整数会失败，不能把转换得到的零当作真实年龄。自定义值通常需要声明元类型，若还要经过排队信号槽等运行时边界，则按要求注册。QVariant 保存某个裸指针时也不会自动获得指向对象的所有权；动态包装并不能代替清晰的类型与生命周期约定。

代码示例

```
QVariant display = QStringLiteral("ready"); // DisplayRole 可返回文本
QVariant edit = 42; // EditRole 也可返回数值
bool ok = false;
const int count = edit.toInt(&ok); // 动态转换要检查结果
if (ok)
    updateCount(count);
QVariant custom = QVariant::fromValue(MyValue{});
qRegisterMetaType<MyValue>(); // 自定义类型跨元对象边界前注册
```

006 Qt QObject 父子对象树如何管理生命周期?

答案　QObject 的父子树是一套所有权规则：父对象销毁时，会删除仍挂在它下面的子对象，子对象销毁时也会从父对象的列表移除。把按钮设为窗口的子对象后，通常无需再单独释放按钮。父子对象必须属于同一线程；迁移线程时，对象本身必须没有父对象，它的子树会一起迁移。

不能同时把同一对象交给父对象和另一个独立拥有者反复删除。栈上对象还要保证构造、析构顺序合理，避免父对象提前删除仍由栈管理的子对象。跨线程清理通常使用 deleteLater()，让所属线程在适当时机销毁对象；它依赖事件循环和线程结束规则，主循环已经停止后不能指望普通延迟删除继续被处理。退出时应安排停止、清理与等待顺序。

代码示例

```
QObject owner; // 栈对象先构造，也会先于局部 child 析构
auto* child = new QObject(&owner); // parent 是唯一所有者且同线程
auto* independent = new Worker; // 无 parent，由调用方负责
independent->moveToThread(&workerThread); // 只能移动无 parent 对象及其子树
connect(&workerThread, &QThread::finished, independent, &QObject::deleteLater);
workerThread.start(); // 退出时须 quit()、wait()，并让事件循环处理延迟删除
child->deleteLater(); // owner 仍会回收；延迟删除依赖所属线程事件循环
```

007 Qt 信号槽依赖什么机制？连接类型如何影响调用?

答案　信号槽把“发生了什么”和“谁来处理”连接起来。带 Q_OBJECT 的类经 moc 生成元对象代码，用来描述信号、属性和运行时调用；函数指针形式的 connect 还能在编译期检查参数是否兼容。发射信号时，Qt 按连接类型决定直接调用槽，还是把参数复制到接收对象线程的事件队列。

AutoConnection 的判断依据是实际发射信号的线程与接收对象的线程归属。相同时直接调用，否则排队；Direct 始终在发射线程同步执行，Queued 等接收线程事件循环处理。BlockingQueued 还会阻塞发送者等待槽返回，同线程使用会死锁。队列连接需要可传递的参数类型和运行中的接收循环，跨线程更新界面应让槽在 GUI 线程执行。

代码示例

```
connect(worker, &Worker::resultReady, window, &Window::showResult);
emit worker->resultReady(result); // AutoConnection 跨线程时排队
connect(worker, &Worker::progress, window, &Window::showProgress, Qt::QueuedConnection);
```

```
connect(this, &Controller::localUpdate, this, &Controller::updateNow, Qt::DirectConnection);
// DirectConnection 在发射线程同步执行；不要用它跨线程碰 GUI
// 同线程 BlockingQueuedConnection 会死锁，接收线程还必须运行事件循环
```

008 Qt 的隐式共享与写时复制是什么？

答案　隐式共享是 Qt 一些值类型减少复制成本的方法。复制 QString 或 QByteArray 时，多个对象可以暂时引用同一块数据，并通过引用计数记录使用者数量；当某个副本要修改内容且数据仍被共享时，它先取得独立数据再修改，这一步叫分离，也叫写时复制。从使用语义上看，两个对象仍是独立的值。

例如把“abc”复制给另一个变量后，只把副本首字节改成大写，原值仍应保持不变。这个机制适合按值返回和传递数据，但修改可能突然触发整块复制，非只读访问也可能引起分离。引用计数的安全性不等于同一个对象可被并发读写；共享实例仍要同步，容器修改后的迭代器和借用地址也应重新检查。

代码示例

```
QByteArray original("abc");
QByteArray copy = original; // 隐式共享：通常只复制引用计数头
copy[0] = 'A'; // 共享时写入会先分离再修改
Q_ASSERT(original == QByteArray("abc")); // 值对象彼此独立
Q_ASSERT(copy == QByteArray("Abc"));
// 写时复制减少值传递开销，但不替代跨线程读写同步
```

009 Qt 容器类和 STL 容器类有什么区别？

答案　Qt 容器和标准库容器都能保存一组对象，选择时主要看接口边界、版本和操作方式。Qt 容器与 Qt API 配合方便，许多容器采用隐式共享，使按值复制暂时只共享数据；标准库容器则围绕标准算法、分配器和范围接口建立了丰富生态，其复制、移动和迭代器规则应按各自类型理解。

例如项目的大多数接口接收 QList，反复转成 std::vector 再转回来可能只有额外开销；纯标准库模块直接使用标准容器通常更自然。Qt 6 中 QList 与 QVector 已采用相同连续存储实现，不能照搬旧版性能比较。是否需要连续内存、频繁中间插入、稳定引用或廉价移动，比“哪家容器更快”更有判断价值，关键路径再用实际数据测量。

代码示例

```
QList<QPointF> points;
points.append(point);
chart->setPoints(points); // Qt接口直接使用Qt容器
std::vector<QPointF> samples;
samples.reserve(count);
std::sort(samples.begin(), samples.end());
// 选择调用边界原生容器，避免不必要的来回转换
```

010 一个信号可以连接多个槽吗？一个槽可以连接多个信号吗？

答案　可以把一个信号连接到多个槽，也可以让一个槽接收多个信号。前者像一次通知触发多个订阅者，例如保存成功后同时刷新列表和更新状态栏；后者适合把按钮点击、菜单动作都汇入同一个保存函数。信号参数必须能与槽参数兼容，槽可以忽略信号末尾不需要的参数。

连接关系是独立的，多执行一次 connect 通常就多建立一条连接，同一动作可能因此被执行两遍。重复初始化时应保存连接句柄、先断开旧连接，或者在适用情况下使用 UniqueConnection；该去重方式不能普遍用于 Lambda。直接连接的槽会在发射期间执行，队列连接则稍后执行，因此多个槽的完成顺序还取决于连接类型和线程，不能仅靠一条信号表达完整事务顺序。

代码示例

```
const auto first = connect(button, &QPushButton::clicked, this, &Window::save);
const auto second = connect(button, &QPushButton::clicked, this, &Window::submit);
// 一次发射会调用两个接收端；同一槽也可连接多个信号
disconnect(first); // 按需移除某条连接
Q_ASSERT(second); // 检查连接句柄仍代表有效连接
// 重复初始化会重复响应；UniqueConnection 适用于成员函数连接
// Lambda 需保存 QMetaObject::Connection 管理其连接生命周期
```

011 信号与槽的优缺点是什么？

答案　信号槽的主要收益是降低对象之间的依赖。发送者只需要声明“数据改变了”，无需知道界面、日志或其他接收者的具体实现；接收者可以自由订阅和取消。函数指针连接可以检查类型，队列连接还提供跨线程调度，使后台计算结果能够回到界面线程处理，便于把业务和展示分开。

代价是连接管理、参数复制和元对象分发有额外成本，异步调用链也不像普通函数调用那样容易追踪。排队只决定槽在哪里执行，并不会自动锁住其他共享变量；界面控件仍须在 GUI 线程使用。调试时同时记录连接建立位置、信号发射位置和槽进入位置，必要时用 dumpObjectInfo() 查看对象信息。需要立即返回计算结果的紧密调用，普通函数往往更直接。

代码示例

```
connect(source, &Source::changed, view, &View::refresh, Qt::QueuedConnection);
emit source->changed(value); // 发送方只发布事件，不依赖具体 UI 实现
// 跨线程排队仅调度调用，不会锁住外部共享变量
// 用 QObject::dumpObjectInfo() 查看连接与对象信息
qInfo() << "Source::changed emitted" << value; // 记录异步链路关键点
// GUI 接收端在主线程处理；锁和对象亲和性仍需独立遵守
```

012 如何自定义信号和槽?

答案　自定义信号通常在 QObject 子类中声明，并在类中加入 Q_OBJECT，让构建系统运行 moc 生成所需代码。signals 区域写出信号名称和参数，例如 dataReady(字节数据)；信号函数一般不自己编写实现，在结果准备好时用 emit 发射。emit 强调通知语义，本身不会自动创建线程或延迟执行。
接收端可以是成员函数，也可以是 Lambda，新式 connect 用函数指针把它们连接起来。为 Lambda 指定上下文对象，可以让连接随该对象销毁而失效，避免回调访问已销毁的界面。跨线程排队时，参数必须能安全复制并符合元类型要求；同样要保证接收线程处理事件。若出现缺失元对象相关链接符号，应先检查 Q_OBJECT 和 moc 构建配置。
代码示例

```
class Worker : public QObject
{
    Q_OBJECT
  signals:
    void dataReady(const QByteArray& bytes); // 信号通常无需函数体
  public slots:
    void run()
    {
        emit dataReady(makePayload());
    }
};
connect(worker, &Worker::dataReady, receiver, &Receiver::handleData);
// 含 Q_OBJECT 的类需要构建系统运行 moc
```

14.2 界面与绘图

001 Graphics View 框架是什么？适用于什么场景?

答案　Graphics View 是用于交互式二维画布的框架，由场景、视图和图元共同工作。图元表示一个可绘制对象，场景保存对象和它们的位置，视图把场景映射到窗口中。流程图中的节点、连线以及地图上的标记都可以各自成为图元，框架还能协助处理选中、移动、碰撞查询和事件分发。
它的优势在于不用自己重新组织全部坐标与命中逻辑，同一个场景还可以同时显示在主编辑区和缩略预览中。视图缩放改变的是观察方式，图元的场景位置可以保持原值。普通设置表单直接使用 Widgets 布局更合适；若画布有大量图元，则仍应测量索引、局部重绘、图元缓存和交互复杂度，框架并不保证任意规模都流畅。
代码示例

```
auto* scene = new QGraphicsScene(this); // 管理图元和场景坐标
auto* item = scene->addEllipse(QRectF(0, 0, 24, 24), QPen(Qt::blue), QBrush(Qt::cyan));
item->setFlag(QGraphicsItem::ItemIsSelectable);
auto* view = new QGraphicsView(scene, this); // 视图处理滚动与变换
view->scale(1.5, 1.5); // 编辑器类场景可缩放和平移
// 普通表单无需 Graphics View；大场景再测索引和局部更新
```

002 QGraphicsScene、QGraphicsView、QGraphicsItem 的关系是什么?

答案　可以把三者理解为物体、世界和观察窗口。QGraphicsItem 是物体，负责自身形状、边界、绘制和交互；QGraphicsScene 是世界，管理图元集合、场景坐标、选择状态和空间查询；QGraphicsView 是观察窗口，把场景的一部分经过缩放、滚动等变换显示出来，并把输入映射回场景。
一个场景可以交给多个视图，因此在主画布移动节点，缩略图也能看到同一个节点的新位置。图元还可以有父图元，子图元的局部坐标和变换会受到父图元影响。需要注意，普通 QGraphicsItem 并不是 QObject，图元父子关系也不能直接等同于 QObject 父子树；如果图元需要信号、槽或属性，可以使用 QGraphicsObject，并明确场景和图元的清理责任。
代码示例

```
auto* scene = new QGraphicsScene(this);
auto* shape = scene->addRect(QRectF(0, 0, 80, 40)); // QGraphicsItem
shape->setFlag(QGraphicsItem::ItemIsMovable);
mainView->setScene(scene); // view 映射同一个 scene
previewView->setScene(scene); // 多个 view 可同时观察
previewView->scale(0.25, 0.25);
// 需 QObject 信号槽的图元可改用 QGraphicsObject
```

003 QPaintDevice、QPaintEngine、QPainter 的关系是什么?

答案　QPaintDevice 表示“画到哪里”，例如窗口控件、内存图像或打印设备；QPainter 表示“怎样画”，提供线条、文字、填充和坐标变换等高层操作；QPaintEngine 则负责把这些操作交给具体后端执行。通常业务代码使用 QPainter，设备选择相应引擎，不需要自己直接调用绘图引擎。

这层分工让相似的绘制逻辑可以输出到屏幕、QImage 或 PDF，但各设备支持的能力、像素格式、分辨率与性能可能不同。例如在透明图像上绘制，要选择支持透明度的格式并初始化背景。绘制期间应让 painter 的作用域清晰，同一设备不能随意被多个 painter 同时操作；窗口绘制仍应遵守 paintEvent 的时机，不能因为用了统一接口就绕开事件规则。

代码示例

```
QImage image(QSize(160, 80), QImage::Format_ARGB32_Premultiplied);
image.fill(Qt::transparent); // QImage 也是 paint device
{
    QPainter painter(&image); // QPainter 发出绘图指令
    painter.drawText(QPoint(8, 32), QStringLiteral("preview"));
} // painter 结束后再读取或显示 image
widget->update(); // QWidget 由窗口绘制后端输出，不要长期持有 painter
```

004 QPainter 的基本使用方法是什么?

答案　QPainter 的常见用法是在控件的 paintEvent() 内创建局部画笔，设置描边、填充、字体和变换，然后发出绘图指令。画笔作用域结束后，本轮绘制随之结束。绘制函数应根据当前状态重建画面，因为窗口遮挡、缩放或系统刷新都会再次触发它，不能只依赖此前留在屏幕上的像素。

业务数据变化时先修改状态，再调用 update() 请求重绘；Qt 可以合并多个更新请求，稍后统一绘制。不要手工调用 paintEvent，也不要在普通按钮回调中直接对 QWidget 作画。图像处理则可以创建指向 QImage 的 painter。绘制路径应保持短小，文件读取、网络请求和耗时计算应提前完成，否则会阻塞界面事件循环。

代码示例

```
void PlotWidget::setValue(double value)
{
    value_ = value;
    update(); // 请求事件循环合并重绘
}
void PlotWidget::paintEvent(QPaintEvent* event)
{
    QPainter painter(this); // 仅在 paintEvent 中画 QWidget
    painter.setClipRegion(event->region()); // 只画失效区域
    painter.drawLine(QPoint(8, 8), mapValue(value_));
} // 不直接调用 paintEvent，也不在其中做 I/O
```

005 QPixmap、QImage、QBitmap 有什么区别？使用场景是什么?

答案　QImage 面向图像数据，能够直接访问像素，适合读写图片、逐像素变换和后台图像计算；QPixmap 面向界面显示，通常与平台的窗口图像资源配合；QBitmap 则是单色位图，常用于只有两种取值的遮罩。因此三者应根据“处理像素”还是“把结果放上屏幕”来选择，而不是简单按清晰度排序。

例如后台线程把照片解码为 QImage，完成滤镜处理后传回 GUI 线程，再转换成 QPixmap 放入标签。发布后不要继续无同步地修改同一份图像实例，跨线程传递应保证值和所有权明确。QPixmap 的具体实现依赖平台，不能笼统断言它一定使用 GPU，也不宜把界面图像资源的线程限制当作 QImage 的限制。格式转换还可能带来复制成本，应避免每帧反复转换。

代码示例

```
// 后台线程解码和逐像素处理使用 QImage
QImage decoded = QImageReader(path).read();
if (decoded.isNull())
    report(QStringLiteral("image decode failed"));
QImage processed = transform(decoded); // 线程间传值时按数据所有权处理
QMetaObject::invokeMethod(
    this,
    [this, processed]
    {
        label->setPixmap(QPixmap::fromImage(processed)); // GUI 线程显示
    },
    Qt::QueuedConnection);
// QBitmap 仅适合单色蒙版；不要假设 QPixmap 一定由 GPU 加速
```

006 QWidget、QDialog、QMainWindow 有什么区别?

答案　QWidget 是 Widgets 界面的基本积木，按钮、面板和顶层窗口都建立在它之上。QDialog 在这个基础上增加对话框的完成语义，能区分接受和取消，适合设置、确认和短表单。QMainWindow 则提供完整主窗口结构，包括中央控件、菜单栏、工具栏、停靠区和状态栏，适合承载应用的主要工作区。

例如编辑器用 QMainWindow 放文档区和工具栏，打开设置时创建 QDialog，设置页内部再由普通 QWidget 和布局组合。模态对话框会限制用户与其他窗口交互，非模态对话框允许继续操作，应按任务选择。三者都属于 GUI 对象，应在 GUI 线程创建和操作；通常通过父子关系管理子控件，并用布局分配尺寸，而不是为每一种窗口重新手算坐标。

代码示例

```
auto* window = new QMainWindow;
auto* editor = new QWidget(window);
window->setCentralWidget(editor);
auto* dialog = new QDialog(window);
connect(ok_button, &QPushButton::clicked, dialog, &QDialog::accept);
// 主窗口管理中央控件；对话框通过accept/reject返回结果
```

007 Qt Designer 和手写代码创建界面有什么区别?

答案　Qt Designer 把控件树和布局保存为 .ui 表单，构建时由 uic 生成初始化代码，适合控件结构相对稳定的窗口。它便于直观看到层级、边距和伸缩关系，但生成文件属于构建产物，直接手改会在下一次生成时丢失。业务连接和数据更新通常放在自己的窗口类中，保持界面定义与逻辑边界清楚。

手写代码更适合按数据数量动态创建控件、复用复杂构造逻辑或实现特殊交互。二者可以组合：Designer 负责固定表单，程序运行后填充模型、绑定信号槽，再按需要添加组件。已有自定义控件可以通过提升为子类接入表单。选择依据是维护和复用方式；无论来源如何，最终仍需检查布局、父子所有权以及真实窗口缩放效果。

代码示例

```
Ui::SettingsForm ui;
ui.setupUi(this); // uic根据.ui生成setupUi代码
for (const Device& device : query_devices())
    ui.deviceLayout->addWidget(new DeviceRow(device));
connect(ui.saveButton, &QPushButton::clicked, this, &Settings::save); // 不手改生成文件
```

008 Qt 中如何实现反走样（抗锯齿）?

答案　抗锯齿通过让边缘像素按覆盖程度混合颜色，减轻斜线和曲线的阶梯感。QPainter 中可以启用 Antialiasing；文字有 TextAntialiasing 提示，缩放图片则可使用 SmoothPixmapTransform。它们解决的是不同绘制对象的问题，设置一个提示并不意味着所有内容都会获得同样的平滑效果。

提示应在本轮 painter 创建后、实际绘制前设置，后端是否支持以及最终效果还受设备像素比和坐标变换影响。例如圆边缘可以更平滑，但把低分辨率头像放大很多倍仍然会模糊，抗锯齿不能补回原图缺失的细节。动画或大量图元绘制时需要测量成本，并结合高分辨率素材、合理坐标和局部重绘改善结果。

代码示例

```
void Canvas::paintEvent(QPaintEvent*)
{
    QPainter painter(this);
    painter.setRenderHint(QPainter::Antialiasing); // 曲线与斜线
    painter.setRenderHint(QPainter::TextAntialiasing); // 文字单独控制
    painter.setRenderHint(QPainter::SmoothPixmapTransform); // 缩放图像
    painter.drawEllipse(QPointF(40, 40), 25, 25);
} // 效果受设备像素比、变换和绘图后端影响
// 高频绘制要在目标设备测量开销
```

009 Qt 中如何实现拖拽功能?

答案　拖放可以分成“源端打包”和“目标端接收”两部分。源端检测到达到拖动距离后创建 QDrag，把文字、文件地址或自定义格式放入 QMimeData，再执行拖动；MIME 类型描述这份数据如何解释，使目标不需要知道源对象的内部结构，也便于在不同控件甚至不同应用间交换。

目标控件先启用接收拖放，在 dragEnterEvent 或 dragMoveEvent 检查格式和允许动作，符合条件才接受；dropEvent 到来时仍需重新验证内容，再真正更新数据。不要传一个仅在源进程有效的裸指针给外部应用，也不能只因拖入事件接受了就信任最终数据。复制与移动语义应明确，文件拖放还应处理路径、权限和读取失败，只有完成实际操作才更新界面状态。

代码示例

```
void SourceWidget::mouseMoveEvent(QMouseEvent* event)
{
    if (!dragging_)
        return;
    auto* mime = new QMimeData;
```

```
    mime->setText(selectedText()); // MIME 可跨 widget/应用传值
    QDrag drag(this);
    drag.setMimeData(mime);
    drag.exec(Qt::CopyAction | Qt::MoveAction);
}
void TargetWidget::dragEnterEvent(QDragEnterEvent* event)
{
    if (event->mimeData()->hasText())
        event->acceptProposedAction();
} // dropEvent 中再次校验 MIME，再读取文本
```

010 Qt 中有哪些常用的布局管理器?

答案　布局管理器负责在可用空间变化时重新安排子控件。QHBoxLayout 和 QVBoxLayout 分别按横向、纵向排列，QGridLayout 使用行列网格，QFormLayout 适合标签与输入框成对出现的表单，QStackedLayout 则让多个页面共享一块区域、一次显示其中一个。通过嵌套这些布局，可以组合较复杂的界面。

布局会结合控件的建议大小、最小尺寸、尺寸策略、伸缩因子和边距分配空间。例如输入框可以吸收窗口增加的宽度，而保存按钮维持自然宽度。把布局安装在父控件上后，不应继续用固定坐标反复调整同一批子控件，否则两套几何规则互相覆盖。布局还应在字体变大、翻译文本变长和窗口缩小时检查，避免仅在设计分辨率下正常。

代码示例

```
auto* form = new QFormLayout;
form->addRow(tr("Name"), nameEdit); // 标签加输入框
auto* grid = new QGridLayout;
grid->addWidget(formWidget, 0, 0); // 网格可放子布局
auto* pages = new QStackedLayout;
pages->addWidget(pageA);
pages->addWidget(pageB);
pages->setCurrentWidget(pageA); // 页面切换不重设子控件坐标
parentWidget->setLayout(grid); // 布局接管几何分配和缩放
```

011 Qt 样式表（QSS）是什么？如何使用?

答案　QSS 是 Qt 的控件样式规则，语法与 CSS 相似，可通过控件或应用的 setStyleSheet() 设置背景、边框、字体和状态效果。选择器可以按控件类型、对象名和动态属性匹配，伪状态则表示悬停、禁用等情况。例如只给对象名为保存按钮的控件着色，可以避免影响其他按钮。

样式主要决定外观，不能替代业务行为、布局或复杂绘图。写了禁用样式并不等于按钮已经不可点击，仍需调用 setEnabled(false)。应用级选择器覆盖面大，可能改变平台原生观感，也可能让大量控件重新计算样式；应控制作用范围。QSS 并不完整实现浏览器 CSS，复杂定制可以考虑控件绘制或 QStyle，同时检查字体、边框和内边距是否影响尺寸。

代码示例

```
button->setObjectName(QStringLiteral("saveButton"));
button->setProperty("accent", true); // 属性可供选择器匹配
qApp->setStyleSheet(QStringLiteral("QPushButton[accent=\"true\"] { background: #2878c8; }"
                                   "QPushButton:hover { border: 1px solid white; }"));
// 应用级规则会覆盖较大范围；复杂绘制仍由控件/QStyle 实现
button->setEnabled(false); // 样式不会替代交互状态逻辑
```

012 Qt 的事件处理机制是什么？事件传递顺序是怎样的?

答案　Qt 的事件循环不断取出系统输入和已排队事件，再把事件交给目标对象。对象的 event() 是统一入口，默认实现会按类型分派到鼠标、键盘、绘制等专用处理函数。postEvent() 把事件放进队列稍后处理，sendEvent() 则立即同步分发，因此后者可能在当前调用尚未结束时进入其他处理逻辑。

事件过滤器可以在目标处理前观察或截断事件，专用处理函数则决定如何响应。accept() 和 ignore() 的意义要结合具体事件类型理解：某些忽略的输入事件可以继续传播到父控件，但不能认为所有事件都沿父子树逐层冒泡。例如重绘事件并不是普通鼠标事件的传播规则。重写 event() 时，对自己不处理的类型通常应调用基类，以保留默认分派和控件行为。

代码示例

```
class Target : public QWidget
{
    void mousePressEvent(QMouseEvent* event) override
    {
        if (event->button() == Qt::LeftButton)
            event->accept();
        else
            event->ignore(); // 某些输入事件可继续向父控件传播
```

```
    }
};
QCoreApplication::postEvent(target, new MyEvent(payload)); // 排队稍后分发
// sendEvent() 同步重入调用；事件循环调用 event() 再分派专用处理函数
```

013 event() 与事件过滤器有什么区别?

答案　event() 属于目标对象本身，是它接收多种事件的统一入口；适合在子类中处理自定义事件，或协调几类输入。事件过滤器则由另一个 QObject 安装到目标上，在目标入口前检查事件，适合统一处理多个输入框的快捷键、记录输入，或者临时拦截某类操作，不必为每个控件创建子类。

过滤器返回 true 表示事件已被过滤，目标不会再收到这一事件；返回 false 才允许继续处理。这与事件对象自身的 accept 状态不是同一个开关。过滤器和目标必须处于同一线程，重写函数时也应检查 watched 是哪个对象。若只是旁观，应放行事件；在 event() 中不处理的事件则交还基类，避免无意破坏键盘、布局或其他默认行为。

代码示例

```
target->installEventFilter(this); // filter 与 target 必须同线程
bool Filter::eventFilter(QObject* watched, QEvent* event)
{
    if (watched == target && event->type() == QEvent::MouseButtonPress)
    {
        handlePress(static_cast<QMouseEvent*>(event));
        return true; // 已处理，目标不会再收到该事件
    }
    return QObject::eventFilter(watched, event); // 保持默认过滤行为
}
```

014 paintEvent()、resizeEvent()、mousePressEvent()等事件函数的作用是什么?

答案　paintEvent、resizeEvent 和 mousePressEvent 是 QWidget 对不同事件类型的专用回调。paintEvent 在控件需要重画时执行，通常响应 update；resizeEvent 可据新尺寸调整缓存；mousePressEvent 可读取按键、位置和修饰键以开始交互。它们由事件系统调用，而不是应用随意调用的普通绘图函数。

回调应快速完成状态更新或绘制，耗时计算放在工作线程，否则鼠标和绘制事件都要排队，界面就会卡住。重写后通常在需要保留默认行为时调用基类实现；例如自定义控件只接管左键，其他键仍交回 QWidget。paintEvent 应能从当前状态重建受影响区域，不能依赖上一次屏幕内容。

代码示例

```
void Canvas::resizeEvent(QResizeEvent* event)
{
    cachedSize_ = event->size(); // 重算与尺寸相关的缓存
    QWidget::resizeEvent(event); // 按需保留基类行为
}
void Canvas::mousePressEvent(QMouseEvent* event)
{
    if (event->button() == Qt::LeftButton)
    {
        beginDrag(event->position());
        return;
    }
    QWidget::mousePressEvent(event);
}
void Canvas::paintEvent(QPaintEvent*)
{
    QPainter(this).drawImage(rect(), cachedImage_);
}
// 事件回调只更新轻状态；耗时工作放到 worker
```

015 如何实现双缓冲绘图?

答案　双缓冲先在屏幕外准备一帧，再把完整结果显示出来，避免逐笔绘制时用户看到擦除和重画的中间过程。对于 QWidget，Qt 通常已经通过 backing store 提供双缓冲，所以自绘控件一般在 paintEvent 中完整画出失效区域即可，不需再为整窗复制一张缓冲图。

手动缓存更适合复用昂贵但静态的背景，再把每帧变化的动画对象叠加绘制。缓存大小、缩放和设备像素比发生变化时应重新生成；只重绘脏区域可以省成本，但必须保证缓存其他部分仍有效。额外缓冲会占用内存，连续保存多张全窗口图像可能比绘制本身更昂贵，需按实际帧率和内存测量。

代码示例

```
// QWidget 已由 backing store 双缓冲，通常不需手写整窗缓冲
void Canvas::setStaticLayer(const QImage& layer)
```

```
{
    cachedLayer_ = layer;
    update();
}
void Canvas::paintEvent(QPaintEvent* event)
{
    QPainter painter(this);
    painter.setClipRegion(event->region());
    painter.drawImage(QPoint(0, 0), cachedLayer_); // 重用静态层
    drawMovingObjects(painter); // 每帧只重绘变化部分
}
// 缓存需随尺寸/DPR 更新；避免保存多份全尺寸图像
```

016 如何实现自定义控件?

答案　自定义控件应从能提供所需行为的基类开始：纯展示或绘图通常继承 QWidget，按钮仍需保留按下、键盘和可访问语义时则继承合适的按钮类。再实现 paintEvent 和必要的输入、尺寸事件，用属性保存状态，并通过信号向外报告用户动作，而不是让外部代码直接改内部像素。

sizeHint 和 minimumSizeHint 告诉布局控件偏好的尺寸，焦点策略决定键盘是否可达；交互控件还应提供可访问名称、状态和键盘操作。资源用父对象所有权或 RAII 管理，控件应在 GUI 线程使用。把业务逻辑放进控件时要控制边界：控件负责输入和呈现，数据持久化或长任务由外部服务处理。

代码示例

```
class Meter : public QWidget
{
    Q_OBJECT
  public:
    QSize sizeHint() const override
    {
        return QSize(120, 32);
    }
  signals:
    void valueActivated(int value); // 交互用信号报告给外部
  protected:
    void paintEvent(QPaintEvent*) override
    {
        QPainter(this).drawText(rect(), text_);
    }
    void mousePressEvent(QMouseEvent*) override
    {
        emit valueActivated(value_);
    }
};
// QWidget 子类留在 GUI 线程；父子树或 RAII 管理其资源
```

017 模态对话框和非模态对话框有什么区别?

答案　模态对话框会暂时阻止指定范围内的其他窗口交互，适合需要立即确认的短流程；非模态对话框打开后，用户仍可操作主窗口，适合查找面板、监控窗口或持续调整的设置。模态范围可能是应用级或窗口级，应根据用户能否切换其他窗口的预期选择。

QDialog::exec() 会启动嵌套事件循环并在关闭后同步返回 Accepted/Rejected，因此调用栈停在等待位置，但期间其他事件仍可能进入，产生重入。open() 或 show() 以异步方式打开，再监听 finished 信号通常更容易维护流程。无论哪种方式，都要清楚管理对话框对象生命周期；通过 parent 持有、完成后 deleteLater 可避免悬空对象。

代码示例

```
auto* dialog = new SettingsDialog(this); // parent 保证窗口对象活到关闭
connect(dialog, &QDialog::finished, dialog, &QObject::deleteLater);
connect(dialog, &QDialog::finished, this, &Window::applySettings);
dialog->open(); // 非模态流程，不启动同步嵌套循环
// 短操作可用 exec() 并检查 Accepted/Rejected
// exec() 返回前 GUI 线程仍可重入处理其他事件
```

14.3 并发与网络

001 QMutex、QReadWriteLock、QSemaphore 有什么区别?

答案　QMutex 让同一时刻只有一个线程进入受保护区，适合保护普通共享状态；QReadWriteLock 把访问分成共享读锁和独占写锁，读多写少时可能让多个读者并行，但频繁写入时需关注等待情况。QSemaphore 管理一定数量的许可，适合限制同时运行的任务数，例如最多打开八个设备槽位。

无论选择哪一种，同一共享数据的所有读写路径都必须遵守一致的同步约定。QMutexLocker、QReadLocker 等 RAII 守卫会在作用域退出时释放锁，因此提前返回也安全。信号量许可必须与实际资源使用对应：获取后所有成功、失败和取消路径都要释放，否则许可逐渐耗尽，后续任务将永久等待。

代码示例

```
QMutex mutex;
{
    QMutexLocker lock(&mutex); // 所有访问共享状态的路径用同一把锁
    ++sharedCount_;
} // 提前 return 也会释放
QReadWriteLock rw;
{
    QReadLocker read(&rw);
    inspect(sharedData_);
} // 多读者
{
    QWriteLocker write(&rw);
    sharedData_ = next;
} // 写者独占
QSemaphore slots(8); // acquire 后使用许可，完成必须 release
```

002 Qt 中有哪些多线程方式？QThread 应如何使用?

答案　Qt 的并行任务可按工作形态选择工具：独立的一次性计算常用 QtConcurrent 或 QThreadPool；需要长期运行并处理事件的对象可放在 QThread 中。常见做法是创建无父对象的 worker QObject，移入线程，再用排队信号槽交互。QThread 对象本身仍属于创建它的线程，它只是管理另一个执行线程，不是 worker 的替代品。

GUI 控件应留在主线程。停止时先请求 worker 协作取消，不能指望 quit() 打断一个正在执行的长函数；然后让事件循环退出并 wait 等待线程结束，再清理对象。需要队列槽的 worker 线程必须运行事件循环；若长任务占住线程执行栈，它也暂时无法处理新的队列请求，取消标志应能被计算过程定期检查。

代码示例

```
auto* thread = new QThread(this); // QThread 对象属于创建线程
auto* worker = new Worker;
worker->moveToThread(thread); // worker 无 parent
connect(thread, &QThread::started, worker, &Worker::process);
connect(worker, &Worker::finished, thread, &QThread::quit);
connect(thread, &QThread::finished, worker, &QObject::deleteLater);
thread->start(); // 工作槽需要 worker 线程事件循环
// 关闭时请求协作取消，再 quit() 并 wait(); quit 不打断长函数
```

003 QtConcurrent 是什么？有什么应用场景?

答案　QtConcurrent 提供较高层的并行算法和 run()，把任务提交给线程池执行，适合互相独立的计算、压缩或批处理，不必自己创建和回收每个线程。返回的 QFuture 表示任务状态和结果；需要完成通知、进度或界面响应时，可配合 QFutureWatcher 监听信号。线程池会复用工作线程，实际并行度受系统和池配置影响。

并行执行不等于自动安全：任务不能直接读写 QWidget，也要避免多个任务无同步修改同一共享对象。完成后用 watcher 在其所属线程中取结果并更新界面；错误传播和取消能力取决于任务及 API 设计，长时间任务不应假定能被强制中断。若任务间有依赖、需要长期事件循环或精细线程亲和性，worker-object 模式可能更合适。

代码示例

```
QFuture<Result> future = QtConcurrent::run(
    [input]
    {
        return calculate(input);
    });
auto* watcher = new QFutureWatcher<Result>(this);
connect(watcher,
        &QFutureWatcher<Result>::finished,
        this,
        [watcher]
        {
```

```
            display(watcher->result()); // 回主线程显示结果
            watcher->deleteLater();
        });
watcher->setFuture(future); // pool 管理工作线程
// 任务不可直接访问 QWidget；长任务需设计取消与错误传播
```

004 Qt 中如何实现线程安全?

答案　线程安全要先明确数据归谁所有。若能让一个线程独占可变 QObject，其他线程通过消息传递不可变结果，通常比到处加锁更容易推理；必须共享时，才用 QMutex 或读写锁保护。所有读写者都要使用同一同步规则，锁只保护受约束的代码，不能覆盖锁外访问。

QObject 的线程亲和性表示事件投递和对象归属的线程，不表示任意成员函数调用都自动安全。跨线程调用事件驱动对象，通常用队列信号槽；GUI 组件只在 GUI 线程操作。析构、取消和线程退出顺序也属于安全性的一部分。数据竞态之外还要检查死锁和对象提前销毁，尤其避免持锁等待一个必须回头获取同一锁的回调。

代码示例

```
// 优先让一个线程独占可变 QObject 状态
connect(worker, &Worker::resultReady, controller, &Controller::applyResult, Qt::QueuedConnection);
void Controller::applyResult(const Result& result)
{
    model_->replace(result); // 模型只在所属线程被修改
    emit viewNeedsRefresh();
}
// 若必须共享裸数据，对所有读写统一加同一把锁
// 线程亲和性本身不等于对象方法线程安全
```

005 Qt 中如何实现线程间通信?

答案　线程间通信首选清楚的消息边界。发送者发出信号，接收对象属于目标线程时使用 QueuedConnection，Qt 把调用排入该线程的事件循环；工作线程完成结果后发给 GUI 窗口，就是典型用法。也可使用带互斥量和条件变量的队列，但生产者、消费者和停止条件都要自己设计。

排队信号参数应能按要求复制或注册元类型，接收线程必须有机会处理事件；如果目标事件循环已停止，槽就不会按预期执行。大对象要考虑复制成本，可传不可变共享值或句柄并明确所有权。无论如何，不要从工作线程直接调用 QWidget 更新界面；消息传递只负责调度，不替代业务层校验。

代码示例

```
connect(worker, &Worker::resultReady, window, &Window::showResult, Qt::QueuedConnection);
worker->startWork(); // 发射端可在工作线程
// Qt 把槽排到 window 所在线程的事件循环
// 参数类型需可复制；自定义类型跨队列要注册元类型
// 事件循环未运行时队列槽不会处理
// worker 只传结果，不直接访问 QWidget
```

006 如何避免多线程中的死锁?

答案　死锁常见于两个线程各自持有一把锁，再等待对方的锁；解决办法之一是给所有代码规定同一把锁的获取顺序，并尽量缩短临界区。持锁时不要做网络 I/O、等待事件或同步等待另一线程，因为对方可能需要这把锁才能继续。使用 RAII 锁守卫可以保证异常和提前返回路径释放资源，但它本身不会消除错误的锁顺序。

tryLock 超时可以让等待者恢复或重试，却必须设计清楚失败时如何回滚部分状态，不能只把死锁变成无声数据错误。BlockingQueuedConnection、互相 wait/join 和锁内发射会触发同步槽的场景也可能形成等待环。复杂系统应记录线程、锁和超时信息，并优先通过单线程拥有状态、异步消息减少锁的数量。

代码示例

```
// 多锁统一规定顺序：先 accountMutex，再 ledgerMutex
QMutexLocker first(&accountMutex);
if (!ledgerMutex.tryLock(100))
{
    rollbackLocalChanges(); // 超时也要恢复一致状态
    return;
}
QMutexLocker second(&ledgerMutex, QMutexLocker::AdoptLock);
commitTransfer(); // 尽量缩短临界区
// 持锁时不要同步等待信号、I/O、另一线程 wait() 或阻塞队列槽
```

14.4 数据、模型与事件

001 Delegate 的作用是什么？如何自定义 Delegate?

答案　Delegate 是 Model/View 中负责单元格外观和编辑交互的对象。视图按需向 delegate 要求绘制某个可见项，因此不必给表格每一行都创建一个真实 QWidget，适合大数据列表。自定义 delegate 通常继承 QStyledItemDelegate，重写 paint() 绘图、sizeHint() 提供建议尺寸；需要编辑时再创建临时编辑器并在模型和编辑器之间传值。

例如状态列直接画绿色圆点，比常驻一个小控件更轻。编辑流程中 createEditor、setEditorData、setModelData 应共同遵守模型角色和数据校验约定；不要绕过模型直接改底层记录，否则视图不会收到更新通知。绘制代码应尊重传入的 option 状态，如选中、禁用和焦点，并尽量复用当前 style 的标准绘制行为来保持平台一致。

代码示例

```
void StatusDelegate::paint(QPainter* p, const QStyleOptionViewItem& o, const QModelIndex& i) const
{
    const bool active = i.data(Qt::UserRole).toBool();
    p->setBrush(active ? Qt::green : Qt::gray);
    p->drawEllipse(o.rect.center(), 5, 5); // 绘制可见单元格，无常驻 QWidget
}
QWidget*
StatusDelegate::createEditor(QWidget* parent, const QStyleOptionViewItem&, const QModelIndex&) const
{
    return new QComboBox(parent); // 编辑时才创建 editor
} // setEditorData/setModelData 在编辑器和 model 间传值
```

002 Model / View 中的数据角色（Role）是什么?

答案　Role 是模型为同一项提供不同用途数据的标识。视图请求 DisplayRole 时通常要显示文本，EditRole 提供编辑值，DecorationRole 可返回图标，ToolTipRole 提供悬停提示。模型的 data(index, role) 根据行列和角色返回 QVariant，因此一条任务记录可以同时展示标题、图标和完成状态，而不必把所有信息拼成一个字符串。

应用可以从 Qt::UserRole 开始定义自己的角色，例如是否已完成，并集中命名这些标识。若在 QML 中暴露模型角色，还需通过 roleNames() 提供稳定名称。读取和编辑应保持角色含义一致，setData 修改后发出 dataChanged 并说明受影响角色；模型返回空值或转换失败时，视图和编辑器都应有明确处理。

代码示例

```
QVariant TaskModel::data(const QModelIndex& i, int role) const
{
    if (!i.isValid() || i.row() >= tasks_.size())
        return {};
    const Task& task = tasks_[i.row()];
    if (role == Qt::DisplayRole)
        return task.title;
    if (role == Qt::DecorationRole)
        return task.icon;
    if (role == Qt::UserRole)
        return task.done;
    return {};
}
// 自定义角色集中声明；QML 列表模型还实现 roleNames()
// 视图按角色取值，不要把所有数据塞成一个显示字符串
```

003 QAbstractItemModel 的作用是什么?

答案　QAbstractItemModel 定义了数据源与视图之间的协议：视图通过行数、列数、父索引、data() 和 flags() 查询内容，模型在数据变化时通知视图。这样一份数据可以交给列表、表格或树形视图，而不必把每条记录变成控件。简单一维列表通常从 QAbstractListModel 继承，表格和树可以选择更通用的基类。

模型更新通知是协议的一部分。插入和删除行前后必须配对调用 beginInsertRows/endInsertRows 等函数，视图据此维护索引、选择和缓存；任意修改底层数组而不通知，画面会陈旧或索引错位。data() 要检查 index 有效性和行列边界，修改时根据角色发 dataChanged。数据库、过滤器等复杂模型还应清楚定义排序和父子关系。

代码示例

```
int TaskModel::rowCount(const QModelIndex& parent) const
{
    return parent.isValid() ? 0 : static_cast<int>(tasks_.size());
}
QVariant TaskModel::data(const QModelIndex& i, int role) const
{
    if (!i.isValid() || i.row() < 0 || i.row() >= rowCount())
        return {};
```

```
    if (role == Qt::DisplayRole)
        return tasks_[i.row()].title;
    return {};
}
// QAbstractListModel 把数据接口提供给 QListView/QML；修改时必须发模型通知
```

004 QCoreApplication::processEvents() 的作用是什么？

答案　QCoreApplication::processEvents() 会在当前调用栈中同步处理待办事件，因此可让短暂初始化期间的界面偶尔响应；但它会在当前逻辑尚未完成时调用其他槽和回调，出现重入。例如数据结构只初始化了一半，事件处理器却开始读取它。它也不会减少计算量，频繁泵事件还会让控制流程难以追踪。

长任务通常应移入 worker，或拆成由 QTimer 安排的小步骤，让主事件循环自然调度。尤其不要在持锁、模型 begin/end 通知期间或对象半初始化时调用 processEvents，因为任意回调可能观察到不一致状态。若少量场景确实需要处理事件，应限定事件类型、缩短范围并确保状态可重入；这属于特殊工具，不应作为常规“解决卡顿”的方法。

代码示例

```
// 不用 processEvents() 在长循环里反复嵌套分发任意回调
auto* watcher = new QFutureWatcher<Result>(this);
connect(watcher,
        &QFutureWatcher<Result>::finished,
        this,
        [watcher]
        {
            showResult(watcher->result());
            watcher->deleteLater();
        });
watcher->setFuture(QtConcurrent::run(
    [input]
    {
        return compute(input);
    }));
// 少量可切成 QTimer::singleShot() 小步任务
// 持锁、改模型通知或对象半初始化时绝不嵌套处理事件
```

005 QDataStream 和 QTextStream 有什么区别？

答案　QDataStream 面向结构化二进制数据，把整数、字符串等按约定顺序序列化；适合程序存档或二进制协议，但读写双方要固定字节序、QDataStream 版本和字段布局，并管理文件格式升级。QTextStream 面向可读文本，负责字符解码、格式化数字和按行读写，更适合日志、配置文本或 CSV 类输出。

二进制格式紧凑但不便人工检查，文本较易读却要明确编码、分隔和转义规则。跨机器或跨版本保存时，不能只依赖本机默认设置；Qt 6 的 QTextStream 默认使用 UTF-8，但稳定文件格式仍建议显式约定。读取不可信文件时，先限制长度和总资源消耗，再分配内存；读写后检查 stream status，避免静默接受截断数据。

代码示例

```
QDataStream binary(&file);
binary.setVersion(QDataStream::Qt_6_0); // 固定序列化版本
binary.setByteOrder(QDataStream::BigEndian); // 文件格式显式约定字节序
binary << quint32(1) << record.id << record.name; // 结构化二进制
if (binary.status() != QDataStream::Ok)
    report(binary.status());
QTextStream text(&textFile);
text.setEncoding(QStringConverter::Utf8);
text << record.name << '\n'; // 面向可读文本
// 读不可信长度时先限幅，再分配内存
```

006 QFile、QFileInfo、QDir 的作用是什么？

答案　QFile 负责打开和读写实际文件，QFileInfo 查询某一路径的大小、时间、权限等元信息，QDir 则表示目录并提供遍历、创建和路径组合。一个常见流程是用 QDir 列出候选项，再通过 QFileInfo 检查是否为文件，最后由 QFile 打开并处理内容。它们关注点不同，组合使用比手工解析路径更清晰。

I/O 操作可能失败，必须检查 open()、read()、write() 的结果及 errorString，而不能假设路径存在且权限允许。用户输入的文件名还要做路径和权限校验，防止意外读写目录之外的内容。使用 QDir 组合路径可避免自行拼分隔符；但路径规范化也不等同于授权判断，符号链接和平台权限仍需按应用安全边界处理。

代码示例

```
QDir dir(rootPath);
for (const QFileInfo& info : dir.entryInfoList(QDir::Files))
{
```

```
    if (!isAllowed(info.fileName()))
        continue; // 校验输入路径/名称
    QFile file(info.filePath());
    if (!file.open(QIODevice::ReadOnly))
    {
        qWarning() << file.errorString();
        continue;
    }
    consume(file.readAll()); // QFile 析构关闭文件
    qInfo() << info.size() << info.lastModified();
}
// 组合路径用 QDir，不手工拼平台分隔符
```

007 QListView、QTableView、QTreeView 有什么区别?

答案　QListView 按一维列表呈现模型项，QTableView 用行和列展示表格，QTreeView 则显示带父子层级的项目。它们都是 Model/View 视图，可共享一个模型而以不同方式呈现，也能配合 delegate 绘制和编辑。比如文件系统模型提供目录层级，可以交给树视图；联系人表格则适合表格视图。

记录很多时，应优先使用模型和 delegate，而不是为每行创建一套按钮、标签等 QWidget。模型负责数据、视图只负责显示交互，滚动时通常仅处理可见项目，资源消耗更可控。模型形状必须与视图预期匹配：树模型要正确实现父子索引，表格要提供列数；选定根索引可让树视图从指定目录开始显示。

代码示例

```
auto* model = new QFileSystemModel(this);
model->setRootPath(rootPath); // 层级数据由模型管理
listView->setModel(model); // 一维项
tableView->setModel(model); // 行列数据
treeView->setModel(model); // 父子层级
treeView->setRootIndex(model->index(rootPath));
// 大量记录用 model/view 和 delegate，避免每行创建 QWidget
```

008 QNetworkAccessManager 的作用是什么?

答案　QNetworkAccessManager 是 Qt 网络请求的协调对象，常用来发起 HTTP 请求并创建对应的 QNetworkReply。请求以异步方式完成，调用 get() 后函数会很快返回，之后通过 finished、进度和错误信号处理响应，因此适合 GUI 程序保持事件循环响应。通常在所属线程复用一个 manager，而不是每次请求都创建一套。

manager、reply 受 QObject 线程亲和性和事件循环约束：在其所属线程使用并读取结果，不要在其他线程直接操作 reply。完成回调中先检查网络错误和 HTTP 状态，再读取数据并安排 deleteLater 清理。网络完成只代表传输结束，不代表服务端业务操作成功；还要检查响应内容、超时、重试策略和用户取消情况。

代码示例

```
auto* manager = new QNetworkAccessManager(this); // 在线程内复用
QNetworkReply* reply = manager->get(QNetworkRequest(QUrl(url)));
connect(reply,
        &QNetworkReply::finished,
        this,
        [reply]
        {
            if (reply->error() == QNetworkReply::NoError)
                consume(reply->readAll());
            else
                qWarning() << reply->errorString();
            reply->deleteLater(); // 读完再延迟销毁
        });
// manager/reply 由所属线程事件循环驱动，不在别的线程直接读取
```

009 QSettings 的作用是什么？如何使用?

答案　QSettings 按组织名和应用名，或显式指定的文件，保存键值配置；平台默认后端可能是注册表或 INI 等形式。它适合窗口尺寸、最近目录和普通偏好设置，可用分组组织路径，并在读取时提供默认值。配置存取位置由应用标识和平台决定，部署时应确认安装和用户权限。

写入配置不等于写入敏感信息安全库。普通设置可能以可读文本保存，口令、令牌等凭据应使用适当的系统凭据存储或加密方案。配置键变更时要设计迁移或兼容默认值；需要确认落盘时可调用 sync 并检查 status，但频繁同步会造成不必要 I/O。读取外部可修改配置时，也要验证值的范围和类型。

代码示例

```
QSettings settings;
settings.beginGroup(QStringLiteral("window"));
```

```
settings.setValue(QStringLiteral("size"), size());
settings.endGroup();
settings.sync(); // 写回后检查持久化状态
if (settings.status() != QSettings::NoError)
    reportSettingsError();
const QSize saved = settings.value(QStringLiteral("window/size"), QSize(800, 600)).toSize();
// 普通配置不存明文口令，升级时按版本迁移键值
```

010 QSqlDatabase、QSqlQuery、QSqlTableModel 的作用是什么?

答案　QSqlDatabase 表示数据库驱动、连接参数和连接本身；QSqlQuery 用来执行参数化 SQL 并读取结果；QSqlTableModel 把表数据映射到 Qt 模型视图，适合简单单表浏览和编辑。查询时应绑定参数而不是拼接用户输入，以避免 SQL 注入和转义错误。连接名要清楚管理，多个连接不要依赖含糊的默认连接。

数据库连接有线程归属：在哪个线程创建，就应在那个线程使用。后台线程需要建立自己的命名连接，不能把同一连接对象拿去并发共享。关闭并移除连接前，相关 query、model 和数据库句柄副本都应先销毁，否则 Qt 可能仍认为连接在使用。连接失败、SQL 执行失败和事务失败都要检查 lastError；界面模型修改后也需明确提交策略。

代码示例

```
const QString name = QUuid::createUuid().toString();
QSqlDatabase db = QSqlDatabase::addDatabase(QStringLiteral("QSQLITE"), name);
db.setDatabaseName(filePath);
if (!db.open())
    return report(db.lastError());
{
    QSqlQuery query(db);
    query.prepare(QStringLiteral("SELECT name FROM user WHERE id=?"));
    query.addBindValue(userId);
    if (!query.exec())
        report(query.lastError());
}
// 连接在哪个线程创建就在哪个线程使用
db.close();
db = {};
QSqlDatabase::removeDatabase(name); // 先销毁 query/model/副本
```

011 QSqlQueryModel、QSqlTableModel、QSqlRelationalTableModel 有什么区别?

答案　这三种 SQL 模型适合不同的数据访问方式。QSqlQueryModel 以一条自定义查询作为结果集，适合连接多表、聚合和报表展示，通常按只读模型使用；QSqlTableModel 围绕单张表提供选择、编辑和提交策略；QSqlRelationalTableModel 扩展单表模型，把外键值显示为关联表中的人类可读字段。

例如订单表中存着客户编号，关联模型可在视图显示客户姓名。但这种便利不等于能自动安全编辑任意多表事务；复杂业务更新应使用参数化查询或自定义模型，并明确事务边界。选模型前先确定是否只读、数据是否来自单表、是否需要外键关联，以及视图编辑如何提交。

代码示例

```
QSqlQueryModel reportModel(this);
reportModel.setQuery(QStringLiteral("SELECT department, count(*) FROM staff GROUP BY department"),
                     db);
// 自定义 JOIN/聚合通常用查询模型，只读展示
QSqlTableModel editable(this, db);
editable.setTable(QStringLiteral("employee"));
editable.select(); // 单表编辑使用 QSqlTableModel
QSqlRelationalTableModel related(this, db);
related.setTable(QStringLiteral("order")); // 外键可映射关联表显示
// 多表编辑需自定义提交/事务边界
```

012 QTcpSocket 和 QUdpSocket 的使用场景是什么?

答案　QTcpSocket 和 QUdpSocket 对应两种传输模型。TCP 建立连接并提供可靠、有序的字节流，适合登录、文件传输和需要完整顺序的交互；但它没有应用消息边界，应用仍要定义长度头或分隔符。UDP 发送一个个独立数据报，开销低且支持广播、组播，但网络可能丢失、重复或乱序，业务协议要自行容忍这些情况。

例如实时位置更新可容忍偶尔丢一条，UDP 可能合适；文件内容完整性要求高则常选 TCP。UDP 数据报过大可能触发 IP 分片，任一分片丢失都会使整个包不可用，设计时应控制载荷大小。二者的异步通知都由对象线程事件循环处理；读取 TCP 时要累计缓冲，读取 UDP 则保留每个数据报边界。

代码示例

```
QTcpSocket tcp;
tcp.connectToHost(host, port); // 可靠有序字节流
```

```
connect(&tcp, &QTcpSocket::readyRead, this, &Client::appendTcpBytes);
QUdpSocket udp;
udp.bind(QHostAddress::AnyIPv4, 0);
udp.writeDatagram(payload, serverAddress, serverPort); // 独立数据报
// TCP 需自行定义消息边界；UDP 要处理丢失、乱序与重复
// 两类 socket 都需在有事件循环的所属线程中使用
```

013 QTimer 的使用方法是什么?

答案　QTimer 是由事件循环驱动的定时通知器。创建后连接 timeout 信号，设置毫秒间隔并 start()，可周期执行轻量操作；设为 singleShot 或调用 singleShot() 可延迟执行一次，例如稍后合并界面刷新。它不会为每个计时器新建线程，回调在 QTimer 所属线程运行。

如果线程正忙或事件队列积压，超时处理会晚于计划时间，因此不适合硬实时控制。要测量真实经过时间，应使用 QElapsedTimer；定时器只用于安排“何时再检查或更新”，不能把两次回调间隔当作精确耗时。对象线程必须有事件循环，若在回调中执行耗时工作，后续定时器和其他事件也会延迟。

代码示例

```
QElapsedTimer elapsed;
elapsed.start(); // 测实际经过时间
QTimer timer(this);
timer.setInterval(500); // 事件循环驱动的周期回调
connect(&timer,
        &QTimer::timeout,
        this,
        [this]
        {
            refreshView();
        });
timer.start();
// 所属线程必须有事件循环；回调繁忙会延迟触发
// QTimer 不是硬实时计时器，不用 timeout 间隔测精确耗时
```

014 Qt 中有哪些实现定时器的方式?

答案　实现定时行为可选高层或低层 API。QTimer 用 timeout 信号表达超时，能方便设置周期、暂停和单次触发，通常是界面代码首选。QObject::startTimer() 返回一个 ID，对象在 timerEvent() 中收到事件后按 ID 区分多个计时器；QBasicTimer 是较轻的封装，仍需由对象处理 timerEvent。

这些方式都由对象所属线程的事件循环投递，不会自动创建计时线程；回调耗时、系统调度和事件队列拥堵都会带来延迟。简单提醒用 QTimer 最易读；对象内部管理少量低层计时器时可考虑 QBasicTimer 或 timer ID。停止定时器并在析构前清理，且不要用定时回调频率推断精确墙钟时间。

代码示例

```
QTimer highLevel(this); // timeout 信号和单次/周期 API
connect(&highLevel, &QTimer::timeout, this, &Worker::tick);
highLevel.setSingleShot(true);
highLevel.start(200);
const int timerId = startTimer(1000); // 低层 ID 由对象管理
void Worker::timerEvent(QTimerEvent* e)
{
    if (e->timerId() == timerId)
        doSmallStep();
    else
        QObject::timerEvent(e);
}
// 两种方式都依赖对象线程事件循环
```

015 Qt 的事件循环机制是什么?

答案　事件循环是 Qt 把异步事件变成对象回调的调度机制。应用调用 QApplication 或 QCoreApplication 的 exec() 后，循环持续接收窗口输入、排队槽、定时器和网络通知，并分发给目标对象。GUI 线程依靠它响应绘制和鼠标；需要定时器或队列槽的工作线程也必须运行自己的事件循环。

如果一个线程长时间运行不返回的函数，该线程的事件循环就暂时无法处理后续事件；没有事件循环时，排队调用和定时通知也不能按预期执行。同步 I/O 或死锁会让窗口看似假死。把长工作移到线程、将处理拆成小步骤，并在应用退出时有序停止循环，才能让事件驱动组件保持响应。

代码示例

```
int main(int argc, char** argv)
{
```

```
    QApplication app(argc, argv); // 创建 GUI 应用上下文
    MainWindow window;
    window.show();
    QTimer::singleShot(0, &window, &MainWindow::startAsyncWork);
    return app.exec(); // 网络/定时器/排队槽由主循环分发
}
// 工作线程需自己的 exec() 才能处理该线程的 queued slot 和 QTimer
// 长时间阻塞某线程会暂停该线程的事件处理
```

016 Qt 的模型 / 视图（Model / View）架构是什么?

答案　Model/View 把数据存储与界面呈现拆开。Model 提供行列、角色数据和修改通知；View 管理展示、选择和滚动；Delegate 负责绘制单元格并在需要时创建编辑器。视图通过 QModelIndex 查询模型，模型通过 dataChanged 和插入删除通知告诉视图发生了什么，因此大列表不必为每条数据创建完整控件。

一个模型可以同时连接列表和表格视图，二者观察同一份数据并以不同方式呈现。数据变更必须按模型 API 通知视图，否则选择状态、索引和画面会不一致。Delegate 不应把业务存储藏在绘制对象里，视图也不应绕开模型直接改数据。该结构适合数据集合；少量固定控件的表单用普通 Widgets 通常更简单。

代码示例

```
class TaskModel : public QAbstractListModel
{ /* 数据与 role API */
};
TaskModel model;
QListView list;
QTableView table;
list.setModel(&model);
table.setModel(&model); // 两个视图观察同一份数据
model.setData(model.index(0, 0), QStringLiteral("done"), Qt::EditRole);
// 模型发 dataChanged/插删通知后视图更新
// Delegate 管绘制与编辑，View 管选择和呈现
```

017 Qt 中如何实现 JSON 数据的解析和生成?

答案　解析 JSON 时先用 QJsonDocument::fromJson() 得到文档，并检查 QJsonParseError；再确认根节点是对象还是数组。字段读取后还应检查实际类型、缺省值和业务范围，例如年龄必须是非负整数，而不能只因 toInt() 能返回一个数就认为数据有效。生成时构造 QJsonObject/Array，再由 QJsonDocument 序列化成 QByteArray。

JSON 适合常见 HTTP 请求和中小文档，接口数据应做结构校验并返回有用错误。Compact 输出省去格式空白，Indented 更便于调试。文档解析会把数据装入内存；特别大的输入应限制字节数，或采用增量解析方案。JSON 中数字表示也有精度边界，长整数标识符可考虑字符串编码，避免客户端转换丢失精度。

代码示例

```
QJsonParseError error;
const QJsonDocument doc = QJsonDocument::fromJson(bytes, &error);
if (error.error != QJsonParseError::NoError || !doc.isObject())
    return report(error);
const QJsonObject object = doc.object();
if (!object.value("id").isDouble())
    return reportSchemaError();
QJsonObject response{{"ok", true}, {"id", object.value("id")}};
const QByteArray output = QJsonDocument(response).toJson(QJsonDocument::Compact);
// 大文档流式读取可用 QJsonStreamReader 等分段方案
```

018 Qt 中如何实现 WebSocket 通信?

答案　QWebSocket 客户端通过 open() 对 ws 或 wss 地址发起握手；握手成功后可发送文本或二进制消息，并连接接收、错误和断开信号。服务端用 QWebSocketServer 监听，再为接入的 socket 安装各自处理逻辑。WebSocket 在 HTTP 握手后提供双向消息通道，消息边界由协议处理，不需要像 TCP 裸流那样自己拼每个字节帧。

收发仍然依赖对象所属线程的事件循环。生产环境要校验证书和服务端身份、验证用户权限、限制消息大小，并为断线重连使用退避策略，避免故障时疯狂重连。TLS 加密并不等于身份认证和业务授权；收到消息后还要做格式、大小和权限校验。关闭时移除连接并清理 socket，不能将失效连接一直留在列表中。

代码示例

```
QWebSocket socket; // 对象需存活在事件循环线程
connect(&socket,
        &QWebSocket::connected,
        this,
        [&socket]
        {
```

```
            socket.sendTextMessage(QStringLiteral("hello"));
        });
connect(&socket, &QWebSocket::textMessageReceived, this, &Client::onMessage);
connect(&socket, &QWebSocket::errorOccurred, this, &Client::onError);
connect(&socket, &QWebSocket::disconnected, this, &Client::scheduleReconnect);
socket.open(QUrl(QStringLiteral("wss://example.invalid/socket")));
// 校验证书、限制消息大小，并为断线重连设置退避
```

019 Qt 中如何实现 XML 数据的解析？

答案　QXmlStreamReader 按顺序读取 XML token，适合边读边处理较大文档，不必一次把整棵树都放进内存。遇到 startElement 时可读取标签名和属性，文本内容应结合当前元素上下文处理；结束时检查 hasError()，并报告错误位置。QXmlStreamWriter 负责按结构输出 XML，并为文本节点正确转义特殊字符。

如果业务需要随机访问任意节点、反复查询树结构，可以使用 DOM，但整份文档会占用更多内存。Qt 6 移除了旧 SAX API，项目需依照目标 Qt 版本选择接口。解析不可信 XML 时要控制文档大小和深度，并依据安全要求禁用或限制外部实体；XML 文本即使经过格式解析，也仍需按业务规则验证。

代码示例

```
QXmlStreamReader xml(&file); // 前向解析，适合大文件
while (!xml.atEnd())
{
    xml.readNext();
    if (xml.isStartElement() && xml.name() == u"item")
        consume(xml.attributes().value(u"id").toString());
}
if (xml.hasError())
    report(xml.errorString(), xml.lineNumber(), xml.columnNumber());
QXmlStreamWriter writer(&out);
writer.writeStartDocument();
writer.writeTextElement(QStringLiteral("name"), userText); // 自动转义
writer.writeEndDocument(); // DOM 仅在需要随机访问整棵树时载入
```

020 Qt 中如何读写文本文件和二进制文件？

答案　文本文件和二进制文件要按内容语义分别处理。文本通常由 QFile 打开，再交给 QTextStream，并明确 UTF-8 等编码和换行约定；原始二进制可直接用 QFile::read/write 操作 QByteArray。若二进制中要保存多种类型，可用 QDataStream，但读取端和写入端必须约定字段顺序、字节序、格式版本及兼容规则。

文件操作要检查打开和读写错误。持久化结构应包含自己的格式版本号，不能只依赖当前 Qt 的默认 stream version；解析不可信文件时要限制长度，避免恶意长度字段触发超大分配。大文件可分块读取，避免 readAll() 一次占用全部内存。文本也可能包含二进制不可用的换行转换，因此要清楚选择文本模式或原始字节模式。

代码示例

```
QFile binary(path);
if (!binary.open(QIODevice::WriteOnly))
    return report(binary.errorString());
QDataStream out(&binary);
out.setVersion(QDataStream::Qt_6_0);
out << quint32(formatVersion) << payload; // 结构化二进制
QFile textFile(textPath);
if (!textFile.open(QIODevice::WriteOnly))
    return;
QTextStream text(&textFile);
text.setEncoding(QStringConverter::Utf8);
text << QString::fromUtf8(message); // 编码与字节序显式约定
// 读取外部文件要验证长度和流状态后再使用数据
```

021 Qt 中如何连接数据库？

答案　建立 Qt SQL 连接时，先用 addDatabase() 指定驱动和唯一连接名，再设置 SQLite 文件名或服务器参数并调用 open()。连接失败应检查 lastError，而不是继续运行查询。之后创建的 QSqlQuery 和模型都依赖该连接，连接名应由应用显式管理，避免不同模块意外使用默认连接。

清理顺序很关键：先销毁所有 query、model 和 QSqlDatabase 副本，关闭数据库，再 removeDatabase。若仍有对象引用连接，删除时会有告警，继续使用也可能造成错误。连接只能在创建它的线程中使用；后台任务要创建自己的连接。部署时还需包含相应 SQL 驱动插件，例如 SQLite 驱动。

代码示例

```
const QString connection = QUuid::createUuid().toString();
{
```

```
    QSqlDatabase db = QSqlDatabase::addDatabase(QStringLiteral("QSQLITE"), connection);
    db.setDatabaseName(filePath);
    if (!db.open())
        return report(db.lastError());
    {
        QSqlQuery query(db);
        runQueries(query);
    } // query 先离开作用域
    db.close();
} // 清除所有 QSqlDatabase 副本、model 后再 remove
QSqlDatabase::removeDatabase(connection); // 否则仍引用连接会告警
```

022 Qt 中有哪些网络编程相关的类?

答案　Qt 网络和数据 API 按协议分层选择。QTcpServer 与 QTcpSocket 适合 TCP 监听和连接，QUdpSocket 负责 UDP 数据报；HTTP 常由 QNetworkAccessManager 创建请求和 reply，WebSocket 则使用 QWebSocket 或 QWebSocketServer。它们是不同协议抽象，不能因为都“收发数据”就互换语义。

网络对象一般依赖对象线程事件循环，异步完成信号应在正确线程处理。选择模块还要检查构建依赖和目标部署内容，例如 Qt Network、WebSockets 模块和 SSL 后端。QFile/QSettings 属于本地文件与配置，SQL 模块处理数据库，不是网络协议类。先确认可靠性、消息边界、连接模型和部署环境，再选最低层足够表达需求的 API。

代码示例

```
QTcpSocket tcp;
tcp.connectToHost(host, port); // 可靠字节流，应用层自行分帧
QUdpSocket udp;
udp.writeDatagram(packet, peer); // 数据报可能丢失
manager.get(QNetworkRequest(url)); // HTTP客户端复用管理器
```

023 exec() 函数的作用是什么?

答案　exec() 的含义要看调用对象。QCoreApplication::exec() 启动应用主事件循环，持续分发输入、排队调用和定时器，直到应用退出并返回状态码。QThread::exec() 为该工作线程启动事件循环，使其中 QObject 的 queued slot 和 QTimer 能工作。两者都不是“启动新线程”的通用函数。

QDialog::exec() 则启动嵌套事件循环并同步等待对话框结果，常用于短模态交互，但在等待期间其他事件仍可能执行，带来重入风险。异步界面通常用 show()/open() 并监听 finished。不要把 processEvents 当作长任务的替代方案；应该移走耗时工作或让状态机异步推进。

代码示例

```
QCoreApplication app(argc, argv);
QTimer::singleShot(0, &worker, &Worker::processChunk); // 让循环自然回到事件队列
return app.exec();
// processEvents() 会同步重入当前线程所有已排队事件
// 不能在半初始化对象、发模型通知或持锁时调用
// 长任务应异步拆分或交给 worker，而非插入嵌套事件泵
```

024 startTimer() 和 QTimer 有什么区别?

答案　QTimer 是较高层接口，用 timeout 信号表达超时，支持周期、单次、启停和间隔设置，代码通常更易组合。startTimer() 是 QObject 的低层接口，返回一个 timer ID，收到 timerEvent 时由对象比较 ID 并决定处理哪个定时器；多个 ID 的启动与停止要由调用方自己维护。

它们都由对象所属线程的事件循环驱动，不会开独立计时线程。回调何时执行受线程忙闲与系统调度影响，因此都不是硬实时计时器。常规界面刷新、延迟搜索用 QTimer；确实需要在对象事件回调中统一管理少量定时器时可用低层 ID。无论何种方式，都要停止不再需要的计时器。

代码示例

```
class TimerOwner : public QObject
{
    void startLowLevel()
    {
        timerId_ = startTimer(250);
    }
    void timerEvent(QTimerEvent* event) override
    {
        if (event->timerId() == timerId_)
            poll();
        else
            QObject::timerEvent(event);
    }
```

```
    int timerId_ = 0;
};
QTimer high(this);
connect(&high, &QTimer::timeout, this, &TimerOwner::poll);
// 高层信号 API 更清楚；两者精度都受事件循环调度
```

025 如何使用 QTcpServer 实现服务器?

答案　实现 TCP 服务端可用 QTcpServer::listen() 绑定地址和端口，newConnection 通知到达后调用 nextPendingConnection() 取得 QTcpSocket。每个客户端都需要独立的接收缓冲和断开清理逻辑。服务器要同时处理监听失败、客户端错误、写缓冲积压和最大连接数，不能只实现 happy path。

TCP 是可靠有序的字节流，readyRead 一次可能只有半条消息，也可能包含多条消息；应按长度前缀或分隔协议解析，并限制未完成消息的缓冲上限。服务端必须进行身份验证和输入校验，避免资源被慢速或恶意客户端耗尽。socket 和 server 应在事件循环所在的线程使用，客户端断开后延迟删除并清除对应状态。

代码示例

```
auto* server = new QTcpServer(this);
connect(server,
        &QTcpServer::newConnection,
        this,
        [server]
        {
            while (server->hasPendingConnections())
            {
                auto* client = server->nextPendingConnection(); // server 暂时拥有
                connect(client,
                        &QTcpSocket::readyRead,
                        this,
                        [client]
                        {
                            appendAndParse(client, client->readAll()); // TCP 字节流
                        });
                connect(client, &QTcpSocket::disconnected, client, &QObject::deleteLater);
            }
        });
if (!server->listen(QHostAddress::Any, port))
    report(server->errorString());
// 限制每连接缓冲与消息长度
```

026 如何使用 Qt 实现 HTTP 请求?

答案　HTTP 请求通常通过复用的 QNetworkAccessManager 异步发出。创建 QNetworkRequest 设置 URL 和必要头部，调用 get 或 post 得到 QNetworkReply；完成后读取网络错误、HTTP 状态码、响应头与正文，再安排 reply 删除。异步方式不会把 GUI 线程停在网络等待上，进度信号可更新进度条或取消状态。

HTTPS 需要正确验证证书，不能为了消除证书错误而关闭校验。网络传输成功和业务成功不是一回事：HTTP 4xx/5xx 仍可能使 reply 的网络错误为 NoError，所以还要检查状态码和响应结构。设置超时、取消和重试策略，避免无限等待或重复提交副作用；请求参数应正确编码，凭据也不应写入日志。

代码示例

```
auto* manager = new QNetworkAccessManager(this);
auto* reply = manager->post(QNetworkRequest(QUrl(url)), body);
connect(reply,
        &QNetworkReply::finished,
        this,
        [reply]
        {
            if (reply->error() != QNetworkReply::NoError)
                report(reply->errorString());
            else if (reply->attribute(QNetworkRequest::HttpStatusCodeAttribute).toInt() >= 400)
                reportHttpStatus(reply->attribute(QNetworkRequest::HttpStatusCodeAttribute));
            else
                consume(reply->readAll());
            reply->deleteLater();
        });
// GUI 线程不要 waitFor...; HTTPS 必须验证证书
```

027 如何使用 Qt 操作 SQLite 数据库?

答案　使用 SQLite 前确认应用部署了 Qt SQL 模块和 QSQLITE 驱动插件。为每个逻辑连接分配唯一名称，设置数据库文件路径并检查 open()。执行 SQL 时通过 QSqlQuery 的 prepare() 与 bindValue() 绑定数据，让驱动负责参数处理，不要把用户输入直接拼接进 SQL 字符串。

查询和连接都应检查 exec、next、lastError 等结果，并在连接所属线程操作。多条相关写操作可用事务。关闭连接前先释放查询、模型和句柄副本，再移除连接名；常见告警通常说明仍有连接引用。数据库路径权限、升级迁移和备份也属于应用职责，SQLite 驱动可用并不代表数据库文件一定可写。

代码示例

```
const QString connection = QUuid::createUuid().toString();
{
    QSqlDatabase db = QSqlDatabase::addDatabase(QStringLiteral("QSQLITE"), connection);
    db.setDatabaseName(filePath);
    if (!db.open())
        return report(db.lastError());
    QSqlQuery query(db);
    query.prepare(QStringLiteral("INSERT INTO user(name) VALUES(?)"));
    query.addBindValue(userName);
    if (!query.exec())
        report(query.lastError());
} // 先析构 query/model/db 副本
QSqlDatabase::removeDatabase(connection); // 部署也要包含 QSQLITE 驱动
```

028 如何处理网络数据的粘包和拆包?

答案　TCP 不保留消息边界，所以 readyRead 一次既可能读到半个包，也可能一次读到连续多个包。接收端将新字节追加到每连接的缓冲区，再按协议循环解析完整帧。长度前缀协议先等够固定头，按明确字节序解出 payload 长度，拒绝超过上限的值，再等待完整正文。

正文不完整时保留缓冲，等下一次数据到来；处理一帧后移除其字节并继续检查后续帧。长度计算要避免整数溢出，同时限制总缓冲大小和连接数，防止恶意客户端宣称超大包而耗尽内存。分隔符协议也要处理转义和编码边界。任何情况下都不要把“一次 read”当成“一条消息”。

代码示例

```
buffer.append(socket->readAll()); // readyRead 一次可能含半帧或多帧
while (buffer.size() >= 4)
{
    const auto* head = reinterpret_cast<const uchar*>(buffer.constData());
    const quint32 length = qFromBigEndian<quint32>(head); // 固定大端长度
    if (length > maxPayload || length > std::numeric_limits<quint32>::max() - 4u)
    {
        socket->abort();
        return;
    }
    if (buffer.size() < 4 + length)
        break; // 半帧留待下次数据
    handle(buffer.mid(4, length));
    buffer.remove(0, 4 + length);
}
// 解析后移除完整包；长度上限避免溢出和内存耗尽
```

029 如何实现数据库的事务处理?

答案　事务把多个数据库操作组合成一个原子工作单元：成功时整体提交，失败时回滚已执行的变更。先确认驱动支持事务并检查 transaction() 返回值，再逐条执行查询、检查每步错误，最后检查 commit() 是否成功。事务缩小了中途失败留下部分数据的风险，但只涵盖数据库连接中的操作。

事务不能撤销已经发出的邮件、调用的远程 API 或写入另一系统的文件；这类副作用需通过事务外的可靠流程、outbox 或补偿处理。事务也不自动解决并发业务冲突，隔离级别和唯一约束要结合数据库语义设计。事务应尽量短，避免持锁等待用户输入或网络响应。提交失败同样要记录错误并决定是否回滚或重试。

代码示例

```
if (!db.transaction())
    return report(db.lastError());
bool ok = false;
QSqlError queryError;
{
    QSqlQuery q(db);
```

```
    q.prepare(sql);
    q.addBindValue(value);
    ok = q.exec();
    if (!ok)
        queryError = q.lastError();
}
if (!ok)
{
    db.rollback();
    return report(queryError);
} // query 先销毁
if (!db.commit())
{
    const auto error = db.lastError();
    db.rollback();
    report(error);
}
// 事务尽量短；外部服务副作用不由数据库回滚
```

030 如何自定义 Model?

答案　自定义列表模型通常继承 QAbstractListModel，实现 rowCount() 和 data()；表格还需 columnCount()，树模型则要正确描述 index() 和 parent()。若支持编辑，flags() 声明可编辑项，setData() 修改底层状态并发出 dataChanged。Model/View 的索引由模型解释，调用者不应把行号长期当作稳定对象身份。

结构变化必须用 beginInsertRows/endInsertRows、beginRemoveRows/endRemoveRows 或 reset 等成对 API 包起来，让视图更新内部索引和选择。传入非法 parent、负数行或越界列时应安全返回。角色名应稳定，编辑验证失败要返回 false 并保持原状态。用模型测试覆盖边界行、通知范围和排序筛选行为，比只检查显示文本更可靠。

代码示例

```
bool Model::insertRows(int row, int count, const QModelIndex& parent)
{
    if (parent.isValid() || count <= 0 || row < 0 || row > rows_.size())
        return false;
    beginInsertRows(parent, row, row + count - 1); // 先通知视图
    rows_.insert(row, count, Item{});
    endInsertRows(); // 修改后闭合通知
    return true;
}
QVariant Model::data(const QModelIndex& i, int role) const
{ /* bounds + role */
}
// 修改已有值后发 dataChanged；移除也用 begin/endRemoveRows
```

14.5 高级特性与 QML

001 Loader、Repeater、ListView 的作用是什么?

答案　Loader、Repeater 和 ListView 都能从数据或组件动态创建界面，但创建策略不同。Loader 按 source 或 sourceComponent 加载一棵组件子树，适合低频页面按需创建；切换 source 时旧对象可能销毁，因此不能把业务数据只存于该页面对象。Repeater 会为模型中的每一项创建 delegate，适合数量有限的固定项。

ListView 面向长列表，主要创建可见区域附近的 delegate，并可复用它们，滚动时减少对象数量。因为 delegate 可能销毁或重用，持久状态应放在 model 中，而不是某个 delegate 的局部属性。访问 Loader 的 item 前检查 status 和 item 是否有效；设计时还要考虑模型变化、异步加载和页面销毁时机。

代码示例

```
// QML: Loader 仅在需要时创建低频页面
Loader
{
id:
    settings;
active:
    settingsVisible;
source:
    "Settings.qml"
}
onLoaded : settings.item.resetView() onStatusChanged
```

```
    : if (status == = Loader.Error) console.warn(errorString())
      // Repeater 会为所有项创建 delegate，适合小集合
      Repeater
{
model:
    5;
delegate:
    Text
    {
    text:
        index
    }
}
// 长列表用 ListView 的可视 delegate；model 状态不能放在会销毁的 delegate 内
```

002 QLibrary 的作用是什么?

答案　QLibrary 在程序运行期间打开普通动态库，并通过 resolve() 查找已导出的符号地址。它适用于加载明确约定的 C 风格函数接口；调用方必须准确匹配函数参数、返回值、调用约定和 ABI，否则即使成功加载也可能崩溃。库文件的名称、依赖路径和平台扩展名也要按部署环境管理。

调用 resolve 前后检查 load() 和 errorString()，并保证函数指针只在库保持加载期间使用。若需要枚举插件元数据、验证接口类型并创建 QObject 实例，应使用 QPluginLoader，而不是把 QLibrary 当作插件框架。卸载前确认没有仍在执行的函数和存活实例依赖该库；普通动态库更新也涉及兼容性与安全信任边界。

代码示例

```
QLibrary library(fileName);
if (!library.load())
    return report(library.errorString());
using CreateFn = QObject* (*)(); // 导出函数 ABI 必须一致
auto create = reinterpret_cast<CreateFn>(library.resolve("createPlugin"));
if (!create)
    return report(library.errorString());
QObject* instance = create(); // 调用期间库必须仍加载
use(instance);
destroyWithLibraryContract(instance);
library.unload(); // 所有函数指针/对象失效后才卸载
```

003 QML 中如何实现动画效果?

答案　QML 动画通过随时间改变属性值来表现状态变化。NumberAnimation 适合数值属性，ColorAnimation 改变颜色，PropertyAnimation 可覆盖更多属性类型；Behavior 可以在属性被设置时自动过渡，State 和 Transition 则适合在界面状态之间定义动画。动画持续时间、缓动曲线决定观感，不改变属性的业务含义。

同一属性应由清楚的动画或状态流程负责；另一段代码持续写入该属性时，动画会被打断或结果难预测。循环动画要有明确停止条件，页面隐藏或对象销毁时也要考虑是否继续运行。动画不能补救主线程卡顿，耗时处理应在后台完成，再把最终状态交给 QML 展示。

代码示例

```
// QML
Rectangle
{
id:
    panel opacity : expanded ? 1.0 : 0.0 Behavior on opacity
    {
        NumberAnimation
        {
        duration:
            200;
            easing.type : Easing.OutCubic
        }
    }
}
// 状态切换也可由 states/Transition 驱动
// 多个动画不要同时命令式写同一属性；隐藏组件时停止循环动画
```

004 QML 中的信号和槽如何使用?

答案　QML 组件可用 signal 声明自己的信号，并通过 onSignalName 处理组件内部通知；Connections 对象可以监听运行时变化的目标。C++ QObject 信号也能连接 QML 方法，QML 信号可由 C++ 的 connect 接收。两端参数必须能通过 Qt 元对象和 QML 类型系统传递，接口名称与参数类型应保持稳定。

信号适合表达已经发生的事件，如用户选择了一个编号；它不应被当作同步返回值接口。处理函数若在 GUI 线程执行长时间计算，会冻结动画和输入，应把工作异步移出后再发结果信号。连接动态对象时要处理目标变成 null 或销毁的情况，也不要在多处重复连接导致一个事件执行多次。

代码示例

```
// QML
Item{
    signal valueChosen(int value) Button{onClicked : valueChosen(7)}
    onValueChosen : console.log("chosen", value)
} // C++ 接 QObject 信号时用 typed connect
connect(&backend, &Backend::finished, this, &Window::showResult);
// QML 参数需可被元类型系统识别；重活不能同步阻塞 UI
```

005 QML 中的属性绑定是什么?

答案　属性绑定是 QML 根据表达式建立的依赖关系。比如宽度绑定到父项的一半，父项尺寸变化时引擎会重新计算表达式并更新宽度；绑定适合表达界面几何和状态间的持续关系。表达式应主要做纯计算，避免读取数据时顺便修改状态，否则依赖重算次数会影响行为。

对绑定属性直接做命令式赋值通常会移除原绑定，之后父项变化也不会再自动更新。若确实要重新建立动态表达式，可使用 Qt.binding()；很多场景更适合修改源状态，让原绑定自然计算结果。工具 qmllint 可以发现部分绑定断裂和未限定引用。复杂表达式则应拆分成有名字的属性，便于调试依赖。

代码示例

```
QProperty<qreal> parent_width, child_width;
child_width.setBinding(
    [&]
    {
        return parent_width.value() * 0.5;
    }); // 依赖变化时自动更新
// 直接setValue会替换绑定；更新数据源通常更清晰
```

006 QML 和 C++ 如何交互?

答案　QML 与 C++ 常通过 QObject 元对象边界交互。C++ 对象可向 QML 暴露 Q_PROPERTY、信号、槽和 Q_INVOKABLE 方法；类型可用 QML_ELEMENT 或注册函数注册，也可以把一个由 C++ 持有的实例交给根对象属性。这样 QML 负责呈现，C++ 负责数据或系统能力，接口仍应设计成稳定、少量的业务操作。

对象的生命周期和所有权需要显式约定。某些由 QML 创建或返回的 QObject 可能按引擎规则转移所有权，不能假设所有指针都由 C++ 父对象回收。上下文属性虽然方便，但隐藏依赖且不利于静态工具分析；优先考虑类型注册或根对象属性。跨线程访问需通过消息传递，不能从 QML 同步调用长任务。

代码示例

```
class Backend : public QObject
{
    Q_OBJECT
    Q_PROPERTY(QString title READ title NOTIFY titleChanged)
  public:
    Q_INVOKABLE void refresh()
    { /* 异步触发工作 */
    }
  signals:
    void titleChanged();
};
qmlRegisterType<Backend>("App", 1, 0, "Backend"); // 类型可见
// QML 引擎使用对象时确认 QObject owner 与销毁顺序
```

007 Q_INVOKABLE 宏的作用是什么?

答案　Q_INVOKABLE 把 QObject 的普通成员函数登记到元对象信息中，使其能够通过 QMetaObject::invokeMethod 按元对象规则调用，也能从 QML 调用。该类必须参与 Qt 元对象系统，通常继承 QObject 并包含 Q_OBJECT；参数和返回值也须是元对象系统可以处理的类型。

它提供调用能力，不自动使函数异步、线程安全或对所有输入可靠。适合暴露简短命令或查询；耗时工作应启动后台任务并用信号报告完成，否则调用线程上的 QML 界面会停顿。不要把内部实现细节都标成可调用接口，最好校验输入并保持方法名和参数语义稳定。

代码示例

```
class Backend : public QObject
{
    Q_OBJECT
  public:
    Q_INVOKABLE QString greeting(const QString& name) const
    {
        return tr("Hello, %1").arg(name);
    }
};
QMetaObject::invokeMethod(&backend, "refresh", Qt::QueuedConnection);
// Q_INVOKABLE 写入元对象；同步耗时函数会卡住调用线程
```

008 Q_PROPERTY 宏的作用是什么?

答案　Q_PROPERTY 在 QObject 子类中声明一个可通过元对象访问的属性，包括属性类型、读取函数，以及可选的写入、通知和绑定相关信息。QML、Designer 和反射代码可按属性名读写它，而不是直接访问 C++ 私有成员。属性定义只是接口描述，真正状态仍由类实现负责维护。

如果属性可变，应让 setter 检查新旧值并在实际变化时发 NOTIFY 信号，否则依赖该属性的 QML 绑定不会刷新或会收到无用通知。只读属性可只提供 READ；读写函数和通知必须对应同一份状态。不要把每个内部字段都曝光，属性也受类型和线程约束，元对象不会自动加锁。

代码示例

```
class Backend : public QObject
{
    Q_OBJECT
    Q_PROPERTY(QString title READ title WRITE setTitle NOTIFY titleChanged)
  public:
    QString title() const
    {
        return title_;
    }
    void setTitle(const QString& v)
    {
        if (title_ == v)
            return;
        title_ = v;
        emit titleChanged();
    }
  signals:
    void titleChanged();

  private:
    QString title_;
};
// NOTIFY 触发 QML 绑定更新；属性读写必须共享同一状态
```

009 Qt Widgets 与 Qt Quick/QML 如何选择?

答案　Qt Widgets 和 Qt Quick/QML 是两套界面开发方式。Widgets 提供成熟的桌面控件、布局和原生风格，适合表单、传统工具以及已有 QWidget 项目；Qt Quick 用 QML 描述界面树、绑定和动画，更适合触控、动态视觉或需要灵活设计的产品。团队技能和目标平台同样影响维护成本。

二者可以在一定场景混用，但跨框架容器、事件焦点、样式、线程和部署会增加复杂度，不应只为单个动画引入整套技术栈。C++ 可为两类界面提供模型和业务逻辑。选型时先做关键交互原型，检查控件可用性、无障碍、性能和部署模块，再根据实际需求决定，不要假设某一套对所有平台都更快。

代码示例

```
QMainWindow desktop; // 传统桌面表单常用Widgets和Layout
QQmlApplicationEngine touch_ui; // 触控、动画界面可选Qt Quick/QML
// 共享业务逻辑放入C++模型，通过明确属性/信号与界面通信
```

010 Qt 中如何实现动态库的加载?

答案　运行时加载普通动态库可以用 QLibrary 加载并解析导出符号；Qt 插件则通过固定接口和元数据发现、验证及创建对象。插件实现通常声明接口版本，用 Q_DECLARE_INTERFACE 描述接口，插件 QObject 类带 Q_PLUGIN_METADATA；宿主用 QPluginLoader 加载实例，再通过 qobject_cast 检查它是否实现预期接口。

插件必须与宿主使用兼容的 ABI、Qt 和编译器配置，部署时还要带齐依赖库。加载成功不代表可信或安全，插件来自不可信来源时它可以在进程内执行代码。插件对象销毁前不能卸载库，因为虚函数和析构代码仍位于库中；版本不匹配、缺失元数据和加载失败都要给出诊断。

代码示例

```
// 普通 C ABI 用 QLibrary；Qt 插件更适合 QPluginLoader
QPluginLoader loader(pluginPath);
QObject* object = loader.instance();
if (!object)
    return report(loader.errorString());
auto* plugin = qobject_cast<MyInterface*>(object);
if (!plugin)
    return report(QStringLiteral("interface mismatch"));
plugin->start();
// 销毁插件对象及其调用者后才能卸载库；宿主/插件 ABI 必须匹配
```

011 Qt 中如何实现属性系统?

答案　QObject 的静态属性由 Q_PROPERTY 在类定义中声明，moc 将其纳入元对象信息，可通过 property()/setProperty() 按名称访问。运行时也能为对象附加动态属性，无需修改类源码；例如给控件设置 busy=true，再由 QSS 属性选择器改变外观。property() 返回 QVariant，因此读取动态属性时要检查值是否存在和类型是否符合预期。

动态属性没有专门的强类型读写函数，也不会自动给每项生成 NOTIFY 信号。变化时对象可收到 QDynamicPropertyChangeEvent，但依赖者需主动处理。它适合临时标记和样式提示，不适合作为核心业务状态存储；静态属性更利于文档、工具和绑定。属性设置仍需在合适线程中进行，反射机制不会自动同步。

代码示例

```
// 静态属性在 QObject 子类中用 Q_PROPERTY + moc 声明
Q_PROPERTY(int count READ count WRITE setCount NOTIFY countChanged)
object->setProperty("count", 3); // 走静态属性的 WRITE
object->setProperty("busy", true); // 未声明时添加动态属性
const QVariant busy = object->property("busy");
// 动态属性无强类型访问器和专属 NOTIFY
// 常驻业务状态优先声明静态属性
```

012 Qt 的反射机制是如何实现的?

答案　Qt 元对象系统是在构建阶段由 moc 为参与系统的 QObject 类生成元信息。它保存类名、信号槽、属性和可调用方法等信息，让 Qt 能按类型分发信号、按名称访问属性和进行受限运行时调用。项目构建必须正确启用 AUTOMOC 或相应 qmake 流程，否则可能出现缺少 vtable 或元对象符号的链接错误。

这不是对任意 C++ 程序的完整反射：它不会自动列出所有普通成员、模板实例或局部变量。需要通过 Qt 元对象访问的类通常要继承 QObject 并有 Q_OBJECT。系统仅覆盖 Qt 声明的元信息，普通业务元数据仍需显式设计。理解这一边界有助于判断何时用属性、信号槽，何时直接调用普通 C++ 接口。

代码示例

```
class Device : public QObject
{
    Q_OBJECT
    Q_PROPERTY(QString name READ name)
  public slots:
    void start()
    { /* ... */
    }
};
const QMetaObject* meta = device->metaObject(); // moc 生成元数据
int propertyIndex = meta->indexOfProperty("name");
int methodIndex = meta->indexOfMethod("start()");
QMetaObject::invokeMethod(device, "start", Qt::QueuedConnection);
// 这套反射不会自动枚举任意 C++ 成员；构建启用 AUTOMOC
```

013 Qt 的对象序列化如何实现?

答案　Qt 不会自动把任意 QObject 实例完整序列化成文件。应用应明确选择需要保存的数据字段，再用 QDataStream 重载运算符保存二进制，或转换成 QVariantMap/QJsonObject 保存结构化文本。保存对象指针、线程状态、信号连接或完整父子对象树都不是简单写几个属性就能完成的。

持久化格式需要版本号、字段默认值和迁移逻辑，读取时要处理缺字段、未知字段和损坏内容。网络传输与本地持久化也要考虑信任边界，不能直接信任反序列化结果。只保存稳定的业务值而不保存临时控件状态，通常更易兼容；反序列化后应重新构造 QObject，并按正常流程恢复关联。

代码示例

```
QJsonObject record{{"version", 2}, {"name", object.name()}};
const QByteArray bytes = QJsonDocument(record).toJson(QJsonDocument::Compact);
QFile file(path);
if (!file.open(QIODevice::WriteOnly))
    return report(file.errorString());
if (file.write(bytes) != bytes.size())
    return report(file.errorString());
QJsonParseError error;
const auto doc = QJsonDocument::fromJson(file.readAll(), &error);
if (error.error != QJsonParseError::NoError)
    return report(error.errorString());
restoreFields(doc.object()); // QObject 指针树/连接不会由属性自动恢复
// 长文档和兼容格式需限制大小并显式迁移 version
```

014 Qt 的插件机制是什么？如何实现插件？

答案　Qt 插件由宿主和扩展模块共享一份版本化接口。接口声明纯虚方法并通过 Q_DECLARE_INTERFACE 注册标识；插件实现类继承 QObject 和接口，使用 Q_PLUGIN_METADATA 导出元数据。宿主用 QPluginLoader 加载库并取得 QObject，再用 qobject_cast 验证接口，失败时报告库错误和接口不匹配。

插件与宿主必须在 Qt、编译器、架构和 ABI 上兼容，且部署依赖齐全。插件加载等同于在宿主进程执行本机代码，不应把不可信插件当作安全沙箱。调用接口期间库必须保持加载；应先停止异步工作和销毁插件对象，再 unload。接口变更要有版本策略，避免新宿主错误调用旧插件。

代码示例

```
QPluginLoader loader(pluginPath);
QObject* root = loader.instance(); // loader 持有库和根对象
if (!root)
    return report(loader.errorString());
auto* iface = qobject_cast<MyInterface*>(root); // 检查接口
if (!iface)
    return report(QStringLiteral("unsupported plugin"));
iface->initialize();
connect(this, &Host::closing, root, &Plugin::stop);
// 停止调用者并销毁对象后才 unload；检查版本和 ABI
```

015 动态属性（Dynamic Property）是什么？

答案　动态属性是在运行时给 QObject 添加的名称和值，不必预先修改类定义；通过 setProperty() 设置、property() 读取。它常被用作临时标记、自动化测试标签或 QSS 选择器输入，方便在既有控件上附加简单状态。值通过 QVariant 承载，因此应约定属性名和类型，读取时检查转换结果。

动态属性不等于完整的业务属性：它没有强类型访问函数和自动 NOTIFY 信号，修改时会产生动态属性变化事件，但属性消费者需自行处理。核心数据更适合 Q_PROPERTY，使接口可以检查、绑定和文档化。设置属性后，QSS 可能需要重新 polish 才应用新规则；动态属性也没有自动提供跨线程安全。

代码示例

```
widget->setProperty("busy", true); // 运行时附加属性
const QVariant value = widget->property("busy");
if (!value.isValid())
    return; // 检查属性是否存在
applyBusyStyle(value.toBool());
connect(widget, &QObject::destroyed, this, &Controller::clearState);
// 动态属性变化还会产生 QDynamicPropertyChangeEvent
// 常驻业务状态改用有类型访问器的 Q_PROPERTY
```

14.6 工程实践

001 .pro 文件的作用是什么？常用配置有哪些？

答案　.pro 文件是 qmake 的项目描述，告诉构建工具要生成应用、库还是子项目，以及源文件、模块、宏和链接依赖。常见变量包括 TEMPLATE、QT、SOURCES、HEADERS、CONFIG、INCLUDEPATH 和 LIBS。比如使用网络 API 时需加入 Network 模块；添加头文件不会自动解决缺失的库链接或运行时插件问题。

构建配置应与实际 Qt 安装和目标平台一致。Qt 6 仍能构建已有 qmake 项目，但新项目也常用 CMake，团队应按工具链和依赖生态选择。不同配置项的效果受 Qt 版本和平台影响，不要盲目复制旧项目的静态链接或编译器标志。构建成功后还需检查运行时所需插件、资源和共享库部署。

代码示例

```
TEMPLATE = app QT += widgets network sql CONFIG += c++ 20 SOURCES += main.cpp worker.cpp HEADERS +=
    worker.h FORMS += mainwindow.ui
#qmake 从.pro 生成构建文件；CMake 项目改用 Qt6::Widgets 等 targets
```

002 C++11 / 14 / 17 / 20 的新特性在 Qt 中如何应用?

答案　现代 C++ 语言特性可以和 Qt API 配合使用，但可用范围取决于编译器、项目标准设置和 Qt 版本。C++11 lambda 常让 connect 的短回调更自然，移动语义和智能指针帮助表达所有权；较新标准中的 optional、结构化绑定等也能用于业务代码。语言标准功能与 Qt 库功能是两回事，开启 C++20 不会自动让 Qt 对象支持任意协程调用。
在构建配置中明确要求项目的 C++ 标准，并确认所有目标平台的编译器都支持。QObject 父子树、线程亲和性和信号槽规则仍按 Qt 约定工作，不能被 STL 智能指针或新语法绕过。复杂 C++ 特性若降低接口可读性或跨编译器兼容性，应谨慎使用；选择能由团队和部署环境稳定支持的标准。

代码示例

```
connect(button,
        &QPushButton::clicked,
        this,
        [this, id = record.id]
        {
            saveRecord(id); // lambda 按值捕获异步所需数据
        });
std::optional<Result> result = findResult(id); // C++17 接口可表达缺失值
if (result)
    show(*result);
// CXX_STANDARD 由项目工具链显式设置；语言标准不等于 Qt API 版本
```

003 Lambda 表达式在 Qt 信号槽中如何使用?

答案　Lambda 可以把短小回调直接写在 connect() 附近，减少为了一个简单动作新增槽函数的负担。新式连接通常写信号、接收对象上下文和 lambda；上下文对象销毁时，连接会断开。捕获列表决定回调保存哪些变量，按值捕获保存副本，按引用捕获则要求引用对象在回调执行时仍然存活。
异步连接可能在较晚时机运行，因此要避免捕获失效的局部引用或已销毁对象。显式上下文比无上下文的长期回调更安全；跨线程连接仍按连接类型在接收者线程调度。复杂分支和可复用业务逻辑最好抽成具名函数，再由 lambda 调用，保持连接点易读并便于测试。

代码示例

```
connect(button,
        &QPushButton::clicked,
        this,
        [this, recordId]
        {
            save(recordId); // context 销毁时连接自动断开
        });
const auto connection = connect(source,
                                &Source::ready,
                                this,
                                [this](const Result& r)
                                {
                                    show(r);
                                });
disconnect(connection); // 动态连接可保存句柄
// 引用捕获的局部变量必须活到回调结束；复杂逻辑放具名槽
```

004 MVC、MVP、MVVM 在 Qt 中如何应用?

答案　MVC、MVP 和 MVVM 都是在拆分数据、显示与交互职责，但具体边界不同。MVC 把模型和视图分开，由控制器协调用户操作；Qt 的 Model/View/Delegate 是一套成熟的数据集合呈现机制。MVP 让 Presenter 面向 View 接口组织流程，便于替换界面和单元测试。MVVM 则由 ViewModel 暴露状态，让 QML 属性绑定自动刷新。
Qt 并不要求项目必须套用某种架构。简单页面多建一层会增加跳转和维护成本；复杂业务需要隔离网络、存储和呈现时，分层才有收益。Widgets 和 QML 可采用不同 UI 组织方式，底层业务服务仍可复用。选择时看测试边界和变化方向，并确保 Model/View 不被误认为完整的业务架构。

代码示例

```
Q_PROPERTY(int progress READ progress NOTIFY progressChanged)
connect(view->startButton, &QPushButton::clicked, view_model, &ImportViewModel::startImport);
// View绑定ViewModel状态；业务服务不依赖具体控件
```

005 Qt 5 和 Qt 6 有什么主要区别?

答案　Qt 6 保留了大量 Qt 5 API，但引入了新的最低工具链要求，重整了构建、图形和部分模块边界。CMake 成为 Qt 6 项目的主流构建方式；QList 与 QVector 在 Qt 6 的实现关系发生变化，此外 Multimedia 等 API 也经历调整。因此旧代码能否编译，不能只看头文件名字相同。

迁移要以目标小版本的 Porting Guide 和模块文档为准，检查已移除模块、签名变更、行为差异、插件部署与编译器 ABI。也要在实际操作系统和显卡后端运行，而不只是本机编译。分阶段升级并保留测试结果，避免同时改构建系统、GUI 和业务逻辑后难以定位差异。

代码示例

```
find_package(Qt6 REQUIRED COMPONENTS Core Widgets Network);
target_link_libraries(app PRIVATE Qt6::Core Qt6::Widgets Qt6::Network);
// 按目标Qt小版本核对迁移指南，并在各平台重建和运行测试
```

006 Qt 6 中移除了哪些模块?

答案　Qt 6 移除了若干旧模块，例如 Qt Script、Qt XmlPatterns 和 Qt Quick Controls 1；旧 Qt WebKit 也不再随 Qt 6 提供，Qt WebEngine 并非源码层面等价替换。部分模块调整了归属或维护状态。Qt Multimedia API 经历重做，不能简单说整个模块都移除了。

项目迁移需要对照具体 Qt 6 小版本的官方模块清单，按实际使用 API 找替代方案。替代库在功能、许可、部署和平台支持上可能不同，升级前要制作小型验证样例。不要只根据编译错误批量换类名，因为模块移除可能反映体系差异；也要检查运行时插件和后端是否可用。

代码示例

```
find_package(Qt6 REQUIRED COMPONENTS Core Widgets);
// 移除模块不能只替换include：先按使用场景选择维护中的替代品
// 在目标版本编译并回归实际行为，记录没有一对一替代的部分
```

007 Qt 6 的新特性有哪些?

答案　Qt 6 的重要变化包括 Qt Quick 图形栈采用 RHI 抽象层，以便在不同图形 API 后端上运行；具体可用 Vulkan、Metal、Direct3D 或 OpenGL 等哪种后端，取决于平台、构建配置和驱动。Qt 6 还改进了 QML 工具链、模块化方式和高 DPI 处理。

抽象层让上层渲染代码不必直接绑定单一 API，但不保证所有设备都自动硬件加速，也不保证不同后端像素结果完全相同。着色器、驱动和平台插件仍会影响实际表现。部署前应在目标 GPU、操作系统和窗口后端测试，遇到兼容问题时依据官方支持矩阵调整渲染后端。

代码示例

```
QQuickWindow window;
qInfo() << window.rendererInterface()->graphicsApi();
load_shader_page(&window);
measure_frame_time_and_memory(); // 在目标GPU/驱动上验证渲染结果
```

008 Qt Creator 的使用技巧有哪些?

答案　Qt Creator 的 Kits 把 Qt 版本、编译器、调试器和目标设备组合起来，构建前应确认项目选中了匹配的 Kit。代码导航、符号搜索和重构工具可帮助理解大型工程；调试时查看调用栈、线程、局部变量和断点，而不是只看日志。Designer 适合编辑 .ui，性能分析器则用于定位真实热点。

当断点信息不完整时检查是否生成并加载调试符号。交叉编译还要确认目标 ABI、设备连接和远程调试配置一致。性能优化前先采样，避免把时间花在非热点代码上。IDE 工具不能替代干净构建和 CI；本地可运行仍需测试部署后的插件、资源和目标设备行为。

代码示例

```
auto kit = active_kit();
auto trace = profiler.capture(
    [&]
    {
        reproduce_slow_click();
    });
auto hot_path = trace.top_cpu_samples();
move_blocking_io_off_ui_thread(hot_path);
// 同一目标Kit下复测交互延迟，并在CI使用该构建配置
```

009 Qt Test 框架是什么？如何使用?

答案　Qt Test 是 Qt 提供的测试框架，可用 QObject 测试类和 private slots 编写用例，并用 QVERIFY、QCOMPARE 检查条件。initTestCase 用于一次性准备，测试槽执行具体行为，清理槽释放资源。QSignalSpy 能观察信号发射，QTest 也能模拟鼠标、键盘和等待条件。

将测试目标链接 Qt::Test，并用合适的宏提供入口后即可放入 CI。优先测试模型、业务规则和错误边界；GUI 事件测试应避免依赖不稳定的绝对延时，必要时等待明确状态。外部网络、时钟和文件系统应抽象或替换为可控对象，降低偶发失败。测试结果仍要覆盖目标平台差异。

代码示例

```
class ModelTest : public QObject
{
    Q_OBJECT
  private slots:
    void rowCountIsUpdated()
    {
        TaskModel model;
        QCOMPARE(model.rowCount(), 0);
        model.append(Task{QStringLiteral("one")});
        QCOMPARE(model.rowCount(), 1);
    }
};
QTEST_GUILESS_MAIN(ModelTest) // CMake target_link_libraries(... Qt6::Test)
// GUI 交互可用 QTest::mouseClick；外部依赖要用 fake 隔离
```

010 Qt 中如何调用平台特定的 API?

答案　跨平台能力优先通过 Qt API 实现；只有 Qt 没有提供所需功能时，再调用 Windows、Linux 或 macOS 的原生接口。用 Q_OS_* 宏将平台代码隔离在适配层，不让系统头文件和句柄泄漏到通用业务模块。平台 API 的窗口句柄、坐标和权限语义都要查对应系统文档。

窗口句柄可能在窗口创建、重建或屏幕切换时变化，使用前检查有效性；高 DPI 下逻辑坐标与物理像素也不同。原生调用失败要检查返回值并提供替代行为，避免只在一个系统成功。跨线程访问系统窗口资源还要遵守操作系统自己的线程规则。每个编译目标都应有构建条件，防止平台宏分支未被测试。

代码示例

```
QWindow* window = /* 已创建的平台窗口 */;
#ifdef Q_OS_WIN
const WId handle = window->winId(); // 仅在平台适配层调用原生 API
useWin32Handle(handle);
#elif defined(Q_OS_MACOS)
useCocoaWindow(window->winId());
#else
usePortableQtPath(window);
#endif
// 检查句柄有效期/DPI；平台头文件不要泄漏到业务层
```

011 Qt 中有哪些调试工具?

答案　调试时按问题类型选工具：Qt Creator 或 GDB/LLDB 用于断点、调用栈和线程；qDebug、qWarning 提供运行上下文；Qt Test 固化可重复行为。内存越界和释放问题可借助 AddressSanitizer、LeakSanitizer 或 Valgrind，性能问题则用采样 profiler 和计时器定位。

偶发崩溃应保留符号化调用栈、精确二进制版本和最小复现条件。发布构建也可以分离保存符号，便于之后分析。日志要考虑并发写入、文件轮转和隐私，不得记录口令或令牌。工具输出是线索，仍需回到实际调用路径验证根因；优化前后使用相同输入和环境比较。

代码示例

```
QElapsedTimer timer;
timer.start();
runRepresentativeOperation();
qInfo() << "operation ms" << timer.elapsed(); // 局部测量而非结论
// Qt Creator Profiler/GDB 看调用栈、线程、CPU 热点
// ASan/LSan 或 Valgrind 查内存；发布包保留匹配符号
// 用同一 Release 配置、同一输入复测改动前后
```

012 Qt 如何实现跨平台?

答案　Qt 通过平台抽象层和插件隔离窗口、输入、文件和网络等操作系统差异，使同一套 API 可以在多个平台构建。平台插件负责与 Windows、macOS、X11 或 Wayland 等窗口系统连接。应用代码大部分可共享，但部署仍要带上正确插件和依赖库。

跨平台不代表行为处处完全一致：字体排版、文件权限、路径大小写、多屏坐标、输入法和系统托盘都可能不同。平台专属能力应封装，缺少能力时提供合理回退。应在目标 OS、架构和图形后端测试安装、升级和运行，不要用单一开发机上的成功推断所有环境正常。

代码示例

```
QString file_path = QDir(selected_directory).filePath("notes.txt");
QSaveFile file(file_path);
if (file.open(QIODevice::WriteOnly))
{
    file.write(contents);
    file.commit();
}
// 用Qt路径/文件API适配平台差异，并在目标系统验证打包
```

013 Qt 应用出现内存泄漏如何定位和处理?

答案　定位内存泄漏时先用 ASan/LSan、Valgrind 或系统堆分析工具重现问题并查看分配栈。区分真正无法释放的对象、仍被引用的缓存，以及程序退出时才释放的延迟资源。Qt 项目常见原因包括没有 parent 的 QObject、裸指针漏删、定时器或 lambda 保留对象，以及 shared_ptr 相互引用形成环。

修复核心是明确唯一所有者，使用 QObject 父子树或 RAII，并让循环引用改用弱引用或显式断开。反复打开关闭相关页面、取消后台任务并检查退出路径，有助于暴露只在生命周期边界出现的泄漏。不要把“进程内存没有立刻下降”直接等同泄漏，分配器可能保留已释放内存；以对象或分配栈证据为准。

代码示例

```
QPointer<Worker> worker = page->worker(); // 观察 QObject 是否仍存活
connect(worker,
        &Worker::done,
        this,
        [worker]
        {
            if (!worker)
                return; // 异步时对象可能已销毁
            consume(worker->result());
        });
page->close(); // 父子树回收其子对象
// 用 sanitizer/堆快照定位孤儿对象、环和重复创建资源
// 修复后反复打开关闭页面并检查退出路径
```

014 Qt 界面如何实现国际化和运行时切换?

答案　国际化从源代码标记用户可见字符串开始：C++ 用 tr()，QML 使用相应翻译函数；lupdate 提取到 .ts，译者用 Qt Linguist 编辑，再由 lrelease 生成 .qm。运行时创建并加载 QTranslator，安装后新产生的翻译查询会使用该目录中的翻译。

运行时切换语言时，已创建控件可能要重新执行 retranslateUi 或刷新模型的展示文本；缓存的翻译字符串也应更新。用复数形式表达数量，不要把动态内容与固定文本随意拼接。不同语言文本长度差异很大，布局需允许伸缩。多个翻译器的安装顺序影响查找优先级，应明确先后并处理加载失败。

代码示例

```
QTranslator translator; // 生命周期覆盖应用翻译期间
if (!translator.load(locale, QStringLiteral(":/i18n/app")))
    reportLoadError();
qApp->installTranslator(&translator);
ui->retranslateUi(window); // Designer 控件刷新
model->refreshTranslatedLabels(); // 自定义模型文本也需重算
// 用户文案统一 tr()；复数用 n 参数，不拼接词片段
// 长词会改变布局，应检查窄窗口和切换时机
```

015 Qt 程序崩溃的常见原因有哪些?

答案　Qt 程序崩溃常由空指针、悬空引用、数组越界、重复释放或对象生命周期错误引起。多线程场景下跨线程直接访问 QWidget、异步 lambda 捕获已销毁对象也很常见。启动时的动态库 ABI 不匹配或插件缺失，则可能在加载或调用平台功能时失败。

应保存符号化崩溃栈、输入数据和精确构建版本，再用 sanitizer 或调试器找到首次错误访问，而不是只处理最终崩溃行。重点检查关闭窗口、取消任务、重复请求和失败回滚等边界。构造最小复现后写回归用例，证明同类生命周期问题不会重现；错误处理也应避免从析构期间继续发出危险回调。

代码示例

```
connect(worker,
        &Worker::finished,
        window,
        [guard = QPointer<Window>(window)](Result r)
        {
            if (!guard)
```

```
            return;
        guard->show_result(r); // 用QObject上下文或QPointer避免访问已销毁窗口
    });
```

016 qDebug()、qWarning()、qCritical() 有什么区别?

答案　qDebug 用于开发诊断，qInfo 表示一般运行信息，qWarning 提醒可能恢复的问题，qCritical 表示严重错误；qFatal 会记录后终止进程。级别帮助筛选和配置输出，但不会自动修复问题，也不代替检查 API 的返回值。开发者仍需决定是否重试、回滚或向用户报告。

可以用消息处理器把日志送到控制台、文件或系统日志，但处理器可能被多个线程调用，应保证线程安全且不能递归调用 qDebug。发布构建可按需关闭调试级别输出，日志要设轮转和保留策略，并过滤口令、令牌、个人数据。严重错误通常应保留足够上下文，但避免输出敏感请求正文。

代码示例

```
qDebug() << "cache miss" << key; // 诊断细节
qInfo() << "service started"; // 运行信息
qWarning() << "retrying" << error; // 可恢复问题
qCritical() << "database unavailable"; // 严重错误仍需明确处理
if (!query.exec())
    handle(query.lastError()); // 日志等级不代替错误检查
// qFatal 会终止进程；生产日志需轮转并脱敏
```

017 qmake 和 CMake 有什么区别?

答案　qmake 用 .pro 文件描述 Qt 工程，语法直接集成 Qt 模块和历史项目习惯，很多旧工程仍在使用。CMake 是通用构建系统，Qt 6 文档广泛采用 imported targets、AUTOMOC/AUTOUIC/AUTORCC 与更广泛的依赖生态。两者都能构建 Qt 项目，区别在于描述方式、工具集成和生态，不是简单的“旧的一定不能用”。

迁移时要逐项映射源文件、模块、资源编译、代码生成、平台条件和第三方依赖，并在目标构建环境验证。不要只把 .pro 改名为 CMakeLists；生成步骤和部署逻辑也要迁移。若旧项目稳定且依赖 qmake，继续维护可能更经济；新项目应结合团队工具链和未来依赖选择。

代码示例

```
find_package(Qt6 REQUIRED COMPONENTS Widgets) qt_add_executable(app main.cpp window.cpp window.h)
        target_link_libraries(app PRIVATE Qt6::Widgets) #AUTOMOC
    / 资源由Qt CMake集成处理
```

018 windeployqt 和 macdeployqt 的作用是什么?

答案　windeployqt 会收集 Windows Qt 应用常用的 DLL、平台插件以及部分 QML 依赖；macdeployqt 则把 Qt 库和插件放入 macOS .app，并处理部分库引用路径。它们减少手工复制工作，但只了解 Qt 生态中的依赖，不能保证所有第三方库和外部资源都被发现。

部署后要在没有开发版 Qt 的干净环境启动，检查平台插件、图像格式、翻译、多媒体后端和 QML 模块。macOS 仍需处理代码签名、公证和权限；Windows 还要核对第三方运行库。自动部署命令成功不等于安装包完整，更新工具版本也可能改变收集行为，交付前应实际安装验证。

代码示例

```
windeployqt(app_path); // Windows收集Qt DLL与平台插件
verify_on_clean_vm(app_path); // 断开开发环境变量后检查独立部署包
```

019 不同平台下 Qt 程序的打包和部署有什么区别?

答案　不同系统的应用部署重点不同。Windows 通常随程序放置 Qt DLL 和 plugins/platforms；macOS 要维护 .app 内框架路径，并完成签名和公证；Linux 则要考虑发行版 ABI、系统库版本和平台插件。各平台的安装器、权限模型和更新机制也不同。

部署工具只能收集部分依赖，第三方库、字体、证书、媒体编解码器和应用资源仍需单独确认。要在目标 OS 与架构的干净环境测试安装、启动、升级和卸载，避免开发机上恰好存在的库掩盖缺失。使用容器或虚拟机可重复部署验证，但最终仍需覆盖真实目标设备和桌面后端。

代码示例

```
std::vector<std::string> required_plugins = inspect_runtime_dependencies(app);
verify_installation_on("Windows");
verify_installation_on("macOS");
verify_installation_on("Linux");
// 每个平台分别处理ABI、签名、插件与证书部署
```

020 信号槽连接失败的常见原因有哪些?

答案　连接失败先看 connect() 返回的 QMetaObject::Connection 是否有效，并确认发送者、接收者对象仍存活。旧式字符串连接依赖运行时字符串匹配，信号槽拼写、const/reference 或参数个数差异都可能失败；函数指针形式通常能在编译阶段发现签名不兼容。

自定义信号所属类需正确参与元对象系统，包含 Q_OBJECT 并由 moc 生成代码。跨线程 queued slot 还要求参数能传递、目标线程事件循环运行。若连接成功却没有执行，检查信号是否实际发射、条件分支、对象线程和过滤器；不要在每次初始化时重复连接造成多次调用。

代码示例

```
const auto connection = QObject::connect(sender, &Sender::ready, receiver, &Receiver::consume);
if (!connection)
    qWarning() << "connect failed"; // 新式签名编译期检查
// 跨线程队列槽依赖接收者事件循环
// 自定义信号类需要 QObject + Q_OBJECT + 正常 moc
// 重复初始化要么只连接一次，要么保存句柄按需断开
```

021 大量数据显示时如何优化性能?

答案　大数据视图应使用 Model/View 和 delegate，只为可见区域绘制，而不是为每条记录创建多个 QWidget。数据量大时可实现 canFetchMore()/fetchMore()，分批拉取，避免一次把全部数据和界面对象放入内存。绘制与 data() 应快速，昂贵转换可缓存，但缓存需要失效规则和大小上限。

批量更新应使用正确的模型通知范围，合并多次 repaint 请求，并用 profiler 找出实际热点。排序、过滤和网络载入可放到后台或专用代理模型，再将结果安全交回模型线程。减少数据量本身常比微调绘制更有效，例如分页、搜索和摘要显示。优化后测量滚动帧率、内存和数据刷新延迟。

代码示例

```
bool Model::canFetchMore(const QModelIndex& parent) const
{
    return !parent.isValid() && loaded_ < total_;
}
void Model::fetchMore(const QModelIndex& parent)
{
    const int count = qMin(batchSize, total_ - loaded_);
    beginInsertRows(parent, loaded_, loaded_ + count - 1);
    loadNextBatch(count);
    loaded_ += count;
    endInsertRows();
}
// delegate 绘制可见行；缓存必须设上限并在数据改变时失效
```

022 如何优化 Qt 应用程序的启动速度?

答案　启动优化先量出时间花在哪里：进程启动、首屏创建、同步磁盘读取、网络请求还是 QML 组件构造。优先只加载用户马上看到的模块和资源，把低频页面、缩略图或非必要检查延迟到首屏出现之后；独立初始化任务可异步运行并显示进度。

Qt 资源系统方便组织和打包文件，但不会自动缩短解码或模块初始化时间。Loader 可以延后创建 QML 页面，却会把成本移到首次打开时，需判断用户体验。分别测冷启动首屏时间和完整功能可用时间；并行初始化也可能争用 CPU 与磁盘。避免在 main() 或构造函数中进行阻塞网络请求。

代码示例

```
// QML：首页先显示，低频页面延后构造
ApplicationWindow
{
    Component.onCompleted : Qt.callLater(() = > settingsLoader.active = true) Loader
    {
    id:
        settingsLoader;
    active:
        false;
    source:
        "Settings.qml"
    }
}
// 网络与磁盘初始化异步执行，首屏只载入必要资源
// 分别测首屏时间与全部启动时间，延迟加载不一定缩短总耗时
```

023 如何优化 Qt 界面的绘制性能?

答案　绘制优化先用 profiler 确认瓶颈是 paintEvent、布局、模型更新还是图形后端。随后缩小 update 区域，只重画变化部分；静态背景可缓存，绘制回调中避免 I/O、复杂计算和大量临时分配。大量列表应借助 model/view 和 delegate，减少真实控件数量。

缓存要在尺寸、主题或数据变化时正确失效，避免使用旧图像；高频刷新还可合并更新。Qt Quick 的渲染后端和硬件加速效果依赖平台，Widgets 的绘制也可能受设备像素比影响。最终要在目标设备上测帧率、CPU 与内存，不应为了减少一次 draw call 引入巨大的全屏缓存。

代码示例

```
void Chart::setPoint(const QPointF& p)
{
    const QRect dirty = markerRect_ | nextMarkerRect(p);
    marker_ = p;
    markerRect_ = nextMarkerRect(p);
    update(dirty); // 仅请求局部重绘
}
void Chart::paintEvent(QPaintEvent* e)
{
    QPainter painter(this);
    painter.setClipRegion(e->region());
    drawCachedGrid(painter);
    drawMarker(painter, marker_);
}
// 不在 paintEvent 做 I/O/复杂计算；Model/View 可虚拟化大列表
```

024 如何减小 Qt 程序的体积?

答案　减小安装包应先看真正的分发产物，而不只是主程序文件。用 Release 构建、移除未使用模块和资源、只部署需要的插件及翻译，并检查第三方依赖；Windows 可分离调试符号，Linux/macOS 也有相应 strip 或符号包工具，但要保留匹配符号供崩溃分析。

静态链接可能让单个文件更大，也不会自动移除不需要的代码，还需显式处理静态插件和许可证义务。部署工具裁剪前要验证所有功能路径，避免删掉运行时才加载的图像格式、平台插件或 QML 模块。以整个安装目录或安装包大小比较优化效果，确保目标环境能启动和使用。

代码示例

```
target_link_options(app PRIVATE "$<$<CONFIG:Release>:-s>") archive_debug_symbols(app);
copy_only_required_plugins(app);
// 测量完整安装包并保留符号文件，不能只统计主exe
```

025 如何减少 Qt 应用程序的内存占用?

答案　内存优化应先用 profiler 或堆快照找到高水位来自哪里：大图、模型数据、缓存、临时副本，还是仍存活的对象。限制队列和缓存容量，离开视图时释放不再需要的资源；大图按显示尺寸解码成缩略图，避免同时保留多份全分辨率 QPixmap。QObject 用父子树管理，普通 C++ 资源用 RAII。

Qt 隐式共享可让只读副本暂时共用数据，但写入时会分离并复制，不能据此假设成本永远为零。不要无证据地频繁 squeeze 容器或清缓存，这可能增加分配和抖动。检查长列表的分页与图像解码峰值，按典型工作流测量。也要区分泄漏与内存分配器保留的空闲区。

代码示例

```
QImageReader reader(path);
reader.setScaledSize(targetPixels); // 按目标显示尺寸解码
QImage thumbnail = reader.read();
if (thumbnail.isNull())
    return report(reader.errorString());
cache.insert(key, thumbnail); // 缓存设置最大容量和淘汰策略
if (cache.size() > maxEntries)
    evictOldest();
thumbnail = {}; // 不再显示时释放引用
// 隐式共享能减副本，写操作仍可能触发 detach
```

026 如何分析 Qt 应用程序的性能瓶颈?

答案　性能分析要在有代表性的场景和 Release 配置下进行，先记录基线，再用 Qt Creator Profiler 或系统采样器找 CPU、内存、QML 和渲染热点。QElapsedTimer 可测某段操作，但单次值受调度、缓存和机器状态影响，应重复运行并看分布。

重点排查主线程同步 I/O、频繁布局和重绘、过量模型通知、锁等待与网络等待。先优化最热路径，再用相同数据和设备复测；同时观察内存和首屏延迟，避免一个指标变好、另一项恶化。Debug 构建的优化级别与符号开销会影响测量，不能直接代表用户发布体验。

代码示例

```
QElapsedTimer timer;
timer.start();
const auto result = runHotPath(input);
```

```
const qint64 elapsedMs = timer.elapsed();
qInfo() << "hot path ms" << elapsedMs; // 测真实路径
// profiler 先找 CPU/内存/渲染热点，不凭猜测优化
// 改动后用相同 Release 构建、设备和输入复测
// 单次耗时会受调度影响，应重复采样
```

027 如何处理不同平台的路径分隔符问题?

答案　路径处理尽量使用 QDir、QFileInfo、QFile 等 Qt API，不要把斜杠或反斜杠硬编码拼接。Qt 通常能接受正斜杠形式，但路径仍要区分普通文件路径、URL 和资源系统路径（如 :/icons/...）；三者编码和访问方式不同。cleanPath 可整理 . 和 ..，却不是权限验证。

若用户提供不可信路径，不能只把它拼到允许根目录后就认为安全；还需规范化后检查根目录边界，并考虑符号链接可能指向外部。不同平台还可能有大小写、盘符和网络路径差异。组合路径用 QDir::filePath，文件访问前验证业务允许范围和权限，避免路径穿越或误写系统文件。

代码示例

```
QDir root(allowedRoot);
const QString candidate = root.filePath(userRelativePath); // 不手拼斜杠
const QString clean = QDir::cleanPath(candidate);
if (!isWithinAllowedRoot(clean, root.absolutePath()))
    return reject();
QFile file(clean);
if (!file.open(QIODevice::ReadOnly))
    return report(file.errorString());
consume(file.readAll());
// cleanPath 不阻止软链接逃逸；边界校验按真实文件系统路径做
```

028 如何实现 Qt 应用程序的单例模式?

答案　进程内确实需要唯一实例时，可以用 C++11 的函数局部静态对象：其初始化在语言层面保证线程安全。Qt QObject 还有线程亲和性和父子所有权问题，因此应在 QApplication 建立后再创建，并确保对象在正确线程使用。构造函数私有、禁用复制可以阻止外部制造第二份实例。

单例引入全局依赖，测试替换、初始化顺序和销毁顺序都更难。很多服务更适合由应用对象创建后显式传给需要它的组件。若单例是 QObject，不要再给它指定另一个 parent，也不要同时交给智能指针拥有，否则可能双重释放。析构期间依赖其他静态单例也可能遇到静态销毁顺序问题。

代码示例

```
class Service : public QObject
{
    Q_OBJECT
  public:
    static Service& instance()
    {
        static Service value;
        return value;
    }
    Service(const Service&) = delete;
    Service& operator=(const Service&) = delete;

  private:
    Service() = default; // 不交给另一个 parent/shared_ptr 重复拥有
};
// 首次调用应在 QApplication 构造之后；优先考虑依赖注入
```

029 如何实现 Qt 应用程序的崩溃捕获和日志记录?

答案　普通可预期错误应由返回值、错误码或异常路径处理，不要等进程崩溃后才记录。崩溃收集通常配置操作系统的 core dump、Windows minidump 或专用崩溃服务，并保存构建版本和匹配的符号文件，再离线生成可读调用栈。致命信号处理上下文受严格限制，不能安全地调用 QFile、qDebug、锁或内存分配，也不能普遍从段错误中恢复继续运行。应用可保存崩溃前的环形日志，但真正转储交给平台机制。崩溃报告应脱敏并取得适当授权，避免上传口令、令牌或用户文件内容。

代码示例

```
install_crash_handler(); // 平台转储机制需在进程启动时安装
log_error_with_context(error, version, request_id);
symbolize_offline(dump, matching_symbols);
// 致命信号处理路径只执行异步信号安全的最小操作
```

030 如何实现 Qt 应用程序的日志系统?

答案　Qt 日志系统可用 qDebug、qInfo、qWarning 和 qCritical 按级别记录，再通过 qInstallMessageHandler() 集中添加时间、线程、类别等上下文并写滚动文件。消息处理器可能由多个线程并发调用，写入共享文件需使用同步保护。处理器内部不能再次调用 Qt 日志函数，否则容易递归；也不应在 GUI 线程同步上传网络日志或做长时间磁盘操作。生产日志要配置容量和轮转，记录中应遮盖凭据、个人信息和敏感正文。若磁盘不可写，应有回退或丢弃策略，避免日志故障反过来阻塞主要功能。

代码示例

```
static QMutex logMutex;
void messageHandler(QtMsgType type, const QMessageLogContext&, const QString& msg)
{
    QMutexLocker lock(&logMutex); // handler 可从多线程并发进入
    writeRotatingLog(type, msg.toUtf8()); // 自有写入函数
    if (type == QtFatalMsg)
        std::abort();
}
qInstallMessageHandler(messageHandler);
// handler 内不能再调 qDebug/qWarning，否则递归；不要同步上传网络
```

031 如何实现 Qt 应用程序的权限管理?

答案　Qt 提供按钮、菜单和系统 API，却不替应用决定谁能做什么。权限模型应由可信业务层定义身份、角色和操作规则；每个敏感动作在执行时都重新授权，不能只在界面隐藏按钮。若数据存储或服务在服务器端，服务器也必须独立验证权限，客户端状态可被修改。

集中权限服务可以让界面查询可用操作、业务层验证命令并记录拒绝事件。日志用于审计时要控制访问和保留期限，避免存入凭据。离线应用也需考虑本地配置可被篡改，界面上的“管理员”布尔值不构成可信认证。权限变更和缓存过期应有明确策略，防止旧授权继续生效。

代码示例

```
if (!authorization.allow(user, "delete", record_id))
    return forbidden();
auto record = repository.load(record_id);
if (record.version != expected_version)
    return conflict();
repository.erase(record_id);
audit.write(user, "delete", record_id); // 后端每次请求独立授权
```

032 如何实现 Qt 应用程序的皮肤切换?

答案　Widgets 可准备多套 QSS，并在用户切换时替换应用或窗口样式表；应集中维护颜色和控件规则，避免各页面各写一份互相冲突的覆盖。QML 可通过统一 Theme 对象暴露颜色、间距与字体，再由各组件绑定，改变主题属性后相关界面会自动重新计算。

皮肤切换不只是更换背景色，还要检查禁用、悬停、焦点和选中状态，以及对比度、键盘操作和可访问性。自绘控件或缓存图像若固化旧颜色，需主动刷新或重建。QSS 不是完整 CSS，复杂状态可能需要 QStyle 或控件绘制；切换时也要确保样式表加载失败能回退到可用主题。

代码示例

```
QFile qssFile(stylePath);
if (!qssFile.open(QIODevice::ReadOnly))
    return report(qssFile.errorString());
qApp->setStyleSheet(QString::fromUtf8(qssFile.readAll())); // Widgets 换肤
// QML: Theme 属性可绑定并驱动组件重绘\ntheme.accent = "#2878c8"; theme.spacing = 12;
// 组件按 theme 属性刷新自绘缓存；选择器尽量限制作用范围
// 验证焦点、禁用态、平台原生外观和可访问性
```

033 如何实现 Qt 应用程序的自动更新功能?

答案　自动更新要先检查新版本清单，再下载适用于当前平台和架构的包。下载后验证可信数字签名或可信哈希，再由独立 updater 在主程序退出后替换文件。TLS 保护传输链路，但不能替代包级签名；清单和版本比较也要防止攻击者诱导安装过期版本。

更新器应分阶段写入临时目录，检查磁盘空间和权限，替换成功后保存版本状态；中断时能继续下载或回滚到上一个可用版本。不要在程序运行中直接覆盖正在使用的二进制。签名密钥要安全管理，失败日志不能包含令牌。上线前测试断网、校验失败、升级中断和回退场景。

代码示例

```
auto manifest = https_get(update_url);
if (!verify_signature(manifest))
    reject_update();
```

```
download_to_temp(manifest.package);
verify_hash_and_platform(temp_file, manifest);
run_standalone_updater(temp_file); // 替换失败时回滚，主程序不自行覆盖正在运行的文件
```

034 如何实现 Qt 应用程序的配置管理?

答案　配置管理可以封装在一个服务中，以 QSettings 统一组织默认值、键名和读取校验。使用 beginGroup() 按模块分组，写入窗口尺寸或普通偏好后，在需要确保落盘时调用 sync() 并检查 status。为配置结构记录版本，升级时迁移旧键或为缺失值提供合理默认。

QSettings 适合普通配置，不适合作为口令或令牌的明文保险箱；敏感凭据应使用系统密钥链或安全存储。同一应用多个进程同时写同一配置时，sync 不会替应用解决所有更新冲突，需定义谁拥有配置或实现锁与合并策略。读取到的值也要验证范围，防止损坏配置导致异常窗口尺寸或资源消耗。

代码示例

```
QSettings settings;
settings.beginGroup(QStringLiteral("config"));
const int version = settings.value("version", 0).toInt();
if (version < 2)
    migrateV1ToV2(settings); // 配置结构版本迁移
settings.setValue("version", 2);
settings.endGroup();
settings.sync();
if (settings.status() != QSettings::NoError)
    reportSettingsError();
// 凭据放系统密钥链，不写普通 INI/注册表键
```

035 如何实现一个自定义的标题栏?

答案　自定义标题栏通常用 FramelessWindowHint 去掉系统装饰，再用布局创建标题、最小化、最大化和关闭按钮。可通过按下时记录全局鼠标位置和窗口偏移来拖动，也可以在平台支持时请求系统原生移动，让系统保留贴边和窗口管理行为。

手工无边框实现不止拖动，还要处理边缘缩放、最大化恢复、双击标题栏、多屏工作区、设备像素比和键盘可访问性。鼠标被其他控件接收时拖动区域也要明确。原生移动接口及可用性有 Qt 版本和平台差异，应在目标系统验证；普通窗口优先保留系统标题栏，减少自定义边缘问题。

代码示例

```
setWindowFlags(Qt::Window | Qt::FramelessWindowHint);
auto* bar = new QWidget(this);
auto* layout = new QHBoxLayout(bar);
layout->addWidget(titleLabel);
layout->addWidget(closeButton);
connect(closeButton, &QPushButton::clicked, this, &QWidget::close);
void TitleBar::mousePressEvent(QMouseEvent* e)
{
    dragOffset_ = e->globalPosition().toPoint() - window()->pos();
}
void TitleBar::mouseMoveEvent(QMouseEvent* e)
{
    if (e->buttons() & Qt::LeftButton)
        window()->move(e->globalPosition().toPoint() - dragOffset_);
}
// 还需双击最大化、边缘缩放、高 DPI 与键盘操作
```

036 如何实现串口通信工具?

答案　串口工具通常通过 Qt SerialPort 模块枚举设备，再设置端口名、波特率、数据位、奇偶校验和停止位后打开。readyRead 到来时读取字节并追加到接收缓冲区，再按设备协议解析完整帧；串口也可能一次只收到半帧。发送端同样要处理写缓冲和设备断开。

设备可能被拔出、占用或配置失败，应监听错误并允许用户重新连接。不要在 GUI 线程长时间调用 waitForReadyRead()；事件驱动方式更适合交互工具。显示十六进制和文本时要区分原始字节与编码后的文本，日志应限制大小。串口配置必须与设备协议一致，校验失败和粘拆包都要在协议层处理。

代码示例

```
QSerialPort port(this);
port.setPortName(portName);
port.setBaudRate(QSerialPort::Baud115200);
port.setDataBits(QSerialPort::Data8);
port.setParity(QSerialPort::NoParity);
```

```
port.setStopBits(QSerialPort::OneStop);
if (!port.open(QIODevice::ReadWrite))
    return report(port.errorString());
connect(&port,
        &QSerialPort::readyRead,
        this,
        [this]
        {
            buffer_ += port.readAll();
            parseFrames(buffer_);
        });
// 检查错误/拔出；GUI 线程不 waitForReadyRead() 阻塞
```

037 如何实现图片查看器?

答案　图片查看器可用 QFileDialog 选择文件，以 QImageReader 检查格式和尺寸后解码，再交给 QLabel 显示 QPixmap，或由自绘控件根据变换矩阵绘制。缩放、平移、旋转应维护统一变换状态，不要每次滚轮都把原图重新采样成新的全尺寸图。

超大或不可信图片可能消耗大量内存，应限制宽高和总像素，必要时按目标大小解码，并在后台线程完成耗时读图。检查 reader 错误和保存结果；EXIF 方向可通过自动变换处理。高 DPI 下还需区分逻辑尺寸和实际像素。显示图像数据在线程间传递时，应明确 QImage 与 GUI 资源的使用边界。

代码示例

```
QImageReader reader(path);
reader.setAutoTransform(true); // 应用 EXIF 方向
reader.setScaledSize(targetPixels); // 避免解完整超大图
QImage image = reader.read();
if (image.isNull())
    return report(reader.errorString());
preview->setPixmap(QPixmap::fromImage(image)); // 在 GUI 线程显示
// 保存用 QImageWriter 并检查错误；外部输入限制像素数
// 滚轮缩放由变换矩阵控制，别反复全尺寸重采样
```

038 如何实现截图功能?

答案　截取屏幕可从 QGuiApplication::screens() 选择目标 QScreen，再用 grabWindow(0) 获取该屏幕图像并保存或预览。多屏布局使用逻辑坐标，而图像像素可能受 devicePixelRatio 影响；选择屏幕和裁剪区域时要做相应换算。保存前检查空结果与文件写入错误。

系统对捕获其他窗口或整个桌面有权限限制。Wayland 等桌面环境可能禁止应用任意读取屏幕，需通过系统授权门户或用户选择窗口/区域的流程。不能用抓取结果来绕过用户隐私授权。目标屏幕可能在运行中变化，应该重新查询并处理无屏幕或捕获失败的情况。

代码示例

```
QScreen* screen = QGuiApplication::primaryScreen();
if (!screen)
    return report(QStringLiteral("no screen"));
QPixmap shot = screen->grabWindow(0); // 平台权限可能限制
if (shot.isNull())
    return report(QStringLiteral("capture unavailable"));
if (!shot.save(outputPath))
    return report(QStringLiteral("save failed"));
// 多屏先按窗口所在 QScreen 选择并换算 DPR/逻辑坐标
// Wayland 等环境应走平台授权门户
```

039 如何实现数据可视化图表?

答案　图表可选择 Qt Graphs、Qt Charts（依目标版本）、第三方库或自绘，先依据平台、交互和数据量确认能力。折线图适合随时间变化的趋势，柱状图适合类别对比；坐标、系列和数据源最好从业务数据层分开管理。动态更新应限制频率，海量点可以抽稀、按时间窗聚合或只显示当前视区。

版本变化要明确：Qt Charts 从 Qt 6.10 起弃用，新项目应评估 Qt Graphs 的支持范围与迁移成本；不应忽略目标小版本差异。图表库不保证适合高频海量散点，实际性能依图形后端和设备而变。自绘能定制外观，但要自行实现轴、命中测试和可访问性。用代表数据测帧率和交互延迟。

代码示例

```
if (samples.size() > visible_limit)
    samples.erase(samples.begin(), samples.begin() + excess);
chart_model.append(timestamp, value);
```

```
timer.start(100ms); // 合并刷新，限制每次显示数据量
// 在目标GPU上测量缩放、滚动和绘制延迟
```

040 如何实现窗口的无边框和拖动?

答案　无边框窗口可通过 FramelessWindowHint 去掉系统装饰，再在自定义标题区域处理鼠标拖动。按下时记录窗口位置与全局鼠标位置的差，左键移动期间按新位置计算窗口坐标。仅让标题空白区开始拖动，避免按钮点击和文本选择被误认为移动。

手动实现还要增加边缘调整尺寸、最大化/还原、屏幕边界、键盘快捷方式和高 DPI 支持。可用平台提供的系统移动/调整大小接口保留系统贴边行为，但 Qt 版本和平台支持不同，不能假设处处可用。触屏和窗口管理器行为也需测试。若没有明确设计需求，使用系统标题栏通常更可靠。

代码示例

```
setWindowFlags(Qt::Window | Qt::FramelessWindowHint);
void Window::mousePressEvent(QMouseEvent* e)
{
    if (e->button() == Qt::LeftButton)
        dragOffset_ = e->globalPosition().toPoint() - pos();
}
void Window::mouseMoveEvent(QMouseEvent* e)
{
    if (e->buttons() & Qt::LeftButton)
        move(e->globalPosition().toPoint() - dragOffset_);
}
// 优先用平台原生 startSystemMove；手动窗口还需边缘缩放
// 多屏边界、最大化恢复和无障碍键盘操作都要处理
```

041 如何实现系统托盘功能?

答案　系统托盘功能由 QSystemTrayIcon 表示。先检查 isSystemTrayAvailable()，设置图标和提示，再绑定 QMenu，并监听 activated() 区分单击、双击或上下文菜单操作。关闭主窗口时可以隐藏到托盘，但必须提供明显的恢复和真正退出入口，否则用户会以为程序已关闭。

不同桌面环境对图标可见性、激活方式和通知支持不完全相同。托盘图标资源必须随安装包部署；图标不可用时应用仍应保留正常窗口操作。退出时停止后台任务、断开设备并保存状态，不能只从托盘菜单直接 quit 而遗留资源。对无托盘环境提供系统菜单或普通窗口替代。

代码示例

```
QSystemTrayIcon tray(QIcon(":/icons/app.png"), this);
if (!QSystemTrayIcon::isSystemTrayAvailable())
    reportNoTray();
auto* menu = new QMenu(this);
menu->addAction(tr("Show"), this, &Window::showNormal);
menu->addAction(tr("Quit"), qApp, &QCoreApplication::quit);
tray.setContextMenu(menu);
connect(&tray,
        &QSystemTrayIcon::activated,
        this,
        [this](auto reason)
        {
            if (reason == QSystemTrayIcon::Trigger)
                showNormal();
        });
tray.show(); // 关闭窗口可 hide 而不是退出，并保留真正退出入口
```

042 如何实现网络调试助手?

答案　网络调试助手可用 QTcpSocket、QTcpServer 和 QUdpSocket 分别做客户端、服务端与数据报测试，界面显示连接状态、收发方向、时间和错误。发送内容应明确是十六进制字节还是编码后的文本；TCP 接收用 QByteArray 缓冲并按协议边界解析，UDP 则保留数据报边界。

自动发送使用 QTimer，但应限制频率和待发送队列，避免设备或网络被压满。日志文本框要限制行数和刷新批次，不能无上限保存所有字节。展示层与传输层分开，连接断开时及时清理缓冲和状态；调试工具也需防止用户输入超大数据、误连敏感端口或把凭据写入日志。

代码示例

```
socket.connectToHost(host, port);
connect(&socket,
        &QTcpSocket::readyRead,
        this,
```

```
        [&]
        {
            rx_buffer += socket.readAll();
            parse_complete_frames(rx_buffer);
        });
socket.write(parse_hex_input(text)); // TCP字节流要由应用协议分帧
```

043 如何实现聊天软件的界面?

答案　聊天界面可把每条消息建模为发送者、正文、时间、状态和头像等字段，用 QAbstractListModel 提供角色，再让 QListView 配合 delegate 绘制左右气泡。输入框、附件按钮和发送状态是独立控件。这样可以分页加载历史，也避免为每条消息创建常驻 QWidget。

网络传输和数据库访问应放在异步服务层，模型在所属线程通过排队结果更新。delegate 的 paint 只负责读取已准备好的显示数据和绘制，不能在绘制时查询数据库或阻塞网络。长消息要按可用宽度换行，状态变化需发出模型通知。大量历史记录通过分页和视区缓存控制内存。

代码示例

```
class MessageModel : public QAbstractListModel
{ /* roles: body/time/status */
};
MessageModel model;
QListView list;
list.setModel(&model);
list.setItemDelegate(new BubbleDelegate(&list)); // delegate 绘制气泡而非每行 QWidget
connect(sendButton,
        &QPushButton::clicked,
        this,
        [this]
        {
            emit sendRequested(input->text());
            input->clear();
        });
connect(
    networkWorker, &Worker::messageReceived, &model, &MessageModel::append, Qt::QueuedConnection);
// 分页载入历史；绘制时不查数据库/不做网络请求
```

044 如何实现视频播放器?

答案　Qt 6 的 QMediaPlayer 可通过 setSource() 加载本地或远端媒体，设置 QVideoWidget 作为画面输出，并提供播放、暂停和进度控制。QAudioOutput 独立管理音量和静音。进度条通常根据 durationChanged 设置范围，再按 positionChanged 更新当前位置；用户拖动则调用 setPosition。

编解码格式依赖平台媒体后端及已安装组件，setSource 成功不保证文件一定能解码。应显示 errorOccurred、缓冲和播放状态，并允许用户看到失败原因。Qt 5 与 Qt 6 的接口差异明显，旧的播放列表 API 也有变化，迁移时查看目标版本文档。测试时覆盖暂停、 seek、音量和后端不支持格式。

代码示例

```
auto* player = new QMediaPlayer(this);
auto* audio = new QAudioOutput(this);
player->setAudioOutput(audio);
auto* video = new QVideoWidget(this);
player->setVideoOutput(video);
connect(player, &QMediaPlayer::errorOccurred, this, &Window::showMediaError);
connect(player, &QMediaPlayer::positionChanged, slider, &QSlider::setValue);
player->setSource(QUrl::fromLocalFile(filePath));
player->play();
// 检查后端格式/缓冲状态；音量由 QAudioOutput 管理
```

045 如何对 Qt 应用程序进行单元测试?

答案　Qt Test 可测试业务对象、模型数据、信号发射和 GUI 交互。QCOMPARE 比较实际值与预期，QVERIFY 检查条件，QSignalSpy 记录某个信号出现几次；QTest 还能模拟鼠标点击和键盘输入。纯逻辑测试通常比依赖屏幕截图稳定，适合先覆盖重要规则和失败路径。

把网络、时钟、文件系统等外部依赖通过接口注入，再使用 fake/mock 控制输入和错误结果，可减少测试偶发性。UI 测试要使用稳定平台插件和明确状态等待，避免固定 sleep 假设界面已经完成。构建时链接 Qt::Test，在 CI 中运行测试目标；测试只证明所覆盖行为，关键用户流程仍要在目标平台验证。

代码示例

```
QSignalSpy spy(&model, &Model::dataChanged);
model.setData(model.index(0, 0), QStringLiteral("new"));
QCOMPARE(spy.size(), 1); // 验证信号和状态
QTest::mouseClick(widget.submitButton(), Qt::LeftButton);
QCOMPARE(widget.statusText(), QStringLiteral("Done"));
// 把网络/时钟/文件系统依赖注入 fake，测试可重复
// CI 链接 Qt6::Test 并运行 QTest suite
```

046 如何自定义消息处理函数?

答案　qInstallMessageHandler() 可以替换 Qt 默认日志出口，把消息统一写到终端、文件或系统日志。处理器收到消息级别、类别、源码位置上下文和文本，可以附加时间戳或线程标识。安装后当前进程的 Qt 日志都会经过它，因此应让处理器行为简单、可预测。

处理器可能从多个线程并发进入，写共享文件要同步或使用线程安全队列。不要在 handler 里再次调用 qDebug 等 Qt 日志函数，否则会递归；也不要执行阻塞网络上传。应用退出时刷新文件并恢复旧 handler。记录之前要做凭据脱敏和文件轮转，磁盘写入失败也不能让整个进程失控。

代码示例

```
static QMutex outputMutex;
void handler(QtMsgType type, const QMessageLogContext& ctx, const QString& msg)
{
    QMutexLocker lock(&outputMutex); // 可能由多个线程调用
    appendWithTimestamp(type, ctx.category, msg.toUtf8());
    // 不在 handler 中再次调用 qDebug，避免递归
}
auto oldHandler = qInstallMessageHandler(handler);
// 退出时刷新并恢复 oldHandler；写入做轮转和隐私脱敏
```

047 如何解决 Qt 中文乱码问题?

答案　中文乱码通常发生在字节与文本转换边界，而不是 QString 本身。先确认源文件编码、网络协议编码、文件编码和终端编码。网络 QByteArray 应按协议明确用 fromUtf8 等方法解码，写入文件则通过 QTextStream 指定 UTF-8 或按格式要求编码。数据显示正常但导出异常时，逐段检查每次字节转换。

Qt 6 默认文本编码为 UTF-8；Qt 5 的本地 8 位编码接口及默认行为不同，不能把旧代码原样当作 UTF-8 处理。终端自身字体和代码页也会影响日志显示，文件里的正确字节可能只是终端呈现不对。不要通过重复转码“试到能看”为止，应该检查原始字节和协议约定。

代码示例

```
QByteArray incoming = socket->readAll(); // 网络层先得到字节
QString text = QString::fromUtf8(incoming); // 按协议编码解码
QFile file(path);
if (!file.open(QIODevice::WriteOnly | QIODevice::Text))
    return;
QTextStream out(&file);
out.setEncoding(QStringConverter::Utf8);
out << text; // 文件编码也明确为 UTF-8
qInfo().noquote() << text; // 日志终端另检查编码设置
// QString 是 Unicode 文本；不要把本地 8 位编码当 UTF-8
```

048 如何解决 Qt 多线程中的数据竞争问题?

答案　数据竞争意味着多个线程同时访问同一内存位置，其中至少一次是写入且没有同步。先确认哪些状态被共享，再优先让一个线程拥有可变对象，把其他线程结果复制成消息发给它。确实需要共享时，用同一把互斥量保护所有相关读写；只给部分代码加锁不能消除竞争。

QueuedConnection 只把槽调用排进接收者线程，不会自动锁住其他共享字段。QObject 的线程亲和性也不代表普通成员函数可同时跨线程调用；事件驱动对象应在所属线程使用，QWidget 只能由 GUI 线程操作。用 ThreadSanitizer 等工具帮助复现定位，修复后检查关闭、取消和对象销毁期间的并发路径。

代码示例

```
connect(worker,
        &Worker::resultReady,
        controller,
        &Controller::apply,
        Qt::QueuedConnection); // 消息传递优先
QMutexLocker lock(&stateMutex); // 所有访问共享状态的路径都持同锁
sharedState = nextState;
// 队列连接只安排槽执行，不保护另一个共享变量
```

```
// GUI 控件只在 GUI 线程访问；可改传不可变副本
// ThreadSanitizer/sanitizer 用于复现并定位竞争
```

049 如何解决 Qt 界面卡顿问题?

答案　界面卡顿要先定位主线程在做什么：CPU 密集计算、同步文件或网络 I/O、锁等待、布局重算还是过量重绘。Profiler 和线程栈能区分这些原因。独立耗时任务可移到 worker，结果通过信号回主线程；大列表用 Model/View，并批量更新数据和合并刷新，减少每条记录创建控件的成本。

setUpdatesEnabled(false) 只适用于很短的批量控件修改，结束后必须恢复；它不会让阻塞 I/O 变快，也不能解决后台线程竞争。避免把 processEvents 当作长期补丁。修复后用相同操作测首帧响应、滚动和更新延迟，确认数据通知与缓存失效仍正确。

代码示例

```
auto* watcher = new QFutureWatcher<Result>(this);
connect(watcher,
        &QFutureWatcher<Result>::finished,
        this,
        [this, watcher]
        {
            model->apply(watcher->result());
            watcher->deleteLater();
        });
watcher->setFuture(QtConcurrent::run(
    [input]
    {
        return crunch(input);
    }));
// 数据批量更新用 begin/end 模型通知；合并 update 区域
// setUpdatesEnabled(false) 只包围短批量界面修改并恢复
// profile 找到主线程热点后再优化
```

050 如何解决 Qt 程序假死的问题?

答案　程序假死通常是 GUI 线程没有返回事件循环：可能在做大计算、同步 I/O、等待锁或同步等待另一个线程。先用采样器和线程栈确认阻塞位置，再将计算提交到 QtConcurrent/QThread、把网络请求改成异步，或拆成可调度的小步骤。

把结果通过 queued signal 返回 GUI，并为长任务设计取消、超时和失败传播。processEvents() 在当前栈中嵌套处理事件，会让尚未完成的状态被其他回调观察，重入风险很高，不应掩盖持续阻塞。排查等待锁时也要分析等待环；只增加超时时间不等于解决死锁。

代码示例

```
QThreadPool::globalInstance()->start(
    [this]
    {
        const Result result = blockingIoOrCompute(); // 移出 GUI 事件回调
        QMetaObject::invokeMethod(
            this,
            [this, result]
            {
                apply(result); // 结果回 GUI 线程
            },
            Qt::QueuedConnection);
    });
// 任务要有取消/超时；分析线程栈定位锁、同步 I/O 或等待环
// processEvents 会重入，不应用来掩盖长期阻塞
```

051 如何解决 Qt 程序在高分屏下显示模糊的问题?

答案　高分屏模糊通常是逻辑尺寸、物理像素和图片资源倍率没有对应好。Qt 6 默认进行高 DPI 处理，布局尺寸通常按逻辑像素安排；像素缓冲图和图片资源需要提供合适分辨率，QImage 等还可用 devicePixelRatio 描述像素与逻辑尺寸关系。SVG 或多倍率资源有助于清晰缩放。

不要同时让 Qt 自动缩放又手工按倍率重复缩放，否则尺寸和绘制位置会错。窗口移到另一块屏幕后，应查询当前 QScreen 的设备像素比，不能永久假定主屏幕配置。Qt 5 的高 DPI 开关和行为依版本变化，迁移需核对目标版本。自绘内容应按逻辑坐标绘制，并在目标倍率下检查文字和细线。

代码示例

```
QScreen* screen = window->screen(); // 多屏移动后重新查询
const qreal dpr = screen ? screen->devicePixelRatio() : 1.0;
```

```
const QSize physical = (logicalSize * dpr).toSize();
QImage image(physical, QImage::Format_ARGB32_Premultiplied);
image.setDevicePixelRatio(dpr); // 布局仍使用逻辑像素
// 图片资源提供合适倍率或 SVG；避免手工重复缩放
// Qt 5 高 DPI 开关随版本变化，按目标版本确认
```

052 如何设计一个 Qt 项目的架构?

答案　Qt 项目可按界面、业务用例和数据/设备适配来划分职责。Widget 或 QML 负责展示和输入，业务逻辑尽量通过普通 C++ 接口完成，不直接依赖某个控件。Model/View 管理集合数据，日志、配置、网络和持久化通过明确接口提供能力；依赖注入让测试能替换真实外部服务。

模块数量应随实际变化点增长，不要先造一堆空壳抽象。界面切换时如果业务规则不变，就说明边界较合理；如果一个 UI 头文件传遍所有层，依赖过深。跨模块使用稳定值类型和少量接口，线程边界与对象所有权也要写清。项目架构是为了降低修改成本，不是追求图表层级。

代码示例

```
class ImportUseCase
{
    FileReader& reader;
    TaskRepository& repository;

  public:
    Result run(Path p)
    {
        return repository.save(reader.read(p));
    }
};
// UI调用用例；测试可注入FakeRepository，业务层不依赖QWidget/QML
```

053 如何避免不必要的信号槽连接?

答案　每次 connect() 通常都会创建一条连接，因此初始化函数重复调用会导致一次信号多次触发同一个槽。固定生命周期的连接应在构造或专用初始化流程中只建立一次；动态订阅则保存 QMetaObject::Connection，在结束时显式断开。Qt::UniqueConnection 可用于符合条件的成员函数连接。

UniqueConnection 不适用于 lambda/functor 的通用去重，因为 Qt 无法按任意闭包身份判断两次连接是否相同。此时由调用方保存连接句柄，按对象生命周期管理；若有大量条目状态变化，用模型和批量通知表达，不要给每行都建立大量临时信号连接。测试可统计一次信号是否产生预期次数。

代码示例

```
QMetaObject::Connection connection;
void Controller::startObserving()
{
    if (connection)
        return; // 固定初始化只连一次
    connection = connect(source, &Source::changed, this, &Controller::refresh);
}
void Controller::stopObserving()
{
    QObject::disconnect(connection);
    connection = {};
}
// UniqueConnection 适用于成员槽，不支持 lambda/functor 去重
// 动态生命周期可保存句柄并显式断开
```

054 如何配置 Qt 的静态编译?

答案　静态 Qt 要先按官方流程构建并安装一套 static Qt Kit，再让应用链接这套库；仅在项目文件加一个 static 标记不会把已有动态 Qt 自动变成静态库。静态平台插件、图像格式和其他模块可能需要显式导入，具体构建选项受 Qt 版本、平台与工具链影响。

静态构建通常让二进制更大，依赖和升级也更复杂。还需逐一确认第三方库是否可静态链接、平台插件注册方式以及 Qt 许可义务；交付形式不同会影响许可要求，应查当前官方许可文本。生成后在干净环境测试平台插件、资源和多媒体功能，并保留符号文件以支持崩溃诊断。

代码示例

```
find_package(Qt6 REQUIRED COMPONENTS Core Widgets)
    target_link_libraries(app PRIVATE Qt6::Core Qt6::Widgets)
    // 静态Qt需匹配工具链并保留静态插件导入；验证许可证与最终依赖
```

055 智能指针在 Qt 中如何使用?

答案　智能指针帮助表达 C++ 对象所有权，但 Qt QObject 还拥有自己的父子对象树机制。QScopedPointer 或 std::unique_ptr 适合单一所有者，QSharedPointer/std::shared_ptr 用于确有共享生命周期的普通对象；对应弱指针可观察共享对象而不增加强引用，打破循环。

QPointer 是 QObject 的观察指针：目标销毁时会自动变空，但它不拥有或延长对象寿命。QObject 通常由 parent 管理，或由明确的单一智能指针管理，不能同时交给 parent 和 shared_ptr，否则可能双重释放。跨线程对象还应按所属线程销毁，必要时通过 deleteLater，而不是任意线程直接 delete。

代码示例

```
auto owned = std::make_unique<PlainResource>(); // 普通 C++ 资源独占
QPointer<QObject> observer = object; // 只观察 QObject，不拥有
if (observer)
    observer->deleteLater(); // 由对象线程事件循环处理
QSharedPointer<Cache> shared = QSharedPointer<Cache>::create(); // 共享非 QObject 数据
QWeakPointer<Cache> weak = shared.toWeakRef(); // 可观察而不形成强引用环
// QObject 用 parent 或单一智能指针所有权，不能两者都拥有
```

第 15 章 OpenCV 与计算机视觉

15.1 图像基础与常用处理

001 Canny 边缘检测

答案　Canny 用来从灰度图中提取边缘。通常先用高斯模糊压制噪声，再根据像素梯度找出亮度变化强的位置；非极大值抑制会让粗响应只保留细线上的局部峰值。这样得到的候选边缘再经过双阈值处理：高于高阈值的是强边缘，介于两阈值之间的弱边缘只有连接到强边缘才会保留。

低阈值和高阈值控制召回与噪点，不能照搬固定数值到所有图像。低对比场景可降低阈值，但也可能引入纹理噪声；模糊过强则会抹掉细小边界。输出是单通道边缘掩码，不是语义轮廓，若需要对象边界还要结合二值化、轮廓提取或后处理。

代码示例

```
cv::Mat gray = cv::imread(path, cv::IMREAD_GRAYSCALE);
if (gray.empty())
    return reportReadFailure();
cv::Mat smooth, edges;
cv::GaussianBlur(gray, smooth, cv::Size(5, 5), 0); // 先抑制噪声
cv::Canny(smooth, edges, lowThreshold, highThreshold);
show(edges);
// 高阈值找强边缘，低阈值参与滞后连接；按图像对比度调参
```

002 Haar 级联分类器进行人脸检测

答案　Haar 级联把人脸检测做成多尺度窗口扫描。加载经过训练的 XML 分类器后，将图像转成灰度；必要时均衡直方图，再调用 detectMultiScale 返回候选矩形。scaleFactor 决定每一层缩小多少，数值越接近 1 通常检查的尺度越密、速度越慢；minNeighbors 控制候选框需要多少相邻支持才保留。

这种方法运行快、无需 GPU，但模型和环境变化会明显影响召回率，侧脸、遮挡、强逆光或非典型人脸可能漏检。均衡直方图会改变对比度，也可能放大噪声，应通过样例判断是否有益。矩形只表示检测器认为可能有人脸，不等于身份识别或可靠的安全认证。

代码示例

```
cv::CascadeClassifier detector;
if (!detector.load(cascadeXml) || detector.empty())
    return reportModelError();
cv::Mat gray, equalized;
cv::cvtColor(frame, gray, cv::COLOR_BGR2GRAY);
cv::equalizeHist(gray, equalized); // 光照允许时增强局部对比
std::vector<cv::Rect> faces;
detector.detectMultiScale(equalized, faces, 1.1, 3, 0, cv::Size(30, 30));
for (const auto& face : faces)
    cv::rectangle(frame, face, cv::Scalar(0, 255, 0), 2);
// scaleFactor/minNeighbors 改变速度与候选框过滤强度
```

003 Mat 和 IplImage 的区别

答案　IplImage 是 OpenCV 旧 C API 的图像结构，通常需要手动申请和释放，调用者还要关注通道数、数据指针和行步长。cv::Mat 是现代 C++ 矩阵类型，析构时以引用计数管理缓冲区，并支持 ROI、类型化访问和大量算法接口。旧代码逐步迁移时可用 cvarrToMat 把旧对象接入 Mat API。

Mat 的普通复制和赋值通常只复制矩阵头并共享像素；如果只读可共享，若需要独立修改或异步保存，就用 clone() 取得自己的数据。把 cvarrToMat 得到的 Mat 当成独立拥有的数据会产生生命周期问题，因为底层可能仍由 IplImage 所有。释放旧图像前先完成借用操作，或先 clone。

代码示例

```
IplImage* legacy = cvLoadImage(path, CV_LOAD_IMAGE_COLOR);
if (!legacy)
    return reportReadFailure();
cv::Mat borrowed = cv::cvarrToMat(legacy); // 仍依赖 legacy 的数据生命周期
cv::Mat owned = borrowed.clone(); // 复制像素后才可独立使用
cvReleaseImage(&legacy); // 旧 C API 必须显式释放
process(owned); // cv::Mat 析构按引用计数回收自己的缓冲区
// 新代码优先用 Mat；ROI/普通复制仍可能共享像素
```

004 OpenCV 如何读写图像并进行 BGR 灰度转换?

答案　imread 用文件名和读取标志得到 cv::Mat，彩色读取默认通道顺序是 BGR，而不是 RGB。灰度转换应使用 COLOR_BGR2GRAY；只有明确需要 RGB 顺序时才转换。imshow 可显示图像，waitKey 会处理 HighGUI 窗口事件，因此窗口程序要周期调用它；无桌面环境或无 GUI 构建中，这条显示路径可能不可用。

写文件时 imwrite 根据扩展名选择编码器，失败会返回 false，例如目录不可写或格式支持缺失。应在读图后检查 empty()，也要检查输出结果。若图像来自摄像头或网络，颜色通道和步长可能另有约定。不要把 BGR 数组直接当 RGB 给其他库，否则颜色会互换；跨库时明确转换一次。

代码示例

```
cv::Mat bgr = cv::imread(path, cv::IMREAD_COLOR); // 默认 BGR
if (bgr.empty())
    return reportReadFailure();
cv::Mat gray;
cv::cvtColor(bgr, gray, cv::COLOR_BGR2GRAY);
cv::imshow("gray", gray);
const int key = cv::waitKey(1); // 驱动 HighGUI 窗口事件
if (key == 'q')
    closeWindow();
if (!cv::imwrite(outputPath, gray))
    reportWriteFailure();
// 无桌面环境不要依赖 imshow；编码器由扩展名选择
```

005 OpenCV 用什么语言编写？支持哪些平台?

答案　OpenCV 核心主要用 C++ 实现，C++ API 覆盖最广，常见官方绑定还包括 Python 和 Java；其他语言接口可能来自额外绑定或特定构建。它可以在桌面、移动和嵌入式环境使用，平台范围包括 Windows、Linux、macOS、Android 与 iOS 等。

“能编译运行”不等于每个平台都包含相同模块和后端。摄像头、硬件编解码、CUDA、GUI 窗口和第三方算法取决于构建选项、驱动、编译器和系统依赖。部署前应查看实际 OpenCV build information，并在目标设备验证所需 API；不要仅依据官网支持平台列表推断某个硬件加速功能可用。

代码示例

```
std::cout << cv::getVersionString() << '\n' << cv::getBuildInformation();
auto image = cv::imread(test_path);
if (image.empty())
    return input_error();
// 构建信息应确认所需模块、摄像头后端和CUDA支持实际存在
```

006 SIFT、SURF 和 ORB 的区别

答案　SIFT、SURF 和 ORB 都通过关键点及其周围外观描述来匹配图像。SIFT 在尺度空间寻找稳定点，常生成 128 维浮点描述子，通常用 L2 距离；SURF 也是尺度不变特征路线，在部分 OpenCV 发行版中由 opencv_contrib 的 xfeatures2d 提供。ORB 用 FAST 选点并计算方向化 BRIEF 二进制描述子，通常更省内存、速度较快。

二进制 ORB 描述子默认适合 Hamming 距离；若配置 WTA_K 为 3 或 4，应使用 Hamming2。特征数量、图像尺度、模糊和重复纹理都会影响结果。三者输出的距离不可直接比较，匹配阈值需按描述子类型和任务验证；SURF 的可用性还受版本、构建和许可政策影响。

代码示例

```
auto sift = cv::SIFT::create(); // 通常产生浮点描述子
auto orb = cv::ORB::create(500); // 二进制描述子
std::vector<cv::KeyPoint> ks, ko;
cv::Mat ds, dorb;
sift->detectAndCompute(gray, cv::noArray(), ks, ds);
orb->detectAndCompute(gray, cv::noArray(), ko, dorb);
```

```
cv::BFMatcher l2(cv::NORM_L2), hamming(cv::NORM_HAMMING);
l2.match(ds, siftMatches);
hamming.match(dorb, orbMatches);
// SURF 需对应 contrib/build 配置；算法效果还受尺度和模糊影响
```

007 cv::findContours() 的作用 & 绘制轮廓

答案　findContours 从单通道二值图中沿前景像素边界追踪点集；前景通常是非零、背景为零。返回的每条轮廓可用于求面积、周长、外接框或形状特征。RETR_EXTERNAL 只保留最外层轮廓，RETR_TREE 还记录嵌套父子关系；CHAIN_APPROX_SIMPLE 会压缩直线上的冗余点，节省空间。

OpenCV 3.2 起 findContours 通常不再修改输入，但如需保留原始掩码或兼顾旧版本，传入 clone 副本更稳妥。轮廓不是自动识别出的物体语义；噪点、断边和二值化质量都会影响结果。绘制可用 drawContours，层级模式下还应按层级关系选择填充或只画某个子轮廓。

代码示例

```
cv::Mat work = binary.clone(); // 保留原掩码，兼顾版本差异
std::vector<std::vector<cv::Point>> contours;
std::vector<cv::Vec4i> hierarchy;
cv::findContours(work, contours, hierarchy, cv::RETR_TREE, cv::CHAIN_APPROX_SIMPLE);
for (std::size_t i = 0; i < contours.size(); ++i)
{
    const double area = cv::contourArea(contours[i]);
    if (area > minArea)
        cv::drawContours(canvas, contours, static_cast<int>(i), cv::Scalar(0, 255, 0), 2);
}
// RETR_TREE 保留层级；外轮廓场景可用 RETR_EXTERNAL
```

008 cv::split() 与 cv::merge()

答案　split 会把多通道图像拆成若干单通道矩阵，例如 BGR 拆成蓝、绿、红三个平面；merge 按给定顺序将兼容的矩阵组合起来。它适用于对某个通道独立处理，例如只均衡亮度或将红通道置零。OpenCV 的常见三通道彩色图顺序是 BGR，要根据实际格式确定通道索引。

拆分和合并会产生额外内存访问和临时矩阵，处理单像素时直接用 Vec3b，逐行处理则可用 ptr 更合适。merge 的各通道必须尺寸和类型匹配。对 HSV、Lab 等颜色空间，通道含义与范围不同，不能把“第一个通道”总当作亮度；若只是颜色空间转换，使用 cvtColor 更明确。

代码示例

```
std::vector<cv::Mat> channels;
cv::split(bgr, channels); // B、G、R 单通道矩阵
channels[2] = cv::Mat::zeros(bgr.size(), CV_8UC1); // 仅保留 B/G
cv::Mat merged;
cv::merge(channels, merged); // 尺寸、深度必须兼容
// 额外 split/merge 会占带宽和内存
cv::Vec3b pixel = bgr.at<cv::Vec3b>(y, x); // 单点访问可直接读通道
// OpenCV 彩色顺序通常是 BGR
```

15.2 特征、变换与视觉算法

009 什么是 OpenCV？它的主要特性有哪些？

答案　OpenCV 是开源的计算机视觉与图像处理库，提供从 Mat 数据容器和几何变换到滤波、阈值、形态学等基础操作。常用模块还覆盖视频读写、特征检测、相机标定、传统机器学习和 DNN 推理。它提供的是算法与数据处理能力，不会自动替应用决定检测目标、参数或业务策略。

C++ 和 Python 常用于组合流程。CPU 优化是常见路径，但 CUDA 等加速需要相应模块、构建配置和硬件，且并非每个函数都有 GPU 实现；Mat 与 GpuMat 之间传输也有成本。评估方案时应检查构建信息、输入格式、精度和目标设备性能，不能仅凭“使用 OpenCV”推断实时或准确。

代码示例

```
auto image = cv::imread(path);
if (image.empty())
    return error("image load failed");
cv::Mat gray, blurred, mask;
cv::cvtColor(image, gray, cv::COLOR_BGR2GRAY);
cv::GaussianBlur(gray, blurred, {5, 5}, 0);
cv::threshold(blurred, mask, 128, 255, cv::THRESH_BINARY);
```

010 光流 (Optical Flow) & Farneback 方法

答案　光流表示连续两帧之间图像纹理的表观位移，不一定等于真实物体的三维运动。Farneback 在局部邻域用多项式近似亮度变化，再估计位移，因此输出稠密场：每个像素位置有水平和垂直两个浮点分量。两帧须尺寸一致，常先转灰度并保留上一帧。

它依赖相邻帧外观相对稳定的假设；快速运动、遮挡、弱纹理、运动模糊和亮度突变都会造成误差。金字塔参数和窗口大小影响可处理运动范围与细节，需根据帧率调试。输出的 flow 是位移，不是速度；若要速度，还要用实际帧时间换算，并过滤不可信区域。

代码示例

```
cv::Mat flow;
cv::calcOpticalFlowFarneback(previousGray, currentGray, flow, 0.5, 3, 15, 3, 5, 1.2, 0);
// flow 为 CV_32FC2；两个分量分别是水平/垂直位移
const cv::Point2f motion = flow.at<cv::Point2f>(y, x);
drawArrow(frame, cv::Point(x, y), cv::Point2f(x, y) + motion);
// 两帧尺寸应一致；弱纹理、遮挡和快速运动会破坏局部假设
previousGray = currentGray.clone(); // 为下一次保留独立帧
```

011 单应性矩阵 (Homography) & 计算

答案　单应性矩阵是 3×3 投影变换，齐次坐标中满足目标点约等于 H 乘源点，再除以齐次尺度。它可描述平面场景从一个视角到另一个视角的映射，也适用于相机纯旋转近似。理论上至少需要四对不共线的对应点；真实特征匹配常含误配，使用 RANSAC 可筛出一致内点再估矩阵。

用 warpPerspective 时要确认 H 的方向与输入、输出坐标定义一致；矩阵空或内点过少时不应继续变换。若场景有明显深度差和相机平移，不同物体的视差不同，单一 H 无法同时对齐，拼接会出现重影。还要计算变换后边界和输出画布，避免结果落在负坐标或被裁切。

代码示例

```
cv::Mat inliers;
cv::Mat H = cv::findHomography(srcPoints, dstPoints, cv::RANSAC, 3.0, inliers);
if (H.empty())
    return reportDegenerateMatches();
cv::Mat warped;
cv::warpPerspective(source, warped, H, outputSize, cv::INTER_LINEAR, cv::BORDER_CONSTANT);
// 至少四对不退化对应点；mask 标出 RANSAC 内点
// 单一 H 不适用于明显深度视差场景
```

012 图像拼接 (Image Stitching)

答案　图像拼接先在两张图中检测关键点、计算描述子并匹配，再用距离比等规则剔除歧义匹配。保留的对应点经 RANSAC 求单应性，把一张图映射到另一张图坐标系。拼接前要投影四角，计算完整画布边界并加平移偏置，否则负坐标区域会被裁掉。

重叠区域需要融合：曝光补偿可减轻整体亮度差，羽化或多频带融合可缓和接缝。ORB 快但不适合所有尺度变化，匹配质量决定最终几何效果。单应性主要适用于近似平面或相机绕光心旋转的场景；近景视差、动态物体和视角变化大时会产生重影或断裂，需要其他拼接模型或深度信息。

代码示例

```
auto orb = cv::ORB::create(1500);
orb->detectAndCompute(leftGray, cv::noArray(), leftKeys, leftDesc);
orb->detectAndCompute(rightGray, cv::noArray(), rightKeys, rightDesc);
cv::BFMatcher(cv::NORM_HAMMING).knnMatch(rightDesc, leftDesc, pairs, 2);
filterRatioMatches(pairs, good); // 比率检验减少歧义匹配
cv::Mat H = cv::findHomography(rightPoints, leftPoints, cv::RANSAC, 3.0, inliers);
if (H.empty())
    return reportNoAlignment();
cv::warpPerspective(rightImage, warped, H, canvasSize); // 先计算平移画布边界
blendOverlap(leftImage, warped); // 羽化/曝光补偿缓解接缝
// 平面/纯旋转较适用；近景视差会重影
```

013 图像旋转

答案　getRotationMatrix2D 生成绕指定中心旋转并缩放的 2×3 仿射矩阵，再由 warpAffine 对像素重采样。角度正负方向要结合图像坐标系确认；使用原图宽高作为输出尺寸时，旋转后的四角可能超出画布而被裁切。

要保留整张图，需根据角度的正弦和余弦计算旋转后包围盒宽高，并把矩阵平移到新画布中心。插值方式会影响边缘清晰度，边界外的像素则由 borderMode 和填充值决定。多次连续旋转会反复插值损失画质，应尽可能从原图一次变换到最终角度。

代码示例

```
const cv::Point2f center(image.cols / 2.0f, image.rows / 2.0f);
cv::Mat M = cv::getRotationMatrix2D(center, angleDegrees, 1.0);
const double radians = angleDegrees * CV_PI / 180.0;
const int outW = static_cast<int>(
    std::ceil(std::abs(image.cols * std::cos(radians)) + std::abs(image.rows * std::sin(radians))));
const int outH = static_cast<int>(
    std::ceil(std::abs(image.rows * std::cos(radians)) + std::abs(image.cols * std::sin(radians))));
M.at<double>(0, 2) += outW / 2.0 - center.x;
M.at<double>(1, 2) += outH / 2.0 - center.y;
cv::warpAffine(image, rotated, M, cv::Size(outW, outH)); // 平移后保留完整画布
```

014 图像金字塔 (Image Pyramid)

答案　图像金字塔表示同一图像的多种分辨率。高斯金字塔每层先用低通滤波抑制高频，再下采样，常用于多尺度检测或粗到细配准。拉普拉斯金字塔保存相邻高斯层之间丢失的细节：将较小层上采样后，从当前高斯层减去它，得到这一层的高频残差。

重建时从最粗层开始逐层上采样并加回对应残差。金字塔融合可在不同尺度平滑混合两幅图，减轻简单硬切换造成的接缝。每层尺寸可能因奇数宽高而取整，重建时需使用原层尺寸和一致边界规则；否则加法尺寸不匹配或图像边缘出现伪影。

代码示例

```
std::vector<cv::Mat> gaussian{image};
for (int level = 0; level < levels; ++level)
{
    cv::Mat next;
    cv::pyrDown(gaussian.back(), next);
    gaussian.push_back(next);
}
std::vector<cv::Mat> laplacian;
for (std::size_t i = 0; i + 1 < gaussian.size(); ++i)
{
    cv::Mat up, detail;
    cv::pyrUp(gaussian[i + 1], up, gaussian[i].size());
    cv::subtract(gaussian[i], up, detail);
    laplacian.push_back(detail);
}
// 重建需用相同尺寸取整与边界规则逐层上采样后加回
```

015 图像阈值化 / 二值化

答案　固定阈值把灰度像素独立分成前景和背景。THRESH_BINARY 规则是像素值大于阈值时写 maxval，否则写零，因此等于阈值也进入零侧。它简单、快速，适合光照较均匀且前景背景分离明显的图像；输出掩码通常是 0 和 255。照明不均时，可以在邻域内计算自适应阈值，让不同区域使用不同亮度基准；若灰度直方图近似双峰，可尝试 Otsu 自动选择全局阈值。Otsu 并不能修复所有光照问题，也需要单通道输入。标签图、分割类别图缩放时要用最近邻插值，避免生成原本不存在的类别值。

代码示例

```
cv::Mat binary;
const double used = cv::threshold(gray, binary, 127, 255, cv::THRESH_BINARY);
// 大于 127 的像素变 255；等于阈值及更小值变 0
cv::threshold(gray, otsu, 0, 255, cv::THRESH_BINARY | cv::THRESH_OTSU);
// Otsu 根据直方图选阈值，仍要求单通道输入
// 光照不均可改 adaptiveThreshold；标签图不要线性插值
```

016 如何使用 ORB 进行特征检测

答案　ORB 同时检测关键点并计算方向化 BRIEF 二进制描述子。通常输入灰度图和可选掩码，detectAndCompute 返回关键点位置及描述子矩阵；maxFeatures 控制最多保留数量，scaleFactor 与 nlevels 决定尺度金字塔。匹配两张图时使用 BFMatcher 和 Hamming 距离，再用近邻距离比或其他几何一致性筛选误配。

ORB 通常速度快、描述子紧凑，但运动模糊、重复纹理和大尺度变化会降低稳定性。默认 WTA_K=2 时使用 NORM_HAMMING；若 WTA_K=3 或 4 则应选择 NORM_HAMMING2。仅按描述距离过滤不能证明匹配正确，配准任务还应使用 RANSAC 检查空间几何关系。

代码示例

```
auto orb = cv::ORB::create(1000, 1.2f, 8);
std::vector<cv::KeyPoint> keypoints;
cv::Mat descriptors;
orb->detectAndCompute(gray, mask, keypoints, descriptors);
```

```
cv::BFMatcher matcher(cv::NORM_HAMMING);
matcher.knnMatch(descriptors, referenceDescriptors, pairs, 2);
for (const auto& pair : pairs)
    if (pair.size() == 2 && pair[0].distance < 0.75f * pair[1].distance)
        good.push_back(pair[0]);
// ORB 描述子是二进制串，匹配用 Hamming 距离
```

017 如何调整图像大小

答案　resize 可通过目标宽高或缩放比例改变图像尺寸。缩小时常用 INTER_AREA 综合原像素覆盖，通常比直接抽样更能减少混叠；放大常用 INTER_LINEAR，INTER_CUBIC 会使用更多邻域像素，可能更平滑但计算更贵。输入图像应先检查非空，输出尺寸也须大于零。

保持宽高比时按同一倍率计算宽和高，再决定补边还是裁剪；直接强行设定目标宽高会拉伸变形。分割标签和二值掩码应使用 INTER_NEAREST，以免插值产生新的类别或灰度值。Mat 的行可能有 stride，外部图像缓冲与视频帧尤其不能假定连续。对视频逐帧缩放也要关注耗时预算。

代码示例

```
if (source.empty())
    return reportReadFailure();
cv::Mat smaller;
cv::resize(source, smaller, cv::Size(), 0.5, 0.5, cv::INTER_AREA); // 缩小
cv::Mat larger;
cv::resize(source, larger, cv::Size(), 2.0, 2.0, cv::INTER_CUBIC); // 放大
// 相同比例保持宽高比；标签/掩码使用 INTER_NEAREST
// resize 输出是 Mat，逐行访问仍须尊重 step/stride
```

018 模板匹配 (Template Matching)

答案　matchTemplate 在源图上滑动模板，并为每个候选位置计算一个匹配响应，结果尺寸通常是搜索图减去模板后再加一。使用相关度类方法时分数越大通常越好；平方差类则取最小值。minMaxLoc 可找出极值位置，匹配框左上角就是该位置，框大小与模板相同。

模板必须不大于搜索图，颜色通道和类型也应匹配。该算法默认不具备尺度或旋转不变性，对亮度和遮挡也可能敏感；可先裁小 ROI、归一化或做图像金字塔。重复纹理可能出现多个高分位置，应设置信心阈值并做非极大值抑制，避免把多个重叠候选都当成目标。

代码示例

```
if (templ.cols > image.cols || templ.rows > image.rows)
    return;
cv::Mat response;
cv::matchTemplate(image, templ, response, cv::TM_CCOEFF_NORMED);
double minScore = 0.0, maxScore = 0.0;
cv::Point minLoc, maxLoc;
cv::minMaxLoc(response, &minScore, &maxScore, &minLoc, &maxLoc);
const cv::Point best = maxLoc; // 相关度方法取最大；平方差取最小
cv::rectangle(image, best, best + templ.size(), cv::Scalar(0, 255, 0), 2);
// 尺度/旋转不天然鲁棒；可先限制 ROI 或建金字塔
```

15.3 视频分析与相机标定

019 直方图均衡化

答案　equalizeHist 先统计单通道 8 位灰度直方图，再按累计分布重映射像素，使原本集中在窄亮度范围的值展开。它可能让低对比细节更明显，但也会放大噪声、改变整体观感，结果不一定更自然。输入必须符合该 API 对通道数和深度的要求。

彩色图像不要分别均衡 B、G、R 三个通道，否则颜色比例变化容易偏色。可转 YCrCb 或 Lab，在亮度通道做均衡后转回。局部照明差异较大时，CLAHE 在分块基础上处理并限制对比度，能避免某些局部过度增强，但分块边界和参数仍要检查。增强仅改善显示/后续算法输入，不会恢复传感器未记录的信息。

代码示例

```
cv::Mat equalized;
cv::equalizeHist(gray, equalized); // 仅单通道 8-bit 灰度
cv::Mat ycrcb;
cv::cvtColor(bgr, ycrcb, cv::COLOR_BGR2YCrCb);
std::vector<cv::Mat> planes;
cv::split(ycrcb, planes);
cv::equalizeHist(planes[0], planes[0]); // 仅均衡亮度 Y
cv::merge(planes, ycrcb);
cv::Mat result;
```

```
cv::cvtColor(ycrcb, result, cv::COLOR_YCrCb2BGR);
// 彩色直接均衡 B/G/R 容易偏色；强光差可用 CLAHE
```

020 视频中颜色跟踪

答案　颜色跟踪通常逐帧把 BGR 转到 HSV，再用 inRange 根据色相、饱和度和明度生成二值掩码。形态学开运算可去小噪点，闭运算可填小孔；轮廓面积过滤后，可用图像矩估计目标质心，再把位置传给后续控制逻辑。

HSV 把色相与亮度分开一些，但对不同光源、阴影和相机白平衡仍敏感，需要实拍调范围。红色的色相值位于 0 度附近的两端，单一区间无法覆盖，需分两次阈值并 OR 合并。相邻帧可能检测到多个相似目标，应再做最近邻、卡尔曼滤波或其他身份关联，不能只选最大轮廓就假设一直是同一物体。

代码示例

```
cv::VideoCapture capture(device);
if (!capture.isOpened())
    return reportCameraFailure();
while (capture.read(frame))
{
    cv::cvtColor(frame, hsv, cv::COLOR_BGR2HSV);
    cv::inRange(hsv, lowerHsv, upperHsv, mask);
    cv::morphologyEx(mask, clean, cv::MORPH_OPEN, kernel); // 去小噪点
    cv::findContours(clean, contours, cv::RETR_EXTERNAL, cv::CHAIN_APPROX_SIMPLE);
    selectLargestAndUpdateCentroid(contours); // 面积筛选/跨帧关联
}
// 红色 Hue 跨首尾范围时分两次 inRange 后 OR 合并
```

021 透视变换 (Perspective Transform)

答案　透视变换用四对对应点求 3×3 单应矩阵，再由 warpPerspective 按矩阵把源图采样到目标画布。四个点的顺序必须一一对应，例如左上、右上、右下、左下；点重复或退化共线会导致解不稳定。目标尺寸、插值方法和边界填充值都应由输出用途指定。

如果只有四对精确角点，可用 getPerspectiveTransform；若有更多带噪声匹配点，应使用 findHomography 并配合 RANSAC 抑制外点。矩阵方向要和期望映射一致，求出的源到目标变换在输出像素采样时通常由函数内部处理逆映射。若输出尺寸不包含变换后范围，结果会被裁切；灰度、颜色和类别掩码也应选相应插值。

代码示例

```
const std::vector<cv::Point2f> sourceCorners{a, b, c, d};
const std::vector<cv::Point2f> targetCorners{p, q, r, s}; // 顺序一一对应
cv::Mat H = cv::getPerspectiveTransform(sourceCorners, targetCorners);
if (H.empty())
    return reportDegenerateCorners();
cv::warpPerspective(
    source, output, H, outputSize, cv::INTER_LINEAR, cv::BORDER_CONSTANT, cv::Scalar());
// 点不可退化/共线；多于四对含噪点改 findHomography + RANSAC
// 输出尺寸和边界填充值按目标画布设置
```

022 高斯模糊

答案　GaussianBlur 用高斯核对每个像素邻域加权平均，削弱高频噪声，常作为边缘检测和阈值处理前的预平滑步骤。核尺寸常用正奇数以有明确中心；sigmaX 为零时由核尺寸计算，sigmaY 为零则沿用 sigmaX。核越大或 sigma 越高，平滑越强，同时细节和边缘也越容易丢失。

边界像素周围缺少完整邻域，borderType 决定如何延伸边缘，例如复制边界或镜像。平滑参数要与噪声尺度匹配，过度模糊会让相邻物体边界融合。输入为空或通道类型异常时应提前处理。模糊并不会去除所有噪声，周期纹理、脉冲噪声等可能需要其他滤波器。

代码示例

```
cv::Mat blurred;
cv::GaussianBlur(source, blurred, cv::Size(5, 5), 0, 0, cv::BORDER_DEFAULT);
// 奇数核宽高；sigmaX 为 0 时由核尺寸推导
// sigmaY 为 0 时沿用 sigmaX
cv::Canny(blurred, edges, low, high); // 平滑降低噪声误检
// 核越大平滑越强，也会损失边缘细节；检查输入为空
```

023 cv::Mat 的拷贝构造和 clone() 有什么区别?

答案　Mat 的拷贝构造或赋值通常只复制矩阵头并增加引用计数，两个 Mat 视图仍指向同一片像素；通过任一视图改写像素，另一视图也会观察到变化。ROI 也是共享底层数据的头部视图。clone() 则分配新存储并复制像素，得到独立图像，适合需要隔离修改或异步保存的场景。〔S14〕

改 Mat 变量本身的头部信息不会自动深拷贝像素。引用计数管理内存寿命，却不保证并发写入安全；两个线程同时修改共享像素仍需锁或各自拥有 clone。传入异步队列前应确认消费者是否只读，以及源缓冲是否可能复用。若只需临时查看，可共享以节省内存；是否复制应由所有权和线程边界决定。〔S14〕

代码示例

```
cv::Mat shared = image; // 浅拷贝矩阵头，像素共享
shared.at<cv::Vec3b>(0, 0)[0] = 255; // 修改会影响 image
cv::Mat independent = image.clone(); // 分配并复制像素
independent.at<cv::Vec3b>(0, 0)[0] = 0; // 原图不变
enqueueForWorker(independent); // 异步/跨线程独立修改更安全
// 只改 Mat 头的赋值不会深拷贝；并发写仍要同步
```

024 OpenCV 的 ROI 为什么可能不是连续内存？处理时要注意什么？

答案　ROI 是 Mat 的矩形视图，通常只新建矩阵头，不会把矩形内容逐行复制。它保留父图的行步长 step，因此 ROI 每行起点之间可能隔着右侧未包含的父图像素；这种矩阵就是非连续内存。用 ptr(row) 逐行访问最稳妥，或检查 isContinuous() 后再决定能否把所有元素视作一段线性数组。〔S14〕

不能仅按 cols 推算下一行地址；应使用 step 或 OpenCV 提供的行指针。需要独立且连续的数据时调用 clone()，但会付出内存和复制时间。ROI 仍依赖父图像素生命周期，父图释放前 Mat 引用计数会保持底层存储有效；但跨线程写入共享 ROI 仍需同步。〔S14〕

代码示例

```
cv::Mat roi = image(rect); // 通常仅新建头，仍沿用父图 stride
for (int y = 0; y < roi.rows; ++y)
{
    const auto* row = roi.ptr<std::uint8_t>(y);
    processRow(row, static_cast<std::size_t>(roi.cols) * roi.elemSize());
}
if (!roi.isContinuous())
    handleRowsSeparately(); // 不可当成单段内存
cv::Mat packed = roi.clone(); // 需要连续独立存储时付出一次复制
// step 描述行间距，不能只用 cols 推出下行地址
```

025 相机标定要估计哪些参数？基本流程是什么？

答案　相机标定要估计内参矩阵中的 fx、fy、cx、cy，以及描述镜头径向和切向畸变的系数；同时还会为每张标定图求相机相对标定板的旋转和平移。常用棋盘格或 Charuco 板，已知板上三维角点，检测它们在图像中的二维位置，再联合拟合相机参数。〔S14〕

应从不同距离、倾角和画面位置拍摄多张清晰图像，让角点覆盖整个视野；重复近似姿态或模糊图会降低估计质量。calibrateCamera 返回重投影误差，但单看一个平均数不够，还要可视化残差并剔除异常视图。之后用 undistort 或预计算映射校正图像。标定板物理尺寸决定外部平移尺度，内参则更依赖图像覆盖和多样姿态。〔S14〕

代码示例

```
// 每个视角保存同一标定板的已知点与检测角点
std::vector<std::vector<cv::Point3f>> objectPoints;
std::vector<std::vector<cv::Point2f>> imagePoints;
cv::Mat cameraMatrix, distortion;
std::vector<cv::Mat> rvecs, tvecs;
const double rms = cv::calibrateCamera(
    objectPoints, imagePoints, imageSize, cameraMatrix, distortion, rvecs, tvecs);
if (!std::isfinite(rms))
    return reportCalibrationFailure();
cv::undistort(testImage, corrected, cameraMatrix, distortion);
// 多姿态覆盖全画面；模糊/重复视角剔除
```

第 16 章 音视频与 FFmpeg

16.1 容器、编码与流媒体协议

001 AAC 编码的 ADTS 和 ADIF 格式有什么区别？

答案　ADTS 和 ADIF 是 AAC 码流的两种封装方式，不代表两种不同的音质。ADTS 通常在每个 AAC access unit 前放同步字、采样率索引、声道配置和帧长度等信息，因此接收端可逐帧扫描并从流中恢复边界，常用于广播或实时传输。ADIF 把头部信息集中在文件开头，适合从头读取的完整文件；若从中间开始接收，就缺少逐帧同步信息。封装结构和文件扩展名不能互相替代，转换时应由正确的解析器或复用器处理。仅凭 ADTS/ADIF 也无法判断 AAC 的采样率、码率和主观质量。

代码示例

```
AVPacket* packet = read_access_unit(input);
write_adts_header(output, packet->size, sample_rate, channels);
write_packet(output, packet);
// ADTS适合逐帧传输；文件容器应按目标封装写入索引与元数据
```

002 AVFormatContext、AVCodecContext、AVStream 的作用分别是什么？

答案　AVFormatContext 表示容器输入或输出，负责文件 I/O、时长和多条媒体流的集合；AVStream 表示其中一条轨道，含时间基、编解码参数等描述；AVCodecContext 则是具体编码器或解码器的工作状态和运行配置。它们分别对应容器、轨道元数据和编解码过程，职责不能混为一谈。

解码时通常先从 AVStream 的 codecpar 复制参数到新建的 AVCodecContext，再打开解码器。packet 按 stream_index 找到对应的轨道和解码上下文；每条音视频轨通常各有一个 codec context。成功打开输入后，所有错误出口都要关闭 format context；分配的 codec context 也需单独释放。

代码示例

```
AVFormatContext* format = nullptr;
if (avformat_open_input(&format, path, nullptr, nullptr) < 0)
    return;
if (avformat_find_stream_info(format, nullptr) < 0)
{
    avformat_close_input(&format);
    return reportProbeError();
}
int index = av_find_best_stream(format, AVMEDIA_TYPE_VIDEO, -1, -1, nullptr, 0);
if (index >= 0)
{
    AVStream* stream = format->streams[index]; // 容器中的轨道/时间基
    AVCodecContext* codec = avcodec_alloc_context3(decoder);
    avcodec_parameters_to_context(codec, stream->codecpar); // 配置解码状态
    if (avcodec_open2(codec, decoder, nullptr) >= 0)
        decode(codec);
    avcodec_free_context(&codec);
}
avformat_close_input(&format); // 成功打开后的所有出口都关闭
```

003 AVPacket 和 AVFrame 有什么区别？分别存储什么数据？

答案　AVPacket 通常承载压缩后的码流数据，以及所属 stream_index、PTS 和 DTS 等包级信息；AVFrame 承载解码后的图像平面或音频样本，编码器也把待编码媒体作为 frame 接收。二者处于编解码管线的不同阶段。复用对象时用 av_packet_unref 和 av_frame_unref 释放本轮引用，避免不断累积缓冲。

一个 packet 不保证对应一个可显示 frame：可能一个包生成多帧，也可能需要多个包，解码器还可能为重排序缓存帧。正确流程是向解码器发送 packet，再循环接收所有可用 frame；文件结束时还需发送空 packet 冲刷延迟帧。对象分配和对象持有的数据引用是两回事，最后分别 free packet/frame。

代码示例

```
AVPacket* packet = av_packet_alloc(); // 压缩码流及 stream_index/时间戳
AVFrame* frame = av_frame_alloc(); // 解码后的平面图像或音频样本
while (av_read_frame(format, packet) >= 0)
{
    if (packet->stream_index == videoIndex)
        sendAndReceive(decoder, packet, frame);
    av_packet_unref(packet); // 释放 packet 持有的数据引用
}
av_frame_unref(frame); // 可复用 frame 时释放本轮缓冲引用
av_packet_free(&packet);
av_frame_free(&frame); // 释放对象本身
// 一个 packet 不保证只产出一帧；decoder 可能缓存延迟帧
```

004 FFmpeg 的主要组件有哪些？各自的作用是什么？

答案　FFmpeg 是一组可组合的多媒体库和命令行工具。libavformat 读写容器并解复用/复用，libavcodec 提供音视频编解码，libavutil 提供通用数据结构、时间和内存工具；libswscale 负责图像缩放与像素格式转换，libswresample 负责音频采样格式、声道和采样率转换。

libavfilter 可构建滤镜处理图，libavdevice 接入采集设备。某个模块是否存在取决于构建时启用的组件、外部库和平台支持；编码器也可能因许可证或依赖而未编译。部署时需检查运行库版本和可用格式/编解码器，不能仅因链接了 FFmpeg 就假设所有格式都支持。

代码示例

```
while (av_read_frame(input, packet) >= 0)
{
    avcodec_send_packet(decoder, packet);
    while (avcodec_receive_frame(decoder, frame) == 0)
        encode_and_mux(frame, output);
} // format读包，codec解码，sws转换，filter处理，codec编码，format复用
```

005 FLV 文件格式的特点是什么？FLV Tag 的类型有哪些？

答案　FLV 是一种容器文件。文件以 FLV 标识和头部开始，之后是一系列 Tag；Tag 头记录类型、数据长度、时间戳和流 ID，数据之后还有 PreviousTagSize，便于解析器检查前一标签长度。常见标签类型承载音频、视频或脚本数据，脚本数据常保存 onMetaData 元信息。

Tag 只说明容器层面数据类型，内部实际编码由音频或视频 codec 标识决定；同一 FLV 可能承载不同编码，支持范围还受播放器与版本影响。时间戳常用于恢复媒体时间顺序，但也要处理回绕和缺失。不能看到 .flv 就判断视频一定是 H.264 或音频一定是 AAC。

代码示例

```
TagHeader tag = read_flv_tag_header(stream);
auto payload = read_exact(stream, tag.data_size);
if (tag.type == TagType::audio)
    audio_parser.push(payload);
if (tag.type == TagType::video)
    video_parser.push(payload);
verify_previous_tag_size(stream);
```

006 H.264 和 H.265 的主要区别是什么？压缩效率和编码复杂度如何对比？

答案　H.265/HEVC 提供比 H.264/AVC 更灵活的块划分、帧内帧间预测和变换方式，编码器可用更多分析来降低冗余。因此在不少内容上，它能以更低码率达到相近主观质量，尤其是高分辨率素材。实际收益随片源、码控、编码器实现、速度预设和播放质量目标变化，没有固定节省比例。

HEVC 编码通常更耗计算，解码复杂度、硬件支持、终端许可和兼容性也要考虑。设备不支持时，即使文件更小也无法播放或会退化到软件解码。选择编码格式应同时看目标码率、画质、延迟、耗电和用户设备覆盖；流媒体通常准备多种编码或转码回退，而不是假设新标准处处更优。

代码示例

```
auto h264 = encode(source, Codec::h264, target_quality);
auto hevc = encode(source, Codec::hevc, target_quality);
measure_quality_bitrate_decode_power(h264, hevc); // 在目标设备验证兼容性与功耗
```

007 H.264 的 Profile 和 Level 分别代表什么含义？

答案　H.264 Profile 描述码流允许使用哪些编码工具和能力子集，例如 Baseline、Main、High；Level 则限制解码所需的资源范围，如最大图像尺寸、宏块处理率、码率和缓冲能力。Profile 和 Level 是兼容性约束，不是画质等级；标成更高 Level 不会自动让图像更清楚。

终端能否播放要综合 profile、level、分辨率、帧率、色度采样、位深和实际码率。编码时设置参数只是提出输出配置，仍应检查编码器实际生成的 SPS 和目标设备支持表。若要兼容旧设备，可采用更普遍支持的组合或提供转码版本。

代码示例

```
AVCodecContext* encoder = avcodec_alloc_context3(codec);
encoder->width = width;
encoder->height = height;
encoder->pix_fmt = AV_PIX_FMT_YUV420P;
encoder->time_base = AVRational{1, 30};
encoder->profile = FF_PROFILE_H264_HIGH; // 工具集子集
encoder->level = 40; // 约束能力上限，数值按 codec 定义
if (avcodec_open2(encoder, codec, nullptr) < 0)
    reportUnsupportedConfig();
avcodec_free_context(&encoder);
// 还要验证目标设备能解该 profile/level/尺寸/帧率组合
```

008 HLS 协议的原理是什么？M3U8 文件的作用是什么？

答案　HLS 把媒体切成多个分片，并以 M3U8 播放列表描述如何获取它们。媒体播放列表列出分片 URI、时长和序号；主播放列表还可以列出多种码率和分辨率，客户端根据网络吞吐在变体间切换。分片常见封装有 MPEG-TS 和 fMP4，直播列表会随新片段产生而更新。

实际延迟不仅由分片长度决定，还受关键帧对齐、编码器 GOP、播放器缓冲和 CDN 缓存影响。客户端必须能访问列表及每个分片，直播列表还可能滑动并移除旧内容。M3U8 是索引文本，不包含全部媒体本身；只复制清单而未部署分片 URI 指向内容，播放会失败。

代码示例

```
auto master = parse_m3u8(fetch(master_url));
auto variant = choose_variant(master, bandwidth, device);
auto playlist = parse_m3u8(fetch(variant.url));
auto segment = fetch(playlist.next_segment());
player.buffer(segment); // 直播时刷新媒体清单并按带宽切换变体
```

009 IDR 帧和 I 帧有什么区别?

答案　I 帧是无需其他图像参考即可解码的一幅图像；IDR 是 H.264 中带参考刷新语义的一种随机访问图像。IDR 解码后，后续图像不会再引用它之前的参考图像，因此可从该点开始建立新的解码序列。不是所有 I 帧都具备这种刷新保证，开放 GOP 中 I 帧后面的图像仍可能引用前面的参考帧。

seek、直播分段和丢包恢复需要真正可独立开始的随机访问点，并确保必要的 SPS/PPS 等参数集可获得。编码器 GOP 结构和封装索引会影响定位速度；不能仅凭帧类型字母 I 就认定任意位置都可无损切段。其他编码标准也有各自随机访问图像类型，概念名称不能直接互换。

代码示例

```
for (AVPacket* packet : packets)
    if (is_idr(packet))
        start_new_segment(packet);
verify_parameter_sets_and_timestamps(segment); // 普通I帧不一定能独立随机访问
```

010 MP4 文件的基本结构是怎样的?什么是 moov、mdat、ftyp?

答案　MP4 基于 ISO Base Media File Format，以层次化 box 组织信息。ftyp 声明文件品牌和兼容标识；mdat 保存压缩音视频样本；moov 保存轨道、时序、编解码配置及样本到数据偏移的索引。播放端先读 moov，才能知道如何在 mdat 中定位各帧。

如果 moov 在文件末尾，渐进播放可能要等下载较多内容才能开始；faststart 通过重写文件把 moov 移到前面，常用于点播文件，但会带来额外读写和处理成本。MP4 只是容器，不表示内部一定采用某一种 codec。流式场景还可使用 fragmented MP4，其组织方式与单个完整 moov/mdat 文件有所不同。

代码示例

```
auto ftyp = mp4.read_box("ftyp");
auto moov = mp4.read_box("moov");
auto mdat = mp4.read_box("mdat");
if (moov.after(mdat))
    faststart_relocate_moov(mp4);
verify_sample_offsets(mp4);
// moov含轨道/样本索引，mdat保存媒体数据，ftyp声明品牌兼容性
```

011 RTMP 协议的特点是什么?适用于什么场景?

答案　RTMP 常用于直播采集端向服务器推送音视频，基于 TCP 连接，通过握手、分块和多路复用消息承载音视频数据；常见默认端口为 1935。TCP 提供有序字节传送，但丢包重传会造成队头阻塞，网络抖动和缓冲设置共同决定延迟，并非协议名称保证固定低延迟。

RTMPS 通常是在 RTMP 连接上增加 TLS 保护，仍需验证服务器证书和鉴权。浏览器对传统 RTMP 播放支持有限，服务端常将输入转成 HLS、DASH 或 WebRTC 供观众播放。RTMP 的适用性还受基础设施和终端支持影响；需要超低互动延迟时，应比较其他传输方案。

代码示例

```
RtmpClient client(server_url);
client.connect();
client.publish(stream_key);
while (auto packet = encoder.next_packet())
    client.write_chunk(packet); // 采集端推送压缩媒体
// 服务端可转封装为HLS/WebRTC；TLS与发布鉴权需单独启用
```

012 RTP 和 RTCP 分别是什么?它们在流媒体传输中的作用是什么?

答案　RTP 为实时媒体包提供序列号、时间戳、负载类型和同步源标识，让接收端判断包顺序、检测丢包，并按媒体采样时钟安排播放。RTP 本身通常不保证送达，也不规定必须使用 UDP；它可以承载在其他传输方式之上，具体配置依会话协商。

RTCP 与 RTP 配套交换发送端和接收端报告，包括丢包、抖动、收发统计，以及用于音视频时钟映射的信息。应用可用这些反馈调整码率或实现拥塞控制，但 RTCP 自身不是可靠重传协议，也不保证媒体一定送达。实时接收端还需要抖动缓冲和迟到包策略，在延迟与连续播放间取舍。

代码示例

```
RtpPacket packet = receive_rtp();
jitter_buffer.insert(packet.sequence, packet.timestamp, packet.payload);
if (packet.sequence_gap())
```

```
    send_rtcp_receiver_report(packet.loss_rate(), packet.jitter());
// RTP承载媒体，RTCP反馈质量；恢复策略由应用协议设计
```

013 RTSP 协议的工作流程是什么？主要包含哪些命令？

答案　RTSP 是控制流媒体会话的协议，客户端通过命令描述能力、建立媒体传输并控制播放。常见流程是 OPTIONS 查询方法，DESCRIBE 获取媒体描述 SDP，SETUP 协商传输通道和端口，PLAY 开始；PAUSE 暂停，TEARDOWN 结束会话。服务端也可能要求认证或采用不同顺序。

媒体数据通常通过 RTP/RTCP 传输，也可能 interleaved 交织在 RTSP TCP 连接上。RTSP 负责会话控制，不应与实际音视频负载传输混为一谈。实现时要解析每条响应状态和会话标识，并按服务器能力协商 UDP/TCP；超时、认证失败和连接清理都需要显式处理。

代码示例

```
RtspSession session(url);
session.options();
auto sdp = session.describe();
session.setup(sdp.audio_track(), transport);
session.play(); // 完成后用TEARDOWN释放会话
```

014 SwrContext 的作用是什么？如何使用它进行音频重采样？

答案　SwrContext 保存 libswresample 的重采样配置和滤波状态，可在输入输出采样率、样本格式或声道布局不同时转换音频。先配置两端参数并初始化，再把 AVFrame 的平面指针、样本数送入 swr_convert；输出容量要按 API 估算值分配。输入一批样本和输出一批样本数量通常不相等，因为重采样比例和内部滤波缓冲会引入延迟。

流结束时需用空输入继续调用转换函数，把滤波器内部积累的尾部样本冲刷出来；遗漏会截断最后一段声音。注意更新版 FFmpeg 的声道布局 API 与旧版通道数接口存在差异，按目标版本选择。每次输出后根据实际返回样本数写数据，最后释放缓冲、布局和 SwrContext。

代码示例

```
SwrContext* swr = nullptr;
check(swr_alloc_set_opts2(
    &swr, &outLayout, AV_SAMPLE_FMT_S16, 44100, &inLayout, AV_SAMPLE_FMT_FLTP, 48000, 0, nullptr));
check(swr_init(swr)); // 参数在初始化前确定
int capacity = swr_get_out_samples(swr, input->nb_samples);
allocateOutputPlanes(capacity, outLayout);
int samples = swr_convert(swr, output, capacity, input->extended_data, input->nb_samples);
writeSamples(output, samples); // 输出数与输入数可不同
int tail = swr_convert(swr, output, capacity, nullptr, 0); // 取滤波器尾部
freeOutputPlanes();
av_channel_layout_uninit(&inLayout);
av_channel_layout_uninit(&outLayout);
swr_free(&swr);
```

015 SwsContext 的作用是什么？如何使用它进行 YUV 到 RGB 的转换？

答案　SwsContext 保存 libswscale 的图像缩放和像素格式转换配置。创建时指定源、目标尺寸和像素格式，再用 sws_scale 提供 AVFrame 的各平面指针、每行 linesize 和待转换行范围。例如 YUV420P 转 RGB24 时，源的 Y、U、V 位于不同 plane，不能把 U/V 当成一个交错平面；目标 RGB 行也可能有对齐填充。

还应依据帧的色彩元数据设置 BT.601/BT.709 等矩阵和 full/limited range。矩阵错误会使颜色偏差，范围不符会导致黑位发灰或亮部不正确。目标缓冲区应按实际 stride 分配。转换完成后释放 SwsContext 和目标 AVFrame；不同 FFmpeg 版本对色彩 API 可能变化，按目标版本文档确认。

代码示例

```
SwsContext* sws = sws_getContext(srcW,
                                 srcH,
                                 AV_PIX_FMT_YUV420P,
                                 dstW,
                                 dstH,
                                 AV_PIX_FMT_RGB24,
                                 SWS_BILINEAR,
                                 nullptr,
                                 nullptr,
                                 nullptr);
if (!sws)
    return reportAllocationFailure();
allocateRgbFrame(dstW, dstH, &dstData, &dstStride); // 含 padding
sws_scale(sws, src->data, src->linesize, 0, srcH, dstData, dstStride);
// 输入 plane 指针和每行 stride 必须来自 AVFrame
```

```
const int* matrix = sws_getCoefficients(SWS_CS_ITU709);
sws_setColorspaceDetails(sws, matrix, 0, matrix, 1, 0, 1 << 16, 1 << 16);
freeRgbFrame(&dstData);
sws_freeContext(sws); // 同时释放源/目标 AVFrame
```

016 UDP 和 TCP 在流媒体传输中各有什么优缺点?

答案　TCP 和 UDP 在可靠性、时延和复杂度之间取舍。TCP 提供可靠有序字节流，丢包时会重传，后续数据可能因此被队头阻塞；它适合能缓冲等待完整数据的传输。UDP 将数据报独立交付，不保证送达、顺序或拥塞控制，应用可自行丢弃过期数据来控制时延。

实时媒体常结合 RTP/UDP、抖动缓冲、NACK 重传或 FEC 前向纠错，按业务允许的时延恢复部分丢包；这些机制不是 UDP 自带。企业网络和防火墙可能限制 UDP，需考虑 TURN/TCP 或其他回退。选型应看时延目标、丢包状况、网络策略、终端支持，以及数据是否必须完整。

代码示例

```
if (media_is_time_sensitive)
    udp.send(packet_with_sequence_and_timestamp);
else
    tcp.send(reliable_ordered_bytes); // 选择前先定义丢包、延迟与重传预算
```

017 WebRTC 的主要特点是什么？适用于什么场景?

答案　WebRTC 面向端到端实时音视频，通常包含媒体采集、回声消除、降噪、拥塞控制、加密和音画同步。ICE 负责尝试候选网络地址建立连接，STUN 帮助发现 NAT 映射；直连失败时可通过 TURN 中继。应用仍要提供信令来交换 SDP 和 ICE 候选，WebRTC 并不规定信令必须通过 WebSocket。

媒体通常优先 UDP，并以拥塞控制与抖动处理平衡时延和丢包；某些 TURN 路径会使用 TCP/TLS。WebSocket 的有序可靠传输容易因重传等待，常用于信令和控制消息，不应简单当作低延迟媒体通道。TURN 的服务器带宽和部署会增加成本，实际穿透成功率也受企业网络影响。〔A127〕

代码示例

```
PeerConnection pc(ice_servers);
pc.add_track(capture_audio_video());
auto offer = pc.create_offer();
signaling.send(offer);
pc.add_remote_candidates(answer.candidates);
// ICE先尝试直连，失败时可通过TURN中继媒体
```

018 YUV420P、YUV420SP（NV12、NV21）的数据排列方式有何不同?

答案　YUV420P（常见名称 I420）使用 Y、U、V 三个独立平面：亮度 Y 保存每个像素，U 和 V 的宽高各约减半。NV12 也采用 4:2:0 色度采样，但把色度放在一个平面中按 UVUV 交错排列；NV21 则按 VUVU 交错。三者色度采样率相同，内存组织不同，不能互相当作同一种像素格式。

真实帧每行可能有对齐填充，因此 stride/linesize 可能大于有效宽度；奇数尺寸时色度平面的取整也由格式定义。读取时应依据实际 pixel format 选择 data[] 平面，并按各自行跨度前进。布局不匹配会产生明显偏色，甚至越界读取；跨库处理前应使用明确的像素格式转换接口。

代码示例

```
// planar YUV420P: Y、U、V 独立 plane
const std::uint8_t *y = frame->data[0], *u = frame->data[1], *v = frame->data[2];
const int yStride = frame->linesize[0], uStride = frame->linesize[1];
samplePlanar(y, u, v, yStride, uStride);
// semiplanar NV12: data[1] 内 UV 交错; NV21 为 VU
const std::uint8_t* uv = nv12->data[1];
sampleInterleavedUv(uv, nv12->linesize[1]);
// 每行用 stride 前进; plane 次序不符会出现明显偏色
```

019 YUV420、YUV422、YUV444 的区别是什么?

答案　4:4:4、4:2:2 和 4:2:0 描述色度相对于亮度的采样密度。4:4:4 为每个亮度样本保存对应 U、V；4:2:2 在水平方向每两个亮度样本共享一组色度；4:2:0 再在垂直方向也约减半。抽样可降低码流或内存占用，通常对自然影像观感影响较小，但鲜艳物体边缘会损失颜色细节。

采样比例只说明分辨率关系，并没有规定数据是独立平面还是交错存储；位深、行对齐和色彩范围还由具体像素格式决定。估算 8 位紧凑存储时要为奇数尺寸色度宽高向上取整；真实 AVFrame 的 linesize 可能带 padding，不能只按像素数分配。字幕、色键或精细彩边需要评估更高色度采样。

代码示例

```
const std::size_t y = static_cast<std::size_t>(width) * height;
const std::size_t cw = (width + 1) / 2, ch = (height + 1) / 2;
const std::size_t bytes420 = y + 2 * cw * ch; // U/V 向上取整
const std::size_t bytes422 = y + 2 * cw * height;
```

```
const std::size_t bytes444 = 3 * y;
// 采样越稀，色度边缘细节越少；实际 frame 还有 stride padding
report(bytes420, bytes422, bytes444);
```

16.2 FFmpeg 解复用、解码与时间戳

020 av_read_frame()读取的是什么数据?

答案　av_read_frame 从容器解复用器读取下一个压缩 packet，并填入 stream_index、时间戳和媒体数据引用；它不负责把压缩内容解码成图像或音频。正常读到包通常返回 0，文件结束返回 AVERROR_EOF，其他负值表示读取错误。包上的 PTS/DTS 要结合所属 AVStream 的 time_base 解释，整数本身并不等于秒。

处理完包后依 stream_index 选择音频、视频或字幕轨，并交给对应模块；一个 packet 可能产出零个或多个 frame。每轮用 av_packet_unref 释放其数据引用，再继续读取，最终用 av_packet_free 释放 packet 对象。需要比较不同轨道时间或送往复用器时，必须在各自时间基之间显式换算时间戳。

代码示例

```
AVPacket* packet = av_packet_alloc();
if (!packet)
    return reportAllocationFailure();
while ((ret = av_read_frame(format, packet)) >= 0)
{
    const AVStream* stream = format->streams[packet->stream_index];
    if (packet->stream_index == videoIndex)
        decode(packet); // 仍是压缩 packet
    inspect(packet->pts, packet->dts, stream->time_base);
    av_packet_unref(packet); // 每次读完都归还 payload 引用
}
if (ret != AVERROR_EOF)
    report(ret); // 区分读错误与正常结束
av_packet_free(&packet); // 再释放 packet 对象
```

021 av_rescale_q()函数的作用是什么？如何进行时间戳转换?

答案　av_rescale_q 用有理数 time_base 缩放整数时间戳，相当于计算 a×源刻度/目标刻度，并按 FFmpeg 规则舍入和处理溢出。它常用于把解复用得到的流时间戳转换到编码器或复用器要求的时间单位。例如源流 time_base 是 1/90000，目标编码时钟是 1/25，就要明确传入源和目标的顺序。

先辨明时间戳属于哪条流、当前使用哪个 time_base，再选择目标刻度；把两个 time_base 反过来会让时间长度严重错误。无效或缺失时间戳需单独处理，不能盲目缩放 AV_NOPTS_VALUE。封装时通常还要根据输出流的 time_base 调整 packet PTS、DTS 和 duration。

代码示例

```
if (pts != AV_NOPTS_VALUE)
{
    const AVRational streamTb = stream->time_base; // 源时间刻度
    const AVRational encoderTb{1, 25}; // 目标每帧 1/25 秒
    const std::int64_t encoderPts = av_rescale_q(pts, streamTb, encoderTb);
    submitTimestamp(encoderPts); // 先确认 bq/cq 次序
}
// 不要直接把 PTS 当毫秒；有理数缩放还可避免乘法溢出
```

022 avcodec_send_packet()和 avcodec_receive_frame()的作用是什么?

答案　avcodec_send_packet 把压缩 packet 输入解码器，avcodec_receive_frame 则取出当前可用的解码帧。一个 packet 可能产生多个 frame，解码器也会缓存重排帧，所以每次成功送入后通常要循环接收到 EAGAIN。若 send 返回 EAGAIN，表示先前输出尚未取完，应先 receive，再重送同一个 packet。send 成功只说明解码器接受了输入，不代表已经输出画面。

读到输入 EOF 后发送一次空 packet 进入冲刷状态，再持续 receive，直到 AVERROR_EOF，才能拿到解码器延迟的最后几帧。EAGAIN 是 send/receive 状态切换信号，不代表媒体坏了。packet 的数据仍由调用方管理，送入后仍应按 API 约定 unref；frame 复用时也要释放上一轮引用。该新式流程不同于旧的 avcodec_decode_video2 调用。

代码示例

```
// receiveAvailable 循环 receive_frame：处理每帧并 unref，直到 EAGAIN/EOF
int ret = avcodec_send_packet(decoder, packet);
if (ret == AVERROR(EAGAIN))
{
    receiveAvailable(decoder, frame);
    ret = avcodec_send_packet(decoder, packet);
}
```

```
if (ret < 0)
    return reportDecodeError(ret);
receiveAvailable(decoder, frame); // receive 到 EAGAIN 后再送下一包
// demux EOF 后发送空包进入 draining mode
while ((ret = avcodec_send_packet(decoder, nullptr)) == AVERROR(EAGAIN))
    receiveAvailable(decoder, frame);
if (ret >= 0)
    receiveUntil(decoder, frame, AVERROR_EOF);
// 最终 av_packet_free、av_frame_free、avcodec_free_context
```

023 time_base 是什么？如何理解时间戳和 time_base 的关系？

答案　time_base 是用有理数表示的时钟单位，写作分子/分母。例如 1/90000 表示一个时间戳刻度相当于 1/90000 秒；时间戳乘上这个比例才能换算成秒。容器流、解码器和编码器可能各自使用不同刻度，所以同一个时刻会对应不同整数，不能直接比较或相加。

帧率描述帧产生或显示的频率，并不等同 time_base；固定帧率时二者可能有简单关系，可变帧率时应读取每帧时间戳。显示同步通常依据 PTS，解码顺序依据 DTS。换算时使用 av_rescale_q 并处理缺失时间戳和舍入，不要简单用整数除法。

代码示例

```
const AVRational tb = stream->time_base; // 每个 timestamp 的秒数
if (frame->pts != AV_NOPTS_VALUE)
{
    const double seconds = frame->pts * av_q2d(tb);
    showAt(seconds); // 展示时钟使用 PTS
}
const AVRational ms{1, 1000};
const std::int64_t timestampMs = av_rescale_q(frame->pts, tb, ms);
// time_base 不是帧率；不同流刻度不可直接比较
```

024 什么是 I 帧、P 帧、B 帧？它们在视频编码中的作用和压缩率有何不同？

答案　I、P、B 描述图像如何利用其他图像预测。I 图像不依赖别的图像即可重建；P 图像通常参考先前图像；B 图像可参考前后图像。帧间预测利用时间上的相似性来降低码率，但解码器必须先拿到参考图像。现代编码还可用多个参考帧和复杂图像结构，不能只凭字母判定码率。

含 B 帧时，解码顺序和显示顺序可能不同：解码器要先收到未来参考图像，再输出更早显示的 B 帧，因此带来重排延迟。seek 和分段应从 IDR 或其他真正可恢复的随机访问点开始。普通 I 帧不一定意味着参考状态刷新；GOP、切片和编码参数都会影响错误恢复。

代码示例

```
for (Frame frame : decode_order)
    decoded.push(decode(frame));
std::sort(decoded.begin(), decoded.end(), by_presentation_timestamp); // B帧解码顺序和显示顺序可不同
// seek从可恢复随机访问点开始，普通I帧未必是IDR
```

025 什么是 NAL Unit？H.264 码流的基本结构是怎样的？

答案　NAL Unit 是 H.264 基本码流的单元，由 NAL 头和负载构成；负载可能是切片、SPS/PPS 参数集或 SEI 等辅助信息。裸流常用 Annex B 起始码分隔 NAL；MP4 等容器通常在样本中用长度前缀，并把参数集存入 codec 配置记录。两种封装不应只靠改扩展名互换。送入特定解码器、容器或网络协议时，可能要使用 bitstream filter 转换起始码与长度前缀，并把参数集放到对方预期位置。一个画面访问单元可由多个 NAL 组成，单个 NAL 也不一定对应一整帧。解析必须结合容器配置和 codec 规则。

代码示例

```
const AVBitStreamFilter* filter = av_bsf_get_by_name("h264_mp4toannexb");
AVBSFContext* bsf = nullptr;
check(av_bsf_alloc(filter, &bsf));
avcodec_parameters_copy(bsf->par_in, stream->codecpar);
bsf->time_base_in = stream->time_base;
check(av_bsf_init(bsf)); // MP4 length-prefix 转 Annex B 起始码
check(av_bsf_send_packet(bsf, packet)); // 成功后 packet 内容所有权转移
while (av_bsf_receive_packet(bsf, filtered) == 0)
    writeAnnexB(filtered);
av_packet_free(&packet);
av_packet_free(&filtered);
av_bsf_free(&bsf);
// 参数集和 NAL 格式按目标协议/容器选择
```

026 什么是 PCM（脉冲编码调制）？采样率、位深度、声道数分别代表什么？

答案　PCM 是采样并量化后的数字音频数据，没有 MP3、AAC 那样的感知有损编码。采样率表示每秒采样次数，位深表示每个样本可用的量化位数，声道数表示独立信号路数。例如 48 kHz、16 位、立体声的未压缩有效速率约为采样率乘位深再乘声道数。

实际文件还可能有容器头、元数据、对齐填充；样本也可按 packed 交错或 planar 平面排列。因此总文件大小不一定正好等于估算值。采样率和位深都影响数据量及可表示信息，传输和播放时还要明确样本格式、声道布局、端序和时间基，不能仅凭文件扩展名推断这些参数。

代码示例

```
const std::uint64_t bitsPerSecond =
    static_cast<std::uint64_t>(sampleRate) * bitsPerSample * channels;
const std::size_t bytesPerSecond = static_cast<std::size_t>((bitsPerSecond + 7) / 8);
const std::size_t blockBytes = bytesPerSecond * blockMilliseconds / 1000;
readPcmBlock(buffer, blockBytes);
playSamples(buffer, sampleRate, sampleFormat, channels);
// PCM 无自描述头：调用方须知道位深/浮点格式与 packed/planar 布局
// 容器头、对齐与设备缓冲还会增加字节数
```

027 什么是 PTS 和 DTS？为什么需要两个时间戳？

答案　DTS 表示压缩帧应按什么顺序送入解码器，PTS 表示解码后的图像或音频样本在媒体时间线上何时呈现。没有帧重排时，两者可能相等或接近；使用 B 帧时，解码和显示顺序不同，PTS 与 DTS 会分离。它们都是按对应流的 time_base 表示的整数。

音画同步时，应把有效 PTS 换算到共同时间单位，再按播放时钟决定呈现时间。PTS 可能缺失为 AV_NOPTS_VALUE，需按协议和媒体类型处理。不能把 PTS 当作文件字节偏移，也不能总用帧号代替时间；DTS 用来满足解码依赖，PTS 负责显示时序。

代码示例

```
if (packet->dts != AV_NOPTS_VALUE)
    decoderOrderClock = av_rescale_q(packet->dts, stream->time_base, commonTb);
if (frame->pts != AV_NOPTS_VALUE)
    displayClock = av_rescale_q(frame->pts, stream->time_base, commonTb);
// 有 B 帧时解码次序和显示次序不同，DTS/PTS 不同
if (frame->pts == AV_NOPTS_VALUE)
    useFallbackTimestamp();
// 同步音视频前先把有效时间戳换到共同时钟
```

16.3 图像、音频与视频编码原理

028 什么是 SIMD？如何使用 SIMD 优化视频处理？

答案　SIMD 是“单指令多数据”：一条向量指令对多个像素或样本同时执行相同运算，例如阈值比较、颜色转换或加法。优化应先让数据连续、批量处理并减少不必要复制；编译器可能自动向量化，OpenCV 和 FFmpeg 自身也常包含 SSE、AVX、NEON 等优化实现。

热点明确后再写平台 intrinsic。需要处理数组尾部不足一个向量的元素、对齐要求和不同 CPU 能力；某机器可运行并不代表所有目标设备都有相同指令集。图像工作经常受内存带宽或缓存限制，算术变快未必明显提速。通过基准测试比较真实输入下的耗时、功耗和结果一致性。

代码示例

```
for (std::size_t i = 0; i + simd_width <= pixels.size(); i += simd_width)
{
    auto lane = load_vector(pixels.data() + i);
    store_vector(mask.data() + i, lane > threshold);
}
for (std::size_t i = vector_end; i < pixels.size(); ++i)
    mask[i] = pixels[i] > threshold; // 标量处理尾部
```

029 什么是 YUV 颜色空间？YUV 和 RGB 有什么区别？

答案　RGB 用红、绿、蓝通道描述显示颜色。数字视频常用 Y′CbCr，把伽马校正后的亮度分量与蓝、红色度差分开，便于降低色度采样率；工程代码常把 Y′CbCr 简称 YUV，但严格说模拟 YUV 与数字 YCbCr 是不同表示。

转换必须匹配 BT.601、BT.709 或 BT.2020 等矩阵，并确认 full range 或 limited range。选择错误会导致颜色偏差、黑位灰或白位不准。RGB 本身也有不同色域和传递函数，通道名称相同不代表色彩空间相同。跨编解码器和显示设备时应传播帧色彩元数据，或明确转换到目标色彩空间。

代码示例

```
SwsContext* convert = makeYuvToRgbContext(frame->width, frame->height);
const int* coeff = sws_getCoefficients(SWS_CS_ITU709); // 色彩矩阵
```

```
sws_setColorspaceDetails(convert, coeff, sourceIsFullRange, coeff, 1, 0, 1 << 16, 1 << 16);
sws_scale(convert, frame->data, frame->linesize, 0, frame->height, rgb, stride);
showRgb(rgb); // 亮度/色度通过矩阵组合到 RGB
sws_freeContext(convert); // 同时检查 source range/full vs limited metadata
```

030 什么是丢包（Packet Loss）？如何处理丢包问题？

答案　丢包是媒体包未到达，或虽到达但已经错过播放期限。RTP 序列号缺口、RTCP 报告和解码器错误可帮助检测。恢复方式取决于媒体：可在剩余时限内用 NACK 请求重传、用 FEC 前向纠错恢复，音频可用 PLC 隐藏短缺帧；视频参考链损坏时可能丢弃并请求关键帧。

实时流不能无期限等待，因为迟到包通常已没有播放价值。拥塞持续时应降码率、缩短或限制队列，并丢弃过期帧；盲目重传会增加排队，令更多数据超时。文件传输则更重视完整性。策略要结合 RTT、GOP 参考关系、业务延迟预算和网络拥塞控制。

代码示例

```
if (packet_is_late(packet, playback_deadline))
    discard(packet);
else if (can_recover_before_deadline(packet, rtt))
    send_nack(packet.sequence);
if (reference_chain_broken())
    request_keyframe(); // 缓冲必须有界，拥塞时降低码率
```

031 什么是关键帧间隔？如何设置合适的关键帧间隔？

答案　关键帧间隔是随机访问点之间相隔的帧数或时间，通常与 GOP 长度相关。间隔较短可缩短 seek、直播新观众切入和错误恢复等待，但关键帧更大，可能增加码率峰值和平均码率；间隔较长通常更利于压缩，却会增加恢复延迟。直播切片通常希望关键帧对齐分片边界，否则新片段可能要等待前一段参考帧。固定帧率可近似用帧数换算时间；可变帧率应按 PTS 安排。编码器场景切换插入关键帧也会改变实际间隔，需检查输出码流并在目标播放器验证。间隔应按带宽、延迟和恢复目标决定。

代码示例

```
encoder->time_base = AVRational{1, fps};
encoder->framerate = AVRational{fps, 1};
encoder->gop_size = fps * targetKeyframeSeconds; // 近似按秒设随机访问间隔
encoder->max_b_frames = 0; // 低延迟场景可减少重排
if (avcodec_open2(encoder, codec, nullptr) < 0)
    reportUnsupportedSettings();
// 分片直播常要求关键帧与切片边界对齐
// 短 GOP 提高恢复频率，也可能增加码率
```

032 什么是分辨率（Resolution）？常见的视频分辨率有哪些？

答案　分辨率是编码帧的像素宽和高，例如 1920×1080。常见档位有 480p、720p、1080p 和 UHD 4K 3840×2160；“p”通常表示逐行扫描。消费设备、电视和影院对 SD、2K、4K 的命名可能不同，交换文件时最好使用明确像素尺寸，而不是只写营销等级。

像素尺寸不等于屏幕物理大小或最终显示比例。非方形像素素材还需看 SAR/PAR 和 DAR，播放器可能据此拉伸或缩放。更高分辨率会提高解码、带宽和存储压力，小屏观看不一定明显更清晰。根据画面细节、终端能力、网络预算和显示尺寸选择编码档位。

代码示例

```
encoder->width = 1920;
encoder->height = 1080; // 编码样本宽高
encoder->sample_aspect_ratio = AVRational{1, 1}; // 方形像素
stream->sample_aspect_ratio = encoder->sample_aspect_ratio;
stream->avg_frame_rate = AVRational{30, 1};
const AVRational dar =
    av_mul_q(av_mul_q(AVRational{encoder->width, encoder->height}, encoder->sample_aspect_ratio),
             AVRational{1, 1});
logDisplayRatio(av_q2d(dar)); // 还需结合目标显示像素比例
// p 通常描述逐行扫描；分辨率不等于显示宽高比
```

033 什么是去块效应滤波（Deblocking Filter）？为什么需要它？

答案　去块滤波在编码块边界附近依据边缘强度、量化和运动信息选择性平滑，减轻低码率画面的方块接缝。它通常是视频标准解码过程的一部分，滤波后的重建图像还可能继续作为后续帧的预测参考。因此它不是对任意图像全局套一次模糊滤镜。

关闭滤波会改变重建画面及后续参考，不能随意当作播放器美化开关；过度平滑也会丢失纹理。不同编码标准有自己的环路滤波工具。排查块效应时还要区分编码量化、显示缩放和采样导致的边缘问题，先检查码率、量化参数和解码输出，再决定是否需要滤波调整。

代码示例

```
Frame raw = decode_without_loop_filter(packet);
Frame filtered = decode_with_deblocking(packet);
compare_boundary_artifacts(raw, filtered); // 去块滤波减轻接缝，也可能损失细节
```

034 什么是宏块（Macroblock）？H.264 中宏块的大小是多少？

答案　H.264 的宏块以 16×16 亮度样本为基本结构，色度块大小取决于 4:2:0、4:2:2 等色度抽样格式。编码器还能把宏块划分为更小的预测分区和变换块，以匹配运动与纹理。宏块不是画面上固定不变、直接显示的方格，而是编码器组织预测和残差的单位。

H.265/HEVC 使用更大的 CTU 起始块，并允许编码树递归划分成多种尺寸，因此结构比“每块 16×16”更灵活。码流内部的宏块或 CTU 由解码器处理，普通播放器用户不需要手动切割图像。分析压缩效率时，还要结合预测模式、量化和熵编码，而不能仅比较块大小。

代码示例

```
Macroblock block = analyze_motion(current_frame, reference_frame, x, y);
auto prediction = motion_compensate(reference_frame, block.vector, block.partition);
encode_residual(current_frame, prediction); // H.264以16x16亮度宏块为基础结构
```

035 什么是容器格式？容器格式和编码格式有什么关系？

答案　容器格式负责把音频、视频、字幕等轨道组织成一个文件或传输对象，保存时间戳、索引和元数据。MP4、Matroska 是容器；H.264、AV1、AAC 是编码格式，规定压缩样本如何表示。一个容器可以封装多种 codec，也可以同一 codec 封在不同容器里。

容器支持某编码并不代表设备一定能播放它，还需匹配 codec profile、level、像素格式、位深和音频配置。转封装只改容器包装，不能改变压缩编码；若目标容器不支持当前 codec，就必须重新编码或选择其他容器。判断兼容性要查看容器、轨道和具体解码器能力。

代码示例

```
auto streams = probe_media(input_path); // 容器描述轨道，codec描述压缩方式
if (target.supports(streams))
    remux(input_path, output_path);
else
    transcode(input_path, output_path);
// 先检查profile、像素格式、音频布局和目标播放器能力
```

036 什么是宽高比（Aspect Ratio）？像素宽高比（PAR）和显示宽高比（DAR）有什么关系？

答案　宽高比描述画面宽度与高度的比例。视频术语中 SAR 常表示样本宽高比，即帧的像素宽/高；PAR 表示单个像素显示时的宽/高；DAR 是屏幕上最终画面的显示比例，关系为 DAR = SAR × PAR。若像素是方形，PAR 为 1，DAR 就等于像素尺寸宽高比。

不同文档有时把 SAR 也写作 sample aspect ratio，简称术语会产生混淆，应看上下文。传统电视视频可能用非方形像素，忽略 PAR 会把圆形显示成椭圆，人物也会被拉伸。缩放时既要保留样本尺寸比例，也要维护 DAR 元数据或正确重采样到方形像素。

代码示例

```
const AVRational sar =
    stream->sample_aspect_ratio.num ? stream->sample_aspect_ratio : AVRational{1, 1};
const AVRational display =
    av_mul_q(AVRational{stream->codecpar->width, stream->codecpar->height}, sar);
const double dar = av_q2d(display); // sample aspect × pixel aspect
setDisplayViewport(dar);
// 元数据缺失时可用 1:1 假设；术语 SAR/PAR 在文档间可能混用
// 不能只凭像素矩形推断最终显示比例
```

037 什么是封装器（Muxer）和解封装器（Demuxer）？

答案　Muxer（封装器）接收编码后的压缩 packet、时间戳和轨道参数，把它们组织为 MP4 等容器并写入样本索引；demuxer 则读取容器结构，输出各轨道的压缩 packet 和时间信息。它们通常不把像素解码成图像，也不重新压缩，因此可用于保持码流不变的转封装。

无损转封装仍受容器能力约束：目标容器可能不支持某 codec、参数集或时间戳形式，需调整配置或转码。转码则涉及解码、可选滤镜和重编码，耗时更多，并可能产生有损失真。无论哪种操作，都要检查输出轨道、时间戳、索引和媒体时长，不能只看命令是否返回成功。

代码示例

```
// Demux：av_read_frame 取压缩 packet，不解码像素
while (av_read_frame(input, packet) >= 0)
{
    const int i = packet->stream_index;
    packet->stream_index = mapInputToOutput(i);
```

```
    av_packet_rescale_ts(
        packet, input->streams[i]->time_base, output->streams[packet->stream_index]->time_base);
    av_interleaved_write_frame(output, packet); // Mux 到新容器
}
av_write_trailer(output); // 写索引/尾部元数据
// stream copy 适用于目标容器支持 codec；按顺序关闭 AVIO/context
```

038 什么是帧内预测和帧间预测？分别采用什么技术实现？

答案　帧内预测只使用当前帧中已经重建的邻近像素来预测当前块，常见方向模式或平面模式；帧间预测从一幅或多幅参考帧取样，用运动矢量和运动补偿形成预测块。编码器把原块与预测块的差作为残差，再经变换、量化和熵编码写入码流。

解码器用相同的参考帧、运动矢量和预测规则重建残差，因此参考图像必须此前正确解码。帧间预测通常利用时间相似性节省码率，但在场景切换、随机访问或遮挡区域，帧内预测更有用。实际编码器会在多种模式间权衡质量、码率和计算量，残差也不总是非零。

代码示例

```
auto intra = predict_from_reconstructed_neighbors(current_block);
auto inter = search_reference_frame(current_block, reference_frames);
auto mode = choose_lowest_rate_distortion(intra, inter);
encode_residual(mode);
```

039 什么是帧率（Framerate）？常见的帧率有哪些？

答案　帧率是每秒采集、编码或显示的帧数，单位 fps。常见电影帧率有 24，电视或网络内容常见 25、30、50、60 fps；高帧率运动素材也会更高。帧率影响运动平滑度和单位时间编码工作量，但不直接表示画面清晰度。

固定帧率可从相邻 PTS 间隔估算；可变帧率素材每帧间隔不同，应以实际时间戳安排播放，标称平均帧率不能替代逐帧时间。提高帧率会增加编码、传输和存储压力，还受快门时间、显示刷新率和解码能力影响。转帧率可能采用丢帧、重复帧或插帧，各方法观感和延迟不同。

代码示例

```
encoder->time_base = AVRational{1, 30}; // 每帧时钟单位
for (std::int64_t index = 0; index < frameCount; ++index)
{
    frame->pts = index; // 固定帧率按编码器时间基递增
    int ret = avcodec_send_frame(encoder, frame);
    if (ret == AVERROR(EAGAIN))
    {
        receivePacketsUntilEagain(encoder, packet);
        ret = avcodec_send_frame(encoder, frame); // 重送尚未接收的帧
    }
    check(ret);
    receivePacketsUntilEagain(encoder, packet);
}
sendNullFrameRetryingEagain(encoder, packet); // 输入结束后发送空帧
receivePacketsUntil(encoder, packet, AVERROR_EOF); // 取完延迟输出
// 可变帧率按源 PTS 换算；最后释放 packet/frame/context
```

040 什么是抖动（Jitter）？如何处理网络抖动？

答案　网络抖动是数据包到达间隔或时延的变化，不等同于丢包。RTP 接收端可以根据包到达时间和媒体时间戳估算抖动，并通过 RTCP 报告。jitter buffer 先缓存，再按媒体时钟平滑输出；缓冲太小容易断续，太大会提高端到端延迟。自适应缓冲可依据最近网络变化调节目标延迟，超出播放期限的数据应丢弃，而不是无限等待。抖动缓冲只吸收有限波动，不能解决持续带宽不足、严重拥塞或长时间断网。实时通话要在连续播放与低延迟之间权衡，并结合丢包恢复和拥塞控制来处理。

代码示例

```
jitter_buffer.insert(packet.sequence, packet.timestamp, packet.arrival_time, packet.payload);
if (jitter_buffer.delay() > max_latency)
    jitter_buffer.discard_expired();
// 按播放时钟取帧，缓冲有界；长期带宽不足应降码率
```

041 什么是推流和拉流？

答案　推流是摄像头、录屏或编码器把媒体发送到服务器；拉流是播放器从服务器取回媒体、缓冲并解码。RTMP 常见于采集端推送，HLS 常用于 CDN 大规模分发，WebRTC 或 SRT 可覆盖其他实时和抗丢包需求，但具体用途取决于实现。

协议只是链路的一部分，端到端延迟还受采集缓冲、编码器 B 帧和 GOP、服务器转发或转码、网络排队、播放器缓冲与显示影响。推流成功只代表某段传输建立，不代表观众可以播放；还需检查发布权限、服务端状态和拉流地址。端到端时间戳应跨环节一致。

代码示例

```
AVFormatContext* input = nullptr;
if (avformat_open_input(&input, pullUrl, nullptr, nullptr) < 0)
    return reportConnectError();
check(avformat_find_stream_info(input, nullptr)); // 拉流后探测轨道
AVFormatContext* output = createOutput(pushUrl); // 推流输出上下文
copyStreamParameters(input, output); // 协议/容器支持对应 codec 时可直通
writeHeader(output);
pumpPackets(input, output); // 仍需有界缓存
writeTrailer(output);
closeOutput(output);
avformat_close_input(&input); // 端到端延迟还取决于 GOP/播放缓冲
```

042 什么是比特流（Bitstream）?

答案　比特流是按顺序排列的压缩媒体数据及解码所需参数。以 H.264 为例，码流由 NAL 单元组成，包含切片、SPS/PPS 等内容。裸 elementary stream 可能用 Annex B 起始码分隔；容器里的 codec packet 则可能使用长度前缀和配置记录，因此包边界与码流分隔符不一定相同。

处理时要分清压缩 packet、访问单元和 NAL 单元的层次。MP4 packet 还带容器时间戳，不能把一个 packet 直接当成一整帧。送入某些设备或协议前，可能需要 bitstream filter 转换格式，并携带参数集。简单搜索起始码在长度前缀格式中会失败，解析应遵守对应封装规范。

代码示例

```
auto packet = demuxer.read_packet();
auto nals = parse_nals(packet.data, bitstream_format);
for (const NalUnit& nal : nals)
    if (nal.is_sps() || nal.is_pps())
        save_parameter_set(nal);
// 一个packet不保证等于一帧，PTS/DTS和stream index也要保留
```

043 什么是码率（Bitrate）? 码率和视频质量有什么关系?

答案　码率是单位时间产生或传输的比特数，常用 kbit/s 或 Mbit/s 表示。若知道文件大小和时长，可用大小乘 8 再除时长粗略估计平均码率，但容器头、音频、字幕和可变码率波动会影响结果。码率越高通常能降低有损压缩失真，却不是质量的直接保证。

编码器效率、编码标准、分辨率、帧率、画面噪声和质量参数都会影响同一码率下的效果。复杂场景需要更多比特，静态画面可用更少；码率控制还可能在短时间产生峰值。比较画质应使用相同来源、显示条件和目标尺寸，结合主观观看及客观指标，而非只比文件大小。

代码示例

```
encoder->bit_rate = 2'000'000; // 目标平均码率示例
encoder->rc_max_rate = 2'500'000; // 可选峰值上限
encoder->rc_buffer_size = 1'000'000; // 缓冲预算由编码器解释
check(avcodec_open2(encoder, codec, nullptr));
// 提高码率通常减轻有损失真，但不保证质量单调
// 内容复杂度、分辨率、帧率和编码模式共同影响效果
```

044 什么是硬件编解码? 和软件编解码有什么区别?

答案　硬件编解码由 GPU 或 SoC 的专用媒体单元处理压缩和解压，常能降低 CPU 占用、功耗或提高并发吞吐；软件编解码在 CPU 上运行，部署较灵活，算法和参数选择也更广。两者的画质和速度取决于具体芯片、编码器实现和设置，不能笼统认为硬件质量一定差或一定快。

使用硬件前要检查 codec、profile、level、位深、分辨率、驱动和 FFmpeg 构建支持。硬件帧与系统内存之间的上传/下载复制可能抵消收益，滤镜也可能迫使数据回 CPU。评估端到端吞吐、功耗、延迟和错误恢复，不仅看编码帧率；还要处理设备不可用时的回退路径。

代码示例

```
AVHWDeviceType type = AV_HWDEVICE_TYPE_NONE;
while ((type = av_hwdevice_iterate_types(type)) != AV_HWDEVICE_TYPE_NONE)
    qInfo() << av_hwdevice_get_type_name(type); // 构建包含的接口
AVBufferRef* device = nullptr;
if (av_hwdevice_ctx_create(&device, AV_HWDEVICE_TYPE_CUDA, nullptr, nullptr, 0) < 0)
    return openSoftwareDecoder(); // 设备/驱动不可用就回退
codec->hw_device_ctx = av_buffer_ref(device); // codec 持有自己的引用
```

```
av_buffer_unref(&device); // decoder 关闭时释放 codec context
// get_format 与目标硬件像素格式回调按设备配置
```

045 什么是编解码器（Codec）?

答案　Codec 是编码器、解码器及其遵循的媒体压缩表示规则。H.264、HEVC 和 AV1 是视频编码标准，AAC、MP3 和 Opus 是音频编码格式。编码后的压缩流通常装入 MP4、Matroska 等容器，由容器保存轨道、时间戳和索引信息。同一 codec 名称也有不同 profile、level、位深、色度抽样和声道配置，目标设备可能只支持其中一部分。文件后缀一般描述容器，不足以说明内部编码。选择 codec 时要考虑压缩效率、授权和专利政策、硬件解码支持、延迟和终端覆盖；应在真实播放器中验证，而不是只看编码器成功。

代码示例

```
auto info = probe_media(input_path);
if (!decoder.supports(info.video_codec, info.profile, info.bit_depth))
    transcode(input_path, output_path);
// codec是压缩格式，容器只是封装轨道和时间信息
```

046 什么是色度采样？为什么要进行色度采样?

答案　色度采样降低 Cb/Cr 色彩分量的空间分辨率，利用人眼通常对亮度细节更敏感这一特点减少存储和带宽。4:4:4 保留每个像素的色度，4:2:2 水平减半，4:2:0 水平和垂直都约减半。自然视频通常能接受一定色度损失。

字幕、细线、彩色边缘和色键处理容易暴露色度抽样的模糊或色边。转换时除了选择 4:2:0 等格式，还要正确指定色度采样位置、色彩矩阵和范围；格式名并不能说明全部颜色语义。高质量图形或后期制作可保留 4:4:4，面向大众播放常以更省带宽的 4:2:0 为主。

代码示例

```
PixelFormat format = is_natural_video ? PixelFormat::yuv420p : PixelFormat::yuv444p;
convert_colorspace(frame, format, matrix, range);
inspect_fine_color_edges(output);
// 字幕边缘/色键可能需要更高色度分辨率
```

047 什么是视频延迟（Latency）？如何降低延迟?

答案　端到端视频延迟包括采集、编码前缓存、编码重排、网络排队和传输、播放器缓冲、解码与显示等环节。先用采集时间戳、发送/接收时间和实际显示时刻打点，找出主要等待处；只调低播放器 buffer 可能让画面更容易卡顿，却没有解决上游编码或网络排队。

降低延迟可缩短编码队列和 GOP、减少或关闭 B 帧、让关键帧配合分片、限制网络队列、缩小抖动缓冲并丢弃过期帧。但每种办法都会影响抗抖动、画质或压缩率。用相同网络条件测延迟分位数和卡顿率，选择符合业务需求的平衡，不应只追求最低单次延迟。

代码示例

```
auto t0 = capture_timestamp(frame);
auto t1 = encoded_timestamp(frame);
auto t2 = received_timestamp(packet);
auto t3 = displayed_timestamp(frame);
report_latency(t0, t1, t2, t3); // 分段测量采集、编码、传输到显示延迟
```

048 什么是视频滤镜？常见的视频滤镜有哪些?

答案　视频滤镜是在解码后的帧上做的处理节点，可以缩放、裁剪、旋转、去噪、调色、叠图或加文字。FFmpeg 可用 -vf 指定单输入滤镜链，或用 -filter_complex 连接多个输入和输出的复杂滤镜图。每个节点可能改变帧尺寸、像素格式、帧率或时间戳，因此相邻节点的输入输出要兼容。

处理顺序会改变结果，例如先缩放再定位 overlay，与先定位再缩放不同。滤镜还可能引入缓冲和延迟；实时场景要检查帧时间戳和背压。命令行滤镜和 libavfilter API 使用同一图形概念，但资源与错误处理方式不同。先做小样验证滤镜参数，再对完整视频运行。

代码示例

```
AVFilterGraph* graph = avfilter_graph_alloc();
AVFilterContext *src = nullptr, *scale = nullptr, *sink = nullptr;
const AVFilter* scaleFilter = avfilter_get_by_name("scale");
check(avfilter_graph_create_filter(
    &src, avfilter_get_by_name("buffer"), "in", bufferArgs, nullptr, graph));
check(avfilter_graph_create_filter(&scale, scaleFilter, "scale", "1280:720", nullptr, graph));
check(avfilter_graph_create_filter(
    &sink, avfilter_get_by_name("buffersink"), "out", nullptr, nullptr, graph));
check(avfilter_link(src, 0, scale, 0));
check(avfilter_link(scale, 0, sink, 0));
// bufferArgs 应含 video_size、pix_fmt、time_base 与 pixel_aspect
check(avfilter_graph_config(graph, nullptr)); // 之后送帧、取处理帧
avfilter_graph_free(&graph); // 每个负值返回都应清理 graph
```

049 什么是转封装（Transmuxing）？它和转码（Transcoding）有什么区别？

答案　转封装是把已有压缩 packet 从一种容器读出，再写进另一容器；若 codec 与参数兼容，可不解码、不重编码，速度较快，也避免再次有损压缩。转码则要解码媒体，按需加滤镜，再用新的编码参数压缩，能够改变分辨率、codec、码率或帧率。

转封装不改变压缩画面，因此无法把 H.264 变成 HEVC；目标容器还必须支持源轨道及 codec 配置。转码会消耗计算资源，重复有损压缩可能累积画质损失。操作前确认需求是换文件包装还是换编码，并检查输出时长、音画同步、字幕轨和目标设备兼容性。

代码示例

```
// 转封装：只改 packet 容器，不解码重编码
while (av_read_frame(input, packet) >= 0)
{
    mapStreamAndRescaleTimestamps(packet, input, output);
    check(av_interleaved_write_frame(output, packet));
    av_packet_unref(packet);
}
av_write_trailer(output); // 源 codec 必须受目标容器支持
// 转码则是 send_packet -> receive_frame -> filter -> send_frame -> receive_packet
// 改尺寸/采样格式需要转码，重编码可能引入失真
```

050 什么是运动补偿（Motion Compensation）？运动矢量在其中起什么作用？

答案　运动补偿根据运动矢量从参考图像取样出预测块，再与当前图像比较。编码器通过运动估计搜索可能的位置，运动矢量描述参考块相对当前块的位移；运动补偿则使用矢量真正生成预测图像。剩余差异作为残差编码，因此矢量不是物体的真实运动轨迹。

现代编码可以使用多个参考帧、亚像素位置和更复杂的分区。更精细搜索可提升预测匹配，也会增加编码计算量。遮挡、光照变化和形变无法只靠平移向量完全描述，需要残差或其他预测模式补足。解码器依赖相同参考帧和运动矢量重建预测，若参考链损坏会影响后续画面。

代码示例

```
auto vector = search_motion_vector(current_block, reference_frame);
auto prediction = interpolate_reference(reference_frame, vector);
auto residual = current_block - prediction;
encode(vector, residual);
```

051 什么是音频帧？一帧音频包含多少个采样点？

答案　音频帧是编码器处理或传输的一个时间块，包含多少采样点由 codec、帧时长和配置决定。AAC-LC 常见每帧 1024 个采样点；48 kHz 时约 21.33 毫秒。Opus 可配置多个帧时长，从短时延到较长效率模式。PCM 本身没有统一压缩帧边界，设备回调块大小由接口和应用决定。

帧时长等于每声道样本数除以采样率；采样数不是所有 codec 都固定。较短帧常有更低处理等待，但每帧头部开销相对更大；较长帧提高效率但可能增加交互延迟。解码后音频还要按 frame.nb_samples、声道布局和样本格式处理，不能把编码 packet 字节数直接当采样点数。

代码示例

```
if (frame->sample_rate > 0 && frame->nb_samples > 0)
{
    const AVRational sampleTb{1, frame->sample_rate};
    const std::int64_t durationUs =
        av_rescale_q(frame->nb_samples, sampleTb, AVRational{1, 1'000'000});
    qInfo() << "audio frame duration us" << durationUs;
}
// nb_samples / sample_rate 才是音频帧时长
// AAC/Opus 编码块大小不同；PCM 回调块不是 codec 固定帧
```

052 什么是音频重采样？在什么情况下需要进行重采样？

答案　音频重采样是对离散采样序列做滤波和速率变换，改变采样率；常常也同时转换样本格式或声道布局。例如输入设备产生 48 kHz 浮点立体声，而编码器要求 44.1 kHz 整数 PCM，就需要重采样并重新组织数据。只修改元数据中的采样率字段不会改变实际样本，播放速度和音高都会错误。

转换器会有滤波延迟，输出样本数也可能与当前输入批次不同；必须累计实际输出，并在流末尾冲刷内部缓存。声道布局转换还涉及混音策略，不能随意复制或丢弃声道。FFmpeg 可用 SwrContext；创建配置和声道布局 API 会随版本演进，按目标版本检查函数签名和所有权。

代码示例

```
SwrContext* swr = nullptr; // 改采样率/格式/声道布局
check(swr_alloc_set_opts2(
    &swr, &stereo, AV_SAMPLE_FMT_S16, 44100, &inputLayout, AV_SAMPLE_FMT_FLTP, 48000, 0, nullptr));
```

```
check(swr_init(swr)); // 不只是改 AVFrame 的 sample_rate 字段
int capacity = swr_get_out_samples(swr, input->nb_samples);
allocateOutput(capacity);
int produced = swr_convert(swr, out, capacity, input->extended_data, input->nb_samples);
consume(out, produced); // 可能有滤波延迟
int tail = swr_convert(swr, out, capacity, nullptr, 0); // flush 尾样本
freeOutput();
av_channel_layout_uninit(&stereo);
av_channel_layout_uninit(&inputLayout);
swr_free(&swr);
```

16.4 FFmpeg 编解码 API 流程

053 使用 FFmpeg 编码视频的基本流程是什么?

答案　FFmpeg 编码视频的基本流程是建立输出容器和输出流，选择编码器，配置尺寸、像素格式、码率控制与 time_base，打开 codec 并写容器头。输入的每一帧都要填充 AVFrame 的像素平面和格式信息，按编码时间顺序设置递增 PTS，再通过 send_frame 送入编码器。

编码器输出的 packet 时间戳通常以 codec time_base 表示，写复用器前要换算到输出流 time_base，并用 interleaved_write_frame 保持多流交错。编码器可能缓存或重排帧，所以一次输入不保证立刻得到一个 packet。结束时发送空 frame 冲刷延迟数据，写 trailer，并按创建顺序释放 codec、流和 format 资源。

代码示例

```
// drainPackets 收到 EAGAIN 为止：换时基、mux 并 unref packet
check(avcodec_open2(encoder, codec, nullptr));
check(avformat_write_header(output, nullptr));
for (std::int64_t i = 0; i < frameCount; ++i)
{
    frame->pts = i; // 按 encoder time_base 递增
    int ret = avcodec_send_frame(encoder, frame);
    if (ret == AVERROR(EAGAIN))
    {
        drainPackets(encoder, output, packet);
        ret = avcodec_send_frame(encoder, frame); // 重送同一帧
    }
    check(ret);
    drainPackets(encoder, output, packet);
}
sendNullRetryingEagain(encoder, output, packet); // EOF 发送空帧
receiveAndMuxUntil(encoder, output, packet, AVERROR_EOF);
av_write_trailer(output);
// 释放 packet/frame/codec; avio_closep 后 avformat_free_context
```

054 使用 FFmpeg 解码视频的基本流程是什么?

答案　FFmpeg 解码视频时先打开输入，用 avformat_find_stream_info 探测各轨道，再找到视频流。根据 AVStream 的 codecpar 参数配置 AVCodecContext，选解码器并打开。主循环调用 av_read_frame，只将目标视频流 packet 送给解码器；send_packet 后循环 receive_frame 取出可用图像，直到 EAGAIN。

处理每帧后应按引用计数规则 unref，其他轨道的 packet 也需释放或交给各自处理器。读到 EOF 后仍要发送空 packet 冲刷解码器，取完缓存的延迟帧直至 AVERROR_EOF。错误出口要释放 packet、frame、codec context 和 format context。时间戳结合流 time_base 解释，解码顺序不一定是显示顺序。遇到 send 的 EAGAIN 时要先取出输出再重送输入包。

代码示例

```
// 已打开输入并用 codecpar 配置 decoder
while (av_read_frame(format, packet) >= 0)
{
    if (packet->stream_index == videoIndex)
    {
        int ret = avcodec_send_packet(decoder, packet);
        if (ret == AVERROR(EAGAIN))
        {
            receiveAvailable(decoder, frame);
            ret = avcodec_send_packet(decoder, packet); // 重送此包
        }
        check(ret);
```

```
        receiveAvailable(decoder, frame); // 到 EAGAIN
    }
    av_packet_unref(packet);
}
sendNullPacketRetryingEagain(decoder, frame);
receiveUntilEof(decoder, frame); // EOF 取完延迟帧
// av_packet_free、av_frame_free、avcodec_free_context、avformat_close_input
```

055 多线程在音视频处理中如何应用?

答案　音视频处理可拆成解复用、解码、滤镜、编码和磁盘/网络 I/O 等阶段，让不同数据块同时经过流水线；FFmpeg 的部分编解码器也支持帧线程或切片线程。这样可能提高多核吞吐，但阶段间有依赖，帧顺序和时间戳仍要保留，音频和视频还要共用正确的同步时钟。

各阶段之间应使用有界队列并建立背压，防止生产持续快于消费，造成内存和直播延迟不断增长。传递 AVFrame/packet 时按引用计数约定管理所有权，并传递 EOF、错误和取消状态。线程数增加不一定更快，CPU、GPU、内存带宽和磁盘可能先饱和；用端到端吞吐和尾部延迟调整流水线。队列阻塞和停止时的清理顺序也需明确。

代码示例

```
// 每个 AVCodecContext 只由所属 stage 线程调用
AVFrame* handoff = av_frame_clone(decoded); // 为消费者增加缓冲引用
if (!handoff)
    reportOutOfMemory();
if (!boundedQueue.tryPush(handoff))
{
    av_frame_free(&handoff);
    dropExpiredFrame();
}
// queue 有容量上限；满时背压或丢过期实时帧
// 消费者处理后 av_frame_free(&ownedFrame)，所有权在队列转移
// EOF/错误也作为显式状态发送，时间戳随 frame 传递
```

16.5 命令行处理与播放器功能

056 如何从视频中提取音频?

答案　若容器与源音频 codec 兼容，可以直接抽取压缩音轨，不经解码重编码，因此速度快且不会再次损失音质。命令行用 -map 指定要抽取的音频轨，-vn 排除视频，-c:a copy 保持原压缩流；例如 MP4 中 AAC 常可直接写入兼容的音频容器。

若目标容器不支持源 codec，或要改采样率、声道布局、音量，就必须解码后重新编码。抽取结果仍是 AAC 等压缩数据，不是 PCM 波形；需要逐样本处理时还需解码。多音轨文件先用 ffprobe 查看轨道索引和语言，操作后检查命令退出状态、输出轨道和时长。

代码示例

```
AVFormatContext* input = open_input(path);
int audio_index = find_stream(input, AVMEDIA_TYPE_AUDIO, language);
AVStream* audio = input->streams[audio_index];
remux_stream(input, audio, output);
// copy压缩packet不重编码；目标容器必须支持该codec
```

057 如何使用 FFmpeg 命令行合并多个视频文件?

答案　要无重编码地首尾拼接多个视频，通常使用 concat demuxer：创建文本清单列出每段文件，再由 FFmpeg 按清单读取并用 -c copy 写出。它在容器层解析并顺序送出轨道，不是把 MP4 文件的原始字节直接连接。

各片段的视频和音频流结构、codec、profile、分辨率、参数集与 time_base 应一致，否则可能失败或出现时间戳跳变。不一致时需先统一转码，或解码后用 concat filter 再编码。清单文件的路径也要限制在可信范围，不能接受不受控用户路径造成任意文件读取。实际还应检查输出首尾是否连续。

代码示例

```
std::vector<std::string> inputs{"part1.mp4", "part2.mp4"};
if (streams_compatible(inputs))
    concat_demux_and_remux(inputs, "joined.mp4");
else
    normalize_and_transcode(inputs, "joined.mp4");
```

058 如何使用 FFmpeg 命令行将视频转换为 GIF?

答案　视频转 GIF 时应限制时长、分辨率和帧率，否则文件很容易变大。GIF 的每帧调色板最多约 256 色，直接逐帧量化可能出现色带和颜色闪烁。较稳妥的流程是先用 palettegen 生成代表性调色板，再以 paletteuse 复用同一调色板，减少帧间颜色漂移。

GIF 的透明度支持有限，复杂视频往往不如 MP4/WebM 高效；要结合循环、透明背景和兼容要求选格式。缩放时应保持宽高比，帧率按运动平滑度决定。滤镜参数依 FFmpeg 版本核验，转换后检查文件大小、渐变色带、透明区域和实际播放效果。

代码示例

```
FilterGraph graph;
graph.add("fps=12,scale=480:-2,palettegen");
auto palette = graph.run(input);
graph.add("paletteuse");
encode_gif(input, palette, output);
// 共享调色板可减少闪烁；宽度和时长控制文件大小
```

059 如何使用 FFmpeg 命令行工具进行视频转码?

答案　FFmpeg 转码需先确定要改变的视频或音频参数，再选择相应编码器；例如用 libx264 编码 H.264，音频改为 AAC。CRF 是特定编码器的质量控制参数，数值较低通常画质更好、文件更大，但不保证固定码率，其他编码器的质量参数语义也可能不同。

终端兼容性决定分辨率、像素格式、profile 和 level；许多播放设备要求 yuv420p，但具体支持仍需验证。转码会消耗算力，也可能产生有损失真。完成后用 ffprobe 检查输出的 codec、尺寸、时长、声道和像素格式，再在目标播放器试播；只换容器时才考虑 -c copy。

代码示例

```
auto decoded = decode(input);
auto resized = scale(decoded, target_width, -2);
auto encoded = encode(resized, Codec::h264, crf = 23, PixelFormat::yuv420p);
write_container(encoded, output); // 转码会重新压缩，之后检查实际播放兼容性
```

060 如何使用 FFmpeg 命令行提取视频的第一帧?

答案　提取视频开头第一张解码图像可使用 -frames:v 1；FFmpeg 会把帧转换到输出图像格式支持的像素格式，例如生成 JPEG。要检查命令返回状态和文件是否产生，因为输入可能没有视频轨、文件损坏或解码失败。多视频轨时要明确选择目标轨道，避免截到封面或备用角度。

取指定时刻可用 -ss；放在 -i 前通常由 demuxer 快速跳到附近 seek point。转码解码时，FFmpeg 通常继续解码并丢弃目标前的帧以提高准确性；stream copy 则不一定能定位到精确帧，因为只能从关键帧开始。截图还要检查旋转元数据和颜色方向。

代码示例

```
AVFrame* frame = seek_and_decode(input, stream_index, timestamp);
if (frame)
    write_image(frame, "first.jpg"); // 精确时刻需解码到目标帧，不是简单复制packet
```

061 如何使用 FFmpeg 命令行给视频添加水印?

答案　图片水印可作为第二路输入，用 overlay 滤镜叠到解码后的视频帧上。滤镜修改像素，所以视频必须重新编码；音频若目标容器兼容可用 -c:a copy 保留。overlay 可指定固定坐标，也能按画面宽高计算右下角位置或用时间表达式控制何时出现。

含透明度的 PNG 要检查 Alpha 通道在处理链中是否保留，JPEG 没有透明度。叠加后还需核对输出分辨率、颜色范围和编码器兼容性；不能一面应用视频滤镜，一面要求视频流 -c:v copy。水印路径和文字若来自用户输入，应采用安全参数传递方式，避免直接拼接 shell 命令。

代码示例

```
FilterGraph graph;
graph.add("overlay=x=W-w-20:y=H-h-20:enable=between(t,0,10)");
auto watermarked = graph.run(video, transparent_logo);
encode_video(watermarked, output);
```

062 如何使用 FFmpeg 命令行调整视频的分辨率?

答案　scale 滤镜可指定目标宽高，也可固定一边让另一边按源比例计算，例如 scale=1280:-2；-2 让自动计算出的尺寸调整为偶数。强制指定宽高可能拉伸画面；若要放入固定画布，可先按比例缩小，再用 pad 补边而保留完整构图。缩放通常意味着视频需要重新编码，还应考虑插值、像素格式、色彩矩阵、range 和 SAR/PAR。像素尺寸改变不应导致显示比例错误。高质量缩放会增加计算开销，实时场景要测帧率；某些编码器要求偶数尺寸。输出后用 ffprobe 查看实际尺寸和比例，再检查是否裁掉重要区域。

代码示例

```
auto output_height = 2 * static_cast<int>(source_height * target_width / source_width / 2);
auto resized = scale(frame, target_width, output_height); // 偶数尺寸适配常见4:2:0编码
```

063 如何减少内存拷贝提升性能?

答案　减少内存拷贝应先用 profiler 找到实际 memcpy 热点，再检查数据是否被反复 clone、颜色转换或在 CPU/GPU 之间搬运。复用 AVFrame/AVPacket 缓冲并传递引用计数对象，可以避免重复分配和复制；硬件解码、滤镜、编码若能共享设备帧，也可能减少主机内存往返。

“零拷贝”仍需要说明缓冲区的所有者、同步时机和最后引用释放时机。GPU 仍在使用时不能复用帧；下游不支持同一硬件格式也可能触发隐式下载。使用有界队列避免长时间持有大量帧。性能改动要对比端到端延迟、内存峰值和吞吐，不能只测一个 memcpy。

代码示例

```
AVFrame* shared = av_frame_clone(decoded); // 新 header，共享引用计数 AVBuffer
if (!shared)
    return reportOutOfMemory();
enqueueOwnedFrame(shared); // 队列拿到所有权，消费后 av_frame_free(&frame)
// 修改前对共享帧调用 av_frame_make_writable()，可能触发复制
if (av_frame_make_writable(shared) < 0)
    reportOutOfMemory();
av_frame_free(&shared); // 释放本端引用，其他持有者仍有效
// 零拷贝仍需同步、格式兼容和最后引用释放时机
```

064 如何判断一个视频文件是否包含音频流?

答案　判断视频文件是否含音频轨，可以用 ffprobe 只选择音频流并输出 stream 索引；输出至少一行通常表示发现了音频轨。程序也可以遍历 AVFormatContext 中的 streams，统计 codec_type 为音频的流。文件可能有多条语言或评论轨，不能把“存在音频”理解为只有一条。

应同时检查 ffprobe 退出码和错误输出：正常没有音频流，与文件损坏、无权限或探测失败不同。若选择默认音轨，还要考虑 disposition、语言和用户偏好，不能固定取第一个索引。网络输入可能需先完成流信息探测才能确定所有轨道。

代码示例

```
AVFormatContext* format = nullptr;
if (avformat_open_input(&format, path, nullptr, nullptr) < 0)
    return reportProbeError();
const int probe = avformat_find_stream_info(format, nullptr);
if (probe < 0)
{
    avformat_close_input(&format);
    return reportProbeError();
}
const int audio = av_find_best_stream(format, AVMEDIA_TYPE_AUDIO, -1, -1, nullptr, 0);
if (audio == AVERROR_STREAM_NOT_FOUND)
    reportNoAudioTrack();
else if (audio < 0)
    reportProbeError();
else
    reportAudioStream(format->streams[audio]);
avformat_close_input(&format); // 无音轨与探测失败是不同状态
```

065 如何在视频上添加水印?

答案　图片水印可用 overlay，文字水印可用 drawtext；滤镜能设定位置、透明度以及只在某时间范围出现。视频滤镜处理的是解码后的图像，所以视频流需要重新编码；音频若编码兼容可用 -c:a copy 保留原压缩数据。文字滤镜还依赖字体文件，中文需要字体包含对应字符。

坐标、字号和边距要按输出画面分辨率设计。先缩放再叠加与先叠加再缩放，水印大小和位置效果不同。滤镜可能改变像素格式、颜色范围和时间戳，需检查输出。不要直接把用户输入的文字、字体路径拼成 shell 命令，以免引号和特殊字符改变命令含义。

代码示例

```
DrawTextOptions options{
    .font_file = font_path, .alpha = 0.75, .x = right_margin, .y = bottom_margin};
FilterGraph graph;
graph.add_drawtext(options);
encode_video(graph.run(frame), output);
// 抽帧核对中文字体、透明度和位置，并确认音画时间戳不变
```

066 如何处理视频文件的旋转信息?

答案　视频旋转信息可能存在于容器显示矩阵或元数据里，也可能已经变换进像素。先用 ffprobe 检查旋转标记，并确认播放器是否自动应用；转码时若输入自动旋转后又加 transpose，就会双重旋转。需要最大兼容时，可把变换真正应用到像素，再清除旧显示矩阵。

烘焙旋转后画面宽高常会互换，显示矩阵也要重置；同时检查 SAR、裁剪框和字幕位置。只删除旋转元数据不会旋转像素。无损转封装可保留矩阵，但目标容器和播放器必须支持。输出后在多个播放器验证方向，避免某个播放器恰好自动纠正而掩盖问题。

代码示例

```
std::size_t sideSize = 0;
const std::uint8_t* matrix = av_stream_get_side_data(stream, AV_PKT_DATA_DISPLAYMATRIX, &sideSize);
if (matrix && sideSize >= 9 * sizeof(std::int32_t))
{
    const double angle = av_display_rotation_get(reinterpret_cast<const std::int32_t*>(matrix));
    reportRotation(angle); // 先确认播放器是否已自动应用
}
// 转码烘焙 transpose 后清理旧显示矩阵，避免双重旋转
// side data 指针属于 stream；复制前不要跨其生命周期保存
```

067 如何实现一个简单的 PCM 音频播放器?

答案　PCM 播放器必须已知原始数据的采样率、整数或浮点样本格式、位深、声道数及交错方式，因为裸 PCM 通常没有自描述头。程序从文件分块读取样本，再送入 SDL 或系统音频 API；若设备格式不同，要先重采样或重排声道。音频回调有严格实时期限，不能在回调里等待磁盘、网络或互斥锁，也不要临时分配大量内存。使用有界环形缓冲区把读取线程与回调解耦；缓冲不足时输出静音或按策略处理。进度按已消费的每声道样本数除以采样率估算，文件字节数还受位深和声道影响。

代码示例

```
while (audio_source.read(pcm_ring.write_span()))
    pcm_ring.commit_written(samples);
void audio_callback(float* out, std::size_t n)
{
    pcm_ring.read_or_silence(out, n);
}
// 音频回调不做磁盘I/O或动态分配；播放秒数=已消费样本数/采样率
```

068 如何实现一个简单的 YUV 播放器?

答案　YUV 裸流没有文件头，播放器必须预先知道宽高、帧率、像素格式、平面顺序和行 stride。逐帧时按各平面实际行跨度读取，再传给渲染器或转换为 RGB。偶数宽高、紧密排列的 YUV420P 每帧常可估为 1.5×宽×高字节，Y 平面占一半，U/V 各占四分之一。

行对齐、奇数尺寸和 NV12/NV21 交错色度会改变实际布局，前述估算不一定适用。裸流缺少时间戳，需要按提供的帧率生成播放时钟；若格式解释错误，会出现偏色、错行或读取越界。设备要求不同布局时应显式转换，逐帧读取后再更新显示，不能依赖扩展名识别。

代码示例

```
// 裸 YUV420P 没有头：宽高/格式/帧率由调用方给定
const int chromaW = (width + 1) / 2, chromaH = (height + 1) / 2;
const std::size_t yBytes = static_cast<std::size_t>(width) * height;
const std::size_t cBytes = static_cast<std::size_t>(chromaW) * chromaH;
readPlane(file, y, width, height, yStride); // 每行按 stride
readPlane(file, u, chromaW, chromaH, uStride);
readPlane(file, v, chromaW, chromaH, vStride);
renderer.uploadYuv420(y, u, v, width, height);
// 紧密偶数尺寸才可简单算 1.5*W*H；处理 EOF/短读
```

069 如何实现视频截图功能?

答案　命令行截图可用 -frames:v 1 输出某一解码帧；-ss 指定目标附近时间。把 -ss 放在 -i 前，demuxer 通常快速跳到前一个 seek point；转码解码时再继续解码并丢弃目标前的帧以获得准确位置。输入后 seek 通常更精确但可能更慢。

stream copy 不能保证跳到任意精确帧，因为压缩帧之间有依赖，需要从随机访问点开始。批量按 fps 抽帧应控制输出数量和文件命名，避免大量图像占满磁盘。还要考虑旋转元数据、色彩范围与 JPEG 有损转换。截图工具应检查输出状态并向用户说明实际取帧时刻。

代码示例

```
const std::int64_t target = av_rescale_q(targetUs, AVRational{1, 1'000'000}, stream->time_base);
decoder->pkt_timebase = stream->time_base; // 解码输出时间戳刻度
check(av_seek_frame(format, videoIndex, target, AVSEEK_FLAG_BACKWARD)); // 先到前一随机访问点
avcodec_flush_buffers(decoder); // 丢弃 seek 前解码器状态
while (av_read_frame(format, packet) >= 0)
{
    if (packet->stream_index == videoIndex)
```

```
        decodeAndCheckPts(packet, target, frame);
    av_packet_unref(packet);
    if (gotFrameAtOrAfterTarget)
        break;
}
convertFrameToRgb(frame, image); // 保存单帧
// 输入侧 seek 后还要解码到目标；失败出口释放 packet/frame/context
```

070 如何实现视频的倍速播放?

答案　倍速播放要让视频、音频和播放时钟保持同一速率。播放器可按倍率推进媒体时钟，依据 PTS 加快视频呈现；超过显示能力时丢帧或调整刷新。音频需单独处理，否则简单跳过样本虽然改变速度，也会改变音调。atempo 等变速滤镜通过时域处理尽量保持音高，但有支持范围和音质代价。

倍速时解码器和网络要持续供给数据，队列需有上限，否则会积累延迟和内存。暂停、seek 和切换倍率时要重设播放时钟基准，避免音画突然跳动。倍率变化也会影响音频处理和丢帧策略，应实测低倍率、高倍率和数据欠载。

代码示例

```
clock.set_rate(playback_speed);
decoder.set_drop_policy(DropLateFrames::enabled);
audio_filter = atempo_chain(playback_speed); // 简单重采样会改变音调
// seek/暂停后重置时钟基准，队列长度保持有界
```

071 如何实现视频的快进和快退?

答案　快进通常先 seek 到目标之前的随机访问点，再继续解码并丢弃目标时间以前的帧；只跳过 packet 可能跳过参考图像，造成花屏。显示准确位置取决于关键帧间隔和解码策略。跳到目标后恢复正常播放时，也要同步音频时钟并清理旧队列。

快退不能简单按文件字节反向读取，因为压缩帧依赖前序参考。常见办法是反复 seek 到较早关键帧，正向解码目标 GOP，把帧缓存后反序显示；长 GOP 和高分辨率会增加等待与内存。快退时通常静音或单独处理音频，并限制缓存大小。

代码示例

```
const std::int64_t target = av_rescale_q(seekUs, AVRational{1, 1'000'000}, stream->time_base);
decoder->pkt_timebase = stream->time_base; // 解码输出时间戳刻度
check(av_seek_frame(format, videoIndex, target, AVSEEK_FLAG_BACKWARD)); // seek 到关键帧附近
avcodec_flush_buffers(decoder); // 清空旧参考帧
while (readVideoPacket(packet))
{
    sendAndReceiveAll(decoder, packet, frame);
    if (frame->pts != AV_NOPTS_VALUE && frame->pts >= target)
        display(frame);
    av_packet_unref(packet);
    if (targetFrameReady)
        break;
}
// 快退可重复 seek 到更早 GOP 并缓存帧；直接跳 packet 会破坏依赖链
// seek 后同步音频时钟；最终 free packet/frame/codec/format
```

072 如何实现视频的拼接和合并?

答案　参数相同的视频首尾拼接可用 concat demuxer 清单，并用 -c copy 避免重新压缩。concat 协议则是按字节连接文件，只适用于少数具有可直接物理拼接结构的容器，并不是一般视频合并方法。输入片段需有相同轨道结构和编码参数，音视频时间戳也要连续。

参数不同的片段应解码后经 concat filter 拼接，先统一画面尺寸、帧率、像素格式和 SAR，并统一音频采样率、格式及声道布局，再编码输出。否则滤镜输入可能不兼容或产生时间跳变。分段首尾还要检查画面、声音和字幕；转码时应重新计算连续 PTS。

代码示例

```
if (all_stream_parameters_match(segments))
    remux_concat(segments, output); // copy仅复用packet
else
{
    auto frames = decode_all(segments);
    auto normalized = normalize(frames);
    encode_concat(normalized, output);
}
```

073 如何实现视频的旋转?

答案　直角旋转可用 transpose 滤镜，例如 clock 和 cclock 表示不同方向，需通过带方向标记的测试帧确认实际结果。任意角度可用 rotate 滤镜。滤镜默认输出画布可能裁掉旋转后的四角，应显式计算输出宽高或配置 ow/oh，并指定边缘填充颜色。

若文件已有旋转显示矩阵，先决定保留元数据还是把旋转烘焙到像素，避免重复变换。像素旋转会改变宽高，需检查 SAR、字幕和水印坐标；滤镜处理需要重新编码视频。完成后核对输出尺寸、播放方向和音频同步，不同播放器对显示矩阵的支持可能不同。

代码示例

```
auto rotation = probe_display_matrix(input);
auto rotated = transpose(frame, rotation);
clear_display_matrix(output); // 像素已旋转时清除旧矩阵，避免播放器旋转两次
```

074 如何实现视频的缩放?需要注意什么?

答案　scale 滤镜或 libswscale 可把帧变为目标宽高。要保持显示比例，常固定一边按比例计算另一边，或使用等比缩放后补边；否则会把画面拉伸。还需考虑源 SAR/PAR、目标色度抽样对偶数尺寸的要求，以及画面重要区域是否需裁切。

颜色转换和缩放要匹配 BT.601/709 等矩阵及 full/limited range，像素格式标签应与真实数据一致。更改尺寸或像素格式一般要重新编码。插值方式影响速度和锐度，实时处理应测帧率。输出后用 ffprobe 检查尺寸、SAR 和格式，并检查色彩和几何比例。

代码示例

```
auto scaled = scale(frame, target_width, keep_aspect_ratio = true);
auto canvas = pad(scaled, canvas_width, canvas_height); // 保留比例并补边
```

075 如何实现视频的裁剪?

答案　crop 滤镜按 crop=w:h:x:y 从每帧左上角偏移 x、y 的位置截出 w×h 区域。例如用输入宽高表达式取中心半宽、半高，可以去掉边缘或聚焦主体。裁剪坐标必须落在原图范围内；动态裁剪还要限制变化速度，避免画面抖动。

4:2:0 等色度抽样常要求裁剪边界对齐到偶数，否则色度平面定位可能不兼容。裁剪会改变像素尺寸，视频通常需要重编码；若要保持显示比例还可能需补边或更新 SAR。裁剪前预览框位置，输出后确认没有误去字幕、台标或关键动作。

代码示例

```
Crop crop{.x = x, .y = y, .width = width, .height = height};
if (crop.inside(frame.bounds()) && crop.aligned_for(PixelFormat::yuv420p))
    encode_video(apply_crop(frame, crop), output);
```

076 如何实现音视频的分离和合并?

答案　分离音视频时可用 -vn 去掉视频、-an 去掉音频，并在目标容器支持时以 -c copy 保留压缩流。-map 可明确从多个输入选择哪条轨道，避免自动选错语言或字幕。合并独立音视频时分别指定视频文件和音频文件，再映射各自轨道到同一输出。

源 codec、采样率、time_base 或目标容器不兼容时需转码。两个输入的起始时间可能不同，必须检查时间戳偏移，避免音画不同步。复制 packet 不会修复损坏数据或改变编码。操作后探测输出轨道数和时长，并在播放器验证音画起点、结尾和同步。

代码示例

```
auto video = select_stream(video_input, MediaType::video);
auto audio = select_stream(audio_input, MediaType::audio);
if (compatible(video.codec, audio.codec, container))
    mux(video, audio, output);
// 有时间戳偏差时先校正音频起点，再检查最终同步
```

077 如何提取视频的关键帧?

答案　ffprobe 可列出第一视频流每个解码帧的 key_frame 标记和时间戳，便于找关键帧时刻。真正提取关键帧图像时，还需 seek 到附近并解码出 AVFrame；压缩 packet 不是可直接保存的 JPEG 或 PNG。按逐帧时间戳定位比按固定秒数猜位置更准确。

codec 的 key_frame 标记不总等于 IDR 或完全独立的随机访问点；开放 GOP 可能有前置依赖。若用于切片或错误恢复，应确认编码结构、参数集和目标解码器能从此处开始。批量提取要限制数量和输出目录，避免长视频写出大量图片。

代码示例

```
// receive_frame 成功后处理，返回 EAGAIN 时送下一 packet
while (decodeNextFrame(decoder, format, packet, frame))
{
    if (frame->flags & AV_FRAME_FLAG_KEY)
    { // 帧标记不一定等于 H.264 IDR
        convertToImage(frame, image);
        saveImage(image, makeTimestampedPath(frame->pts));
```

```
    }
    av_frame_unref(frame); // 接收后复用前释放引用
}
// 真正随机访问还需 seek 到关键帧并解码依赖帧
// 结束时 free frame/packet/codec/format
```

078 如何计算一帧 YUV420P 图像占用的内存大小?

答案　偶数宽高、8 位、紧密排列的 planar YUV420P，一帧可见数据大小为 WH + 2×(W/2)×(H/2)，即 1.5WH 字节。1920×1080 时为 3,110,400 字节。这个估算假设三个平面没有额外行填充，而且 U/V 尺寸正好是宽高各一半。
AVFrame 实际分配通常按每个 plane 的 linesize 乘以平面高度计算，stride 对齐会增加空间。奇数宽高时色度尺寸要按格式向上取整；硬件帧还可能位于专用显存，CPU 地址空间可见字节数不一定代表设备实际占用。逐行访问和分配时应读取实际帧参数，不能只用可见图像大小推断；计算值也不含播放器缓存和其他工作区。
代码示例

```
const std::size_t visibleY = static_cast<std::size_t>(width) * height;
const std::size_t chromaW = (width + 1) / 2, chromaH = (height + 1) / 2;
const std::size_t visible = visibleY + 2 * chromaW * chromaH; // Y/U/V
const std::size_t allocated = static_cast<std::size_t>(std::abs(frame->linesize[0])) * height +
                              static_cast<std::size_t>(std::abs(frame->linesize[1])) * chromaH +
                              static_cast<std::size_t>(std::abs(frame->linesize[2])) * chromaH;
// linesize 含对齐；这里只是可见容量 vs 常规 planar 分配估算
```

079 如何读取一个视频文件的基本信息（时长、分辨率、码率等)?

答案　ffprobe 能同时读取容器级信息和逐轨信息。使用 -show_entries 指定 duration、bit_rate、codec_type、width、height、r_frame_rate 等，再输出 JSON 供程序解析。分辨率只适用于视频轨，音频要看采样率和声道；多轨文件应遍历每个 stream，而不是假设只有一条。
duration 与 bit_rate 可能缺失、估算或代表整个容器，并不一定等于某条流的精确值；可变帧率视频的 r_frame_rate 也未必是实际平均帧率。探测时检查命令退出码和 JSON 解析错误。编辑和同步若要求精确，应依据 packet/frame 时间戳，而非只依赖容器摘要。网络流有时需多读数据才能得到完整信息。
代码示例

```
AVFormatContext* format = nullptr;
if (avformat_open_input(&format, path, nullptr, nullptr) < 0)
    return reportOpenError();
if (avformat_find_stream_info(format, nullptr) < 0)
{
    avformat_close_input(&format);
    return reportProbeError();
}
qInfo() << "duration us" << format->duration; // 单位 AV_TIME_BASE
for (unsigned i = 0; i < format->nb_streams; ++i)
{
    const AVStream* s = format->streams[i];
    qInfo() << s->codecpar->codec_type << s->codecpar->width << s->codecpar->height
            << s->avg_frame_rate.num << s->avg_frame_rate.den;
}
avformat_close_input(&format); // duration/bitrate 可能缺失或估算
```

080 常见的硬件加速方案有哪些?

答案　常见硬件编解码接口包括 NVIDIA NVENC/NVDEC、Intel Quick Sync、AMD AMF、Apple VideoToolbox、Linux VA-API，以及 Windows DXVA/D3D11VA。它们是厂商或系统媒体接口；支持的 codec、profile、位深、表面格式和并发量取决于芯片、驱动及 FFmpeg 构建配置。
ffmpeg -encoders、-decoders、-hwaccels 列表可作初筛，但出现接口名不代表设备一定能建立上下文。硬件帧与软件滤镜之间可能触发昂贵复制。应在目标机器创建真实设备、处理代表视频，并测吞吐、功耗、画质和失败回退，不要只在开发机推断。同一接口在不同代芯片上的能力也可能不同，因此以目标设备实测作为依据。
代码示例

```
AVHWDeviceType type = AV_HWDEVICE_TYPE_NONE;
while ((type = av_hwdevice_iterate_types(type)) != AV_HWDEVICE_TYPE_NONE)
{
    qInfo() << av_hwdevice_get_type_name(type); // 当前构建可识别的后端
}
// 再按 codec 的 AVCodecHWConfig 选择像素格式和设备类型
// NVENC/VideoToolbox/VA-API 等能力由硬件、驱动、构建共同决定
// 实际创建设备并解码验证；失败时回退软件路径
```

16.6 格式、性能与播放质量

081 常见的视频封装格式有哪些？MP4、FLV、MKV、AVI 各有什么特点？

答案　MP4 通常兼容性较广，常用于点播和下载；Matroska/MKV 封装灵活，适合多轨、字幕和附件；FLV 曾广泛用于 RTMP 直播工作流；AVI 是较早容器，对新 codec、时间戳和部分媒体功能支持有限。它们在轨道索引、元数据和时间组织方式上不同。

容器不决定内部采用什么视频或音频 codec，也不保证播放器支持其中每种 profile、字幕类型与参数。选择时考虑目标终端、是否需要多音轨字幕、点播或直播、索引恢复能力。容器转换有时只需转封装；若 codec 不兼容则要转码，并评估处理耗时和画质损失。还要确认目标平台的字幕与流式播放实现。

代码示例

```
Container target = choose_container(player, tracks, subtitles);
if (target.supports_all(tracks))
    remux(tracks, target);
else
    transcode_unsupported_tracks(tracks, target);
```

082 常见的音频编码格式有哪些？AAC、MP3、Opus 各有什么特点？

答案　MP3 兼容面很广，AAC 常见于移动设备和网络视频，Opus 面向低延迟交互并兼顾语音和音乐；三者都是有损音频编码。AAC 和 MP3 生态成熟，Opus 在低码率语音和实时通话中常有优势，但实际效果不能只凭格式名称判断。质量会受编码器实现、码率、采样率、声道布局和输入素材影响，同码率下也没有适用于所有场景的固定排序。还要核对目标播放器和容器是否支持目标 codec。若追求存档或后期制作，应使用无损或未压缩格式，不要把高码率有损文件称为无损。

代码示例

```
for (Codec codec : {Codec::aac, Codec::mp3, Codec::opus})
{
    auto sample = encode(source_audio, codec, bitrate);
    compare_quality_size_and_cpu(sample);
} // 语音与音乐、目标终端兼容性共同决定选择
```

083 直播推流时如何处理网络波动？

答案　推流端应监控发送队列长度、RTT、丢包和估计带宽，依据网络变化调整分辨率、帧率或码率，避免持续发送超过链路容量。发送队列必须有上限；实时内容过期后应优先丢弃，而不是无限排队，否则延迟会越来越大。关键帧间隔会影响丢包后的画面恢复时间。

可结合重传、FEC 或丢弃过期帧，但恢复策略必须符合剩余播放时限。TCP 会可靠重传，却可能因队头阻塞加剧时延；UDP 延迟策略灵活，但需要应用协议检测丢包与拥塞。服务端和播放器缓冲也会影响结果，需监测端到端延迟和卡顿率。

代码示例

```
if (network.queue_age() > max_latency)
    encoder.set_bitrate(lower_rate);
if (packet.is_past_play_deadline())
    drop(packet);
report_loss_rtt_and_stall();
// 发送队列有界；按互动延迟要求调码率、重传或FEC
```

084 码率控制有哪些方式？CBR、VBR、ABR、CRF 分别适用于什么场景？

答案　CBR 让目标码率较稳定，适合码率预算严格的广播或受限链路，但复杂场景可能画质下降，静态场景则可能浪费比特。VBR 根据画面复杂度调整码率，常用于质量优先的文件。ABR 以较长时间平均码率为目标，允许短期波动。CRF 是部分编码器提供的恒定质量控制，码率会随内容改变，不能承诺固定文件大小或峰值带宽。编码器之间对 CBR、VBR、ABR 和 CRF 的名称及参数含义可能不同，还会有 VBV 等峰值限制设置。根据分发平台、带宽和存储预算选择，并测量平均与峰值码率。

代码示例

```
encoder->bit_rate = targetBps; // 目标平均速率
encoder->rc_max_rate = peakBps;
encoder->rc_buffer_size = bufferBits; // CBR/VBV 类约束
av_opt_set(encoder->priv_data, "crf", "23", 0); // 某些编码器的质量模式
if (avcodec_open2(encoder, codec, nullptr) < 0)
    return reportOptionError();
writePacketsUntilFlush(encoder);
// CRF 不保证固定文件大小；选项名称/范围依具体编码器
// 同一模式还受分辨率、场景复杂度和编码器实现影响
```

085 视频播放出现花屏的可能原因有哪些?

答案　花屏可能来自网络丢包或损坏 packet、参考帧缺失、NAL 格式不匹配、解码器驱动问题，也可能是显示端把像素格式、plane 指针或 stride 解释错。先用 ffprobe 检查流信息，再用软件解码确认码流本身是否异常；若软件正常、硬件异常，重点查驱动和硬解路径。

逐阶段对比压缩 packet、解码后的 AVFrame 和渲染纹理，记录错误日志、关键帧恢复点和尺寸。YUV420P、NV12/NV21 平面顺序混淆会呈现明显色块或偏色。花屏只在丢包后出现时，等待随机访问点或请求关键帧可能恢复；先区分网络、解码和显示问题再修复。

代码示例

```
if (frame->format == AV_PIX_FMT_NONE)
    reportMissingPixelFormat();
for (int plane = 0; plane < AV_NUM_DATA_POINTERS; ++plane)
{
    if (frame->data[plane])
        qInfo() << plane << frame->linesize[plane];
}
if (frame->hw_frames_ctx)
{
    check(av_hwframe_transfer_data(cpuFrame, frame, 0)); // 对比软件可读画面
}
// 检查 packet 丢失、参考帧、plane/stride、range 与矩阵
// 用关键帧恢复并比较软件/硬件解码区分输入与渲染问题
```

086 视频播放卡顿的可能原因有哪些？如何优化?

答案　先分别测网络接收/解复用、解码、滤镜、渲染和音频回调耗时，观察 packet/frame 队列深度与 PTS。卡顿可能由网络欠载、解码跟不上、主线程绘制过慢或音视频时钟漂移造成；盲目加大 buffer 会暂时掩盖短抖动，却增加延迟。根据测量结果优化对应阶段：启用目标设备支持的硬解、异步 I/O、预读和有界队列，或降低输出尺寸与滤镜成本。欠载时补充数据，过载时丢弃过期帧并降码率。保持音频主时钟和视频同步策略清楚，比较卡顿率、CPU、内存和延迟的变化。

代码示例

```
report(decode_latency(), render_latency(), queue_depth(), frame_pts_drift());
if (decoder_is_bottleneck())
    enable_supported_hardware_decode_or_lower_resolution();
// buffer变大可能减少卡顿，也会提高端到端延迟
```

087 解释 GOP（Group of Pictures）的概念，GOP 大小对视频质量和码率有什么影响?

答案　GOP 是从随机访问图像开始、由后续预测图像组成的编码结构，常用帧数或时间表示长度。长 GOP 减少关键帧比例，通常提升压缩效率，但新观众切入、seek 和丢包后的恢复会更慢。短 GOP 带来更多随机访问点，利于切片对齐与恢复，但关键帧会增加码率。

GOP 的开闭结构、B 帧和 IDR 设置也影响图像依赖。开放 GOP 中边界后的图像可能引用前一段，切片不一定能独立解码。帧率变化时按时间定义间隔更可靠。流媒体应结合分段时长、延迟、抗丢包和带宽测试，而不是只用固定 GOP 数值。

代码示例

```
encoder->gop_size = fps * gopSeconds; // 约每 gopSeconds 秒一次随机访问点
encoder->max_b_frames = 2; // 预测效率/重排延迟的取舍
encoder->keyint_min = encoder->gop_size / 2; // 如编码器支持
check(avcodec_open2(encoder, codec, nullptr));
for (AVFrame* f : frames)
    sendFrameAndDrainPackets(encoder, f);
// 切片边界通常要对齐 IDR/随机访问帧
// 长 GOP 省关键帧开销但切入/恢复更慢；最后 free codec context
```

088 解释熵编码的概念，CAVLC 和 CABAC 有什么区别?

答案　熵编码把量化后的符号按统计概率转换为紧凑码字，本身是无损步骤；有损信息主要在量化阶段被丢弃。H.264 的 CAVLC 根据邻近上下文做可变长度编码，结构较简单；CABAC 采用上下文建模和算术编码，通常压缩效率更高，但解码计算量也更大。

编码器和解码器必须支持同一 profile 与熵编码工具。H.264 Baseline 不支持 CABAC，Main 和 High profile 可支持；因此设置编码参数时要检查输出 profile 和目标硬件能力。CABAC 不是任何视频标准都采用的统一名称，也不能简单把压缩率优势等同于画质提升。

代码示例

```
auto symbols = quantize(transform(residual));
if (profile == Profile::baseline)
```

```
    write_cavlc(symbols);
else
    write_cabac(symbols, context_model);
// 熵编码可无损还原量化符号；量化本身可能有损
```

089 运动估计（Motion Estimation）的原理是什么？常用的运动估计算法有哪些？

答案　运动估计是编码器为当前图像块寻找参考帧中相似区域的过程。候选位置与当前块的偏移形成运动矢量，之后编码器用它生成预测块，并对预测残差编码。搜索可在整数像素甚至亚像素位置进行；更细搜索通常增加计算量，但可能找到更好的匹配。

全搜索检查许多候选，效果常较好但成本高；菱形、六边形或金字塔分层搜索通过减少候选加快处理。实际编码器还会结合多参考帧、亚像素细化和率失真代价。运动矢量描述压缩预测关系，不一定对应物体真实运动，遮挡、形变和光照变化都会影响估计。

代码示例

```
auto candidates = fast_motion_search(current_block, reference_frame);
auto best = minimize_rate_distortion(candidates, lambda);
auto vector = refine_subpixel(best);
encode_motion_vector(vector); // 搜索结果不一定等于真实物体运动
```

090 量化（Quantization）在视频编码中的作用是什么？QP 值如何影响视频质量？

答案　量化把变换后的系数按量化步长缩小并取整，是视频有损失真的主要来源之一。H.264 的 QP 通常范围为 0–51；QP 越高，步长越大，细节被舍弃得更多，质量往往降低。降低 QP 通常保留更多细节，但会产生更大的码流。

QP 与码率不是简单线性关系。H.264 中 QP 每增加 6，量化步长约翻倍，但实际码率变化还取决于纹理、运动、预测模式和码控；固定 QP 的不同场景码率也会不同。CRF 等质量控制会动态调整各帧参数。应以目标内容测试画质、码率峰值和存储量。

代码示例

```
// 仅当所选编码器支持其私有参数时设置
if (av_opt_set(encoder->priv_data, "qp", "23", 0) < 0)
    reportUnsupportedQpMode();
check(avcodec_open2(encoder, codec, nullptr));
sendFrameAndDrainPackets(encoder, frame);
// H.264 QP 越高量化步长越大；较低通常保留细节也增码率
// QP 每升 6 步长约翻倍，但码率不保证减半
// CRF 与固定 QP 是不同控制模式
```

091 零拷贝技术在音视频处理中如何应用？

答案　零拷贝是减少整帧在 CPU 内存或设备表面之间复制的做法，可通过引用计数 AVBufferRef、DMA-BUF、硬件帧共享、内存映射或同一设备上的表面传递实现。它能降低内存带宽和延迟，但只有解码器、滤镜与编码器支持兼容的像素格式、设备和上下文时才能贯通。

同步栅栏、缓冲区所有权和生命周期仍必须正确管理；消费者完成前不能复用生产者的表面。跨设备、颜色格式转换或软件滤镜常会触发读回和复制，所以“零拷贝”通常是减少某段链路复制，不一定全流程完全没有复制。用性能工具确认真实数据路径，并同时监测队列背压和显存占用。

代码示例

```
AVFrame* handoff = av_frame_clone(decoded); // 共享 AVBufferRef 帧数据
if (!handoff)
    return reportOutOfMemory();
queue.push(handoff); // 队列持有引用，消费者处理完 av_frame_free
if (needsWrite)
    check(av_frame_make_writable(handoff)); // 共享时可能复制
// 硬件帧贯通还要求相同设备/格式并处理同步 fence
// 跨设备读回或像素格式转换会重新产生拷贝
```

092 音频采样格式有哪些？Planar 和 Packed 格式有什么区别？

答案　采样格式描述单个样本是整数还是浮点、占多少位，例如 S16、S32 或 FLT；它与声道布局是两项独立信息。Packed 把同一采样时刻各声道的样本交错存放，例如立体声 L0、R0、L1、R1；Planar 则为每个声道单独提供 plane，例如 FLTP 有左右两路 float 数组。

读取 AVFrame 时检查 format、ch_layout、nb_samples 和 data/extended_data；不能仅按 channels 推断内存布局。高声道数时额外 plane 还可能使用 extended_data。采样位宽和布局错误会造成噪声、声道交换或越界。进入编码器或重采样器前应确认它支持所选格式，必要时用 libswresample 转换。还要用 linesize/采样数正确计算各 plane 的可读长度。

代码示例

```
const auto format = static_cast<AVSampleFormat>(frame->format);
qInfo() << av_get_sample_fmt_name(format) << frame->nb_samples << frame->sample_rate;
```

```
if (av_sample_fmt_is_planar(format))
{
    for (int ch = 0; ch < frame->ch_layout.nb_channels; ++ch)
        consumeChannel(frame->extended_data[ch], frame->nb_samples); // 每声道 plane
}
else
{
    consumePacked(frame->data[0], frame->nb_samples * frame->ch_layout.nb_channels);
}
// 读取前检查 ch_layout/linesize; planar 和 packed 指针布局不同
```

第 17 章 Python

17.1 Python 对象、类型与容器

001 Python 的动态类型和强类型分别约束什么？

答案　“动态类型”说的是名字与类型的关系：名字不必预先声明一个固定的静态类型，运行时可以先绑定整数对象，之后再绑定字符串对象。对象本身始终有确定类型，该类型决定它支持哪些操作。动态类型不代表对象可以随意改造成任何类型。

“强类型”通常描述类型之间的运算边界较明确，例如字符串和整数不能直接相加，想转换时应显式调用 int 或 str，并处理转换失败。这两个维度并不矛盾：Python 是动态类型语言，也通常被称为强类型语言；数值类型之间仍有语言定义的混合运算规则，强类型不等于完全没有隐式转换。〔N161、N162〕

代码示例

```
value = 3  # 名字重新绑定后，引用的是另一个类型的对象
value = "3"
result = int(value) + 2  # 跨类型运算先显式转换
```

002 Python 常见类型、对象可变性和函数传参是什么关系？

答案　Python 的变量名保存的是对象的绑定。给另一个名字赋值通常只是让它也指向同一个对象，并不复制对象。数字、str、tuple、frozenset 等常见对象不可变；list、dict、set 和 bytearray 可变。tuple 不可变只表示其元素引用不能被换掉，元素所指的列表仍可能修改。

函数调用时，形参获得对象引用：给形参重新绑定新对象只影响局部名字，但通过共享引用原地修改列表，调用者也能看见。+= 会按类型尝试原地更新：列表常在原对象上追加，整数则通常产生新整数再重绑名字。判断函数副作用要看具体对象的可变性与别名关系，而不能把 Python 传参笼统说成值拷贝或引用传递。〔N161、N162〕

代码示例

```
def modify(items, number):
    items.append(1)  # 原地修改共享列表；整数加法只重新绑定局部名字
    number += 1
values, count = [], 0
modify(values, count)  # values变为[1]，count仍为0
```

003 Python 中如何区分对象身份、值相等和真假判断？

答案　is 判断两个名字是否指向同一个对象，== 则调用对象的相等比较，表达业务值是否相等。除 None 等单例判断外，通常用 == 比较数值和文本。None、False、0 和空列表是不同对象或类型，但在条件语句中可能都为假；判断可选值时要写 is None，以免把合法的零或空容器误判为缺失。

CPython 可能缓存小整数或驻留字符串，让某些值恰好共享身份，但这是实现细节，不能用 is 比较数字或字符串。类还可以通过 __bool__ 或 __len__ 自定义真假，因此真假不等于是否为 None。写条件时先确定需要判断身份、值相等、还是对象的真假语义。〔N161、N163〕

代码示例

```
value = 0
if value is None: print("missing")  # 零是有效值，不等于未提供
left, right = [1], [1]
print(left == right, left is right)  # 值相等，但不是同一对象
```

004 and、or 为什么不一定返回 bool？

答案　and 和 or 会短路，并返回选中的操作数本身，而不保证返回 bool。x and y 在 x 为假时直接返回 x，否则计算并返回 y；x or y 在 x 为真时直接返回 x，否则才计算并返回 y。not 则会对真假值取反并返回布尔值。短路可以跳过昂贵计算，也可能让右侧函数完全不运行。

value or default 是常见默认值写法，但当 value 为 0、空字符串或空容器时也会采用 default。若仅 None 表示缺失，应用显式条件判断。不要用 and/or 写依赖于对象真假值的复杂赋值；明确的 if 或条件表达式通常更容易看出右侧是否会执行。〔N162、N163〕

代码示例

```
name = ""
result = name or "anonymous"  # and/or返回操作数，并按需短路
count = 0
fallback = 10 if count is None else count  # 只把None当作缺失值
```

005 type 与 isinstance 如何处理继承和抽象接口？

答案　type(obj) 返回对象的实际类型；type(obj) is T 判断精确类型，所以 T 的子类不会匹配。isinstance(obj, T) 判断是否属于该类型或其子类，也能与抽象基类的注册关系配合。比如 bool 是 int 的子类，因此 isinstance(True, int) 为真。

当代码确实需要区分某个精确表示时，可用 type 检查；通常应按对象提供的行为或抽象协议编程，避免维护大量具体类型分支。若 bool 不应被当作整数输入，可以先明确排除它或要求精确的 int 类型。类型检查是接口选择工具，不会自动验证对象方法的全部语义或副作用。〔N164、N161〕

代码示例

```
value = True  # bool是int的子类
if isinstance(value, bool): raise TypeError("bool is not accepted")
if type(value) is int: use_exact_integer(value)  # 精确类型检查
```

006 Python 的 list、tuple、dict、set 和 deque 如何选择？

答案　按常见操作选择容器：list 适合可变顺序数据和下标访问，也可用 append/pop 实现栈；tuple 适合位置意义固定的组合；dict 保存键值映射；set 适合成员判断和集合运算。dict 键与 set 元素必须可哈希，且相等对象会被视为同一键或成员。deque 适合两端频繁增删。

优先队列可用 heapq 在列表上维护最小堆，查看堆顶 O(1)，插入和移除 O(log n)；多线程场景可考虑 queue.PriorityQueue。Python 没有内建通用链表或树容器，通常用自定义类、嵌套对象或 dict/list 邻接表表示。容器性能还受操作比例和数据规模影响，选型要按语义与测量决定。〔N163、N165〕

代码示例

```
stack = []; stack.append(task); latest = stack.pop()
lookup = {"id": 7}; unique = {1, 2, 3}; queue = collections.deque()
queue.appendleft(first); queue.append(last)  # 按访问方式选择容器
```

007 CPython 的列表如何保存异构元素，常见操作复杂度是什么？

答案　CPython 的 list 通常是一个可调整容量的对象指针数组，元素对象本体可能分散在内存各处；所以列表能保存不同类型，是因为指针可以指向不同类型对象。按下标访问通常 O(1)，查找值通常 O(n)，中间插入或删除要移动后续引用，通常也是 O(n)。

append 每次添加一个元素，extend 会依次加入可迭代对象中的所有元素；insert 把一个对象放到指定位置。动态数组会预留额外容量，使连续尾部追加的均摊成本为 O(1)，但单次扩容可能重新分配并搬动 O(n) 个引用。具体增长公式属于 CPython 实现细节，不是 Python 语言的性能保证。〔N166、N163〕

代码示例

```
items = ["A", 7, object()]  # list可保存不同类型对象的引用
items.append("last")  # 尾部追加均摊O(1)
items.insert(1, "middle")  # 中间插入要移动后续引用，O(n)
```

008 Python 容器复制、切片和重复为什么会产生共享嵌套对象？

答案　变量赋值只是增加一个对象引用；切片、list.copy 和 copy.copy 通常创建新的外层容器，却保留内部元素引用。因此修改某个嵌套列表时，浅拷贝两边仍会看到变化。deepcopy 会递归复制并记录对象映射以处理环和重复引用，但类型可以定制其行为，外部资源也未必适合复制。

[[0] * 3] * 3 会把同一个内层列表重复三次，写入一行的元素会影响其他行。要构造相互独立的行，应在推导式的每次迭代里新建列表。tuple 本身不能替换其中引用，但里面若保存列表，列表仍然可变。只有在确实需要独立嵌套状态时才使用深拷贝。〔N167、N163〕

代码示例

```
wrong = [[0] * 3] * 3  # 三行引用同一个内层列表
matrix = [[0] * 3 for _ in range(3)]  # 每行独立创建
matrix[0][0] = 1
assert matrix[1][0] == 0  # 修改第一行不影响其他行
```

009 sort 与 sorted 的修改行为及稳定排序规则是什么？

答案　list.sort 在原列表上排序并返回 None；sorted 接收任意可迭代对象并返回新列表，不改变输入。两者都支持 key 提取比较键和 reverse 反转方向，并保证稳定排序：比较键相同的项目保留原有相对顺序。多个字段可组合成元组 key 按字典序比较。

若不同字段需要不同排序方向，可从次要字段开始多次稳定排序，再按主要字段排序。排序前要确认键之间可比较，特别是混合 None、字符串和数字时，默认排序未必有效。key 应表达业务排序含义，避免依赖对象地址、类型偶然次序或未经规范化的输入。〔N163、N164〕

代码示例

```
ordered = sorted(users, key=lambda user: (user.age, user.name))  # 返回新列表且排序稳定
users.sort(key=lambda user: user.age)  # 原地排序，返回None
```

010 str、bytes 和字符串拼接要注意哪些编码边界？

答案　str 表示 Unicode 文本，bytes 保存字节序列；两者之间用 encode 和 decode 根据明确编码转换。文件、网络和数据库边界应统一编码，遇到坏数据时按协议决定拒绝、替换或报告，不要随意忽略错误。字符串不可变，循环反复拼接可能产生多次分配与复制。

已有多个片段时可用 join 一次组合；若流式写出大结果，则逐块写文件，避免先把全部内容放进一个大字符串。文本字符数、用户看到的字形数和 UTF-8 字节数可能不同，截断或协议长度计算要先确定单位。bytes 可以含零字节，不能依赖文本终止符来计算二进制数据长度。〔N163、N168〕

代码示例

```
text = "猫"
data = text.encode("utf-8")  # 字符数与编码后的字节数不同
joined = "\n".join(["first", "second"])  # 多段文本一次拼接
```

011 推导式与生成器表达式在执行时机和内存上有什么差别?

答案　列表、集合和字典推导式会立即遍历输入并创建结果，适合需要重复使用、随机访问或后续修改的情况。生成器表达式按需提供下一个值，通常只保存迭代状态，适合大数据流水线。惰性不会减少必须处理的元素数量，也不能保证下游不缓存所有结果。

生成器表达式的最左侧可迭代表达式会立即求值，后续取值、条件和计算一般在消费时执行，所以异常和外部状态变化的时机不同。若表达式带副作用或有复杂嵌套，普通 for 循环更容易读懂。选生成器的理由应是延迟或流式处理，而不是认为它对所有代码都自动更快。〔N162、N163〕

代码示例

```
records = (parse(line) for line in file)  # 生成器按需读取和解析
first = next(records)  # 此时才处理第一行
all_rows = [parse(line) for line in file]  # 列表推导式立即构造结果
```

012 CPython 字典如何查找、处理冲突并扩容?

答案　CPython 的 dict 通常用哈希表定位键：先算哈希，根据索引找到候选槽位，再按探测规则处理冲突，并用相等比较确认键是否相同。删除的槽位需要保留一种“曾占用”的标记，否则查找可能在探测过程中提前误以为键不存在。空闲比例下降后，表可能扩容并重建索引。

查找和插入平均通常为 O(1)，但哈希冲突、重建或输入分布可能让操作变慢，最坏情况并非 O(1)。现代 CPython 将索引与条目存储区分开，并有实例属性等专门布局；探测细节、容量阈值和自由线程构建下的实现会随版本变化。面试应说明一般原理和平均复杂度，不把某个版本源码常数说成语言保证。〔N169、N163〕

代码示例

```
value = table.get(key)  # 字典平均O(1)查找，冲突时继续探测
if key not in table: table[key] = default_value()
# 删除后保留探测标记，扩容和冲突会增加开销
```

013 dict 保持的是插入顺序还是键的排序?

答案　Python 3.7 起，字典保持插入顺序是语言保证；CPython 3.6 中已经有这一行为，但当时属于实现特性。字典不会自动按键排序。更新已存在键的值通常仍留在原位置；删除一个键后再插入，它通常会出现在新的末尾位置。需要按键或值排序时，应显式用 sorted 生成排序结果。两个字典做相等比较主要比较键值映射，不比较插入顺序。迭代顺序保证也不代表可以随意在遍历中增删键；迭代期间改变大小仍可能抛异常或造成不可靠逻辑。〔N163〕

代码示例

```
mapping = {"b": 2, "a": 1}
print(list(mapping), sorted(mapping))  # 前者是插入顺序，后者显式排序
```

014 可哈希对象需要满足什么约束，为什么 1、True 和 1.0 会共用字典键?

答案　可哈希对象在作为字典键或集合元素期间，哈希值必须保持稳定；并且若两个对象按 == 相等，它们的哈希值必须相同。反过来哈希相同并不代表相等，查表时仍会比较键。list、dict 等可变类型通常不可哈希；tuple 只有内部每个元素都可哈希时才能作为键。

自定义类实现 __eq__ 后要一起考虑 __hash__。如果哈希依赖的字段在对象已经放进字典后改变，查找可能无法到达原槽位。Python 中 1、True、1.0 按数值比较相等且哈希相同，因此会映射到同一键；要区分业务类型时，构造带类型标签的复合键。〔N161、N163〕

代码示例

```
mapping = {1: "first"}
mapping[True] = "bool"; mapping[1.0] = "float"
assert len(mapping) == 1  # 三个数值键相等且哈希相同
```

015 为什么遍历 dict 时不应直接增删键?

答案　字典迭代器和 keys/items 等视图会观察底层字典。遍历时增删键改变了字典大小，可能抛 RuntimeError，也可能让循环结果不符合预期；这次运行没报错不表示代码可靠。只更新已有键的值与改变键集合不同，但仍应避免把复杂副作用夹在迭代中。

删除满足条件的键时，可以先生成待删键列表，再逐一删除；或者遍历 list(d) 得到键快照，也可构造一个新字典。若另一个线程并发改字典，复制快照本身也需要锁或其他同步协议。单个下标访问安全与一整段遍历更新安全不是一回事。〔N163〕

代码示例

```
remove = [key for key, value in mapping.items() if should_remove(value)]
for key in remove: del mapping[key]  # 先收集，再改变字典大小
```

016 dict、defaultdict、Counter 及缺失键 API 如何选择？

答案　d[key] 在键缺失时会抛出 KeyError，适合缺键就应视为错误的场景。get(key, default) 缺失时返回默认值，但不会修改字典；setdefault 会在缺失时写入默认值并返回。传入 setdefault 的默认表达式会在函数调用前先求值，即使键已存在也可能创建昂贵对象。defaultdict 的工厂通常在方括号访问缺失键时调用，get 并不会触发自动创建。分组累积可用 defaultdict(list)，频次统计可用 Counter；Counter 对缺失计数通常按零处理。选择时应考虑读取是否会意外改变容器：不应让一次查询悄悄新增条目。默认值若昂贵，或需要根据具体键生成，用显式 if key not in d 会更清楚，也更容易加日志和错误处理。〔N163、N165〕

代码示例

```
value = mapping.get(key, default)  # 缺失时读取默认值，不插入
buckets = collections.defaultdict(list); buckets[k].append(item)
counts = collections.Counter(items)  # 缺失计数按0处理
```

017 set 与 frozenset 如何去重，为什么不能承诺元素顺序？

答案　set 与 frozenset 都按哈希与相等关系识别元素，支持去重、成员判断和交并差。set 可原地添加删除，因此自身不可哈希；frozenset 不可变，可在内部元素均可哈希时用作字典键或另一个集合的元素。成员判断平均通常很快，但哈希冲突仍可能增加成本。

集合不保证保持插入顺序，遍历顺序不能当作稳定接口或协议。若需要按输入顺序去重，可用 dict.fromkeys 按插入顺序保留第一次出现的元素，再转成列表；这要求元素可哈希且相等语义符合需求。若结果要排序，应明确调用 sorted。〔N163〕

代码示例

```
items = ["a", "b", "a"]
unique = set(items)  # 集合去重但不保证迭代顺序
ordered_unique = list(dict.fromkeys(items))  # 保留首次出现顺序
```

17.2 Python 函数、迭代与对象协议

018 Python 的函数对象和 lambda 适合怎样的接口？

答案　函数是 Python 对象，可以像其他值一样赋给名字、存到列表、传入另一个函数或由函数返回，因此回调和策略函数很常见。lambda 适合只包含一个表达式的小函数，表达式结果就是返回值；它不会让调用自动并行，也不具有普通 def 没有的魔法执行模型。

map、filter 等简单操作可以组合，但通常返回惰性迭代器；列表推导式常更容易同时看清输入、映射和条件。多步业务逻辑、需要说明文档或调试时，应写有名字的普通函数。避免嵌套多层 lambda；压缩字符数往往牺牲可读性和错误定位能力。〔N161、N162、N164〕

代码示例

```
def add_tax(amount): return amount * 1.1
positive = filter(lambda value: value > 0, values)  # 简单表达式适合lambda
result = [value * 2 for value in positive]
```

019 Python 参数的 /、*、*args 和 **kwargs 如何约束调用？

答案　默认形参一般可按位置或关键字传递。斜线 / 左侧的参数只能按位置传；裸星号 * 或 *args 之后的形参只能按关键字传。定义侧的 *args 收集多余位置实参为元组，**kwargs 收集多余关键字实参为字典；调用侧的 * 与 ** 则把已有列表、元组或映射展开。

同一形参被位置和关键字重复赋值、或调用方传入不支持的关键字，会得到 TypeError。仅限位置参数可以避免内部命名成为用户依赖；仅限关键字参数适合有多个容易混淆的选项。设计公共函数时要明确稳定 API，修改参数规则前检查已有调用，避免悄悄改变用户代码含义。〔N170、N171〕

代码示例

```
def send(destination, /, payload, *, retries=3):
    perform_send(destination, payload, retries)
options = {"retries": 2}; send(*args, **options)  # /和*限定参数调用方式
```

020 可变默认参数为什么会被多次函数调用共享？

答案　默认参数表达式只在 def 执行、创建函数时求值一次，结果保存在函数对象上。若默认值是列表，后续缺省调用会复用同一个列表；函数每次 append 都会改变下一次调用看到的状态。这种共享可能很隐蔽，因为代码看起来像是每次都创建一个空列表。

常见写法是用 None 作为“未传”标记，并在函数内部创建新容器。如果 None 本身是合法输入，可用独立哨兵对象区分缺省。可变默认值可以被有意用作缓存，但必须清楚地记录生命周期、并发保护和清理方式；否则应避免借函数默认值隐藏长期状态。〔N170〕

代码示例

```
def collect(value, items=None):
    if items is None: items = []  # 每次缺省调用都创建新列表
    items.append(value); return items
```

021 LEGB、global 和 nonlocal 如何影响名字绑定?

答案　LEGB 是函数名字查找的常见概括：先查局部 Local，再查外层函数 Enclosing，接着模块 Global，最后是内置 Built-in。若函数体内给名字赋值，该名字通常会在整个函数中被认作局部变量，所以先读后赋可能触发 UnboundLocalError，而不会读取模块全局值。

global 让当前作用域的赋值绑定到模块名字；nonlocal 指向最近一层已存在的外部函数变量。它们改变的是名字绑定，不是修改列表内容等可变对象所必需的声明。依赖关系若能通过参数和返回值表达，通常比隐藏共享状态更好测试。类体、推导式和注解作用域还有额外规则。〔N172〕

代码示例

```
def counter():
    value = 0
    def increment():
        nonlocal value  # 修改最近一层外部函数的绑定
        value += 1; return value
    return increment
```

022 闭包如何保存外层变量，循环 lambda 的晚绑定问题怎么解决?

答案　闭包会保存访问外层函数变量所需的绑定单元，通常不是把变量当前值自动复制成快照。循环里创建多个 lambda 并引用同一个循环变量时，等到调用时变量已经指向最后一个值，于是多个函数都读到相同结果，这就是常见的晚绑定问题。

可以用工厂函数为每轮创建独立局部变量，也可用默认参数在定义 lambda 时固定当时的对象引用。默认参数只固定引用；如果对象后来原地修改，闭包仍会看到修改。若要重绑外层名字可使用 nonlocal。闭包也会让捕获对象继续存活，应避免无意保留大型对象或资源。〔N172、N170〕

代码示例

```
functions = [lambda i=i: i for i in range(3)]  # 默认参数在创建时捕获当前值
assert [f() for f in functions] == [0, 1, 2]
```

023 装饰器怎样替换函数，带参装饰器和异步包装有哪些边界?

答案　@装饰器定义后，Python 大致执行 原函数名 = 装饰器(原函数)，随后把返回值绑定回原名字。多个装饰器从上到下应用、运行时从最外层包装进入。带参数装饰器通常先通过工厂接收配置，再返回实际装饰函数。functools.wraps 帮助保留名称、文档和 __wrapped__ 信息，但不保证包装对象的所有行为都与原函数完全相同。

包装 async def 时，普通同步包装器如果只调用原函数，会得到协程对象却没有等待其实际运行；计时、异常处理和清理可能因此包错范围。异步包装器通常也要声明为 async 并 await 原协程。还要决定是否保留取消语义、签名和异常链。〔N171、N173、N174〕

代码示例

```
import functools
@functools.wraps(function)
def timed(*args, **kwargs):
    start = time.perf_counter(); result = function(*args, **kwargs)  # 包装器保留元信息
    log(time.perf_counter() - start); return result
```

024 可迭代对象、迭代器与 for 循环的协议是什么?

答案　可迭代对象实现 iter 协议，iter(obj) 会取得一个迭代器；迭代器实现 next 协议，每次 next 返回下一个值，耗尽时抛 StopIteration。for 循环会自动执行这套协议并正常结束。容器如列表通常能反复 iter，每次返回新的迭代器；生成器本身通常是一次性迭代器，保存当前消费位置。

需要多次遍历时，接口应接收可重复遍历的容器、接收一个创建迭代器的工厂，或显式缓存输入。若拿到的是已消费一部分的迭代器，再次调用 iter 通常不会重置它。不要假设所有 iterable 都能回退或重复读取；处理文件流、生成器和网络输入时更要明确消费语义。〔N161、N164〕

代码示例

```
items = [1, 2, 3]
first = iter(items); print(next(first))  # 容器可重新创建迭代器
stream = (value for value in items); print(next(stream), next(stream))  # 生成器保留位置
```

025 yield、return 和 yield from 怎样控制生成器执行?

答案　含 yield 的函数被调用时先返回生成器对象，函数体通常等到消费时才执行。每次 yield 交出一个值并暂停，局部状态会保留；再次请求下一个值时从暂停处继续。return 结束生成器，返回值成为 StopIteration 的 value，不会作为普通生成元素继续产出。

yield from 会把迭代和更多生成器协议委托给子迭代器，包括发送值、异常、关闭及获取其返回值，并非单纯缩写 for 循环。生成器可流式产生数据，减少一次性结果内存，但会让局部引用对象继续存活。提前停止消费时应明确关闭生成器，并用 with 或 finally 管理文件等资源。〔N162、N161〕

代码示例

```
def batches(source):
    for batch in source: yield batch  # yield暂停并保留局部状态
    return "done"  # return结束生成器，不作为普通产出值
```

026 实例方法、classmethod 和 staticmethod 怎样绑定参数?

答案　类里的普通函数经实例读取时遵循描述符绑定：得到的绑定方法会在调用时自动把该实例作为第一个参数。参数名 self 只是约定，不是 Python 关键字。classmethod 把类作为第一个参数传入，适合创建对象的工厂方法；通过派生类调用时能收到实际派生类。

staticmethod 不自动传 self 或 cls，适合逻辑上归属于类命名空间、但不依赖实例和类状态的辅助函数。方法的绑定行为由函数描述符及装饰器决定，而不是参数名字决定。需要访问实例数据时应使用实例方法；希望工厂尊重继承层次时通常用 classmethod，而不是把基类名称写死。〔N161、N175、N176〕

代码示例

```
class Base:
    @classmethod
    def from_text(cls, text): return cls(parse(text))  # cls支持派生类工厂
    @staticmethod
    def valid(value): return value >= 0  # 不自动绑定实例或类
```

027 __new__ 和 __init__ 在对象创建中分别负责什么?

答案　调用类时通常先执行 __new__ 创建并返回对象，再对返回的类实例执行 __init__ 初始化状态。__new__ 是静态创建入口，通常需调用 super().__new__(cls)；它在创建不可变类型子类时尤其有用。__init__ 负责给已创建对象设置状态，必须返回 None。

若 __new__ 返回的对象不是该类的实例，通常不会走该类的 __init__ 初始化流程。类调用还可被元类定制，因此这不是 C++“分配内存然后构造”规则的直接翻译。应避免在 __init__ 中执行不可回滚的大量外部副作用；对象构造失败时，已取得的资源要由 RAII 式上下文管理清理。〔N161〕

代码示例

```
class Item:
    def __new__(cls, *args): return super().__new__(cls)  # 创建对象
    def __init__(self, value): self.value = value  # 初始化已创建对象
```

028 类属性和实例属性如何查找，可变类属性为什么会共享?

答案　类属性定义在类对象上；实例读取属性时，除数据描述符等更高优先级规则外，找不到实例属性就可能回退到类属性。若类属性保存一个列表，所有实例可能读到同一个列表；对它 append 是修改共享对象。执行 self.items = [] 通常是在当前实例上建立新的属性绑定。

需要每个实例独立状态时，在 __init__ 中创建可变对象；dataclass 字段可用 default_factory 为每个实例产生新值。描述符和 __slots__ 会影响通常的属性查找与存储方式，因此不能简化为“实例字典总是优先”。类属性适合常量或有意共享的状态，使用可变类属性要说明共享意图。〔N175、N176、N177〕

代码示例

```
class Record:
    def __init__(self): self.items = []  # 每个实例拥有独立列表
first, second = Record(), Record(); first.items.append(1)
assert second.items == []  # 避免共享可变类属性
```

029 Python 的鸭子类型如何表达多态，何时需要 Protocol 或抽象基类?

答案　鸭子类型按行为而非继承关系使用对象：函数需要 read 或迭代能力时，只要传入对象按协议提供该操作即可。它仍依赖明确调用约定，方法缺失或语义不符仍会在运行时失败。静态类型检查需要描述结构化接口时可使用 typing.Protocol；抽象基类 abc.ABC 可规定抽象方法或提供运行时分类机制。

运行时 Protocol 检查通常只能验证有限的属性存在性，不能完整保证参数签名、返回值含义、异常和资源所有权。接口文档应说明这些行为约束。选择结构化检查还是名义继承取决于协作需求：开放的第三方实现适合 Protocol，需强制共享实现或注册关系时可用 ABC。〔N161、N178、N175〕

代码示例

```
def read_all(source): return source.read()  # 依赖行为，不要求固定父类
class FileSource:
    def read(self): return "file data"
```

030 Python 多继承的 MRO 与 super 为什么不是“直接调用父类”?

答案　Python 多继承的方法解析顺序通常使用 C3 线性化：它保留类声明中的局部顺序和单调性，使多条继承路径合并成一致的查找顺序；若无法构造这样的顺序，类定义会失败。查看 Class.__mro__ 可直接确认实际顺序。

super() 不是“直接调用源码里某个父类”，而是从当前类在实际实例 MRO 中的位置之后继续查找。协作式多继承中的各层通常要使用兼容参数并继续调用 super，才能让每个实现恰好参与一次。忽略这套协议可能跳过初始化或重复处理。〔N175、N161〕

代码示例

```
class Root:
    def __init__(self, **kwargs): super().__init__(**kwargs)  # 沿MRO协作初始化
class Left(Root): pass
class Right(Root): pass
class Join(Left, Right): pass
print(Join.__mro__)  # 查看实际方法解析顺序
```

031 描述符、property 与属性访问钩子如何协作?

答案　实现 __get__、__set__ 或 __delete__ 的对象遵循描述符协议；普通函数的绑定和 property 都建立在这套机制上。常规实例属性读取时，数据描述符优先于实例字典；实例字典通常又优先于非数据描述符，再按类的 MRO 查找。property 可以在保持 obj.name 形式的同时校验赋值。

__getattribute__ 参与几乎所有常规属性访问，覆盖后应委托 object 或父类实现，否则容易递归。__getattr__ 是普通查找抛 AttributeError 后的后备路径。访问规则还有槽位、元类等细节，故属性钩子要谨慎实现并测试缺失、异常与描述符行为。〔N176、N161〕

代码示例

```
class Temperature:
    def __init__(self): self._value = 0
    @property
    def value(self): return self._value
    @value.setter
    def value(self, new_value):
        if new_value < -273: raise ValueError("below absolute zero")
        self._value = new_value  # property在访问时执行校验
```

032 __repr__、__str__ 等特殊方法怎样让对象支持内置操作?

答案　__repr__ 通常面向开发和调试，尽量准确地描述对象；__str__ 面向用户阅读。未定义 __str__ 时通常使用 repr 相关表示。__len__、__iter__、__contains__ 和算术运算等特殊方法，让自定义对象参与 Python 内置操作。实现时要遵守约定的返回类型和语义。

解释器隐式调用 len(obj) 等操作时，通常在对象类型上查找特殊方法，不是简单读取实例同名属性，所以给单个实例添加 __len__ 不一定改变 len 的行为。相等比较与哈希也必须一致。repr 适合诊断，不应当作稳定序列化格式；跨版本存储应使用明确的序列化协议。〔N161〕

代码示例

```
class Queue:
    def __init__(self, items): self._items = items
    def __len__(self): return len(self._items)  # 支持len()
    def __repr__(self): return f"Queue({self._items!r})"  # 用于诊断
```

033 __slots__ 能节省哪些内存，为什么它不是完整的封装工具?

答案　__slots__ 通常让实例通过固定槽位保存指定属性，避免每个对象都拥有常规 __dict__；当实例数量很大且属性集合固定时，可能减少内存占用，也限制动态添加未声明字段。节省多少取决于继承结构、实例字典、弱引用和 Python 实现，不能仅凭写了 slots 就断定更省。

若需要弱引用，通常要声明 __weakref__ 槽位；基类或派生类引入实例字典时，动态属性能力和内存收益会改变，重复槽名也可能造成问题。slots 不是私有访问控制，外部仍可读取公开槽位。应在真实对象规模下比较内存和接口需求，再决定是否使用。〔N161〕

代码示例

```
class Point:
    __slots__ = ("x", "y")  # 固定字段可减少每实例的动态字典开销
    def __init__(self, x, y): self.x, self.y = x, y
```

034 下划线命名与双下划线名字改写能提供真正的私有成员吗?

答案　单前导下划线通常是“内部实现”的约定，也会影响某些 from 模块 import * 的导入行为，但不会阻止调用者访问。类体中双前导下划线名字通常会被改写为带类名的属性，主要用于减少派生类意外撞名；知道改写规则的人仍然能访问它，因此不是安全边界。

首尾双下划线通常保留给 Python 特殊协议方法，不要随意创造类似名字。API 是否公开应靠文档、模块边界和稳定性承诺表达，而不是期待名字隐藏阻止访问。若需要真正隔离敏感数据，不能依赖 Python 属性命名，而应使用适当的进程或安全边界。〔N175、N161〕

代码示例

```
class Base:
    def __init__(self): self.__state = "base data"  # 名字改写减少继承撞名
obj = Base(); print(obj._Base__state)  # 不是访问控制或安全边界
```

17.3 Python 异常、模块与工程接口

035 try、except、else、finally 如何影响异常和返回值?

答案　try 中没有异常离开时执行 else；匹配到异常时执行 except；无论正常还是异常退出，finally 都用于清理。return 会先确定待返回值，再运行 finally；如果 finally 自己 return 或抛出异常，会覆盖先前待完成的返回或异常，容易把故障吞掉。

因此 finally 应尽量只释放资源，文件和锁通常更适合交给 with 管理。try 块应只包围确实要捕获的操作，避免把后续业务代码的 bug 一并误认成预期异常。进程被强制终止、解释器崩溃或断电时，不能依赖 finally 一定执行；重要持久操作要有事务或恢复机制。〔N171〕

代码示例

```
try:
    result = perform()
except RecoverableError as error:
    handle(error)
else:
    consume(result)  # 仅try正常完成时执行
finally:
    release_resource()  # 不在finally里return，以免覆盖异常
```

036 Python 应如何选择异常基类、保留回溯并建立异常链?

答案　普通业务异常通常继承 Exception；BaseException 还包括 KeyboardInterrupt、SystemExit 等控制流程异常，宽泛捕获它们可能阻止用户中断或程序退出。若当前层无需恢复或转换错误，就不要捕获后原样再记录一遍，让异常向上传递可保留清晰的故障边界。

except 块中的裸 raise 会重新抛出当前异常并保留原回溯；raise e 会在当前抛出位置增加回溯信息。需要把底层错误转换成接口异常时，可用 raise NewError(...) from e 建立原因链。自定义错误应包含调用者需要判断的上下文，但避免把密码、令牌等敏感数据写入异常消息。〔N179、N171〕

代码示例

```
try:
    read_config()
except OSError as cause:
    raise ConfigError("cannot read config") from cause  # 保留异常链
```

037 with 和 contextmanager 怎样确保资源清理，什么时候会吞掉异常?

答案　with 语句先调用上下文管理器的 __enter__，成功后执行代码块，退出时调用 __exit__。若代码块抛异常，异常信息会交给 __exit__；它返回真值就会抑制异常。若 __enter__ 自身失败，通常不会调用该管理器的 __exit__，所以获取资源失败时也要自行保持异常安全。

contextlib.contextmanager 允许用生成器表示上下文：yield 前获取资源，finally 中释放；若捕获异常后没有重新抛出，异常可能被视为已处理。文件、锁和事务都应明确成功提交与失败回滚边界，不能让垃圾回收决定释放时机。组合多个资源时，退出顺序与获取顺序相反。〔N180、N171〕

代码示例

```
from contextlib import contextmanager
@contextmanager
def opened(path):
    file = open(path, encoding="utf-8")
    try: yield file
    finally: file.close()  # 即使块内异常也执行清理
```

038 模块、包与 import 缓存如何影响循环导入和程序入口?

答案　模块提供独立名字空间，包负责组织模块；普通包通常包含 __init__.py，命名空间包遵循不同的查找规则。首次导入通常先把模块对象放入 sys.modules，再执行顶层代码，因此循环导入可能看到尚未初始化完成的对象。与其盲目重试，不如抽出共同依赖或延后确有必要的导入。

import m 绑定模块对象，from m import x 把当时的 x 对象绑定到本地名字；之后 m.x 被重新赋值不会自动更新本地绑定。脚本直接运行时 __name__ 是 __main__，用入口判断隔离启动副作用，可让模块导入与进程池加载更安全。顶层代码应避免执行昂贵或不可逆操作。〔N181、N182〕

代码示例

```
def main(): start_application()
if __name__ == "__main__": main()  # 导入时不自动执行应用入口
```

039 虚拟环境解决什么隔离问题，怎样让依赖安装可复现？

答案　venv 为项目提供独立的包安装位置和解释器入口，可减少两个项目依赖版本互相干扰。它不是操作系统级容器，不会自动隔离系统库、环境变量或文件系统，也不会固定 Python 解释器版本。用 python -m pip 安装可以明确 pip 对应当前解释器，避免把包装进另一个环境。

要可复现，应记录支持的 Python 版本与依赖约束，必要时提交锁定文件和制品校验信息；环境目录本身通常可重建，不提交版本控制。含本地扩展时还要记录操作系统、架构、编译器和外部动态库。开发机安装成功只是一次验证，不保证其他平台得到相同产物。〔N183〕

代码示例

```
python -m venv .venv
.venv/bin/python -m pip install -r requirements.txt  # 隔离安装锁定依赖
# 在干净CI中按相同Python版本重建环境
```

040 类型注解、TypedDict、dataclass 和校验模型分别提供什么保证？

答案　Python 类型注解主要供静态检查器、IDE 和文档工具使用，解释器默认不会逐次强制验证参数或字段类型。Any 会放宽检查；T | None 表示允许缺失类型，但不等于参数有默认值。TypedDict 描述普通字典的键结构，运行时仍是字典，也不自动校验外部数据。

dataclass 可生成初始化、比较等方法，但默认不验证传入字段类型。Pydantic 等模型可在运行时校验和转换输入，具体行为受严格模式、额外字段策略等配置影响。来自 API、配置文件或用户的值必须在边界做实际校验；只写注解不能当作信任输入的依据。〔N178、N177、N184〕

代码示例

```
class Request(TypedDict):
    username: str
    count: int
raw = json.loads(payload)  # 注解不验证外部JSON；边界处仍要运行时校验
request = RequestModel.model_validate(raw)
```

041 读取大文件怎样避免一次性占满内存，文本流还要注意什么？

答案　不指定大小的 read 通常读取剩余全部内容，可能占用大量内存；readline 读一行，readlines 则通常建立行列表。直接遍历文件对象可逐行处理，二进制大文件可按固定字节块读取。不过若处理代码又把所有块累积到列表，整体内存仍会随文件增长。

用 with 关闭文件，并在文本模式明确选择编码和解码错误策略；文本读取可能进行换行转换。需要协议中的原始字节偏移、固定长度块或精确分段时用二进制模式，字符数不能直接当磁盘字节位置。还要处理异常长单行、截断文件和流式解析状态。〔N168、N164〕

代码示例

```
with open(path, "r", encoding="utf-8") as stream:
    for line in stream: process(line)  # 逐行处理，额外内存有界
with open(path, "rb") as stream:
    while chunk := stream.read(64 * 1024): parser.feed(chunk)
```

042 JSON 的 dump/load 与 dumps/loads 有什么区别和限制？

答案　json.dump 和 json.load 分别向文件类对象写入或从中读取 JSON；dumps 和 loads 则把 JSON 文本变成字符串或从字符串解析。它们转换的是受 JSON 数据模型约束的数据，并非任意 Python 对象快照。对象键会变成字符串，tuple、set、自定义类、循环引用和精确数值等都要定义明确编码策略。

不要用 eval 解析不可信输入。连续调用多个 dump 也不会自动形成一个合法的单文档 JSON 流，因为 JSON 文档之间没有天然边界；流式记录可约定 NDJSON 等格式。对外数据还应限制体积与嵌套深度、校验字段类型，并约定日期、浮点特殊值和兼容策略。〔N185〕

代码示例

```
with open(path, "w", encoding="utf-8") as stream: json.dump(value, stream)
text = json.dumps(value); parsed = json.loads(text)
# 校验解析后的结构和大小；不要用eval解析不可信文本
```

17.4 Python 并发与 asyncio

043 GIL 如何影响 Python 并行，为什么仍然需要业务锁？

答案　在传统启用 GIL 的 CPython 中，同一时刻通常只有一个线程执行 Python 字节码。GIL 是 CPython 实现层为了保护解释器内部对象状态而采用的同步机制，不是 Python 语言对所有实现的要求。阻塞 I/O 和部分原生扩展会释放 GIL，因此线程仍适合很多 I/O 场景；纯 Python CPU 计算可评估多进程或原生扩展。

CPython 3.13 起提供可选的自由线程构建；运行时和扩展兼容情况要按环境确认，部分扩展可能影响 GIL 是否启用。GIL 不是业务锁：多个操作组成的检查与更新、跨对象不变量仍要显式同步；自由线程执行时更需要清楚的数据保护。〔N186、N187、N188〕

代码示例

```
with ThreadPoolExecutor() as pool:
    results = list(pool.map(fetch, urls))  # I/O等待可重叠
with lock:
    if balance >= amount: balance -= amount  # GIL不替代业务锁
```

044 Python 的 Lock 和 RLock 怎样选择，重入锁能消除死锁吗?

答案　Lock 是普通互斥锁，适合保护简单共享临界区。RLock 会记录持有它的线程和递归获取次数，允许同一线程在已经持锁时再次获取；每次获取都必须匹配一次释放。若函数调用链可能重入同一资源，RLock 可避免线程把自己阻塞住。通常用 with 自动释放锁。

重入锁只解决同线程重复获取的问题，不会消除不同线程以相反顺序获取多把锁形成的环，也不能修复长时间持锁或锁内慢 I/O。协程在同一事件循环线程中运行时，使用线程 Lock 可能阻塞整个循环，应使用 asyncio 的异步同步原语。先梳理锁顺序和临界区范围，再决定是否需要 RLock。〔N187、N189〕

代码示例

```
with lock:
    update_shared_state()  # with异常退出也会释放锁
with reentrant_lock:
    nested_call()  # 同线程可再次获取，但不能消除所有死锁
```

045 queue.Queue 如何协调线程任务，task_done 和 join 表示什么?

答案　queue.Queue 用锁和条件变量协调线程间传递任务；设置 maxsize 后，生产者在队列满时可以等待，形成一定背压。消费者 get 取出任务后，最终应为每个任务调用一次 task_done。Queue.join 等待的是所有入队任务的未完成计数归零，不是等待消费者线程退出。

若任务处理抛异常而没有在 finally 中调用 task_done，join 可能一直等待；是否重试失败任务是另一层业务策略。empty 和 qsize 只是某个时刻的观察，另一线程可能随后改变队列状态。停止哨兵、显式关闭或版本支持的 shutdown 需与生产者和消费者退出顺序统一设计。〔N190〕

代码示例

```
item = queue.get()
try: process(item)
finally: queue.task_done()  # 每个get对应一次task_done
queue.join()  # 等待未完成计数归零，不等于线程退出
```

046 Python 线程池、进程池和 Future 的结果传播有什么区别?

答案　ThreadPoolExecutor 在同一进程的线程里执行任务，适合共享对象或等待 I/O；ProcessPoolExecutor 在独立进程中运行，更适合可并行的 CPU 工作，但进程启动、数据传输和序列化都有额外成本。Future 表示将来的结果，result 会等待并在调用处重新抛出任务异常；cancel 不能强制杀死已经开始执行的任务。

进程间可用 Queue/Pipe 发消息；Value、Array 或 shared_memory 能共享数据但仍需同步；Manager 的代理接口方便，却要承担进程通信成本。任务和参数通常要可 pickle，优先使用模块顶层函数；spawn/forkserver 下入口应受 __main__ 判断保护。还要明确关闭池、等待任务和超时策略。〔N191、N188〕

代码示例

```
if __name__ == "__main__":
    with ProcessPoolExecutor() as pool:
        future = pool.submit(cpu_bound_function, payload)
        result = future.result(timeout=5)  # 子进程异常在result处传播
```

047 协程对象、Task、Future 与事件循环怎样完成异步调度?

答案　调用 async def 函数通常只创建协程对象，并不立即执行函数体。await 会等待一个可等待对象；Task 把协程交给事件循环调度，Future 表示将来可完成的结果。事件循环通常在一个线程中协作运行任务，任务只有在等待未完成操作时才让出执行权，所以每个 await 并不必然切换到别的任务。

连续 await 两个协程通常是先完成一个再开始下一个。若要在等待期间重叠运行，需要创建 Task 并持有、等待结果。异步可以隐藏 I/O 等待，不会自动把 CPU 计算分配到多个核心。必须处理任务异常、取消、资源释放，并确保所有创建的任务最终有明确所有者。〔N174〕

代码示例

```
async def main():
    first = asyncio.create_task(fetch_first())
    second = asyncio.create_task(fetch_second())
    return merge(await asyncio.gather(first, second))  # 等待时交还事件循环
```

048 asyncio.gather 与 TaskGroup 遇到子任务异常时怎样处理?

答案　gather 按输入顺序返回各任务结果。默认模式遇到子任务异常会把首个异常传播给等待者，但其他子任务通常不会仅因此自动全部取消；return_exceptions=True 会把异常作为结果元素交回，调用方必须逐项检查。调用者取消 gather 与子任务失败是不同的取消路径。

TaskGroup 从 Python 3.11 提供结构化并发：退出上下文前会等待组内任务；一个任务发生非取消异常时，通常会取消其他任务，完成清理后再按异常组规则报告。取消并不会回滚已经执行的外部副作用，因此写数据库或调用远程服务仍要设计幂等与补偿。〔N174〕

代码示例

```
async with asyncio.TaskGroup() as group:
    group.create_task(fetch_first())
    group.create_task(fetch_second())
# 离开上下文时等待子任务；一项失败会取消其余任务并汇总异常
```

049 异步函数里怎样处理同步 I/O 和 CPU 密集工作?

答案　事件循环线程中若运行 time.sleep、同步网络请求或长时间 CPU 计算，循环无法调度其他任务，它们会一起停顿。优先使用原生异步 I/O 库；只能调用阻塞 I/O 时，可用 asyncio.to_thread 或 run_in_executor 把调用移到线程池。传统启用 GIL 的环境中，纯 Python CPU 计算通常评估进程池或能释放 GIL 的扩展。

把操作移出循环并不消除队列和取消边界：取消 await 通常不会停止已在线程中执行的同步函数。网络或数据库调用仍应设置库自身的超时，避免后台线程一直阻塞。CPU 作业还要限制并发和传输开销，不能因为用了 executor 就认定可以无限提交。〔N174、N191〕

代码示例

```
async def fetch_page(url):
    page = await asyncio.to_thread(blocking_http_get, url)  # 不阻塞事件循环
    return parse(page)
```

050 asyncio 的超时和任务取消为什么是协作式的?

答案　Task.cancel 是向协程请求取消；在后续适合的暂停点，协程会收到 CancelledError。它不是立即杀死任意 Python 指令。被取消代码仍应在 finally 或异步上下文管理器中释放资源，通常清理完成后继续传播取消；随意吞掉异常会干扰超时和 TaskGroup 等上层逻辑。

asyncio.timeout 与 wait_for 可建立不同形式的超时边界，但超时发生后等待清理也会占时间，所以实际总耗时可能超过标称时间。网络、数据库和外部工具调用还应设置底层操作超时。若远程请求已提交而本地任务被取消，要能查询执行结果，或用幂等键和补偿逻辑避免重复产生副作用。〔N174〕

代码示例

```
async def work():
    resource = await open_resource()
    try: await perform(resource)
    finally: await resource.close()  # 取消是协作式，清理可能继续耗时
```

051 Semaphore 和有界队列为什么比一次创建海量 Task 更适合限并发?

答案　Semaphore 限制同时进入某项操作的任务数，例如同时访问连接池的请求数量；有界 asyncio.Queue 则限制等待任务的积压，队列满时 put 会等待，形成背压。只在任务内部加信号量仍可能先创建数十万 Task，占用大量内存，所以大输入应由少量工作协程从有界队列取活。

使用 async with 管理信号量令牌，消费者无论成功、失败或取消，都要按协议调用 task_done。生产者结束、工作协程异常和整体取消时也要让其他等待者能退出，否则队列 join 可能永久阻塞。限制并发数量和限制积压量是两个不同目标，常需同时设置。〔N189、N174〕

代码示例

```
queue = asyncio.Queue(maxsize=100)  # 有界队列产生背压
async def worker():
    while True:
        job = await queue.get()
        try: await process(job)
        finally: queue.task_done()
```

052 contextvars 如何隔离异步请求上下文，它和线程局部存储有什么不同?

答案　threading.local 按线程保存状态，但多个 asyncio Task 往往在同一个线程轮流运行，因此线程局部变量不能区分不同请求。ContextVar 把值放在当前执行上下文中，asyncio 会为任务维护上下文，适合请求 ID 或追踪标记。新建任务通常继承当前上下文绑定的副本，但这不是任务间传消息的通道。

set 会返回 token，使用 finally 调用 reset 可恢复先前值，避免后续代码读到残留请求状态。复制上下文通常复制名字到对象的绑定，不会深复制绑定对象；若其中放可变字典，任务仍可能共享并发修改。上下文适合不可变标记，不宜暗藏复杂的共享业务状态。〔N192、N174〕

代码示例

```
request_id = ContextVar("request_id")
async def handle(value):
    token = request_id.set(value)
```

```
    try: await call_downstream()
    finally: request_id.reset(token)  # 按异步任务恢复上下文绑定
```

17.5 CPython 内存与执行机制

053 CPython 对象为什么比其业务数值占用更多内存?

答案　CPython 对象通常需要记录类型、引用管理状态和对象内容，容器还要保存元素引用、容量或索引结构。因此一个整数对象占用的内存远多于其逻辑数值所需的机器位数。对象对齐、小对象分配器和内存池也会影响实际分配及进程常驻内存。

sys.getsizeof 通常给出对象的直接大小，不会自动递归统计它引用的所有对象；若把多个容器大小相加，又可能把同一个共享对象重复计算。对象布局随 Python 实现、版本和构建方式变化。需要查内存时结合 tracemalloc 等分配快照和进程 RSS，不要把某个 CPython 版本测到的固定字节数当作所有对象的常数。〔N193、N161〕

代码示例

```
shallow = sys.getsizeof(values)  # 不包含引用对象图的全部大小
seen = set(); total = measure_reachable_objects(values, seen)
rss = read_process_rss()  # RSS也包含解释器、扩展和分配器保留页
```

054 引用计数、循环垃圾回收和 del 分别改变什么?

答案　传统 CPython 主要用引用计数跟踪对象的强引用数量；引用降到零时通常可以回收。但这只是 CPython 常见实现方式，不能由此断定所有 Python 实现和所有构建都会在某个精确时刻销毁对象。两个容器对象互相引用会形成环，引用计数本身无法发现外部已经没有入口，因此还需要循环垃圾回收器。

del name 通常先移除当前作用域中的名字绑定，不是强制销毁对象；其他引用、缓存、任务或循环仍可延长寿命。GC 的分代策略、阈值和自由线程构建细节会变化，避免依赖某个固定代数。文件、锁和连接等外部资源应使用 with 或专门所有者明确管理，不能等垃圾回收器择时处理。〔N194、N161、N186〕

代码示例

```
alias = values
del values  # 只删除当前名字，alias仍引用原对象
left.append(right); right.append(left)  # 不可达循环由循环GC处理
```

055 Python 对象删除后 RSS 为什么不降，怎样定位真实内存增长?

答案　Python 对象被释放后，分配器可能把内存留在自己的空闲池中，供后续对象重用，而不是立刻把页归还操作系统。因此对象数量下降时 RSS 仍保持较高，不能仅凭任务管理器判断泄漏。碎片、缓存或本地扩展还会影响观测。要比较真实增长，应在稳定负载下记录分配和对象持有变化。

tracemalloc 可在可疑阶段前后拍快照，比较 Python 分配位置；它不覆盖所有 GPU、C 扩展或外部库分配。全局容器、无界缓存、闭包和未完成任务若仍持有引用，GC 也不会回收对象。先追踪保留链和生命周期，再评估是否需要 collect；强行回收不能解决仍被引用的对象。〔N195、N193、N194〕

代码示例

```
tracemalloc.start(); before = tracemalloc.take_snapshot()
run_fixed_workload(); after = tracemalloc.take_snapshot()
for stat in after.compare_to(before, "lineno")[:10]: print(stat)
# 再对比RSS；C扩展分配可能不出现在Python追踪中
```

056 CPython 的字节码、pyc 缓存与 dis 分析分别解决什么问题?

答案　CPython 通常将源代码解析成代码对象和字节码，再由解释器执行；导入模块时可以把编译结果放进 pyc 缓存，后续根据版本和源文件校验信息判断能否复用。pyc 主要减少再次导入时的解析和编译开销，不是独立于 Python 版本的稳定机器码，也不代表函数运行会自动更快。

dis 可以显示字节码，帮助理解取值、调用和控制流，但具体操作码会随 Python 版本和专门化优化变化。某段字节码也不构成线程原子性或语言语义保证。用 dis 探索实现有帮助，性能结论仍要用目标 Python、实际输入与剖析器测量。〔N182、N196〕

代码示例

```
import dis
def add(left, right): return left + right
dis.dis(add)  # 当前解释器字节码；pyc是编译缓存，不是稳定机器码
```

057 定位 Python 慢请求时如何区分算法、CPU、I/O 和等待?

答案　先用固定输入和相同负载记录端到端耗时、吞吐及尾延迟，再拆分 Python 计算、外部请求、锁等待和队列排队。cProfile 可查看函数调用次数与累计时间；timeit 适合隔离小片段比较。异步服务还要检查事件循环是否被同步 I/O 或长时间计算阻塞。

总耗时高但 CPU 利用率低，优先检查 I/O、下游延迟、连接池和锁等待；CPU 持续繁忙时再看算法复杂度、重复工作和数据布局。分析器自身会增加开销，原生扩展的细节可能需要系统采样工具。每次优化后用相同工作负载复测，否则难判断实际收益。〔N197、N174〕

代码示例

```
start = time.perf_counter(); serve_representative_request()
profile = cProfile.Profile(); profile.enable(); serve_representative_request(); profile.disable()
profile.print_stats(sort="cumtime")  # 同时观察端到端延迟和调用画像
```

058 functools.lru_cache 为什么可能造成内存增长和过期结果?

答案　lru_cache 按函数参数缓存返回结果，参数需要可哈希；设置有限 maxsize 后会按最近使用策略逐出旧项。缓存会持有键和结果，若装饰实例方法还可能让 self 随条目长期存活，因此长期服务要观察容量和对象寿命。它更适合同一输入稳定地产生同一输出的计算。

依赖时间、配置或外部状态的结果要设计失效策略，否则缓存会过期。线程安全表示内部结构保持一致，不保证同一个缺失键在并发时只计算一次。可用 cache_info 观察命中、未命中和容量，需要时 cache_clear。关键字参数组合、可变返回值被调用者修改等，也可能造成浪费或共享副作用。〔N173〕

代码示例

```
@functools.lru_cache(maxsize=1024)
def compute(config_version, key): return expensive_lookup(config_version, key)
info = compute.cache_info(); compute.cache_clear()  # 失效旧版本并释放缓存引用
```

第 18 章 工具命令与容器

18.1 Linux 命令与现场排查

001 ps 的进程快照和 top 的采样信息应怎样解读？

答案　ps 输出一次进程快照，适合脚本筛选和指定字段；top 持续刷新采样，便于观察变化。ps 的 -e、-p、-L、-o 分别可查看全部进程、指定 PID、线程和自定义列；BSD 风格 aux 与 UNIX 风格 -ef 参数规则不同，不要混为一套语法。top 的 CPU 值按采样方式展示，多线程进程在某些模式可超过 100%；不能和 ps 生命周期平均值直接比较。VSZ/VIRT 是虚拟地址空间，RSS/RES 是驻留内存，两者差值不代表泄漏。容器能用的 CPU 还受 quota 和亲和性限制，top 显示多少 CPU 行并不等于配额。

排查单个程序时可先用 ps 保存 PID、线程和内存字段，再用 top 观察几轮变化；如果关注线程，可切换线程显示。一次快照适合回答“现在是什么状态”，连续采样适合回答“状态怎样变化”，两种视角结合更容易避免误判。〔N198、N199〕

代码示例

```
ps -p 1234 -L -o pid,tid,stat,pcpu,rss,vsz,comm
top -H -p 1234  # 多次采样并与容器CPU quota/亲和性对照
```

002 按进程名结束程序时怎样避免匹配错误并保留清理机会？

答案　结束程序前先用 pgrep -a 或 ps 核对 PID、命令行和所属用户。对确认无误的目标先发 SIGTERM，给程序机会刷盘、关闭连接并正常退出。pkill 的模式范围可能比预期宽，不能把宽泛 grep 匹配结果直接批量传给 kill；长期脚本还要考虑 PID 可能被其他进程复用。

只有明确确认后果且程序不响应时，才考虑 SIGKILL。它不能被程序捕获，也无法保证业务清理；处于不可中断内核等待的进程不一定立即消失。僵尸进程已经退出，只等父进程回收，继续发信号无效。服务通常优先由 systemd 或容器编排器按服务身份停止。〔N200、N198〕

代码示例

```
pgrep -a -x worker  # 先核对完整命令和身份
kill -TERM 1234     # 给进程正常清理机会；确认无响应再评估KILL
```

003 如何用 ss 和进程信息定位监听端口与连接异常？

答案　ss 可查看套接字状态、地址、端口和队列，-lntp 常用于查看 TCP 监听者，-u 查看 UDP，-a 查看更广范围，-n 避免名称解析；查看进程关联可能需要权限。过滤本机目标端口能缩小范围，但还应区分绑定在回环、指定网卡或通配地址，以及 IPv4/IPv6 和容器网络命名空间。

LISTEN 只说明本机套接字监听，不能证明外部客户端可达。队列列的单位随状态变化：LISTEN 时 Recv-Q 通常是已完成、等待 accept 的连接数，Send-Q 常为 backlog 上限；ESTABLISHED 时通常反映字节队列。结合内核计数增量和应用 accept 速度判断，不能因出现 RST 就断言队列满。〔N201、N69、N70〕

代码示例

```
ss -lntp 'sport = :8080'  # 检查监听地址、进程和端口
ss -ant state established  # 结合Recv-Q/Send-Q解释连接队列
```

004 ping、nc 和 curl 分别能验证哪一层的连通性？

答案　ping 主要验证 ICMP 是否有回应；防火墙或网络策略可能丢弃 ICMP，因此失败不必然说明业务不可用，成功也不表示 TCP 端口开放。nc -vz 可尝试 TCP 建连，帮助区分拒绝与超时，但它不校验 HTTP 等应用协议。UDP 没有同样的握手，探测命令无报错也不等于服务可用。

curl 可以继续验证 HTTP 状态码、TLS 证书和响应耗时。排查从域名解析、路由、监听、防火墙、传输连接到应用协议逐层验证，并设置连接和总超时。不要用忽略证书校验掩盖 TLS 问题；探测成功只证明对应层面的检查通过。〔N202、N203、N201〕

代码示例

```
getent hosts example.com
ping -c 2 example.com       # ICMP失败不等于TCP服务不可达
nc -vz -w 3 example.com 443
curl --connect-timeout 3 --max-time 10 -I https://example.com/
```

005 free、top 和 vmstat 怎样区分内存不足、缓存与实际换页？

答案　系统可用内存不等于 free 一栏：MemAvailable 更接近当前可供应用使用而无需大量换页的估计值，部分页缓存可以回收。进程 RSS 也不是独占内存，多个进程共享页会重复计数；容器还需单独检查 cgroup 当前用量和上限。vmstat 1 的第一行通常汇总自启动以来的统计，后续才是采样区间；si/so 用于观察换入换出。sar -B 可看扫描和回收，其中后台扫描、直接回收及成功回收页分别反映不同压力。应结合持续可用量、major fault、PSI 和 OOM 记录判断；单看已用 swap 或一个百分比不足以下结论。〔N204、N205、N206〕

代码示例

```
free -h                     # 重点看MemAvailable
vmstat 1 5                  # 观察后续si/so采样
```

```
sar -B 1 5
cat /sys/fs/cgroup/memory.current /sys/fs/cgroup/memory.max
```

006 sed、awk、grep 和 sort 如何组合处理日志而不混淆统计口径?

答案　sed 擅长按行替换和编辑，awk 适合解析字段、筛选并累加；修改文件前先保留原件或输出到新文件，注意正则、分隔符和 shell 引号是不同解析层。grep -c 统计含匹配内容的行数，不是所有出现次数；若一行可多次出现，应逐次输出匹配再计数。

统计访问日志前先定义 PV 范围、状态码、时间窗和访客标识。一行请求不天然等于页面浏览，可能是静态资源、机器人或重试；仅用 IP 估计 UV 会受 NAT、代理和动态地址影响。固定空格切字段无法可靠解析含引号和空格的 User-Agent，应按日志格式使用解析器。大日志要流式处理，不要整体读进 shell 变量。〔N207、N208、N209〕

代码示例

```
sed 's/old/new/g' input.log > replaced.log
grep -F -o 'needle' input.log | wc -l
sort -k3,3nr access.log | head -n 10  # 先确认日志字段格式
```

007 Linux load average 和 CPU 使用率为什么可能完全不同?

答案　load average 是一段时间内任务数量的平滑平均，通常覆盖 1、5、15 分钟；可运行任务和不可中断等待任务都可能计入。它不是 CPU 使用率百分比。计算密集会抬高负载，磁盘等待也可能让负载很高而 CPU 并不忙。

可用 uptime 或 top 看整体，再以 mpstat 查看每核 user、system、iowait 和 idle，结合运行队列、D 态任务与 I/O 延迟判断原因；pidstat 可定位进程 CPU。容器看到的全机负载未必与自身配额相称。不能把 load 除以逻辑核数当成精确利用率，也不能仅靠一个阈值判断服务健康。

例如负载高、各核 idle 仍多且 D 态进程增加时，应优先查存储等待；若运行队列持续大于可用 CPU 数且 user 时间高，才更像 CPU 竞争。结合采样区间和服务延迟判断，比单次看 1 分钟负载可靠。〔N210、N199、N206〕

代码示例

```
uptime
mpstat -P ALL 1            # 区分usr/sys/iowait/idle
pidstat -u 1
ps -eo state,pid,comm | awk '$1 == "D" { print }'
```

18.2 Git 版本控制与协作

008 Git 工作区、暂存区和提交对象怎样流转?

答案　工作区保存当前编辑内容，暂存区记录下一次提交的候选快照，提交对象保存树、父提交和元数据；本地仓库与远端仓库各有自己的对象和引用。git add 把当时文件内容写入暂存区，之后继续编辑并不会自动更新同一暂存内容；git commit 主要提交暂存区。

fetch 和 push 用于与远端交换对象并更新引用，本地提交成功并不代表已上传。分支是指向提交的可移动引用。提交前分别检查工作区差异、暂存区差异和待推送提交，才能准确知道最终会记录与发布什么。

实际提交的是暂存区里的文件快照，不是编辑器当前展示的全部内容。若 add 后继续修改同一文件，需再次 add；审查 staged diff 可避免把不相关或敏感文件意外带入提交。〔N211、N212〕

代码示例

```
git status --short
git diff
git diff --cached
git log --oneline '@{u}..HEAD'
git commit                 # 只提交暂存区快照，不会自动push
```

009 merge、rebase 与快进合并怎样改变提交历史，冲突如何处理?

答案　merge 把分支关系合在一起，必要时创建有多个父节点的合并提交；若当前分支是目标分支的祖先，可以 fast-forward，只移动分支引用。rebase 则把一组提交的改动重放到新基线上，通常生成新提交 ID，线性历史更清楚，但会改写提交身份。

冲突时先理解两边修改意图，再编辑文件、检查结果并暂存；随后按操作流程继续或 abort。rebase 适合整理尚未共享的提交，共享分支改写会影响协作者，不能把无条件强推当成冲突解决。pull --rebase 表示获取后重放本地提交，与 merge 型 pull 不同。

若团队需要保留合并节点以记录集成关系，可选择 merge；若只整理私有提交序列可选择 rebase。解决冲突后要跑构建和测试，因为文本合并成功并不保证行为正确。〔N213、N211〕

代码示例

```
git fetch origin
git rebase origin/main      # 仅重放本地提交；共享历史需先协调
git add path/to/conflict
git rebase --continue       # git rebase --abort可撤销本次rebase
```

010 fetch 和 pull 对本地分支有什么不同影响?

答案　git fetch 下载远端新增对象并更新本地远程跟踪引用，通常不自动修改当前分支或工作区，适合先审查远端变化。pull 会先 fetch，再按配置和选项执行 merge、rebase 或仅快进，因此可能改变当前分支与文件并出现冲突。origin/main 是本地保存的远程跟踪引用，不是实时查询服务器。操作前确认当前分支、upstream 和未提交修改；需要谨慎集成时先 fetch，比较提交和文件差异后再决定 merge 或 rebase。
例如先 git fetch origin 再查看 origin/main..HEAD 与 HEAD..origin/main，可以分清本地领先和远端领先的提交；只有理解两边差异后才集成。〔N214、N211〕

代码示例

```
git status --short --branch
git fetch origin           # 只更新远端跟踪引用
git log --oneline HEAD..origin/main
git diff HEAD origin/main   # pull还会进一步集成到当前分支
```

011 git diff 的常见比较对象是什么，A..B 与 A...B 如何理解?

答案　git diff 不带提交参数时，主要比较工作区与暂存区；--cached 比较暂存区与提交，通常默认 HEAD；指定两个提交会比较两个快照。对于 diff，A..B 大体等价于直接比较 A 与 B；A...B 则以共同合并基点为起点比较到 B，常用于查看分支相对基线引入了什么。
这套点号含义不能套到 git log：日志中的 A..B 通常列出 B 可到达、A 不可到达的提交集合。可以用 --stat 先看改动规模，再看具体上下文。未追踪文件通常不在普通 diff 中显示，评审前还需查看 status。
审查前明确自己要比较文件快照还是提交集合。分支差异可先看三点 diff，再查看 status 中未追踪文件；这样不会把“没出现在补丁里”误当作“仓库没有这项改动”。〔N212〕

代码示例

```
git diff --cached
git diff A B
git diff A...B             # 比较merge-base到B的补丁
git log A..B               # 提交集合，与diff的点号语义不同
```

012 HEAD 和 detached HEAD 分别指向什么，怎样保留临时提交?

答案　通常 HEAD 是指向当前分支的符号引用；检出一个具体提交而非分支时，HEAD 可直接指向该提交，这叫 detached HEAD。游离状态下仍能编辑和提交，但新提交没有分支名指向；切换走以后可能只暂时留在 reflog 中，之后会被清理。
若要长期保留临时提交，可在当前位置创建分支或标签，让引用明确指向它；也可记录提交 ID 后补建引用。detached HEAD 不表示不能修改，但 reflog 不是长期备份。开始实验前检查当前状态，完成后为需要保留的历史取稳定名称。短期只读检查提交可以保持游离状态；一旦准备提交实验结果，先建分支能让后续切换、推送和协作都有明确名字。〔N215〕

代码示例

```
git switch --detach a1b2c3d
# 在游离HEAD上提交后，用分支名保存该提交
git switch -c experiment-save
```

013 reset 的三种模式与 revert 适用于哪些撤销场景?

答案　git reset 移动当前分支或 HEAD 到目标提交；--soft 保留暂存区和工作区，--mixed 通常重置暂存区但保留工作区，--hard 还会重置受版本控制的工作区文件，并可能覆盖妨碍检出的未追踪内容。它适合整理未共享的本地历史，执行前要检查目标和差异。
git revert 则通过新增反向变更提交撤销旧提交，保留已有历史，更适合共享分支；若后续提交依赖原改动，反向应用仍会冲突。撤销合并提交要明确选择主线父节点，并理解它会影响之后的再次合并。二者都不保证恢复所有未提交或外部副作用。
要撤销已推送的单次修改，revert 往往更容易让其他克隆保持历史一致；reset --hard 前应确认没有需要保留的工作区内容。〔N216、N217〕

代码示例

```
git show a1b2c3d           # 先审查撤销内容
git revert a1b2c3d         # 共享历史用新提交反向撤销
git reset --hard a1b2c3d   # 会覆盖工作区，先备份并确认
```

014 commit --amend 为什么会更换提交 ID?

答案　git commit --amend 会用当前暂存区内容及指定元数据替换最近提交，通常创建新的提交对象，因此提交 ID 会改变；只改说明也会变，因为说明是提交对象的一部分。它适合修正尚未共享的最近提交。工作区修改不会自动进入 amend，需先检查并按需加入暂存区。

如果旧提交已经被同事拉取，远端替换历史会让他们面对不同的提交基线；应按团队约定协调改写或提交新的修正。amend 还可能改变作者/提交者时间和签名。操作前查看 staged diff 和原提交内容，确保不会误把无关或敏感文件放入新提交。〔N211〕

代码示例

```
git add path/to/fix
git commit --amend          # 创建新提交对象，会改变提交ID
# 已推送的提交先协调团队，不要独自改写共享历史
```

015 switch、restore 和 checkout 怎样划分切分支与恢复文件的职责?

答案　checkout 历史上既能切换分支、检出提交，也能恢复路径，参数形式不同会产生不同效果。switch 主要负责切换或创建分支，restore 主要负责把路径内容从指定来源恢复到工作区或暂存区，意图更清楚。新命令不改变底层 Git 对象模型，只是把旧命令的多种职责拆开。

restore 的来源选项与 --staged 会影响究竟覆盖哪一区域，它不会自动创建提交保存旧内容。恢复前用 status 和 diff 检查修改；可能需要的文件先复制或提交。较旧 Git 版本可能没有 switch/restore，应确认版本和本机帮助。〔N211、N215〕

代码示例

```
git diff -- path/to/file
git restore --source=HEAD -- path/to/file
git restore --staged -- path/to/file  # 恢复前核对目标区域
```

016 stash 能暂存哪些修改，apply 和 pop 发生冲突时怎么办?

答案　git stash 通常保存已追踪文件的暂存区与工作区修改；-u 额外包含未追踪文件，-a 还会包含被忽略的文件。它是本地仓库中的临时记录，不会随普通 push 上传。apply 应用修改并保留 stash 记录，pop 通常在应用成功后再删除记录；冲突时需要像其他合并冲突一样检查和解决。

若希望恢复原暂存状态，可了解 --index 选项及其冲突限制。stash 适合短时间切换任务，不是长期备份；重要工作应建分支并提交。使用 -a 之前检查忽略目录里是否有体积巨大、私密或生成文件，避免意外打包敏感内容。〔N218〕

代码示例

```
git stash push -u -m 'temporary work'
git stash apply             # 应用但保留记录
# 解决冲突并确认恢复无误后再执行 git stash drop
```

017 密钥进入 Git 历史后，为什么只删除文件或加 gitignore 不够?

答案　凭据一旦进入 Git 历史，应首先在凭据所属服务撤销或轮换，因为旧提交、远端克隆、缓存和日志都可能已有副本。之后界定受影响分支、标签和协作者，按托管平台指南用专用工具改写相关历史，并协调重新获取基线。删除当前文件或新增 .gitignore 都不会让旧凭据失效。

历史改写会改变提交 ID，也无法保证收回他人的克隆或外部缓存；扫描器没命中不等于从未泄露。上传前应分别检查工作区、暂存区、准备推送内容和相关历史，凭据使用安全存储并添加合理忽略规则。沟通和工单中也只使用脱敏信息，避免再次传播凭据。〔N219、N220〕

代码示例

```
# 先在服务端撤销/轮换泄露的凭据
git status --short
git ls-files
gitleaks git . --log-opts='--all --full-history' --redact
```

018 cherry-pick 怎样选择单个变更，有哪些重复和依赖风险?

答案　cherry-pick 将选定提交引入的变更应用到当前分支，并通常创建一个新的提交对象，适合把一个修复移植到维护分支。它不自动搬运相关分支的全部上下文；若提交依赖此前重构、配置或其他提交，可能有冲突，也可能无冲突但构建或行为不正确。

可用 -x 在新提交说明中记录来源提交 ID，便于追溯。遇到冲突后按流程继续或中止；挑选合并提交需要明确主线父节点。后续再合并原分支时还要考虑重复变更与历史关系。审查补丁并跑测试，不能只凭提交标题决定它是独立修复。〔N211〕

代码示例

```
git switch maintenance
git show 4f2c1a0       # 核对修复依赖
git cherry-pick -x 4f2c1a0
# 解决冲突后运行相关测试并检查新提交
```

019 删除本地和远端分支后，提交是否立即消失?

答案　git branch -d 删除本地分支引用，并在通常情况下检查该分支已合并；-D 强制跳过检查。删除远端分支用 push remote --delete，清理本地过期远程跟踪引用则是另一项操作，例如 fetch --prune。它们分别修改不同仓库中的引用。

删除分支不会立刻抹掉提交对象；其他分支、标签、reflog 或克隆仍可能引用这些提交，之后才可能被垃圾回收。删除前核对分支所有者、合并状态和是否需要备份。敏感内容已进入提交时，删分支不能替代轮换凭据及历史清理。〔N211、N214〕

代码示例

```
git branch --merged main
git branch -d feature
git push origin --delete feature
git fetch --prune           # 删除前确认已合并；其他引用仍可保留提交
```

020 主干开发、功能分支与 Fork/PR 工作流怎样选择？

答案　工作流要按发布节奏、维护版本、团队规模与权限选择。Git Flow 用 develop、release 等长期分支组织版本发布，适合有明确周期的项目，但增加维护成本；GitHub Flow、GitLab Flow 常用短期分支和评审；主干开发要求小步集成和可靠自动验证，失败时能快速修复。

Fork 是托管平台上的独立仓库副本，PR/MR 是协作平台上的评审和合入请求，不是 Git 核心命令。无论哪种方式，都需明确基线、评审人、自动检查和合并策略。长期分支偏离越久，冲突和测试组合通常越多；分支数量本身并不能保证安全。〔N221、N211〕

代码示例

```
git switch -c feature/short-lived
# 小步提交并运行CI
git push -u origin feature/short-lived
# 发起PR，经评审与检查后按团队策略合并
```

021 git log 怎样查看分支关系并限制到相关变更？

答案　git log 展示提交图，可用 --oneline 简化摘要、--graph 显示分支合并线、--decorate 显示分支和标签；加 --all 可纳入本地可见的更多引用。这些都是查看方式，不会修改仓库。也可按作者、时间、提交消息或路径过滤，并加 -p 阅读补丁。

注意 git log 中 A..B 表示 B 可到达、A 不可到达的提交集合，这与 git diff 的 A..B 比较两个快照不同。--follow 对单个路径追踪重命名具有启发式限制，不能保证找全所有历史。定位回归要结合提交关系和真实补丁，不能只凭提交日期或作者直接判定原因。〔N222〕

代码示例

```
git log --oneline --graph --decorate --all
git log --author='Name' --since='2026-01-01' -- path/to/file
git log -p -- path/to/file
git log A..B                # 显示提交集合
```

022 轻量标签、附注标签与可验证发布有什么关系？

答案　轻量标签主要是一个指向 Git 对象的引用；附注标签则创建独立标签对象，可以保存打标签人、说明和可选的密码学签名。两者都可标记发布版本，但标签名本身不保证被签过名、内容可信或永不移动。验证签名还依赖可信公钥和身份管理。

push 并不总会上传所有本地标签，发布时应显式选定要推送的标签，避免把临时标签一并发布。用户已经依赖的版本标签通常应保持不变，修正应发布新版本。发布检查还要核对标签指向的提交和产物摘要。〔N223〕

代码示例

```
git tag -a v2.0 -m 'release 2.0'
git tag -v v2.0             # 有签名时验证身份
git push origin v2.0        # 显式发布标签
```

023 blame 和 bisect 如何帮助定位回归，分别有什么局限？

答案　git blame 按行显示当前内容最近关联的提交，适合找到相关变更和设计说明，但格式化、代码搬移和大规模重构会影响结果；它不是责任归属工具。git bisect 则在已知正常和异常的提交间，按提交图缩小范围，配合稳定的测试命令可定位引入回归的候选提交。

bisect 需要区分“好”“坏”和“当前无法测试”的提交。环境不稳定或中间提交不可构建会误导二分。找到候选后仍需阅读补丁、复现并验证因果；提交可能只是触发已有缺陷，不一定是唯一根因。〔N224、N222〕

代码示例

```
git bisect start
git bisect bad
git bisect good 9d3f1a2
# 每步运行同一回归测试并标记good/bad/skip
git bisect reset
```

024 gitignore 为什么忽略不了已经追踪的文件?

答案　.gitignore 主要告诉 Git 哪些未追踪路径不应自动加入索引；已经被追踪的文件不会因为后来写入忽略规则就停止跟踪，也不会从旧提交消失。若要保留本地文件但不再追踪，应对指定路径执行 git rm --cached 并提交索引变化，同时添加适当 ignore 规则。

可用 git check-ignore -v 查看具体生效规则，留意规则相对路径、目录匹配和否定项。忽略文件也不能保护已提交的秘密；旧历史中的凭据要另行撤销和清理。团队共享忽略规则、个人编辑器配置和系统缓存可放在不同位置，避免把个人习惯强加给全组。〔N220〕

代码示例

```
printf '\nconfig.local\n' >> .gitignore
git check-ignore -v config.local
git rm --cached config.local  # 保留本地文件并停止追踪
# 已提交过的凭据仍需轮换并清理历史
```

18.3 Docker 容器与部署工具

025 Linux 容器依靠什么隔离和限额，cgroup v2 与虚拟机有何不同?

答案　Linux 容器常用 namespaces 隔离进程、挂载、网络等可见视图，cgroups 统计和限制 CPU、内存等资源，再配合 capabilities、seccomp 等机制缩小权限。cgroup v2 使用统一层级并按规则启用控制器，cpu.max、memory.max 的具体限制和统计要依据目标内核文档解释。

容器通常共享宿主机内核；虚拟机一般运行独立客体内核，所以二者的隔离边界与兼容性不同。容器不等于轻量虚拟机。Docker Desktop 运行 Linux 容器时通常还经过虚拟机，其启动、内存和性能开销受部署影响，不能用固定数量级概括。

容器中的进程视图隔离并不会自动限制所有资源或赋予最小权限；部署仍需配置资源上限、用户权限和系统调用策略。分析问题时同时确认宿主机、容器和 cgroup 的观察层级。〔N225、N226〕

代码示例

```
cat /proc/self/ns/mnt /proc/self/ns/net
cat /sys/fs/cgroup/cpu.max
cat /sys/fs/cgroup/memory.current /sys/fs/cgroup/memory.max
# 容器共享宿主内核；虚拟机通常运行独立客体内核
```

026 Docker 客户端、Engine、镜像、容器与 Registry 怎样协作?

答案　Docker 客户端通过 API 向 Docker Engine 守护进程发送构建、运行和管理请求，Engine 再协调实际容器运行组件。镜像包含文件系统层与运行配置，是创建实例的模板；容器是在镜像上启动的运行实例，通常带有自己的可写层和隔离参数。同一镜像可派生多个容器。

Registry 用来存放和分发镜像；标签是可移动引用，digest 对应具体内容。停止容器不等于删除镜像，删除容器也不等于删除所挂载卷。远程 Engine API 权限、Registry 凭据和镜像来源都是独立安全边界；客户端命令所在机器未必是容器的真实宿主机。

部署时最好记录镜像 digest、容器参数和卷来源，避免只记录可移动标签后无法确定实际运行内容。清理容器与清理持久数据应分开审查。〔N225〕

代码示例

```
docker pull registry.example/app@sha256:0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef
docker run --name app -v app-data:/data registry.example/
app@sha256:0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef
docker inspect app --format '{{.Image}} {{.Mounts}}'
```

027 镜像分层和容器可写层如何影响缓存与磁盘占用?

答案　镜像通常由多层只读文件系统层和配置组成，容器再叠加自己的可写层。修改底层文件时，存储驱动可能采用 copy-up；不同指令的缓存和层行为也不完全相同。RUN、COPY 等会改变构建文件系统，ENV、CMD 等主要改变配置，不能认为每条指令都新增相同大小的数据。

后续层删除文件通常不会抹去先前层中的内容，所以把大文件或秘密先复制进去再在后续层删除，仍可能留在镜像历史。合理排序稳定依赖与常变源码可提高构建缓存复用；用镜像历史和磁盘统计确认效果。

构建缓存命中依赖指令及输入上下文是否改变；清理缓存只影响构建存储，不会自动缩小已经发布的历史镜像层。〔N227、N228、N229〕

代码示例

```
COPY requirements.txt /app/
RUN pip install -r /app/requirements.txt
COPY source/ /app/source/
RUN build-app
# 稳定依赖层在前；清理后续层不能删除先前层中的秘密
```

028 Dockerfile 的构建指令与运行配置如何区分，COPY 与 ADD 怎样选择？

答案　FROM 选择基础镜像或构建阶段，WORKDIR 设置目录，RUN 在构建时执行命令，COPY 加入构建上下文或其他阶段文件；USER、ENV、CMD 等设置镜像运行配置。EXPOSE 只是声明容器端口，不会自动发布到宿主机；VOLUME 声明挂载点，实际挂什么由运行时决定。

ADD 除复制外还支持特定归档解包和远程来源等功能，细节受 Dockerfile 语法版本影响；普通本地文件优先用更明确的 COPY。ARG 不是秘密存储通道，构建参数可能进入元数据或历史；秘密用专用 BuildKit secret 等方式。用 .dockerignore 缩小构建上下文并排除凭据和生成物。

运行时发布端口和挂载目录要在 docker run 或 Compose 配置中显式设置；Dockerfile 中的声明并不替代部署配置。〔N227、N228〕

代码示例

```
FROM build-image AS build
WORKDIR /app
COPY source/ .
RUN build-app
FROM runtime-image
COPY --from=build /app/server /server
USER nonroot
EXPOSE 8080  # 声明端口，不会自动发布到宿主机
```

029 CMD 与 ENTRYPOINT 如何组合，为什么 exec 形式影响退出信号？

答案　ENTRYPOINT 通常指定容器启动的主程序，CMD 提供默认命令或参数。使用 exec 数组形式时，docker run 后追加的参数通常替换 CMD 并传给入口，--entrypoint 可明确覆盖入口。要回答具体 argv，应把镜像设置与运行命令逐项组合，不能说入口永远不能被改。

shell 形式会先运行 shell，参数展开和信号传递与 exec 形式不同；包装脚本通常要用 exec 把 shell 替换为应用，使主进程更直接收到停止信号。但 exec 形式不保证业务能优雅退出，应用仍需处理信号、清理资源并回收子进程。

给容器停止信号留出合理宽限时间，并检查应用收到信号后的清理路径；仅改变启动语法不能保证进程不会遗留子进程。〔N227、N230〕

代码示例

```
ENTRYPOINT ["/app/server"]
CMD ["--port", "8080"]
# docker run image --port 9090会替换CMD参数
# 入口脚本应exec应用，使应用接收停止信号
```

030 多阶段构建怎样缩小运行镜像，为什么不应盲目选择最小基础镜像？

答案　多阶段构建可让前一阶段带齐编译器和开发依赖，运行阶段只从构建阶段复制需要的产物；也可用 --target 单独构建测试或调试阶段。配合 .dockerignore、合适的运行基础和同层清理临时缓存，通常能减少运行镜像体积。

不要盲目选择最小镜像而丢掉证书、时区、动态库或诊断工具。Alpine 使用 musl，与基于 glibc 构建的 C++ 程序兼容性需核实；静态链接也不是所有程序都成立。可复现还需固定基础镜像 digest、依赖版本和外部输入校验，并在目标环境验证。

比较镜像体积后还应运行 smoke test，确认可执行文件所需动态库、用户权限和默认目录均齐全；最小化不能以运行失败为代价。〔N231、N228、N227〕

代码示例

```
FROM compiler-image AS build
COPY source/ /src/
RUN build-app --output /out/server
FROM runtime-image AS runtime
COPY --from=build /out/server /server
USER nonroot
ENTRYPOINT ["/server"]
```

031 volume、bind mount、tmpfs 与容器可写层各适合保存什么？

答案　容器可写层跟随容器实例，容器重建后不应假定数据还在；关键数据应放到持久存储。Docker volume 由 Docker 管理，适合跨容器重建保留数据；bind mount 直接映射宿主路径，适合开发或共享文件，但依赖宿主目录布局并可能遮住镜像里的原目录内容。

tmpfs 用于临时内存文件，通常随容器停止消失，但还需考虑宿主机换页。volume 持久化不等于备份或高可用，数据库仍需一致性备份。配置挂载时明确只读权限、UID/GID 和宿主机可访问范围，避免给容器不必要的写权限。〔N229〕

代码示例

```
docker run --mount type=volume,src=db-data,dst=/var/lib/db image
# 开发源码可用bind mount，临时数据可用tmpfs
# 删除容器不会自动代表应该删除数据卷
```

032 Docker bridge、host、none、container 和 overlay 网络怎样影响容器通信?

答案　自定义 bridge 网络适合同一主机上的容器通信，通常能按容器或服务名解析；默认 bridge 的服务发现行为不同。host 模式共享宿主网络命名空间，网络隔离变少，-p 发布端口通常没有意义；none 则基本不配置外部网络。container:<name|id> 让容器共享指定实例的网络命名空间，因而共享接口和 localhost，通常不能单独发布一套端口。overlay 网络可连接支持该驱动的多主机容器，不能据此推断 Kubernetes 的所有网络实现都相同。目标端口、宿主发布端口与 EXPOSE 声明是不同概念，应逐项核对。〔N232、N230〕

代码示例

```
docker network create app-net
docker run --network app-net --name db database-image
docker run --network app-net --publish 8080:8080 api-image
# 同一用户bridge网络中服务可按名称通信
```

033 Docker Compose 描述了什么，服务依赖为什么不等于业务就绪?

答案　Compose 用声明式文件描述多服务应用的镜像或构建、网络、卷、环境和运行参数，可用 docker compose up、ps、logs、down 统一管理一组服务。项目网络通常支持通过服务名访问，因此配置可以减少手动容器间地址维护。服务启动顺序不等于业务就绪：数据库进程存在时，可能仍在恢复或拒绝连接。应配置 healthcheck、适当依赖条件，并让应用自身有超时和重试。Compose 能用于单机生产场景，但不提供 Kubernetes 式跨节点调度。down 与 down --volumes 对数据影响不同，真实凭据也不应明文提交。〔N233、N232〕

代码示例

```
services:
  db:
    image: database@sha256:digest
    healthcheck:
      test: ["CMD", "db-check"]
  app:
    build: .
    depends_on:
      db:
        condition: service_healthy
# 应用仍需重试连接并用自身探针报告业务就绪
```

034 docker run 的参数怎样决定运行实例，日常管理命令如何分工?

答案　docker run 创建并启动新容器，start 启动已有实例；-d 后台运行，-it 分配交互终端，-p 发布端口，--mount 指定存储，--network 选择网络，--memory/--cpus 设置资源约束，-u 选择容器用户。--restart 是重启策略，不等于健康检查；--rm 适合一次性容器。

ps -a、logs、inspect、stats 分别查看生命周期、输出、配置和资源观测；stop/restart 管理运行状态，rm 删除容器，rmi 删除镜像，build/pull/push 处理镜像。默认端口发布可能绑定更广的宿主地址，需时明确指定；环境变量也不是保密库，不要把真实令牌写进可共享命令。

命令行选项会共同决定容器安全与持久化边界，启动后用 inspect 核实最终配置。运行参数可审查但不一定能安全保存秘密，部署凭据应由受控配置机制注入。〔N230、N225〕

代码示例

```
docker run --name demo --rm -p 127.0.0.1:8080:8080 \
  --memory 512m --cpus 1 --user 1000 \
  -e APP_ENV=prod example/app:version
docker inspect demo
docker stats demo
```

035 docker exec 与 attach 对容器进程有什么不同影响?

答案　docker exec 在仍运行的容器里启动一个额外进程，可直接执行诊断命令，也可尝试打开镜像中已有的 shell；极简镜像可能没有 bash 或任何 shell。它不会改变容器主进程。docker attach 则连接主进程已有标准输入输出，输入和终端信号可能直接影响应用，退出组合键也要按配置处理。

exec 要求容器仍运行，不能让已退出的实例复活。调试优先读 logs 和 inspect，临时修改容器里的文件不会自动进入之后构建的镜像；应把修复写回源文件和 Dockerfile。还要考虑 exec 启动的诊断进程对运行状态的影响。〔N230、N225〕

代码示例

```
docker logs demo
docker inspect demo
# 容器运行时再exec诊断进程；attach会连到主进程标准流
docker exec -it demo sh
```

036 容器启动后立即退出，应从哪些证据判断原因?

答案　容器启动后马上退出，先看 docker ps -a、logs 和 inspect，核对退出码、启动命令、错误字段及 OOMKilled。主进程完成一次性任务后退出属于正常情况；服务主进程启动失败或主动后台化，也会使容器停止。加 -d 只让客户端后台返回，不能让已退出的进程继续活着。

137 常与 SIGKILL 有关，但不能单独证明是 OOM；要结合 OOMKilled 标记、内核事件和资源限额判断。再核对工作目录、挂载遮挡、权限、动态库、端口和依赖就绪，必要时有控制地覆盖入口复现。最终把修复写回可重建配置，并验证新镜像。〔N230、N225〕

代码示例

```
docker ps -a
docker logs demo
docker inspect demo --format '{{.State.ExitCode}} {{.State.OOMKilled}}'
# 结合启动命令、权限、挂载和宿主OOM证据排查
```

037 docker commit 为什么不能替代可审查的镜像构建流程?

答案　docker commit 根据容器当前可写层和部分配置生成镜像，可用于临时保存调查结果，但不会把挂载卷中的数据正常纳入提交，也不是进程内存检查点。它隐含了容器运行期间的手工变化，难以审查和复现，因此不适合作为持续交付的唯一构建方式。

docker build 从 Dockerfile、上下文和依赖执行，能纳入版本控制与审查，但也不自动保证可复现；还要固定基础镜像、依赖版本和外部下载内容。两种方式都可能带入秘密。容器里验证过的修复应回写源代码和构建步骤，并从干净环境重建。〔N228、N227、N229〕

代码示例

```
# 把验证过的修改写回Dockerfile和源码
docker build --no-cache -t app:reviewed .
# 扫描最终镜像层中的秘密；docker commit隐藏手工步骤且不含卷数据
```

038 Docker 构建的镜像与 Kubernetes 的容器运行时是什么关系?

答案　Docker 提供镜像构建、分发和容器运行工具；Kubernetes 负责声明式集群编排、调度、扩缩容和恢复。Kubernetes 通过 CRI 与受支持的运行时通信，例如 containerd 或 CRI-O。Kubernetes 1.24 移除内置 dockershim，不表示 Docker 构建的标准镜像不能运行，因为镜像格式与运行时接口是不同层次。

外部兼容适配器及具体部署仍应按集群文档判断。镜像能拉取也不能保证在节点正常运行；还需匹配 CPU 架构、操作系统、权限、卷和网络配置。Docker 命令行不是 Kubernetes 的运行时必需组件。

迁移时分别验证镜像可解析、容器可启动、探针可通过、存储能挂载和网络策略生效，不要把镜像兼容当成整个应用已经兼容。〔N234、N225〕

代码示例

```
docker build -t registry.example/app:1.2 .
docker push registry.example/app:1.2
kubectl apply -f deployment.yaml  # Kubernetes通过CRI让节点运行时启动镜像
```

039 Docker prune 的清理范围为什么必须逐项确认?

答案　先用 docker system df 和容器、卷列表确认占用与引用关系，再决定清理哪一类资源。image prune 默认主要清除悬空镜像，-a 会扩大到未被容器使用的镜像；system prune 会涉及多类未使用资源，但默认并不等于删除所有卷。

volume prune 的默认范围和选项会随 Docker 版本及卷类型而有细节，执行前要看本机 --help 和文档。停止容器仍可能关联需要恢复的数据，“当前未引用”也不代表业务无用。清理前核对备份、卷名和容器历史，不能把 prune -a --volumes 当作无风险通用命令。

生产清理应先确认资源负责人、恢复路径和维护窗口，再逐类执行并回读结果；清理命令的范围依版本及选项变化。〔N235、N229〕

代码示例

```
docker system df -v
docker volume ls
docker volume inspect db-data
# 确认备份和数据归属后再运行相应prune命令
```

第 19 章 后端进阶与平台工程

19.1 存储选型与代码组织

001 MongoDB 的文档模型、副本集与分片分别解决什么问题?

答案　MongoDB 用 BSON 文档保存字段与嵌套数据；把经常一起读写的小对象嵌入文档，可少做连接。但无限增长的数组和大量重复字段会让文档变大，也会增加更新成本。MongoDB 有索引、模式校验和事务，并非没有结构或不能事务。关系库与文档库应按查询方式、约束、关联和运维能力比较。

副本集在主节点接收写入并复制到其他节点，读写 concern 决定确认及读取语义；分片用分片键把数据分散到多机，mongos 根据键路由。副本主要处理可用性和冗余，分片主要扩展容量与吞吐，二者可以配合但不能解决所有热点。〔N236、N237〕

代码示例

```
auto doc = document{{"_id", 42}, {"name", "Aki"}, {"addresses", addresses}};
collection.insert_one(doc); // 文档嵌入适合一起读取且大小可控的子对象
replica_set.wait_for_write_concern(majority); // read/write concern决定确认级别
```

002 HDFS 如何把大文件存储和故障恢复分布到集群?

答案　HDFS 把大文件切成较大的块。NameNode 管命名空间和块位置等元数据，DataNode 保存块内容。读取时客户端先问 NameNode 副本在哪，再直接从合适的 DataNode 读块并校验 checksum。写入时 NameNode 返回写入节点链，客户端发给第一个 DataNode，数据沿副本管线传递，确认再沿链返回。

NameNode 用 fsimage 和 edit log 管理元数据；高可用部署的 Standby 跟随共享日志，故障时准备接管。心跳、块报告用于发现故障和恢复副本，纠删码也可减少某些场景存储开销。HDFS 适合大文件顺序高吞吐，不适合大量小文件或低延迟随机改写；副本不是异地备份，也防不了误删、权限错误和站点故障。〔N238〕

代码示例

```
for (const auto& block : file_blocks)
{
    auto nodes = namenode.allocate_replicas(block, replication_factor);
    datanode_pipeline.write(nodes, block); // NameNode管元数据，正文由DataNode传输
    verify_checksum(block);
}
```

003 业务分支变多时，怎样运用策略、工厂和观察者模式?

答案　先判断业务分支是否真有多种变化。数量少、规则清楚时 if 或 switch 更容易读；多种算法共用接口时，可用策略模式把选择和执行分开；工厂集中创建对象；观察者让事件产生者通知多个订阅者。若规则本身可表述成数据，表驱动可能更简单，但有顺序、副作用的流程不能机械换成哈希表。

设计模式解决的是某类变化，不是给代码增加抽象层的目的。要说明创建对象的所有权和生命周期、策略怎样选择、订阅者怎么取消，以及多余间接层的测试成本。代码质量看可读性、边界和修改成本，不看使用了多少模式。〔N239〕

代码示例

```
// 示意：创建、选择、执行分别承担明确责任
auto strategy = factory.create(request.type);
Result result = strategy->execute(request);
events.publish(result); // 订阅者失败策略与资源生命周期另行约定
```

19.2 订单、认证与接口边界

004 订单创建、支付回调与库存释放如何组成一致的状态机?

答案　先列出订单状态和合法迁移，例如待支付、已支付、待发货、已发货、完成或取消；售后可另有申请、退款处理中、完成等逆向状态。用条件更新或版本号原子争夺迁移，支付回调以渠道交易号唯一去重，并核对金额、币种和订单。超时任务只可关闭仍待支付订单，创建与本库库存预留尽量同一事务，跨服务则需可恢复消息和补偿。

取消与支付并发时要定义晚到款处理。退款请求超时可能渠道已成功但响应丢失，先按退款流水号查渠道状态，再幂等重试并对账，不能直接重复扣款或报失败。状态变更用 Outbox 驱动下游，使业务状态和事件保持可恢复。〔N127〕

代码示例

```
if (payment.valid_for(order) && order.compare_exchange_status(Pending, Paid))
{
    outbox.append_once(payment.transaction_id, OrderPaid{order.id});
}
else
    handle_duplicate_or_late_payment(payment); // 超时关单也用条件更新竞争
```

005 用户密码应如何保存、校验和渐进升级？

答案　密码应使用专门设计的慢哈希保存，而不是明文、可逆加密或快速 SHA-256/MD5。每条记录用随机盐，并保存算法标识和成本参数；可用成熟库提供的 Argon2id、scrypt 或合适的 bcrypt/PBKDF2。成本要按生产硬件调校，既让离线猜测昂贵，也不能让登录被轻易拖垮。

盐可以和摘要同库；可选 pepper 要放在数据库之外的密钥管理位置并安排轮换。登录时由库安全验证，若旧记录算法或成本不足，成功后重新散列并更新。还要做在线限速、泄漏密码检查和安全重置流程；日志、异常及监控不能记录原密码或完整凭据。〔N240〕

代码示例

```
auto stored = password_hasher.verify(input, user.password_hash, user.salt, user.cost);
if (!stored)
    return unauthorized();
if (user.cost < current_cost)
    user.update_hash(password_hasher.hash(input, secure_random_salt()));
create_session(user.id); // 仅保存专用慢哈希，不记录明文密码
```

006 单点登录如何用统一身份提供方建立各业务会话？

答案　单点登录并非让不同网站共享同一个裸 Cookie。用户在统一身份提供方认证后，业务系统按协议验证身份结果，再建立自己的会话。OIDC 授权码流程中，业务方校验重定向 URI、state 和 PKCE，后端换授权码并核对 ID Token 签名、issuer、audience、过期时间及需要的 nonce。

OAuth 主要表达授权委托，OIDC 在其上增加身份层。退出时要说明只是关闭业务本地会话，还是还要通知中央会话和其他业务。还要处理短期令牌、密钥轮换、跨站 Cookie 和登录 CSRF；身份提供方登录成功并不代表用户有每个业务的权限。〔N241〕

代码示例

```
auto state = secure_random();
auto verifier = pkce_verifier();
redirect_to_idp(state, pkce_challenge(verifier));
if (!constant_time_equal(callback.state, state))
    reject();
validate_id_token(callback.token, issuer, audience, nonce); // 再建立本地会话
```

007 对外 API 如何同时处理身份、权限、重放和重复请求？

答案　TLS 保护传输；认证回答“调用者是谁”，授权则在每次资源操作中检查对象归属和允许动作，不能只靠前端隐藏按钮。接口还要限制字段、长度、速率和计算成本，并避免日志记录凭据。若用应用层签名，要明确签哪些内容，例如方法、路径、正文摘要和时间窗口。

签名本身不证明业务权限，也不能自动防重放；需结合 nonce 或服务端状态拒绝重复请求。幂等键负责识别业务重试，应绑定调用者、请求内容及已保存结果。统一版本、错误码、审计及密钥轮换，避免客户端和服务端各自猜规则。〔N242、N243〕

代码示例

```
verify_tls_and_signature(request);
check_timestamp_and_unused_nonce(request);
if (!authorization.allows(caller, request.resource, request.action))
    return forbidden();
return idempotency_store.execute_once(request.key,
                                      request.body_hash,
                                      [&]
                                      {
                                          return handle(request);
                                      });
```

19.3 容量、隔离与运维

008 如何把“一秒内响应”转成可测量、可执行的延迟目标？

答案　“一秒内响应”应变成可测量的 SLI/SLO：定义成功请求比例、观察窗口和延迟分位数，例如 99% 的成功请求在 1 秒内完成，而不是保证所有请求永远低于一秒。把端到端预算分给入口排队、应用、数据库和外部调用，并沿调用链传播 deadline。

客户端超时不会自动停止服务器已开始的业务；服务端需检查取消和截止时间，尽早停止无用工作，否则重试可能与原操作并行。连接池、线程池和队列设上限，重试受剩余预算与幂等性约束。用 trace 和压测看尾延迟及饱和点，过载时快速拒绝或降级，避免无界等待。〔N244〕

代码示例

```
Deadline deadline = now() + 900ms;
auto response = dependency.call(request, deadline - 100ms);
```

```
if (now() >= deadline)
    cancel_downstream();
record_success_rate_and_latency(response, percentile = 99); // 把“一秒”变成端到端SLO
```

009 Java、Go 与 CPython 的垃圾回收机制为什么不能混为一谈？

答案　Java 的 GC 行为依所选收集器而变，常见实现会用可达性分析、分代、并发标记或整理的不同组合，不能说所有收集器都用同一复制算法。Go 使用并发标记清扫和写屏障，在暂停时间、GC CPU 和堆空间之间权衡，应结合运行时指标和内存限制调优。

常规 CPython 主要靠引用计数回收对象，同时检测部分容器构成的不可达引用环；细节会随版本和构建变化。三者都不能回收仍被业务引用的数据，也不保证及时关闭文件、连接等外部资源。使用 finally、with、defer 或显式 Close 等结构化清理资源，而不是等 GC。〔N245、N246、N194〕

代码示例

```
File file = open_file(path);
try
{
    auto data = file.read_all();
}
catch (...)
{
    file.close();
    throw;
}
// GC管理内存对象，不替代文件、锁、socket等外部资源的确定性关闭
```

010 四层与七层负载均衡按什么粒度转发？

答案　四层负载均衡按 IP、端口和传输层连接选择后端，通常不理解 HTTP 请求内容；七层代理终止连接并解析 HTTP，可按域名、路径、头部或租户路由，也能应用请求级超时。七层会增加解析和连接管理成本，但不能简单断言总比某个四层方案慢。

要把 TLS 在哪里终止、长连接复用、源地址如何传递、健康检查及跨区费用一起考虑。层次决定可见信息，轮询、最少连接和一致性哈希则决定如何挑后端，是不同维度。

如果只需在 TCP 连接建立时做后端分发，四层通常更直接；若需要按 URL 路径分流或读 HTTP 头，才需要七层解析。两层也可串联部署，各自承担不同决策。〔N247〕

代码示例

```
auto backend = layer4_route(destination_ip, destination_port); // TCP/UDP连接粒度
if (http.host == "api.example" && http.path.starts_with("/v2"))
    backend = api_pool.select();
// 七层代理解析应用请求；转发来源头只信任受控代理写入的值
```

011 突发流量或慢依赖出现时，怎样用资源隔离限制故障范围？

答案　舱壁隔离把有限资源按依赖、租户或关键路径分开，可用独立并发配额、连接池、线程池甚至进程边界，防止慢服务占满所有资源。但如果多个业务仍共用同一个 Redis、数据库或 MQ，大键、热点分区或连接耗尽仍可能一起拖垮它们；关键依赖必要时要按故障域拆开。

每个隔离区都要有有限队列、超时和拒绝策略，否则只是把无限积压搬到别处。限流控制进入速率，熔断停止调用故障依赖，舱壁限制影响面，三者可协作但不互相替代。切得太细会浪费空闲资源，应按真实饱和指标分配保底和弹性额度，并用故障注入验证关键路径。〔N248〕

代码示例

```
ThreadPool orders{20, 40};
ThreadPool reports{4, 8}; // 独立线程和队列限制故障范围
if (reports.queue_full())
    reject_or_defer(report_request);
// 若共享数据库已饱和，还需限制关键查询或进一步隔离依赖
```

012 监控指标用推还是拉？怎样避免指标系统本身失控？

答案　拉模式由采集端定时抓取目标，统一控制间隔、超时并产生 up 健康信号；推模式由应用主动发送，适合短任务或网络边界，但接收端需处理背压、重试和身份认证。Prometheus 主要使用拉取；Pushgateway 适用范围较窄，不是常驻服务的默认转发站。

指标标签应低基数，不能无界加入用户 ID、完整 URL 或请求 ID。要保留单次事件细节用日志，跨服务路径用追踪。告警应围绕用户影响、错误率、尾延迟和资源饱和，而不只看平均值；监控系统本身也要限制序列数量、采集频率和保留时间。〔N249〕

代码示例

```
metrics.counter("http_requests_total", {{"route", "/orders"}, {"status", "500"}});
metrics.histogram("http_latency_seconds", {{"route", "/orders"}});
// 标签保持低基数；request_id等细节放脱敏日志或trace
```

013 旧库怎样迁到新库，同时保持增量追平和可回滚？

答案　迁移先定义新旧模式兼容和转换规则，取得一致快照并记录对应日志位置；随后用 CDC 或等价增量通道持续重放后续变化。只做一次全量复制就切流，会漏掉复制期间的新写入。增量处理要可幂等恢复，并处理删除、更新顺序与表结构变更。

按主键范围核对行数、校验和及业务约束。追平后用写入栅栏、短暂冻结或受控路由确定最终边界，再分阶段放量。回滚必须考虑新库已经接受的写入：要么反向同步，要么设明确回滚窗口和写入策略，不能简单把读开关拨回旧库。〔N250〕

代码示例

```
auto snapshot = source.export_consistent_snapshot();
target.import(snapshot.data);
cdc.apply_from(snapshot.binlog_position);
compare_row_counts_and_hashes();
source.fence_writes();
cdc.wait_until_caught_up();
router.switch_small_cohort(target);
// 回滚前必须处理新库切流后的写入，避免丢失
```

19.4 容器编排与平台边界

014 容器运行时和 Kubernetes 分别负责什么？

答案　容器运行时负责按镜像创建、启动和隔离进程；Kubernetes 则在集群上协调声明的期望状态。Pod 是调度和共享部分资源的基本单元，Deployment 管副本和滚动更新，Service 提供稳定入口并选择后端。调度器、控制器和节点组件一起让实际状态逐步靠近配置。

Kubernetes 通过 CRI 对接兼容运行时，不要求节点安装 Docker Engine。它也不会自动替应用保证数据一致、备份或容量；还要配置 requests/limits、探针、持久卷、网络和权限策略，并测试节点故障和数据恢复。

例如声明三个副本后，一个 Pod 退出，控制器会尝试创建替代 Pod；如果镜像拉取失败或资源不足，实际副本数仍可能低于目标。声明式配置表达期望，不是对服务可用性的保证。〔N251〕

代码示例

```
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: app
          image: registry.example/app@sha256:digest
# 编排控制期望副本；探针和资源限制按业务设置
```

第 20 章 大模型与 AI 应用工程

20.1 Agent 架构与运行控制

001 什么是 Agent？它与直接调用大模型有什么区别？

答案　Agent 是围绕目标持续观察环境、选择行动并根据结果调整下一步的系统。一次普通大模型调用主要把当前输入变成输出；Agent 把模型接到工具和环境反馈上，可能经历多轮决策。这里模型提出“调用工具”只是一个请求，真正读文件、查业务或改状态的是宿主程序，程序应验证调用再执行。

判断是否是 Agent，不看提示词有多长、界面是不是聊天，也不看有没有调用过一次工具，而要看系统是否能用执行结果决定下一步和何时结束。自主行动范围由宿主授权，不代表拥有无限权限。任务简单且路径固定时，单次模型调用或确定流程往往更清楚。〔A1、A2〕

代码示例

```
Goal goal{"查杭州天气并建议是否带伞"};
auto proposal = model.plan(goal, tools);
if (proposal.calls_tool())
{
    auto result = host.validate_and_execute(proposal.tool_call());
    proposal = model.continue_with(result);
}
return proposal.final_text(); // 宿主执行工具，不让模型直接访问环境
```

002 Agent 的核心架构与 Harness 运行循环如何设计？

答案　Harness 是驱动 Agent 完成工作的运行时：保存任务状态，按需整理上下文，把模型提出的行动交给执行器，再把真实结果写回状态，决定继续、等待还是结束。模型、工具接口、状态存储、上下文管理、调度器和结果校验是主要边界；规划和长期记忆可按任务复杂度加入。

运行时还要设步数、费用和时限预算，记录版本并支持中断恢复。完成条件应是可验证的业务结果。快照只能保存已经写入的状态，恢复时仍需核对数据库或外部工具是否已执行，避免重复副作用。会话内循环依赖当前进程；独立调度器则可持久安排下一轮。周期任务还要记录时区、触发时间、任务 ID 和重入策略，处理停机错过的轮次。不同产品的会话及恢复规则各不相同，必须按接口文档核实。〔A3、A4、I111〕

代码示例

```
while (!task.done() && task.budget_remaining())
{
    auto proposal = model.next(task.context());
    auto result = host.execute_with_policy(proposal);
    task.persist(result);
    if (acceptance_test(task))
        task.mark_done();
} // 恢复前先核对外部操作ID，避免重复副作用
```

003 Workflow、Agent 与 Tool 有什么区别，怎样组合？

答案　Tool 提供一次明确能力，例如查资料或计算；Workflow 按预先定义的控制流组织步骤；Agent 则允许模型依据目标和观察动态决定后续行动。差别主要在控制权，而不是彼此排斥：固定流程可以包含一个 Agent 节点，Agent 也可以把子流程封装成工具来调用。

链式步骤适合阶段固定的任务，路由适合按类别分流，并行适合互不依赖的输入，生成后校验适合有明确验收标准的输出。流程固定不代表结果必然正确，里面的模型或服务仍会失败。应按路径变化程度、验证成本和维护复杂度选足够简单的结构。〔A5〕

代码示例

```
auto address = workflow.validate(order.address);
auto kind = agent.classify(order.description); // 仅动态决策交给Agent
if (kind == Kind::refund)
    return refund_workflow.run(order);
return exchange_workflow.run(order); // 固定步骤仍用普通Workflow
```

004 ReAct、Plan-and-Execute 与 Reflection 怎样工作，如何选用？

答案　ReAct 反复进行决策、行动、观察，适合必须边查边决定的问题；每次工具结果都可能改变下一步。Plan-and-Execute 先形成计划再逐项执行，适合依赖关系清楚的长任务，但遇到环境变化要重新规划。Reflection 在执行后加入评价和修订，前提是能拿到测试、规则或人工反馈。

这些方法可以组合，例如先拆计划，执行时用 ReAct，并在阶段末根据测试校验。反思模型可能重复原来的错误，应优先引入独立证据并限制修订次数。实现只需保存结构化行动、观察和结果，不必要求模型展示完整的隐含推理。〔A2、A6〕

代码示例

```
Plan plan = planner.create(goal);
for (Step& step : plan)
{
    auto observation = tools.execute(step.action);
    if (!tests.pass(observation))
        plan.replan(step, observation);
}
// Reflection要依据测试或工具证据，不能只让模型自我确认
```

005 如何规划与拆分复杂任务，使 Agent 的计划可执行、可验收?

答案　先把目标改写成可验收交付物和约束，再拆成有输入、输出、前置条件与检查方法的子任务，按依赖关系排序。粒度要能独立验证并保留业务含义，不要只按文件或技术层机械拆；只有互不依赖的步骤才适合并行。执行后用真实结果更新状态，条件变化时局部重规划。

CoT 组织中间推理，ToT 搜索多个候选并回退，GoT 允许中间结果复用和合并；这些表示思考组织方式，不代表执行成功。编程任务可按可运行的业务增量推进，并验证跨模块集成，避免每个子任务单独通过却整体不可用。〔A7、A8、A9〕

代码示例

```
Task validate = make_task("定义登录契约");
Task implement = make_task("实现凭证校验", depends_on(validate));
Task test = make_task("验证成功和失败路径", depends_on(implement));
run_independent_tasks_in_parallel();
run_after_dependencies(test); // 每步都有验收条件
```

006 Agent 的短期与长期记忆如何存储、检索和治理?

答案　短期记忆记录当前任务的对话、目标、进度和待处理结果；可以持久化以支持重启。长期记忆保存跨任务仍有价值的事实、事件或操作经验。存储结构应跟访问目的匹配：精确属性适合数据库，语义相似查找可用向量，需要关系遍历时再考虑图。

写入时按单条事实或事件记录主体、来源、时间、置信度和适用范围，不要把整段对话一股脑塞进去。读取时按权限、相关性和时效筛选，再放入上下文。还要处理冲突、过期、删除和租户隔离；命中记忆只是候选证据，不能覆盖当前指令或权威业务数据。文件型记忆可先读索引再取主题细节。有些实现会把单次提取与跨任务整合分开，但不能据此推断所有产品都启用相同的记忆机制。〔A10、I90、I91、I92〕

代码示例

```
MemoryFact fact{"user", "output_language", "Chinese", source_id, now()};
if (authorized_to_store(fact) && !conflicts_with_existing(fact))
    memory.save_with_provenance(fact);
// 新任务检索时检查权限、时效和当前明确指令
```

007 何时采用多 Agent，怎样组织协作与动态交接?

答案　多 Agent 让多个各有任务和上下文的决策循环协作，常见结构有主管分派、并行研究和专门角色接力。只有子任务能清楚分工、需要隔离上下文或确有专业差异时，多 Agent 的额外调用和协调成本才值得；单 Agent 也能并行调用互不依赖的工具。

协作前要约定任务契约、输入输出、共享状态写入权和汇总责任。动态交接应限制目标，并传递目标、证据、未决事项和剩余预算，防止彼此转派。子 Agent 的结果需要验证与消歧，角色不同不代表结果独立可信；共享文件或数据库的并发冲突还要单独设计。〔A11〕

代码示例

```
auto a = spawn_agent(task_a, EvidenceList{});
auto b = spawn_agent(task_b, EvidenceList{});
a.wait_with_deadline(deadline);
b.wait_with_deadline(deadline);
return supervisor.verify_and_synthesize(a.result(), b.result()); // 仅独立子任务并行
```

008 上下文怎样压缩，触发阈值与保留内容如何确定?

答案　压缩上下文前先统计消息、工具定义和检索材料的实际 token，再为模型输出、后续工具结果及余量预留空间；不能统一按窗口某个百分比截断。可保留近期消息、总结旧对话、抽取结构化状态，或移除可重新读取的大工具结果。摘要应保留目标、约束、关键决策、证据位置、未决操作和工具调用关联，并标注代码/资料版本。

压缩结果应低于触发阈值，摘要也有损，所以保留原始记录供回查。提示缓存只减少重复计算，不会腾出上下文窗口。Responses API 的 compaction 可能包含加密且不透明的 compaction item；使用时应按文档将返回的完整窗口或必要项目继续传递，不能把该项当普通文字摘要改写或任意删除。〔A12、A13、I93〕

代码示例

```
if (estimate_tokens(input) + reserved_output > safe_budget)
{
    input = responses_compact(input); // 保留返回的完整状态项，不把其当可编辑摘要
```

```
}
request(input); // 另存任务目标和外部操作ID，便于恢复
```

009 何时手写 Agent，何时使用框架？

答案　框架是否合适取决于任务复杂度和你要自己承担多少运行保障，不取决于它是否流行。简单工具循环、特殊协议或需要精确掌控请求时，直接用模型 SDK 可少一层抽象，容易追踪输入、工具结果和错误。需要持久恢复、复杂分支或人工中断时，框架可能省去重复建设，但也增加状态模型、升级兼容和调试成本。

先拿代表性任务测成功率、延迟、成本和故障恢复，再决定抽象层级。手写仍需实现预算、校验和观测；使用框架仍要理解实际请求、状态变化和边界。两种方式都不替代业务结果验证。〔A1〕

代码示例

```
for (Implementation impl : {direct_sdk, agent_framework})
{
    auto result = evaluate(impl, success_cases, timeout_cases, cancellation_cases);
    report(result.success_rate, result.p99, result.cost, result.recovery_time);
} // 按可观测性、权限控制和维护成本选择
```

010 什么是上下文工程？编程 Agent 应如何选择代码上下文？

答案　上下文工程是在每轮模型调用前，把当前决策需要的指令、任务状态、工具说明、记忆和外部证据组织进有限输入。先明确要解决的问题，再定向读取相关材料；区分用户约束、可信系统规则和不可信外部内容，并保留证据来源与版本。编程任务要从入口、类型、调用者、依赖、测试和有效配置还原真实调用链，不能因打开仓库就假设模型读过全部代码。

信息不足时继续定向读取，信息过多时摘要并保留回查位置。评价的是决策是否正确、遗漏率和成本，而不是塞入多少材料。检索内容与用户指令有冲突时，必须按信任层级处理，不能让仓库或网页内容擅自改变权限。〔A14、A15〕

代码示例

```
auto context = read_interface_and_implementation();
context.add(callers_and_tests());
context.add(current_config_and_version());
context.mark_external_documents_untrusted(); // 只放与修改直接相关并可核验的证据
```

011 多轮任务如何管理状态、交接并可靠恢复？

答案　把对话消息、业务事实、任务进度和跨会话记忆分开，用任务 ID 与目标版本关联；目标变更后重新判断旧计划是否仍适用。交接要保留约束、决策、完成证据、待办和未知事项。代码任务还要记工作区基线、已保存及未跟踪文件、未提交变更、测试对应版本和仍运行的外部任务。

恢复时重新读当前代码和配置，核对快照之后的变化；操作结果未知时先查状态。聊天记录不保存环境，重放任务也不保证外部操作只执行一次。还要区分会话恢复、文件回退和远程控制：例如 Claude Code Remote Control 仍在原机器运行，云端新会话可能有独立环境。检查点只覆盖其追踪的文件，不能撤销数据库或网络副作用。〔A4、A16、I95、I94〕

代码示例

```
task.persist_checkpoint(git_state, test_revision, pending_operation_ids);
if (operation.status_unknown())
    operation.query_by_id(); // 回退代码不会撤销已发生的外部操作
```

012 如何设计 Agent 的评测集、判定器和线上指标？

答案　评测集要包含代表性任务、初始环境、允许操作和可判定目标，并覆盖长链路、异常、权限边界及容易误判完成的情况。结果层检查环境是否达到目标；轨迹层检查必要步骤、禁止动作和证据；运行层记录延迟、成本和人工介入。有效路径可能不止一种，不能要求工具调用序列完全相同。

优先用确定性规则和真实环境判定，主观质量再由人工或校准过的模型评审。随机执行需多次试验，分别报告单次成功和重复稳定性；还要做版本回归和线上分布监控。安全失败不能由平均分掩盖。〔A17〕

代码示例

```
auto run = evaluate(agent, test_case);
assert(run.order_status == expected_status && !run.used_unauthorized_tool);
metrics.record(run.success, run.p95, run.cost, run.human_handoff); // 评轨迹也评结果
```

013 如何识别并治理 Agent 的重复调用、路径震荡和死循环？

答案　重复循环常来自目标不清、状态没更新、工具反馈不够，或失败后仍套用同一策略。只数工具名称会把正常分页误报成死循环；应比较规范化参数、目标对象、返回值和业务进度，判断同一观察下是否重复行动，以及是否在若干状态间来回切换。

先补足状态和反馈，再设置连续无进展轮数、总步数、时限和费用上限。触发时可请求缺失信息、换策略、局部重规划或转人工。达到限制后保留已完成结果、失败原因和恢复点；不能只把最大轮数调高，否则只会放大时间与成本。〔A18〕

代码示例

```
Signature signature{tool_name, normalize(args), target_id, state.version};
if (same_signature_without_progress(signature, 3))
    stop_and_replan_or_ask_user();
```

```
if (step_budget_exhausted())
    persist_resume_state(); // 合法分页需确认游标前进
```

014 Agent 延迟升高时，如何用 Trace 定位瓶颈？

答案　先区分首个有用响应的时间和整项任务完成时间，再按任务类型、输入规模和版本比较分布，平均值容易掩盖长尾。Trace 应串起上下文构建、检索、每轮模型调用、工具调用、排队、等待和重试，并记录输入输出量、调用轮次及缓存命中。沿关键路径判断是单轮变慢，还是规划多绕了几轮；并行分支的时间不能简单相加。

模型首个 token 慢要看排队和输入处理，输出慢要看生成长度，工具慢则继续查它的下游。可减少重复检索和无效上下文、并行独立操作、复用稳定结果；优化后同时检查成功率和成本，避免以提前失败换取较短延迟。〔A19、A20〕

代码示例

```
Trace trace(task_id);
trace.span("context_build", token_count, elapsed);
for (const auto& call : model_and_tool_calls)
    trace.record(call.start, call.end, call.status);
// 并行跨度按关键路径计算，再比较优化前后成功率与P99
```

015 子 Agent 失联时如何接管，怎样避免并发修改冲突？

答案　子 Agent 超时只能说明暂时没响应，不能证明它已经停止。主管要保存任务拥有者、心跳、进度和代次；接管时让旧执行者的写入资格失效，并让存储或发布端校验代次，防止迟到结果覆盖新结果。接替者从已确认状态继续，对结果未知的外部操作先查询再决定。

并行写入要明确文件或对象的所有者，代码任务可用独立工作树和固定基线，由汇总者审查合并。文本无冲突不代表业务兼容，还要跑集成验证。独立上下文不等于独立文件系统：同目录的 Agent 可互相覆盖；Git worktree 隔离工作树和索引，但共享仓库对象，数据库、端口及外部服务仍需单独隔离。〔A21、A11、I96、I97〕

代码示例

```
if (lease.generation != current_generation)
    reject_stale_result();
Worktree isolated = create_worktree(task_id, current_generation + 1);
result = agent_b.resume(last_confirmed_checkpoint); // 发布端用代次隔离迟到写入
```

016 什么是任务幻觉？如何证明 Agent 确实完成了操作？

答案　任务幻觉是把计划、工具请求或受理回执说成已经完成，例如未跑测试却报告全通过。应区分准备执行、已发起、已受理、执行成功和结果验证，让最终结论引用执行器给出的证据。证据需关联任务与调用 ID、目标对象、实际操作、时间和适用版本。

修改后还要验证业务后置条件：重新读取状态，或确认测试运行对应当前工作区。异步受理、部分完成和结果未知应保持不同状态，模型不能自行把它们升级成成功。报告明确完成范围和证据位置；没有证据的部分就补查或继续执行。〔A22、A23〕

代码示例

```
auto receipt = executor.start_deployment(version);
auto status = executor.query(receipt.id);
if (status == Accepted || status == Running)
    report("尚未验证完成");
if (status == Succeeded && health_check().version == version)
    report_verified_success();
```

017 Agent 查询数据库时，如何防止越权、泄漏和查询语义错误？

答案　数据库权限必须由服务端依据真实用户和租户执行，不能相信模型生成的 user_id。优先提供有业务含义的查询接口，限制表、字段、行范围、返回量和资源消耗，并使用无法绕过行策略的最小权限账号；只读也可能泄漏数据。如果允许模型写 SQL，应解析语法并限制可操作对象和语句种类。开头是 SELECT 或用了参数绑定，都不能限制任意读取权限。提供准确关系、指标口径和时间范围，并用样例校验查询语义。输出要脱敏；数据库结果是数据，不应被当作更改权限或发起动作的新指令。〔A24、A25〕

代码示例

```
auto ast = parse_sql(model_generated_sql);
if (ast.has_write() || !policy.allows(ast.tables(), ast.columns(), tenant))
    reject();
auto rows = db.execute_with_row_limit_and_timeout(ast, tenant);
redact(rows); // 结果仍是数据
```

018 Agent 如何从反馈改进，又避免把错误经验固化？

答案　所谓自进化通常是根据运行反馈更新提示、记忆、技能或策略，不代表模型自动改了权重。收集与任务版本绑定的轨迹、真实结果和人工反馈，先区分是决策错、工具故障还是需求含糊，再提炼带适用条件的候选经验。经验要保留来源、反例和版本。

候选规则先在独立回归集上检查质量、安全和成本，再逐步启用并保留回滚与过期删除。单次成功不能证明规则普遍有效；网页或工具里的不可信指令不能直接写成高优先级规则。线上失败应进入审查流程，避免未经验证的自动更新污染后续任务。〔A26、A25〕

代码示例

```
auto candidate = derive_rule(repeated_failures, conditions, counterexamples);
if (regression_set.pass(candidate) && candidate.unauthorized_calls <= baseline)
    gray_release(candidate);
else
    discard(candidate); // 单次成功不自动改生产策略
```

019 Agent 何时请求人工确认？权限与外部内容的信任边界如何控制？

答案　先分清需求澄清、行动审批和人工接管：缺关键业务规则时才澄清；超出现有授权或影响范围明显改变时才审批；系统无法可靠继续时再交接。用户已授权范围内的必要步骤应继续做完。审批页面要展示具体对象、动作、参数和后果，并绑定操作者、内容版本与有效范围；脚本或目标改变后重新确认。

网页、仓库和工具返回只能作为数据，不能授权新权限或撤销用户约束。执行端要强制认证和访问控制，拥有某个工具不等于能操作所有资源。审批策略和操作系统沙箱是独立控制：关闭提示不一定关闭隔离；所谓规划模式也不能只靠提示词做到只读。〔A27、A25、I98〕

代码示例

```
Approval approval = request_human_approval(hash(action), actor, expiry);
if (hash(action) != approval.action_hash || now() > approval.expiry)
    require_new_approval();
sandbox.enforce_filesystem_network_and_tool_limits(); // 网页或模型文本不能扩大权限
```

020 相同输入为什么出现不同执行结果？如何定位并控制差异？

答案　用户消息相同，不代表模型请求或执行环境相同。记忆、检索结果、工具数据、系统指令、模型版本、并发顺序以及采样和数值计算都会变化。排查时保存每轮真实请求、状态版本、工具结果和运行配置，比较两条轨迹最早分歧发生在哪一步，再判断是模型决策还是外部状态造成。

固定模型版本、输入和随机种子、降低采样随机性只能减少一部分波动，不能保证跨环境完全复现。历史事件回放、从快照重新请求模型、重放真实工具是三类实验。复现要控制外部依赖和副作用，并多次运行测稳定度。〔A28、A16〕

代码示例

```
auto first = replay_with_recorded_inputs(run_a);
auto second = replay_with_recorded_inputs(run_b);
compare_at_first_divergence(first.trace, second.trace); // 有副作用的真实任务不盲目重放
```

20.2 RAG 检索与知识治理

021 RAG 解决什么问题，离线建库与在线回答怎样衔接？

答案　RAG 在生成时检索外部证据，补充私有知识或较新事实，也便于给出来源；资料变化不必每次都重新训练模型。离线先解析、清洗、切块并保存原文位置、版本和权限，再建关键词或向量索引。在线按问题和对话构造查询，先应用用户权限和资料适用条件，再召回、可选重排或扩展上下文，模型据证据回答并引用。

向量只是检索定位方式，生成模型实际读到的是取回文本；RAG 也可用全文或结构化查询。改写和重排都属可选优化。证据不足时应补查、澄清或拒答，而不是由模型填空。〔A34、A35、A36〕

代码示例

```
auto chunks = index.build(parse_documents(files, version, access_policy));
auto evidence = index.search(question, filter_by_user_permissions(user));
return evidence.empty() ? ask_or_decline() : answer_with_citations(evidence); // 引用原文版本
```

022 RAG 与微调如何取舍，能否组合使用？

答案　RAG 把可更新的资料在回答时放进输入；微调用样本改变模型参数，更适合稳定的任务策略、格式或表达方式。频繁变化的事实、私有文档问答和可追溯证据通常先评估 RAG，但它有检索漏召和上下文成本。微调需要训练数据和验证，知识纠错及删除较难定位，也不保证逐条记住事实。

两者可以组合：微调模型更好地遵循格式和使用证据，RAG 维护当前事实；也可训练检索器改善召回。RAG 的定义不要求模型完全不训练。最终按事实更新频率、引用要求、业务正确率及总成本比较。〔A34〕

代码示例

```
if (fact_changes_often)
    evidence = retrieval.search(question, current_version);
else if (format_is_stable)
    model = fine_tune(model, format_examples);
return model.answer(question, evidence); // 微调不自动同步知识更新
```

023 RAG 怎样组织文档片段，并减少切块造成的语义断裂？

答案　切块要同时考虑检索定位和阅读理解。块太大容易混入多个主题，太小会失去定义、代词指向和适用条件。可按标题、段落和句子组织，再用 token 预算限制；表格切分时重复表头，代码保留必要结构。每块记录文档 ID、位置、版本和父子关系，原文独立保存。

重叠能缓解边界截断，却不保证语义完整。可先搜小块，再扩展相邻句或父章节补足上下文。Late Chunking 在长文本编码后按块池化，保留跨块语境，但有模型窗口和计算成本限制。块大小和重叠应通过业务召回及答案评测选择，而不是只凭经验定一个固定比例。〔A37、A38〕

代码示例

```
auto hit = search("refund period");
auto parent = document.load_parent(hit.parent_id);
auto context = expand_with_neighbors_and_table_header(hit, parent);
answer_with_source_page(context); // 重叠切块不能替代文档结构
```

024 Embedding 有哪些表示方法，RAG 如何选型和评估？

答案　Embedding 把文本映射到向量空间，以便比较语义相似度。Word2Vec 等给词静态表示，BERT 类编码上下文；检索双编码器分别编码查询与文档，可以预先算好文档向量。对比学习让相关样本更接近、负例更可区分。交叉编码器同时读取查询和候选，通常用于小候选集重排，精度潜力较高但成本也更大。

选择时检查语言、领域、最大长度、部署限制和吞吐，并用真实问题和标注片段测 Recall、排序和延迟。维数更高或排行榜更好不保证业务效果。距离度量、归一化和查询指令要符合模型约定；查询与文档编码必须兼容，换不兼容模型通常要重算整个索引。〔A39、A40〕

代码示例

```
auto candidates = vector_index.search(embed(query), top_k = 100);
auto ranked = cross_encoder.rerank(query, candidates);
report(recall_at_k(ranked), ndcg_at_k(ranked), p95_latency()); // 更换模型要重建向量
```

025 向量数据库怎样选型，如何定位规模与性能瓶颈？

答案　选型先看“检索是否够准、服务是否能稳定承载、团队是否能维护”，不要只比较向量条数。HNSW 把向量连成多层邻居图，查询时从粗层逐步走到近邻；增加搜索候选通常能找回更多结果，但会多花时间和内存。IVF 先把空间分成许多簇，只查部分簇，量化再压缩向量，能省空间但可能损失精度。

容量估算不能只用“条数×向量维度”：还要算图或倒排索引、原文、元数据、缓存和副本。压测时固定硬件、数据分布和过滤条件，同时测召回率、并发吞吐与尾延迟；把权限过滤、持续写入和后台建索引也放进负载。若内存抖动明显，分别检查索引驻留和后台任务；若过滤后结果少，则查过滤成本与候选策略。先测现有数据库或搜索引擎能否达标，再评估独立向量库的更新一致性、扩容与运维成本。〔A41、A42、A43、A36〕

代码示例

```
for (IndexConfig config : candidates)
{
    auto result = benchmark(index.build(data, config), queries, permissions);
    report(result.recall, result.p95, result.throughput, result.memory);
} // 同时测持续写入和后台合并
```

026 关键词与向量检索怎样互补，多路召回如何融合？

答案　关键词检索和向量检索擅长不同的匹配。倒排索引配合 BM25，能让稀有词、词频和文档长度影响得分，所以“E_CONNRESET”“型号 X7”这类精确词通常很好找。向量检索把语句映射到数值空间，能找“如何取消订阅”和“停止自动续费”这样的近义表达，但也可能把相似型号混为一谈。

混合召回可以并行查两路或多路结果，先按片段 ID 去重，再融合排序。原始 BM25 分数与向量相似度的范围不同，直接相加没有天然意义；RRF 只看各路名次，把每个结果按 1/(常数+名次) 累加，较容易融合。也可以先校准分数再加权。融合后仍可重排，而且每一路都要应用相同的租户、权限和版本过滤。额外召回会增加算力和噪声，需用真实问题集验证增益。〔A44、A45〕

代码示例

```
auto lexical = bm25.search(query, tenant_filter);
auto semantic = vectors.search(embed(query), tenant_filter);
auto merged = reciprocal_rank_fusion(deduplicate(lexical, semantic));
// 两路必须应用一致的权限过滤，避免融合泄漏
```

027 Query Rewrite、HyDE 与多查询扩展分别解决什么问题？

答案　Query Rewrite 是把口语或依赖历史的提问改成更适合检索的句子，例如把“那它退款多久”补成“订单 A 的退款到账需要几天”。改写必须保留实体、时间、版本和否定条件；若指代不清，先追问。多查询扩展会从不同角度生成几条查询并分别检索，去重融合后通常覆盖更广，但也带来额外调用、重复片段和噪声。

HyDE 先让模型写一段假设性的文档，再用它的向量寻找真实资料。假设文本只是检索线索，不能被当作事实证据直接回答。Step-back 先问上位原理或背景，再用结果帮助回答细节，例如先找“退款状态如何流转”，再找某订单状态。

工程上保留原问题作为一路，与改写结果共同评测；比较证据召回和最终答题质量，检查改写有没有偷换意图。这些技巧都不保证比原问题更准。〔A46、A47〕

代码示例

```
auto queries = make_queries_without_dropping_constraints(original_question);
auto results = parallel_retrieve(queries);
auto candidates = deduplicate_and_rank(results); // HyDE文本是假设，不作为引用证据
```

028 RAG 落地效果差时，怎样定位并优化问题?

答案　RAG 答错时先追踪一条失败问题经过的每个环节，不能上来就换更大的模型。先确认资料中确实有答案，PDF 解析没有丢表格、页眉或阅读顺序；再确认正确片段已进入索引、在召回结果里、重排后仍保留，最后看模型是否按证据作答。每一步都留存输入和结果，才能定位错误发生在哪一层。

如果证据在源文件却没入库，应修解析和增量流程；如果事实被切成两个块，可补标题、邻接块或父章节；如果问法与材料用词不同，再试查询改写或混合检索。召回到正确片段但它排名靠后，才适合评估重排；重排只能整理已有候选，找不回从未召回的证据。每次只改一类因素，在同一测试集做消融，同时观察质量、拒答、延迟和费用，避免堆组件后无法判断收益。〔A35、A48〕

代码示例

```
auto source = find_answer_in_original_documents(question);
auto retrieval = index.search(question, top_k);
auto answer = generate_with_citations(retrieval);
compare_each_stage(source, retrieval, answer); // 在首次丢失证据的阶段修复
```

029 Self-RAG、CRAG 与 Agentic RAG 怎样调整检索流程?

答案　这几类方法都把“要不要检索、检索结果是否可用、是否继续查”变成显式步骤，但实现机制不同。Self-RAG 的论文方案训练模型生成反思标记，用于判断是否需要检索、材料是否相关以及回答是否有依据；普通提示词让模型自我检查，不能直接称为同一算法。CRAG 会评估检索质量，再决定保留、精炼或补充检索，也可以引入外部搜索。Agentic RAG 则把检索当成可循环调用的工具：模型根据当前证据决定下一条查询，直到证据足够或达到停止条件。循环能够处理一次召回没覆盖的问题，也会放大判断错误、调用费用和等待时间。应限制总调用、轮数和上下文长度；连续几轮没有新增证据时应退出或降级，并规定外部搜索的数据边界。用固定问题集比较单次检索与循环方案的端到端效果。〔A49、A50〕

代码示例

```
for (int round = 0; round < 3 && evidence.insufficient(); ++round)
{
    auto more = search(make_query_for_missing_evidence(question, evidence));
    if (!adds_relevant_evidence(more))
        break;
    evidence.merge_deduplicated(more);
} // 预算耗尽仍不足就说明无法确认
```

030 什么时候用知识图谱增强 RAG，GraphRAG 增加了什么能力?

答案　知识图谱适合把关系当作一等数据处理，例如“公司 A 持有公司 B，B 又控制 C”，回答需沿关系走两跳。系统可先从文档找到实体，再遍历图中的边，并回到原文片段核验证据。向量检索负责相似语义，图遍历负责明确关系；实体消歧、关系抽取错误、有效时间和更新成本都要管理，图中每条关系应保留来源。

微软 GraphRAG 的一种方案会抽取实体关系、划分社区并生成分层摘要，便于回答“这批材料总体讨论了哪些主题”之类全库问题；局部问题则围绕命中实体取邻居和原文。LightRAG 结合低层实体检索、高层关系或主题检索、向量匹配和图邻域来组织证据，不要求同样预生成社区报告。普通片段检索配合查询分解也能解决一部分多跳问题，因此先按问题类型和实测收益决定是否建图，不把论文的成本比例当成普遍保证。〔A51、I99、I100〕

代码示例

```
auto entities = graph.find_entities(question);
auto paths = graph.expand(entities, max_hops = 2);
for (auto& path : paths)
    path.attach_source_and_validity();
answer_only_from_verified_sources(paths); // 图边也可能是抽取错误
```

031 怎样分层量化 RAG 效果，避免只看最终答案?

答案　评估要把检索、生成和线上业务分开看。测试集应保存固定的资料快照、问题和相关证据，并包含答案不存在、版本冲突等边界。Hit@K 看前 K 条是否至少有一条相关证据；Recall@K 看所有相关证据中找回了多少；MRR 关心第一条相关结果排第几；nDCG 还能按相关程度给排序打分。它们衡量的是检索，不代表最终回答正确。

生成层要检查是否回答了问题、事实是否准确、关键陈述是否得到证据支持，以及引用是否真的指向对应资料。忠实复述错误文档仍然是错答。Ragas 或模型裁判可以辅助批量评测，但需拿人工判断样本校准偏差。线上还应看解决率、纠错、拒答、延迟尾部和总成本；阈值依业务风险制定，任何单一指标都不能独自证明系统可用。〔A52、A48〕

代码示例

```
auto hits = retrieve(question, k);
metrics.hit_at_k = !hits.relevant().empty();
metrics.recall_at_k = hits.relevant_count() / relevant_count;
metrics.mrr = reciprocal_rank(first_relevant_rank(hits));
// 另评生成事实、引用、拒答和人工抽查
```

032 RAG 知识库怎样增量更新、失效和回滚?

答案　给每份资料分配稳定 ID，记录内容哈希、版本和片段清单。新文件、修改和删除可由事件或定时比对发现；处理任务应可重试且幂等，重复收到同一更新不会造出重复片段。修改后只重做受影响文档的解析、切分和编码；若切分边界难以对应，重建整份文档通常更稳。旧片段必须能关联回源文件和版本，便于审计与撤回。

发布时先写入新版本，检查数量、抽样内容和检索回归，再切换活动版本或集合别名，最后清理旧数据，避免先删后加造成服务空窗。删除还要同步处理原文、索引和缓存，并确认新查询已看不到它。更换编码模型或切分策略时，可并行构建新索引并保留回滚入口。图增强库还要根据来源依赖重算实体、关系和摘要；被其他有效文档支持的节点不能误删。〔A53、A54、I100、A51〕

代码示例

```
auto staged = ingest_new_version(document_id, content_hash, version);
if (validate_count_and_retrieval(staged))
    atomic_switch_active_version(document_id, version);
else
    keep_previous_version_active(); // 失败时旧版继续服务
```

033 RAG 检索到互相冲突的资料时，应该依据什么裁决?

答案　检索相关性只说明资料可能与问题有关，不能替代可信度判断。入库时应保存来源、负责人、审批状态、版本、适用产品或地区，以及发布时间和生效区间。查询时先匹配用户范围和提问时间，排除不适用、未生效或已废止内容，再按事先约定的来源优先级处理。发布时间较新不必然意味着它覆盖当前地区或产品。

如果同一条件下两份有效资料给出相反数字，应把冲突显式传到回答阶段。能够按适用范围和审批规则裁决时，引用有效资料并说明版本；规则不足时，列出分歧、追问必要条件，或交由知识负责人确认，不要让模型猜多数意见。还应记录答案实际引用的文档版本，方便资料被撤回时定位受影响的回答。〔A55、A36〕

代码示例

```
auto applicable = filter_by_scope_date_and_permissions(retrieve(question));
if (contradictory(applicable) && !has_approved_precedence(applicable))
    request_clarification();
else
    answer_with_selected_version_and_citations(applicable); // 分数不等于权威等级
```

034 长上下文为何出现中间遗忘，大窗口能否替代 RAG?

答案　窗口能容纳很多输入，不代表模型会平均利用每一处信息。Lost in the Middle 描述过一些模型在长文本中间位置的关键信息上表现下降；具体程度依模型、任务和干扰内容而变，不能当作所有模型都遵守的定律。长输入还会增加预填充计算和 KV 缓存，花费与延迟也会上升。扩大上下文并不会自动解决知识更新、权限和来源追溯。

只有少量文件且要全局比较时，直接把相关全文放入上下文可能更简单；资料量大、问题局部且重复出现时，先检索通常更省。常见组合是先用检索定位，再读取命中章节的完整上下文供模型综合。评估时应把证据分别放在开头、中间、末尾，改变噪声与证据数量，并测答案完整性、真实延迟和费用；单一的“针在草堆”测试不足以决定架构。〔A56、A57〕

代码示例

```
for position in ("start", "middle", "end"):
    prompt = place_evidence_with_fixed_distractors(evidence, position)
    score = evaluate_model(prompt, question)
    report(position, score)  # 比较位置敏感性，不泛化到所有模型
```

035 预建索引检索与 Agent 自主文件搜索怎样选型和组合?

答案　预建索引和 Agent 自主搜索的区别，在于资料是提前组织还是执行时探索。索引先完成解析和切分，适合大型资料库中高频的局部问答，查询快且路径可复用；代价是更新流程和索引质量需要持续维护。自主搜索会列目录、匹配内容、打开原文，再依据发现继续查，适合代码依赖追踪等线索难以预先枚举的任务。

自主搜索每多读一层就会增加耗时和调用成本，也可能因关键词不准或过早停止漏掉证据。可以先用索引给出可能的文件、位置和版本，再让 Agent 打开原文核验或补查。无论哪种方式，都要确认资料范围、权限和引用完整性，并限制读取量和工具预算。找到几条高分结果或搜过几轮，只能说明执行了搜索，不能证明所有证据都已覆盖。〔A34、A58〕

代码示例

```
auto locations = index.search(question, authorized_paths);
for (const auto& location : locations)
{
    auto source = read_with_neighbors(location);
    verify_citation_and_version(source);
```

```
}
// 定向检索后再限额查调用者；未读范围不能声称已覆盖
```

20.3 LLM 工具调用与协议

036 Function Calling 怎样形成模型决策与工具执行的闭环？

答案　工具调用是模型与宿主程序协作的一个闭环。宿主先给模型工具名称、用途和参数格式；模型可直接回答，也可提出一次或多次工具调用请求。模型返回调用意图并不代表工具已经执行，宿主必须校验参数、权限和业务条件，再真正调用 API 或程序。执行结果被送回模型后，它可以继续请求其他工具，也可以生成最终回答。

工具调用字段和结果回传格式会随接口而不同，因此宿主要保留调用 ID 与结果的对应关系。多个互不依赖的调用可以并行，存在先后依赖时则要等前一步结果。流式响应里参数常以片段到达，不能把未完成的 JSON 当成可执行命令；应按调用标识聚合完整输入，再解析和校验。结构化参数有助于机器读取，但不保证意图正确、事实真实或操作获授权。〔A109、I101〕

代码示例

```
auto proposal = model.request_tool_call(question, tool_schema);
auto args = parse_complete_stream_and_validate(proposal.arguments);
if (authorization.allows(user, args))
{
    auto result = tool.execute(args);
    return model.continue_with(result);
}
```

037 模型的工具调用能力如何训练和评测？

答案　训练工具调用时，常把工具定义、用户请求、模型选择、结构化参数、工具真实返回值和最终回答组成一条轨迹，再用监督微调教模型如何选工具和填参数。数据不应只有“调用正确并成功”的样例，还要包括无需工具、缺少参数时追问、多个步骤有依赖、可并行任务以及工具失败后的恢复，否则模型容易把任何问题都变成工具调用。

合成轨迹可以扩充覆盖面，但应先校验结构、实际运行可执行工具、去重并抽查结果，避免虚构工具输出污染数据。偏好优化或带执行反馈的强化学习可以进一步权衡成功率、调用次数与成本，却不是必经路线。评测时用未见过的工具、改写过的描述和边界输入，分开统计工具选择、参数准确、合理拒绝及端到端任务完成度，不能只测 JSON 能否解析。〔A110〕

代码示例

```
trajectories = []
for case in task_cases:
    if case.needs_clarification: trajectories.append(ask_user(case))
    elif case.needs_tool: trajectories.append(run_tool_and_record_real_result(case))
    else: trajectories.append(answer_without_tool(case))
# 分别评工具选择、参数正确性和最终任务成功率
```

038 MCP 的 Host、Client、Server 及 Tools、Resources、Prompts 各负责什么？

答案　MCP 是宿主应用与外部服务交换能力和上下文的协议。Host 管理模型、用户会话和授权，可在内部为每个 MCP Server 建立一个 Client；每个 Client 负责和对应服务通信。Server 可提供 Tools（可执行操作）、Resources（可读取的内容）和 Prompts（可参数化的消息模板）。这些名称描述接口类别，不代表工具天然安全；读取资料和修改状态都要由宿主按权限控制。

一次典型调用是宿主通过 Client 发现 Server 提供的能力，把合适的工具定义交给模型；模型返回调用请求后，宿主再让 Client 转发给 Server，并把真实结果交回模型。MCP 使用 JSON-RPC 消息约定，但统一通信格式并不会替应用完成业务逻辑、权限审批或模型选择。落地时还要核实传输、能力协商、认证和错误处理。〔A111、A112〕

代码示例

```
auto capabilities = client.discover(server);
auto proposal = model.choose_tool(question, filter_by_permissions(capabilities.tools));
auto result = host.validate_and_execute(client, proposal); // Host落实业务授权
```

039 Function Calling、MCP 与 Skill 怎样分工，什么时候值得引入 MCP？

答案　Function Calling 是模型接口表达结构化调用意图的方式；MCP 规定应用与服务怎样发现并交换工具和上下文；Skill 则是描述一类任务做法的说明、流程和可选资源。它们不是固定的三层依赖：Skill 可以直接调用 CLI，程序也可以直接使用 MCP Server，宿主还能把 MCP 发现的工具映射为模型支持的调用格式。

少量工具只服务于一个应用时，直接注册和执行往往更简单。若多个兼容宿主需要复用、统一发现和维护外部工具，或者已有 MCP 服务可接入，协议层的复用价值会增加。引入前还要考虑版本兼容、部署、授权和运维成本。工具数量多、使用了 Agent 或希望流程更先进，都不能单独证明必须采用 MCP；先明确需要解决的重复工作再选择。〔A113、A114〕

代码示例

```
skill = load_skill_summary(task)
if skill.applies and service_is_shared_across_hosts:
```

```
    result = mcp_client.call("weather", validated_city)
else:
    result = run_restricted_cli(["weather", validated_city])
# 工具是否复用和当前宿主能力决定是否引入MCP
```

040 怎样判断推理模型能否接入工具与 MCP?

答案　判断兼容性时拆开看三层：MCP Client/Server 协议由宿主或连接器实现，模型接口决定能否以受支持的格式提出结构化调用，具体推理后端还要能正确解析该格式。模型通常不需要直接理解 MCP 报文；宿主可以把发现到的工具定义映射成模型接口接受的描述，再把返回的调用意图转换为 MCP 请求。

若接口没有原生工具调用，也可尝试文本格式适配，但需自行解析并严格校验，可靠性通常需要额外验证。不同模型版本、模板和解码器可能对工具参数或推理模式有不同限制，应查看对应接口文档并实测完整的“提出调用—执行—回传—继续”流程。若某接口要求携带续接或推理相关字段，应遵循该版本约定，不能假设所有模型都相同。〔A115、A111〕

代码示例

```
auto tool_schema = adapt_mcp_tools_to_model(client.discover());
auto response = model.call(tool_schema, request);
if (response.has_tool_call())
{
    auto result = client.call(validate(response.tool_call()));
    response = model.continue_with(result);
}
```

041 Skill 如何通过渐进式加载进入 Agent 的上下文?

答案　Skill 通常是一个包含 SKILL.md 的能力目录：前置元数据写名称和用途，正文描述步骤，还可附带脚本、参考资料和模板。宿主先扫描目录并检查元数据，只向模型提供轻量的名称与描述清单；任务匹配或用户明确指定后，才读取对应正文，执行步骤需要时再加载文件资源。这个“先选、后读”的过程减少无关内容占据上下文。

要区分 Skill 已安装、对模型可见、正文已读取和动作已执行这几种状态。加载只把指令放入上下文，不会改模型权重，也不会自动运行脚本。Skill 描述应说明适用与不适用任务，避免多个条目宽泛重复；用触发和不触发样例检查误选、漏选，并按具体宿主验证上下文管理、压缩保留和脚本授权。〔A114、A116、I103〕

代码示例

```
auto registry = scan_skill_descriptions(skill_directory);
auto selected = model.select_relevant_skill(task, registry);
if (selected)
{
    auto instructions = read_skill_and_referenced_files(selected);
    execute_authorized_steps(instructions);
}
// 只看摘要不代表执行过完整步骤
```

042 A2A 如何支持 Agent 协作，与 MCP 的能力边界有什么区别?

答案　A2A 面向相互独立的 Agent 之间委托任务。接收方可发布 Agent Card，说明能做什么、如何连接及认证要求；调用方发现能力后发起消息或任务，接收方可以直接返回结果，也可以报告运行状态并交付产物。对耗时较长的任务，调用方还需处理状态更新、补充输入、取消和失败恢复。

MCP 主要让一个应用连接工具和上下文服务；A2A 主要让一个 Agent 把工作交给另一个具有独立执行能力的 Agent。被委托的 Agent 内部仍可使用 MCP，因此两者可以配合。卡片中的能力声明不能代替信任判断和权限授予。A2A 的 AgentSkill 是 Agent Card 中描述能力的条目，也不要与含 SKILL.md、供宿主加载的 Skill 包混为一谈；接入时需核实协议及传输版本。〔A117、A118〕

代码示例

```
auto card = discover_agent_card(endpoint);
verify_identity_and_authorization(card);
auto task = remote_agent.submit(goal, minimum_required_input);
while (task.running())
    task = remote_agent.wait_or_poll(task.id, deadline);
verify_artifact(task.result()); // accepted不等于completed
```

043 MCP 的 stdio、Streamable HTTP 与早期 HTTP+SSE 有何区别?

答案　stdio 通常由客户端启动本地 Server，通过标准输入和输出交换按行分隔的 JSON-RPC；标准输出只能放协议消息，日志放 stderr。Streamable HTTP 则把消息 POST 到单个 MCP 端点，响应可以是 JSON，也可以是与请求关联的 SSE 流。旧式 HTTP+SSE 使用独立的消息 POST 与服务端事件流端点。

按 2025-03-26 规范，Streamable HTTP 取代旧式 HTTP+SSE；2026-07-28 规范将旧传输正式列为弃用并设有迁移期。新旧版本在初始化握手、会话标识和请求元数据方面也有变化，不能只看都用了 SSE 就假定兼容。实际集成应明确双方协商的协议版和 SDK 版本；stdio 的子进程隔离、远程服务认证和网络访问仍需单独管理。〔A119、A120〕

代码示例

```
if (server_is_local)
    transport = StdioTransport(child_process);
else
    transport = StreamableHttpTransport(endpoint, authenticated_session);
transport.validate_protocol_version(); // 断线恢复和认证按协商版本处理
```

044 LLM 网关能统一哪些能力，切换模型与缓存有哪些限制?

答案　LLM 网关位于应用和模型提供方之间，可统一请求入口、凭据保管、租户限额、计费和审计，也可做路由、负载均衡或故障转移。它减少应用直接维护多套上游接入的工作，但不能让不同模型的语义自动一致。工具调用字段、结构化输出、上下文上限和流式事件都需要适配；路由到备用模型前要确认任务仍受支持，以及数据发往何处。

缓存也必须按语义边界隔离。缓存键应考虑租户、权限、模型与提示版本、资料时效及请求参数；个性化结果或实时状态不能只因向量相似就复用。网关要观测上游错误、尾延迟和总成本，并限制敏感日志及重试放大。流式内容一旦已发送给用户，中途透明切换通常难以拼接，需明确失败和恢复策略。〔A121〕

代码示例

```
auto key = hash(tenant, permissions, model_version, prompt_version, request.parameters);
if (request.is_cacheable() && cache.contains(key))
    return cache.get(key);
auto result = model_gateway.route(request);
if (result.complete())
    cache.put_with_ttl(key, result);
```

045 工具数量很多时，如何缩小候选集并评估 Tool Routing?

答案　工具很多时，先做目录和权限过滤，再让模型从小候选集里选择。注册表应记录工具命名空间、用途边界、参数结构、版本和授权条件。先按租户、角色、环境和任务状态剔除不可用工具，再用规则或分类器选领域，之后可用关键词、向量检索或重排找出少量候选，最后才把完整定义交给模型。候选与问题相关不等于用户有权调用。

候选不足时可追问，或按规则逐步扩大搜索范围；高风险操作不能因为排在第一名就自动执行。评测时分开看候选召回率、模型选择、无工具时是否正确拒绝、参数准确率和最终任务成功率，同时观察定义长度、延迟与越权暴露。只优化提示 token 数，可能把真正需要的工具也一并过滤掉。〔A122、A112〕

代码示例

```
auto authorized = filter_tools_by_role(tool_registry, caller);
auto candidates = rerank(retrieve_relevant_tools(task, authorized));
auto proposal = model.choose(candidates);
authorize_again_before_execution(proposal);
```

046 模型工具调用遇到格式错误、失败或结果未知时怎样恢复?

答案　先分清失败发生在哪层：模型没选对工具、参数格式错误、业务校验拒绝、权限不足、服务暂时不可用，还是副作用已经发生但响应丢了。格式问题可返回明确校验信息并让模型有限次修正；业务条件和授权必须由服务端检查。瞬时错误只在总预算内退避重试，避免模型、SDK 和调度器各自重试造成调用风暴。

写操作需在执行前保存稳定幂等键和调用状态。超时后先查真实执行记录，再决定重试、恢复或交人工处理；超时不能说明操作一定没发生。回传给模型的错误应是可继续规划的结构化信息，不泄露机密。达到预算或轮数上限时应明确失败或待处理。取消调用也不会自动撤销已完成副作用；外部完成状态应以可核验记录和产物为准。〔A112、A123、I102〕

代码示例

```
persist_call_state(idempotency_key, CallState::pending);
auto result = tool.execute(args, idempotency_key);
if (result.timed_out())
    result = query_actual_status(idempotency_key);
// 状态未知时不伪造成功，也不盲目重复不可逆操作
```

047 为什么封装工具 CLI，何时直接接入、何时再包装为 MCP?

答案　CLI 把确定流程包装成独立命令，方便 Agent、开发者、脚本和定时任务以相同方式调用，也便于脱离聊天现场重现问题。供程序使用的 CLI 应有稳定参数或标准输入、结构化 stdout、诊断 stderr、退出码和版本信息，并关闭交互式确认。退出码为零只说明进程按约定结束，仍要检查业务状态与产物。

宿主调用命令时应把参数作为独立数组传递，避免把模型生成文本拼成 shell 命令；还要校验路径和业务输入。单一宿主适配成熟 CLI 往往最省事；多个兼容宿主希望复用并统一发现和调用时，可在 CLI 外加 MCP Server。CLI 可以访问远程 API，MCP 也可以通过 stdio 使用本地进程，所以应根据复用范围、部署方式和维护成本选择。〔A124、A119〕

代码示例

```
ProcessResult result = spawn(argv_array, shell = false, timeout = deadline);
if (result.exit_code != 0)
    return structured_error(result.stderr);
auto status = parse_and_verify(result.stdout); // 多宿主复用后可再封装为MCP工具
```

048 怎样管理 Agent 生成代码的执行权限、资源与沙箱生命周期?

答案　把 Agent 生成的代码当作不可信程序运行。由沙箱外的运行器为每个任务分配隔离目录：输入只读、仅输出目录可写，网络默认关闭或按任务放行，不把宿主凭据和管理接口交进去。资源预算要覆盖 CPU、内存、进程数、磁盘、输出量和总时限；超时或取消时终止整个进程组，避免主进程退出而后台子进程继续运行。

普通容器仍共享宿主内核，隔离强度需按威胁模型评估；更强的用户态内核或虚拟机可能增加启动和资源成本。完成后只导出允许的产物，校验路径、大小和业务内容，再清理临时环境并保存执行证据。隔离能限制程序影响范围，却不能证明输出计算正确，也不能替代业务验收。〔A125、N226〕

代码示例

```
auto sandbox =
    create_isolated_workspace(read_only_inputs, network = false, cpu_limit, memory_limit);
auto process = sandbox.run(generated_code, timeout);
if (process.cancelled() || process.timed_out())
    sandbox.kill_process_group();
validate_allowlisted_artifacts(process.outputs());
sandbox.destroy(); // 沙箱不证明结果正确
```

20.4 大模型原理与工程

049 什么是大语言模型，与传统 NLP 的任务适配方式有何不同?

答案　大语言模型是在大量文本等数据上学习统计规律、再迁移到多种语言任务的模型；“大” 没有一个适用于所有工作的固定参数门槛。主流生成模型常用自回归目标：根据已经看到的 token 预测下一个 token，再重复生成。指令跟随、示例提示或微调可把这一能力用于问答、摘要等任务。

上下文学习是把示例和新指令放入当前输入，模型权重并未更新；微调才会改变参数。传统 NLP 也有预训练和生成模型，不能简化成 “一种任务配一个分类器”。统一文本接口降低了试用不同任务的成本，却不保证任何任务都适合大模型；规则清楚的分类或检索，用专用小模型可能更便宜、更稳定。〔A76、A77〕

代码示例

```
prompt = "Classify this request as billing or technical: ..."
label = model.generate(prompt)  # 上下文学习不更新模型参数
# 微调则使用训练数据更新权重，需用独立集合评测泛化
```

050 Transformer 如何计算注意力，Encoder 与 Decoder 有何区别?

答案　注意力让序列中每个位置按相关程度读取其他位置的信息。模型把输入分别投影成查询 Q、键 K、值 V，再计算 $\mathrm{softmax}(QK^{\top}/\sqrt{d_k})V$；除以根号维度可控制点积尺度，降低 softmax 过度饱和风险。多头注意力用多组投影分别学习关系，再合并结果；Transformer 层通常还含前馈网络、残差和归一化。

编码器可让位置双向读取输入，适合把整段文本编码成表示；原始解码器用因果掩码阻止读取未来 token，并可通过交叉注意力读取编码器输出。仅解码器架构通常没有编码器交叉注意力。训练时目标序列已知，可并行计算各位置并加掩码；生成时未来 token 尚未产生，仍需逐 token 展开。〔A78〕

代码示例

```
scores = query @ key.T / math.sqrt(head_dim)
scores = scores.masked_fill(future_token_mask, float("-inf"))  # Decoder因果掩码
weights = softmax(scores, dim=-1)
context = weights @ value  # Encoder通常可看双向上下文
```

051 MHA、MQA、GQA 与 FlashAttention 分别优化什么?

答案　MHA 为每个查询头保留独立的 K/V 头；MQA 让多个查询头共享一组 K/V，GQA 则把查询头分组后共享较少的 K/V 头。后两者主要减少历史 KV 缓存容量和解码时读取带宽，具体质量影响取决于训练和任务。稠密注意力预填充通常需要随序列长度平方增长的计算，解码则反复读取历史 K/V。

FlashAttention 通过分块并在线更新 softmax，减少显存中的大中间矩阵及往返读写，同时保持精确注意力计算语义；它没有把标准稠密注意力的算术复杂度改成线性。GQA 可与 FlashAttention 结合，但速度取决于硬件、序列长度、批量与算子实现，应以目标负载测吞吐和显存。〔A79、A80、A81〕

代码示例

```
kv_heads = {"MHA": 32, "GQA": 8, "MQA": 1}[attention_type]
cache_bytes = 2 * layers * sequence_length * kv_heads * head_dim * bytes_per_element
output = flash_attention(query, key, value)  # 分块计算，实测目标硬件收益
```

052 位置编码有什么作用，sin/cos、RoPE 与 ALiBi 如何取舍?

答案　自注意力本身若只看 token 内容，不知道这些 token 原先的次序；位置机制把顺序或距离信息加入计算。sin/cos 方法把固定频率的位置向量加到词向量上。RoPE 按位置旋转 Q、K 的二维分量，使点积带有相对位置差。ALiBi 则直接在注意力分数中加入随距离变化、不同头斜率不同的偏置。

这些方案提供不同归纳偏置，不能仅凭是否“相对位置”就断言效果优劣。RoPE 可配合插值或缩放扩展长度，但模型接口接受更长输入，不代表长距离检索和跨段推理质量可靠。位置扩展要在训练长度之外验证不同位置、间隔与任务的表现。〔A78、A82、A83〕

代码示例

```
q_rot = rope(query, positions); k_rot = rope(key, positions)
scores = q_rot @ k_rot.T / math.sqrt(head_dim)
# RoPE旋转Q/K；ALiBi则向分数加相对位置偏置
```

053 Tokenizer 怎样把文本交给模型，子词分词有哪些工程边界？

答案　Tokenizer 把文本编码成词表中的 token ID，模型再通过嵌入表把 ID 映射成向量。子词分词是在词表容量与序列长度间折中：BPE 反复合并常见相邻片段，Unigram 从候选片段中选择概率较高的分解；SentencePiece 是支持多种算法的工具框架，不是某一种分词算法。

字节级覆盖或字节回退可缓解未登录字符问题，但具体行为受词表、规范化及特殊 token 配置影响。训练和推理必须使用匹配的 tokenizer、聊天模板和角色边界，否则同一字符串可能变成不同输入。截断预算与费用要按目标模型实际编码结果计算，不能用汉字数按固定比例换算。〔A84〕

代码示例

```
token_ids = tokenizer.encode(chat_template.format(user_text=text))
if len(token_ids) > context_limit - output_reserve:
    token_ids = truncate_by_priority(token_ids)  # 用目标模型的tokenizer估算
```

054 预训练、SFT 与后训练分别解决什么问题？

答案　预训练通常使用大规模清洗语料学习语言规律、知识和任务模式，自回归模型常以预测下一个 token 为目标。SFT 用指令与示范答案继续训练，模型会学习格式，也可能学习任务行为、工具调用和拒答方式。后训练是预训练之后的广义阶段，常含 SFT、偏好优化或强化学习；奖励信号可由人类、模型或可执行验证产生。

这不是每种模型都必须严格顺序执行的一套固定配方，也可能包含领域继续预训练或多轮交替。工程上要检查训练数据质量和评测泄漏，保留独立测试集；领域收益可能伴随通用能力下降，因此应同时做目标领域和通用能力回归。〔A85〕

代码示例

```
model = pretrain(model, next_token_corpus)  # 语言建模
model = sft(model, instruction_examples)  # 指令行为
model = post_train(model, preference_or_reward_data)  # 偏好/策略
```

055 Scaling Law 如何指导算力分配，涌现是否有固定规模门槛？

答案　Scaling Law 是在特定数据、优化器和训练流程下观察损失如何随参数量、训练 token 数和算力预算变化的经验关系，可用于估算扩展训练的边际收益。Chinchilla 研究强调在固定训练算力下，模型规模和训练 token 应一起增长；常说的约二十 token/参数是其条件下的经验比例，不是所有模型的数据上限。

若目标部署期推理成本也重要，训练阶段可能投入更多算力，把较小模型训练得更充分。所谓“涌现”有时指能力指标在规模变化中突然变高，但离散评分方式也会制造这种观感，因此不存在通用固定参数门槛。应检查数据质量、指标和经验关系适用区间，再以目标任务评测验证。〔A86、A87〕

代码示例

```
for params in compute_candidate_scales(budget):
    loss = estimate_training_loss(params)
    deployment_cost = estimate_inference_cost(params)
    compare_on_target_tasks(loss, deployment_cost)  # 小规模试验验证趋势
```

056 全量微调、LoRA 与 QLoRA 如何选择，LoRA 的收益和边界是什么？

答案　SFT、DPO 描述训练目标或算法；全量微调、LoRA 描述更新哪些参数，是两个不同维度。LoRA 冻结底座权重，用低秩矩阵近似更新量 $\Delta W=(\alpha/r)BA$，减少梯度和优化器状态。QLoRA 在量化底座上训练适配器，可降低底座权重显存，但激活、梯度及推理缓存仍占资源。

适配器便于保存、切换和共享同一底座，条件适合时也能合并回权重；量化和多适配器服务延迟要实测。冻结底座不能保证通用能力不退化，秩、目标层和训练数据仍会影响能力。知识频繁变化可先考虑检索；稳定风格、格式或任务行为再结合质量和预算选择微调，并在领域集和通用回归集上验证。〔A88、A89〕

代码示例

```
for parameter in base_model.parameters(): parameter.requires_grad = False
output = base_model(x) + (alpha / rank) * (x @ A @ B)
train(A, B); evaluate_domain_and_general_regressions()  # QLoRA还需要激活与优化器内存
```

057 PPO、DPO、GRPO 与拒绝采样怎样用于后训练？

答案　经典 RLHF 流程常用偏好标注训练奖励模型，再用 PPO 优化策略；训练中通常还要参考策略 KL 约束和价值基线，因此工程环节较多。DPO 从 KL 正则化的偏好优化关系推导出直接训练偏好回答对的目标，不必单独训练奖励模型，但不等同于任意 PPO 训练。

GRPO 对同一问题采样多条回答，以组内奖励估计相对优势，减少独立价值模型需求，但仍需生成与评分成本。拒绝采样微调是先生成候选、筛选后再做监督训练；RLAIF 说明反馈可来自 AI。方法可组合，效果取决于偏好数据、奖励可验证性、探索需求和预算。〔A85、A90、A91〕

代码示例

```
candidates = sample_responses(model, prompt, count=8)
scored = [(answer, verifier(answer)) for answer in candidates]
accepted = [answer for answer, score in scored if score.passes]
# 拒绝采样依赖可靠验证器；PPO/DPO/GRPO需要各自反馈信号
```

058 贪心、Beam Search 与温度、Top-K、Top-P 怎样影响生成？

答案　解码每一步都要从下一个 token 的概率分布里选值。贪心总选当前最高概率，计算开销较低但只做局部选择；Beam Search 保留多个候选前缀，通常使用带长度处理的序列分数搜索，成本更高，也不保证全局最优或事实正确。采样先按温度缩放 logits：低温使分布更尖，高温更平。Top-K 保留固定数量的高分候选；Top-P 保留累计概率达到阈值的最小候选集，再归一化采样。开放生成要权衡多样性、稳定性和成本，没有对所有任务都最佳的温度。低温不保证完全可复现或答案正确；组合顺序、参数默认值及支持范围要查模型接口并用固定任务集比较。〔A92、A93〕

代码示例

```
logits = model.next_token_logits(context) / temperature
filtered = top_k_or_top_p(logits, k=40, p=0.9)
next_id = sample(softmax(filtered), rng); context.append(next_id)
```

059 KV Cache 与跨请求前缀缓存有什么区别，如何控制成本？

答案　因果自回归生成时，历史位置的 K/V 不会因后续 token 到来而改变，因此 KV cache 可保存过去各层的键和值，每一步只计算新增 token 的表示。它通常不必缓存每个历史查询向量。容量大致随层数、批量、序列长度、KV 头数、头维度和元素字节数增长；GQA 应按 KV 头数而不是查询头数估算。

跨请求前缀缓存进一步复用完全相同 token 前缀的预填充计算，主要节省 prompt prefill，并不直接返回最终答案，也不会免除后续生成。命中依模型、适配器、位置与实现策略。把稳定提示放前面可帮助复用，但要做租户隔离和缓存淘汰，并用真实请求分布测命中率和成本。〔A94、A95〕

代码示例

```
kv_bytes = 2 * layers * sequence_length * kv_heads * head_dim * bytes_per_element
request_cache = allocate_kv_cache(kv_bytes, concurrency)
# 跨请求前缀缓存复用prefill，不等于缓存最终回答
```

060 LLM 量化怎样节省资源，GPTQ、AWQ 与文件格式有何区别？

答案　量化用较少位数近似表示权重、激活或 KV，能减小存储和数据带宽需求，但总显存还包含尺度、零点、缓存和临时工作区。权重低比特并不表示所有乘加都以同一低精度执行；实际速度由硬件指令、反量化算子和批量形态决定。GPTQ 用校准激活和二阶近似逐层量化，并补偿误差；AWQ 按激活统计缩放权重以保护显著通道，并非简单保留固定少数高精度权重。NF4 是非均匀四位表示，常用于 QLoRA；GGUF 是模型存储格式，不是量化算法。应先核对推理后端支持，再比较实际任务质量、首 token 延迟和吞吐，不能只按位数推断质量损失。〔A96、A97、A89〕

代码示例

```
quantized = quantize(weights, method="GPTQ")
report(weight_bytes(quantized), kv_cache_bytes, throughput, task_quality)
# 格式、量化算法和后端算子分别核对兼容性
```

061 怎样把 Prompt 做成可迭代、可验证的工程接口？

答案　先把提示词当作一个有输入和输出约定的接口来设计：说明任务目标、受众、背景、可用依据、结果结构和失败条件，再用少量样例覆盖常见边界。角色称呼不会凭空增加知识。用分隔符区分外部文档和指令，把检索到的文本视为数据，不能让其中内容扩大系统权限。

程序需要消费结果时，优先使用模型接口支持的结构化输出，并在应用端再次验证字段和业务规则；“请返回 JSON”本身不保证结果可解析。把提示、样例、模型版本和采样配置一起版本化，用独立测试集对比成功率、格式错误、拒答质量、延迟和成本。每次修改围绕假设并加入失败样例回归，避免凭一次顺眼输出判断效果。〔A98〕

代码示例

```
prompt = build_prompt(task, untrusted_data=delimit(user_text))
result = model.generate(prompt, response_schema=TicketSchema)
validated = TicketSchema.validate_json(result)  # 格式合法仍需业务语义检查
```

062 CoT 为什么可能改善推理，使用时有哪些成本和可靠性边界？

答案　CoT 是让模型在给出结论前产生中间步骤的一类提示或训练方式，可把复杂问题拆成连续子问题；部分推理模型也会使用额外计算完成内部推理。它在一些数学和逻辑任务上可能有帮助，但中间文本不是正确性的证明，也不能替代外部事实检索或程序验证。

中间步骤可能出现错误、事后合理化或把错误继续传递；额外 token 还会提高延迟和费用。自一致性通过采样多条推理路径并聚合答案，有时能改善特定任务，却无法消除相关错误。对外通常给可核验的简要依据，再用计算器、测试或来源核查关键结论；提示方式应匹配模型接口并用任务评测验证。〔A99、A100〕

代码示例

```
candidates = [model.solve(problem) for _ in range(5)]
verified = [answer for answer in candidates if calculator.check(answer)]
return choose_verified(verified)  # 推理文本不能代替外部校验
```

063 大模型为什么产生幻觉，RAG 为什么也不能保证回答忠实？

答案　语言模型根据训练中学到的分布生成连贯文本，流畅并不等于真实。训练数据错误、知识过期、问题本身不可答或提示中缺少证据，都可能让模型给出貌似合理的错误；降低温度也不能保证事实正确。对重要内容需要外部证据和拒答策略。

RAG 能提供带出处的资料，却不能保证忠实：检索可能漏掉关键段落、找到过期版本或错误片段，模型也可能忽略证据或拼接矛盾信息。应保留资料来源与版本，让答案围绕证据写作，并核对每个引用是否真正支持结论；证据不足时澄清或拒答。评测分开看知识源、召回、上下文组织与生成错误，并测事实正确率、证据支持率和拒答质量。〔A34、A101〕

代码示例

```
evidence = retrieve_versioned_sources(question, permissions=user.permissions)
answer = model.generate(question, evidence=evidence, require_citations=True)
if not every_claim_supported(answer, evidence): return clarify_or_decline()
```

064 MoE 如何解耦总参数与激活参数，部署时要付出什么代价？

答案　稀疏 MoE 常把 Transformer 的部分前馈层替换成多个专家网络，由路由器为每个 token 选择少数专家并合并输出，也可能另设共享专家。总参数表示可存储容量，激活参数只表示单次 token 经过的部分；两者不能直接据此推算与同规模稠密模型的延迟或质量。

路由负载不均可能导致专家过热、容量溢出或跨卡通信放大，可用均衡损失、路由偏置和调度缓解；专家也不保证天然学成预设语义领域。服务部署仍需让所需专家权重可访问，显存、卸载和通信成本由总权重与并行方案决定。选型要同时测容量、通信、吞吐和尾延迟。〔A102〕

代码示例

```
expert_ids, weights = router.top_k(token, k=2)
outputs = dispatch_to_experts(token, expert_ids)
result = combine(outputs, weights); monitor_expert_load(expert_ids)
```

065 vLLM、SGLang、TGI 与 llama.cpp 如何按实际负载选型？

答案　先核对目标模型、聊天模板、工具调用、量化格式和硬件是否被具体版本支持，再比较推理框架。vLLM 常以分页 KV 管理和连续批处理提高并发；SGLang 通过前缀复用与调度优化处理复杂共享前缀场景，两者能力持续重叠，不能预设固定胜负。llama.cpp 使用 GGUF，支持本地 CPU 与多种 GPU 后端，适合本地和边缘运行。

截至 2026 年 10 月，Hugging Face 官方 TGI 仓库已归档并说明进入维护模式，只接受轻量维护；新项目还要考虑该状态带来的后续维护成本。性能测试要复现真实输入输出长度、并发及缓存命中率，同时测首 token 延迟、后续 token 时延、吞吐与显存；用单一短提示比较不足以代表线上负载。〔A103、A95、A104、A105、A106〕

代码示例

```
for backend in ("vllm", "sglang", "tgi", "llama.cpp"):
    result = benchmark(backend, fixed_workload, model, target_hardware)
    report(result.first_token_p99, result.tokens_per_second, result.memory)
```

066 如何评测并选择业务使用的大模型？

答案　先明确业务任务、数据边界、部署方式和延迟预算，用同一组业务样本、提示配置和资源约束比较模型。公开知识、推理或代码基准只能作参考，要注意训练污染、评分口径及采样次数不同。业务评测集按场景、难度和失败类型分层，并保留未参与调参的留出集。

客观任务用断言、单元测试或结构检查；主观答案可用盲评和模型裁判，但需人工校准长度、顺序和自偏好偏差。最终看任务成功率、严重错误与拒答质量、稳定性、尾延迟及完成一次任务的总成本，而非只看每 token 单价。可按任务路由多个模型，并持续检查降级路径的数据边界和能力。〔A107、A108〕

代码示例

```
for model in candidates:
    result = evaluate(model, held_out_cases, fixed_prompt, fixed_budget)
    report(result.task_success, result.severe_errors, result.p99, result.cost)
# 留出集不用于反复调参
```

20.5 LangChain 与应用框架

067 LangChain 的 Chain、Runnable 与 LCEL 如何组织数据流?

答案　Chain、Runnable 与 LCEL 分别解决流程组织、步骤协议和组合表达问题。Runnable 把模型、提示模板、解析器等统一成可调用组件，常见接口包括同步调用、异步调用、批处理和流式输出；LCEL 用管道符把这些组件组成 RunnableSequence，也可用 RunnableParallel 将同一输入送到多个分支，再把结果合并为字典。
这样的组合让数据流更容易测试和复用，但不会自动保证字段名、类型或错误语义一致，因此每一步仍要约定输入输出并处理失败。固定步骤的流程适合用 Runnable 组合；若要由模型动态选择工具、等待人工确认或从中断处恢复，则更适合 agent 或显式状态图。〔A59、A60、A61〕

代码示例

```
from langchain_core.runnables import RunnableLambda, RunnableParallel
pipeline = RunnableLambda(load_input) | RunnableLambda(format_prompt) | model | parser
parallel = RunnableParallel(summary=summarizer, keywords=keyword_extractor)
```

068 LangChain 怎样分层，旧项目升级到 v1 要检查哪些边界?

答案　升级前先分清几个层次：langchain-core 提供消息、Runnable、工具等核心协议；langchain 侧重创建 agent、结构化输出和中间件；LangGraph 承担带状态的执行与编排。LangChain v1 将旧版 Chain、部分 retriever 与索引接口移入 langchain-classic，因此旧项目可能遇到导入路径变化，不能只看安装成功就判断兼容。
迁移时应锁定彼此匹配的版本，逐项检查导入、消息角色与内容、工具参数 schema、结构化结果、流式事件、checkpoint 恢复，以及重试时外部副作用是否会重复。最后用真实请求跑关键工作流，因为 API 能调用并不代表状态和业务语义保持不变。〔A62、A63〕

代码示例

```
# 锁定并验证匹配版本的langchain与langchain-core依赖
result = agent.invoke(input, config={"configurable": {"thread_id": thread_id}})
assert result["messages"]  # 回归消息、工具参数、流式事件和恢复状态
```

069 使用 LangChain 构建 Agent 时，如何注册和可靠执行工具?

答案　在 LangChain agent 中，工具通常由带类型注解的函数或 @tool 定义，类型信息用来构造模型可见的参数 schema；create_agent 接收模型和工具集合。模型返回工具名、参数和调用 ID 后，运行时负责校验参数并执行工具，再以匹配该 ID 的 ToolMessage 把结果交回模型，循环到生成最终答复或达到步数限制。
工具描述和 schema 只是调用接口，并不构成授权；身份、租户与权限应由应用从可信请求上下文注入，再由服务端验证每次操作。还要限制重试和调用次数、区分可恢复错误，并让写操作具备幂等键或人工审批。结构化输出保证格式约束，不保证内容事实正确。〔A64、A65、A66〕

代码示例

```
@tool
def get_order(order_id: str) -> dict:
    """查询订单；服务端仍需校验当前用户权限。"""
    return repository.load_authorized(order_id, current_user())
agent = create_agent(model, tools=[get_order])
```

070 LangChain 的 checkpointer、store 与 thread_id 分别负责什么?

答案　create_agent 建立在 LangGraph 的状态执行之上。checkpointer 保存某个 thread 的图状态和执行进度；后续以相同 thread_id 调用，才能在同一会话中继续或从中断点恢复。store 则用于跨 thread 保存可按 namespace/key 读取的数据，也可配置索引做语义检索；它与单次执行里的 state、由应用注入的 context 用途不同。
生产环境要使用持久化后端，并按用户和租户隔离 thread 与 store 命名空间。内存版 saver 在进程退出后不保留检查点，不能当作持久化存储。恢复执行时某些节点可能重跑，所以外部写入应支持幂等或补偿；checkpoint 也不意味着必须把全部历史原样送进模型上下文。〔A67、A68、A4〕

代码示例

```
config = {"configurable": {"thread_id": authorized_thread_id}}
agent.invoke({"messages": [user_message]}, config=config)  # checkpointer保存thread状态
store.put((tenant_id, "profile"), key="name", value=name)  # store跨thread共享资料
```

071 LangChain、LangGraph 与 LlamaIndex 如何取舍和组合?

答案　LangChain、LangGraph 和 LlamaIndex 的重心不同，但边界并非互斥。LangChain 提供模型、消息、工具等统一接口，并提供常见 agent 创建方式；其 create_agent 本身运行在 LangGraph 之上。LangGraph 更适合显式描述状态、节点、路由、并行汇合、中断和恢复，让长流程控制逻辑可见。
LlamaIndex 重点在数据摄取、索引和检索增强，也提供 Agent 与 Workflow 能力。系统可以把 LlamaIndex 检索封装成 LangChain 工具，再由 LangGraph 编排检索、审核和回答。选型应看已有组件、团队熟悉度、持久化和调试需求，并用实际工作流验证；不要把 LangChain 简化成只支持线性链，也不要认为记忆或 agent 只存在于某一个框架。〔A69、A63、A70〕

代码示例

```
state = {"question": question, "evidence": [], "approved": False}
state["evidence"] = llama_index_retriever.invoke(state["question"])
# LangGraph管理审核、分支和恢复；LangChain可封装通用工具
result = graph.invoke(state, config={"configurable": {"thread_id": thread_id}})
```

072 LangChain4j 怎样把模型、工具和知识检索接入 Java 服务？

答案　LangChain4j 是面向 Java 的独立库，提供聊天模型、向量化、向量存储和检索等接口。AI Services 可根据 Java 接口生成代理，把提示构造、模型调用、工具循环或检索步骤封装起来；工具可通过 @Tool 暴露，ChatMemoryProvider 与 @MemoryId 可按用户或会话选择对话记忆，也能与 Spring Boot 集成。

但这些便利接口不会替代服务端的授权和业务校验。ChatMemory 适合维护模型当前需要的上下文，可能裁剪或淘汰旧消息；完整审计记录应单独保存。还需检查所选模型提供商实际支持的工具调用和结构化输出能力。长任务的持久化、事务边界和补偿仍应由业务工作流承担；并发请求使用同一 memory id 时也要验证实现的线程安全与隔离。〔A71、A72、A73〕

代码示例

```
interface Assistant { String answer(@MemoryId String sessionId, String question); }
Assistant assistant =
AiServices.builder(Assistant.class).chatMemoryProvider(memoryProvider).tools(orderTools).build();
String result = assistant.answer(sessionId, question); // 工具仍在服务端检查权限
```

073 Deep Research 怎样规划检索、核验证据并决定停止？

答案　Deep Research 适合需要多来源核验、范围较大的问题。先明确研究范围、时间边界和交付格式，再把问题拆成可独立验证的子目标；并行检索彼此独立的部分，优先阅读官方文档、论文或原始数据，并记录来源及其支持的具体结论。

协调步骤要检查覆盖缺口、相互矛盾的证据和仍未确认的主张，必要时再次检索，然后综合为有出处的结论。转载同一来源不算独立佐证，应区分直接事实、推断与未知，并设置停止条件，例如目标覆盖充分、没有新增证据，或时间与调用预算用尽。LangGraph 可用于编排这类循环，open_deep_research 是一种具体实现；简单事实查询通常不需要完整研究流程。〔A74、A75〕

代码示例

```
question = define_scope_time_and_deliverable()
subtasks = decompose_into_verifiable_questions(question)
evidence = parallel_research(subtasks, prefer_primary_sources=True)
report = synthesize_with_citations(evidence, mark_unknowns=True)  # 达到覆盖或预算上限后停止
```

20.6 AI 编程与交付验证

074 代码补全、对话生成与 Agent 编程有什么区别？

答案　代码补全、对话生成和 Agent 编程的主要区别，是它们能接触并操作多少工程环境。补全通常根据光标附近的代码和上下文给出几个局部候选，开发者挑选后仍要自己组织修改；对话生成先理解自然语言问题，可解释概念、生成函数或提出补丁，但往往要人把代码放入项目并反馈运行结果。Agent 则可以在授权的工具范围内搜索仓库、读文件、编辑代码、运行测试，再根据输出继续修正，形成“观察—行动—反馈”的循环。

所以应按任务的步骤和可验证性选择方式：重复样板适合补全，讨论设计或理解单段代码适合对话，需要跨文件定位且能用测试验收的任务才适合交给 Agent。Agent 能执行命令，不等于它理解了所有业务，也不代表结果天然安全；人仍需检查差异、确认测试覆盖真实需求并决定是否交付。代码写得多、界面里有聊天框，都不能单独说明系统属于哪一类。〔A29〕

代码示例

```
auto completion = request_local_code_candidate(context); // 人决定是否采纳
if (needs_repository_changes)
    agent.modify_scoped_files();
review(diff);
run_acceptance_tests(); // Agent编程必须检查实际差异和结果
```

075 怎样把开发需求转为可验收的规格？Vibe Coding 与规格驱动如何取舍？

答案　可验收规格把“我想要一个好用的功能”变成双方都能判断的行为约定。要先说明当前行为、目标用户和触发场景，再列输入、输出、单位、合法范围、边界值、失败时的结果，以及不能发生的副作用；例如余额不足时订单应失败，库存不得减少。每个要点最好配一个具体例子，避免模型替产品人员猜测尚未决定的业务规则。代码结构等事实可以从仓库查证；涉及取舍的规则若仍有歧义，应先提问或记录假设。

Vibe Coding 借助快速生成和运行反馈探索方向，适合低成本试做、需求尚在变化的阶段；规格驱动先写清可观察的契约，再拆计划、任务和测试，便于长期维护、协作与追责。它们可先探索后收敛：把原型中确认的行为写成验收项，继续迭代时同步规格、实现和测试。规格不是文件名清单或一次写死的流程，更不能因为写得详细就证明实现正确。较大任务还应让需求、澄清、计划、任务和测试彼此可追溯，并优先澄清会影响数据或后续决策的高风险歧义。〔A9〕

代码示例

```
EXPECT_EQ(place_order(item, 2), InventoryError::insufficient);
EXPECT_EQ(database.order_count(), 0);
EXPECT_EQ(inventory.quantity(item), 1);
// 规格同时写出结果和禁止副作用，另测成功路径
```

076 Coding Agent 如何在大型仓库中找到真正相关的代码？

答案　在大型仓库中找代码，先从可复现的线索缩小问题：错误文本、接口路由、日志字段、测试名称或用户可见行为，都比“看看相关模块”更具体。文本检索能找字符串，符号检索能找到定义和引用；再从入口追到调用者、依赖注入、回调和配置，确认实际运行路径。语义检索按含义找相似候选，适合补充线索，却可能只命中旧实现或测试代码；语法树识别语法结构，语言服务还能依据类型和作用域解析重载及引用。

沿路径检查输入怎样变成状态变化或外部副作用，必要时用断点或日志核对推断。若搜不到，先看工作目录、忽略规则、生成代码、子模块和读取结果是否被截断。最后把入口、处理函数和关键依赖串成可复查的链路，并用当前定义、调用者和测试确定修改位置；不能因为某函数名字相似，就假定它一定是线上实际使用的实现。〔A15、A30〕

代码示例

```
rg -n 'error text|test name' src tests
rg -n 'handler_name|write_order' src tests
# 从路由入口追到处理函数、依赖和数据库写入，再用断点确认调用路径
```

077 如何审查 AI 生成的代码，识别表面合理的错误实现？

答案　审查 AI 生成的代码，先把提交前后的实际差异与需求并排比较，包含新增、修改、删除、配置和依赖；只看新代码容易漏掉被删掉的校验或测试。随后从真实输入沿调用链追踪到状态和外部操作，检查空值、越界、权限、错误传播、资源释放及并发时序。发现可疑行为时，应指出它能从哪个入口到达，并构造一个足以暴露问题的最小输入或反例，这比笼统说“这里可能不安全”更可验证。

判断正确性要依靠独立的契约和证据：编译、静态检查可发现部分结构错误，针对性测试与集成运行用于验证行为和组合边界。若测试直接复制实现中的算法，二者可能一起错；应根据需求手工推导预期结果。最终结论要说明审查的版本、真实运行过的命令和结果，也要指出未验证的环境。模型自称正确、代码外观看起来合理或测试全绿，都不能单独构成充分保证。〔A23、A22〕

代码示例

```
EXPECT_THROW(pay(balance = 10, amount = 11), InsufficientFunds);
EXPECT_EQ(balance, 10); // 失败时不能产生扣款副作用
// 审查校验顺序、删除的检查和最小反例
```

078 怎样利用报错、日志与测试，引导 AI 定位并修复根因？

答案　报错、日志和测试要组成可以重复的证据，而不是只贴一行错误文本。先固定代码版本、运行环境、输入和复现命令，并写清预期与实际结果；保留完整堆栈和带请求标识的前后日志，避免把并发请求的记录拼在一起。堆栈通常指出异常最后出现的位置，不一定是根因：应沿调用链检查出错值从哪里产生，核对函数契约、有效配置、线程或异步时序，以及失败前发生过哪些状态变化。

接着把候选原因写成不同的假设，设计最小输入或局部观测，让每次实验都能排除一部分可能性，并保存命令和观察结果。找到根因后尽量做小范围修复，先确认原来的复现确实失败，再确认修复后通过，并跑受影响路径的回归检查。不要通过吞掉异常、随意填默认值或放宽断言来制造表面成功。若问题无法稳定复现，应如实标明证据局限，并继续缩小输入或环境差异。〔A31〕

代码示例

```
./app --config ./test-config.json 2>run.log
rg -n 'request_id|config_field' logs/ run.log
# 用缺字段与空字符串分别复现，修复后重跑原命令和相邻测试
```

079 如何让 AI 生成的测试覆盖关键业务，而非只提高覆盖率？

答案　有价值的测试从业务规则出发，而不是先追求行覆盖率。把规则拆成决策条件、状态变化、权限主体、边界输入和各阶段可能的失败，再根据风险补上重复请求、并发和外部服务异常。每个用例都应检查应有结果，也要检查失败时禁止发生的事，例如付款失败不能写入成功订单。预期结果应由规格、领域规则或独立样例计算；若测试把实现的同一段逻辑复制进断言，就可能让同一个错误同时逃过检查。

单元测试适合隔离局部逻辑，集成测试验证协议和组件组合，模拟依赖时也要还原真实超时、重试或失败语义。覆盖率只说明执行经过哪些代码，变异测试可故意改动条件来检查断言是否敏感，但需排除不改变行为的等价变异。重要需求应能追溯到验证它的用例。迁移旧系统时，若规则要求兼容，可对同一输入比较新旧结果；若业务规则已变，应按新规格制定预期，不能把旧程序自动当作正确答案。〔A32、A22、I104〕

代码示例

```
TEST(Order, RejectsQuantityAboveInventory)
{
    EXPECT_EQ(place_order(item, 2), InventoryError::insufficient);
    EXPECT_EQ(order_count(), 0);
```

```
    EXPECT_EQ(stock(item), 1); // 验证状态与副作用
}
```

080 Coding Agent 如何避免被过期的代码描述、索引或摘要误导？

答案　代码摘要、符号索引和仓库地图都是从源码生成的缓存，能加快查找，却可能落后于当前工作区。可给缓存记录仓库、文件路径、符号、内容指纹和生成条件；使用前比较实际输入是否变化。单记录提交号不够，因为开发者可能已有未提交修改；接口、配置、依赖或生成规则变化时，也要使受影响的索引和已拼装的上下文失效，并按依赖范围重建。

异步重建时还要防止竞态：旧任务可能比新任务更晚结束，因此发布结果前应再次确认输入指纹仍是最新值。搜索或语义检索只提供候选位置，真正修改前要回到当前源码，检查定义、调用方和测试。缓存与源码版本一致只能说明材料较新，不能证明摘要理解正确；涉及行为的判断仍需由代码路径和运行证据支持。〔A30、A33〕

代码示例

```
auto before = hash_file(path);
auto summary = index_source(path);
if (hash_file(path) != before)
    discard(summary); // 源码在索引期间变化，拒绝发布旧摘要
else
    store_summary(path, before, summary);
```

20.7 编程 Agent 扩展与持续运行

081 AGENTS.md、CLAUDE.md 等项目规则怎样按作用域加载并维护？

答案　AGENTS.md、CLAUDE.md 一类文件是给编程助手看的项目说明，可写构建命令、目录约定、代码风格和验证步骤；它们提供上下文，不能自行授予读写文件、联网或执行命令的权限。文件名相同也不保证所有宿主都读取它：发现哪些名字、从哪里开始、怎样处理覆盖和冲突，都由具体产品及版本决定。应先查正在使用的客户端文档或实际加载记录，再写规则，不能把一种工具的规则套给另一种。

以当前 Codex 文档为例，它会从 Codex 全局配置目录检查指令，再沿项目根目录到启动工作目录查找规则；每个目录至多采用一个匹配文件，较深目录的规则追加在前面的内容之后并可覆盖冲突。具体可识别的文件名和合并细节仍应以所用版本为准。根目录文件宜短且稳定，专题细则按需放到局部目录；修改后应在实际客户端确认作用域和结果，并删除过期命令。〔I105、I90、I103〕

代码示例

```
rules = [global_rules]
for directory in parents_from_project_root_to_start_dir:
    candidate = directory / "AGENTS.md"
    if candidate.exists(): rules.append(candidate.read_text())
# 按宿主定义的作用域和覆盖顺序合并，并运行任务验证生效
```

082 Hooks 如何在 Agent 生命周期中执行校验，与提示词约束有什么区别？

答案　Hook 是宿主在会话生命周期或工具调用的特定事件上自动运行的处理器，通常接收结构化事件信息，再用输出或退出状态给宿主反馈。提示词只是告诉模型应该怎样做，模型可能误解或忽略；Hook 则由宿主在相应事件到达时调用代码。比如工具执行前可以检查命令并拒绝本次调用；工具执行后可以检查结果、记录日志或要求进一步处理，但外部写入等副作用已经发生时，事后拒绝不能把它撤销。

各产品支持的事件、匹配字段和阻断语义会随宿主及版本变化，配置前要核对当前文档和事件输入格式。处理器应有明确超时和失败策略：校验服务不可用时是拒绝执行还是允许继续；停止事件还能否触发续跑，也要防止循环。还要检查工具匹配范围是否覆盖实际通道。Hook 是自动化检查点，不是完整访问控制；本地脚本自身也可能读写文件或执行命令，必须按其实际权限审查。〔I106、I107〕

代码示例

```
def before_tool(event):
    if event.command in destructive_commands:
        return deny("destructive command")  # 工具尚未执行
    return allow()
# after-tool hook只能记录结果，副作用可能已经发生
```

083 Agent 插件怎样打包和分发能力，与 Skill、MCP 有何关系？

答案　插件通常是宿主可安装、可版本化分发的一组扩展文件，清单描述身份和组件；它可以把 Skill、MCP 服务、Hook 等打包在一起，但可包含哪些部件取决于宿主。Skill 主要提供任务知识、步骤和参考资料，帮助模型按特定流程工作；MCP 则约定客户端怎样连接外部服务并调用工具或读取资源。插件承担安装、配置和更新这些部件的包装，因此它们分属不同层次，可以组合，不能互相替代。

插件市场帮助用户发现和分发内容，却不能证明发布者可信或脚本安全；“安装成功”也不保证能力已启用、在当前产品界面可见或获得所需授权。应检查来源、版本、清单、依赖及启动命令，了解 MCP 实际可访问的数据和 Hook 的执

行权限，再用受控任务确认生效范围。插件中的本地程序可能继承当前用户权限，因此更新时也要审查差异，不能因它叫扩展包就跳过代码审查。〔I108、I109、I110〕

代码示例

```yaml
name: example-plugin
version: 1.2.0
skills:
  - path: skills/review/SKILL.md
mcp_servers:
  - command: ["trusted-server", "--readonly"]
hooks:
  before_tool: hooks/check.py
# 审查来源、权限、依赖和实际执行能力后再启用
```

084 OpenClaw 的 Gateway、Agent 与 Node 各负责什么？

答案　在 OpenClaw 的架构中，Gateway 是常驻的连接与控制中心，负责接入消息渠道、管理会话并把事件路由到合适的执行端；Agent 根据任务调用模型和工具完成工作；Node 则是经过配对的设备端，向系统声明它能提供的设备能力，例如执行获准的命令或读取传感器。Node 不等同于另一台消息网关，Agent 的任务执行也不等于 Gateway 的渠道管理。具体模块和可用能力应按部署版本核对。

自托管只表示服务由谁部署和管理，不自动意味着拥有全部设备权限、保留完整历史或保证消息永不丢失。上线时要检查客户端与 Node 如何认证、配对，实际开放哪些命令或传感器能力，以及账号和会话怎样隔离。断线后重新连上只证明连接恢复；除非协议明确提供且验证了补发机制，否则不能假定离线期间的每条事件都会重放。〔I112〕

代码示例

```cpp
auto message = gateway.receive(channel_event);
auto agent = router.select_authorized_agent(message.session);
auto result = node.invoke(agent.authorized_capability("read_sensor"));
gateway.reply(message.session, agent.format(result)); // 断线恢复还要校验事件序号
```

20.8 生成内容标记与溯源

085 大模型文本水印怎样嵌入和检测，有哪些可靠性边界？

答案　文本水印可以在生成采样时，利用密钥和前文上下文对候选词元的选择做轻微偏置，让较长文本累积出可统计检测的信号；它通常不需要插入不可见字符，也不等于修改模型权重。检测器按对应方案计算统计量，再用预设阈值判断文本是否与该水印机制相符。这个判断有误报和漏报概率，说明的是统计证据，不是逐句证明作者身份、具体使用了哪款模型或人参与了多少。

文本很短、表达空间受限（如代码、固定格式或低熵文本）时，可供累积的信号少，检测能力会下降；删改、翻译和转述也可能破坏统计特征。因此无法保证水印总能检出，也无法保证任何改写都能去除。它还要与隐藏 Unicode 字符、文件元数据和 C2PA 等内容凭证区分：那些机制记录或携带的信息不同。使用检测结果时，应报告所用检测方案和阈值，并避免把“相符”夸大为作者归属结论。〔I114、I113〕

代码示例

```python
tokens = sample_with_watermark(model, prompt, secret_key)
score = detect_watermark(tokens, secret_key)
confidence = compare_to_threshold(score, calibrated_threshold)
# 短文本、改写和翻译会削弱统计证据；报告置信度而非绝对结论
```

参考文献

S1 仅用于面经考点调研；S2–S22 为语言标准、官方手册、RFC、原始论文与 API 文档，用于核对答案中的规则、行为和适用边界。

- [S1] 面经与题目主题调研（仅用于发现考点）：C++ 面试问题集锦 · C++ 面试题卡片集
- [S2] C++ Working Draft：语言规则、库设施与并发规范：在线工作草案
- [S3] GCC 优化选项：LTO 与 profile guided optimization：GCC 手册
- [S4] Itanium C++ ABI：对象布局、调用接口和运行时约定示例：ABI 规范
- [S5] Linux man-pages：fork、listen、epoll、io_uring 与 socket 选项：fork(2) · listen(2) · epoll(7) · io_uring(7) · socket(7)
- [S6] Boost.Asio：Proactor 异步操作模型：设计说明
- [S7] IETF TCP 与 HTTP 规范：TCP RFC 9293 · HTTP/2 RFC 9113 · HTTP/3 RFC 9114 · HTTP 语义 RFC 9110 · TLS 1.3 RFC 8446
- [S8] IETF DNS 概念与设施 RFC 1034：RFC 1034
- [S9] MySQL Reference Manual：InnoDB redo、MVCC、锁和 binlog：Redo log · MVCC · 锁 · Binlog 与两阶段提交 · undo · 索引 · 隔离级别
- [S10] Redis 文档：热 Key、大 Key 与分布式锁：热点与大 Key 观测 · 分布式锁和 fencing token
- [S11] Raft 原始论文：In Search of an Understandable Consensus Algorithm：扩展版论文
- [S12] Apache Kafka 设计：幂等生产者与事务语义：设计文档
- [S13] Debezium 文档：Outbox Event Router：Transactional Outbox
- [S14] OpenCV 4.10 文档：cv::Mat 与相机标定：cv::Mat · 相机标定
- [S15] Microsoft Azure Architecture Center：Circuit Breaker Pattern：熔断器模式
- [S16] Qt 6 官方文档：QObject 生命周期、线程与信号槽：QObject · Threads and QObjects · QThread · Signals & Slots
- [S17] Qt 6 官方文档：容器、模型/视图与数据库：容器 · QAbstractItemModel · QSqlQuery · QSqlDatabase
- [S18] Qt 6 官方文档：QML、插件与版本迁移：QML/C++ 集成 · QPluginLoader · Qt 6 迁移
- [S19] Qt 6 官方文档：图表与多媒体：Qt Charts · Qt Graphs · Qt Multimedia
- [S20] FFmpeg 官方文档：命令行、编解码 API、滤镜与拼接：文档入口 · 解码 API · 滤镜手册 · FAQ
- [S21] OpenCV 4.x 官方文档：图像处理、矩阵与标定：文档入口 · cv::Mat · 相机标定
- [S22] ROS 2 QoS 官方设计文档：QoS 策略 · Deadline、Liveliness 与 Lifespan

S23 为本次增补的选题与去重来源；N 系列为新增答案核对的官方资料。新增题目按考点独立整理，近义问法统一收录。

- [S23] 小林 coding 面试栏目（2026-10-29 读取）：C++ · Python · MySQL · Redis · 计算机网络 · 操作系统 · 数据结构与算法 · 消息队列 · 分布式 · 系统设计 · Linux 命令 · Git · Docker · 后端进阶路线图
- [N1] C++ 工作草案：复合类型与指针转换
- [N2] C++ 工作草案：指针转换
- [N3] C++ 工作草案：函数调用
- [N4] C++ 工作草案：相等运算
- [N5] C11 公开工作草案 N1570：6.5.9
- [N6] C++ 工作草案：sizeof
- [N7] C++ 工作草案：类与访问控制
- [N8] C++ 工作草案：联合体
- [N9] C++ 工作草案：构造与析构期间的对象
- [N10] C++ 工作草案：虚函数覆盖
- [N11] Itanium C++ ABI：对象与虚表布局
- [N12] C++ 工作草案：对象生命周期
- [N13] Linux man-pages：C 字符串拷贝
- [N14] Linux man-pages：strlen
- [N15] Linux man-pages：memcpy
- [N16] C++ 工作草案：对象表示
- [N17] C++ 工作草案：new 表达式
- [N18] Linux man-pages：malloc 与分配机制
- [N19] C++ 工作草案：显式默认化函数

- [N20] C++ 工作草案：删除函数
- [N21] C++ 工作草案：范围 for 语句
- [N22] C++ 工作草案：枚举声明
- [N23] C++ 工作草案：变长参数模板
- [N24] GCC 预处理器：头文件搜索
- [N25] Linux man-pages：dlopen 与动态装载
- [N26] x86-64 System V ABI 规范
- [N27] System V ABI：ELF 对象文件格式
- [N28] Linux 内核 x86 启动协议
- [N29] Linux man-pages：系统调用与库函数包装
- [N30] Linux man-pages：POSIX 线程共享与独立资源
- [N31] Linux man-pages：调度策略
- [N32] Linux 内核文档：页表层次和地址转换
- [N33] Linux man-pages：多线程进程 fork 的约束
- [N34] Linux man-pages：信号机制
- [N35] Linux man-pages：POSIX 信号量
- [N36] Linux 内核文档：EEVDF 调度
- [N37] Linux man-pages：pthread 自旋锁
- [N38] Linux 内核文档：锁类型与 PREEMPT_RT 差异
- [N39] E. W. Dijkstra：银行家算法原稿 EWD108
- [N40] Linux 内核文档：多代 LRU 页面回收
- [N41] Linux man-pages：文件描述符与打开文件描述
- [N42] Linux man-pages：unlink 后文件存续规则
- [N43] Linux 内核文档：Page Cache
- [N44] Linux man-pages：mmap(2)
- [N45] Linux man-pages：fsync/fdatasync 的持久化边界
- [N46] Linux 内核文档：通用 IRQ 处理
- [N47] Linux 内核文档：工作队列
- [N48] Linux man-pages：read(2) 返回值与错误
- [N49] Linux man-pages：epoll(7) 就绪通知
- [N50] IETF RFC 1122：Internet 主机分层要求
- [N51] IETF RFC 9110：HTTP 方法、范围请求与代理语义
- [N52] IETF RFC 9112：HTTP/1.1 报文边界与连接管理
- [N53] IETF RFC 9111：HTTP 缓存及验证
- [N54] Linux man-pages：hosts(5) 与本地名称映射
- [N55] IETF RFC 4033：DNSSEC 安全性与边界
- [N56] IETF RFC 7766：DNS over TCP 实施要求
- [N57] IETF RFC 6265：Cookie 状态管理
- [N58] Mozilla MDN：浏览器 Set-Cookie 属性
- [N59] WHATWG HTML：Web Storage
- [N60] IETF RFC 7519：JWT 结构与声明
- [N61] IETF RFC 8725：JWT 验证最佳实践
- [N62] gRPC 官方文档：RPC 核心概念与调用生命周期
- [N63] IETF RFC 6455：WebSocket 协议
- [N64] WHATWG Fetch：CORS 协议
- [N65] Amazon CloudFront：边缘分发工作流程
- [N66] IETF RFC 9293：TCP 报文和连接状态机
- [N67] Linux man-pages：accept(2)
- [N68] CISA/FBI/MS-ISAC：DDoS 响应指南
- [N69] Linux 内核文档：TCP SYN 队列与 syncookies 参数
- [N70] Linux man-pages：tcp(7) 与半关闭状态参数
- [N71] IETF RFC 826：ARP 地址解析

- [N72] IETF RFC 792：ICMP 差错与回显消息
- [N73] IETF RFC 3022：传统 NAT/NAPT
- [N74] Linux man-pages：send(2) 的返回值与错误
- [N75] OWASP：XSS 防护速查
- [N76] OWASP：CSRF 防护速查
- [N77] Redis 官方文档：运行与开发概览
- [N78] MongoDB：关系与非关系数据模型
- [N79] Microsoft：数据库规范化
- [N80] MySQL JOIN 语法
- [N81] MySQL 子查询优化
- [N82] MySQL CHAR 与 VARCHAR
- [N83] MySQL BLOB 与 TEXT
- [N84] MySQL 整数类型
- [N85] MySQL 精确定点运算
- [N86] MySQL IP 地址转换函数
- [N87] MySQL 外键约束
- [N88] MySQL 函数与运算符
- [N89] MySQL 聚合函数
- [N90] MySQL TRUNCATE TABLE
- [N91] MySQL SELECT 语法
- [N92] MySQL 窗口函数
- [N93] MySQL 存储引擎
- [N94] MySQL Buffer Pool
- [N95] MySQL Binary Log 与组提交
- [N96] MySQL 移除基于文件的元数据存储
- [N97] MySQL 独立表空间
- [N98] MySQL InnoDB AUTO_INCREMENT
- [N99] MySQL 索引条件下推
- [N100] MySQL 列索引与前缀索引
- [N101] MySQL 死锁处理
- [N102] MySQL 元数据锁
- [N103] MySQL InnoDB 参数
- [N104] MySQL Checkpoint
- [N105] MySQL Doublewrite Buffer
- [N106] MySQL LIMIT 优化
- [N107] MySQL 复制实现
- [N108] MySQL 半同步复制
- [N109] MySQL GTID 概念
- [N110] Azure：分片模式
- [N111] Redis 7.2 源码：ZSet、渐进 rehash 与 listpack
- [N112] Redis 7.2 listpack source
- [N113] Redis 7.2 dictionary source
- [N114] Redis 7.2 SDS source
- [N115] Redis 7.2 event loop source
- [N116] Redis 延迟排查
- [N117] Redis 事务
- [N118] Redis Pipeline
- [N119] Redis EXPIRE
- [N120] Redis 复制
- [N121] Redis Sentinel
- [N122] Redis Cluster 规范
- [N123] Redis 客户端缓存

- [N124] Azure：Cache Aside 模式
- [N125] Redis SCAN
- [N126] Redis Streams
- [N127] MySQL InnoDB 锁定读
- [N128] MySQL 命名锁函数
- [N129] ZooKeeper Recipes
- [N130] Google SRE：处理过载
- [N131] RocketMQ 5.0 架构
- [N132] RabbitMQ 交换机
- [N133] Kafka 4.1 设计
- [N134] RocketMQ 5.0 顺序消息
- [N135] Kafka 4.1 监控
- [N136] RocketMQ 5.0 消费重试
- [N137] RabbitMQ Dead Letter Exchanges
- [N138] RocketMQ 5.0 定时消息
- [N139] RocketMQ 5.0 事务消息
- [N140] RocketMQ 4.x 存储架构
- [N141] Kafka 4.1 Consumer 配置
- [N142] RabbitMQ 消息确认
- [N143] RabbitMQ Consumer Prefetch
- [N144] RabbitMQ Quorum Queues
- [N145] Jepsen：一致性模型
- [N146] gRPC Deadlines
- [N147] ZooKeeper Internals：ZAB
- [N148] OpenTelemetry Traces
- [N149] Apache Seata AT 模式
- [N150] C++ 工作草案：优先队列适配器
- [N151] C++ 工作草案：sort 的要求
- [N152] GCC libstdc++：排序算法实现
- [N153] Sedgewick/Wayne Algorithms：红黑树参考实现
- [N154] Sedgewick/Wayne Algorithms：AVL 树参考实现
- [N155] Redis 8.0 源码：有序集合跳表实现
- [N156] Sedgewick/Wayne Algorithms：Trie 参考实现
- [N157] OpenDSA 教材：外部排序与多路归并
- [N158] Sedgewick/Wayne Algorithms：邻接表图实现
- [N159] Jeff Erickson Algorithms：Greedy Algorithms
- [N160] Jeff Erickson Algorithms：Dynamic Programming
- [N161] Python 数据模型
- [N162] Python 表达式
- [N163] Python 内置类型
- [N164] Python 内置函数
- [N165] Python collections 模块
- [N166] CPython listobject.c 实现
- [N167] Python copy 模块
- [N168] Python io
- [N169] CPython dictobject.c 实现
- [N170] Python 函数与参数
- [N171] Python 复合语句
- [N172] Python 执行模型与名字绑定
- [N173] Python functools 模块
- [N174] Python asyncio 任务
- [N175] Python 类教程

- [N176] Python 描述符指南
- [N177] Python dataclasses
- [N178] Python typing
- [N179] Python 异常
- [N180] Python contextlib 模块
- [N181] Python 导入系统
- [N182] Python 编译与字节码缓存
- [N183] Python venv
- [N184] Pydantic 模型
- [N185] Python JSON 编解码
- [N186] Python 自由线程指南
- [N187] Python threading
- [N188] Python multiprocessing
- [N189] Python asyncio 同步原语
- [N190] Python queue
- [N191] Python concurrent.futures
- [N192] Python contextvars
- [N193] CPython 内存管理
- [N194] Python gc
- [N195] Python tracemalloc
- [N196] Python dis
- [N197] Python 性能分析器
- [N198] procps-ng ps 手册
- [N199] procps-ng ps、top、free 与 vmstat 手册
- [N200] Linux kill(2) 信号语义
- [N201] iproute2 ss 手册
- [N202] OpenBSD nc 手册
- [N203] curl 官方手册
- [N204] procps-ng free 手册
- [N205] Linux /proc/loadavg 与 /proc/meminfo
- [N206] procps-ng vmstat 手册
- [N207] GNU sed 手册
- [N208] GNU awk 手册
- [N209] GNU coreutils 手册
- [N210] Linux /proc/loadavg
- [N211] Git 官方命令手册
- [N212] Git diff
- [N213] Git rebase
- [N214] Git fetch
- [N215] Git checkout 与 detached HEAD
- [N216] Git reset
- [N217] Git revert
- [N218] Git stash
- [N219] GitHub 移除敏感数据指南
- [N220] Git gitignore
- [N221] Pro Git 分布式工作流
- [N222] Git log
- [N223] Git tag
- [N224] Git bisect
- [N225] Docker 架构与对象
- [N226] Linux cgroup v2 文档
- [N227] Dockerfile 参考

- [N228] Docker 构建最佳实践
- [N229] Docker 存储概览
- [N230] Docker run 参考
- [N231] Docker 多阶段构建
- [N232] Docker 网络概览
- [N233] Docker Compose 文档
- [N234] Kubernetes 容器运行时
- [N235] Docker 资源清理
- [N236] MongoDB：文档建模
- [N237] MongoDB：副本集
- [N238] Apache Hadoop：HDFS 架构
- [N239] C++ Core Guidelines：接口、工厂与类设计
- [N240] OWASP：密码存储
- [N241] OpenID Connect Core 1.0
- [N242] OWASP：REST 接口安全
- [N243] RFC 9421：HTTP 消息签名
- [N244] Google SRE：服务级别目标
- [N245] Java 25：垃圾回收调优导论
- [N246] Go：垃圾回收指南
- [N247] NGINX：TCP/UDP 负载均衡
- [N248] Azure Architecture：舱壁隔离模式
- [N249] Prometheus：架构与抓取
- [N250] AWS DMS：持续变更捕获
- [N251] Kubernetes：声明式编排
- [N252] Linux man-pages：execve 与 ELF 解释器

S24 为大模型六栏目的考点来源；A 系列为该章节独立整理答案时核对的原始论文、协议规范与官方文档。框架和协议的具体版本以各条引用及项目依赖为准。

- [S24] 小林面试笔记：大模型面试六栏目（2026-10-29 读取）：Agent · RAG · LLM 工具调用 · 大模型工程 · LangChain · AI 编程
- [A1] Building effective AI agents \ Anthropic
- [A2] ReAct: Synergizing Reasoning and Acting in Language Models
- [A3] Effective harnesses for long-running agents \ Anthropic
- [A4] Persistence - Docs by LangChain
- [A5] Workflows and agents - Docs by LangChain
- [A6] Self-Refine: Iterative Refinement with Self-Feedback
- [A7] Tree of Thoughts: Deliberate Problem Solving with Large Language Models
- [A8] Graph of Thoughts: Solving Elaborate Problems with Large Language Models
- [A9] spec-kit/spec-driven.md at main · github/spec-kit · GitHub
- [A10] Memory overview - Docs by LangChain
- [A11] How we built our multi-agent research system \ Anthropic
- [A12] Context editing - Claude Platform Docs
- [A13] Token counting - Claude Platform Docs
- [A14] Effective context engineering for AI agents \ Anthropic
- [A15] Features
- [A16] Use time-travel - Docs by LangChain
- [A17] Demystifying evals for AI agents \ Anthropic
- [A18] Graph API overview - Docs by LangChain
- [A19] OpenTelemetry：GenAI Agent spans（发展中的语义约定）
- [A20] OpenTelemetry：GenAI metrics（发展中的语义约定）
- [A21] How to do distributed locking — Martin Kleppmann’s blog
- [A22] Evaluation - SWE-bench
- [A23] What to look for in a code review | eng-practices

- [A24] PostgreSQL: Documentation: 18: 5.9. Row Security Policies
- [A25] LLM06:2025 Excessive Agency - OWASP Gen AI Security Project
- [A26] Reflexion: Language Agents with Verbal Reinforcement Learning
- [A27] Interrupts - Docs by LangChain
- [A28] Reproducibility — PyTorch 2.15 documentation
- [A29] GitHub Copilot on GitHub.com and GitHub Mobile - GitHub Docs
- [A30] The clangd index
- [A31] Google SRE - Troubleshooting Methodology: A Learning Path
- [A32] Mutant states and metrics | Stryker Mutator
- [A33] Hermeticity - Bazel Documentation
- [A34] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
- [A35] Ranking and reranking | Elastic Docs
- [A36] Milvus 官方文档：Filtered Search
- [A37] Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models
- [A38] Dense X Retrieval: What Retrieval Granularity Should We Use?
- [A39] Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks - ACL Anthology
- [A40] MTEB: Massive Text Embedding Benchmark
- [A41] Milvus 官方文档：Index Explained
- [A42] Milvus 官方文档：HNSW
- [A43] Milvus 官方文档：HNSW_SQ
- [A44] Similarity settings | Elasticsearch Reference
- [A45] Reciprocal rank fusion | Elasticsearch Reference
- [A46] Precise Zero-Shot Dense Retrieval without Relevance Labels
- [A47] Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models
- [A48] Ragas: Automated Evaluation of Retrieval Augmented Generation
- [A49] Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection
- [A50] Corrective Retrieval Augmented Generation
- [A51] From Local to Global: A Graph RAG Approach to Query-Focused Summarization
- [A52] Ranking evaluation | Elasticsearch Reference
- [A53] Milvus 官方文档：Upsert Entities
- [A54] Milvus 官方文档：Manage Aliases
- [A55] Knowledge Conflicts for LLMs: A Survey
- [A56] Lost in the Middle: How Language Models Use Long Contexts
- [A57] Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach
- [A58] SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering
- [A59] Runnable | langchain_core | LangChain Reference
- [A60] RunnableSequence | langchain_core | LangChain Reference
- [A61] RunnableParallel | langchain_core | LangChain Reference
- [A62] LangChain v1 migration guide - Docs by LangChain
- [A63] LangGraph overview - Docs by LangChain
- [A64] Agents - Docs by LangChain
- [A65] Tools - Docs by LangChain
- [A66] create_agent | langchain | LangChain Reference
- [A67] Short-term memory - Docs by LangChain
- [A68] Long-term memory - Docs by LangChain
- [A69] LangChain overview - Docs by LangChain
- [A70] High-Level Concepts | Developer Documentation
- [A71] Introduction | LangChain4j
- [A72] AI Services | LangChain4j
- [A73] Chat Memory | LangChain4j
- [A74] GitHub - langchain-ai/open_deep_research • GitHub

- [A75] open_deep_research/src/open_deep_research/deep_researcher.py at main · langchain-ai/open_deep_research · GitHub
- [A76] Language Models are Few-Shot Learners
- [A77] BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
- [A78] Attention Is All You Need
- [A79] Fast Transformer Decoding: One Write-Head is All You Need
- [A80] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints
- [A81] FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness
- [A82] RoFormer: Enhanced Transformer with Rotary Position Embedding
- [A83] Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation
- [A84] SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing
- [A85] Training language models to follow instructions with human feedback
- [A86] Training Compute-Optimal Large Language Models
- [A87] Are Emergent Abilities of Large Language Models a Mirage?
- [A88] LoRA: Low-Rank Adaptation of Large Language Models
- [A89] QLoRA: Efficient Finetuning of Quantized LLMs
- [A90] Direct Preference Optimization: Your Language Model is Secretly a Reward Model
- [A91] DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models
- [A92] The Curious Case of Neural Text Degeneration
- [A93] Generation strategies · Hugging Face
- [A94] Cache strategies · Hugging Face
- [A95] Automatic Prefix Caching - vLLM
- [A96] GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers
- [A97] AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration
- [A98] Prompt engineering overview - Claude Platform Docs
- [A99] Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
- [A100] Self-Consistency Improves Chain of Thought Reasoning in Language Models
- [A101] FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation
- [A102] DeepSeek-V3 Technical Report
- [A103] Efficient Memory Management for Large Language Model Serving with PagedAttention
- [A104] Welcome to SGLang - SGLang Documentation
- [A105] Text Generation Inference · Hugging Face
- [A106] GitHub - ggml-org/llama.cpp: LLM inference in C/C++ · GitHub
- [A107] Holistic Evaluation of Language Models
- [A108] Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena
- [A109] Tool use with Claude - Claude Platform Docs
- [A110] Toolformer: Language Models Can Teach Themselves to Use Tools
- [A111] Architecture - Model Context Protocol
- [A112] Tools - Model Context Protocol
- [A113] Specification - Model Context Protocol
- [A114] Specification - Agent Skills
- [A115] Extended thinking - Claude Platform Docs
- [A116] How to add skills support to your agent - Agent Skills
- [A117] Overview - A2A Protocol
- [A118] A2A and MCP - A2A Protocol
- [A119] Overview - Model Context Protocol
- [A120] Key Changes - Model Context Protocol
- [A121] LiteLLM AI Gateway | liteLLM
- [A122] Tool search tool - Claude Platform Docs
- [A123] Retry | gRPC
- [A124] subprocess — Subprocess management — Python 3.15.2 documentation

- [A125] Introduction to gVisor security - gVisor
- [A126] WHATWG HTML：Server-sent events
- [A127] IETF RFC 8825：WebRTC Protocols Overview

S25 为图解系列的选题与去重来源；I 系列补充本轮答案核对的一手资料。涉及具体软件行为时，以所引版本和官方文档为准。

- [S25] 小林图解系列（2026-10-29 读取）：图解网络 · 图解系统 · 图解 MySQL · 图解 Redis · 图解 Agent · 图解 Claude Code · 图解 Codex
- [I1] Linux 内核文档：Ethernet Bridging 的 FDB 学习、老化与泛洪
- [I2] Linux 内核文档：NAPI 中断、预算轮询和完成
- [I3] Linux 内核文档：sk_buff 元数据、共享缓冲和克隆
- [I4] IETF RFC 5077：TLS Session Tickets
- [I5] IETF RFC 7918：TLS False Start 条件与风险
- [I6] IETF RFC 6298：TCP RTO 计算与退避
- [I7] IETF RFC 5961：Challenge ACK 与严格 RST 验证
- [I8] IETF RFC 7413：TCP Fast Open 与重放边界
- [I9] IETF RFC 9000：QUIC 流、包号、流控与路径验证
- [I10] IETF RFC 9204：QPACK 与阻塞流限制
- [I11] IETF RFC 8200：IPv6 首部与分片规则
- [I12] IETF RFC 4861：IPv6 邻居发现
- [I13] IETF RFC 2131：DHCPv4 租约、续租与中继
- [I14] IETF RFC 3376：IGMPv3 成员关系与源过滤机制
- [I15] Linux 内核文档：NUMA Memory Policy
- [I16] Linux 内核文档：进程地址、VMA、页表与反向映射
- [I17] Linux 内核文档：内核内存分配接口与 GFP 上下文
- [I18] Linux 内核文档：物理内存的 Node/Zone、PCP 及水位
- [I19] Linux 内核文档：虚拟内存参数、swappiness 与压缩
- [I20] Linux 内核文档：Overcommit Accounting
- [I21] Linux 内核文档：blk-mq 多队列、合并及 I/O 调度
- [I22] Linux 内核文档：ext4 块组和分配布局
- [I23] Linux 内核文档：跨总线设备 IO、MMIO 访问与顺序
- [I24] Linux man-pages：sendfile 部分传输和源数据生命周期
- [I25] Linux man-pages：pthread_create 资源与线程数限制
- [I26] Linux man-pages：管道原子写、EOF 和 SIGPIPE
- [I27] Linux man-pages：posix_fadvise 访问模式与预读提示
- [I28] Linux 内核文档：VFS 对象和文件系统操作分派
- [I29] IETF RFC 4291：IPv6 地址类型及无广播地址
- [I30] Linux 内核文档：内存模型与 PFN 到页描述符
- [I31] Linux 内核文档：slab 对象缓存和分配阶
- [I32] Linux 内核文档：内存热插拔与下线迁移限制
- [I33] Linux 内核源码：伙伴系统页分配与合并
- [I34] IETF RFC 7323：TCP 窗口缩放、时间戳与 PAWS
- [I35] IETF RFC 5681：TCP Reno 拥塞控制基础
- [I36] Linux 内核文档：ext4 inode 数据映射与 extent 树
- [I37] IETF RFC 2018：TCP SACK 选择确认
- [I38] IETF RFC 2883：TCP D-SACK 重复接收报告
- [I39] IETF RFC 6633：废弃 ICMP Source Quench
- [I40] Linux 内核文档：lockdep 锁依赖与循环检测
- [I41] OSTEP 作者教材第 32 章：死锁四条件与预防
- [I42] Arm：Armv8-A Instruction Set Architecture（顺序语义、流水线与分支）
- [I43] RISC-V ISA：寄存器位宽与指令长度编码
- [I44] Microsoft：Physical Address Extension
- [I45] Microsoft：ILP32 与 LLP64 数据模型

- [I46] Microsoft：32 位程序在 64 位 Windows 上的兼容边界
- [I47] C++ 工作草案：基本类型、补码与整数算术
- [I48] Oracle Numerical Computation Guide：IEEE Arithmetic
- [I49] Python 官方教程：浮点数的误差与限制
- [I50] Python 官方文档：math.isclose 与容差比较
- [I51] Linux perf：perf stat 硬件计数与 IPC
- [I52] Linux man-pages：sched_setaffinity
- [I53] GCC：__builtin_expect 与分支概率提示
- [I54] C++ 工作草案：likely 与 unlikely 属性
- [I55] Intel SDM Volume 3：缓存控制与一致性（第 14 章）
- [I56] Linux 内核文档：缓存一致性与内存屏障
- [I57] Linux 内核文档：proc 文件系统与中断统计
- [I58] seL4：微内核与用户态服务的边界
- [I59] Microsoft：Windows 用户态与内核态组件
- [I60] Linux 内核文档：PREEMPT_RT 的执行上下文差异
- [I61] MySQL 8.4：一致性非锁定读与快照边界
- [I62] MySQL 8.4：不同语句设置的 InnoDB 锁
- [I63] MySQL 8.4：InnoDB 行格式
- [I64] MySQL 8.4：联合索引的范围优化
- [I65] MySQL 8.4：表达式类型转换与索引
- [I66] MySQL 8.4：EXPLAIN 输出格式
- [I67] MySQL 8.4：列数与行大小限制
- [I68] MySQL 8.4：Buffer Pool 抗扫描污染
- [I69] MySQL 8.4：binlog 刷盘与组提交参数
- [I70] MySQL 8.4 C API：mysql_reset_connection 会话清理
- [I71] MySQL 8.4：wait_timeout 与连接限制
- [I72] MySQL 8.4.0 源码：页目录二分与记录链查找
- [I73] MySQL 8.4.0 源码：页目录槽及记录分组界限
- [I74] MySQL 8.4：mysqldump 一致性备份与 DDL 限制
- [I75] MySQL 8.4：自适应哈希索引
- [I76] MySQL 8.4：Change Buffer
- [I77] Redis 官方：持久化、混合格式与 7.0 多文件 AOF
- [I78] Redis 官方：近似 LRU、LFU 与内存淘汰
- [I79] Redis 官方：6.2、7.0、7.2 紧凑编码差异
- [I80] Redis 7.2.0 源码：List 的 listpack 与 quicklist 转换
- [I81] Redis 7.2.0 源码：Set 的 intset、listpack 与字典转换
- [I82] Redis 7.2.0 源码：整数集合整体升级与删除
- [I83] Redis 7.2.0 源码：共享复制缓冲区与 backlog
- [I84] Redis 官方：HyperLogLog 误差与空间
- [I85] Redis 官方：LMOVE 与可靠队列模式
- [I86] Redis 官方：Pub/Sub 至多一次投递语义
- [I87] Redis 官方：UNLINK 的异步内存回收
- [I88] MySQL 8.4.0 源码：ReadView 版本可见性判定
- [I89] MySQL 8.4：ORDER BY、内存 filesort 与临时磁盘文件
- [I90] Claude Code 官方文档：How Claude remembers your project
- [I91] OpenAI Codex 公开实现：记忆第一阶段提取（固定版本）
- [I92] OpenAI Codex 公开实现：记忆第二阶段整合（固定版本）
- [I93] OpenAI 官方文档：Compaction
- [I94] Claude Code 官方文档：Checkpointing
- [I95] Claude Code 官方文档：Remote Control
- [I96] Claude Code 官方文档：Create custom subagents
- [I97] Git 官方文档：git-worktree

- [I98] OpenAI 官方文档：Agent approvals and security
- [I99] LightRAG: Simple and Fast Retrieval-Augmented Generation（原始论文）
- [I100] HKUDS LightRAG 官方仓库：检索模式、增量更新与删除重建
- [I101] Claude API 官方文档：Streaming messages
- [I102] Claude API 官方文档：Handle tool calls
- [I103] OpenAI 官方文档：Rethinking skills and prompts for GPT-6 Astra
- [I104] Anthropic 官方实践：How to prepare for AI-driven code modernization projects
- [I105] OpenAI Codex 官方文档：Custom instructions with AGENTS.md
- [I106] Claude Code 官方文档：Hooks reference
- [I107] OpenAI 官方文档：Hooks
- [I108] Claude Code 官方文档：Create plugins
- [I109] OpenAI 官方文档：Plugins concepts
- [I110] Claude Code 官方文档：Plugin security
- [I111] Claude Code 官方文档：Run prompts on a schedule
- [I112] OpenClaw 官方文档：Gateway architecture
- [I113] Anthropic 官方公告：Claude text watermark
- [I114] Nature 原始论文：Scalable watermarking for identifying large language model outputs